
Introduction to Numerical Methods and Analysis with Python

Brenton LeMesurier (College of Charleston, South Carolina) with

Jun 18, 2024

CONTENTS

I	Frontmatter	3
1	Introduction	5
1.1	Topics	5
1.2	Some References	5
II	Main	7
2	Root-finding	9
2.1	Root Finding by Interval Halving (Bisection)	9
2.2	Solving Equations by Fixed Point Iteration (of Contraction Mappings)	18
2.3	Newton's Method for Solving Equations	33
2.4	Taylor's Theorem and the Accuracy of Linearization	47
2.5	Measures of Error and Order of Convergence	49
2.6	The Convergence Rate of Newton's Method	52
2.7	Root-finding Without Derivatives	54
3	Linear Algebra and Simultaneous Equations	67
3.1	Row Reduction/Gaussian Elimination	67
3.2	Machine Numbers, Rounding Error and Error Propagation	86
3.3	Partial Pivoting	95
3.4	Solving $Ax = b$ with LU factorization, $A = LU$	98
3.5	Solving $Ax = b$ With Both Pivoting and LU factorization	107
3.6	Error bounds for linear algebra, condition numbers, matrix norms, etc.	115
3.7	Iterative Methods for Simultaneous Linear Equations	123
3.8	Faster Methods for Solving $Ax = b$ for Tridiagonal and Banded matrices, and Strict Diagonal Dominance	128
3.9	Computing Eigenvalues and Eigenvectors: the Power Method, and a bit beyond	132
3.10	Solving Nonlinear Systems of Equations by generalizations of Newton's Method — a brief introduction	137
4	Polynomial Collocation and Approximation	139
4.1	Polynomial Collocation (Interpolation/Extrapolation)	139
4.2	Error Formulas for Polynomial Collocation	147
4.3	Choosing the collocation points: the Chebyshev method	154
4.4	Piecewise Polynomial Approximating Functions and Spline Interpolation	155
4.5	Least-squares Fitting to Data	161
4.6	Least-squares Fitting to Data: Appendix on The Geometrical Approach	171
5	Derivatives and Definite Integrals	173
5.1	Approximating Derivatives by the Method of Undetermined Coefficients	173
5.2	Richardson Extrapolation	178

5.3	Definite Integrals, Part 1: The Building Blocks	182
5.4	Definite Integrals, Part 2: The Composite Trapezoid and Midpoint Rules	189
5.5	Definite Integrals, Part 3: The (Composite) Simpson’s Rule and Richardson Extrapolation	194
5.6	Definite Integrals, Part 4: Romberg Integration	197
6	Minimization	201
6.1	Finding the Minimum of a Function of One Variable Without Using Derivatives — A Brief Introduction	201
6.2	Finding the Minimum of a Function of Several Variables — Coming Soon	203
7	Initial Value Problems for Ordinary Differential Equations	205
7.1	Basic Concepts and Euler’s Method	205
7.2	Runge-Kutta Methods	223
7.3	A Global Error Bound for One Step Methods	238
7.4	Systems of ODEs and Higher Order ODEs	239
7.5	Error Control and Variable Step Sizes	256
7.6	An Introduction to Multistep Methods: Leap-frog	270
7.7	Adams-Bashforth Multistep Methods	281
7.8	Implicit Methods: Adams-Moulton	304
7.9	Predictor-Corrector Methods — Coming Soon	306
7.10	Introduction to Implicit Methods and Stiff Equations	306
III	Exercises	307
8	Exercises on the Bisection Method	309
8.1	A test case	309
8.2	Exercise 1	309
8.3	The bisection method algorithm in “pseudocode”	310
8.4	Exercise 2	310
8.5	Exercise 3	310
9	Exercises on Fixed Point Iteration	311
9.1	Exercise 1	311
10	Exercises on Error Measures and Convergence	313
10.1	Exercise 1	313
11	Exercises on Newton’s Method	315
11.1	Exercise 1	315
11.2	Exercise 2	315
12	Exercises on Root-finding Without Derivatives	317
12.1	Exercise 1: Comparing Root-finding Methods	317
13	Exercises on Machine Numbers, Rounding Error and Error Propagation	319
13.1	Exercise 1	319
13.2	Exercise 2	319
13.3	Exercise 3	319
14	Exercises on Solving Simultaneous Linear Equations	321
14.1	Exercise 1	321
14.2	Exercise 2	321
14.3	Exercise 3	321
14.4	Some Relevant Algorithms	322

15 Exercises on Approximating Derivatives, the Method of Undetermined Coefficients and Richardson Extrapolation	323
15.1 Exercise 1	323
15.2 Exercise 2	323
15.3 Exercise 3	324
15.4 Exercise 4	324
15.5 Exercise 5	324
15.6 Exercise 6	324
 IV Python Tutorial	 325
16 Introduction	327
17 Getting Python Software for Scientific Computing	329
17.1 Anaconda, featuring JupyterLab, Spyder, and IPython	329
17.2 Colab (a.k.a. Colaboratory); a purely online alternative for Jupyter notebooks	329
18 Suggestions and Notes on Python and Jupyter Notebook Usage	331
19 Python Basics	333
19.1 Using Python interactively	333
19.2 Running some or all cells in a notebook	333
19.3 Numbers and basic arithmetic	334
19.4 Boolean values (True-False)	336
19.5 Comparisons	337
19.6 Character strings	337
19.7 Naming quantities, and displaying information with <code>print</code>	338
19.8 Some mathematical functions and constants: module <code>math</code>	339
19.9 Logarithmic functions	342
19.10 Notes on organization and presentation of course work	342
20 Notes on Python Coding Style (under construction)	345
20.1 Restrictions on characters used in names	345
20.2 Naming style	345
21 Python Variables, Including Lists and Tuples, and Arrays from Package Numpy	347
21.1 Foreword	347
21.2 Numerical variables	347
21.3 Text variables	348
21.4 Lists	349
21.5 Tuples	351
21.6 Naming rules for variables	352
21.7 The immutability of tuples (and also of text strings)	352
21.8 Numpy arrays: for vectors, matrices, and beyond	353
22 Decision Making With <code>if</code>, <code>else</code>, and <code>elif</code>	361
22.1 Introduction	361
22.2 Handling multiple possibilities	364
22.3 There can be as many <code>elif</code> clauses as you want.	364
22.4 Plan before you code!	365
23 Defining and Using Python Functions	367
23.1 Introduction	367
23.2 Variables “created” inside a function definition are local to it	368

23.3	Note: With tuples, parentheses are optional	370
23.4	Single-member tuples: not an oxymoron	371
23.5	Documenting functions with triple quoted comments, and <code>help**</code>	371
23.6	Exercise A. A robust function for solving quadratics	372
23.7	Keyword arguments: specifying input values by name	373
23.8	Functions as input to other functions	374
23.9	Optional input arguments and default values	375
23.10	Optional topic: anonymous functions, a.k.a. lambda functions	376
24	Iteration with <code>for</code>	379
24.1	Introduction	379
24.2	Repeating a predetermined number of times, with <code>for</code> loops	379
24.3	Repeating for a range of equally spaced integers with <code>range()</code>	380
24.4	Ranges that start elsewhere	381
24.5	Ranges of equally-spaced integers	381
24.6	Decreasing sequences of integers	382
25	Iteration with <code>while</code>	387
25.1	Introduction	387
25.2	Repeating an initially unknown number of times, with <code>while</code> loops	387
25.3	Appending to lists, and our first use of Python <i>methods</i>	389
26	Code Files, Modules, and an Integrated Development Environment	393
26.1	Introduction	393
26.2	Integrated Development Environments: Spyder et al	393
27	Recursion (vs iteration)	397
27.1	Introduction	397
27.2	Iterative form	397
27.3	Recursive form	398
27.4	Bonus Material: Tail Recursion	400
27.5	A final challenge (optional, and maybe hard!)	401
28	Plotting Graphs with Matplotlib	403
28.1	Introduction: Matplotlib and Pyplot	403
28.2	Sources on Matplotlib	403
28.3	Choosing where the graphs appear	403
28.4	Producing arrays of “x” values with the numpy function <code>linspace</code>	404
28.5	Basic graphs with <code>plot</code>	405
28.6	Smoother graphs	407
28.7	Multiple curves on a single figure	408
28.8	Two curves with a single <code>plot</code> command	409
28.9	Multiple curves in one figure	410
28.10	Plotting sequences	412
28.11	Plotting curves in separate figures (from a single cell)	412
28.12	Decorating the Curves	415
28.13	Exercises	416
28.14	Getting help from the documentation	417
28.15	P. S. A shortcut revealed: the IPython “magic” command <code>pylab</code>	417
29	Numpy Array Operations and Linear Algebra	419
29.1	The matrix transpose A^T is given by <code>A.T</code>	421
29.2	<i>Slicing</i> : Extracting rows, columns, and other rectangular chunks from matrices	421
30	Package Scipy and More Tools for Linear Algebra	425

30.1	Introduction	425
31	Summation and Integration	427
31.1	Introduction	427
31.2	Exercises	427
31.3	Further Exercises	429
32	Random Numbers, Histograms, and a Simulation	431
32.1	Introduction	431
32.2	Module <code>random</code> within package <code>numpy</code>	431
32.3	Uniformly distributed real numbers: <code>numpy.random.rand</code>	432
32.4	Normally distributed real numbers: <code>numpy.random.randn</code>	433
32.5	Histogram plots with <code>matplotlib.random.hist</code>	433
32.6	Random integers: <code>numpy.random.randint</code>	434
33	Formatted Output and Some Text String Manipulation	439
33.1	The function <code>print()</code>	439
33.2	F-string formatting	440
33.3	Formatting numbers	441
34	Classes, Objects, Attributes, Methods: Very Basic Object-Oriented Programming in Python	447
34.1	Example A: Class <code>VeryBasic3Vector</code>	447
34.2	Example B: Class <code>BasicNVector</code>	448
34.3	Inheritance: new classes that build on old ones	450
35	Exceptions and Exception Handling	453
35.1	Introduction	453
35.2	Handling and Raising Exceptions	453
35.3	Catching any “exceptional” situation and handling it specially	454
35.4	This try/except structure does two things:	455
35.5	Catching and displaying the exception error message	455
35.6	Handling specific exception types	455
35.7	Handling multiple exception types	456
35.8	Summary: a while-try-except pattern for interactive programs	456
35.9	Exercise D-B. Handling division by zero in Newton’s method	457
V	Appendices	459
36	Notebook for generating the module <code>numericalMethods</code>	461
36.1	Index	461
36.2	Zero Finding: solving $f(x) = 0$ or $g(x) = x$	461
36.3	Linear Algebra	469
36.4	Polynomial Collocation and Approximation	475
36.5	Solving Initial Value Problems for Ordinary Differential Equations	476
36.6	For some examples in Chapter Initial Value Problems for Ordinary Differential Equations	486
37	Linear algebra algorithms using 0-based indexing and semi-open intervals	487
37.1	The naive Gaussian elimination algorithm	487
37.2	The LU factorization with L unit lower triangular, by Doolittle’s direct method	488
37.3	Forward substitution with a unit lower triangular matrix	488
37.4	Backward substitution with an upper triangular matrix	489
37.5	Versions with maximal element partial pivoting	490
37.6	Tridiagonal matrix algorithms	492
37.7	Banded matrix algorithms	493

38 Revision notes and plans	495
38.1 Recent changes	495
38.2 To Do	495
39 Bibliography	497
Bibliography	499
Proof Index	501

Brenton LeMesurier College of Charleston, Charleston, South Carolina lemesurierb@cofc.edu, with contributions by Stephen Roberts (Australian National University)

Last Revised June 13, 2024

For notes on recent changes and plans for further revisions, see [Revision notes and plans](#).

This is published at <http://lemesurierb.people.cofc.edu/introduction-to-numerical-methods-and-analysis-python/>

The primary language used for computational examples is Python and the related packages Numpy and Matplotlib, and it also contains a tutorial on using Python with those packages; this is excerpted from the Jupyter book *Python for Scientific Computing* by the same author.

I am working on an evolution of this to cover further topics, and with some more advanced material on analysis of methods, to make it suitable for courses up to introductory graduate level.

There is also a parallel edition that presents examples using the [Julia programming language](#) in place of Python; this can be found at the predictable location <http://lemesurierb.people.cofc.edu/introduction-to-numerical-methods-and-analysis-julia/>

Both of these are based on [Elementary Numerical Analysis with Python](#), my notes for the course *Elementary Numerical Analysis* at the University of Northern Colorado in Spring 2021), in turn based in part on Jupyter notebooks and other materials for the courses MATH 245, MATH 246, MATH 445 and MATH 545 at the College of Charleston, South Carolina, as well as MATH 375 at the University of Northern Colorado.

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

Part I

Frontmatter

INTRODUCTION

This book addresses the design and analysis of methods for computing numerical values for solutions to mathematical problems. Often, only accurate approximations are possible rather than exact solutions, so a key mathematical goal is to assess the accuracy of such approximations.

Given that most numerical methods allow any degree of accuracy to be achieved by working hard enough, the next level of analysis is assessing *cost*, or equivalently *speed*, or more generally the *efficiency of resource usage*. The most natural question then is how much time and other resources are needed to achieve a given degree of accuracy.

1.1 Topics

The main areas of interest are:

1. Finding the zeros of a function: solving $f(x) = 0$.
2. Solving systems of simultaneous linear equations; in matrix-vector notation, solving $Ax = b$ for x .
3. Fitting polynomials to a collection of data points, either exactly (colocation) or approximately (least-squares).
4. Approximating a function by a polynomial, or several polynomials.
5. Approximating derivatives and definite integrals.
6. Solving ordinary differential equations.
7. Finding the minimum of a function.

Although it is the last major topic, the numerical solution of differential equations will often be mentioned earlier as a motivation for other topics. However, we start in a simpler setting: the problem of finding the zeros of a real-valued function: solving $f(x) = 0$.

1.2 Some References

- [Sauer, 2022] *Numerical Analysis* by Timothy Sauer, 3rd edition.
- [Burden *et al.*, 2016] *Numerical Analysis* by Richard L. Burden and J. Douglas Faires, 9th edition.
- [Chenney and Kincaid, 2012] *Numerical Mathematics and Computing* by Ward Cheney and David Kincaid.
- [Kincaid and Cheney, 1990] *Numerical Analysis* by David Kincaid and Ward Cheney, Brooks/Cole, 1990.
- [SciPy Lecture Notes] online at <https://scipy-lectures.org/>; a free reference mainly on the SciPy package, but also with some general information on using Python for scientific computing. It is available both as a web-site or downloadable as PDF or HTML — for off-line access, I prefer the HTML version rather than the PDF.

See also the *Bibliography*.

Part II

Main

ROOT-FINDING

2.1 Root Finding by Interval Halving (Bisection)

References:

- Section 1.1 *The Bisection Method* in *Numerical Analysis* by Sauer [Sauer, 2022]
- Section 2.1 *The Bisection Method* in *Numerical Analysis* by Burden, Faires and Burden [Burden *et al.*, 2016]

(See the *Bibliography*.)

2.1.1 Introduction

One of the most basic tasks in numerical computing is finding the roots (or “zeros”) of a function — solving the equation $f(x) = 0$ where $f : \mathbb{R} \rightarrow \mathbb{R}$ is a continuous function from and to the real numbers. As with many topics in this course, there are multiple methods that work, and we will often start with the simplest and then seek improvement in several directions:

- **reliability** or *robustness* — how good it is at avoiding problems in hard cases, such as division by zero.
- *accuracy* and guarantees about accuracy like estimates of how large the error can be — since in most cases, the result cannot be computed exactly.
- *speed* or *cost* — often measure by minimizing the amount of arithmetic involved, or the number of times that a function must be evaluated.

Example 1.1.1 (Solve $x = \cos x$)

This is a simple equation for which there is no exact formula for a solution, but we can easily ensure that there is a solution, and moreover, a unique one. It is convenient to put the equation into “zero-finding” form $f(x) = 0$, by defining

$$f(x) := x - \cos x.$$

Also, note that $|\cos x| \leq 1$, so a solution to the original equation must have $|x| \leq 1$. So we will start graphing the function on the interval $[a, b] = [-1, 1]$.

Remark 1.1.1 (On Python)

This is our first use of two Python packages that you might not have seen before: *Numpy* and *Matplotlib*. If you want to learn more about them, see for example the Python Review sections on *Python Variables, Lists, Tuples, and Numpy arrays* and *Graphing with Matplotlib*

Or for now, just learn from the examples here.

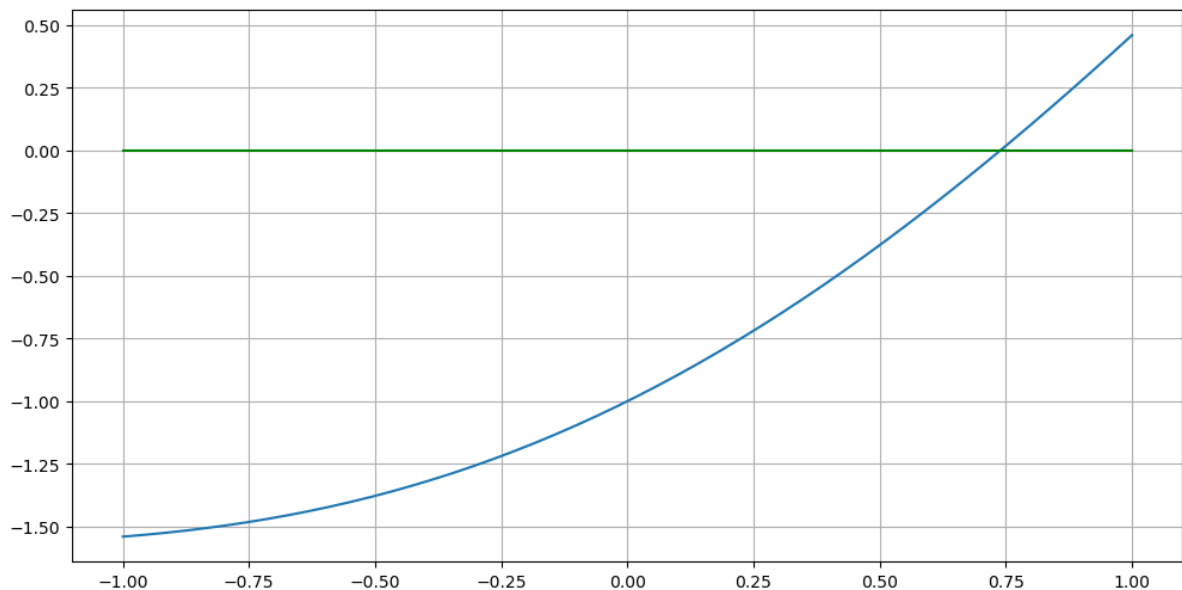
```
# We will often need resources from the modules numpy and pyplot:
import numpy as np
import matplotlib.pyplot as plt

# We can also import items from a module individually, so they can be used by "first_
# name only".
# Here this is done for mathematical functions; in some later sections it will be_
# done for all imports.
from numpy import cos
```

```
def f(x):
    return x - cos(x)
```

```
a = -1; b = 1
```

```
x = np.linspace(a, b) # 50 equally spaced values from a to b
plt.figure(figsize=(12,6)) # Create an "empty" graph, 12 wide, 6 high
plt.plot(x, f(x))
plt.plot([a, b], [0, 0], 'g') # Mark the x-axis in green
plt.grid(True); # Add a graph paper background
```



```
# If you want to see what "linspace" gives, run this cell
print(x)
```

```
[-1.          -0.95918367 -0.91836735 -0.87755102 -0.83673469 -0.79591837
 -0.75510204 -0.71428571 -0.67346939 -0.63265306 -0.59183673 -0.55102041
 -0.51020408 -0.46938776 -0.42857143 -0.3877551  -0.34693878 -0.30612245
 -0.26530612 -0.2244898  -0.18367347 -0.14285714 -0.10204082 -0.06122449
 -0.02040816  0.02040816  0.06122449  0.10204082  0.14285714  0.18367347
  0.2244898   0.26530612  0.30612245  0.34693878  0.3877551   0.42857143]
```

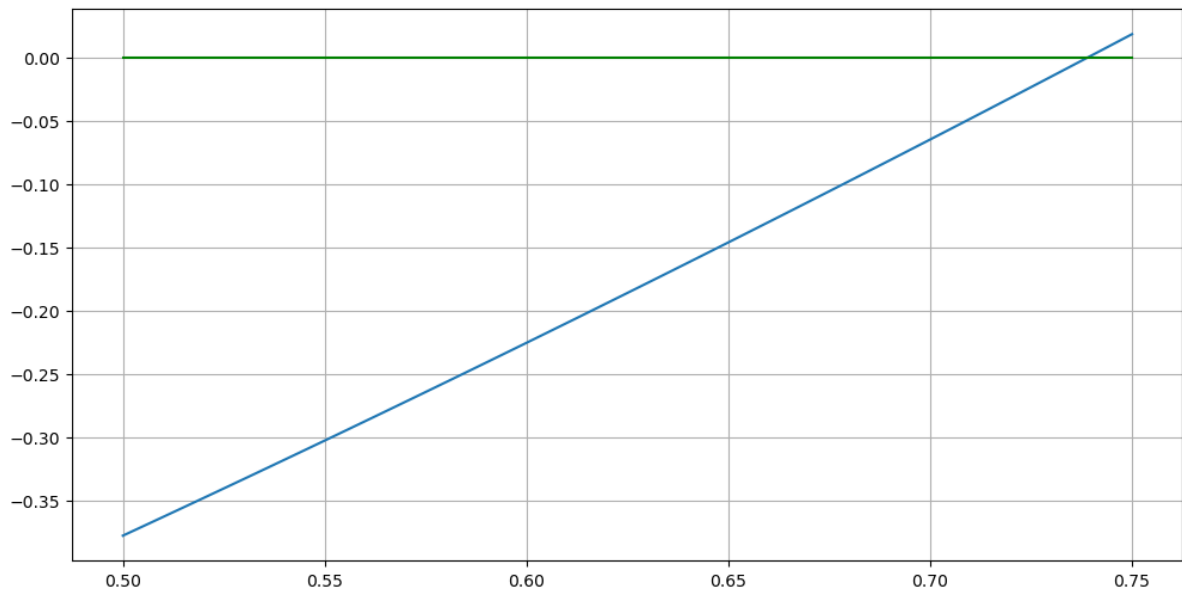
(continues on next page)

(continued from previous page)

```
0.46938776 0.51020408 0.55102041 0.59183673 0.63265306 0.67346939
0.71428571 0.75510204 0.79591837 0.83673469 0.87755102 0.91836735
0.95918367 1.          ]
```

This shows that the zero lies between 0.5 and 0.75, so zoom in:

```
a = 0.5; b = 0.75
x = np.linspace(a, b)
plt.figure(figsize=(12,6))
plt.plot(x, f(x))
plt.plot([a, b], [0, 0], 'g')
plt.grid(True);
```



And we could repeat, getting an approximation of any desired accuracy.

However this has two weaknesses: it is very inefficient (the function is evaluated about fifty times at each step in order to draw the graph), and it requires lots of human intervention.

To get a procedure that can be efficiently implemented in Python (or another programming language of your choice), we extract one key idea here: finding an interval in which the function changes sign, and then repeatedly find a smaller such interval within it. The simplest way to do this is to repeatedly divide an interval known to contain the root in half and check which half has the sign change in it.

Graphically, let us start again with interval $[a, b] = [-1, 1]$, but this time focus on three points of interest: the two ends and the midpoint, where the interval will be bisected:

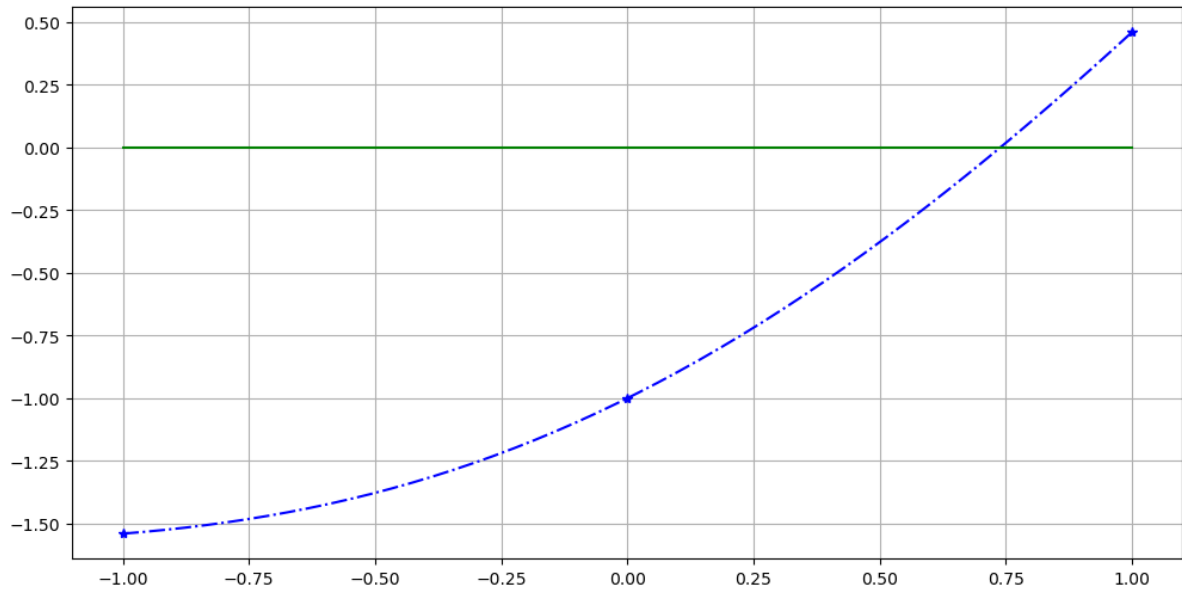
```
a = -1
b = 1
c = (a+b)/2
```

```
acb = [a, c, b]
plt.figure(figsize=(12,6))
plt.plot(acb, f(acb), 'b*')
# And just as a visual aid:
```

(continues on next page)

(continued from previous page)

```
x = np.linspace(a, b)
plt.plot(x, f(x), 'b-.')
plt.plot([a, b], [0, 0], 'g')
plt.grid(True);
```

**Remark 1.1.2 (On Python)**

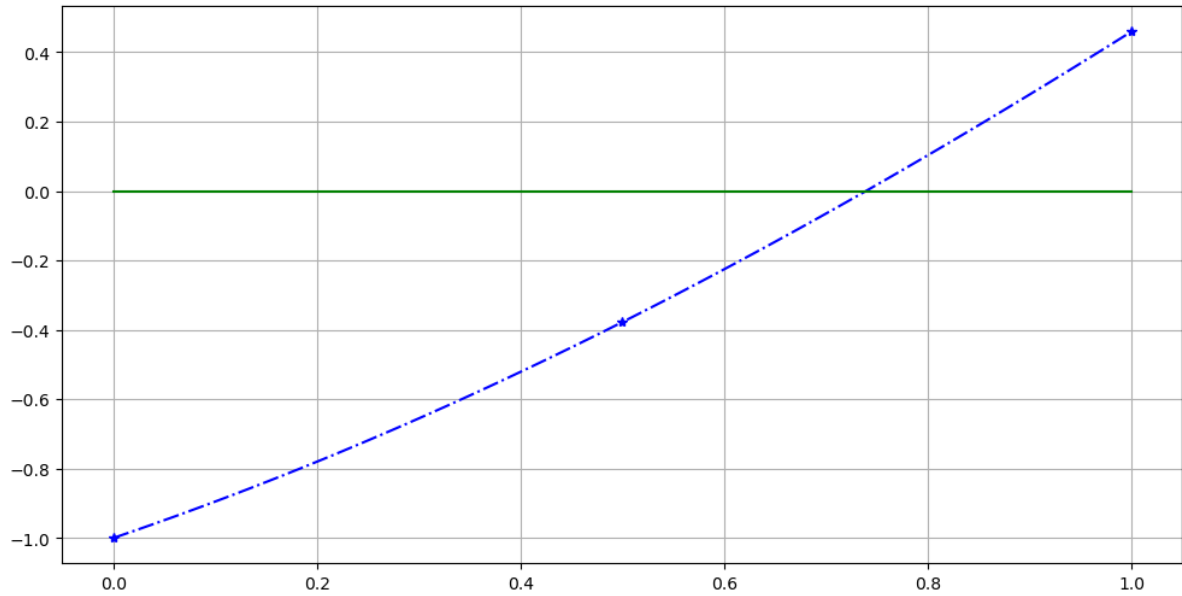
Note on line 3 above that the function `cos` from Numpy (full name `numpy.cos`) can be evaluated simultaneously on a list of numbers; the version `math.cos` from module `math` can only handle one number at a time. This is one reason why we will avoid `math` in favor of `numpy`.

$f(a)$ and $f(c)$ have the same sign, while $f(c)$ and $f(b)$ have opposite signs, so the root is in $[c, b]$; update the a, b, c values and plot again:

```
a = c # new left end is old center
b = b # redundant, as the right end is unchanged
c = (a+b)/2
print(f"{a=}, {b=}, {c=}")
```

```
a=0.0, b=1, c=0.5
```

```
acb = [a, c, b]
x = np.linspace(a, b)
plt.figure(figsize=(12,6))
plt.plot(acb, f(acb), 'b*', x, f(x), 'b-.')
plt.plot([a, b], [0, 0], 'g')
plt.grid(True);
```

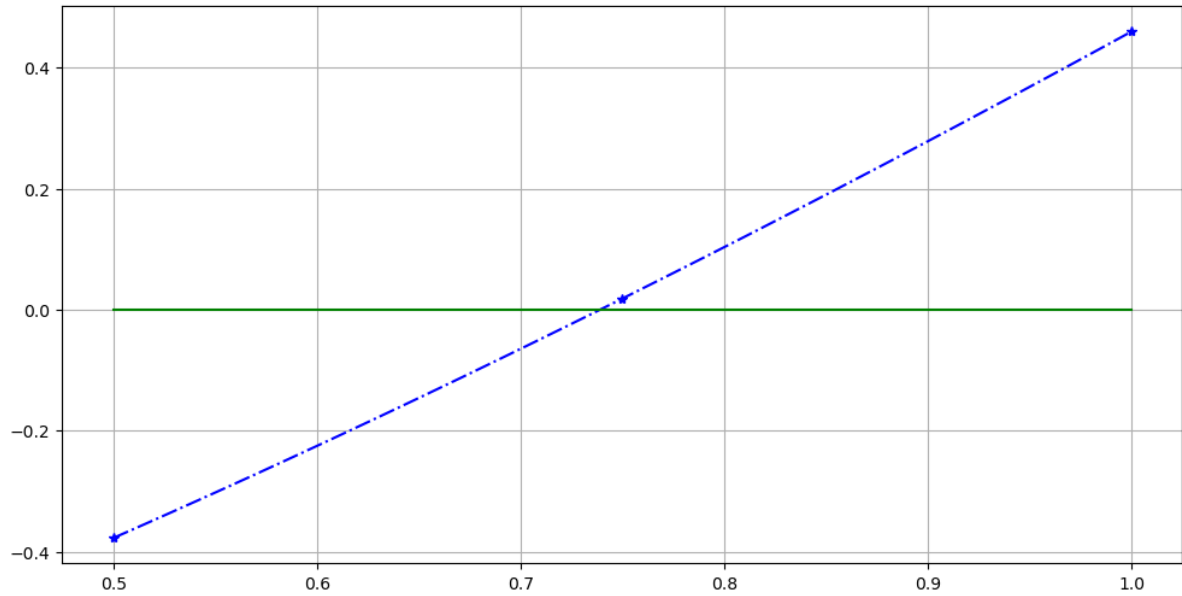


Again $f(c)$ and $f(b)$ have opposite signs, so the root is in $[c, b]$, and ...

```
a = c # new left end is old center again
# skipping the redundant "b = b" this time
c = (a+b)/2
print(f"{a=}, {b=}, {c=}")
```

```
a=0.5, b=1, c=0.75
```

```
acb = [a, c, b]
x = np.linspace(a, b)
plt.figure(figsize=(12,6))
plt.plot(acb, f(acb), 'b*', x, f(x), 'b-.')
plt.plot([a, b], [0, 0], 'g')
plt.grid(True);
```

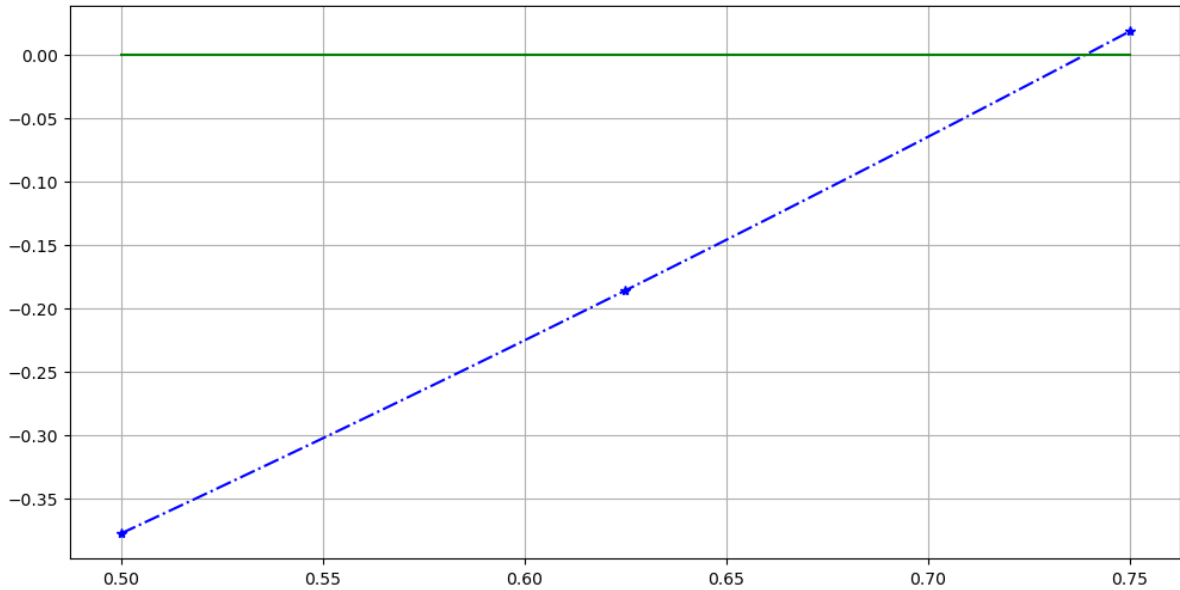


This time $f(a)$ and $f(c)$ have opposite sign, so the root is at left, in $[a, c]$:

```
# this time, the value of a does not need to be updated ...
b = c # ... and the new right end is the former center
c = (a+b)/2
print(f"{a=}, {b=}, {c=}")
```

```
a=0.5, b=0.75, c=0.625
```

```
acb = [a, c, b]
x = np.linspace(a, b)
plt.figure(figsize=(12,6))
plt.plot(acb, f(acb), 'b*', x, f(x), 'b-.')
plt.plot([a, b], [0, 0], 'g')
plt.grid(True);
```



2.1.2 A first algorithm for the bisection method

Now it is time to dispense with the graphs, and describe the procedure in mathematical terms:

- if $f(a)$ and $f(c)$ have opposite signs, the root is in interval $[a, c]$, which becomes the new version of interval $[a, b]$.
- otherwise, $f(c)$ and $f(b)$ have opposite signs, so the root is in interval $[c, b]$

Pseudo-code for describing algorithms

As a useful bridge from the mathematical description of an algorithm with words and formulas to actual executable code, these notes will often describe algorithms in *pseudo-code* — a mix of words and mathematical formulas with notation that somewhat resembles code in a language like Python.

This is also preferable to going straight to code in a particular language (such as Python) because it makes it easier if, later, you wish to implement algorithms in a different programming language.

Note well one feature of the pseudo-code used here: **assignment** is denoted with a left arrow:

$x \leftarrow a$

is the instruction to cause the value of variable x to become the current value of a .

This is to distinguish from

$x = a$

which is a **comparison**: the true-or-false assertion that the two quantities *already* have the same value.

Unfortunately however, Python (like most programming languages) does not use this notation: instead assignment is done with $x = a$ so that asserting equality needs a different notation: this is done with $x == a$; note well that double equal sign!

Also, the pseudo-code marks the end of blocks like `if`, `for` and `while` with a line `end`. Many programming languages do something like this, but Python does not: instead it uses only the end of indentation as the indication that a block is finished.

With those notational issues out of the way, the key step in the bisection strategy is the update of the interval:

Algorithm 1.1.1 (one step of bisection)

```

$$c \leftarrow \frac{a+b}{2}$$
if  $f(a)f(c) < 0$  then:  
     $b \leftarrow c$   
else:  
     $a \leftarrow c$   
end
```

This needs to be repeated a finite number of times, and the simplest way is to specify the number of iterations. (We will consider more refined methods soon.)

Algorithm 1.1.2 (bisection, first version)

- Get an initial interval $[a, b]$ with a sign-change: $f(a)f(b) < 0$.
 - Choose N , the number of iterations.
 - for i from 1 to N :

$$c \leftarrow \frac{a+b}{2}$$
 if $f(a)f(c) < 0$ then:
 $b \leftarrow c$
 else:
 $a \leftarrow c$
 end
end
• The approximate root is the final value of c .
-

A Python version of the iteration is not a lot different:

```
for i in range(N):  
    c = (a+b)/2  
    if f(a) * f(c) < 0:  
        b = c  
    else:  
        a = c
```

(If you wish to review for loops in Python, see the Python Review section on [Iteration with for](#))

Exercise 1

Create a Python function `bisection1` which implements the first algorithm for bisection above, which performs a fixed number N of iterations; the usage should be: `root = bisection1(f, a, b, N)`

Test it with the above example: $f(x) = x - \cos x = 0$, $[a, b] = [-1, 1]$

(If you wish to review the defining and use of functions in Python, see the Python Review section on [Defining and Using Python Functions](#))

2.1.3 Error bounds, and a more refined algorithm

The above method of iteration for a fixed number of times is simple, but usually not what is wanted in practice. Instead, a better goal is to get an approximation with a guaranteed maximum possible error: a result consisting of an approximation $\tilde{r}$ to the exact root r and also a bound E_{max} on the maximum possible error; a guarantee that $|r - \tilde{r}| \leq E_{max}$. To put it another way, a guarantee that the root r lies in the interval $[\tilde{r} - E_{max}, \tilde{r} + E_{max}]$.

In the above example, each iteration gives a new interval $[a, b]$ guaranteed to contain the root, and its midpoint $c = (a + b)/2$ is with a distance $(b - a)/2$ of any point in that interval, so at each iteration, we can have:

- $\tilde{r}$ is the current value of $c = (a + b)/2$
- $E_{max} = (b - a)/2$

2.1.4 Error tolerances and stopping conditions

The above algorithm can *passively* state an error bound, but it is better to be able to solve to a desired degree of accuracy; for example, if we want a result “accurate to three decimal places”, we can specify $E_{max} \leq 0.5 \times 10^{-3}$.

So our next goal is to *actively* set an accuracy target or *error tolerance* E_{tol} and keep iterating until it is met. This can be achieved with a `while` loop; here is a suitable algorithm:

Algorithm 1.1.3 (bisection with error tolerance)

- Input function f , interval endpoints a and b , and an error tolerance E_{tol}
 - Evaluate $E_{max} = (b - a)/2$
 - while $E_{max} > E_{tol}$: $c \leftarrow (a + b)/2$ if $f(a)f(c) < 0$ then: $b \leftarrow c$ else: $a \leftarrow c$ end
 $E_{max} \leftarrow (b - a)/2$ end
 - Output $\tilde{r} = c$ as the approximate root and E_{max} as a bound on its absolute error.
-

(If you wish to review while loops, see the Python Review section on [Iteration with while](#))

Exercise 2

Create a Python function implementing this better algorithm, with usage `root = bisection2(f, a, b, E_tol)`

Test it with the above example: $f(x) = x - \cos x$, $[a, b] = [-1, 1]$, this time accurate to within 10^{-4} .

Use the fact that there is a solution in the interval $(-1, 1)$.

2.2 Solving Equations by Fixed Point Iteration (of Contraction Mappings)

References:

- Sections 6.1.1 *Euler's Method* in [Sauer, 2022]
- Section 5.2 *Euler's Method* in [Burden *et al.*, 2016]
- Sections 7.1 and 7.2 of [Chenney and Kincaid, 2012]

2.2.1 Introduction

In the next section we will meet *Newton's Method for Solving Equations* for root-finding, which you might have seen in a calculus course. This is one very important example of a more general strategy of **fixed-point iteration**, so we start with that.

```
# We will often need resources from the modules numpy and pyplot:
import numpy as np

# We can also import items from a module individually, so they can be used by "first-
↪name only".
# In this book this is done mostly for mathematical functions with familiar names:
from numpy import cos
# and some very frequently use functions for graphing:
from matplotlib.pyplot import figure, plot, title, legend, grid
```

2.2.2 Fixed-point equations

A variant of stating equations as *root-finding* ($f(x) = 0$) is *fixed-point* form: given a function $g : \mathbb{R} \rightarrow \mathbb{R}$ or $g : \mathbb{C} \rightarrow \mathbb{C}$ (or even $g : \mathbb{R}^n \rightarrow \mathbb{R}^n$; a later topic), find a *fixed point* of g . That is, a value p for its argument such that

$$g(p) = p$$

Such problems are interchangeable with root-finding. One way to convert from $f(x) = 0$ to $g(x) = x$ is defining

$$g(x) := x - w(x)f(x)$$

for any “weight function” $w(x)$.

One can convert the other way too, for example defining $f(x) := g(x) - x$. We have already seen this when we converted the equation $x = \cos x$ to $f(x) = x - \cos x = 0$.

Compare the two setups graphically: in each case, the x value at the intersection of the two curves is the solution we seek.

```
def f_1(x): return x - cos(x)
def g_1(x): return cos(x)
```

```
a = -1
b = 1

x = np.linspace(a, b)
```

(continues on next page)

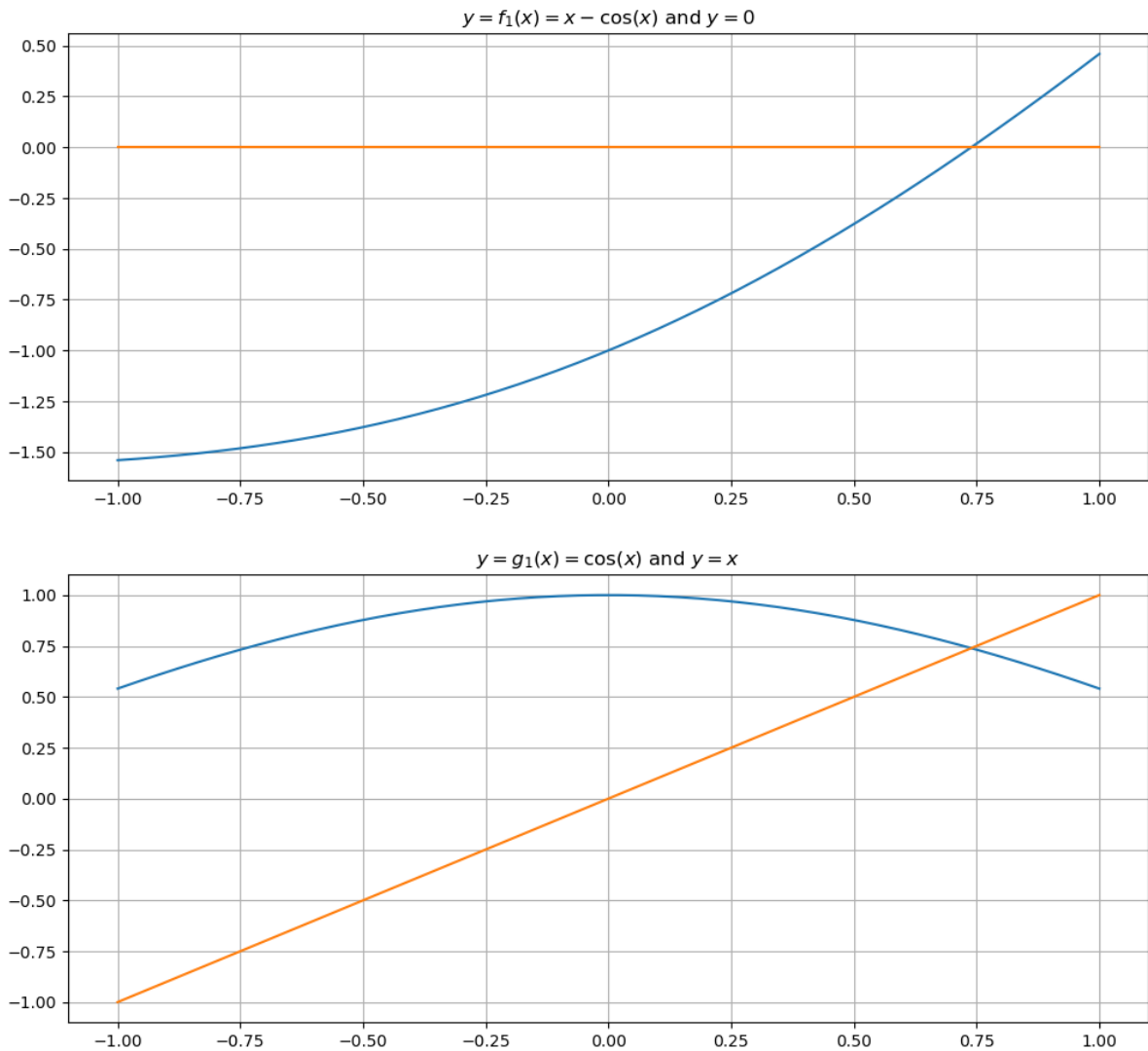
(continued from previous page)

```

figure(figsize=(12,5))
title("$y = f_1(x) = x - \cos(x)$ and $y=0$")
plot(x, f_1(x))
plot([a, b], [0, 0])
grid(True)

figure(figsize=(12,5))
title("$y = g_1(x) = \cos(x)$ and $y=x$")
plot(x, g_1(x))
plot(x, x)
grid(True)

```



The fixed point form can be convenient partly because we almost always have to solve by successive approximations, or *iteration*, and fixed point form suggests *one* choice of iterative procedure: start with any first approximation x_0 , and iterate with

$$x_1 = g(x_0), x_2 = g(x_1), \dots, x_{k+1} = g(x_k), \dots$$

Proposition 1.2.1

If g is continuous, and **if** the above sequence $\{x_0, x_1, \dots\}$ converges to a limit p , then that limit is a fixed point of function g : $g(p) = p$.

Proof. From $\lim_{k \rightarrow \infty} x_k = p$, continuity gives

$$\lim_{k \rightarrow \infty} g(x_k) = g(p).$$

On the other hand, $g(x_k) = x_{k+1}$, so

$$\lim_{k \rightarrow \infty} g(x_k) = \lim_{k \rightarrow \infty} x_{k+1} = p.$$

Comparing gives $g(p) = p$.

That second “if” is a big one. Fortunately, it can often be resolved using the idea of a *contraction mapping*.

Definition 1.2.1 (Mapping)

A function $g(x)$ defined on a closed interval $D = [a, b]$ which sends values back into that interval, $g : D \rightarrow D$, is sometimes called a *map* or *mapping*.

(*Aside:* The same applies for a function $g : D \rightarrow D$ where D is a subset of the complex numbers, or even of vectors $\mathbb{R}^n$ or $\mathbb{C}^n$.)

A mapping is sometimes thought of as moving a region S within its domain D to another such region, by moving each point $x \in S \subset D$ to its image $g(x) \in g(S) \subset D$.

A very important case is mappings that shrink the region, by reducing the distance between points:

Proposition 1.2.2

Any continuous mapping on a closed interval $[a, b]$ has at least one fixed point.

Proof. Consider the “root-finding cousin”, $f(x) = x - g(x)$.

First, $f(a) = a - g(a) \leq 0$, since $g(a) \geq a$ so as to be in the domain $[a, b]$ — similarly, $f(b) = b - g(b) \geq 0$.

From the Intermediate Value Theorem, f has a zero p , where $f(p) = p - g(p) = 0$.

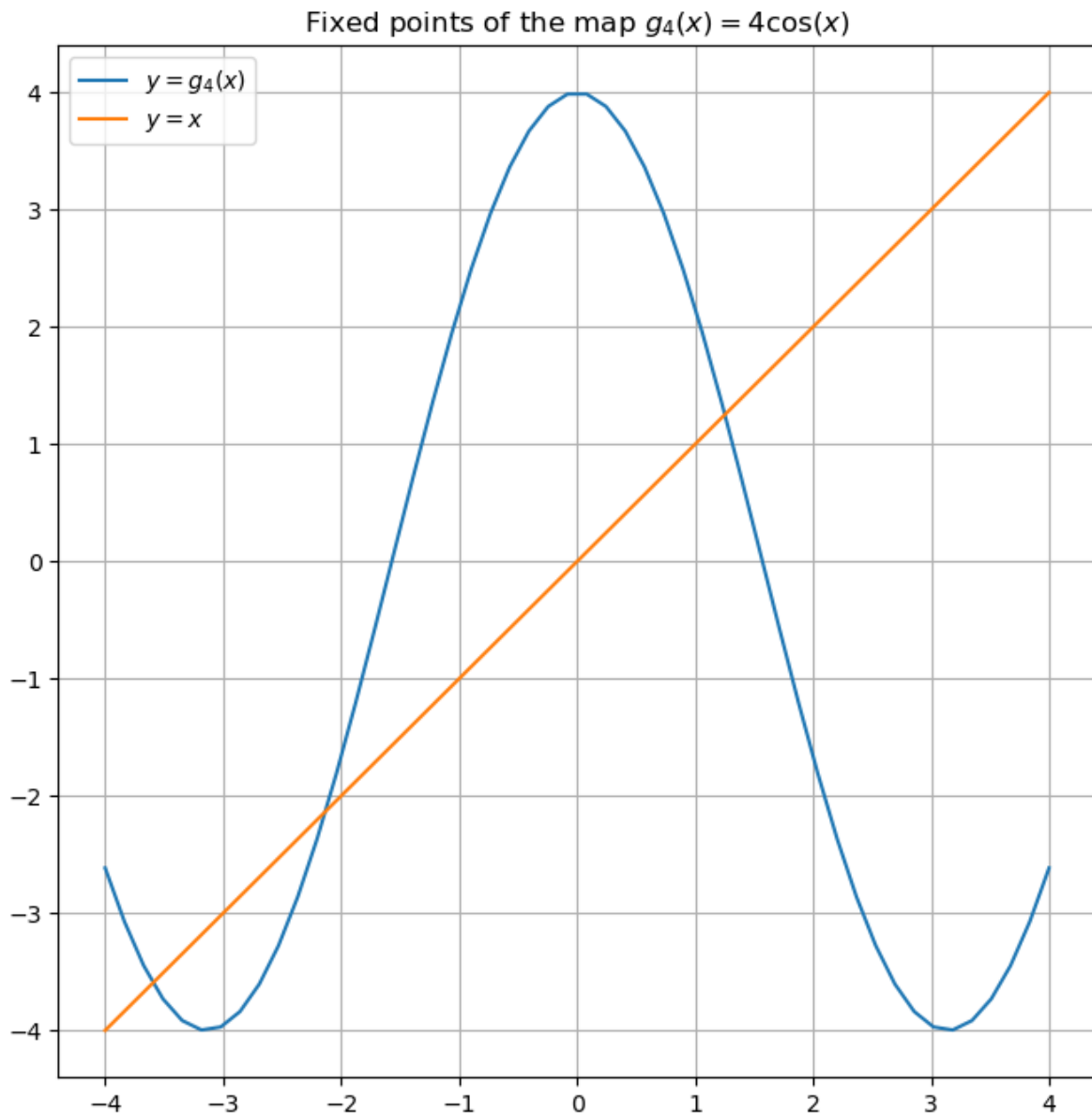
In other words, the graph of $y = g(x)$ goes from being above the line $y = x$ at $x = a$ to below it at $x = b$, so at some point $x = p$, the curves meet: $y = x = p$ and $y = g(p)$, so $p = g(p)$.

Example 1.2.1

Let us illustrate this with the mapping $g_4(x) := 4 \cos x$, for which the fact that $|g_4(x)| \leq 4$ ensures that this is a map of the domain $D = [-4, 4]$ into itself:

```
def g_4(x): return 4*cos(x)
a = -4
b = 4
x = np.linspace(a, b)

figure(figsize=(8,8))
title("Fixed points of the map  $g_4(x) = 4 \cos(x)$ ")
plot(x, g_4(x), label="$y=g_4(x)$")
plot(x, x, label="$y=x$")
legend()
grid(True);
```



This example has multiple fixed points (three of them). To ensure both the existence of a **unique** solution, and coverage of the iteration to that solution, we need an extra condition.

Definition 1.2.2 (Contraction Mapping)

A mapping $g : D \rightarrow D$, is called a *contraction* or *contraction mapping* if there is a constant $C < 1$ such that

$$|g(x) - g(y)| \leq C|x - y|$$

for any x and y in D . We then call C a *contraction constant*.

(*Aside:* The same applies for a domain in $\mathbb{R}^n$: just replace the absolute value $|\dots|$ by the vector norm $\|\dots\|$.)

Remark 1.2.1

It is not enough to have $|g(x) - g(y)| < |x - y|$ or $C = 1$! We need the ratio $\frac{|g(x) - g(y)|}{|x - y|}$ to be *uniformly* less than one for all possible values of x and y .

Theorem 1.2.1 (A Contraction Mapping Theorem)

Any contraction mapping on a closed, bounded interval $D = [a, b]$ has exactly one fixed point p in D . Further, this can be calculated as the limit $p = \lim_{k \rightarrow \infty} x_k$ of the iteration sequence given by $x_{k+1} = g(x_k)$ for *any* choice of the starting point $x_0 \in D$.

Proof. The main idea of the proof can be shown with the help of a few pictures.

First, uniqueness: between any two of the multiple fixed points above — call them p_0 and p_1 — the graph of $g(x)$ has to rise with secant slope 1: $(g(p_1) - g(p_0))/(p_1 - p_0) = (p_1 - p_0)/(p_1 - p_0) = 1$, and this violates the contraction property.

So instead, for a contraction, the graph of a contraction map looks like the one below for our favorite example, $g(x) = \cos x$ (which we will soon verify to be a contraction on interval $[-1, 1]$):

The second claim, about **convergence** to the fixed point from any initial approximation x_0 , will be verified below, once we have seen some ideas about measuring errors.

An easy way of checking whether a differentiable function is a contraction

With differentiable functions, the contraction condition can often be easily verified using derivatives:

Theorem 1.2.2 (A derivative-based fixed point theorem)

If a function $g : [a, b] \rightarrow [a, b]$ is differentiable and there is a constant $C < 1$ such that $|g'(x)| \leq C$ for all $x \in [a, b]$, then g is a contraction mapping, and so has a unique fixed point in this interval.

Proof. Using the Mean Value Theorem, $g(x) - g(y) = g'(c)(x - y)$ for some c between x and y . Then taking absolute values,

$$|g(x) - g(y)| = |g'(c)| \cdot |x - y| \leq C|x - y|.$$

Example 1.2.2 ($g(x) = \cos(x)$ is a contraction on interval $[-1, 1]$)

Our favorite example $g(x) = \cos(x)$ is a contraction, but we have to be a bit careful about the domain.

For all real x , $g'(x) = -\sin x$, so $|g'(x)| \leq 1$; this is almost but not quite enough.

However, we have seen that iteration values will settle in the interval $D = [-1, 1]$, and considering g as a mapping of this domain, $|g'(x)| \leq \sin(1) = 0.841 \dots < 1$: that is, now we have a contraction, with $C = \sin(1) \approx 0.841$.

And as seen in the graph above, there is indeed a unique fixed point.

The contraction constant C as a measure of how fast the approximations improve (the smaller the better)

It can be shown that if C is small (at least when one looks only at a reduced domain $|x - p| < R$) then the convergence is “fast” once $|x_k - p| < R$.

To see this, we define some jargon for talking about errors. (For more details on error concepts, see section [Measures of Error and Order of Convergence](#).)

Definition 1.2.3 (Error)

The *error* in $\tilde{x}$ as an approximation to an exact value x is

$$\text{error} := (\text{approximation}) - (\text{exact value}) = \tilde{x} - x$$

This will often be abbreviated as E .

Definition 1.2.4 (Absolute Error)

The *absolute error* in $\tilde{x}$ an approximation to an exact value x is the magnitude of the error: the absolute value $|E| = |\tilde{x} - x|$.

(*Aside:* This will later be extended to x and $\tilde{x}$ being vectors, by again using the vector norm in place of the absolute value. In fact, I will sometimes blur the distinction by using the “single line” absolute value notation for vector norms too.)

In the case of x_k as an approximation of p , we name the error $E_k := x_k - p$. Then C measures a worst case for how fast the error decreases as k increases, and this is “exponentially fast”:

Proposition 1.2.3

$|E_{k+1}| \leq C|E_k|$, or $|E_{k+1}|/|E_k| \leq C$, and so

$$|E_k| \leq C^k |x_0 - p|$$

That is, the error decreases at worst in a geometric sequence, which is exponential decrease with respect to the variable k .

Proof. $E_{k+1} = x_{k+1} - p = g(x_k) - g(p)$, using $g(p) = p$. Thus the contraction property gives

$$|E_{k+1}| = |g(x_k) - g(p)| \leq C|x_k - p| = C|E_k|$$

Applying this again,

$$|E_k| \leq C|E_{k-1}| \leq C \cdot C|E_{k-2}| = C^2|E_{k-2}|$$

and repeating $k - 2$ more times,

$$|E_k| \leq C^k|E_0| = C^k|x_0 - p|.$$

Remark 1.2.2

We will often use this “recursive” strategy of relating the error in one iterate to that in the previous iterate.

Example 1.2.3 (Solving $x = \cos x$ with a naive fixed point iteration)

We have seen that *one* way to convert the example $f(x) = x - \cos x = 0$ to a fixed point iteration is $g(x) = \cos x$, and that this is a contraction on $D = [-1, 1]$

Here is what this iteration looks like:

```
a = 0
b = 1
x = np.linspace(a, b)
iterations = 10

# Start at left
print(f"Solving x = cos(x) starting to the left, at x_0 = {a}")
x_k = a
figure(figsize=(8,8))
title(f"Solving $x = \cos x$ starting to the left, at $x_0$ = {a}")
plot(x, x, "g")
plot(x, g(x), "r")
grid(True)
for k in range(iterations):
    g_x_k = g_1(x_k)
    # Graph evaluation of g(x_k) from x_k:
    plot([x_k, x_k], [x_k, g(x_k)], "b")
    x_k_plus_1 = g_1(x_k)
    # Connect to the new x_k on the line y = x:
    plot([x_k, g_1(x_k)], [x_k_plus_1, x_k_plus_1], "b")
    # Update names: the old x_k+1 is the new x_k
    x_k = x_k_plus_1
    print(f"x_{k+1} = {x_k_plus_1}")

# Start at right
print(f"Solving x = cos(x) starting to the right, at x_0 = {b}")
x_k = b
figure(figsize=(8,8))
title(f"Solving $x = \cos(x)$ starting to the right, at $x_0$ = {b}")
plot(x, x, "g")
plot(x, g(x), "r")
grid(True)
for k in range(iterations):
    g_x_k = g_1(x_k)
```

(continues on next page)

(continued from previous page)

```

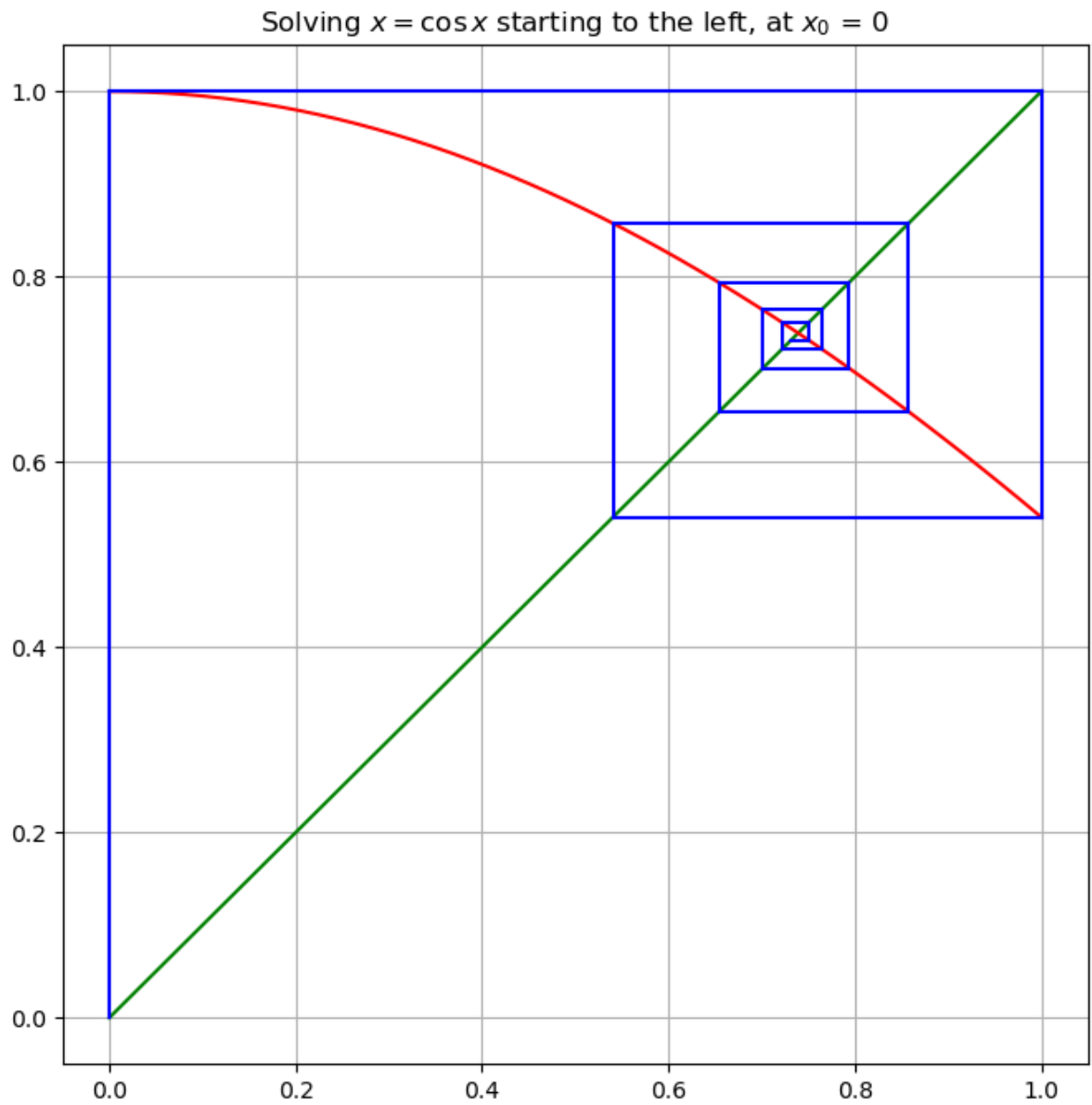
# Graph evalation of g(x_k) from x_k:
plot([x_k, x_k], [x_k, g(x_k)], "b")
x_k_plus_1 = g_1(x_k)
#Connect to the new x_k on the line y = x:
plot([x_k, g_1(x_k)], [x_k_plus_1, x_k_plus_1], "b")
# Update names: the old x_k+1 is the new x_k
x_k = x_k_plus_1
print(f"x_{k+1} = {x_k_plus_1}")

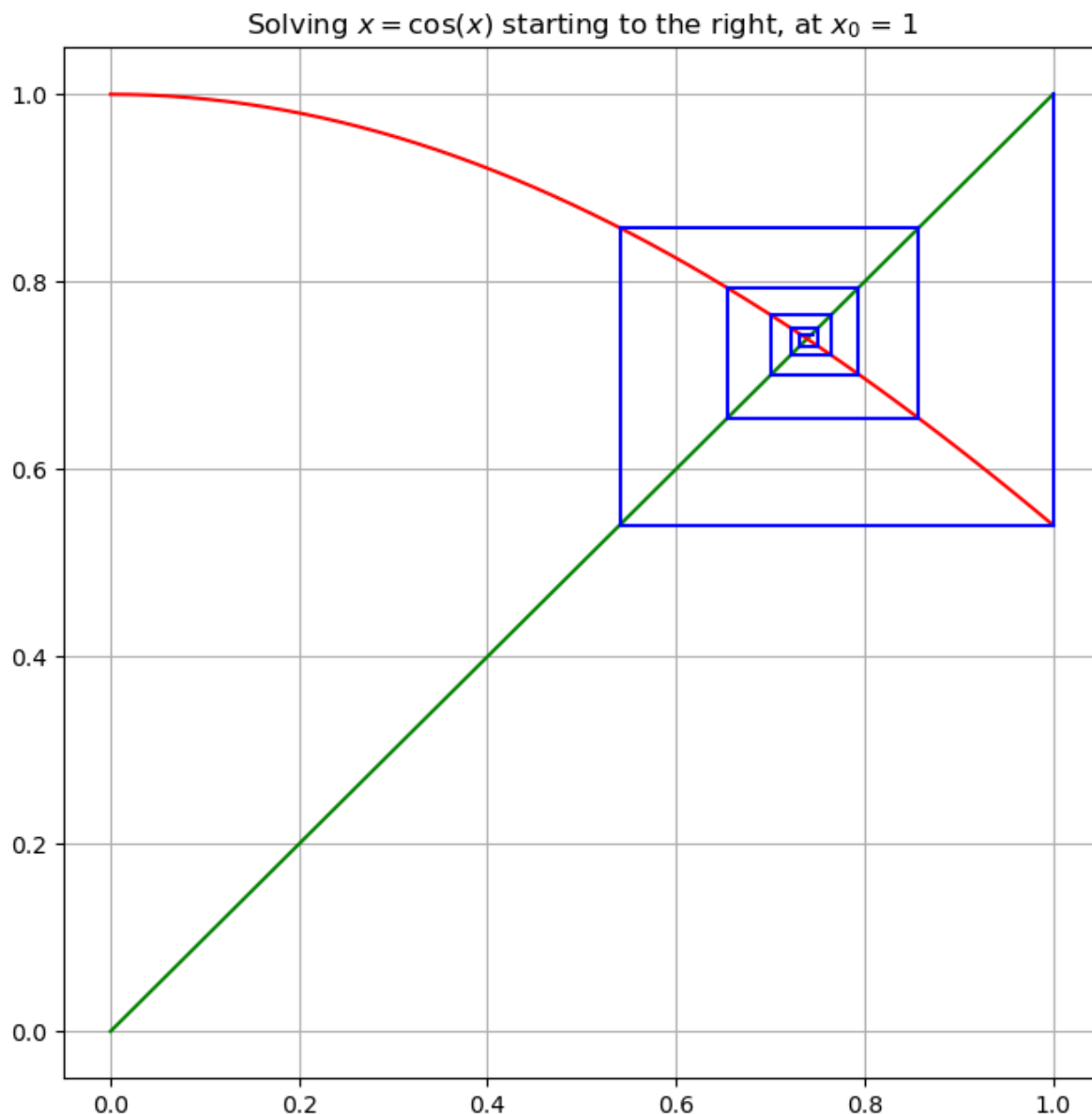
```

```

Solving x = cos(x) starting to the left, at x_0 = 0
x_1 = 1.0
x_2 = 0.5403023058681398
x_3 = 0.8575532158463933
x_4 = 0.6542897904977792
x_5 = 0.7934803587425655
x_6 = 0.7013687736227566
x_7 = 0.7639596829006542
x_8 = 0.7221024250267077
x_9 = 0.7504177617637605
x_10 = 0.7314040424225098
Solving x = cos(x) starting to the right, at x_0 = 1
x_1 = 0.5403023058681398
x_2 = 0.8575532158463933
x_3 = 0.6542897904977792
x_4 = 0.7934803587425655
x_5 = 0.7013687736227566
x_6 = 0.7639596829006542
x_7 = 0.7221024250267077
x_8 = 0.7504177617637605
x_9 = 0.7314040424225098
x_10 = 0.7442373549005569

```





In each case, one gets a “box spiral” in to the fixed point. It always looks like this when g is *decreasing* near the fixed point.

If instead g is *increasing* near the fixed point, the iterates approach monotonically, either from above or below:

Example 1.2.4 (Solving $f(x) = x^2 - 5x + 4 = 0$ in interval $[0, 3]$)

The roots are 1 and 4; for now we aim at the first of these, so we chose a domain $[0, 3]$ that contains just this root.

Let us get a fixed point for by “partially solving for x ”: solving for the x in the $5x$ term:

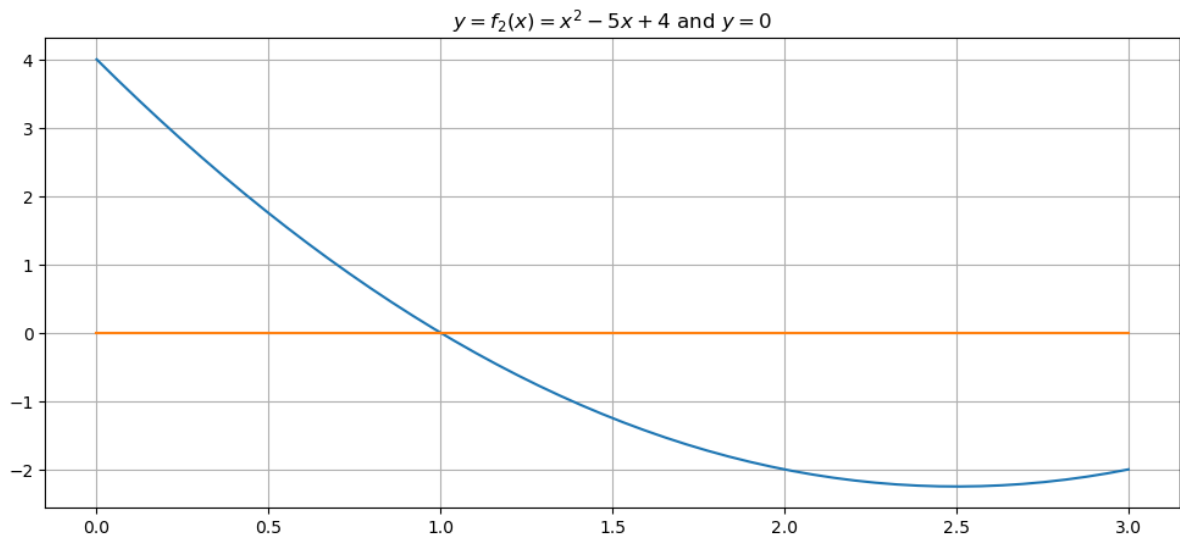
$$x = g(x) = (x^2 + 4)/5$$

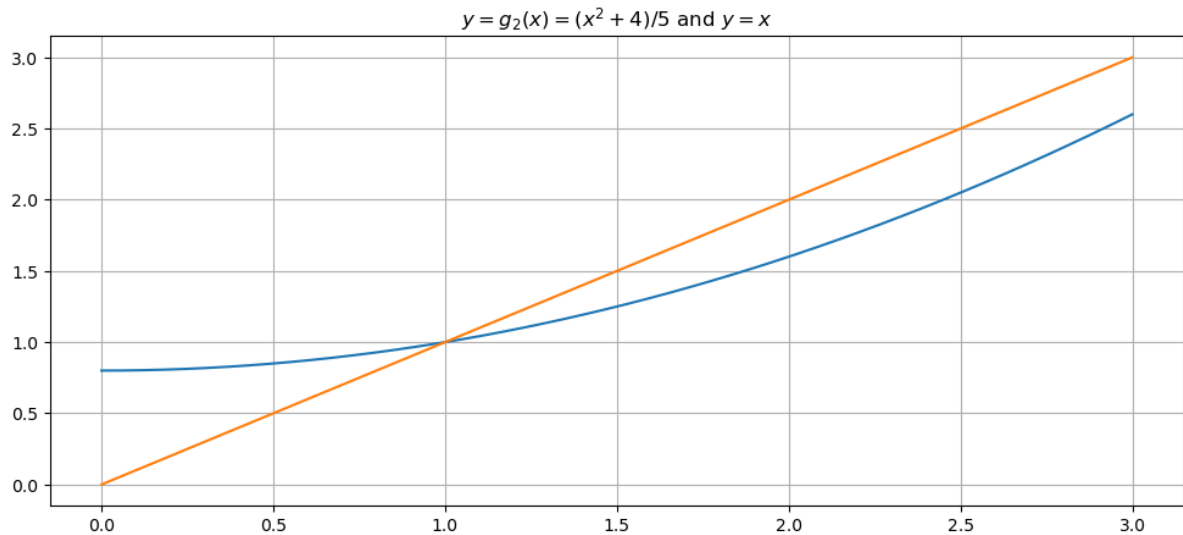
```
def f_2(x): return x**2 - 5*x + 4
def g_2(x): return (x**2 + 4)/5
```

```
a = 0
b = 3
x = np.linspace(a, b)

figure(figsize=(12,5))
title("$y = f_2(x) = x^2-5x+4$ and $y = 0$")
plot(x, f_2(x))
plot([a, b], [0, 0])
grid(True)

figure(figsize=(12,5))
title("$y = g_2(x) = (x^2 + 4)/5$ and $y=x$")
plot(x, g_2(x))
plot(x, x)
grid(True)
```

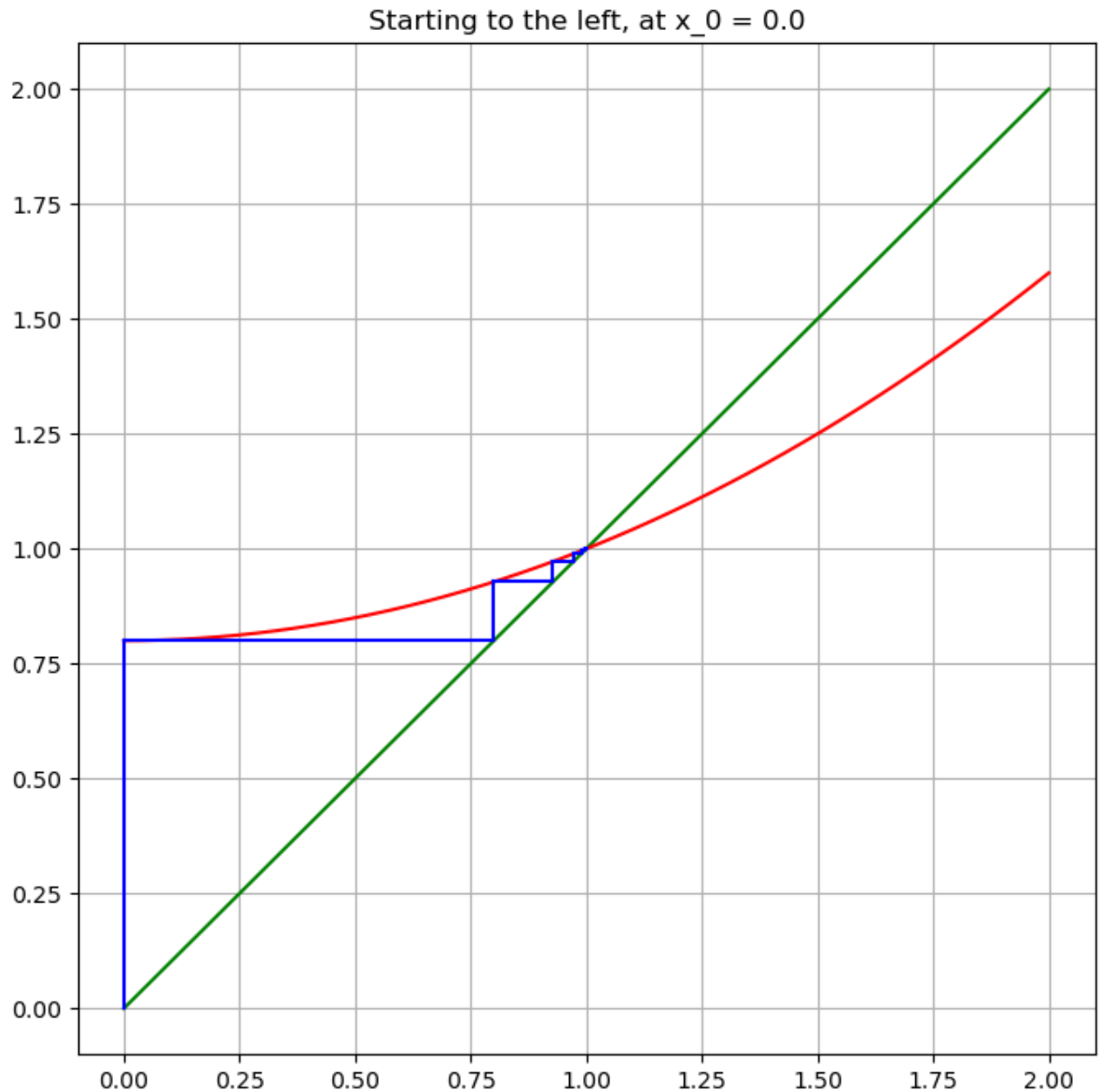




```
iterations = 10
# Start at left
a = 0.0
b = 2.0
x = np.linspace(a, b)
```

```
x_k = a
figure(figsize=(8,8))
title(f"Starting to the left, at x_0 = {a}")
grid(True)
plot(x, x, "g")
plot(x, g_2(x), "r")
for k in range(iterations):
    g_x_k = g_2(x_k)
    # Graph evaluation of g(x_k) from x_k:
    plot([x_k, x_k], [x_k, g_2(x_k)], "b")
    x_k_plus_1 = g_2(x_k)
    #Connect to the new x_k on the line y = x:
    plot([x_k, g_2(x_k)], [x_k_plus_1, x_k_plus_1], "b")
    # Update names: the old x_k+1 is the new x_k
    x_k = x_k_plus_1
    print(f"x_{k+1} = {x_k_plus_1}")
```

```
x_1 = 0.8
x_2 = 0.9280000000000002
x_3 = 0.9722368000000001
x_4 = 0.9890488790548482
x_5 = 0.9956435370319303
x_6 = 0.9982612105666906
x_7 = 0.9993050889044148
x_8 = 0.999722132142052
x_9 = 0.9998888682989302
x_10 = 0.999955549789623
```



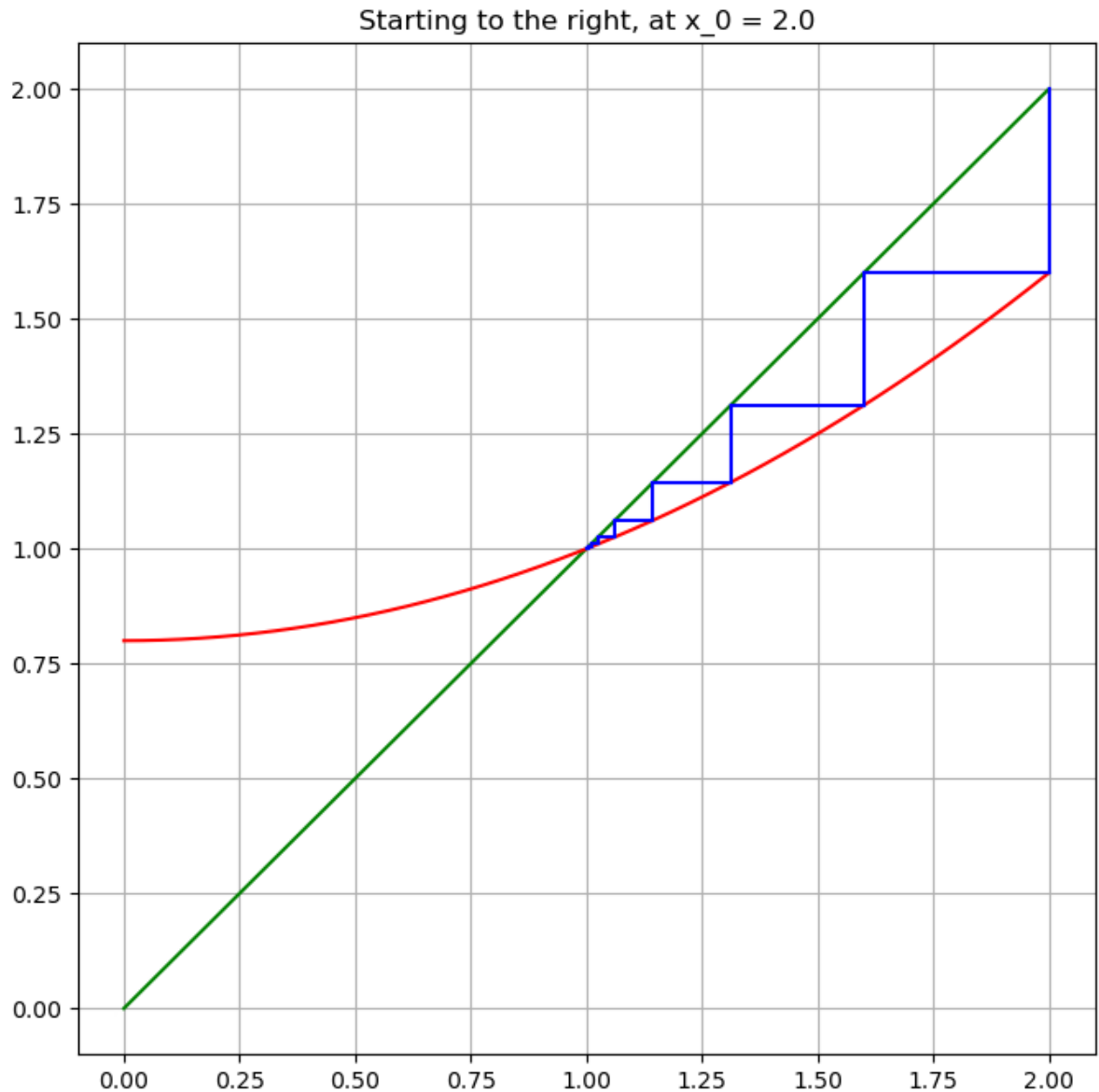
```
# Start at right
a = 0.0
b = 2.0
x = np.linspace(a, b)
x_k = b
figure(figsize=(8,8))
title(f"Starting to the right, at x_0 = {b}")
grid(True)
plot(x, x, "g")
plot(x, g_2(x), "r")
for k in range(iterations):
    g_x_k = g_2(x_k)
    # Graph evaluation of g(x_k) from x_k:
    plot([x_k, x_k], [x_k, g_2(x_k)], "b")
    x_k_plus_1 = g_2(x_k)
    #Connect to the new x_k on the line y = x:
```

(continues on next page)

(continued from previous page)

```
plot([x_k, g_2(x_k)], [x_k_plus_1, x_k_plus_1], "b")
# Update names: the old x_k+1 is the new x_k
x_k = x_k_plus_1
print(f"x_{k+1} = {x_k_plus_1}")
```

```
x_1 = 1.6
x_2 = 1.312
x_3 = 1.1442688
x_4 = 1.061870217330688
x_5 = 1.0255136716907844
x_6 = 1.010335658164943
x_7 = 1.0041556284319177
x_8 = 1.0016657052223
x_9 = 1.0006668370036975
x_10 = 1.000266823735797
```



2.2.3 Exercises

Exercise 1

The equation $x^3 - 2x + 1 = 0$ can be written as a fixed point equation in many ways, including

$$1. \ x = \frac{x^3 + 1}{2}$$

and

$$2. \ x = \sqrt[3]{2x - 1}$$

For each of these options:

(a) Verify that its fixed points do in fact solve the above cubic equation.

(b) Determine whether fixed point iteration will converge to the solution $r = 1$. (assuming a “good enough” initial approximation).

Note: computational experiments can be a useful start, but prove your answers mathematically!

2.3 Newton’s Method for Solving Equations

References:

- Sections 1.2 *Fixed-Point Iteration* and 1.4 *Newton’s Method* of [Sauer, 2022]
- Sections 2.2 *Fixed-Point Iteration* and 2.3 *Newton’s Method and Its Extensions* of [Burden et al., 2016]

2.3.1 Introduction

Newton’s method for solving equations has a number of advantages over the bisection method:

- It is usually faster (but not always, and it can even fail completely!)
- It can also compute complex roots, such as the non-real roots of polynomial equations.
- It can even be adapted to solving systems of non-linear equations; that topic will be visited later.

```
# We will often need resources from the modules numpy and pyplot:
import numpy as np
from numpy import sin, cos
from matplotlib.pyplot import figure, plot, title, legend, grid
```

2.3.2 Derivation as a contraction mapping with “very small contraction coefficient C ”

You might have previously seen Newton’s method derived using tangent line approximations. That derivation is presented below, but first we approach it another way: as a particularly nice contraction mapping.

To compute a root r of a differentiable function f , we design a contraction mapping for which the contraction constant C becomes arbitrarily small when we restrict to iterations in a sufficiently small interval around the root: $|x - r| \leq R$.

That is, the error ratio $|E_{k+1}|/|E_k|$ becomes ever smaller as the iterations get closer to the exact solution; the error is thus reducing ever faster than the above geometric rate C^k .

This effect is in turn achieved by getting $|g'(x)|$ arbitrarily small for $|x - r| \leq R$ with R small enough, and then using the above connection between $g'(x)$ and C . This can be achieved by ensuring that $g'(r) = 0$ at a root r of f — so long as the root r is *simple*: $f'(r) \neq 0$ (which is generically true, but not always).

To do so, seek g in the above form $g(x) = x - w(x)f(x)$, and choose $w(x)$ appropriately. At the root r ,

$$g'(r) = 1 - w'(r)f(r) - w(r)f'(r) = 1 - w(r)f'(r) \quad (\text{using } f(r) = 0,)$$

so we ensure $g'(r) = 0$ by requiring $w(r) = 1/f'(r)$ (hence the problem if $f'(r) = 0$).

We do not know r , but that does not matter! We can just choose $w(x) = 1/f'(x)$ for all x values. That gives

$$g(x) = x - f(x)/f'(x)$$

and thus the iteration formula

$$x_{k+1} = x_k - f(x_k)/f'(x_k)$$

(That is, $g(x) = x - f(x)/f'(x)$.)

You might recognize this as the formula for Newton's method.

To explore some examples of this, here is a Python function implementing Newton's method.

```
def newton_method(f, Df, x0, errorTolerance, maxIterations=20, demoMode=False):
    """Basic usage is:
    (rootApproximation, errorEstimate, iterations) = newton_method(f, Df, x0,
    ↪errorTolerance)
    There is an optional input parameter "demoMode" which controls whether to
    - print intermediate results (for "study" purposes), or to
    - work silently (for "production" use).
    The default is silence.
    """
    if demoMode: print("Solving by Newton's Method.")
    x = x0
    for k in range(maxIterations):
        fx = f(x)
        Dfx = Df(x)
        # Note: a careful, robust code would check for the possibility of division by_
    ↪zero here,
        # but for now I just want a simple presentation of the basic mathematical_
    ↪idea.
        dx = fx/Dfx
        x -= dx # Aside: this is shorthand for "x = x - dx"
        errorEstimate = abs(dx)
        if demoMode:
            print(f"At iteration {k+1} x = {x} with estimated error {errorEstimate:0.
    ↪3}, backward error {abs(f(x)):0.3}")
        if errorEstimate <= errorTolerance:
            iterations = k
            return (x, errorEstimate, iterations)
    # If we get here, it did not achieve the accuracy target:
    iterations = k
    return (x, errorEstimate, iterations)
```

Remark (A Python module)

From now on, all functions like this that implement numerical methods are also collected in the module file `numericalMethods.py`

Thus, you could omit the above `def` and instead import `newton_method` with

```
from numericalMethods import newton_method
```

Example

Let's start with our favorite equation, $x = \cos x$.

Remark (On Python style)

Since function names in Python (and most programming languages) must be alpha-numeric (with the underscore `_` as a "special guest letter"), I will avoid primes in notation for derivatives as much as possible: from now on, the derivative of

f is most often denoted as Df rather than f' .

```
def f_1(x): return x - cos(x)
def Df_1(x): return 1. + sin(x)
```

```
(root, errorEstimate, iterations) = newton_method(f_1, Df_1, x0=0., errorTolerance=1e-
↪8, demoMode=True)
print()
print(f"The root is approximately {root}")
print(f"The estimated absolute error is {errorEstimate:0.3}")
print(f"The backward error is {abs(f_1(root)):0.3}")
print(f"This required {iterations} iterations")
```

```
Solving by Newton's Method.
At iteration 1 x = 1.0 with estimated error 1.0, backward error 0.46
At iteration 2 x = 0.7503638678402439 with estimated error 0.25, backward error 0.
↪0.189
At iteration 3 x = 0.7391128909113617 with estimated error 0.0113, backward error
↪4.65e-05
At iteration 4 x = 0.7390851333852839 with estimated error 2.78e-05, backward
↪error 2.85e-10
At iteration 5 x = 0.7390851332151606 with estimated error 1.7e-10, backward error
↪1.11e-16

The root is approximately 0.7390851332151606
The estimated absolute error is 1.7e-10
The backward error is 1.11e-16
This required 4 iterations
```

Here we have introduced another way of talking about errors and accuracy, which is further discussed in the section on *Measures of Error and Order of Convergence*.

Definition (Backward Error)

- The **backward error** in $\tilde{x}$ as an approximation to a root of a function f is $f(\tilde{x})$.
- The **absolute backward error** is its absolute value, $|f(\tilde{x})|$. However sometimes the latter is simply called the backward error — as the above code does.

This has the advantage that we can actually compute it without knowing the exact solution!

The backward error also has a useful geometrical meaning: if the function f were changed by this much to a nearby function $\tilde{f}$ then $\tilde{x}$ could be an exact root of $\tilde{f}$. Hence, if we only know the values of f to within this backward error (for example due to rounding error in evaluating the function) then $\tilde{x}$ could well be an exact root, so there is no point in striving for greater accuracy in the approximate root.

We will see this in the next example.

Graphing Newton's method iterations as a fixed point iteration

Since this is a fixed point iteration with $g(x) = x - (x - \cos(x))/(1 + \sin(x))$, let us compare its graph to the ones seen in the section on *fixed point iteration*. Now g is neither increasing nor decreasing at the fixed point, so the graph has an unusual form.

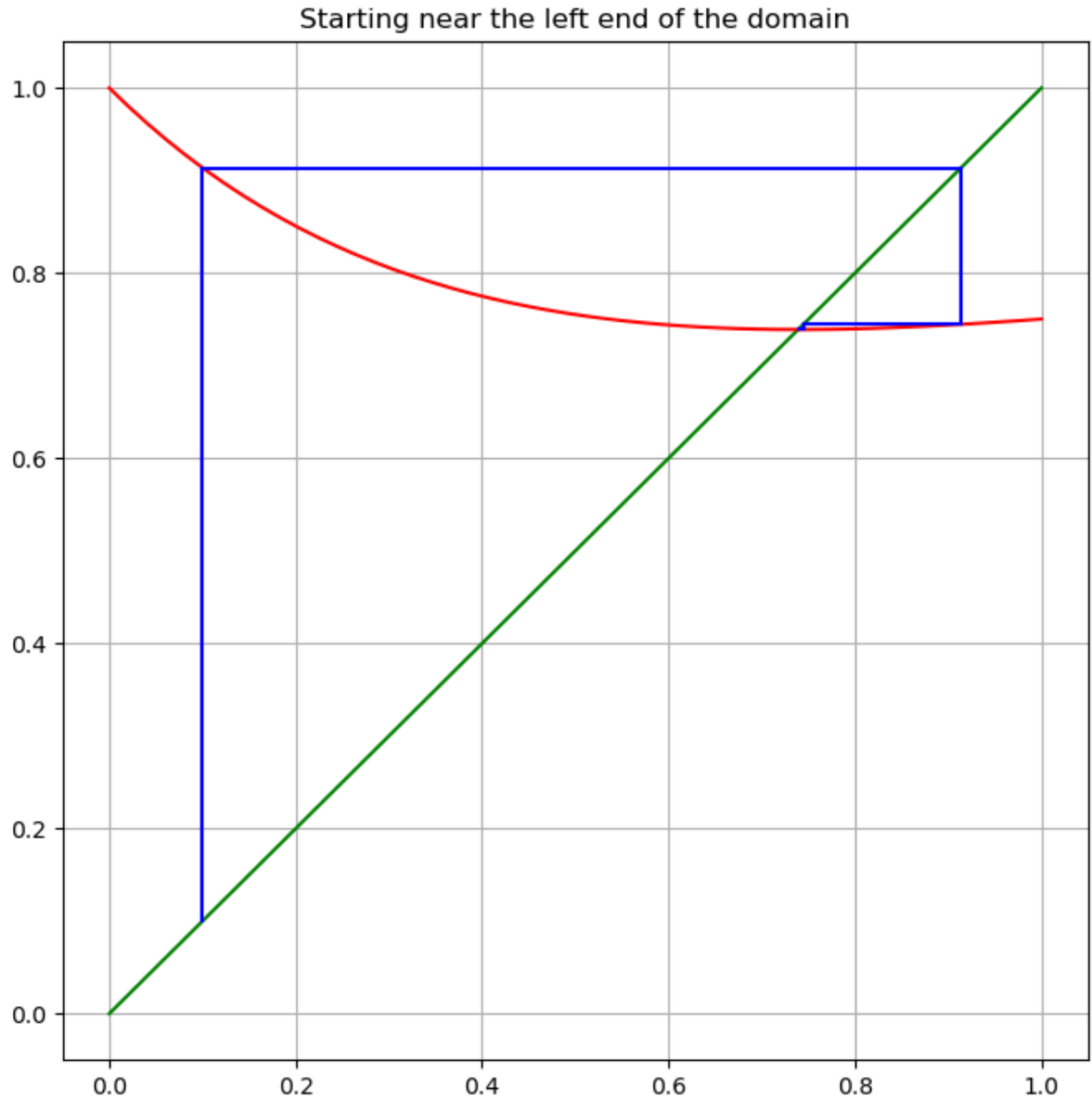
```
def g(x):
    return x - (x - cos(x))/(1 + sin(x))
a = 0
b = 1

# An array of x values for graphing
x = np.linspace(a, b)

iterations = 4 # Not so many are needed now!

# Start at left
description = 'Starting near the left end of the domain'
print(description)
x_k = 0.1
print(f"x_0 = {x_k}")
figure(figsize=(8,8))
title(description)
grid(True)
plot(x, x, 'g')
plot(x, g(x), 'r')
for k in range(iterations):
    g_x_k = g(x_k)
    # Graph evaluation of g(x_k) from x_k:
    plot([x_k, x_k], [x_k, g(x_k)], 'b')
    x_k_plus_1 = g(x_k)
    #Connect to the new x_k on the line y = x:
    plot([x_k, g(x_k)], [x_k_plus_1, x_k_plus_1], 'b')
    # Update names: the old x_k+1 is the new x_k
    x_k = x_k_plus_1
    print(f"x_{k+1} = {x_k}")
```

```
Starting near the left end of the domain
x_0 = 0.1
x_1 = 0.9137633861014282
x_2 = 0.7446642419816996
x_3 = 0.7390919659607759
x_4 = 0.7390851332254692
```



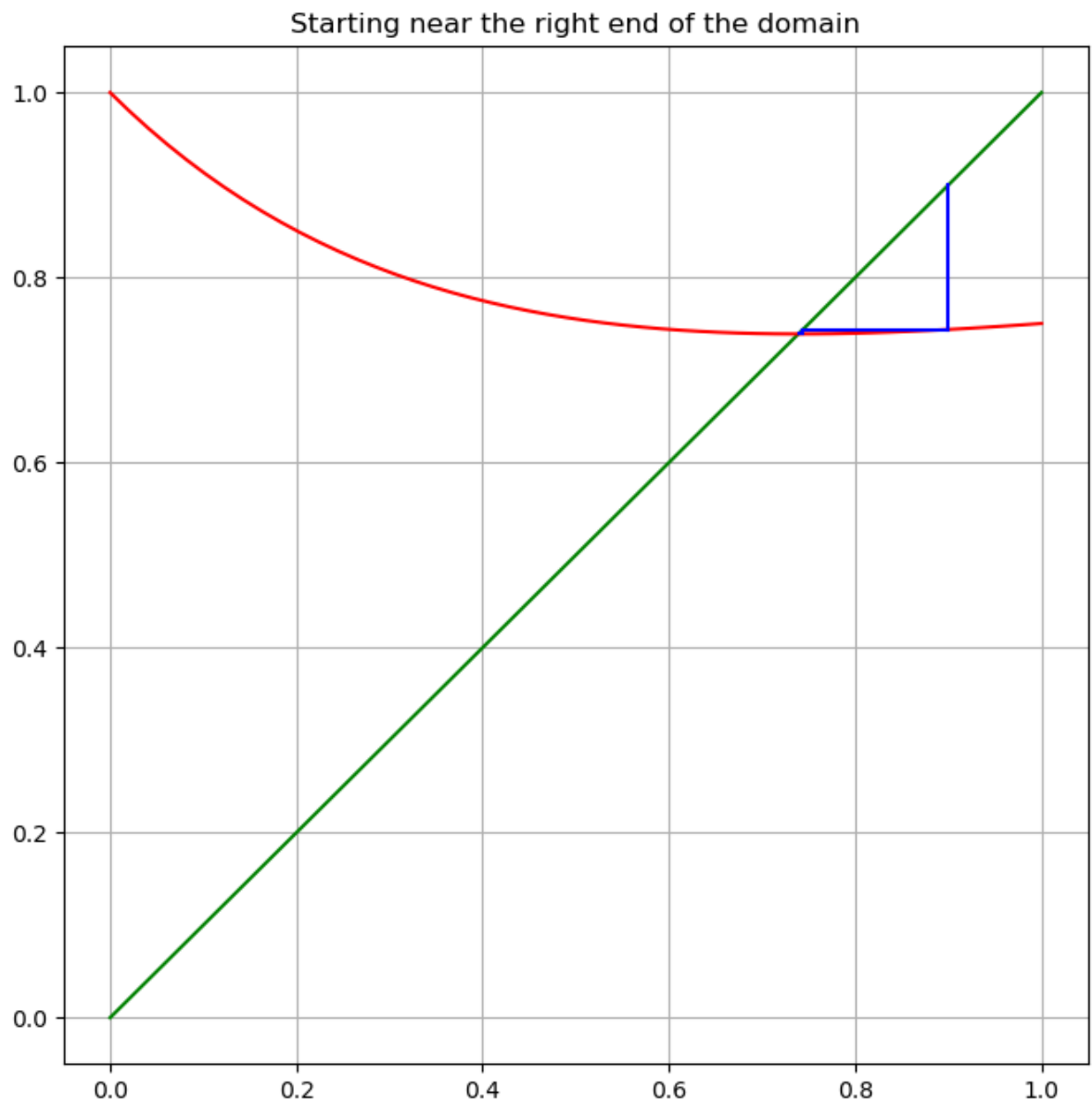
```
# Start at right
description = 'Starting near the right end of the domain'
print(description)
x_k = 0.9
print(f"x_0 = {x_k}")
figure(figsize=(8,8))
title(description)
grid(True)
plot(x, x, 'g')
plot(x, g(x), 'r')
for k in range(iterations):
    g_x_k = g(x_k)
    # Graph evaluation of g(x_k) from x_k:
    plot([x_k, x_k], [x_k, g(x_k)], 'b')
    x_k_plus_1 = g(x_k)
```

(continues on next page)

(continued from previous page)

```
#Connect to the new  $x_k$  on the line  $y = x$ :  
plot([ $x_k$ ,  $g(x_k)$ ], [ $x_{k\_plus\_1}$ ,  $x_{k\_plus\_1}$ ], 'b')  
# Update names: the old  $x_{k+1}$  is the new  $x_k$   
 $x_k = x_{k\_plus\_1}$   
print(f" $x_{k+1} = {x_k}$ ")
```

```
Starting near the right end of the domain  
 $x_0 = 0.9$   
 $x_1 = 0.7438928778417369$   
 $x_2 = 0.7390902113045812$   
 $x_3 = 0.7390851332208545$   
 $x_4 = 0.7390851332151607$ 
```



In fact, wherever you start, all iterations take you to the right of the root, and then approach the fixed point monotonically

— and very fast. We will see an explanation for this in *The Convergence Rate of Newton's Method*.

Example (Pushing to the limits of standard 64-bit computer arithmetic)

Next, demand more accuracy; this time silently. As we will see in a later section, 10^{-16} is about the limit of the precision of standard (IEEE64) computer arithmetic with 64-bit numbers.

So let's try to compute the root as accurately as we can within these limits:

```
(root, errorEstimate, iterations) = newton_method(f_1, Df_1, x0=0, errorTolerance=1e-
↪16)
print()
print(f"The root is approximately {root}")
print(f"The estimated absolute error is {errorEstimate}")
print(f"The backward error is {abs(f_1(root)):0.4}")
print(f"This required {iterations} iterations")
```

```
The root is approximately 0.7390851332151607
The estimated absolute error is 6.633694101535508e-17
The backward error is 0.0
This required 5 iterations
```

Observations:

- It only took one more iteration to meet the demand for twice as many decimal places of accuracy.
- The result is “exact” as far as the computer arithmetic can tell, as shown by the zero backward error: we have indeed reached the accuracy limits of computer arithmetic.

2.3.3 Newton's method works with complex numbers too

We will work almost entirely with real values and vectors in $\mathbb{R}^n$, but actually, everything above also works for complex numbers. In particular, Newton's method works for finding roots of functions $f : \mathbb{C} \rightarrow \mathbb{C}$; for example when seeking all roots of a polynomial.

Remark (Notation for complex number in Python)

Python uses `j` for the square root of -1 (as is also sometimes done in engineering) rather than `i`.

In general, the complex number $a + bi$ is expressed as `a+bj` (note: `j` at the end, and no spaces). As you might expect, imaginary numbers can be written without the `a`, as `bj`.

However, the coefficient `b` is always needed, even when $b = 1$: the square roots of -1 are `1j` and `-1j`, not `j` and `-j`, and the latter pair still refer to a variable `j` and its negation.

```
z = 3+4j
print(z)
print(abs(z))
```

```
(3+4j)
5.0
```

```
print(1j)
```

```
1j
```

```
print(-1j)
```

```
(-0-1j)
```

but:

```
print(j)
```

```
-----  
NameError                                Traceback (most recent call last)  
Input In [12], in <cell line: 1>()  
----> 1 print(j)  
  
NameError: name 'j' is not defined
```

Giving `j` a value does not interfere:

```
j = 100
```

```
print(1j)
```

```
1j
```

```
print(j)
```

```
100
```

Example (All roots of a cubic)

As an example, let us seek all three cube roots of 8, by solving $x^3 - 8 = 0$ and trying different initial values x_0 .

```
def f_2(x): return x**3 - 8  
def Df_2(x): return 3*x**2
```

First, $x_0 = 1$

```
(root1, errorEstimate1, iterations1) = newton_method(f_2, Df_2, x0=1.,  
    errorTolerance=1e-8, demoMode=True)  
print()  
print(f"The first root is approximately {root1}")  
print(f"The estimated absolute error is {errorEstimate1}")  
print(f"The backward error is {abs(f_2(root1)):0.4}")  
print(f"This required {iterations1} iterations")
```



```

Solving by Newton's Method.
At iteration 1 x = 3.3333333333333335 with estimated error 2.33, backward error 29.
↪0
At iteration 2 x = 2.4622222222222222 with estimated error 0.871, backward error 6.
↪93
At iteration 3 x = 2.081341247671579 with estimated error 0.381, backward error 1.
↪02
At iteration 4 x = 2.003137499141287 with estimated error 0.0782, backward error 0.
↪0377
At iteration 5 x = 2.000004911675504 with estimated error 0.00313, backward error
↪5.89e-05
At iteration 6 x = 2.0000000000120624 with estimated error 4.91e-06, backward
↪error 1.45e-10
At iteration 7 x = 2.0 with estimated error 1.21e-11, backward error 0.0

The first root is approximately 2.0
The estimated absolute error is 1.2062351117801901e-11
The backward error is 0.0
This required 6 iterations

```

Next, start at $x_0 = i$ (a.k.a. $x_0 = j$):

```

(root2, errorEstimate2, iterations2) = newton_method(f_2, Df_2, x0=1j,
↪errorTolerance=1e-8, demoMode=True)
print()
print(f"The second root is approximately {root2}")
print(f"The estimated absolute error is {errorEstimate2:0.3}")
print(f"The backward error is {abs(f_2(root2)):0.3}")
print(f"This required {iterations2} iterations")

```

```

Solving by Newton's Method.
At iteration 1 x = (-2.6666666666666665+0.6666666666666667j) with estimated error
↪2.69, backward error 27.2
At iteration 2 x = (-1.4663590926566705+0.6105344098423685j) with estimated error
↪1.2, backward error 10.2
At iteration 3 x = (-0.23293230984230862+1.157138282313884j) with estimated error
↪1.35, backward error 7.21
At iteration 4 x = (-1.920232195343855+1.5120026439880303j) with estimated error 1.
↪72, backward error 13.4
At iteration 5 x = (-1.1754417924325353+1.4419675366055333j) with estimated error
↪0.748, backward error 3.76
At iteration 6 x = (-0.9389355523964146+1.7160019741718067j) with estimated error
↪0.362, backward error 0.741
At iteration 7 x = (-1.0017352527552088+1.7309534907089796j) with estimated error
↪0.0646, backward error 0.0246
At iteration 8 x = (-0.9999988050398477+1.7320490713246675j) with estimated error
↪0.00205, backward error 2.53e-05
At iteration 9 x = (-1.0000000000014002+1.7320508075706016j) with estimated error
↪2.11e-06, backward error 2.67e-11
At iteration 10 x = (-1+1.7320508075688774j) with estimated error 2.22e-12,
↪backward error 1.99e-15

The second root is approximately (-1+1.7320508075688774j)
The estimated absolute error is 2.22e-12
The backward error is 1.99e-15
This required 9 iterations

```

This root is in fact $-1 + i\sqrt{3}$.

Finally, $x_0 = 1 - i$

```
(root3, errorEstimate3, iterations3) = newton_method(f_2, Df_2, x0=1-1j,
    errorTolerance=1e-8, demoMode=False)
print()
print(f"The third root is approximately {root3}")
print(f"The estimated absolute error is {errorEstimate3}")
print(f"The backward error is {abs(f_2(root3)):0.4}")
print(f"This required {iterations3} iterations")
```

```
The third root is approximately (-1-1.7320508075688772j)
The estimated absolute error is 3.629748304956849e-15
The backward error is 1.986e-15
This required 9 iterations
```

This root is in fact $-1 - i\sqrt{3}$.

2.3.4 Newton's method derived via tangent line approximations: linearization

The more traditional derivation of Newton's method is based on the very widely useful idea of *linearization*; using the fact that a differentiable function can be approximated over a small part of its domain by a straight line — its tangent line — and it is easy to compute the root of this linear function.

So start with a first approximation x_0 to a solution r of $f(x) = 0$.

Step 1: Linearize at x_0 .

The tangent line to the graph of this function with center x_0 , also known as the *linearization of f at x_0* , is

$$L_0(x) = f(x_0) + f'(x_0)(x - x_0).$$

(Note that $L_0(x_0) = f(x_0)$ and $L'_0(x_0) = f'(x_0)$.)

Step 2: Find the zero of this linearization

Hopefully, the two functions f and L_0 are close, so that the root of L_0 is close to a root of f ; close enough to be a better approximation of the root r than x_0 is.

Give the name x_1 to this root of L_0 : it solves $L_0(x_1) = f(x_0) + f'(x_0)(x_1 - x_0) = 0$, so

$$x_1 = x_0 - f(x_0)/f'(x_0)$$

Step 3: Iterate

We can then use this new value x_1 as the center for a new linearization $L_1(x) = f(x_1) + f'(x_1)(x - x_1)$, and repeat to get a hopefully even better approximate root,

$$x_2 = x_1 - f(x_1)/f'(x_1)$$

And so on: at each step, we get from approximation x_k to a new one x_{k+1} with

$$x_{k+1} = x_k - f(x_k)/f'(x_k)$$

And indeed this is the same formula seen above for Newton's method.

Illustration: a few steps of Newton's method for $x - \cos(x) = 0$.

This approach to Newton's method via linearization and tangent lines suggests another graphical presentation; again we use the example of $f(x) = x - \cos(x)$. This has $Df(x) = 1 + \sin(x)$, so the linearization at center a is

$$L(x) = (a - \cos(a)) + (1 + \sin(a))(x - a)$$

For Newton's method starting at $x_0 = 0$, this gives

$$L_0(x) = -1 + x$$

and its root — the next iterate in Newton's method — is $x_1 = 1$

Then the linearization at center x_1 is

$$L_1(x) = (1 - \cos(1) + (1 + \sin(1))(x - 1), \approx 0.4596 + 1.8415(x - 1)$$

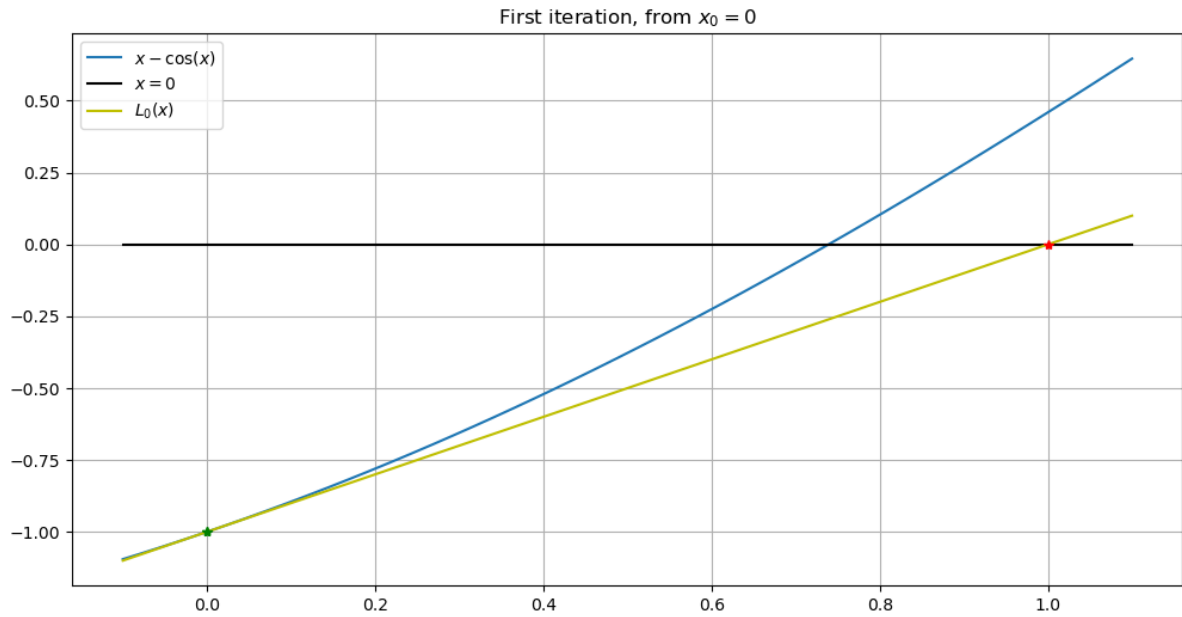
giving $x_2 \approx 1 - 0.4596/1.8415 \approx 0.7504$.

Let's graph a few steps.

```
def L_0(x): return -1 + x
```

```
figure(figsize=(12,6))
title('First iteration, from $x_0 = 0$')
left = -0.1
right = 1.1
x = np.linspace(left, right)
plot(x, f_1(x), label='$x - \cos(x)$')
plot([left, right], [0, 0], 'k', label="$x=0$") # The x-axis, in black
x_0 = 0
plot([x_0], [f_1(x_0)], 'g*')
plot(x, L_0(x), 'y', label='$L_0(x)$')
plot([x_0], [f_1(x_0)], 'g*')
x_1 = x_0 - f_1(x_0)/Df_1(x_0)
print(f'{x_1=}')
plot([x_1], [0], 'r*')
legend()
grid(True);
```

```
x_1=1.0
```

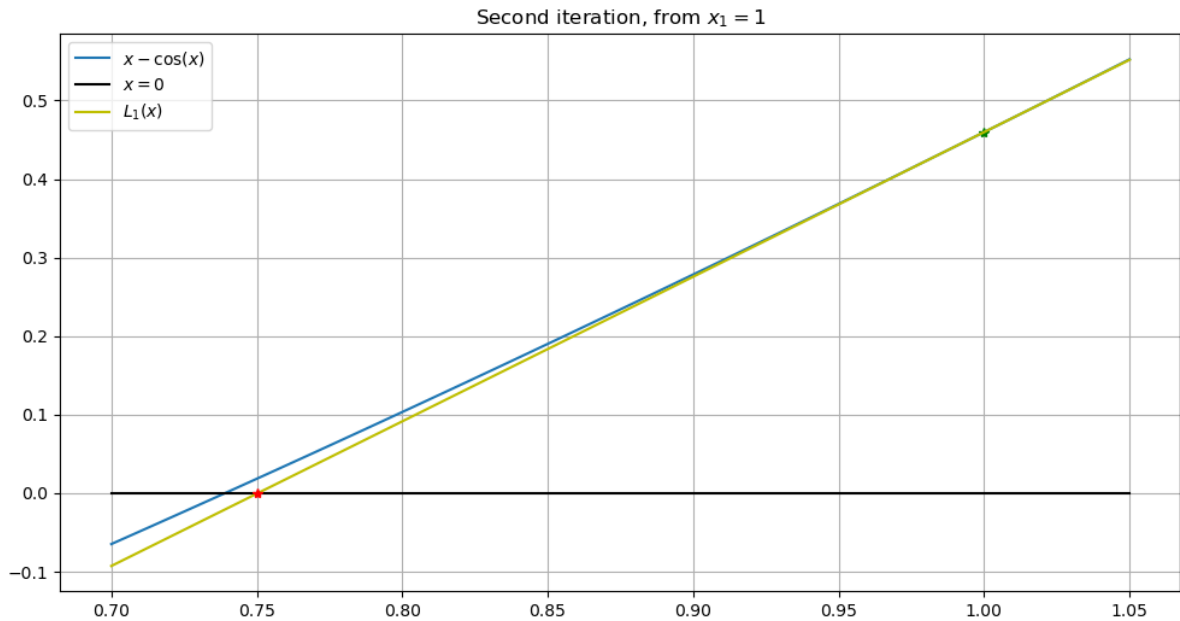


```
def L_1(x): return (x_1 - cos(x_1)) + (1 + sin(x_1))*(x - x_1)

figure(figsize=(12,6))
title('Second iteration, from  $x_1 = 1$ ')
# Shrink the domain
left = 0.7
right = 1.05
x = np.linspace(left, right)

plot(x, f_1(x), label='$x - \cos(x)$')
plot([left, right], [0, 0], 'k', label="$x=0$") # The x-axis, in black
plot([x_1], [f_1(x_1)], 'g*')
plot(x, L_1(x), 'y', label='$L_1(x)$')
x_2 = x_1 - f_1(x_1)/Df_1(x_1)
print(f'{x_2=}')
plot([x_2], [0], 'r*')
legend()
grid(True);
```

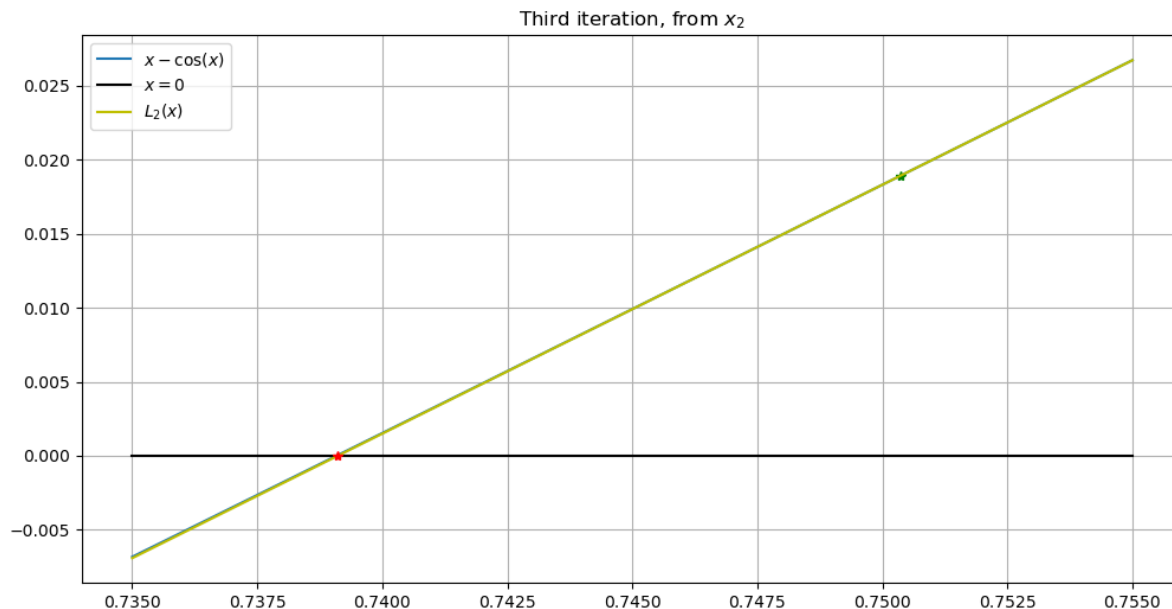
```
x_2=0.7503638678402439
```



```
def L_2(x): return (x_2 - cos(x_2)) + (1 + sin(x_2))*(x - x_2)

figure(figsize=(12,6))
title('Third iteration, from $x_2$')
# Shrink the domain some more
left = 0.735
right = 0.755
x = np.linspace(left, right)

plot(x, f_1(x), label='$x - \cos(x)$')
plot([left, right], [0, 0], 'k', label="$x=0$") # The x-axis, in black
plot([x_2], [f_1(x_2)], 'g*')
plot(x, L_2(x), 'y', label='$L_2(x)$')
x_3 = x_2 - f_1(x_2)/Df_1(x_2)
plot([x_3], [0], 'r*')
legend()
grid(True);
```



2.3.5 How accurate and fast is this?

For the bisection method, we have seen in *Root Finding by Interval Halving* a fairly simple way to get an upper limit on the absolute error in the approximations.

For absolute guarantees of accuracy, things do not go quite as well for Newton's method, but we can at least get a very "probable" *estimate* of how large the error can be. This requires some calculus, and more specifically Taylor's theorem, reviewed in the section on *Taylor's Theorem*.

So we will return to the question of both the speed and accuracy of Newton's method in *The Convergence Rate of Newton's Method*.

On the other hand, the example graphs above illustrate that the successive linearizations become ever more accurate as approximations of the function f itself, so that the approximation x_3 looks "perfect" on the graph — the speed of Newton's method looks far better than for bisection. This will also be explained in the section on *The Convergence Rate of Newton's Method*.

2.3.6 Exercises

Exercise 1

a) Show that Newton's method applied to

$$f(x) = x^k - a$$

leads to fixed point iteration with function

$$g(x) = \frac{(k-1)x + \frac{a}{x^{k-1}}}{k}.$$

b) Then verify mathematically that the iteration $x_{k+1} = g(x_k)$ has super-linear convergence.

Exercise 2

a) Create a Python function for Newton's method, with usage

```
(root, errorEstimate, iterations, functionEvaluations) = newton_method(f, Df, x_0,
↪errorTolerance, maxIterations)
```

(The last input parameter `maxIterations` could be optional, with a default like `maxIterations=100`.)

b) based on your function `bisection2` create a third (and final!) version with usage

```
(root, errorBound, iterations, functionEvaluations) = bisection(f, a, b,
↪errorTolerance, maxIterations)
```

c) Use both of these to solve the equation

$$f_1(x) = 10 - 2x + \sin(x) = 0$$

i) with [estimated] absolute error of no more than 10^{-6} , and then

ii) with [estimated] absolute error of no more than 10^{-15} .

Note in particular how many iterations and how many function evaluations are needed.

Graph the function, which will help to find a good starting interval $[a, b]$ and initial approximation x_0 .

d) Repeat, this time finding the unique real root of

$$f_2(x) = x^3 - 3.3x^2 + 3.63x - 1.331 = 0$$

Again graph the function, to find a good starting interval $[a, b]$ and initial approximation x_0 .

e) This second case will behave differently than for f_1 in part (c): describe the difference. (We will discuss the reasons in class.)

2.4 Taylor's Theorem and the Accuracy of Linearization

References:

- Theorem 0.8 in Section 0.5 *Review of Calculus* in [Sauer, 2022].
- Section 1.1 *Review of Calculus* in [Burden et al., 2016], from Theorem 1.14 onward.

2.4.1 Taylor's theorem

Taylor's theorem is most often stated in the form

Theorem (Taylor's Theorem, with center a)

When all the relevant derivatives exist,

$$\begin{aligned} f(x) = & f(a) + f'(a)(x-a) + \frac{1}{2}f''(a)(x-a)^2 \dots \frac{f^{(k)}(a)}{k!}(x-a)^k + \dots \\ & + \frac{f^{(n)}(a)}{n!}(x-a)^n + R_n(x) \end{aligned} \quad (2.1)$$

The polynomial part of this,

$$T_n(x) = f(a) + f'(a)(x-a) + \dots + \frac{f^{(k)}(a)}{k!}(x-a)^k + \dots + \frac{f^{(n)}(a)}{n!}(x-a)^n \quad (2.2)$$

is the **Taylor polynomial of degree n with center a for function f** , and the *remainder* is

$$R_n(x) = \frac{f^{(n+1)}(c_x)}{(n+1)!}(x-a)^{n+1} \quad (2.3)$$

with the value c_x lying between a and x , and so depending on x .

This gives information about the absolute error in the polynomial $T_n(x)$ as an approximation of $f(x)$:

$$|f(x) - T_n(x)| \leq \frac{M_{n+1}}{(n+1)!}|x-a|^{n+1}$$

where M_{n+1} is the maximum absolute value of $f^{(n+1)}$ over the relevant interval between a and x .

Of course we typically do not know much about that constant M_{n+1} , so often the most important thing is the power law rate $|x-a|^{n+1}$ at which the error reduces as x approaches a .

Taylor polynomials are therefore most useful when the quantity $h := x - a$ is small, and we will most often use them in situations where the limit as $h \rightarrow 0$ is relevant. It is convenient to change the notation a bit, treating h as the variable:

Theorem (Taylor's Theorem, h form)

When all the relevant derivatives exist,

$$T_n(h) = f(a) + f'(a)h + \dots + \frac{f^{(k)}(a)}{k!}h^k + \dots + \frac{f^{(n)}(a)}{n!}h^n \quad (2.4)$$

with this polynomial in h approximating $f(a+h)$ in that

$$f(a+h) - T_n(h) = R_n(h) = \frac{f^{(n+1)}(c_h)}{(n+1)!}h^{n+1} \text{ with } |c_h - a| < |h|. \quad (2.5)$$

(Note: h may be negative!)

This gives a bound on the absolute error

$$|f(a+h) - T_n(h)| \leq \frac{M_{n+1}}{(n+1)!}|h|^{n+1} \quad (2.6)$$

with

$$M_{n+1} := \max_{|x-a| \leq |h|} |f^{(n+1)}(x)|$$

2.4.2 Error formula for linearization

A very common use of Taylor's Theorem is the rather simple case $n = 1$; *linearization*, to approximate a twice differentiable function by a linear one. (This will be even more so when we come to system of equations, since the only such systems that we can systematically solve exactly are linear systems.)

Taylor's Theorem for the linearization $L(x) = f(a) + f'(a)(x-a)$ of f at a then says that

$$f(x) - L(x) = \frac{f''(c_x)}{2}h^2, \quad |c_x - a| < |x - a| \quad (2.7)$$

or in terms of h ,

$$f(a+h) = f(a) + f'(a)h + \frac{f''(c_h)}{2}h^2, \quad |c_h - a| < |h| \quad (2.8)$$

Thus there is an error bound

$$|f(a+h) - (f(a) + f'(a)h)| \leq \frac{M_2}{2}h^2, \quad \text{where } M_2 = \max_{|x-a|<|h|} |f''(x)| \quad (2.9)$$

Of course sometimes it is enough to use the maximum over the whole domain, $M_2 = \max |f''(x)|$.

2.5 Measures of Error and Order of Convergence

References:

- Section 1.3.1 *Forward and backward error* of [Sauer, 2022], on *measures of error*;
- Section 2.4 *Error Analysis for Iterative Methods* of [Burden et al., 2016], on *order of convergence*.

These notes cover a number of small topics:

- Measures of error: absolute, relative, forward, backward, etc.
- Measuring the rate of convergence of a sequence of approximations.
- Big-O and little-o notation for describing how small a quantity (usually an error) is.

2.5.1 Error measures

Several of these have been mentioned before, but they are worth gathering here.

Consider a quantity $\tilde{x}$ considered as an approximation of an exact value x . (This can be a number or a vector.)

Definition 1.5.1 (Error)

The **error** in $\tilde{x}$ is $\tilde{x} - x$ (or $x - \tilde{x}$; different sources use both versions and either is fine so long as you are **consistent**.)

Definition 1.5.2 (Absolute Error)

The **absolute error** is the absolute value of the error: $|\tilde{x} - x|$. For vector quantities this means the norm $\|\tilde{x} - x\|$, and it can be any norm, so long as we again choose one and use it consistently. Two favorites are the *Euclidean norm* $\|x\| = \sqrt{\sum |x_i|^2}$, denoted $\|x\|_2$, and the *maximum norm* (also mysteriously known as the *infinity norm*):

$$\|x\|_{\max} = \|x\|_{\infty} = \max_i |x_i|.$$

For real-valued quantities, the absolute error is related to the number of **correct decimal places**: p decimal places of accuracy corresponds roughly to absolute error no more than 0.5×10^{-p} .

Definition 1.5.3 (Relative Error)

The **relative error** is the ratio of the absolute error to the size of the exact quantity: $\frac{\|\tilde{x} - x\|}{\|x\|}$ (again possibly with vector norms).

This is often more relevant than absolute error for inherently positive quantities, but is obviously unwise where $x = 0$ is a possibility. For real-valued quantities, this is related to the number of **significant digits**: accuracy to p significant digits corresponds roughly to relative error no more than 0.5×10^{-p} .

When working with computer arithmetic, p significant **bits** corresponds to relative error no more than $2^{-(p+1)}$.

Backward error (and forward error)

An obvious problem is that we usually do not know the exact solution x , so cannot evaluate any of these; instead we typically seek **upper bounds** on the absolute or relative error. Thus, when talking of approximate solutions to an equation $f(x) = 0$ the concept of *Definition 1.3.1* **backward error** introduced in the section on *Newton's Method for Solving Equations* can be very useful, for example as a step in getting bounds on the size of the error; to recap

Definition 1.5.4 (Backward Error)

The **backward error** in $\tilde{x}$ as an approximate solution to the equation $f(x) = 0$ is $f(\tilde{x})$; the amount by which function f would have to be changed in order for $\tilde{x}$ to be an exact root.

For the case of solving simultaneous linear equations in matrix-vector form $Ax = b$, this is $b - A\tilde{x}$, also known as the **residual**.

Definition 1.5.5 (Absolute Backward Error)

The absolute backward error is — as you might have guessed — the absolute value of the backward error: $|f(\tilde{x})|$. This is sometimes also called simply the backward error. (The usual disclaimer about vector quantities applies.)

For the case of solving simultaneous linear equations in matrix-vector form $Ax = b$, this is $\|b - A\tilde{x}\|$, also known as the **residual norm**.

Remark 1.5.1

- One obvious advantage of the backward error concept is that you can actually evaluate it without knowing the exact solution x .
 - Also, one significance of backward error is that if the values of $f(x)$ are only known to be accurate within an absolute error of E then any approximation with absolute backward error less than E could in fact be exact, so there is no point in seeking greater accuracy.
 - The names *forward error* and *absolute forward error* are sometimes used as synonyms for *error* etc. as defined above, when they need to be distinguished from backward errors.
-

2.5.2 Order of convergence of a sequence of approximations

Definition 1.5.6

We have seen that for the sequence of approximations x_k to a quantity x given by the fixed point iteration $x_{k+1} = g(x_k)$, the absolute errors $E_k := |x_k - x|$ typically have

$$\frac{E_{k+1}}{E_k} \rightarrow C = |g'(x)|.$$

so that eventually the errors diminish in a roughly geometric fashion: $E_k \approx KC^k$. This is called **linear convergence**.

Aside: Why “linear” rather than “geometric”? Because there is an approximately linear relationship between consecutive error values,

$$E_{n+1} \approx CE_n.$$

This is a very common behavior for iterative numerical methods, but we will also see that a few methods do even better; for example, when Newton’s method converges to a simple root r of f (one with $f'(r) \neq 0$)

$$E_{k+1} \approx CE_k^2$$

This is called **quadratic convergence**. More generally:

Definition 1.5.7 (convergence of order p)

This is when

$$E_{k+1} \approx CE_k^p, \text{ or more precisely, } \lim_{k \rightarrow \infty} \frac{E_{k+1}}{E_k^p} \text{ is finite.}$$

We have already observed experimentally the intermediate result that “ $C = 0$ ” for Newton’s method in this case; that is,

$$\frac{E_{k+1}}{E_k} \rightarrow 0. \tag{2.10}$$

Definition 1.5.8 (super-linear convergence)

When the successive errors behave as in Equation (2.10) the convergence is **super-linear**. This includes any situation with order of convergence $p > 1$.

For most practical purposes, if you have established super-linear convergence, you can be happy, and not worry much about refinements like the particular order p .

2.5.3 Big-O and little-o notation

Consider the error formula for approximation of a function f with the Taylor polynomial of degree n , center a :

$$|f(a+h) - T_n(h)| \leq \frac{M_{n+1}}{(n+1)!} |h|^{n+1} \text{ where } M_{n+1} = \max |f^{(n+1)}(x)|.$$

Since the coefficient of h^{n+1} is typically not known in practice, it is wise to focus on the power law part, and for this the “big-O” and little-o” notation is convenient.

If a function $E(h)$ goes to zero at least as fast as h^p , we say that it is *of order* h^p , written $O(h^p)$.

More precisely, $E(h)$ is no bigger than a multiple of h^p for h small enough; that is, there is a constant C such that for some positive number δ

$$\frac{|E(h)|}{|h|^p} \leq C \text{ for } |h| < \delta.$$

Another way to say this is in terms of the *lim-sup*, if you have seen that jargon:

$$\limsup_{h \rightarrow 0} \frac{|E(h)|}{|h|^p} \text{ is finite.}$$

This can be used to rephrase the above Taylor's theorem error bound as

$$f(x) - T_n(x) = O(|x - a|^{n+1})$$

or

$$f(a + h) - T_n(h) = O(h^{n+1}),$$

and for the case of the linearization,

$$f(a + h) - L(x) = f(a + h) - (f(a) + f'(a)h) = O(h^2).$$

Little-o notation, for “negligibly small terms”

Sometimes it is enough to say that some error term is small enough to be neglected, at least when h is close enough to zero. For example, with a Taylor series we might be able to neglect the powers of $x - a$ or of h higher than p .

We will thus say that a quantity $E(h)$ is *small of order* h^p , written $o(h^p)$ when

$$\lim_{h \rightarrow 0} \frac{|E(h)|}{|h|^p} = 0.$$

Note the addition of the word **small** compared to the above description of the big-O case!

With this, the Taylor's theorem error bound can be stated as

$$f(x) - T_n(x) = o(|x - a|^n),$$

or

$$f(a + h) - T_n(h) = o(h^n),$$

and for the case of the linearization,

$$f(a + h) - L(x) = f(a + h) - (f(a) + f'(a)h) = o(h).$$

2.6 The Convergence Rate of Newton's Method

References:

- Section 1.4.1 *Quadratic Convergence of Newton's Method* in [Sauer, 2022].
- Theorem 2.9 in Section 2.4 *Error Analysis of Iterative Methods* in [Burden *et al.*, 2016], but done quite differently.

Jumping to the punch line, we will see that when the iterates x_k given by Newton's method converge to a *simple root* r (that is, a *solution of* $f(r) = 0$ with $f'(r) \neq 0$) then the errors $E_k = x_k - r$ satisfy

$$E_{k+1} = O(E_k^2) \text{ and therefore } E_{k+1} = o(E_k)$$

In words, the error at each iteration is *of the order of* the square of the previous error, and so is *small of order* the previous error.

(Yes, it this a slight abuse of the notation as defined above, but all will become clear and rigorous soon.)

The first key step is getting a recursive relationship between consecutive errors E_k and E_{k+1} from the recursion formula for Newton's method,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Start by subtracting r :

$$E_{k+1} = x_{k+1} - r = x_k - \frac{f(x_k)}{f'(x_k)} - r = E_k - \frac{f(x_k)}{f'(x_k)}$$

The other key step is to show that the two terms at right are very close, using the linearization of f at x_k with the error E_k as the small term h , and noting that $r = x_k - E_k$:

$$0 = f(r) = f(x_k - E_k) = f(x_k) - f'(x_k)E_k + O(E_k^2)$$

Solve for $f(x_k)$ to insert into the numerator above: $f(x_k) = f'(x_k)E_k + O(E_k^2)$. (There is no need for a minus sign on that last term; big-O terms can be of either sign, and this new one is a different but still small enough quantity!)

Inserting above,

$$E_{k+1} = E_k - \frac{f'(x_k)E_k + O(E_k^2)}{f'(x_k)} = E_k - E_k + \frac{O(E_k^2)}{f'(x_k)} = \frac{O(E_k^2)}{f'(x_k)} \rightarrow \frac{O(E_k^2)}{f'(r)} = O(E_k^2)$$

As $k \rightarrow \infty$, $f'(E_k) \rightarrow f'(r) \neq 0$, so the term at right is still no larger than a multiple of E_k^2 : it is $O(E_k^2)$, as claimed.

If you wish to verify this more carefully, note that

- this $O(E_k^2)$ term is no bigger than $\frac{M}{2} E_k^2$ where M is an upper bound on $|f''(x)|$, and
- once E_k is small enough, so that x_k is close enough to r , $|f'(x_k)| \geq |f'(r)|/2$.

Thus the term $\frac{O(E_k^2)}{f'(x_k)}$ has magnitude no bigger than $\frac{M/2}{|f'(r)|/2} E_k^2 = \frac{M}{|f'(r)|} E_k^2$, which meets the definition of being of order E_k^2 .

A more careful calculation actually shows that

$$\lim_{k \rightarrow \infty} \frac{|E_{k+1}|}{E_k^2} = \left| \frac{f''(r)}{2f'(r)} \right|,$$

which is the way that this result is often stated in texts. For either form, it then easily follows that

$$\lim_{k \rightarrow \infty} \frac{|E_{k+1}|}{|E_k|} = 0,$$

giving the *super-linear convergence* already seen using the Contraction Mapping Theorem, now restated as $E_{k+1} = o(E_k)$.

2.6.1 A Practical error estimate for fast-converging iterative methods

One problem for Newton's Method (and many other numerical methods we will see) is that there is not a simple way to get a guaranteed upper bound on the absolute error in an approximation. Our best hope is finding an interval that is guaranteed to contain the solution, as the Bisection Method does, and we can *sometimes* also do that with Newton's Method for a real root. But that approach fails as soon as the solution is a complex number or a vector.

Fortunately, when convergence is "fast enough" in some sense, the following *heuristic* or "rule of thumb" applies in many cases:

The error in the latest approximation is typically smaller than the difference between the two most recent approximations.

When combined with the *backward error*, this can give a fairly reliable measure of accuracy, and so can serve as a fairly reliable *stopping condition* for the loop in an iterative calculation.

When is a fixed point iteration “fast enough” for this heuristic?

This heuristic can be shown to be reliable in several important cases:

Proposition

For the iterations x_k given by a contraction mapping that has $C \leq 1/2$,

$$|E_k| \leq |x_k - x_{k-1}|,$$

or in words *the error in x_k is smaller than the change from x_{k-1} to x_k* , so the above guideline is valid.

Proposition

For a super-linearly convergent iteration, eventually $|E_{k+1}|/|E_k| < 1/2$, and from that point onwards in the iterations, the above applies again.

I leave verification as an exercise, or if you wish, to discuss in class.

2.7 Root-finding Without Derivatives

References:

- Section 1.5.1 *Secant Method and variants* in [Sauer, 2022]
- Section 2.3 *Newton’s Method and its Extensions* in [Burden *et al.*, 2016]; just the later sub-sections, on *The Secant Method* and *The Method of False Position*).

2.7.1 Introduction

We have already seen one method for solving $f(x) = 0$ without needing to know any derivatives of f : the *Bisection Method*, a.k.a. *Interval Halving*. However, we have also seen that that method is far slower than Newton’s Method.

Here we explore methods that are almost the best of both worlds: about as fast as Newton’s method but not needing derivatives.

The first of these is the Secant Method. Later in this course we will see how this has been merged with the Bisection Method and *Polynomial Interpolation* to produce the current state-of-the-art approach; only perfected in the 1960’s.

```
# We will often need resources from the modules numpy and pyplot:
import numpy as np
from numpy import abs, cos

# Since we do a lot of graphics in this section, some more short-hands:
from matplotlib.pyplot import figure, title, plot, xlabel, ylabel, grid, legend
from matplotlib.pyplot import show # This might be needed with MS Windows

# Also, some from the module for this book:
from numericalMethods_module import newton_method
```

2.7.2 Using Linear Approximation Without Derivatives

One quirk of the *Bisection Method* is that it only uses the sign of the values $f(a)$ and $f(b)$, not their magnitudes. If one of these is far smaller than the other, one might guess that the root is closer to that end of the interval. This leads to the idea of:

- starting with an interval $[a, b]$ known to contain a zero of f ,
- connecting the two points $(a, f(a))$ and $(b, f(b))$ with a straight line, and
- finding the x -value c where this line crosses the x -axis. In the words, approximating the function by a *secant line*, in place of the *tangent line* used in Newton's Method.

First Attempt: The Method of False Position

The next step requires some care. The first idea (from almost a millenium ago) was to use this new approximation c as done with bisection: check which of the intervals $[a, c]$ and $[c, b]$ has the sign change and use it as the new interval $[a, b]$; this is called *The Method of False Position* (or *Regula Falsi*, since the academic world used latin in those days.)

The secant line between $(a, f(a))$ and $(b, f(b))$ is

$$L(x) = \frac{f(a)(b-x) + f(b)(x-a)}{b-a}$$

and its zero is at

$$c = \frac{af(b) - f(a)b}{f(b) - f(a)}$$

This is easy to implement, and an example will show that it sort of works, but with a weakness that hampers it a bit:

```
def false_position(f, a, b, errorTolerance=1e-15, maxIterations=15, demoMode=False):
    """Solve f(x)=0 in the interval [a, b] by the Method of False Position.
    This code also illustrates a few ideas that I encourage, such as:
    - Avoiding infinite loops, by using for loops sand break
    - Avoiding repeated evaluation of the same quantity
    - Use of descriptive variable names
    - Use of "camelCase" to turn descriptive phrases into valid Python variable names
    - An optional "demonstration mode" to display intermediate results.
    """
    if demoMode: print(f"Solving by the Method of False Position.")
    fa = f(a)
    fb = f(b)
    for iteration in range(maxIterations):
        if demoMode: print(f"\nIteration {iteration}:")
        c = (a * fb - fa * b) / (fb - fa)
        fc = f(c)
        if fa * fc < 0:
            b = c
            fb = fc # N.B. When b is updated, so must be fb = f(b)
        else:
            a = c
            fa = fc
        errorBound = b - a
        if demoMode:
            print(f"The root is in interval [{a}, {b}]")
            print(f"The new approximation is {c}, with error bound {errorBound:0.4}, ↵
↵backward error {abs(fc):0.4}")
```

(continues on next page)

(continued from previous page)

```

    if errorBound < errorTolerance:
        break
    # Whether we got here due to accuracy of running out of iterations,
    # return the information we have, including an error bound:
    root = c # the newest value is probably the most accurate
    return (root, errorBound)

```

Remark 1.7.1

For a more concise presentation, you could omit the above `def` and instead import this function with

```
from numericalMethods_module import false_position
```

```
def f(x): return x - cos(x)
```

```

(root, errorBound) = false_position(f, a=-1, b=1, demoMode=True)
print(f"\nThe Method of False Position gave approximate root is {root},")
print(f"with estimate error {errorBound:0.4}, backward error {abs(f(root)):0.4}")

```

Solving by the Method of False Position.

Iteration 0:

The root is in interval [0.5403023058681398, 1]

The new approximation is 0.5403023058681398, with error bound 0.4597, backward
error 0.3173

Iteration 1:

The root is in interval [0.7280103614676172, 1]

The new approximation is 0.7280103614676172, with error bound 0.272, backward
error 0.01849

Iteration 2:

The root is in interval [0.7385270062423998, 1]

The new approximation is 0.7385270062423998, with error bound 0.2615, backward
error 0.000934

Iteration 3:

The root is in interval [0.7390571666782676, 1]

The new approximation is 0.7390571666782676, with error bound 0.2609, backward
error 4.68e-05

Iteration 4:

The root is in interval [0.7390837322783136, 1]

The new approximation is 0.7390837322783136, with error bound 0.2609, backward
error 2.345e-06

Iteration 5:

The root is in interval [0.7390850630385933, 1]

The new approximation is 0.7390850630385933, with error bound 0.2609, backward
error 1.174e-07

Iteration 6:

(continues on next page)

(continued from previous page)

```

The root is in interval [0.7390851296998365, 1]
The new approximation is 0.7390851296998365, with error bound 0.2609, backward_
error 5.883e-09

Iteration 7:
The root is in interval [0.7390851330390691, 1]
The new approximation is 0.7390851330390691, with error bound 0.2609, backward_
error 2.947e-10

Iteration 8:
The root is in interval [0.7390851332063397, 1]
The new approximation is 0.7390851332063397, with error bound 0.2609, backward_
error 1.476e-11

Iteration 9:
The root is in interval [0.7390851332147188, 1]
The new approximation is 0.7390851332147188, with error bound 0.2609, backward_
error 7.394e-13

Iteration 10:
The root is in interval [0.7390851332151385, 1]
The new approximation is 0.7390851332151385, with error bound 0.2609, backward_
error 3.708e-14

Iteration 11:
The root is in interval [0.7390851332151596, 1]
The new approximation is 0.7390851332151596, with error bound 0.2609, backward_
error 1.776e-15

Iteration 12:
The root is in interval [0.7390851332151606, 1]
The new approximation is 0.7390851332151606, with error bound 0.2609, backward_
error 1.11e-16

Iteration 13:
The root is in interval [0.7390851332151607, 1]
The new approximation is 0.7390851332151607, with error bound 0.2609, backward_
error 0.0

Iteration 14:
The root is in interval [0.7390851332151607, 1]
The new approximation is 0.7390851332151607, with error bound 0.2609, backward_
error 0.0

The Method of False Position gave approximate root is 0.7390851332151607,
with estimate error 0.2609, backward error 0.0

```

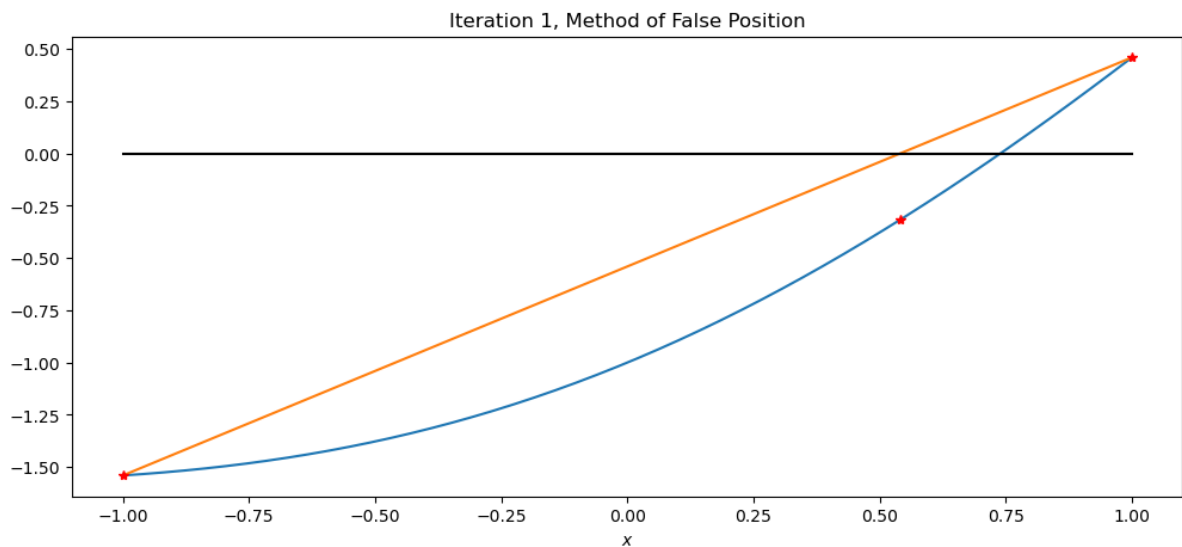
The good news is that the approximations are approaching the zero reasonably fast — far faster than bisection — as indicated by the backward errors improving by a factor of better than ten at each iteration.

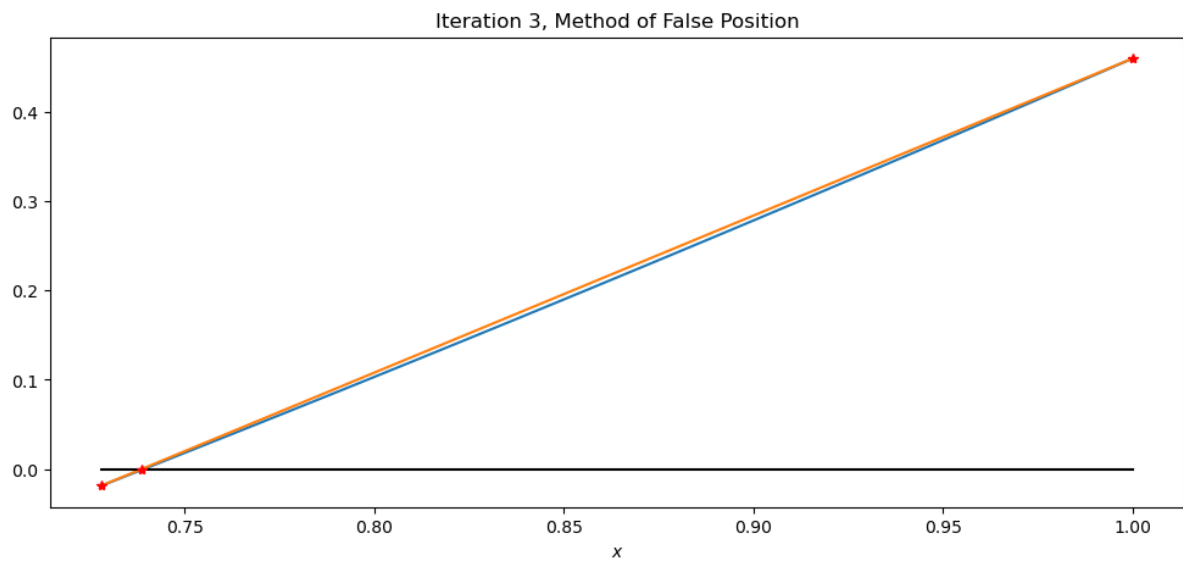
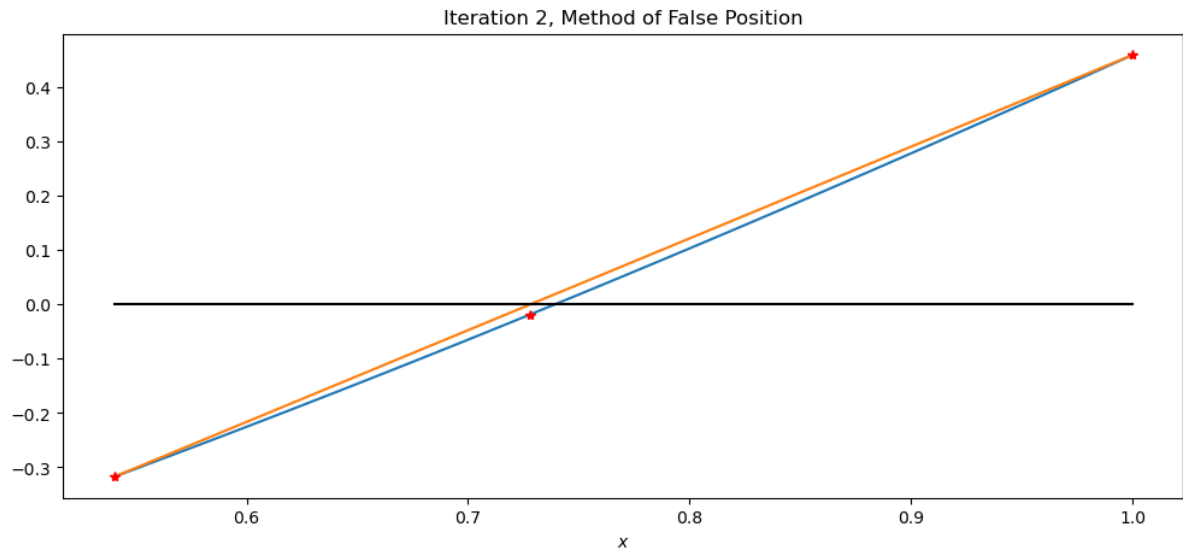
The bad news is that one end gets “stuck”, so the interval does not shrink on both sides, and the error bound stays large.

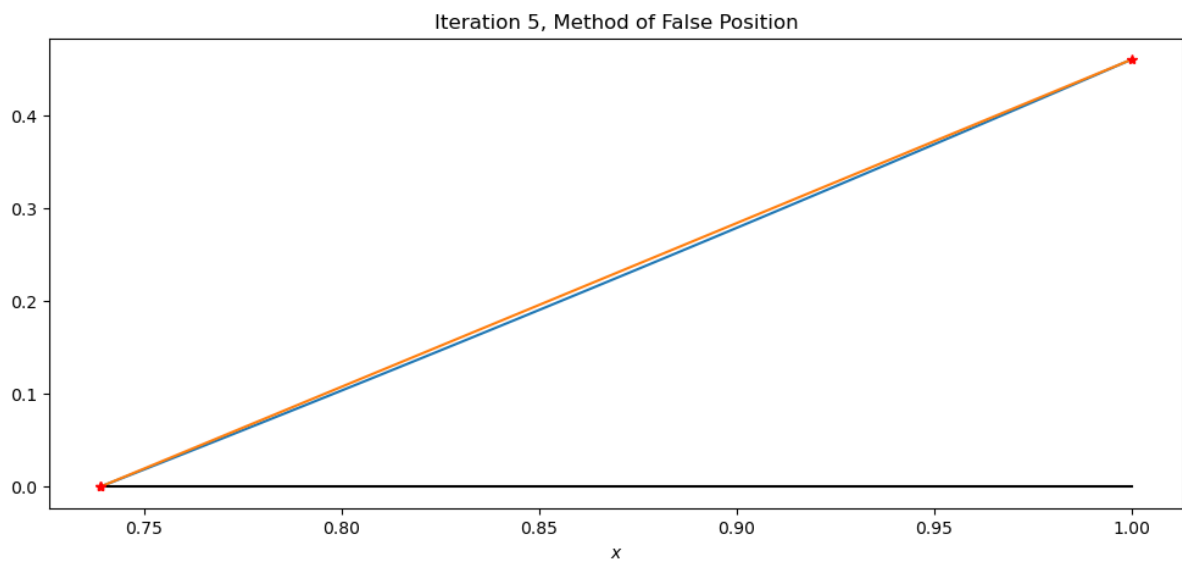
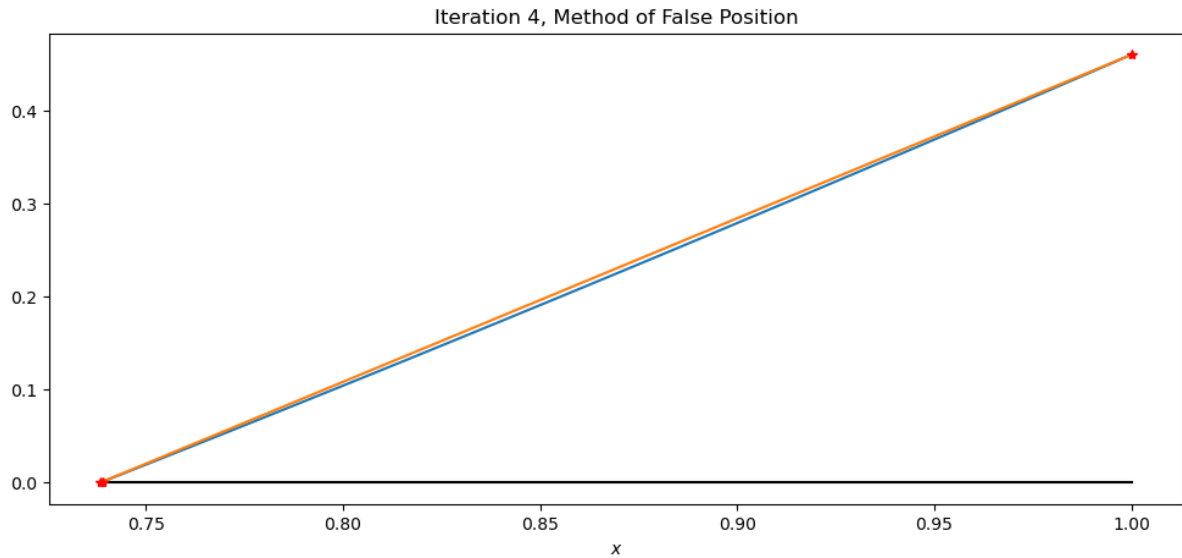
This behavior is generic: with function f of the same convexity on the interval $[a, b]$, the secant line will always cross on the same side of the zero, so that one end-point persists; in this case, the curve is concave up, so the secant line always crosses to the left of the root, as seen in the following graphs.

```
def graph_false_position(f, a, b, maxIterations=5):
    """Graph a few iterations of the Method of False Position for solving  $f(x)=0$  in
    the interval  $[a, b]$ .
    """
    fa = f(a)
    fb = f(b)
    for iteration in range(maxIterations):
        c = (a * fb - fa * b) / (fb - fa)
        fc = f(c)
        abc = [a, b, c]
        left = np.min(abc)
        right = np.max(abc)
        x = np.linspace(left, right)
        figure(figsize=[12, 5])
        title(f"Iteration {iteration+1}, Method of False Position")
        xlabel("$x$")
        plot(x, f(x))
        plot([left, right], [f(left), f(right)]) # the secant line
        plot([left, right], [0, 0], 'k') # the x-axis line
        plot(abc, f(abc), 'r*')
        #show() # The Windows version of JupyterLab might need this command
        if fa * fc < 0:
            b = c
            fb = fc # N.B. When b is updated, so must be fb = f(b)
        else:
            a = c
            fa = fc
```

```
graph_false_position(f, a=-1, b=1)
```







Refinement: Always Use the Two Most Recent Approximations — The Secant Method

The basic solution is to always discard the oldest approximation — at the cost of not always having the zero surrounded! This gives the Secant Method.

For a mathematical description, one typically enumerates the successive approximations as x_0, x_1 , etc., so the notation above gets translated with $a \rightarrow x_{k-2}, b \rightarrow x_{k-1}, c \rightarrow x_k$; then the formula becomes the recursive rule

$$x_k = \frac{x_{k-2}f(x_{k-1}) - f(x_{k-2})x_{k-1}}{f(x_{k-1}) - f(x_{k-2})}$$

Two difference from above:

- previously we could assume that $a < b$, but now we do not know the order of the various x_k values, and
- the root is not necessarily between the two most recent values, so we no longer have the simple error bound. (In fact, we will see that the zero is typically surrounded two-thirds of the time!)

Instead, we use the *magnitude* of $b - a$ which is now $|x_k - x_{k-1}|$, and this is only an *estimate* of the error. This is the same as used for Newton's Method; as there, it is still useful as a condition for ending the iterations and indeed tends to be pessimistic, so that we typically do one more iteration than needed — but it is not on its own a complete guarantee of having achieved the desired accuracy.

Pseudo-code for a Secant Method Algorithm

Algorithm 1.7.1 (Secant Method)

Input function f , interval endpoints x_0 and x_1 , an error tolerance E_{tol} , and an iteration limit N

for k from 2 to N

$$x_k \leftarrow \frac{x_{k-2}f(x_{k-1}) - f(x_{k-2})x_{k-1}}{f(x_{k-1}) - f(x_{k-2})}$$

Evaluate the error estimate $E_{est} \leftarrow |x_k - x_{k-1}|$

if $E_{est} \leq E_{tol}$

 End the iterations

else

 Go around another time

end

end

Output the final x_k as the approximate root and E_{est} as an estimate of its absolute error.

Python Code for this Secant Method Algorithm

We could write Python code that closely follows this notation, accumulating a list of the values x_k .

However, since we only ever need the two most recent values to compute the new one, we can instead just store these three, in the same way that we recycled the variables a , b and c . Here I use more descriptive names though:

```
def secant_method(f, a, b, errorTolerance=1e-15, maxIterations=15, demoMode=False):
    """Solve f(x)=0 in the interval [a, b] by the Secant Method."""
    if demoMode:
        print(f"Solving by the Secant Method.")
    # Some more descriptive names
    x_older = a
    x_more_recent = b
    f_x_older = f(x_older)
    f_x_more_recent = f(x_more_recent)
    for iteration in range(maxIterations):
        if demoMode: print(f"\nIteration {iteration}:")
        x_new = (x_older * f_x_more_recent - f_x_older * x_more_recent) / (f_x_more_
→ recent - f_x_older)
        f_x_new = f(x_new)
        (x_older, x_more_recent) = (x_more_recent, x_new)
        (f_x_older, f_x_more_recent) = (f_x_more_recent, f_x_new)
        errorEstimate = abs(x_older - x_more_recent)
    if demoMode:
```

(continues on next page)

(continued from previous page)

```

        print(f"The latest pair of approximations are {x_older} and {x_more_
↪recent},")
        print(f"where the function's values are {f_x_older:0.4} and {f_x_more_
↪recent:0.4} respectively.")
        print(f"The new approximation is {x_new}, with estimated error
↪{errorEstimate:0.4}, backward error {abs(f_x_new):0.4}")
        if errorEstimate < errorTolerance:
            break
        # Whether we got here due to accuracy of running out of iterations,
        # return the information we have, including an error estimate:
        return (x_new, errorEstimate)

```

Note: As above, you could omit the above def and instead import this function with

```
from numericalMethods_module import secant_method
```

```

(root, errorEstimate) = secant_method(f, a=-1, b=1, demoMode=True)
print(f"\nThe Secant Method gave approximate root is {root},")
print(f"with estimated error {errorEstimate:0.4}, backward error {abs(f(root)):0.4}")

```

Solving by the Secant Method.

Iteration 0:

The latest pair of approximations are 1 and 0.5403023058681398,
 where the function's values are 0.4597 and -0.3173 respectively.
 The new approximation is 0.5403023058681398, with estimated error 0.4597, backward_
 ↪error 0.3173

Iteration 1:

The latest pair of approximations are 0.5403023058681398 and 0.7280103614676172,
 where the function's values are -0.3173 and -0.01849 respectively.
 The new approximation is 0.7280103614676172, with estimated error 0.1877, backward_
 ↪error 0.01849

Iteration 2:

The latest pair of approximations are 0.7280103614676172 and 0.7396270126307336,
 where the function's values are -0.01849 and 0.000907 respectively.
 The new approximation is 0.7396270126307336, with estimated error 0.01162, ↪
 ↪backward error 0.000907

Iteration 3:

The latest pair of approximations are 0.7396270126307336 and 0.7390838007832722,
 where the function's values are 0.000907 and -2.23e-06 respectively.
 The new approximation is 0.7390838007832722, with estimated error 0.0005432, ↪
 ↪backward error 2.23e-06

Iteration 4:

The latest pair of approximations are 0.7390838007832722 and 0.7390851330557805,
 where the function's values are -2.23e-06 and -2.667e-10 respectively.
 The new approximation is 0.7390851330557805, with estimated error 1.332e-06, ↪
 ↪backward error 2.667e-10

Iteration 5:

The latest pair of approximations are 0.7390851330557805 and 0.7390851332151607,

(continues on next page)

(continued from previous page)

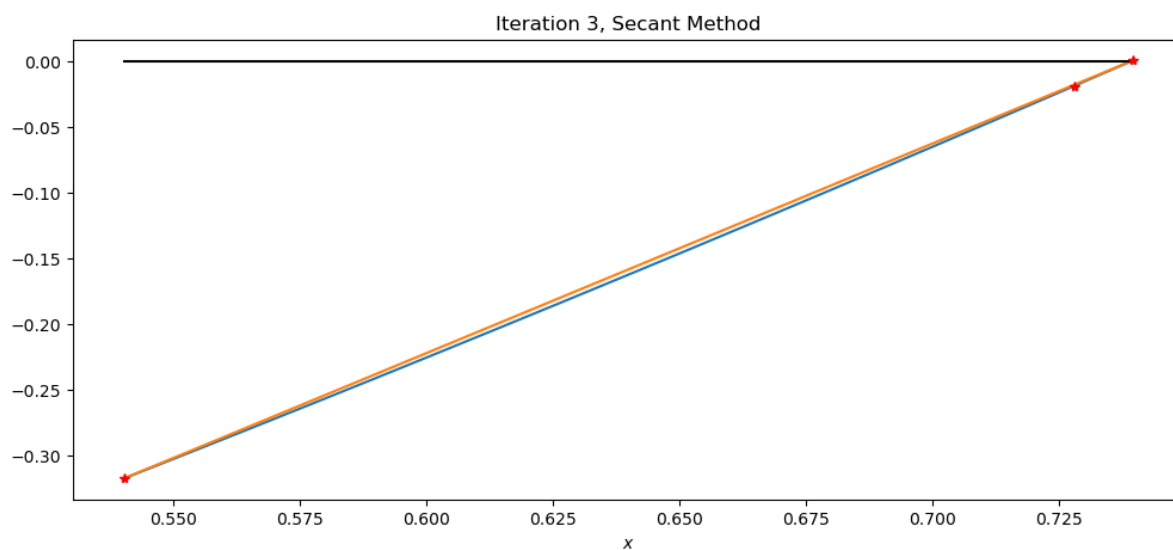
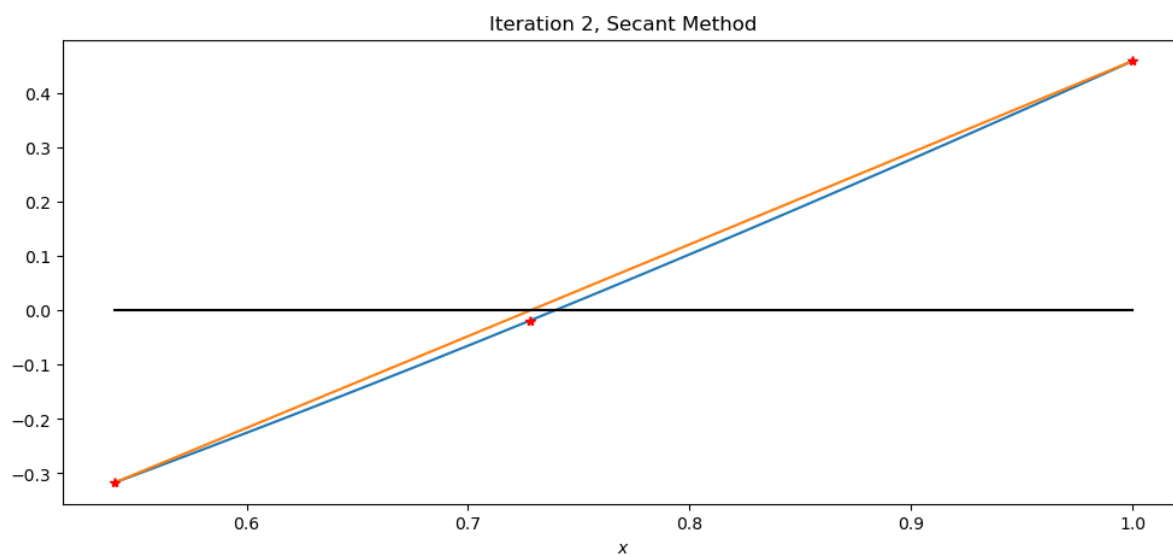
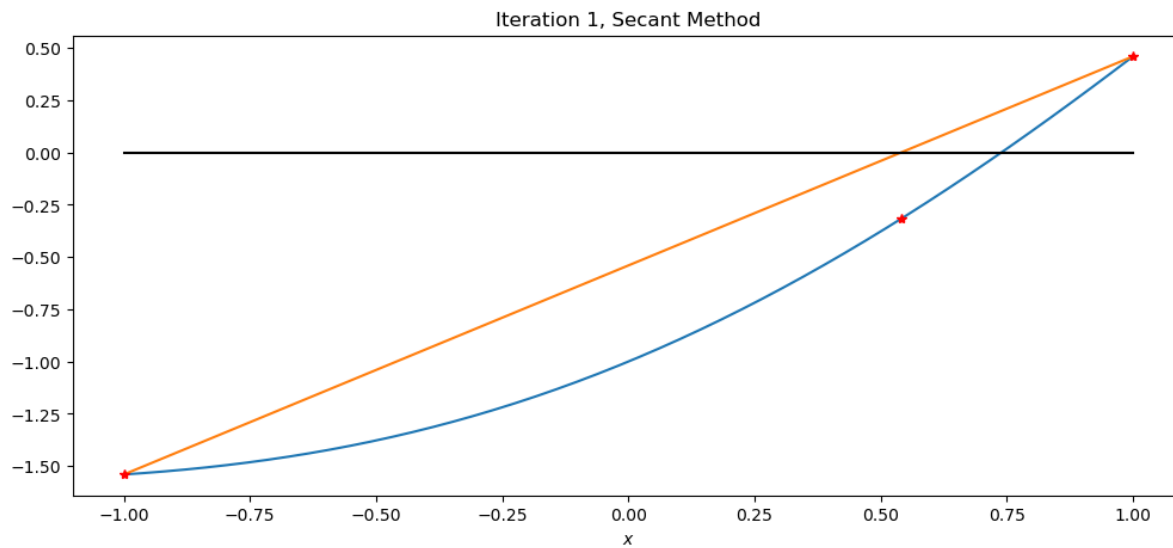
where the function's values are $-2.667\text{e-}10$ and 0.0 respectively.
 The new approximation is 0.7390851332151607 , with estimated error $1.594\text{e-}10$,
 backward error 0.0

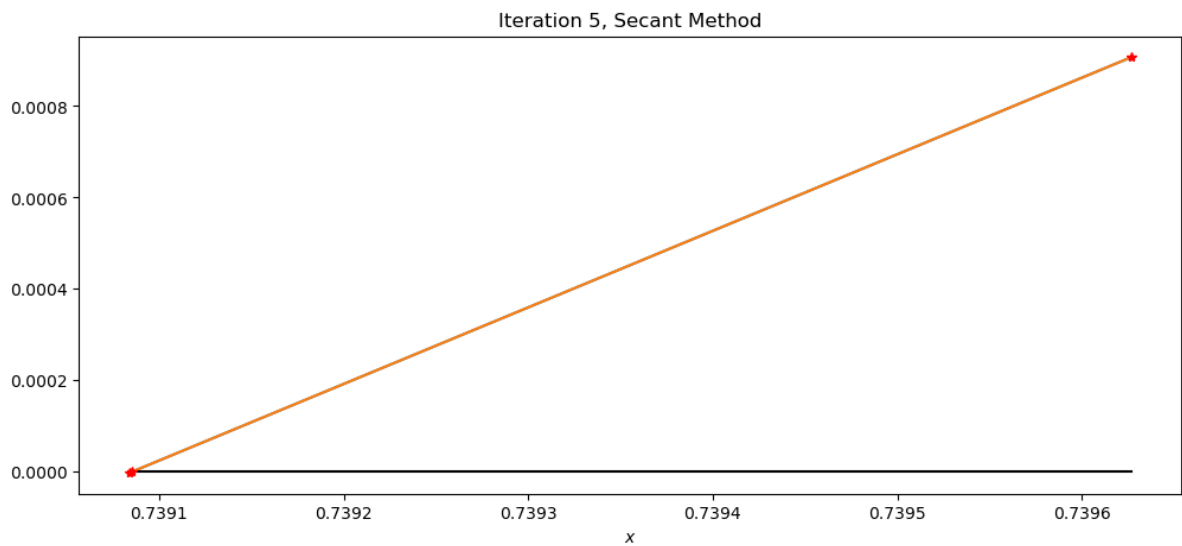
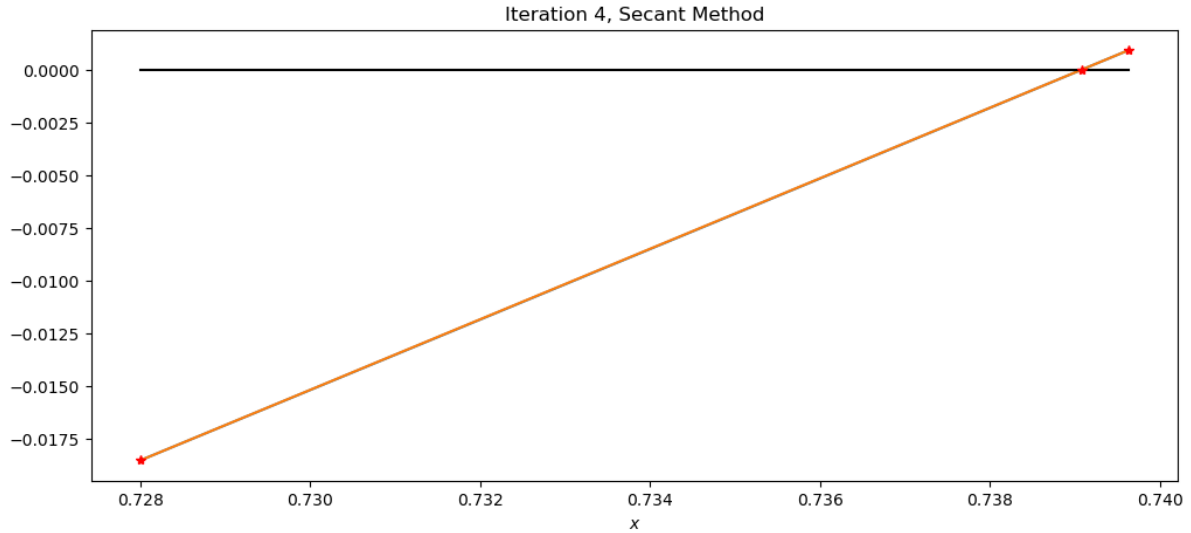
Iteration 6:
 The latest pair of approximations are 0.7390851332151607 and 0.7390851332151607 ,
 where the function's values are 0.0 and 0.0 respectively.
 The new approximation is 0.7390851332151607 , with estimated error 0.0 , backward
 error 0.0

The Secant Method gave approximate root is 0.7390851332151607 ,
 with estimated error 0.0 , backward error 0.0

```
def graph_secant_method(f, a, b, maxIterations=5):
    """Graph a few iterations of the Secant Method for solving  $f(x)=0$  in the interval
     $[a, b]$ ."""
    x_older = a
    x_more_recent = b
    f_x_older = f(x_older)
    f_x_more_recent = f(x_more_recent)
    for iteration in range(maxIterations):
        x_new = (x_older * f_x_more_recent - f_x_older * x_more_recent) / (f_x_more_
        recent - f_x_older)
        f_x_new = f(x_new)
        latest_three_x_values = [x_older, x_more_recent, x_new]
        left = np.min(latest_three_x_values)
        right = np.max(latest_three_x_values)
        x = np.linspace(left, right)
        figure(figsize=[12,5])
        title(f"Iteration {iteration+1}, Secant Method")
        xlabel("$x$")
        plot(x, f(x))
        plot([left, right], [f(left), f(right)]) # the secant line
        plot([left, right], [0, 0], 'k') # the x-axis line
        plot(latest_three_x_values, f(latest_three_x_values), 'r*')
        # show() # The Windows version of JupyterLab might need this command
        (x_older, x_more_recent) = (x_more_recent, x_new)
        (f_x_older, f_x_more_recent) = (f_x_more_recent, f_x_new)
        errorEstimate = abs(x_older - x_more_recent)
```

```
graph_secant_method(f, a=-1, b=1)
```





Observations

- This converges faster than the *First Attempt: The Method of False Position* (and far faster than Bisection).
- The majority of iterations do have the root surrounded (sign-change in f), but every third one — the second and fifth — do not.
- Comparing the error estimate to the backward error, the error estimate is in fact quite pessimistic (and so fairly trustworthy); in fact, it is typically of similar size to the backward error at the previous iteration.

The last point is a quite common occurrence: the available error estimates are often “trailing indicators”, closer to the error in the previous approximation in an iteration. For example, recall that we saw the same thing with Newton’s Method when we used $|x_k - x_{k-1}|$ to estimate the error $E_k := x_k - r$ and saw that it is in fact closer to the previous error, E_{k-1} .

LINEAR ALGEBRA AND SIMULTANEOUS EQUATIONS

3.1 Row Reduction/Gaussian Elimination

References:

- Section 2.1.1 *Naive Gaussian elimination* of [Sauer, 2022].
- Section 6.1 *Linear Systems of Equations* of [Burden et al., 2016].
- Section 7.1 of [Chenney and Kincaid, 2012].

3.1.1 Introduction

The problem of solving a system of n simultaneous linear equations in n unknowns, with matrix-vector form $Ax = b$, is quite thoroughly understood as far as having a good general-purpose methods usable with any $n \times n$ matrix A : essentially, Gaussian elimination (or row-reduction) as seen in most linear algebra courses, combined with some modifications to stay well away from division by zero: *partial pivoting*. Also, good robust software for this general case is readily available, for example in the Python packages NumPy and SciPy.

Nevertheless, this basic algorithm can be very slow when n is large – as it often is when dealing with differential equations (even more so with *partial* differential equations). We will see that it requires about $n^3/3$ arithmetic operations.

Thus I will summarise the basic method of row reduction or Gaussian elimination, and then build on it with methods for doing things more robustly, and then with methods for doing it faster in some important special cases:

1. When one has to solve many systems $Ax^{(m)} = b^{(m)}$ with the same matrix A but different right-hand side vectors $b^{(m)}$.
2. When A is *banded*: most elements are zero, and all the non-zero elements $a_{i,j}$ are near the main diagonal: $|i - j|$ is far less than n . (*Aside on notation*: “far less than” is sometimes denoted $\ll$, as in $|i - j| \ll n$.)
3. When A is *strictly diagonally dominant*: each diagonal element $a_{i,i}$ is larger in magnitude than the sum of the magnitudes of all other elements in the same row.

Other cases not (yet) discussed in this text are

4. When A is *positive definite*: symmetric ($a_{i,j} = a_{j,i}$) and with all eigenvalues positive. This last condition would seem hard to verify, since computing all the eigenvalues of A is harder than solving $Ax = b$, but there are important situations where this property is automatically guaranteed, such as with *Galerkin* and *finite-element methods* for solving boundary value problems for differential equations.
5. When A is *sparse*: most elements are zero, but not necessarily with all the non-zero elements near the main diagonal.

Remark 2.1.1 (Module `numpy.linalg` with standard nickname `la`)

Python package Numpy provides a lot of useful tools for numerical linear algebra through a module `numpy.linalg`.

Just as package `numpy` is used so often that there is a conventional nickname `np`, so `numpy.linalg` is usually nicknamed `la`.

```
import numpy as np
import numpy.linalg as la

# As in recent sections, we import some items from modules individually, so they can
# be used by "first name only".
from numpy import array, inf, zeros_like, empty
```

3.1.2 Strategy for getting from mathematical facts to a good algorithm and then to its implementation in [Python] code

Here I take the opportunity to illustrate some useful strategies for getting from mathematical facts and ideas to good algorithms and working code for solving a numerical problem. The pattern we will see here, and often later, is:

Step 1. Get a basic algorithm:

1. Start with mathematical facts (like the equations $\sum_{j=1}^n a_{ij}x_j = b_i$).
2. Solve to get an equation for each unknown — or for an updated approximation of each unknown — in terms of other quantities.
3. Specify an order of evaluation in which all the quantities at right are evaluated earlier.

In this, it is often best to start with a verbal description before specifying the details in more precise and detailed mathematical form.

Step 2. Refine to get a more robust algorithm:

1. Identify cases that can lead to failure due to division by zero and such, and revise to avoid them.
2. Avoid inaccuracy due to problems like severe rounding error. One rule of thumb is that anywhere that a zero value is a fatal flaw (in particular, division by zero), a very small value is also a hazard when rounding error is present. So **avoid very small denominators**. (We will soon examine this through the phenomenon of **loss of significance**, and its extreme case **catastrophic cancellation**.)

Step 3. Refine to get a more efficient algorithm

For example,

- Avoid repeated evaluation of exactly the same quantity.
- Avoid redundant calculations, such as ones whose value can be determined in advance; for example, values that can be shown in advance to be zero.
- Compare and choose between alternative algorithms.

3.1.3 Gaussian Elimination, a.k.a. Row Reduction

We start by considering the most basic algorithm, based on ideas seen in a linear algebra course.

The problem is best stated as a collection of equations for individual numerical values:

Given coefficients $a_{i,j}$, $1 \leq i \leq n$, $1 \leq j \leq n$ and right-hand side values b_i , $1 \leq i \leq n$, solve for the n unknowns x_j , $1 \leq j \leq n$ in the equations $\sum_{j=1}^n a_{i,j}x_j = b_i$, $1 \leq i \leq n$.

In verbal form, the basic strategy of *row reduction* or *Gaussian elimination* is this:

- **Choose** one equation and use it to eliminate one **chosen** unknown from all the other equations, leaving that chosen equation plus $n - 1$ equations in $n - 1$ unknowns.
- Repeat recursively, at each stage using one of the remaining equations to eliminate one of the remaining unknowns from all the other equations.
- This gives a final equation in just one unknown, preceded by an equation in that unknown plus one other, and so on: solve them in this order, from last to first.

Determining those choices, to produce a first algorithm: “Naive Gaussian Elimination”

A precise algorithm must include rules specifying all the choices indicated above. The simplest “naive” choice, which works in most but not all cases, is to eliminate from the top to bottom and left to right:

- Use the first equation to eliminate the first unknown from all other equations.
- Repeat recursively, at each stage using the first remaining equation to eliminate the first remaining unknown. Thus, at step k , equation k is used to eliminate unknown x_k .
- This gives one equation in just the last unknown x_n ; another equation in the last two unknowns x_{n-1} and x_n , and so on: solve them in this reverse order, evaluating the unknowns from last to first.

This usually works, but can fail because at some stage the (updated) k -th equation might not include the k -th unknown: that is, its coefficient might be zero, leading to division by zero.

We will refine the algorithm to deal with that in the section on *Partial Pivoting*.

Remark 2.1.2 (Using Numpy for matrices, vectors and their products)

As of version 3.5 of Python, vectors, matrices, and their products can be handled very elegantly using Numpy arrays, with the one quirk that the product is denoted by the at-sign @. That is, for a matrix A and compatible matrix or vector b both stored in Numpy arrays, their product is given by $A @ b$.

This means that, along with my encouragement to totally ignore *Python* arrays in favor of *Numpy* arrays, and to usually avoid Python *lists* when working with numerical data, I also recommend that you ignore the now obsolescent Numpy `matrix` data type, if you happen to come across it in older material on Numpy.

Aside: Why not $A * b$? Because that is the more general “point-wise” array product: $c = A * b$ gives array c with $c[i, j]$ equal to $A[i, j] * b[i, j]$, which is not how matrix multiplication works.

3.1.4 The general case of solving $Ax = b$, using Python and NumPy

The problem of solving $Ax = b$ in general, when all you know is that A is an $n \times n$ matrix and b is an n -vector, can in most cases be handled well by using standard software rather than by writing your own code. Here is an example in Python, solving

$$\begin{bmatrix} 4 & 2 & 7 \\ 3 & 5 & -6 \\ 1 & -3 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

using the array type from package numpy and the function solve from the linear algebra module numpy.linalg.

```
A = array([[4., 2., 7.], [3., 5., -6.], [1., -3., 2.]])
print(f"A = \n{A}")
b = array([2., 3., 4.])
print(f"b = {b}")
print(f"A @ b = {A @ b}")
```

```
A =
[[ 4.  2.  7.]
 [ 3.  5. -6.]
 [ 1. -3.  2.]]
b = [2. 3. 4.]
A @ b = [42. -3.  1.]
```

Remark 2.1.3 (floating point numbers versus integers)

It is important to specify that the entries are real numbers (type “float”); otherwise Numpy does integer arithmetic.

One way to do this is as above: putting a decimal point in the numbers (or to be lazy, in at least one of them!)

Another is to tell the function array that the type is float:

```
A = array([[4, 2, 7], [3, 5, -6], [1, -3, 2]], dtype=float)
b = array([2, 3, 4], dtype=float)
```

```
x = la.solve(A, b)
print("numpy.linalg.solve says that the solution of Ax = b is")
print(f"x = {x}")
# Check the backward error, also known as the residual
r = b - A @ x
print(f"\nAs a check, the residual (or backward error) is")
print(f"    r = b-Ax = {r},")
print(f"and its infinity (or 'maximum') norm is ||r|| = {la.norm(r, inf)}")
print("\nAside: another way to compute this is with max(abs(r)):")
print(f"||r|| = {max(abs(r))}")
print(f"and its 1-norm is ||r|| = {la.norm(r, 1)}")
```

```
numpy.linalg.solve says that the solution of Ax = b is
x = [ 1.81168831 -1.03246753 -0.45454545]
```

```
As a check, the residual (or backward error) is
    r = b-Ax = [0.0000000e+00 0.0000000e+00 4.4408921e-16],
and its infinity (or 'maximum') norm is ||r|| = 4.440892098500626e-16
```

(continues on next page)

(continued from previous page)

```

Aside: another way to compute this is with max(abs(r)):
||r|| = 4.440892098500626e-16
and its 1-norm is ||r|| = 4.440892098500626e-16

```

Remark 2.1.4 (Not quite zero values and rounding)

Some values here that you might hope to be zero are instead very small non-zero numbers, with exponent 10^{-16} , due to rounding error in computer arithmetic. For details on this (like why “-16” in particular) see [Machine Numbers, Rounding Error and Error Propagation](#).

3.1.5 The naive Gaussian elimination algorithm, in pseudo-code

Here the elements of the transformed matrix and vector after step k are named $a_{i,j}^{(k)}$ and $b_i^{(k)}$, so that the original values are $a_{i,j}^{(0)} = a_{i,j}$ and $b_i^{(0)} = b_i$.

The name $l_{i,k}$ is given to the multiple of row k that is subtracted from row i at step k . This naming might seem redundant, but it becomes very useful later, in the section on [LU factorization](#).

Algorithm 2.1.1 (naive Gaussian elimination)

for k from 1 to $n-1$ *Step k : get zeros in column k below row k :*

 for i from $k+1$ to n

Evaluate the multiple of row k to subtract from row i :

$$l_{i,k} = a_{i,k}^{(k-1)} / a_{k,k}^{(k-1)} \quad \text{If } a_{k,k}^{(k-1)} \neq 0!$$

Subtract ($l_{i,k}$ times row k) from row i in matrix A ...:

 for j from 1 to n

$$a_{i,j}^{(k)} = a_{i,j}^{(k-1)} - l_{i,k} a_{k,j}^{(k-1)}$$

 end

 ... and at right, subtract ($l_{i,k}$ times b_k) from b_i :

$$b_i^{(k)} = b_i^{(k-1)} - l_{i,k} b_k^{(k-1)}$$

 end

The rows before $i = k$ are unchanged, so they are omitted from the update; however, in a situation where we need to complete the definitions of $A^{(k)}$ and $b^{(k)}$ we would also need the following inside the `for k` loop:

Algorithm 2.1.2 (Inserting the zeros below the main diagonal)

for i from 1 to k

 for j from 1 to n

$$a_{i,j}^{(k)} = a_{i,j}^{(k-1)}$$

 end

$$b_i^{(k)} = b_i^{(k-1)}$$

end

However, the algorithm will usually be implemented by overwriting the previous values in an array with new ones, and then this part is redundant.

The next improvement in efficiency: the updates in the first k columns at step k give zero values (that is the key idea of the algorithm!), so there is no need to compute or store those zeros, and thus the only calculations needed in the above `for j from 1 to n` loop are covered by `for j from k+1 to n`. Thus from now on we use only the latter: except when, for demonstration purposes, we need those zeros.

Thus, the standard algorithm looks like this:

Algorithm 2.1.3 (basic Gaussian elimination)

```

for k from 1 to n-1      Step k: Get zeros in column k below row k:
    for i from k+1 to n    Update only the rows that change: from k+1 on:
        Evaluate the multiple of row k to subtract from row i:

$$l_{i,k} = a_{i,k}^{(k-1)} / a_{k,k}^{(k-1)} \quad \text{If } a_{k,k}^{(k-1)} \neq 0!$$

        Subtract ( $l_{i,k}$  times row k) from row i in matrix A, in the columns that are not automatically zero:
        for j from k+1 to n

$$a_{i,j}^{(k)} = a_{i,j}^{(k-1)} - l_{i,k} a_{k,j}^{(k-1)}$$

        end
        and at right, subtract ( $l_{i,k}$  times  $b_k$ ) from  $b_i$ :

$$b_i^{(k)} = b_i^{(k-1)} - l_{i,k} b_k^{(k-1)}$$

    end
end

```

Remark 2.1.5 (Syntax for for loops and 0-based array indexing)

Since array indices in Python (and in Java, C, C++, C#, Swift, etc.) start from zero, not from one, it will be convenient to express linear algebra algorithms in a form compatible with this.

- Every index is one less than in the above! Thus in an array with n elements, the index values i are $0 \leq i < n$, **excluding n** , which is the half-open interval of integers $[0, n)$.
- In the indexing of an array, one can refer to the part the array with indices $a \leq i < b$, **excluding b** , with the **slice** notation `a:b`.
- Similarly, when specifying the range of consecutive integers i , $a \leq i < b$ in a `for` loop, one can use the expression `range(a, b)`.

Also, when indices are processed in order (from low to high), these notes will abuse notation slightly, referring to the values as a set — specifically, a semi-open interval of integers.

For example, the above loop

```
for j from k+1 to n:
```

first gets all indices lowered by one, to


```
for j from k to n-1:
```

and then this will sometimes be described in terms of the set of j values:

```
for j in [k,n):
```

which in Python becomes

```
for j in range(k, n):
```

This new notation needs care initially, but helps with clarity in the long run. For one thing, it means that the indices of an n -element array, $[0, n-1)$, are described by `range(0, n)` and by `0:n`. In fact, the case of “starting at the beginning”, with index zero, can be abbreviated: `range(n)` is the same as `range(0, n)`, and `:b` is the same as `0:b`.

Another advantage is that the index ranges $a:b$ and $b:c$ together cover the same indices as $a:c$, with no gap or duplication of b , and likewise `range(a, b)` and `range(b, c)` combine to cover `range(a, c)`.

3.1.6 The naive Gaussian elimination algorithm, in Pythonic zero-based pseudo-code

Here the above notational shift is made, along with eliminating the above-noted redundant formulas for values that are either zero or are unchanged from the previous step. It is also convenient for k to be the index of the row being used to reduce subsequent rows, and so also the index of the column in which values below the main diagonal are being set to zero.

Algorithm 2.1.4

for k in [0, n-1):	for i in [k+1, n):	$l_{i,k} = a_{i,k}^{(k)} / a_{k,k}^{(k)}$	If $a_{k,k}^{(k)} \neq 0!$	for j in [k+1, n):	$a_{i,j}^{(k+1)} =$
$a_{i,j}^{(k)} - l_{i,k} a_{k,j}^{(k)}$	end	$b_i^{(k+1)} = b_i^{(k)} - l_{i,k} b_k^{(k)}$	end end		

3.1.7 The naive Gaussian elimination algorithm, in Python

Conversion to actual Python code is now quite straightforward; there is little more to be done than:

- Change the way that indices are described, from b_i to `b[i]` and from $a_{i,j}$ to `A[i, j]`.
- Use case consistently in array names, since the quirk in mathematical notation of using upper-case letters for matrix names but lower case letters for their elements is gone! In these notes, matrix names will be upper-case and vector names will be lower-case (even when a vector is considered as 1-column matrix).
- Rather than create a new array for each matrix $A^{(0)}$, $A^{(1)}$, etc. and each vector $b^{(0)}$, $b^{(1)}$, we overwrite each in the same array.

Remark 2.1.6

We will see that this simplicity in translation is quite common once algorithms have been expressed with zero-based indexing. The main ugliness is with loops that count backwards; see below.

```
for k in range(n-1):
    for i in range(k+1, n):
        L[i,k] = A[i,k] / A[k,k]
        for j in range(k+1, n):
            A[i,j] -= L[i,k] * A[k,j]
        b[i] -= L[i,k] * b[k]
```

To demonstrate this, some additions are needed:

- Putting this algorithm into a function.
- Getting the value n needed for the loop, using the fact that it is the length of vector b .
- Creating the array L .
- Copying the input arrays A and b into new ones, U and c , so that the original arrays are not changed. That is, when the row reduction is completed, U contains $A^{(n-1)}$ and c contains $b^{(n-1)}$.

Also, for some demonstrations, the zero values below the main diagonal of U are inserted, though usually they would not be needed.

```
def rowreduce(A, b):
    """To avoid modifying the matrix and vector specified as input,
    they are copied to new arrays, with the method .copy()
    Warning: it does not work to say "U = A" and "c = b";
    this makes these names synonyms, referring to the same stored data.
    """
    U = A.copy()
    c = b.copy()
    n = len(b)
    # The function zeros_like() is used to create L with the same size and shape as A,
    # and with all its elements zero initially.
    L = np.zeros_like(A)
    for k in range(n-1):
        for i in range(k+1, n):
            # compute all the L values for column k:
            L[i,k] = U[i,k] / U[k,k] # Beware the case where U[k,k] is 0
            for j in range(k+1, n):
                U[i,j] -= L[i,k] * U[k,j]

            # Put in the zeros below the main diagonal in column k of U;
            # this is not important for calculations, since those elements of U are
            # not used in backward substitution,
            # but it helps for displaying results and for checking the results via
            # residuals.
            U[i,k] = 0.

        c[i] -= L[i,k] * c[k]
    return (U, c)
```

Note: As usual, you could omit the above `def` and instead import this functions with

```
from numericalMethods import rowreduce
```

```
(U, c) = rowreduce(A, b)
print(f"Row reduction gives\nU =\n{U}")
print(f"c = {c}")
```

```

Row reduction gives
U =
[[ 4.      2.      7.  ]
 [ 3.      3.5    -11.25]
 [ 1.     -3.5    -11.  ]]
c = [2.   1.5 5. ]

```

Let's take advantage of the fact that we have used `la.solve` to get a very accurate approximation of the solution x of $Ax = b$; this should also solve $Ux = c$, so check the backward error, a.k.a. the *residual*:

```

r = c - U@x
print(f"\nThe residual (backward error) c-Ux is {r}, with maximum norm {max(abs(r))}.".
↪")

```

```

The residual (backward error) c-Ux is [ 0.          -5.43506494 -5.42532468], with
↪maximum norm 5.4350649350649345.

```

Remark 2.1.7 (Array slicing in Python)

Operations on a sequence of array indices, with “slicing”: vectorization

Python code can specify vector operations on a range of indices $[c, d)$, referred to with the slice notation $c:d$. For example, the *slice* notation $A[c:d, j]$ refers to the array containing the $d - c$ elements $A[i, j]$ for i in the semi-open interval $[c, d)$.

Thus, each of the three arithmetic calculations above can be specified over a range of index values in a single command, eliminating all the inner-most `for` loops; this is sometimes called *vectorization*. Only `for` loops that contains other `for` loops remain.

Apart from mathematical elegance, this usually allows far faster execution.

```

for k in range(n-1):
    L[k+1:n,k] = U[k+1:n,k] / U[k,k] # compute all the L values for column k
    for i in range(k+1, n):
        U[i,k+1:n] -= L[i,k] * U[k,k+1:n] # Update row i
    c[k+1:n] -= L[k+1:n,k] * c[k] # update c values

```

I will break my usual guideline by redefining `rowreduce`, since this is just a different statement of exactly the same algorithm:

```

def rowreduce(A, b, demomode=False):
    """To avoid modifying the matrix and vector specified as input,
    they are copied to new arrays, with the method .copy()
    Warning: it does not work to say "U = A" and "c = b";
    this makes these names synonyms, referring to the same stored data.
    """
    U = A.copy()
    c = b.copy()
    n = len(b)
    # The function zeros_like() is used to create L with the same size and shape as A,
    ↪ and with all its elements zero initially.
    L = np.zeros_like(A)
    for k in range(n-1):
        if demomode: print(f"Step {k=}")

```

(continues on next page)

(continued from previous page)

```

# compute all the L values for column k:
L[k+1:,k] = U[k+1:n,k] / U[k,k] # Beware the case where U[k,k] is 0
if demomode:
    print(f"The multipliers in column {k+1} are {L[k+1:,k]}")
for i in range(k+1, n):
    U[i,k+1:n] -= L[i,k] * U[k,k+1:n] # Update row i
    # Insert the below-diagonal zeros in column k;
    # this is not important for calculations, since those elements of U are
↪not used in backward substitution,
    # but it helps for displaying results and for checking the results via
↪residuals.
    U[i,k] = 0.0

c[k+1:n] -= L[k+1:n,k] * c[k] # update c values
if demomode:
    # insert zeros in U:
    U[k+1:, k] = 0.
    print(f"The updated matrix is\n{U}")
    print(f"The updated right-hand side is\n{c}")
return (U, c)

```

Remark 2.1.8 (another way to select some rows or columns of a matrix)

As a variant on slicing, one can give a list of indices to select rows or columns of a matrix; for example:

```
A[[r1 r2 r3], :]
```

gives a three row part of array A and

```
A[2:, [c1 c2 c3 c4]]
```

selects the indicated four columns — but only from row 2 onwards.

This gives another way to describe the update of the lower-right block $U[k+1:n, k+1:n]$ with a single matrix multiplication: it is the *outer product* of part of column k of L after row k by the part of row k of U after column k .

To specify that the pieces of L and U are identified as a 1-column matrix and a 1-row matrix respectively, rather than as vectors, the above “row/column list” method must be used, with the list being just $[k]$ in each case.

```

def rowreduce(A, b, demomode=False):
    """To avoid modifying the matrix and vector specified as input,
    they are copied to new arrays, with the method .copy()
    Warning: it does not work to say "U = A" and "c = b";
    this makes these names synonyms, referring to the same stored data.
    """
    U = A.copy()
    c = b.copy()
    n = len(b)
    # The function zeros_like() is used to create L with the same size and shape as A,
    ↪ and with all its elements zero initially.
    L = np.zeros_like(A)
    for k in range(n-1):
        if demomode: print(f"Step {k=}")
        # compute all the L values for column k:

```

(continues on next page)

(continued from previous page)

```

L[k+1:,k] = U[k+1:n,k] / U[k,k] # Beware the case where U[k,k] is 0
if demomode:
    print(f"The multipliers in column {k+1} are {L[k+1:,k]}")
U[k+1:n,k+1:n] -= L[k+1:n,[k]] @ U[[k],k+1:n] # The new "outer product"
↪method.
# Insert the below-diagonal zeros in column k;
# this is not important for calculations, since those elements of U are not
↪used in backward substitution,
# but it helps for displaying results and for checking the results via
↪residuals.
U[k+1:n,k] = 0.0

c[k+1:n] -= L[k+1:n,k] * c[k] # update c values
if demomode:
    U[k+1:n, k] = 0. # insert zeros in column k of U:
    print(f"The updated matrix is\n{U}")
    print(f"The updated right-hand side is\n{c}")
return (U, c)
    
```

Repeating the above testing:

3.1.8 Backward substitution with an upper triangular matrix

The transformed equations have the form

$$\begin{aligned}
 u_{1,1}x_1 + u_{1,2}x_2 + u_{1,3}x_3 + \cdots + u_{1,n}x_n &= c_1 \\
 &\vdots \\
 u_{i,i}x_i + u_{i+1,i+1}x_{i+1} + \cdots + u_{i,n}x_n &= c_i \\
 &\vdots \\
 u_{n-1,n-1}x_{n-1} + u_{n-1,n}x_n &= c_{n-1} \\
 u_{nn}x_n &= c_n
 \end{aligned}$$

and can be solved from bottom up, starting with $x_n = c_n/u_{n,n}$.

All but the last equation can be written as

$$u_{i,i}x_i + \sum_{j=i+1}^n u_{i,j}x_j = c_i, \quad 1 \leq i \leq n-1$$

and so solved as

$$x_i = \frac{c_i - \sum_{j=i+1}^n u_{i,j}x_j}{u_{i,i}}, \quad \text{If } u_{i,i} \neq 0$$

This procedure is **backward substitution**, giving the algorithm

Algorithm 2.1.5

$$x_n = c_n/u_{n,n} \text{ for } i \text{ from } n-1 \text{ down to } 1 \quad x_i = \frac{c_i - \sum_{j=i+1}^n u_{i,j}x_j}{u_{i,i}} \text{ end}$$

This works so long as none of the main diagonal terms $u_{i,i}$ is zero, because when done in this order, everything on the right hand side is known by the time it is evaluated.

For future reference, note that the elements $u_{k,k}$ that must be non-zero here, the ones on the **main diagonal** of U , are the same as the elements $a_{k,k}^{(k)}$ that must be non-zero in the row reduction stage above, because after stage k , the elements of row k do not change any more: $a_{k,k}^{(k)} = a_{k,k}^{(n-1)} = u_{k,k}$.

3.1.9 The backward substitution algorithm in zero-based pseudo-code

Again, a zero-based version is more convenient for programming in Python (or Java, or C++):

Algorithm 2.1.6

$$x_{n-1} = c_{n-1}/u_{n-1,n-1} \text{ for } i \text{ from } n-2 \text{ down to } 0 \quad x_i = \frac{c_i - \sum_{j=i+1}^{n-1} u_{i,j}x_j}{u_{i,i}} \text{ end}$$

Remark 2.1.9 (Indexing from the end of an array and counting backwards)

To express the above backwards counting in Python, we have to deal with the fact that `range(a, b)` counts upwards and excludes the “end value” b . The first part is easy: the extended form `range(a, b, step)` increments by `step` instead of by one, so that `range(a, b, 1)` is the same as `range(a, b)`, and `range(a, b, -1)` counts down: $a, a-1, \dots, b+1$.

But it still stops just before b , so getting the values from $n-1$ down to 0 requires using $b = -1$, and so the slightly quirky expression `range(n-1, -1, -1)`.

One more bit of Python: for an n -element single-index array v , the sum of its elements $\sum_{i=0}^{n-1} v_i$ is given by `sum(v)`. Thus $\sum_{i=a}^{b-1} v_i$, the sum over a subset of indices $[a, b)$, is given by `sum(v[a:b])`.

And remember that multiplication of Numpy arrays with `*` is pointwise.

The backward substitution algorithm in Python

With all the above Python details, the core code for backward substitution is:

```
x[n-1] = c[n-1]/U[n-1,n-1]
for i in range(n-2, -1, -1):
    x[i] = (c[i] - sum(U[i,i+1:] * x[i+1:])) / U[i,i]
```

Remark 2.1.10

Note that the backward substitution algorithm and its Python coding have a nice mathematical advantage over the row reduction algorithm above: the precise mathematical statement of the algorithm does not need any intermediate quantities distinguished by superscripts $^{(k)}$, and correspondingly, all variables in the code have fixed meanings, rather than changing at each step.

In other words, all uses of the equal sign are mathematically correct as equations!

This can be advantageous in creating algorithms and code that is more understandable and more readily verified to be correct, and is an aspect of the *functional programming* approach. We will soon go part way to that *functional* ideal, by rephrasing Gaussian elimination in a form where all variables have clear, fixed meanings, corresponding to the natural mathematical description of the process: the method of **LU factorization** introduced in *Solving $Ax = b$ with LU factorization, $A = LU$* .

Remark 2.1.11 (Another way to count backwards along an array)

On the other hand, there is an elegant way access array elements “from the top down”. Firstly (or “lastly”) $x[-1]$ is the last element: the same as $x[n-1]$ when $n = \text{len}(x)$, but without needing to know that length n .

More generally, $x[-i]$ is $x[n-i]$.

Thus, one possibly more elegant way to describe backward substitution is to count with an increasing index, the “distance from the bottom”: from $x[n-1]$ which is $x[-1]$ to $x[0]$, which is $x[-n]$. That is, index $-i$ replaces index $n-i$:

```
x[-1] = c[-1]/U[-1,-1]
for i in range(2, n+1):
    x[-i] = (c[-i] - sum(U[-i,1-i:] * x[1-i:])) / U[-i,-i]
```

There is still the quirk of having to “overshoot”, referring to $n+1$ in `range` to get to final index $-n$.

As a final demonstration, we put this second version of the code into a complete working Python function and test it:

```
def backwardsubstitution(U, c, demomode=False):
    """Solve U x = c for b."""
    n = len(c)
    x = np.zeros(n)
    x[-1] = c[-1]/U[-1,-1]
    if demomode: print(f"x_{n} = {x[-1]}")
    for i in range(2, n+1):
        x[-i] = (c[-i] - sum(U[-i,1-i:] * x[1-i:])) / U[-i,-i]
        if demomode: print(f"x_{n-i+1} = {x[-i]}")
    return x
```

which as usual is also available via

```
from numericalMethods import backwardsubstitution
```

```
x = backwardsubstitution(U, c)
print(f"x = {x}")
r = b - A@x
print(f"\nThe residual b - Ax = {r},")
print(f"with maximum norm {max(abs(r)):.3}.")
```

```
x = [ 1.81168831 -1.03246753 -0.45454545]
```

```
The residual b - Ax = [0.0000000e+00 0.0000000e+00 4.4408921e-16],
with maximum norm 4.44e-16.
```

Since one is often just interested in the solution given by the two steps of row reduction and then backward substitution, they can be combined in a single function by composition:

```
def solvelinearsystem(A, b): return backwardsubstitution(*rowreduce(A, b));
```

Remark 2.1.12 (On Python)

The `*` here takes the value to its right (a single tuple with two elements U and c) and “unpacks” it to the two separate variables U and c needed as input to `backwardsubstitution`

```
solveLinearsystem(A, b)
```

```
array([ 1.81168831, -1.03246753, -0.45454545])
```

3.1.10 Two code testing hacks: starting from a known solution, and using randomly generated examples

An often useful strategy in developing and testing code is to create a test case with a known solution; another is to use random numbers to avoid accidentally using a test case that is unusually easy.

Preferred Python style is to have all `import` statements at the top, but since this is the first time we've heard of module `random`, I did not want it to be mentioned mysteriously above.

```
import random
```

```
x_random = empty(len(b)) # An array the same length as b, with no values specified.
# yet
for i in range(len(x)):
    x_random[i] = random.uniform(-1, 1) # gives random real value, from uniform
# distribution in [-1, 1]
print(f"x_random = {x_random}")
```

```
x_random = [0.5291947 0.82976479 0.49678882]
```

Create a right-hand side `b` that automatically makes `x_random` the correct solution:

```
b_random = A @ x_random
```

```
print(f"A =\n{A}")
print(f"\nb_random = {b_random}")
(U, c_random) = rowreduce(A, b_random)
print(f"\nU=\n{U}")
print(f"\nResidual c_random - U@x_random = {c_random - U@x_random}")
x_computed = backwardsubstitution(U, c_random)
print(f"\nx_computed = {x_computed}")
print(f"\nResidual b_random - A@x_computed = {b_random - A@x_computed}")
print(f"\nBackward error |b_random - A@x_computed| = {max(abs(b_random - A@x_
# computed))}")
print(f"\nError x_random - x_computed = {x_random - x_computed}")
print(f"\nAbsolute error |x_random - x_computed| = {max(abs(x_random - x_computed))}")
```

```
A =
[[ 4.  2.  7.]
 [ 3.  5. -6.]
 [ 1. -3.  2.]]

b_random = [ 7.2538301  2.7556751 -0.96652202]

U=
[[ 4.    2.    7. ]
```

(continues on next page)

(continued from previous page)

```
[ 0.    3.5  -11.25]
[ 0.    0.   -11.   ]]

Residual c_random - U@x_random = [0. 0. 0.]

x_computed = [0.5291947  0.82976479 0.49678882]

Residual b_random - A@x_computed = [0. 0. 0.]

Backward error |b_random - A@x_computed| = 0.0

Error x_random - x_computed = [0. 0. 0.]

Absolute error |x_random - x_computed| = 0.0
```

3.1.11 What can go wrong? Three examples

Example 2.1.1 (An obvious division by zero problem)

Consider the system of two equations

$$\begin{aligned}x_2 &= 1 \\x_1 + x_2 &= 2\end{aligned}$$

It is easy to see that this has the solution $x_1 = x_2 = 1$; in fact it is already in “reduced form”. However when put into matrix form

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

the above algorithm fails, because the first *pivot element* a_{11} is zero:

```
A1 = array([[0., 1.], [1., 1.]])
b1 = array([1., 1.])
(U1, c1) = rowreduce(A1, b1)
print(f"U1 = \n{U1}")
print(f"c1 = {c1}")
x1 = backwardsubstitution(U1, c1)
print(f"x1 = {x1}")
```

```
U1 =
[[ 0.    1.]
 [ 0. -inf]]
c1 = [ 1. -inf]
x1 = [nan nan]
```

```
/var/folders/zk/qv7t2p8x33ldzh_sg8b854lr0000gn/T/ipykernel_18954/2577478211.py:15:␣
␣RuntimeWarning: divide by zero encountered in true_divide
␣L[k+1:,k] = U[k+1:n,k] / U[k,k] # Beware the case where U[k,k] is 0
```

(continues on next page)

(continued from previous page)

```
/Users/brenton/Library/CloudStorage/OneDrive-CollegeofCharleston/numerical-methods-
and-analysis/numerical-methods-and-analysis-python/docs/numerical_methods.
py:355: RuntimeWarning: invalid value encountered in double_scalars
x[-1] = c[-1]/U[-1,-1]
```

Remark 2.1.13 (On Python “Infinity” and “Not a Number”)

- `inf`, meaning “infinity”, is a special value given as the result of operations like division by zero. Surprisingly, it can have a sign! (This is available in Python from package Numpy as `numpy.inf`)
- `nan`, meaning “not a number”, is a special value given as the result of calculations like $0/0$. (This is available in Python from package Numpy as `numpy.nan`)

Example 2.1.2 (A less obvious division by zero problem)

Next consider this system

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 2 \\ 1 & 2 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \\ 5 \end{bmatrix}$$

The solution is $x_1 = x_2 = x_3 = 1$, and this time none of the diagonal elements is zero, so it is not so obvious that a division by zero problem will occur, but:

```
A2 = array([[1., 1., 1.], [1., 1., 2.], [1., 2., 2.]])
b2 = array([3., 4., 5.])
```

```
(U2, c2) = rowreduce(A2, b2)
print(f"U2 = \n{U2}")
print(f"c2 = {c2}")
x2 = backwardsubstitution(U2, c2)
print(f"x2 = {x2}")
```

```
U2 =
[[ 1.  1.  1.]
 [ 0.  0.  1.]
 [ 0.  0. -inf]]
c2 = [ 3.  1. -inf]
x2 = [nan nan nan]
```

```
/var/folders/zk/qv7t2p8x33ldzh_sg8b854lr0000gn/T/ipykernel_18954/2577478211.py:15:
RuntimeWarning: divide by zero encountered in true_divide
L[k+1:,k] = U[k+1:n,k] / U[k,k] # Beware the case where U[k,k] is 0
```

What happens here is that the first stage subtracts the first row from each of the others ...

```
A2[1,:] -= A2[0,:]
b2[1] -= b2[0]
A2[2,:] -= A2[0,:]
b2[2] -= b2[0]
```

... and the new matrix has the same problem as above at the next stage:

```
print(f"Now A2 is \n{A2}")
print(f"and b2 is {b2}")
```

```
Now A2 is
[[1. 1. 1.]
 [0. 0. 1.]
 [0. 1. 1.]]
and b2 is [3. 1. 2.]
```

Thus, the second and third equations are

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

with the same problem as in [Example 2.1.1](#).

Example 2.1.3 (Problems caused by inexact arithmetic: “divison by almost zero”)

The equations

$$\begin{bmatrix} 1 & 10^{16} \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 + 10^{16} \\ 2 \end{bmatrix}$$

again have the solution $x_1 = x_2 = 1$, and the only division that happens in the above algorithm for row reduction is by that pivot element $a_{11} = 1, \neq 0$, so with exact arithmetic, all would be well. But:

```
A3 = array([[1., 1e16], [1. , 1.]])
b3 = array([1. + 1e16, 2.])
print(f"A3 = \n{A3}")
print(f"b3 = {b3}")
```

```
A3 =
[[1.e+00 1.e+16]
 [1.e+00 1.e+00]]
b3 = [1.e+16 2.e+00]
```

```
(U3, c3) = rowreduce(A3, b3)
print(f"U3 = \n{U3}")
print(f"c3 = {c3}")
x3 = backwardsubstitution(U3, c3)
print(f"x3 = {x3}")
```

```
U3 =
[[ 1.e+00  1.e+16]
 [ 0.e+00 -1.e+16]]
c3 = [ 1.e+16 -1.e+16]
x3 = [2. 1.]
```

This gets $x_2 = 1$ correct, but x_1 is completely wrong!

One hint is that b_1 , which should be $1 + 10^{16} = 10000000000000001$, is instead just given as 10^{16} .

On the other hand, all is well with less large values, like 10^{15} :

```
A3a = array([[1., 1e15], [1. , 1.]])
b3a = array([1. + 1e15, 2.])
print(f"A3a = \n{A3a}")
print(f"b3a = {b3a}")
```

```
A3a =
[[1.e+00 1.e+15]
 [1.e+00 1.e+00]]
b3a = [1.e+15 2.e+00]
```

```
(U3a, c3a) = rowreduce(A3a, b3a)
print(f"U3a = \n{U3a}")
print(f"c3a = {c3a}")
x3a = backwardsubstitution(U3a, c3a)
print(f"x3a = {x3a}")
```

```
U3a =
[[ 1.e+00  1.e+15]
 [ 0.e+00 -1.e+15]]
c3a = [ 1.e+15 -1.e+15]
x3a = [1. 1.]
```

Example 2.1.4 (Avoiding small denominators)

The first equation in [Example 2.1.3](#) can be divided by 10^{16} to get an equivalent system with the same problem:

$$\begin{bmatrix} 10^{-16} & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 + 10^{-16} \\ 2 \end{bmatrix}$$

Now the problem is more obvious: this system differs from the system in [Example 2.1.1](#) just by a tiny change of 10^{-16} in that pivot elements a_{11} , and the problem is *division by a value very close to zero*.

```
A4 = array([[1e-16, 1.], [1. , 1.]])
b4 = array([1. + 1e-16, 2.])
print(f"A4 = \n{A4}")
print(f"b4 = {b4}")
```

```
A4 =
[[1.e-16 1.e+00]
 [1.e+00 1.e+00]]
b4 = [1. 2.]
```

```
(U4, c4) = rowreduce(A4, b4)
print(f"U4 = \n{U4}")
print(f"c4 = {c4}")
x4 = backwardsubstitution(U4, c4)
print(f"x4 = {x4}")
```

```
U4 =
[[ 1.e-16  1.e+00]
```

(continues on next page)

(continued from previous page)

```
[ 0.e+00 -1.e+16]]
c4 = [ 1.e+00 -1.e+16]
x4 = [2.22044605 1.      ]
```

One might think that there is no such small denominator in [Example 2.1.3](#), but what counts for being “small” is magnitude relative to other values — 1 is very small compared to 10^{16} .

To understand these problems more (and how to avoid them) we will explore [Machine Numbers, Rounding Error and Error Propagation](#) in the next section.

3.1.12 When naive Gaussian elimination is safe: diagonal dominance

There are several important cases when we can guarantee that these problems do not occur. One obvious case is when the matrix A is diagonal and non-singular (so with all non-zero elements); then it is already row-reduced and with all denominators in backward substitution being non-zero.

A useful measure of being “close to diagonal” is *diagonal dominance*:

Definition 2.1.1 (Strict Diagonal Dominance)

A matrix A is **row-wise strictly diagonally dominant**, sometimes abbreviated as just **strictly diagonally dominant** or **SDD**, if

$$\sum_{1 \leq k \leq n, k \neq i} |a_{i,k}| < |a_{i,i}|$$

Loosely, each main diagonal “dominates” in size over all other elements in its row.

Definition 2.1.2 (Column-wise Strict Diagonal Dominance)

If instead

$$\sum_{1 \leq k \leq n, k \neq i} |a_{k,i}| < |a_{i,i}|$$

(so that each main diagonal element “dominates its column”) the matrix is called **column-wise strictly diagonally dominant**.

Note that this is the same as saying that the transpose A^T is SDD.

Aside: If only the corresponding non-strict inequality holds, the matrix is called *diagonally dominant*.

Theorem 2.1.1

For any strictly diagonally dominant matrix A , each of the intermediate matrices $A^{(k)}$ given by the naive Gaussian elimination algorithm is also strictly diagonally dominant, and so the final upper triangular matrix U is. In particular, all the diagonal elements $a_{i,i}^{(k)}$ and $u_{i,i}$ are non-zero, so no division by zero occurs in any of these algorithms, including the backward substitution solving for x in $Ux = c$.

The corresponding fact also true if the matrix is column-wise strictly diagonally dominant: that property is also preserved at each stage in naive Gaussian elimination.

Thus in each case the diagonal elements — the elements divided by in both row reduction and backward substitution — are in some sense safely away from zero. We will have more to say about this in the sections on *pivoting* and *LU factorization*

For a column-wise SDD matrix, more is true: at stage k , the diagonal dominance says that the pivot element on the diagonal, $a_{k,k}^{(k-1)}$, is larger (in magnitude) than any of the elements $a_{i,k}^{(k-1)}$ below it, so the multipliers $l_{i,k}$ have

$$|l_{i,k}| = |a_{i,k}^{(k-1)} / a_{k,k}^{(k-1)}| < 1.$$

As we will see when we look at the effects of rounding error in the sections on *Machine Numbers*, *Rounding Error and Error Propagation* and *Error bounds for linear algebra* keeping intermediate values small is generally good for accuracy, so this is a nice feature.

Remark 2.1.14 (Positive definite matrices)

Another class of matrices for which naive Gaussian elimination works well is **positive definite matrices** which arise in any important situations; that property is in some sense more natural than diagonal dominance. However that topic will be left for later.

3.2 Machine Numbers, Rounding Error and Error Propagation

References:

- Sections 0.3 *Floating Point Representation of Real Numbers* and 0.4 *Loss of Significance* in [Sauer, 2022].
- Section 1.2 *Round-off Errors and Computer Arithmetic* of [Burden et al., 2016].
- Sections 1.3 and 1.4 of [Chenney and Kincaid, 2012].

3.2.1 Overview

The naive Gaussian elimination algorithm seen in the section *Row Reduction/Gaussian Elimination* has several related weaknesses which make it less robust and flexible than desired.

Most obviously, it can fail even when the equations are solvable, due to its naive insistence on always working from the top down. For example, as seen in *Example 2.1.1* of that section, it fails with the system

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

because the formula for the first multiplier $l_{2,1} = a_{2,1}/a_{1,1}$ gives $1/0$.

Yet the equations are easily solvable, indeed with no reduction needed: the first equation just says $x_2 = 1$, and then the second gives $x_1 = 2 - x_2 = 1$.

All one has to do here to avoid this problem is change the order of the equations. Indeed we will see that such reordering is *all that one ever needs to do*, so long as the original equation has a unique solution.

However, to develop a good strategy, we will also take account of errors introduced by rounding in computer arithmetic, so that is our next topic.

3.2.2 Robustness and well-posedness

The above claim raises the concept of **robustness** and the importance of both existence and uniqueness of solutions.

Definition 2.2.1 (Well-Posed)

A problem is **well-posed** if it is stated in a way that it has a unique solution. (Note that this might include asking for the set of all solutions, such as asking for all roots of a polynomial.)

For example, the problem of finding the root of a continuous, monotonic function $f : [a, b] \rightarrow \mathbb{R}$ with $f(a)$ and $f(b)$ of opposite sign is well-posed. Note the care taken with details to ensure both existence and uniqueness of the solution.

Definition 2.2.2 (Robust)

An algorithm for solving a class of problems is **robust** if it is guaranteed to solve any well-posed problem in the class.

For example, the bisection method is robust for the above class of problems. On the other hand, Newton's method is not, and if we dropped the specification of monotonicity (so allowing multiple solutions) then the bisection method in its current form would not be robust: it would fail whenever there is more than one solution in the interval $[a, b]$.

3.2.3 Rounding error and accuracy problems due to “loss of significance”

There is a second slightly less obvious problem with the naive algorithm for Gaussian elimination, closely related to the first. As soon as the algorithm is implemented using any rounding in the arithmetic (rather than, say, working with exact arithmetic on rational numbers) division by values that are very close to zero can lead to very large intermediate values, which thus have very few correct decimals (correct bits); that is, very large absolute errors. These large errors can then propagate, leading to low accuracy in the final results, as seen in [Example 2.1.2](#) and [Example 2.1.4](#) of *Row Reduction/Gaussian Elimination*.

This is the hazard of *loss of significance*, discussed in Section 0.4 of [Sauer, 2022] and Section 1.4 of [Chenney and Kincaid, 2012].

So it is time to take Step 2 of the strategy described in the previous notes:

2. Refine to get a more robust algorithm

1. Identify cases that can lead to failure due to division by zero and such, and revise to avoid them.
2. Avoid inaccuracy due to problems like severe rounding error. One rule of thumb is that anywhere that a zero value is a fatal flaw (in particular, division by zero), a very small value is also a hazard when rounding error is present. So avoid very small denominators. ...

3.2.4 The essentials of machine numbers and rounding in machine arithmetic

As a very quick summary, standard computer arithmetic handles real numbers using *binary machine numbers* with p significant bits, and rounding off of other numbers to such *machine numbers* introduces a relative error of at most 2^{-p} . The current dominant choice for machine numbers and arithmetic is IEEE-64, using 64 bits in total and with $p = 53$ significant bits, so that $1/2^p \approx 1.11 \cdot 10^{-16}$, giving about fifteen significant digits. (The other bits are used for an exponent and the sign.)

(Note: in the above, I ignore the extra problems with real numbers whose magnitude is too large or too small to be represented: *underflow* and *overflow*. Since the allowable range of magnitudes is from $2^{-1022} \approx 2.2 \cdot 10^{-308}$ to $2^{1024} \approx 1.8 \cdot 10^{308}$, this is rarely a problem in practice.)

With other systems of binary machine numbers (like older 32-bit versions, or higher precision options like 128 bits) the significant differences are mostly encapsulated in that one number, the **machine unit**, $u = 1/2^p$.

Binary floating point machine numbers

The basic representation is a binary version of the familiar *scientific* or *decimal floating point* notation: in place of the form $\pm d_0.d_1d_2 \dots d_{p-1} \times 10^e$ where the *fractional part* or *mantissa* is $f = d_0.d_1d_2 \dots d_{p-1} = d_0 + \frac{d_1}{10} + \dots + \frac{d_{p-1}}{10^{p-1}}$.

Binary floating point machine numbers with p significant bits can be described as

$$\pm(b_0.b_1b_2 \dots b_{p-1})_2 \times 2^e = \pm \left(b_0 + \frac{b_1}{2} + \frac{b_2}{2^2} + \dots + \frac{b_{p-1}}{2^{p-1}} \right) \times 2^e$$

Just as decimal floating point numbers are typically written with the exponent chosen to have non-zero leading digit $d_0 \neq 0$, **normalized** binary floating point machine numbers have exponent e chosen so that $b_0 \neq 0$. Thus in fact $b_0 = 1$ — and so it need not be stored; only $p - 1$ bits are needed to stored for the mantissa.

Worst case rounding error

It turns out that the relative errors are determined solely by the number of significant bits in the mantissa, regardless of the exponent, so we look at that part first.

Rounding error in the mantissa, $(1.b_1b_2 \dots b_{p-1})_2$

The spacing of consecutive mantissa values $(1.b_1b_2 \dots b_{p-1})_2$ is one in the last bit, or 2^{1-p} . Thus rounding of any intermediate value x to the nearest number of this form introduces an absolute error of at most half of this: $u = 2^{-p}$, which is called the *machine unit*

How large can the *relative* error be? It is largest for the smallest possible denominator, which is $(1.00 \dots 0)_2 = 1$, so the relative error due to rounding is also at most 2^{-p} .

Rounding error in general, for $\pm(1.b_1b_2 \dots b_{p-1})_2 \cdot 2^e$.

The sign has no effect on the absolute error, and the exponent changes the spacing of consecutive machine numbers by a factor of 2^e . This scales the maximum possible absolute error to 2^{e-p} , but in the relative error calculation, the smallest possible denominator is also scaled up to 2^e , so the largest possible relative error is again the machine unit, $u = 2^{-p}$.

One way to describe the machine unit u (sometimes called *machine epsilon*) is to note that the next number above 1 is $1 + 2^{1-p} = 1 + 2u$. Thus $1 + u$ is at the threshold between rounding down to 1 and rounding up to a higher value.

IEEE 64-bit numbers: more details and some experiments

For completely full details, you could read about the [IEEE 754 Standard for Floating-Point Arithmetic](#) and specifically the [binary64](#) case. (For historical reasons, this is known as “Double-precision floating-point format”, from the era when computers were typically used 32-bit words, so 64-bit numbers needed two words.)

In the standard IEEE-64 number system:

- 64 bit words are used to store real numbers (a.k.a. *floating point* numbers, sometimes called *floats*.)
- There are $p = 53$ bits of precision, so that 52 bits are used to store the mantissa (fractional part).
- The sign is stored with one bit s : effectively a factor of $(-1)^s$, so $s = 0$ for positive, $s = 1$ for negative.
- The remaining 11 bits are use for the exponent, which allows for $2^{11} = 2048$ possibilities; these are chosen in the range $-1023 \leq e \leq 1024$.
- However, so far, this does not allow for the value zero! This is handled by giving a special meaning for the smallest exponent $e = -1023$, so the smallest exponent for *normalized* numbers is $e = -1022$.
- At the other extreme, the largest exponent $e = 1024$ is used to encode “infinite” numbers, which can arise when a calculation gives a value too large to represent. (Python displays these as `inf` and `-inf`). This exponent is also used to encode “Not a Number”, for situations like trying to divide zero by zero or multiply zero by `inf`.
- Thus, the exponential factors for normlaized numbers are in the range $2^{-1022} \approx 2 \times 10^{-308}$ to $2^{1023} \approx 9 \times 10^{307}$. Since the mantissa ranges from 1 to just under 2, the range of magnitudes of normalized real numbers is thus from $2^{-1022} \approx 2 \times 10^{-308}$ to just under $2^{1024} \approx 1.8 \times 10^{308}$.

Some computational experiments:

```
p = 53
u = 2**(-p)
print(f"For IEEE-64 arithmetic, there are {p} bitd of precision and the machine unit is u={u}, ")
print(f"and the next numbers above 1 are 1+2u = {1+2*u}, 1+4u = {1+4*u} and so on.")
for factor in [3, 2, 1.000000000001, 1]:
    onePlusSmall = 1 + factor * u
    print(f"1 + {factor}u rounds to {onePlusSmall}")
    difference = onePlusSmall - 1
    print(f"\tThis is more than 1 by {difference:.4}, which is {difference/u} times u")
```

```
For IEEE-64 arithmetic, there are 53 bitd of precision and the machine unit is u=1.
1102230246251565e-16,
and the next numbers above 1 are 1+2u = 1.0000000000000002, 1+4u = 1.
00000000000000004 and so on.
1 + 3u rounds to 1.0000000000000004
    This is more than 1 by 4.441e-16, which is 4.0 times u
1 + 2u rounds to 1.0000000000000002
    This is more than 1 by 2.22e-16, which is 2.0 times u
1 + 1.000000000001u rounds to 1.0000000000000002
    This is more than 1 by 2.22e-16, which is 2.0 times u
1 + 1u rounds to 1.0
    This is more than 1 by 0.0, which is 0.0 times u
```

```
print("On the other side, the spacing is halved:")
print(f"the next numbers below 1 are 1-u = {1-u}, 1-2u = {1-2*u} and so on.")
for factor in [2, 1, 1.000000000001/2, 1/2]:
```

(continues on next page)

(continued from previous page)

```
oneMinusSmall = 1 - factor * u
print(f"1 - {factor}u rounds to {oneMinusSmall}")
difference = 1 - oneMinusSmall
print(f"\tThis is less than 1 by {difference:.4}, which is {difference/u} times u
↪")
```

On the other side, the spacing is halved:
the next numbers below 1 are $1-u = 0.9999999999999999$, $1-2u = 0.9999999999999998$,
↪and so on.

```
1 - 2u rounds to 0.9999999999999998
    This is less than 1 by 2.22e-16, which is 2.0 times u
1 - 1u rounds to 0.9999999999999999
    This is less than 1 by 1.11e-16, which is 1.0 times u
1 - 0.5000000000000005u rounds to 0.9999999999999999
    This is less than 1 by 1.11e-16, which is 1.0 times u
1 - 0.5u rounds to 1.0
    This is less than 1 by 0.0, which is 0.0 times u
```

Next, look at the extremes of very small and very large magnitudes:

```
print(f"The smallest normalized positive number is {2**(-1022)=}")
print(f"The largest mantissa is binary (1.1111...) with 53 ones: {2 - 2**(-52)=:0.20}.
↪..")
print(f"The largest normalized number is {(2 - 2**(-52))*2.**1023=}")
print(f"If instead we round that mantissa up to 2 and try again, we get {2*2.**1023=}
↪")
```

```
The smallest normalized positive number is 2**(-1022)=2.2250738585072014e-308
The largest mantissa is binary (1.1111...) with 53 ones: 2 - 2**(-52)=1.
↪999999999999999778...
The largest normalized number is (2 - 2**(-52))*2.**1023=1.7976931348623157e+308
If instead we round that mantissa up to 2 and try again, we get 2*2.**1023=inf
```

What happens if we compute positive numbers smaller than that smallest normalized positive number 2^{-1022} ?

```
for S in [0, 1, 2, 52, 53]:
    exponent = -1022-S
    print(f" 2**(-1022-{S}) = 2**({exponent}) = {2**exponent}")
```

```
2**(-1022-0) = 2**(-1022) = 2.2250738585072014e-308
2**(-1022-1) = 2**(-1023) = 1.1125369292536007e-308
2**(-1022-2) = 2**(-1024) = 5.562684646268003e-309
2**(-1022-52) = 2**(-1074) = 5e-324
2**(-1022-53) = 2**(-1075) = 0.0
```

These extremely small values are called *denormalized numbers*. Numbers with exponent $2^{-1022-S}$ have fractional part with S leading zeros, so only $p - S$ significant bits. So when the shift S reaches $p = 53$, there are no significant bits left, and the value is truly zero.

3.2.5 Propagation of error in arithmetic

The only errors in the results of Gaussian elimination come from errors in the initial data (a_{ij} and b_i) and from when the results of subsequent arithmetic operations are rounded to machine numbers. Here, we consider how errors from either source are propagated — and perhaps amplified — in subsequent arithmetic operations and rounding.

In summary:

- When *multiplying* two numbers, the relative error in the sum is no worse than slightly more than the sum of the relative errors in the numbers multiplied. (the be pedantic, it is at most the sum of those relative plus their product, but that last piece is typically far smaller.)
- When *dividing* two numbers, the relative error in the quotient is again no worse than slightly more than the sum of the relative errors in the numbers divided.
- When *adding* two **positive** numbers, the relative error is no more that the larger of the relative errors in the numbers added, and the absolute error in the sum is no larger than the sum of the absolute errors.
- When *subtracting* two **positive** numbers, the absolute error is again no larger than the sum of the absolute errors in the numbers subtracted, **but the relative error can get far worse!**

Due to the differences between the last two cases, this discussion of error propagation will use “addition” to refer only to adding numbers of the same sign, and “subtraction” when subtracting numbers of the same sign.

More generally, we can think of rewriting the operation in terms of a pair of numbers that are both positive, and assume WLOG that all input values are positive numbers.

Notation: $x_a = x(1 + \delta_x)$ for errors and $fl(x)$ for rounding

Two notations will be useful.

Firstly, for any approximation x_a of a real value x , let $\delta_x = \frac{x_a - x}{x}$, so that $x_a = x(1 + \delta_x)$.

Thus, $|\delta_x|$ is the relative error, and δ_x helps keep track of the sign of the error.

Also, introduce the function $fl(x)$ which does rounding to the nearest machine number. For the case of the approximation $x_a = fl(x)$ to x given by rounding, the above results on machine numbers then give the bound $|\delta_x| \leq u = 2^{-p}$.

Propagation of error in products

Let x and y be exact quantities, and $x_a = x(1 + \delta_x)$, $y_a = y(1 + \delta_y)$ be approximations. The approximate product $(xy)_a = x_a y_a = x(1 + \delta_x)y(1 + \delta_y)$ has error

$$x(1 + \delta_x)y(1 + \delta_y) - xy = xy(1 + \delta_x + \delta_y + \delta_x\delta_y) - xy = xy(1 + \delta_{xy})$$

Thus the relative error in the product is

$$|\delta_{xy}| \leq |\delta_x| + |\delta_y| + |\delta_x||\delta_y|$$

For example if the initial errors are due only to rounding, $|\delta_x| \leq u = 2^{-p}$ and similarly for $|\delta_y|$, so the relative error in $x_a y_a$ is at most $2u + u^2 = 2^{1-p} + 2^{-2p}$. In this and most situations, that final “product of errors” term $\delta_x \delta_y$ is far smaller than the first two, giving to a very good approximation

$$|\delta_{xy}| \leq |\delta_x| + |\delta_y|$$

This is the above stated “sum of relative errors” result.

When the “input errors” in x_a and y_a come just from rounding to machine numbers, so that each has p bits of precision, $|\delta_x|, |\delta_y| \leq 1/2^p$ and the error bound for the product is $1/2^{p-1}$: at most one bit of precision is lost.

Exercise 1

Derive the corresponding result for quotients.

Propagation or error in sums (of positive numbers)

With x_a and y_a as above (and positive), the approximate sum $x_a + y_a$ has error

$$(x_a + y_a) - (x + y) = (x_a - x) + (y_a - y)$$

so the absolute error is bounded by $|x_a - x| + |y_a - y|$; the sum of the absolute errors.

For the relative errors, express this error as

$$(x_a + y_a) - (x + y) = (x(1 + \delta_x) + y(1 + \delta_y)) - (x + y) = x\delta_x + y\delta_y$$

Let δ be the maximum of the relative errors, $\delta = \max(|\delta_x|, |\delta_y|)$; then the absolute error is at most $(|x| + |y|)\delta = (x + y)\delta$ and so the relative error is at most

$$\frac{(x + y)\delta}{|x + y|} = \delta = \max(|\delta_x|, |\delta_y|)$$

That is, *the relative error in the sum is at most the sum of the relative errors*, again as advertised above.

When the “input errors” in x_a and y_a come just from rounding to machine numbers, the error bound for the sum is no larger: no precision is lost! Thus, if you take any collection of non-negative numbers, round them to machine numbers so that each has relative error at most u , then the sum of these rounded values also has relative error at most u .

Propagation or error in differences (of positive numbers): loss of significance/loss of precision

The above calculation for the absolute error works fine regardless of the signs of the numbers, so the absolute error of a difference is still bounded by the sum of the absolute errors:

$$|(x_a - y_a) - (x - y)| \leq |x_a - x| + |y_a - y|$$

But for subtraction, the denominator in the relative error formulas can be far smaller. WLOG let $x > y > 0$. The relative error bound is

$$\frac{|(x_a - y_a) - (x - y)|}{|x - y|} \leq \frac{x\delta_x + y\delta_y}{x - y}$$

Clearly if $x - y$ is far smaller than x or y , this can be far larger than the “input” relative errors $|\delta_x|$ and $|\delta_y|$.

The extreme case is where the values x and y round to the same value, so that $x_a - y_a = 0$, and the relative error is 1: “100% error”, a case of *catastrophic cancellation*.

Exercise 2

Let us move slightly away from the worst case scenario where the difference is exactly zero to one where it is close to zero; this will illustrate the idea mentioned earlier that *wherever a zero value is a problem in exact arithmetic, a very small value can be a problem in approximate arithmetic*.

For $x = 8.024$ and $y = 8.006$,

- Round each to three significant figures, giving x_a and y_a .
- Compute the absolute errors in each of these approximations, and in their difference as an approximation of $x - y$.
- Compute the relative errors in each of these three approximations.

Then look at rounding to only two significant digits!

Upper and lower bounds on the relative error in subtraction

The problem is worst when x and y are close in relative terms, in that y/x is close to 1. In the case of the errors in x_a and y_a coming just from rounding to machine ennumbers, we have:

Theorem 2.2.1 (Loss of Precision)

Consider $x > y > 0$ that are close in that they agree in at least q significant bits and at most r significant bits:

$$\frac{1}{2^r} < 1 - \frac{y}{x} < \frac{1}{2^q}.$$

Then when rounded to machine numbers which are then subtracted, the relative error in that approximation of the difference is greater than that due to rounding by a factor of between 2^q and 2^r .

That is, subtraction loses between q and r significant bits of precision.

Exercise 3

(a) Illustrate why computing the roots of the quadratic equation $ax^2 + bx + c = 0$ with the standard formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

can sometimes give poor accuracy when evaluated using machine arithmetic such as IEEE-64 floating-point arithmetic. This is not always a problem, so identify specifically the situations when this could occur, in terms of a condition on the coefficients a , b , and c . (It is sufficient to consider real value of the coefficients. Also as an aside, there is no loss of precision problem when the roots are non-real, so you only need consider quadratics with real roots.)

(b) Then describe a careful procedure for always getting accurate answers. State the procedure first with words and mathematical formulas, and then express it in pseudo-code.

Example 2.2.1 (Errors when approximating derivatives)

To deal with differential equations, we will need to approximate the derivative of function from just some values of the function itself. The simplest approach is suggested by the definition of the derivative

$$Df(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

by using

$$Df(x) \approx D_h f(x) := \frac{f(x+h) - f(x)}{h}$$

with a small value of h — but this inherently involves the difference of almost equal quantities, and so loss of significance.

Taylor's theorem give an error bound if we assume exact arithmetic — worse for larger h . Then the above results give a measure of rounding error effects — worse for smaller h .

This leads to the need to balance these error sources, to find an optimal choice for h and the corresponding error bound.

Denote the error in approximately calculating $D_h f(x)$ with machine arithmetic as $\tilde{D}_h f(x)$.

The error in this as an approximating of the exact derivative is

$$E = \tilde{D}_h f(x) - Df(x) = (\tilde{D}_h f(x) - D_h f(x)) + (D_h f(x) - Df(x))$$

which we will consider as the sum of two pieces, $E = E_A + E_D$ where

$$E_A = \tilde{D}_h f(x) - D_h f(x)$$

is the error due to machine Arithmetic in evaluation of the difference quotient $D_h f(x)$, and

$$E_D = D_h f(x) - Df(x)$$

is the error in this difference quotient as an approximation of the exact derivative $Df(x) = f'(x)$. This error is sometimes called the **discretization error** because it arises when we replace the derivative by a discrete algebraic calculation.

Bounding the Arithmetic error E_A

The first source of error is rounding of $f(x)$ to a machine number; as seen above, this gives $f(x)(1 + \delta_1)$, with $|\delta_1| \leq u$, so absolute error $|f(x)\delta_1| \leq |f(x)|u$.

Similarly, $f(x + h)$ is rounded to $f(x + h)(1 + \delta_2)$, absolute error at most $|f(x + h)|u$.

Since we are interested in fairly small values of h (to keep E_D under control), we can assume that $|f(x + h)| \approx |f(x)|$, so this second absolute error is also very close to $|f(x)|u$.

Then the absolute error in the difference in the numerator of $D_h f(x)$ is at most $2|f(x)|u$ (or only a tiny bit greater).

Next the division. We can assume that h is an exact machine number, for example by choosing h to be a power of two, so that division by h simply shifts the power of two in the exponent part of the machine number. This has no effect on the relative error, but scales the absolute error by the factor $1/h$ by which one is multiplying: the absolute error is now bounded by

$$|E_A| \leq \frac{2|f(x)|u}{h}$$

This is a critical step: the difference has a small absolute error, which conceals a large relative error due to the difference being small; now the absolute error gets amplified greatly when h is small.

Bounding the Discretization error E_D

As seen in *Taylor's Theorem and the Accuracy of Linearization* — for the basic case of linearization — we have

$$f(x + h) - f(x) = Df(x)h + \frac{f''(c_x)}{2}h^2$$

so

$$E_D = \frac{f(x + h) - f(x)}{h} - Df(x) = \frac{f''(c_x)}{2}h$$

and with $M_2 = \max |f''|$,

$$|E_D| \leq \frac{M_2}{2}h$$

Bounding the total absolute error, and minimizing it

The above results combine to give an upper limit on how bad the total error can be:

$$|E| \leq |E_A| + |E_D| \leq \frac{2|f(x)|u}{h} + \frac{M_2}{2}h$$

As anticipated, the errors go in opposite directions: decreasing h to reduce E_D makes E_A worse, and vice versa. Thus we can expect that there is a “goldilocks” value of h — neither too small nor too big — that gives the best overall bound on the total error.

To do this, let's clean up the notation: let

$$A = 2|f(x)|u, \quad D = \frac{M_2}{2},$$

so that the error bound for a given value of h is

$$E(h) = \frac{A}{h} + Dh$$

This can be minimized with a little calculus:

$$\frac{dE(h)}{dh} = -\frac{A}{h^2} + D$$

which is zero only for the unique critical point

$$h = h^* = \sqrt{\frac{A}{D}} = \sqrt{\frac{2|f(x)|u}{M_2/2}} = 2\sqrt{\frac{|f(x)|}{M_2}}\sqrt{u} = K\sqrt{u}$$

using the short-hand $K = 2\sqrt{\frac{|f(x)|}{M_2}}$.

This is easily verified to give the global minimum of $E(h)$; thus, the best error bound we can get is for this value of h :

$$E \leq E^* := E(h^*) = \frac{2|f(x)|u}{K\sqrt{u}} + \frac{M_2}{2}K\sqrt{u} = \left(\frac{2|f(x)|}{K} + K\frac{M_2}{2}\right)\sqrt{u}$$

Conclusions from this example

In practical cases, we do not know the constant K or the coefficient of $\sqrt{u}$ in parentheses — but that does not matter much!

The most important — and somewhat disappointing — observation here is that both the optimal size of h and the resulting error bound is roughly proportional to the square root of the machine unit u . For example with p bits of precision, $u = 2^{-p}$, the best error is of the order of $2^{-p/2}$, or about $p/2$ significant bits: at best we can hope for about half as many significant bits as our machine arithmetic gives.

In decimal terms: with IEEE-64 arithmetic $u = 2^{-53} \approx 10^{-16}$, so giving about sixteen significant digits, and $\sqrt{u} \approx 10^{-8}$, so $\tilde{D}_h f(x)$ can only be expected to give about half as many; eight significant digits.

This is a first indication of why machine arithmetic sometimes needs to be so precise — more precise than any physical measurement by a factor of well over a thousand.

It also shows that when we get to computing derivatives and solving differential equations, we will often need to do a better job of approximating derivatives!

3.3 Partial Pivoting

References:

- Section 2.4.1 *Partial Pivoting* of [Sauer, 2022].
- Section 6.2 *Pivoting Strategies* of [Burden *et al.*, 2016].
- Section 7.1 of [Chenney and Kincaid, 2012].

Remark 2.3.1

Some references describe the method of *scaled* partial pivoting, but here we present instead a version without the “scaling”, because not only is it simpler, but modern research shows that it is essentially always as good, once the problem is set up in a “sane” way.

3.3.1 Introduction

The basic row reduction method can fail due to division by zero (and to have very large rounding errors when a denominator is extremely close to zero). A more robust modification is to swap the order of the equations to avoid these problems: *partial pivoting*. Here we look at a particularly robust version of this strategy, *Maximal Element Partial Pivoting*.

```
# As in recent sections, we import some items from modules individually, so they can
    be used by "first name only".
from numpy import array
```

3.3.2 What can go wrong with Naive Gaussian Elimination?

We have noted two problems with the naive algorithm for Gaussian elimination: total failure due to the division by zero, and loss of precision due to dividing by very small values — or more precisely calculations that lead to intermediate values far larger than the final results. The culprits in all cases are the same: the denominators are first the *pivot elements* $a_{k,k}^{(k-1)}$ in evaluation of $l_{i,k}$ during row reduction and then the $u_{k,k}$ in back substitution. Further, those $a_{k,k}^{(k-1)}$ are the final updated values at indices (k, k) , so are the same as $u_{k,k}$. Thus it is exactly these *main diagonal elements* that we must deal with.

3.3.3 The basic fix: Partial Pivoting

The basic strategy is that at step k , we can swap equation k with any equation i , $i > k$. Note that this involves swapping those rows of array A and also those elements of the array b for the right-hand side: $b_k \leftrightarrow b_i$.

This approach of swapping equations (swapping rows in arrays A and b) is called **pivoting**, or more specifically **partial pivoting**, to distinguish from the more elaborate strategy where columns of A are also reordered (which is equivalent to reordering the unknowns in the equations). The row that is swapped with row k is sometimes called the **pivot row**, and the new denominator is the corresponding **pivot element**.

This approach is robust so long as one is using exact arithmetic: it works for any well-posed system because so long as the $Ax = b$ has a unique solution — so that the original matrix A is non-singular — at least one of the $a_{i,k}^{(k-1)}$, $i \geq k$ will be non-zero, and thus the swap will give a new element in position (k, k) that is non-zero. (I will stop caring about superscripts to distinguish updates, but if you wish to, the elements of the new row k could be called either $a_{k,j}^{(k)}$ or even $u_{k,j}$, since those values are in their final state.)

3.3.4 Handling rounding error: Maximal Element Partial Pivoting

The final refinement is to seek the smallest possible magnitudes for intermediate values, and thus the smallest absolute errors in them, by making the multipliers $l_{i,k}$ small, in turn by making the denominator $a_{k,k}^{(k-1)} = u_{k,k}$ as large as possible in magnitude:

At step k , choose the pivot row $p_k \geq k$ so that $|a_{p_k,k}^{(k-1)}| \geq |a_{i,k}^{(k-1)}|$ for all $i \geq k$. If there is more than one such element of largest magnitude, use the lowest value: in particular, if $p_k = k$ works, use it and do not swap!

3.3.5 Swapping rows in Python

I will not give a detailed algorithm for this, since we will soon implement an even better variant.

However, here are some notes on swapping values and how to avoid a possible pitfall.

Exercise 1, on Python coding

a) Explain why we cannot just swap the relevant elements of rows k and p with:

```
for j in range(k,n):
    A[k,j] = A[p,j]
    A[p,j] = A[k,j]
```

or with vectorized “slicing”:

```
A[k,k:] = A[p,k:]
A[p,k:] = A[k,k:]
```

Describe what happens instead.

b) A common strategy to avoid this problem uses an intermediate temporary copy of each value being “moved”. This can be combined with slicing, but be careful: arrays (including slices of arrays) must be copied with the method `.copy()`:

```
temp = A[k,k:].copy()
A[k,k:] = A[p,k:]
A[p,k:] = temp
```

3) Python also has an elegant alternative for swapping a pair of values:

```
for j in range(k,n): ( A[k,j] , A[p,j] ) = ( A[p,j] , A[k,j] )
```

This can also be done with slicing, with care to copy those slices:

```
( A[k,k:] , A[p,k:] ) = ( A[p,k:].copy() , A[k,k:].copy() )
```

Some demonstrations

No row reduction is done here, so entire rows are swapped rather than just the elements from column k onward:

```
A = array([[1. , -6. , 2.],[3. , 5. , -6.],[4. , 2. , 7.]])
n = 3
print(f"Initially,\nA =\n {A}")
```

```
Initially,
A =
[[ 1. -6.  2.]
 [ 3.  5. -6.]
 [ 4.  2.  7.]
```

```
k = 0
p = 2
temp = A[k,k:].copy()
```

(continues on next page)

(continued from previous page)

```
A[k,k:] = A[p,k:]
A[p,k:] = temp
print("After swapping rows 1 <-> 3 (row indices 0 <-> 2) using slicing and a
↳temporary row,")
print(f"A =\n {A}")
```

```
After swapping rows 1 <-> 3 (row indices 0 <-> 2) using slicing and a temporary
↳row,
A =
[[ 4.  2.  7.]
 [ 3.  5. -6.]
 [ 1. -6.  2.]]
```

```
k = 1
p = 2
for j in range(n):
    ( A[k,j] , A[p,j] ) = ( A[p,j] , A[k,j] )
print(f"After swapping rows 2 <-> 3 using a loop and tuples of elements, no temp,")
print(f"A =\n {A}")
```

```
After swapping rows 2 <-> 3 using a loop and tuples of elements, no temp,
A =
[[ 4.  2.  7.]
 [ 1. -6.  2.]
 [ 3.  5. -6.]]
```

```
k = 0
p = 1
( A[k,k:] , A[p,k:] ) = ( A[p,k:].copy() , A[k,k:].copy() )
print(f"After swapping rows 1 <-> 2 using tuples of slices, no loop or temp,")
print(f"A =\n {A}")
```

```
After swapping rows 1 <-> 2 using tuples of slices, no loop or temp,
A =
[[ 1. -6.  2.]
 [ 4.  2.  7.]
 [ 3.  5. -6.]]
```

3.4 Solving $Ax = b$ with LU factorization, $A = LU$

References:

- Section 2.2 *The LU Factorization* of [Sauer, 2022].
- Section 6.5 *Matrix Factorizations* of [Burden *et al.*, 2016].
- Section 8.1 *Matrix Factorizations* of [Chenney and Kincaid, 2012].

3.4.1 Avoiding repeated calculation, excessive rounding and messy notation: LU factorization

Putting aside pivoting for a while, there is another direction in which the algorithm for solving linear systems $Ax = b$ can be improved. It starts with the idea of being more efficient when solving multiple system with the same right-hand side: $Ax^{(m)} = b^{(m)}, m = 1, 2, \dots$

However it has several other benefits:

- allowing a strategy to reduce rounding error, and
- a simpler, more elegant mathematical statement.

We will see how to merge this with partial pivoting in *Solving $Ax = b$ With Both Pivoting and LU factorization*.

Some useful jargon:

Definition 2.4.1 (Triangular matrix)

A matrix is **triangular** if all its non-zero elements are either on the main diagonal or to one side of it. There are two possibilities:

- Matrix U is **upper triangular** if $u_{ij} = 0$ for all $i > j$.
- Matrix L is **lower triangular** if $l_{ij} = 0$ for all $j > i$.

One important example of an upper triangular matrix is U formed by row reduction; note well that it is much quicker and easier to solve $Ux = c$ than the original system $Ax = b$ exactly because of its triangular form.

We will soon see that the multipliers $l_{ij}, i > j$ for row reduction that were introduced in *Row Reduction/Gaussian Elimination* help to form a very useful lower triangular matrix L .

The key to the LU factorization idea is finding a **lower triangular** matrix L and an **upper triangular** matrix U such that $LU = A$, and then using the fact that it is far quicker to solve a linear system when the corresponding matrix is triangular.

Indeed we will see that, if naive Gaussian elimination for $Ax = b$ succeeds, giving row-reduced form $Ux = c$:

1. The matrix A can be factorized as $A = LU$ with U an $n \times n$ upper triangular matrix and L an $n \times n$ lower triangular matrix.
2. There is a unique such factorization with the further condition that L is **unit lower triangular**, which means the extra requirement that the value on its main diagonal are unity: $l_{k,k} = 1$. This is called the **Doolittle Factorization** of A .
3. In the Doolittle factorization, the matrix U is the one given by naive Gaussian elimination, and the elements of L below its main diagonal are the multipliers arising in naive Gaussian elimination. (The other elements of L , on and above the main diagonal, are the ones and zeros dictated by it being unit lower triangular: the same as for those elements in the $n \times n$ identity matrix.)
4. The transformed right-hand side c arising from naive Gaussian elimination is the solution of the system $Lc = b$, and this is solvable by an procedure called **forward substitution**, very similar to the backward substitution used to solve $Ux = c$.

Putting all this together: if naive Gaussian elimination works for A , we can introduce the name c for Ux , and note that $Ax = (LU)x = L(Ux) = Lc = b$. Then solving of the system $Ax = b$ can be done in three steps:

1. Using A , find the Doolittle factors, L and U .
2. Using L and b , solve $Lc = b$ to get c . (Forward substitution)
3. Using U and c , solve $Ux = c$ to get x . (Backward substitution)

3.4.2 The direct method for the Doolittle LU factorization

If you believe the above claims, we already have one algorithm for finding an LU factorization; basically, do naive Gaussian elimination, but ignore the right-hand side b until later. However, there is another “direct” method, which does not rely on anything we have seen before about Gaussian elimination, and has other advantages as we will see.

(If I were teaching linear algebra, I would be tempted to start here and skip Gaussian Elimination!)

This method starts by considering the apparently daunting task of solving the n^2 simultaneous and nonlinear equations for the initially unknown elements of L and U :

$$\sum_{k=1}^n l_{i,k} u_{k,j} = a_{i,j} \quad 1 \leq i \leq n, \quad 1 \leq j \leq n.$$

The first step is to insert the known information; the already-known values of elements of L and U . For one thing, the sums above stop when either $k = i$ or $k = j$, whichever comes first, due to all the zeros in L and U :

$$\sum_{k=1}^{\min(i,j)} l_{i,k} u_{k,j} = a_{i,j} \quad 1 \leq i \leq n, \quad 1 \leq j \leq n.$$

Next, when $i \leq j$ —so that the sum ends at $k = i$ and involves $l_{i,i}$ —we can use $l_{i,i} = 1$.

So break up into two cases:

On and above the main diagonal ($i \leq j$, so $\min(i, j) = i$):

$$\sum_{k=1}^{i-1} l_{i,k} u_{k,j} + u_{i,j} = a_{i,j} \quad 1 \leq i \leq n, \quad i \leq j \leq n.$$

Below the main diagonal ($i > j$, so $\min(i, j) = j$):

$$\sum_{k=1}^{j-1} l_{i,k} u_{k,j} + l_{i,j} u_{j,j} = a_{i,j} \quad 2 \leq i \leq n, \quad 1 \leq j \leq i.$$

In each equation, the last term in the sum has been separated, so that we can use them to “solve” for an unknown:

$$u_{i,j} = a_{i,j} - \sum_{k=1}^{i-1} l_{i,k} u_{k,j} \quad 1 \leq i \leq n, \quad i \leq j \leq n.$$

$$l_{i,j} = \frac{a_{i,j} - \sum_{k=1}^{j-1} l_{i,k} u_{k,j}}{u_{j,j}} \quad 2 \leq i \leq n, \quad 1 \leq j \leq i.$$

Here comes the characteristic step that gets us from valid equations to a useful algorithm: we can arrange these equations in an order such that all the values at right are determined by an earlier equation!

First look at what they say for the first row and first column.

With $i = 1$ in the first equation, there is no sum, and so: $u_{1,j} = a_{1,j}$, $1 \leq j \leq n$, which is the familiar fact that the first row is unchanged in naive Gaussian elimination.

Next, with $j = 1$ in the second equation, there is again no sum: $l_{i,1} = \frac{a_{i,1}}{u_{1,1}} = \frac{u_{i,1}}{u_{1,1}}$, $2 \leq i \leq n$, which is indeed the multipliers in the first step of naive Gaussian elimination.

Remember that one way to think of Gaussian elimination is *recursively*: after step k , one just applies the same process recursively to the smaller $n - k \times n - k$ matrix in the bottom-right-hand corner. We can do something similar here; at stage k :

1. First use the first of the above equations to solve first for row k of U , meaning just $u_{k,j}$, $j \geq k$,

2. Then use the second equation to solve for column k of L : $l_{i,k}, i > k$.

Algorithm 2.4.1 (Doolittle factorization)

Stage $k = 1$ is handled by the simpler special equations above, and for the rest:

for k from 2 to n

 for j from k to n *Get the non-zero elements in row k of U*

$$u_{k,j} = a_{k,j} - \sum_{s=1}^{k-1} l_{k,s} u_{s,j}$$

 end

 for i from $k+1$ to n *Get the non-zero elements in column k of L (except the 1's on its diagonal)*

$$l_{i,k} = \frac{a_{i,k} - \sum_{s=1}^{k-1} l_{i,s} u_{s,k}}{u_{k,k}}$$

 end

end

Note well that in the formulas to evaluate at the right,

1. The terms $l_{k,s}$ are for $s < k$, so from a column s that has already been computed for a previous k value.
2. The terms $u_{s,j}$ are for $s < k$, so from a row s that has already been computed for a previous k value.
3. The denominator $u_{k,k}$ in the second inner loop is computed just in time, in the first inner loop for the same k value.

So the only thing that can go wrong is the same as before: a zero pivot element $u_{k,k}$.

Remark 2.4.1 (On this algorithm)

1. For $k = n$, the second inner loop is redundant, so could be eliminated. Indeed it might need to be eliminated in actual code, where “empty loops” might not be allowed. On the other hand, allowing empty loops makes the above correct also for $k = 1$; then the `for k` loop encompasses the entire factorization algorithm.
 2. This direct factorization algorithm avoids any intermediate modification of arrays, and thus eliminates all those superscripts like $a_{i,j}^{(k)}$. This is not only nicer mathematically, but can help to avoid mistakes like code that inadvertently modifies the array containing the matrix A and then uses it to compute the residual, $b - Ax$. More generally, such purely mathematical statements of algorithms can help to avoid coding errors; this is part of the philosophy of the *functional programming* approach.
 3. Careful examination shows that the product $l_{k,s} u_{s,j}$ that is part of what is subtracted at location (k, j) is the same as what is subtracted there at stage k of Gaussian elimination, just with different names. More generally, *every piece of arithmetic is the same as before*, except arranged in a different order, so that the $k - 1$ changes made to an element in row k are done together, via those sums.
 4. Rewrites with zero-based indexing will be provided later.
-

```
# Import some items from modules individually, so they can be used by "first name only"
↪ ".
from numpy import array, zeros_like, identity
```

```

def lu_factorize(A, demoMode=False):
    """Compute the Doolittle LU factorization of A.
    Sums like  $\sum_{s=1}^{k-1} l_{k,s} u_{s,j}$  are done as matrix products;
    in the above case, row matrix  $L[k, 1:k-1]$  by column matrix  $U[1:k-1, j]$  gives the
    sum for a given j,
    and row matrix  $L[k, 1:k-1]$  by matrix  $U[1:k-1, k:n]$  gives the relevant row vector.
    """
    n = len(A) # len() gives the number of rows in a 2D array.
    # Initialize U as the zero matrix;
    # correct below the main diagonal, with the other entries to be computed below.
    U = zeros_like(A)
    # Initialize L as the identity matrix;
    # correct on and above the main diagonal, with the other entries to be computed
    below.
    L = identity(n)
    # Column and row 1 (i.e Python index 0) are special:
    U[0,:] = A[0,:]
    L[1:,0] = A[1:,0]/U[0,0]
    if demoMode:
        print(f"After step k=0")
        print(f"U=\n{U}")
        print(f"L=\n{L}")
    for k in range(1, n-1):
        U[k,k:] = A[k,k:] - L[k,k:] @ U[:k,k:]
        L[k+1:,k] = (A[k+1:,k] - L[k+1:,:k] @ U[:k,k])/U[k,k]
        if demoMode:
            print(f"After step {k=}")
            print(f"U=\n{U}")
            print(f"L=\n{L}")
    # The last row (index "-1") is special: nothing to do for L
    U[-1,-1] = A[-1,-1] - sum(L[-1,:-1]*U[:-1,-1])
    if demoMode:
        print(f"After the final step, k={n-1}")
        print(f"U=\n{U}")
    return (L, U)

```

A test case on LU factorization

```
A = array([[4, 2, 7], [3, 5, -6], [1, -3, 2]], dtype=float)
```

```
print(f"A=\n{A}")
```

```

A=
[[ 4.  2.  7.]
 [ 3.  5. -6.]
 [ 1. -3.  2.]]

```

```
(L, U) = lu_factorize(A, demoMode=True)
```

```

After step k=0
U=
[[4. 2. 7.]

```

(continues on next page)

(continued from previous page)

```

[0. 0. 0.]
[0. 0. 0.]]
L=
[[1.  0.  0. ]
 [0.75 1.  0. ]
 [0.25 0.  1. ]]
After step k=1
U=
[[ 4.  2.  7. ]
 [ 0.  3.5 -11.25]
 [ 0.  0.  0. ]]
L=
[[ 1.  0.  0. ]
 [ 0.75 1.  0. ]
 [ 0.25 -1.  1. ]]
After the final step, k=2
U=
[[ 4.  2.  7. ]
 [ 0.  3.5 -11.25]
 [ 0.  0. -11. ]]

```

```

print(f"A=\n{A}")
print(f"L=\n{L}")
print(f"U=\n{U}")
print(f"L times U is \n{L@U}")
print(f"The 'residual' A - LU is \n{A - L@U}")

```

```

A=
[[ 4.  2.  7.]
 [ 3.  5. -6.]
 [ 1. -3.  2.]]
L=
[[ 1.  0.  0. ]
 [ 0.75 1.  0. ]
 [ 0.25 -1.  1. ]]
U=
[[ 4.  2.  7. ]
 [ 0.  3.5 -11.25]
 [ 0.  0. -11. ]]
L times U is
[[ 4.  2.  7.]
 [ 3.  5. -6.]
 [ 1. -3.  2.]]
The 'residual' A - LU is
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

```

Forward substitution: solving $Lc = b$ for c

This is the last piece missing. The strategy is very similar to backward substitution, but slightly simplified by the ones on the main diagonal of L . The equations $Lc = b$ can be written much as above, separating off the last term in the sum:

$$\sum_{j=1}^n l_{i,j} c_j = b_i, \quad 1 \leq i \leq n$$

$$\sum_{j=1}^i l_{i,j} c_j = b_i, \quad 1 \leq i \leq n$$

$$\sum_{j=1}^{i-1} l_{i,j} c_j + c_i = b_i, \quad 1 \leq i \leq n$$

Then solve for c_i :

$$c_i = b_i - \sum_{j=1}^{i-1} l_{i,j} c_j$$

These are already in usable order: the right-hand side in the equation for c_i involves only the c_j values with $j < i$, determined by earlier equations if we run through index i in increasing order.

First, $i = 1$

$$c_1 = b_1 - \sum_{j=1}^0 l_{1,j} c_j = b_1$$

Next, $i = 2$

$$c_2 = b_2 - \sum_{j=1}^1 l_{2,j} c_j = b_2 - l_{2,1} c_1$$

Next, $i = 3$

$$c_3 = b_3 - \sum_{j=1}^2 l_{3,j} c_j = b_3 - l_{3,1} c_1 - l_{3,2} c_2$$

I leave as an exercise expressing this in pseudo-code (adjusted to zero-based indexing); here it is in Python; also available from the module `numericalMethods` with

```
from numericalMethods import forwardSubstitution
```

Exercise 1

A) Express this forward substitution strategy as pseudo-code, adjusting to Python's zero-based indexing. Spell out all the sums in explicit rather than using ' Σ ' notation for sums any matrix multiplication short-cut.

B) Then implement it "directly" in a Python function, with format:

```
function forwardSubstitution(L, b)
    . . .
    return c
```


Again do this with explicit evaluation of each sum rather than using the function `sum` or any matrix multiplication short-cut.

C) Test it, using this often-useful “reverse-engineering” tactic:

1. Create suitable test arrays `L` and `c`. (Use n at least three, and preferably larger.)
2. Compute their product, with `b = L * c`
3. Check if `c_solution = forwardSubstitution(L, b)` gives the correct value (within rounding error.)

As usual, there is also an implementation available from module `numericalMethods`, at ... *and forward substitution* ..., so this is used here. (It is not in the form asked for in the above exercise!)

A test case on forward substitution

```
b = array([2., 3., 4.])
```

```
c = forwardSubstitution(L, b)
```

```
print(f"c = {c}")
print(f"The residual b - Lc is {b - L@c}")
print(f"\t with maximum norm {max(abs(b - L@c)) :0.3}")
```

```
c = [2.  1.5 5. ]
The residual b - Lc is [0. 0. 0.]
        with maximum norm 0.0
```

Completing the test case, with backward substitution

As this step is unchanged, just import the version seen in a previous section.

```
from numericalMethods import backwardSubstitution
```

```
x = backwardSubstitution(U, c)
```

```
print(f"The residual c - Ux for the backward substitution step is {c - U@x}")
print(f"\t with maximum norm {max(abs(c - U@x)) :0.3}")
print(f"The residual b - Ax for the whole solving process is {b - A@x}")
print(f"\t with maximum norm {max(abs(b - A@x)) :0.3}")
```

```
The residual c - Ux for the backward substitution step is [0. 0. 0.]
        with maximum norm 0.0
The residual b - Ax for the whole solving process is [ 0.0000000e+00 -4.4408921e-
↵16  8.8817842e-16]
        with maximum norm 8.88e-16
```

Exercise 2

(An ongoing activity.)

Start building a Python module `linearalgebra` in a file `linearalgebra.py`, with all our linear algebra functions: for now, `forwardSubstitution(L, b)` as above and also `rowReduce(A, b)` and `backwardSubstitution(U, c)` from the section [Row Reduction/Gaussian Elimination](#)

Include testing/demonstration at the bottom of the module definition file, in the block of the `if` statement

```
if __name__ == "__main__":
```

We will add the Doolittle method and such in a while, and could use this module in assignments and projects.

Creating modules

One way to do this is with Spyder (or another Python IDE). However, if you prefer working primarily with JupyterLab and Jupyter notebooks, one way to create this module is to first put the function `def`'s and testing code in a notebook `linearalgebra.ipynb` and then convert that to `linearalgebra.py` with the JupyterLab menu command

```
File > Export Notebook As ... > Export Notebook to Executable Script
```

As an example of creating a module, I am creating one as we go along in this course, via the notebook [Notebook for generating the module numericalMethods](#) and Python code file `numericalMethods.py` derived from that, which defines module `numericalMethods`.

The notebook version is in the Appendices of this Jupyter book.

3.4.3 When does LU factorization work?

It was seen in the section [Partial Pivoting](#) that naive Gaussian elimination works (in the sense of avoiding division by zero) so one good result is that

Theorem 2.4.1

Any SDD matrix has a Doolittle factorization $A = LU$, with the diagonal elements of U all non-zero, so backward substitution also works.

For any column-wise SDD matrix, this LU factorization exists and is also “optimal”, in the sense that it follows what you would do with maximal element partial pivoting.

This nice second property can be got for SDD matrices via a twist, or actually a transpose.

For an SDD matrix, its transpose $B = A^T$ is column-wise SDD and so has the nice Doolittle factorization described above: $B = L_B U_B$, with L_B being column-wise diagonally dominant and having ones on the main diagonal.

Transposing back, $A = B^T = (L_B U_B)^T = U_B^T L_B^T$, and defining $L = U_B^T$ and $U = L_B^T$,

- L is lower triangular
- U is upper triangular, row-wise diagonally dominant and with ones on its main diagonal: it is “unit upper triangular”.
- Thus LU is another LU factorization of A , with U rather than L being the factor with ones on its main diagonal.

3.4.4 Crout decomposition

This sort of LU factorization is called the **Crout decomposition**; as with the Doolittle version, if such a factorization exists, it is unique.

Theorem 2.4.2

Every SDD matrix has a Crout decomposition, and the factor U is SDD.

Remark 2.4.2

As was mentioned at the end of the section *Row Reduction/Gaussian Elimination* naive Gaussian elimination also works for *positive definite* matrices, and thus so does the Doolittle LU factorization. However, there is another LU factorization that works even better in that case, the *Cholesky factorization*; this topic might be returned to later.

3.5 Solving $Ax = b$ With Both Pivoting and LU factorization

References:

- Section 2.4 *The $PA=LU$ Factorization* of [Sauer, 2022].
- Section 6.5 *Matrix Factorizations* of [Burden *et al.*, 2016].
- Section 8.1 *Matrix Factorizations* of [Chenney and Kincaid, 2012].

3.5.1 Introduction

The last step in producing an algorithm for solving the general case of n simultaneous linear equations in n variables that is *robust*, *efficient* and with good control of *rounding error* is to combine the ideas of **partial pivoting** from *Partial Pivoting* and **LU factorization** from *Solving $Ax = b$ with LU factorization*, $A = LU$.

This is sometimes described in three parts:

- *permute* (reorder) the rows of the matrix A by multiplying it at left by a suitable *permutation matrix* P ; one with a single “1” in each row and each column and zeros elsewhere;
- Get the LU factorization of this matrix: $PA = LU$.
- To solve $Ax = b$
 - Express as $PAx = LUx = Pb$ (which just involves computing Pb , which reorders the elements of b)
 - Solve $Lc = Pb$ for c by forward substitution
 - Solve $Ux = c$ for x by backward substitution: as before, this gives $LUx = Lc = Pb$ and $LUx = PAx$, so $PAx = Pb$; since a permutation matrix P is invertible (just unravel the row swaps), this ensures that $Ax = b$.

This gives a nice formulas in terms of matrices; however we can describe it a bit more compactly and efficiently by just talking about the permutation of the rows, described by a *permutation vector* — an n component vector $\pi = [\pi_1, \pi_2, \dots, \pi_n]$ whose elements are the integers from 1 to n in some order. So that is how the algorithm will be described below.

(Aside: I use the conventional name π for a permutation vector, partly to distinguish from the notation p_i used for *pivot* rows; however, feel free to use the name p instead, especially in Julia code.)

A number of details of this sketch will now be filled in, including the very useful fact that the permutation vector (or matrix) can be constructed “on the fly”, as rows are swapped in partial pivoting.

3.5.2 Row swapping is all you need

Let us look at maximal element partial pivoting, but described in terms of the entries of the factors L and U , and updating matrix A with a succession of row swaps.

(For now, I omit what happens to the right-hand side vector b ; that is where the permutation vector p will come in, as addressed below.)

What happens if pivoting occurs at some stage k , with swapping of row k with a row $p_k > 5$?

One might fear that the process has to start again from the top using the modified version of matrix A , but in fact all previous work can be reused, just swapping those rows “everywhere”.

Example: what happens at stage 5 ($k = 5$)?

To see this with a concrete example consider what happens if at stage $k = 5$ we swap rows 5 and 10 of A .

A) Firstly, what happens to matrix A ?

The previous steps of the LU factorization process only involved entries of A in its first four rows and first four columns, and this row swap has no effect of them. Likewise, in row reduction, changes at and below row $k = 5$ have no effect on the first four rows of the row reduced form, U .

Thus, the only change here is to swap the entries of A between rows 5 and 10. What is more, the subsequent calculations only involve columns of index $j = 5$ upwards, so in fact we only need to update those entries. This can be written as

$$a_{5,j} \leftrightarrow a_{10,j}, \quad 5 \leq j \leq n$$

Thus if we are working in Python with A stored in a numpy array, the update is the slice operation

```
( A[5, 5:], A[10, 5:] ) = ( A[10, 5:], A[5, 5:] )
```

(except for that pesky Pythonic down-shifting of indices; to be seen in pseudo-code later!)

B) Next, look at the work done so far on U .

That just consists of the previous rows $1 \leq i \leq 4$, and the swapping of rows 5 with 10 has no effect up there:

Values already computed in U are unchanged.

C) Finally, look at the work done so far on the multipliers $l_{i,j}$; that is, matrix L .

The values computed so far are the first four columns of L ; the multiples $l_{i,j}$, $1 \leq j \leq 4$ of row j subtracted from row $i > j$. These *do* change: for example, the multiple $l_{5,2}$ of row 2 is now subtracted from what was row 5 but is now row 10: thus, the new value of $l_{10,2}$ is the previous value of $l_{5,2}$.

Likewise, the same is true in reverse: the new value of $l_{5,2}$ is the previous value of $l_{10,2}$. This applies for all of the first four rows, so second index $1 \leq j \leq 4$:

The entries of L computed so far are swapped between rows 5 and 10, leaving the rest unchanged.

As this is again only for some columns — the first four — the swaps needed are:

$$l_{5,j} \leftrightarrow l_{10,j}, \quad 1 \leq j \leq 4$$

or in Python slice notation for an array L :

$(L[5, :4], L[10, :4]) = (L[10, :4], L[5, :4])$

The general pattern

The example above extends to all stages k of row reduction or computing the LU factorization or a permute version of matrix A , where we adjust the pivot element at position (k, k) by first swapping row k with a row $p_k, \geq k$. (Allowing that sometimes no swap is needed, so that $p_k = k$.)

Gathering the key formulas above, this part of the algorithm is

Algorithm 2.5.1

for k from 1 to $n-1$

Find the pivot row $p_k, \geq k$.

if $p_k > k$

Swap $l_{k,j} \leftrightarrow l_{p_k,j}, \quad 1 \leq j < k$

Swap $a_{k,j} \leftrightarrow a_{p_k,j}, \quad k \leq j \leq n$

end

end

Pseudo-code for LU factorization with row swapping (first version)

Here I also adopt slice notation; for example, $a_{k,k:n}$ denotes the slice $[a_{k,k} \dots a_{k,n}]$.

Algorithm 2.5.2 (LU factorization with row swapping, I)

for k from 1 to n

Find the pivot element:

$p = k$ (*p will be the index of the pivot row*)

for i from $k+1$ to n

if $|u_{i,k}| > |u_{p,k}|$

$p \leftarrow i$

end

end

if $p > k$ (*Swap rows*)

$l_{k,1:k-1} \leftrightarrow l_{p,1:k-1}$

$a_{k,k:n} \leftrightarrow a_{p,k:n}$

end

for j from k to n (*Get the non-zero elements in row k of U*)

$u_{k,j} = a_{k,j} - \sum_{s=1}^{k-1} l_{k,s} u_{s,j}$

```

end
for i from k+1 to n    (Get the non-zero elements in column k of L — except the 1's on its diagonal)
    
$$l_{i,k} = \frac{a_{i,k} - \sum_{s=1}^{k-1} l_{i,s} u_{s,k}}{u_{k,k}}$$

end
end

```

But what about the right-hand side, b ?

One thing is missing from this strategy so far: if we are solving with a given right-hand-side column vector b , we would also swap its rows at each stage, with

$$b_k \leftrightarrow b_{p_k}$$

but with the LU factorization we need to keep track of these swaps for use later.

This turns out to mesh nicely with another detail: we can avoid actually copying array entries around by just keeping track of the order in which we use rows to get zeros in other rows. Our goal will be a permutation vector $\pi = [\pi_1, \pi_2, \dots, \pi_n]$ which says:

- First use row π_1 to get zeros in column 1 of the $n - 1$ other rows.
- Then use row π_2 to get zeros in column 2 of the $n - 2$ remaining rows.
- ...

To do this:

- first, initialize an array $\pi = [1, 2, \dots, n]$
- at stage k , if the pivot element is in row $p_k \neq k$, swap the corresponding elements in π (rather than swapping entire rows of arrays):

$$\pi_k \leftrightarrow \pi_{p_k}$$

Introducing the name A' for the new version of matrix A , its row k has entries $a'_{k,j} = a_{\pi_k,j}$.

This pattern persists through each row swap: instead of computing a succession of updated versions of matrix A , we leave it alone and just change the row indices:

All references to entries of A are now done with permuted row index: $a_{\pi_i,j}$

The same applies to the array L of multipliers:

All references to entries of L are now done with $l_{\pi_i,j}$.

Finally, since these row swaps also apply to the right-hand side b , we do the same there:

All references to entries of b are now done with b_{π_i} .

Pseudo-code for LU factorization with a permutation vector

Algorithm 2.5.3 (LU factorization with row swapping, II)

Initialize the permutation vector, $\pi \leftarrow [1, 2, \dots, n]$

for k from 1 to n

 Find the pivot element:

$p \leftarrow k$ (*p will be the index of the pivot row*)

 for i from k+1 to n

 if $|u_{i,k}| > |u_{p,k}|$:

$p \leftarrow i$

 end

 if $p > k$ (*Just swap indices, not rows*)

$\pi_k \leftrightarrow \pi_p$

 end

 for j from k to n (*Get the non-zero elements in row k of U*)

$u_{k,j} \leftarrow a_{k,j} - \sum_{s=1}^{k-1} l_{k,s} u_{s,j}$

 end

 for i from k+1 to n (*Get the non-zero elements in column k of L — except the 1's on its diagonal*)

$l_{i,k} \leftarrow \frac{a_{i,k} - \sum_{s=1}^{k-1} l_{i,s} u_{s,k}}{u_{k,k}}$

 end

end

Remark 2.5.1

For the version with a permutation matrix P , instead:

- start with an array P that is the identity matrix, and then
- swap its rows $k \leftrightarrow p_k$ at stage k instead of swapping the entries of π or the rows of A and L .

```
from numpy import array, zeros_like
```

```
def plu(A, demoMode=False):
    """Compute the Doolittle PA=LU factorization of A —
    but with the permutation recorded as permutation vector, not as the permutation
    ↪ matrix P.
    Sums like  $\sum_{s=1}^{k-1} l_{k,s} u_{s,j}$  are done as matrix products;
    in the above case, row matrix L[k, 1:k-1] by column matrix U[1:k-1, j] gives the
    ↪ sum for a give j,
    and row matrix L[k, 1:k-1] by matrix U[1:k-1, k:n] gives the relevant row vector.
    """
```

(continues on next page)

(continued from previous page)

```

n = len(A)  # len() gives the number of rows in a 2D array.
perm = array(range(n))
# Initialize U as the zero matrix;
# correct below the main diagonal, with the other entries to be computed below.
U = zeros_like(A)
# Also, initialize L as the zero matrix;
# the 1's will also be filled in as we go.
L = zeros_like(A)
for k in range(n-1):
    if demoMode: print(f"k={k}")

    # Find the pivot element in column k:
    pivot_row = k
    abs_u_ik_max = abs(A[perm[k],k])
    for row in range(k+1, n):
        abs_u_ik = abs(A[perm[row],k])
        if abs_u_ik > abs_u_ik_max:
            pivot_row = row
            abs_u_ik_max = abs_u_ik
    if pivot_row > k: # "swap"
        if demoMode: print(f"Swap row {k} with row {pivot_row}")
        (perm[k], perm[pivot_row]) = (perm[pivot_row], perm[k])
    else:
        if demoMode: print(f"No row swap needed.")
    U[k,k:] = A[perm[k],k:] - L[perm[k],:k] @ U[:k,k:]
    L[perm[k],k] = 1.
    for row in range(k+1,n):
        L[perm[row],k] = ( A[perm[row],k] - L[perm[row],:k] @ U[:k,k] ) / U[k,k]
    if demoMode:
        print(f"permuted A is:")
        for row in range(n):
            print(A[perm[row],:])
        print(f"intermediate U is\n{U}")
        print(f"intermediate L is\n{L}")
        print(f"perm={perm}")
    # The last row (index "-1") is special: nothing to do for L except put in the 1_
    on the "permuted main diagonal"
    U[n-1,n-1] = A[perm[n-1],n-1] - sum(L[perm[n-1],:n-1]*U[:n-1,n-1])
    L[perm[n-1],n-1] = 1.
    if demoMode:
        print(f"After the final step, k={n-1}")
        print(f"U=\n{U}")
    return (L, U, perm)

```

```

A = array([[1. , -3. , 22.], [3. , 5. , -6.], [4. , 235. , 7.], ])
print(f"A=\n{A}")
(L, U, perm) = plu(A, demoMode=True)
print("\nFunction plu gives")
print(f"row permutation {perm}")
print(f"L=\n{L}")
print(f"U=\n{U}")
print(f"The 'residual' A - LU is \n{A - L@U}")

```

```

A=
[[ 1. -3. 22.]

```

(continues on next page)

(continued from previous page)

```

[ 3.  5. -6.]
[ 4. 235.  7.]]
k=0
Swap row 0 with row 2
permuted A is:
[ 4. 235.  7.]
[ 3.  5. -6.]
[ 1. -3. 22.]
intermediate U is
[[ 4. 235.  7.]
 [ 0.  0.  0.]
 [ 0.  0.  0.]]
intermediate L is
[[0.25 0.  0. ]
 [0.75 0.  0. ]
 [1.  0.  0. ]]
perm=[2 1 0]
k=1
No row swap needed.
permuted A is:
[ 4. 235.  7.]
[ 3.  5. -6.]
[ 1. -3. 22.]
intermediate U is
[[ 4. 235.  7. ]
 [ 0. -171.25 -11.25]
 [ 0.  0.  0. ]]
intermediate L is
[[0.25 0.36058394 0. ]
 [0.75 1.  0. ]
 [1.  0.  0. ]]
perm=[2 1 0]
After the final step, k=2
U=
[[ 4. 235.  7. ]
 [ 0. -171.25 -11.25 ]
 [ 0.  0. 24.30656934]]

Function plu gives
row permutation [2 1 0]
L=
[[0.25 0.36058394 1. ]
 [0.75 1.  0. ]
 [1.  0.  0. ]]
U=
[[ 4. 235.  7. ]
 [ 0. -171.25 -11.25 ]
 [ 0.  0. 24.30656934]]
The 'residual' A - LU is
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

```

Matrix L is not actually lower triangular, due to the permutation of its rows, but is still fine for a version of forward substitution, because

- row π_1 only involves x_1 (multiplied by 1) and so can be used to solve for x_1

- row π_2 only involves x_1 and x_2 (the latter multiplied by 1) and so can be used to solve for x_2
- ...

Definition 2.5.1 (Psychologically [lower] triangular)

A matrix like this — one that is a row-permutation of a [lower] triangular matrix — is called **psychologically [lower] triangular**. (Maybe because it believes itself to be such?)

Forward and backward substitution with a permutation vector

To solve $Lc = b$, all one has to change from the formulas for forward substitution seen in the previous section *Solving $Ax = b$ with LU factorization*, $A = LU$ is to put the permuted row index π_i in both L and b :

$$c_i = b_{\pi_i} - \sum_{j=1}^{i-1} l_{\pi_i, j} c_j, \quad 1 \leq i \leq n$$

```
def forwardsubstitution(L, b, perm):
    """Solve  $Lc = b$  for  $c$ , with permutation of the rows of  $L$  and of  $b$ ."""
    n = len(b)
    c = zeros_like(b)
    c[0] = b[perm[0]]
    for i in range(1, n):
        c[i] = b[perm[i]] - L[perm[i], :i] @ c[:i]
    return c
```

```
b = array([2., 3., 4.])
```

```
print(f"b = {b}")
```

```
b = [2. 3. 4.]
```

```
c = forwardsubstitution(L, b, perm)
print(f"c={c}")
```

```
c=[4. 0. 1.]
```

Then the final step, solving $Ux = b$ for x , needs no change, because U had no rows swapped, so we are done; we can import the function `backwardsubstitution` seen previously

```
from numericalMethods_module import backwardsubstitution
```

```
x = backwardsubstitution(U, c)
print(f"x={x}")
r = b - A@x
print(f"The residual r = b - Ax is \n{r}, with maximum norm {max(abs(r))}")
```

```
x=[ 1.08678679 -0.0027027  0.04114114]
The residual r = b - Ax is
[0. 0. 0.], with maximum norm 0.0
```

3.6 Error bounds for linear algebra, condition numbers, matrix norms, etc.

References:

- Section 2.3.1 *Error Magnification and Condition Number* of [Sauer, 2022].
- Section 7.5 *Error Bounds and Iterative Refinement* of [Burden et al., 2016] — but you may skip the last part, on *Iterative Refinement*; that is not relevant here.
- Section 8.4 of [Chenney and Kincaid, 2012].

3.6.1 Residuals, backward errors, forward errors, and condition numbers

For an approximation x_a of the solution x of $Ax = b$, the *residual* $r = Ax_a - b$ measures error as *backward error*, often measured by a single number, the *residual norm* $\|Ax_a - b\|$. Any norm could be used, but the maximum norm is usually preferred, for reasons that we will see soon.

The corresponding (dimensionless) measure of relative error is defined as

$$\frac{\|r\|}{\|b\|}.$$

However, these can greatly underestimate the *forward* errors in the solution: the absolute error $\|x - x_a\|$ and relative error

$$Rel(x_a) = \frac{\|x - x_a\|}{\|x\|}$$

To relate these to the residual, we need the concepts of a *matrix norm* and the *condition number* of a matrix.

3.6.2 Matrix norms induced by vector norms

Given any vector norm $\|\cdot\|$ — such as the maximum (“infinity”) norm $\|\cdot\|_\infty$ or the Euclidean norm (length) $\|\cdot\|_2$ — the corresponding *induced matrix norm* is

$$\|A\| := \max_{x \neq 0} \frac{\|Ax\|}{\|x\|}, = \max_{\|x\|=1} \|Ax\|$$

This maximum exists for either of these vector norms, and for the infinity norm there is an explicit formula for it: for any $m \times n$ matrix,

$$\|A\|_\infty = \max_{i=1}^m \sum_{j=1}^n |a_{ij}|$$

(On the other hand, it is far harder to compute the Euclidean norm of a matrix: the formula requires computing eigenvalues.)

Note that when the matrix is a vector considered as a matrix with a single column — so $n = 1$ — the sum goes away, and this agrees with the infinity vector norm. This allows us to consider vectors as being just matrices with a single column, which we will often do from now on.

3.6.3 Properties of (induced) matrix norms

These induced matrix norms have many properties in common with Euclidean length and other vector norms, but there can also be products, and then one has to be careful.

1. $\|A\| \geq 0$ (positivity)
2. $\|A\| = 0$ if and only if $A = 0$ (definiteness)
3. $\|cA\| = |c| \|A\|$ for any constant c (absolute homogeneity)
4. $\|A + B\| \leq \|A\| + \|B\|$ (sub-additivity or the triangle inequality),
and when the product of two matrices makes sense (including matrix-vector products),
5. $\|AB\| \leq \|A\| \|B\|$ (sub-multiplicativity)

Note the failure to always have equality with products. Indeed one can have $AB = 0$ with A and B both non-zero, such as when A is a singular matrix and B is a null-vector for it.

Remark 2.6.1 (Other matrix norms)

There are other matrix norms of use in some contexts, in particular the [Frobenius norm](#). Then the above properties are often used to *define* what it is to be a matrix form, much as the first four define what it is to be a vector norm.

Remark 2.6.2 (numpy.linalg.norm)

Python package Numpy provides the function `numpy.linalg.norm` for evaluating matrix norms. The default usage `numpy.linalg.norm(A)` computes $\|A\|_2$, which one can also specify explicitly with `numpy.linalg.norm(A, 2)`; to get the maximum norm $\|A\|_\infty$, one uses `numpy.linalg.norm(A, numpy.inf)`.

3.6.4 Relative error bound and condition number

It can be proven that, for any choice of norm,

$$\text{Rel}(x_a) = \frac{\|x - x_a\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|r\|}{\|b\|},$$

where the last factor $\frac{\|r\|}{\|b\|}$ is the *relative backward error*.

Since we can (though often with considerable effort, due to the inverse!) compute the right-hand side when the infinity norm is used, we can compute an upper bound on the relative error. From this, an upper bound on the absolute error can be computed if needed.

The *growth factor* between the relative backward error measured by the residual and the relative (forward) error is called the *condition number*, $K(A)$:

$$\kappa(A) := \|A\| \|A^{-1}\|$$

so that the above bound on the relative error can be restated as

$$\text{Rel}(x_a) = \frac{\|x - x_a\|}{\|x\|} \leq \kappa(A) \frac{\|r\|}{\|b\|}$$

Actually there is a different condition number for each choice of norm; we work with

$$\kappa_\infty(A) := \|A\|_\infty \|A^{-1}\|_\infty$$

Note that for a singular matrix, this is undefined: we can intuitively say that the condition number is then infinite.

At the other extreme, the identity matrix I has norm 1 and condition number 1 (using any norm), and this is the best possible because in general $\kappa(A) \geq 1$. (This follows from property 5, sub-multiplicativity.)

Aside: estimating $\|A^{-1}\|_\infty$ and thence the condition number, and `numpy.linalg.cond`

In Python, good approximations of condition numbers are given by the function `numpy.linalg.cond`.

As with `numpy.linalg.norm`, the default `numpy.linalg.cond(A)` gives $\kappa_2(A)$, based on the Euclidian length $\|\cdot\|_2$ for vectors; to get the infinity norm version $\kappa_\infty(A)$ use `numpy.linalg.cond(A, numpy.inf)`.

This is not done exactly, since computing the inverse is a lot of work for large matrices, and good estimates can be got far more quickly. The basic idea is start with the formula

$$\|A^{-1}\| = \max_{\|x\|=1} \|A^{-1}x\|$$

and instead compute the maximum over some finite selection of values for x : call them $x^{(k)}$. Then to evaluate $y^{(k)} = A^{-1}x^{(k)}$, express this through the equation $Ay^{(k)} = x^{(k)}$. Once we have an LU factorization for A (which one probably would have when exploring errors in a numerical solution of $Ax = b$) each of these systems can be solved relatively fast: Then

$$\|A^{-1}\| \approx \max_k \|y^{(k)}\|.$$

3.6.5 Well-conditioned and ill-conditioned problems and matrices

Condition numbers, giving upper limit on the ratio of forward error to backward error, measure the amplification of errors, and have counterparts in other contexts. For example, with an approximation r_a of a root r of the equation $f(x) = 0$, the ratio of forward error to backward error is bounded by $\max 1/|f'(x)| = \frac{1}{\min |f'(x)|}$, where the maximum only need be taken over an interval known to contain both the root and the approximation. This condition number becomes “infinite” for a multiple root, $f'(r) = 0$, related to the problems we have seen in that case.

Careful calculation of an approximate solution x_a of $Ax = b$ can often get a *residual* that is at the level of machine rounding error, so that roughly the relative backward error is of size comparable to the machine unit, u . The condition number then guarantees that the (forward) relative error is no greater than about $u \kappa(A)$.

In terms of significant bits, with p bit machine arithmetic, one can hope to get $p - \log_2(\kappa(A))$ significant bits in the result, but can not rely on more, so one loses $\log_2(\kappa(A))$ significant bits. Compare this to the observation that one can expect to lose at least $p/2$ significant bits when using the approximation $Df(x) \approx D_h f(x) - (f(x+h) - f(x))/h$.

A **well-conditioned problem** is one that is not too highly sensitive to errors in rounding or input data; for an equation $Ax = b$, this corresponds to the condition number of A not being too large; the matrix A is then sometimes also called well-conditioned. This is of course vague, but might typically mean that $p - \log_2(\kappa(A))$ is a sufficient number of significant bits for a particular purpose.

A problem that is not deemed well-conditioned is called **ill-conditioned**, so that a matrix of uncomfortably large condition number is also sometimes called ill-conditioned. An ill-conditioned problem might still be well-posed, but just requiring careful and precise solution methods.

Example 2.6.1 (the Hilbert matrices)

The $n \times n$ Hilbert matrix H_n has elements

$$H_{i,j} = \frac{1}{i+j-1}$$

For example

$$H_4 = \begin{bmatrix} 1 & 1/2 & 1/3 & 1/4 \\ 1/2 & 1/3 & 1/4 & 1/5 \\ 1/3 & 1/4 & 1/5 & 1/6 \\ 1/4 & 1/5 & 1/6 & 1/7 \end{bmatrix}$$

and for larger or smaller n , one simply adds or remove rows below and columns at right.

These matrices arise in important situations like finding the polynomial of degree $n - 1$ that fits given data in the sense of minimizing the root-mean-square error — as we will discuss later in this course if there is time and interest.

Unfortunately as n increases the condition number grows rapidly, causing severe rounding error problems. To illustrate this, I will do something that one should usually avoid: compute the inverse of these matrices. This is also a case that shows the advantage of the LU factorization, since one computes the inverse by successively computing each column, by solving n different systems of equations, each with the same matrix A on the left-hand side.

```
import numpy as np
from numpy import inf
from numericalMethods import lu_factorize, forwardsubstitution, backwardsubstitution, \
    solveLinearsystem
from numpy.linalg import norm, cond
from numpy.random import random
```

```
def inverse(A):
    """Use sparingly; there is usually a way to avoid computing inverses that is
    faster and with less rounding error!"""
    n = len(A)
    A_inverse = np.zeros_like(A)
    (L, U) = lu_factorize(A)
    for i in range(n):
        b = np.zeros(n)
        b[i] = 1.0
        c = forwardsubstitution(L, b)
        A_inverse[:,i] = backwardsubstitution(U, c)
    return A_inverse
```

```
def hilbert(n):
    H = np.zeros([n,n])
    for i in range(n):
        for j in range(n):
            H[i,j] = 1.0/(1.0 + i + j)
    return H
```

```
for n in range(2,6):
    H_n = hilbert(n)
    print(f"H_{n} is")
    print(H_n)
    H_n_inverse = inverse(H_n)
    print("and its inverse is")
    print(H_n_inverse)
    print("to verify, their product is")
    print(H_n @ H_n_inverse)
    print()
```

```

H_2 is
[[1.          0.5          ]
 [0.5         0.33333333]]
and its inverse is
[[ 4. -6.]
 [-6. 12.]]
to verify, their product is
[[1. 0.]
 [0. 1.]]

H_3 is
[[1.          0.5          0.33333333]
 [0.5         0.33333333 0.25        ]
 [0.33333333 0.25         0.2         ]]
and its inverse is
[[ 9. -36. 30.]
 [-36. 192. -180.]
 [ 30. -180. 180.]]
to verify, their product is
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

H_4 is
[[1.          0.5          0.33333333 0.25        ]
 [0.5         0.33333333 0.25         0.2         ]
 [0.33333333 0.25         0.2         0.16666667]
 [0.25        0.2         0.16666667 0.14285714]]
and its inverse is
[[ 16. -120. 240. -140.]
 [-120. 1200. -2700. 1680.]
 [ 240. -2700. 6480. -4200.]
 [-140. 1680. -4200. 2800.]]
to verify, their product is
[[ 1.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00]
 [-3.55271368e-15  1.00000000e+00 -1.13686838e-13 -1.13686838e-13]
 [-3.55271368e-15  5.68434189e-14  1.00000000e+00  0.00000000e+00]
 [ 0.00000000e+00 -5.68434189e-14  0.00000000e+00  1.00000000e+00]]

H_5 is
[[1.          0.5          0.33333333 0.25         0.2         ]
 [0.5         0.33333333 0.25         0.2         0.16666667]
 [0.33333333 0.25         0.2         0.16666667 0.14285714]
 [0.25        0.2         0.16666667 0.14285714 0.125        ]
 [0.2         0.16666667 0.14285714 0.125         0.11111111]]
and its inverse is
[[ 2.500e+01 -3.000e+02  1.050e+03 -1.400e+03  6.300e+02]
 [-3.000e+02  4.800e+03 -1.890e+04  2.688e+04 -1.260e+04]
 [ 1.050e+03 -1.890e+04  7.938e+04 -1.176e+05  5.670e+04]
 [-1.400e+03  2.688e+04 -1.176e+05  1.792e+05 -8.820e+04]
 [ 6.300e+02 -1.260e+04  5.670e+04 -8.820e+04  4.410e+04]]
to verify, their product is
[[ 1.00000000e+00  4.54747351e-13  0.00000000e+00  0.00000000e+00
  0.00000000e+00]
 [-2.84217094e-14  1.00000000e+00 -1.81898940e-12 -1.81898940e-12
  1.81898940e-12]
 [-4.26325641e-14  9.09494702e-13  1.00000000e+00  0.00000000e+00
  0.00000000e+00]]

```

(continues on next page)

(continued from previous page)

```

-2.72848411e-12]
[ 2.84217094e-14 -2.27373675e-13  9.09494702e-13  1.00000000e+00
 1.81898940e-12]
[ 2.84217094e-14 -2.27373675e-13  1.81898940e-12 -3.63797881e-12
 1.00000000e+00]]

```

Note how the inverses have some surprisingly large elements; this is the matrix equivalent of a number being very close to zero and so with a very large reciprocal.

Since we have the inverses, we can compute the matrix norms of each H_n and its inverse, and thence their condition numbers; then this can be compared to the approximations of these condition numbers given by `numpy.linalg.cond`

```

for n in range(2,6):
    H_n = hilbert(n)
    print(f"H_{n} is")
    print(H_n)
    print(f"with infinity norm {norm(H_n, inf)}")
    H_n_inverse = inverse(H_n)
    print("and its inverse is")
    print(H_n_inverse)
    print(f"with infinity norm {norm(H_n_inverse, inf)}")
    print(f"Thus the condition number of H_{n} is {norm(H_n, inf) * norm(H_n_inverse, inf)}")
    print(f"For comparison, numpy.linalg.cond gives {cond(H_n, inf)}")
    print()

```

```

H_2 is
[[1.          0.5         ]
 [0.5         0.33333333]]
with infinity norm 1.5
and its inverse is
[[ 4. -6.]
 [-6. 12.]]
with infinity norm 18.000000000000007
Thus the condition number of H_2 is 27.000000000000001
For comparison, numpy.linalg.cond gives 27.000000000000001

H_3 is
[[1.          0.5         0.33333333]
 [0.5         0.33333333 0.25        ]
 [0.33333333 0.25        0.2         ]]
with infinity norm 1.8333333333333333
and its inverse is
[[ 9. -36.  30.]
 [-36. 192. -180.]
 [ 30. -180. 180.]]
with infinity norm 408.000000000000165
Thus the condition number of H_3 is 748.0000000000003
For comparison, numpy.linalg.cond gives 748.0000000000027

H_4 is
[[1.          0.5         0.33333333 0.25        ]
 [0.5         0.33333333 0.25        0.2         ]
 [0.33333333 0.25        0.2         0.16666667]

```

(continues on next page)

(continued from previous page)

```

[0.25      0.2      0.16666667 0.14285714]]
with infinity norm 2.083333333333333
and its inverse is
[[ 16. -120.  240. -140.]
 [ -120. 1200. -2700. 1680.]
 [ 240. -2700.  6480. -4200.]
 [ -140. 1680. -4200. 2800.]]
with infinity norm 13619.999999996671
Thus the condition number of H_4 is 28374.999999993062
For comparison, numpy.linalg.cond gives 28374.999999997388

H_5 is
[[1.      0.5      0.33333333 0.25      0.2      ]
 [0.5      0.33333333 0.25      0.2      0.16666667]
 [0.33333333 0.25      0.2      0.16666667 0.14285714]
 [0.25      0.2      0.16666667 0.14285714 0.125      ]
 [0.2      0.16666667 0.14285714 0.125      0.11111111]]
with infinity norm 2.2833333333333333
and its inverse is
[[ 2.500e+01 -3.000e+02  1.050e+03 -1.400e+03  6.300e+02]
 [-3.000e+02  4.800e+03 -1.890e+04  2.688e+04 -1.260e+04]
 [ 1.050e+03 -1.890e+04  7.938e+04 -1.176e+05  5.670e+04]
 [-1.400e+03  2.688e+04 -1.176e+05  1.792e+05 -8.820e+04]
 [ 6.300e+02 -1.260e+04  5.670e+04 -8.820e+04  4.410e+04]]
with infinity norm 413279.999999164
Thus the condition number of H_5 is 943655.9999980911
For comparison, numpy.linalg.cond gives 943655.9999999335

```

Next, experiment with solving equations, to compare residuala with actual errors.

I will use the testing strategy of starting with a known solution x , from which the right-hand side b is computed; then slight simulated error is introduced to b . Running this repeatedly with use of different random “errors” gives an idea of the actual error.

```

for n in range(2,6):
    print(f"{n=}")
    H_n = hilbert(n)
    x = np.linspace(1.0, n, n)
    print(f"x is {x}")
    b = H_n @ x
    print(f"b is {b}")
    error_scale = 1e-8
    b_imperfect = b + 2.0 * error_scale * (random(n) - 0.5) # add random "errors"
    # between -error_scale and error_scale
    print(f"b has been slightly changed to {b_imperfect}")
    x_computed = solvelinearsystem(H_n, b_imperfect)
    residual = b - H_n @ x_computed
    relative_backward_error = norm(residual, inf)/norm(b, inf)
    print(f"The residual maximum norm is {norm(residual, inf)}")
    print(f"and the relative backward error ||r||/||b|| is {relative_backward_error:0.4}")
    absolute_error = norm(x - x_computed, inf)
    relative_error = absolute_error/norm(x, inf)
    print(f"The absolute error is {absolute_error:0.4}")
    print(f"The relative error is {relative_error:0.4}")
    error_bound = cond(H_n, inf) * relative_backward_error

```

(continues on next page)

(continued from previous page)

```

print(f"For comparison, the relative error bound from the formula above is {error_
bound:0.4}")
print(f"\nBeware: the relative error is larger than the relative backward error_
by a factor {relative_error/relative_backward_error:0.8}")
print()

```

```

n=2
x is [1. 2.]
b is [2.          1.16666667]
b has been slightly changed to [2.          1.16666667]
The residual maximum norm is 7.2687527108428185e-09
and the relative backward error ||r||/||b|| is 3.634e-09
The absolute error is 9.218e-08
The relative error is 4.609e-08
For comparison, the relative error bound from the formula above is 9.813e-08

Beware: the relative error is larger than the relative backward error by a factor_
12.682367

n=3
x is [1. 2. 3.]
b is [3.          1.91666667 1.43333333]
b has been slightly changed to [3.00000001 1.91666667 1.43333333]
The residual maximum norm is 7.658041312197383e-09
and the relative backward error ||r||/||b|| is 2.553e-09
The absolute error is 1.367e-06
The relative error is 4.555e-07
For comparison, the relative error bound from the formula above is 1.909e-06

Beware: the relative error is larger than the relative backward error by a factor_
178.4471

n=4
x is [1. 2. 3. 4.]
b is [4.          2.71666667 2.1          1.72142857]
b has been slightly changed to [4.          2.71666667 2.1          1.72142857]
The residual maximum norm is 6.2034000158917024e-09
and the relative backward error ||r||/||b|| is 1.551e-09
The absolute error is 5.916e-05
The relative error is 1.479e-05
For comparison, the relative error bound from the formula above is 4.401e-05

Beware: the relative error is larger than the relative backward error by a factor_
9536.5336

n=5
x is [1. 2. 3. 4. 5.]
b is [5.          3.55          2.81428571 2.34642857 2.01746032]
b has been slightly changed to [5.00000001 3.55          2.81428571 2.34642857 2.
01746032]
The residual maximum norm is 9.61878221517054e-09
and the relative backward error ||r||/||b|| is 1.924e-09
The absolute error is 0.000799
The relative error is 0.0001598
For comparison, the relative error bound from the formula above is 0.001815

```

(continues on next page)

(continued from previous page)

Beware: the relative error is larger than the relative backward error by a factor ≈ 83063.018

We see in these experiments that:

- As the condition number increases, the relative error becomes increasingly larger than the backward error computed from the residual.
- It is less than the above bound $\text{Rel}(x_a) = \frac{\|x - x_a\|}{\|x\|} \leq \kappa(A) \frac{\|r\|}{\|b\|}$, and typically quite a bit less.

3.7 Iterative Methods for Simultaneous Linear Equations

References:

- Section 2.5 *Iterative Methods* in [Sauer, 2022], sub-sections 2.5.1 to 2.5.3.
- Chapter 7 *Iterative Techniques in Linear Algebra* in [Burden *et al.*, 2016], sections 7.1 to 7.3.
- Section 8.4 in [Chenney and Kincaid, 2012].

3.7.1 Introduction

This topic is a huge area, with lots of ongoing research; this section just explores the first few methods in the field:

1. The Jacobi Method.
2. The Gauss-Seidel Method.

The next three major topics for further study are:

3. The Method of Successive Over-Relaxation (“SOR”). This is usually done as a modification of the Gauss-Seidel method, though the strategy of “over-relaxation” can also be applied to other iterative methods such as the Jacobi method.
4. The Conjugate Gradient Method (“CG”). This is beyond the scope of this course; I mention it because in the realm of solving linear systems that arise in the solution of differential equations, CG and SOR are the basis of many of the most modern, advanced methods.
5. Preconditioning.

3.7.2 The Jacobi method

The basis of the Jacobi method for solving $Ax = b$ is splitting A as $D + R$ where D is the diagonal of A :

$$\begin{aligned} d_{i,i} &= a_{i,i} \\ d_{i,j} &= 0, \quad i \neq j \end{aligned}$$

so that $R = A - D$ has

$$\begin{aligned} r_{i,i} &= 0 \\ r_{i,j} &= a_{i,j}, \quad i \neq j \end{aligned}$$

Visually

$$D = \begin{bmatrix} a_{11} & 0 & 0 & \dots \\ 0 & a_{22} & 0 & \dots \\ 0 & 0 & a_{33} & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

It is easy to solve $Dx = b$: the equations are just $a_{ii}x_i = b_i$ with solution $x_i = b_i/a_{ii}$.

Thus we rewrite the equation $Ax = Dx + Rx = b$ in the fixed point form

$$Dx = b - Rx$$

and then use the familiar fixed point iteration strategy of inserting the current approximation at right and solving for the new approximation at left:

$$Dx^{(k)} = b - Rx^{(k-1)}$$

Note: We could make this look closer to the standard fixed-point iteration form $x_k = g(x_{k-1})$ by dividing out D to get

$$x^{(k)} = D^{-1}(b - Rx^{(k-1)}),$$

but — as is often the case — it will be better to avoid matrix inverses by instead solving this easy system. This “inverse avoidance” becomes far more important when we get to the Gauss-Seidel method!

Exercise 1: Implement and test the Jacobi method

Write and test Python functions for this.

A) As usual start with a most basic version that does a fixed number of iterations

```
x = jacobi_basic(A, b, n)
```

B) Then refine this to apply an error tolerance, but also avoiding infinite loops by imposing an upper limit on the number of iterations:

```
x = jacobi(A, b, errorTolerance, maxIterations)
```

Test this with the matrices of form T below for several values of n , increasingly geometrically. To be cautious initially, try $n = 2, 4, 8, 16, \dots$

3.7.3 The underlying strategy

To analyse the Jacobi method — answering questions like for which matrices it works, and how quickly it converges — and also to improve on it, it helps to described a key strategy underlying it, which is this: approximate the matrix A by another one E one that is easier to solve with, chosen so that the discrepancy $R = A - E$ is small enough. Thus, repeatedly solving the new easier equations $Ex^{(k)} = b^{(k)}$ plays a similar role to repeatedly solving tangent line approximations in Newton’s method.

Of course to be of any use, E must be somewhat close to A ; the remainder R must be small enough. We can make this requirement precise with the use of *matrix norms* introduced in *Error bounds for linear algebra, condition numbers, matrix norms, etc.* and an upgrade of the contraction mapping theorem seen in *Solving Equations by Fixed Point Iteration (of Contraction Mappings)*.

Thus consider a general *splitting* of A as $A = E + R$. As above, we rewrite $Ax = Ex + Rx = b$ as $Ex = b - Rx$ and thence as $x = E^{-1}b - (E^{-1}R)x$. (It is alright to use the matrix inverse here, since we are not actually computing it; only using it for a theoretical argument!) The fixed point iteration form is thus

$$x^{(k)} = g(x^{(k-1)}) = c - Sx^{(k-1)}$$

where $c = E^{-1}b$ and $S = E^{-1}R$.

For vector-valued functions we extend the previous [Definition 1.2.2](#) in Section *Solving Equations by Fixed Point Iteration (of Contraction Mappings)* as:

Definition 2.7.1 (Vector-valued contraction mapping)

For a set D of vectors in $\mathbb{R}^n$, a mapping $g : D \rightarrow D$ is called a *contraction* or *contraction mapping* if there is a constant $C < 1$ such that

$$\|g(x) - g(y)\| \leq C\|x - y\|$$

for any x and y in D . We then call C a *contraction constant*.

Next, the contraction mapping theorem [Theorem 1.2.1](#) extends to

Theorem 2.7.1 (Contraction mapping theorem for vector-valued functions)

- Any contraction mapping g on a closed, bounded set $D \in \mathbb{R}^n$ has exactly one fixed point p in D .
- This can be calculated as the limit $p = \lim_{k \rightarrow \infty} x^{(k)}$ of the iteration sequence given by $x^{(k)} = g(x^{(k-1)})$ for *any* choice of the starting point $x^{(0)} \in D$.
- The errors decrease at a guaranteed minimum speed: $\|x^{(k)} - p\| \leq C\|x^{(k-1)} - p\|$, so $\|x^{(k)} - p\| \leq C^k\|x^{(0)} - p\|$.

With this, it turns out that the above iteration converges if S is “small enough” in the sense that $\|S\| = C < 1$ — and it is enough that this works for *any* choice of matrix norm!

Theorem 2.7.2

If $S := E^{-1}R = E^{-1}A - I$ has $\|S\| = C < 1$ for any choice of matrix norm, then the iterative scheme $x^{(k)} = c - Sx^{(k-1)}$ with $c = E^{-1}b$ converges to the solution of $Ax = b$ for any choice of the initial approximation $x^{(0)}$. (Aside: the zero vector is an obvious and popular choice for $x^{(0)}$.)

Incidentally, since this condition guarantees that there exists a unique solution to $Ax = b$, it also shows that A is non-singular.

Proof. (sketch)

The main idea is that for $g(x) = c - Sx$,

$$\|g(x) - g(y)\| = \|(c - Sx) - (c - Sy)\| = \|S(y - x)\| \leq \|S\|\|y - x\| \leq C\|x - y\|,$$

so with $C < 1$, it is a contraction.

(The omitted more “technical” detail is to find a suitable bounded domain D that all the iterates $x^{(k)}$ stay inside it.)

What does this say about the Jacobi method?

For the Jacobi method, $E = D$ so E^{-1} is the diagonal matrix with elements $1/a_{i,i}$ on the main diagonal, zero elsewhere. The product $E^{-1}A$ then multiplies each row i of A by $1/a_{i,i}$, giving

$$E^{-1}A = \begin{bmatrix} 1 & a_{1,2}/a_{1,1} & a_{1,3}/a_{1,1} & \cdots \\ a_{2,1}/a_{2,2} & 1 & a_{2,3}/a_{2,2} & \cdots \\ a_{3,1}/a_{3,3} & a_{3,2}/a_{3,3} & 1 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

so that subtracting the identity matrix to get S cancels the ones on the main diagonal:

$$S = E^{-1}A - I = \begin{bmatrix} 0 & a_{1,2}/a_{1,1} & a_{1,3}/a_{1,1} & \cdots \\ a_{2,1}/a_{2,2} & 0 & a_{2,3}/a_{2,2} & \cdots \\ a_{3,1}/a_{3,3} & a_{3,2}/a_{3,3} & 0 & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

Here is one of many places that using the maximum-norm, a.k.a. ∞ -norm, makes life much easier! Recalling that this is given by

$$\|A\|_{\infty} = \max_{i=1}^n \left(\sum_{j=1}^n |a_{i,j}| \right),$$

- First, sum the absolute values of elements in each row i ; with the common factor $1/|a_{i,i}|$, this gives $(|a_{i,1}| + |a_{i,2}| + \cdots |a_{i,i-1}| + |a_{i,i+1}| + \cdots |a_{i,n}|) / |a_{i,i}|$.

Such a sum, skipping index $j = i$, can be abbreviated as

$$\left(\sum_{1 \leq j \leq n, j \neq i} |a_{i,j}| \right) / |a_{i,i}|$$

- Then get the norm as the maximum of these:

$$C = \|E^{-1}A\|_{\infty} = \max_{i=1}^n \left[\left(\sum_{1 \leq j \leq n, j \neq i} |a_{i,j}| \right) / |a_{i,i}| \right]$$

and the contraction condition $C < 1$ becomes the requirement that each of these n “row sums” is less than 1:

Multiplying each of the inequalities by the denominator $|a_{i,i}|$ gives n conditions

$$\left(\sum_{1 \leq j \leq n, j \neq i} |a_{i,j}| \right) < |a_{i,i}|$$

This is strict diagonal dominance, as in [Definition 2.1.1](#) in the section *Row Reduction/Gaussian Elimination*, and as discussed there, one way to think of this is that such a matrix A is close to its main diagonal D , which is the intuitive condition that the approximation of A by D as done in the Jacobi method is “good enough”.

And indeed, combining this result with [Theorem 2.7.2](#) gives:

Theorem 2.7.3 (Convergence of the Jacobi method)

The Jacobi Method converges if A is strictly diagonally dominant, for any initial approximation $x^{(0)}$.

Further, the error goes down by at least a factor of $\|I - D^{-1}A\|$ at each iteration.

By the way, other matrix norms give other conditions guaranteeing convergence; perhaps the most useful of these others is that it is also sufficient for A to be column-wise strictly diagonally dominant as in [Definition 2.1.2](#).

3.7.4 The Gauss-Seidel method

To recap, two key ingredients for a splitting $A = E + R$ to be useful are that

- the matrix E is “easy” to solve with, and
- it is not too far from A .

The Jacobi method choice of E being the main diagonal of A strongly emphasizes the “easy” part, but we have seen another larger class of matrices for which it is fairly quick and easy to solve $Ex = b$: *triangular matrices*, which can be solved with forward or backward substitution, not needing row reduction.

The Gauss-Seidel Method takes E be the lower triangular part of A , which intuitively leaves more of its entries closer to A and makes the remainder $R = A - E$ “smaller”.

To discuss this and other splittings, we write the matrix as $A = L + D + U$ where:

- D is the diagonal of A , as for Jacobi
- L is the strictly lower diagonal part of A (just the elements with $i > j$)
- U is the strictly upper diagonal part of A (just the elements with $i < j$)

That is,

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & \dots \\ a_{2,1} & a_{2,2} & a_{2,3} & \dots \\ a_{3,1} & a_{3,2} & a_{3,3} & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & \dots \\ a_{2,1} & 0 & 0 & \dots \\ a_{3,1} & a_{3,2} & 0 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} + \begin{bmatrix} a_{1,1} & 0 & 0 & \dots \\ 0 & a_{2,2} & 0 & \dots \\ 0 & 0 & a_{3,3} & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} + \begin{bmatrix} 0 & a_{1,2} & a_{1,3} & \dots \\ 0 & 0 & a_{2,3} & \dots \\ 0 & 0 & 0 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix} = L + D + U$$

Thus $R = L + U$ for the Jacobi method.

So now we use $E = L + D$, which will be called A_L , the *lower triangular part of A* , and the remainder is $R = U$. The fixed point form becomes

$$A_L x = b - Ux$$

giving the fixed point iteration

$$A_L x^{(k)} = b - Ux^{(k-1)}$$

Here we definitely do not use the inverse of A_L when calculating! Instead, solve with forward substitution.

However to analyse convergence, the mathematical form

$$x^{(k)} = A_L^{-1}b - (A_L^{-1}U)x^{(k-1)}$$

is useful: the iteration map is now $g(x) = c - Sx$ with $c = (L + D)^{-1}b$ and $S = (L + D)^{-1}U$.

Arguing as above, we see that convergence is guaranteed if $\|(L + D)^{-1}U\| < 1$. However it is not so easy in general to get a formula for $\|(L + D)^{-1}U\|$; what one can get is slightly disappointing in that, despite the $R = U$ here being in some sense “smaller” than the $R = L + U$ for the Jacobi method, the general convergence guarantee looks no better:

Theorem 2.7.4 (Convergence of the Gauss-Seidel method)

The Gauss-Seidel method converges if A is strictly diagonally dominant, for any initial approximation $x^{(0)}$.

However, in practice the convergence rate as given by $C = C_{GS} = \|(L + D)^{-1}U\|$ is often better than for the $C = C_J = \|D^{-1}(L + U)\|$ for the Jacobi method.

Sometimes this reduces the number of iterations enough to outweigh the extra computational effort involved in each iteration and make this faster overall than the Jacobi method — but not always.

Exercise 2: Implement and test the Gauss-Seidel method, and compare to Jacobi

Do the two versions as above and use the same test cases.

Then compare the speed/cost of the two methods: one way to do this is by using Python's “stop watch”, function `time`. `time`: see the description of [Python module time](#) in the Python manual.

3.7.5 A family of test cases, arising from boundary value problems for differential equations

The **tri-diagonal** matrices T of the form

$$\begin{aligned} t_{i,i} &= 1 + 2h^2 \\ t_{i,i+1} &= t_{i+1,i} = -h^2 \\ t_{i,j} &= 0, \quad |i - j| > 1 \end{aligned}$$

and variants of this arise in the solutions of boundary value problems for ODEs like

$$\begin{aligned} -u''(x) + Ku &= f(x), \quad a \leq x \leq b \\ u(a) &= u(b) = 0 \end{aligned}$$

and related problems for partial differential equations.

Thus these provide useful initial test cases — usually with $h = (b - a)/n$.

3.8 Faster Methods for Solving $Ax = b$ for Tridiagonal and Banded matrices, and Strict Diagonal Dominance

Reference:

Section 6.6 *Special Types of Matrices* in [Burden *et al.*, 2016], the sub-sections on *Band Matrices* and *Tridiagonal Matrices*.

3.8.1 Tridiagonal systems

Differential equations often lead to the need to solve systems of equations $Tx = b$ where the matrix T has this special form:

Definition 2.8.1 (Tridiagonal matrix)

A matrix T is **tridiagonal** if the only non-zero elements are on the main diagonal and the diagonals adjacent to it on either side, so that $T_{i,j} = 0$ if $|i - j| > 1$. That is, the system looks like:

$$Tx = \begin{bmatrix} d_1 & u_1 & & & & \\ l_1 & d_2 & u_2 & & & \\ & l_2 & d_3 & u_3 & & \\ & & \ddots & \ddots & \ddots & \\ & & & l_{n-2} & d_{n-1} & u_{n-1} \\ & & & & l_{n-1} & d_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

with all “missing” entries being zeros. The notation used here suggests one efficient way to store such a matrix: as three 1D arrays d , l and u .

(Such equations also arise in other important situations, such as *spline interpolation*)

It can be verified that LU factorization preserves all the non-zero values, so that the Doolittle algorithm — if it succeeds without any division by zero — gives $T = LU$ with the form

$$L = \begin{bmatrix} 1 & & & & & \\ L_1 & 1 & & & & \\ & L_2 & 1 & & & \\ & & \ddots & \ddots & & \\ & & & L_{n-2} & 1 & \\ & & & & L_{n-1} & 1 \end{bmatrix}, U = \begin{bmatrix} D_1 & u_1 & & & & \\ & D_2 & u_2 & & & \\ & & D_3 & u_3 & & \\ & & & \ddots & \ddots & \\ & & & & D_{n-1} & u_{n-1} \\ & & & & & D_n \end{bmatrix}$$

Note that the first non-zero element in each column is unchanged, as with a full matrix, but now it means that the upper diagonal elements u_i are unchanged.

Again, one way to describe and store this information is with just the two new 1D arrays L and D , along with the unchanged array u .

3.8.2 Algorithms

Algorithm 2.8.1 (LU factorization)

```

D1 = d1
for i from 2 to n
    Li-1 = li-1/Di-1
    Di = di - Li-1ui-1
end

```

Algorithm 2.8.2 (Forward substitution)

```

c1 = b1
for i from 2 to n
    ci = bi - Li-1ci-1
end

```

Algorithm 2.8.3 (Backward substitution)

```

xn = cn/Dn
for i from n-1 down to 1
    xi = (ci - uixi+1)/Di
end

```

3.8.3 Generalizing to banded matrices

As we have seen, approximating derivatives to higher order of accuracy and approximating derivatives of order greater than two requires more than three nodes, but the locations needed are all close to the ones where the derivative is being approximated. For example, the simplest symmetric approximation of the fourth derivative $D^4 f(x)$ used values from $f(x - 2h)$ to $f(x + 2h)$. Then row i of the corresponding matrix has all its non-zero elements at locations $(i, i - 2)$ to $(i, i + 2)$: the non-zero elements lie in the narrow “band” where $|i - j| \leq 2$, and thus on five “central” diagonals.

This is a **penta-digonal matrix**, and an example of the larger class of **banded matrices**: ones in which all the non-zero elements have indices $-p \leq j - i \leq q$ for p and q smaller than n — usually far smaller; $p = q = 2$ for a penta-digonal matrix.

Let us recap the general Doolittle algorithm for computing an LU factorization:

Algorithm 2.8.4 (Doolittle algorithm for computing an LU factorization)

The top row is unchanged:

```
for j from 1 to n
```

$$u_{1,j} = a_{1,j}$$

```
end
```

The left column requires no sums:

```
for i from 2 to n
```

$$l_{i,1} = a_{i,1}/u_{1,1}$$

```
end
```

The main loop: for k from 2 to n

```
  for j from k to n
```

$$u_{k,j} = a_{k,j} - \sum_{s=1}^{k-1} l_{k,s} u_{s,j}$$

```
  end
```

```
  for i from k+1 to n
```

$$l_{i,k} = (a_{i,k} - \sum_{s=1}^{k-1} l_{i,s} u_{s,k}) / u_{k,k}$$

```
  end
```

```
end
```

Eliminating redundant calculation in the above

With a banded matrix, many of the entries at right are zero, particularly in the two sums, which is where most of the operations are. Thus we can rewrite, exploiting the fact that all elements with indices $j - i < -p$ or $j - i > q$ are zero. To start with, the top diagonal is not modified, as already noted for the tridiagonal case: $u_{k,k+q} = a_{k,k+q}$ for $1 \leq k \leq n - q$.

Algorithm 2.8.5 (LU factorization of a banded matrix)

The top row is unchanged:

```
for j from 1 to 1+q
```

```

     $u_{1,j} = a_{1,j}$ 
end
The top non-zero diagonal is unchanged:
for k from 1 to n - q
     $u_{k,k+q} = a_{k,k+q}$ 
end
The left column requires no sums:
for i from 2 to 1+p
     $l_{i,1} = a_{i,1}/u_{1,1}$ 
end
The main loop:
for k from 2 to n
    for j from k to min(n, k+q-1)
        
$$u_{k,j} = a_{k,j} - \sum_{s=\max(1,k-p,j-q)}^{k-1} l_{k,s}u_{s,j}$$

    end
    for i from k+1 to min(n, k+p-1)
        
$$l_{i,k} = \left( a_{i,k} - \sum_{s=\max(1,i-p,k-q)}^{k-1} l_{i,s}u_{s,k} \right) / u_{k,k}$$

    end
end
end

```

It is common for a banded matrix to have equal band-width on either side, $p = q$, as with tridiagonal and pentadiagonal matrices. Then the algorithm is somewhat simpler:

Algorithm 2.8.6 (LU factorization of a banded matrix, $p = q$)

```

The top row is unchanged:
for j from 1 to 1+p
     $u_{1,j} = a_{1,j}$ 
end
The top non-zero diagonal is unchanged: for k from 1 to n - p
     $u_{k,k+p} = a_{k,k+p}$ 
end
The left column requires no sums:
for i from 2 to 1+p
     $l_{i,1} = a_{i,1}/u_{1,1}$ 
end
The main loop:

```

```

for k from 2 to n
  for j from k to min(n, k+p-1)
    
$$u_{k,j} = a_{k,j} - \sum_{s=\max(1,j-p)}^{k-1} l_{k,s} u_{s,j}$$

  end
  for i from k+1 to min(n, k+p)
    
$$l_{i,k} = \left( a_{i,k} - \sum_{s=\max(1,i-p)}^{k-1} l_{i,s} u_{s,k} \right) / u_{k,k}$$

  end
end
end

```

3.8.4 Strict diagonal dominance helps again

These algorithms for banded matrices do no pivoting, and that is highly desirable, because pivoting creates non-zero elements outside the “band” and so can force one back to the general algorithm. Fortunately, we have seen one case where this is fine: the matrix being either row-wise or column-wise strictly diagonally dominant.

3.9 Computing Eigenvalues and Eigenvectors: the Power Method, and a bit beyond

References:

- Section 12.1 *Power Iteration Methods* of [Sauer, 2022].
- Section 7.2 *Eigenvalues and Eigenvectors* of [Burden *et al.*, 2016].
- Chapter 8, *More on Linear Equations* of [Chenney and Kincaid, 2012], in particular section 3 *Power Method*, and also section 2 *Eigenvalues and Eigenvectors* as background reading.

The *eigenproblem* for a square $n \times n$ matrix A is to compute some or all non-trivial solutions of

$$A\vec{v} = \lambda\vec{v}.$$

(By non-trivial, I mean to exclude $\vec{v} = 0$, which gives a solution for any λ .) That is, to compute the eigenvalues λ (of which generically there are n , but sometimes less) and the eigenvectors $\vec{v}$ corresponding to each.

With eigenproblems, and particularly those arising from differential equations, one often needs only the few smallest and/or largest eigenvalues. For these, the *power method* described next can be adapted, leading to the *shifted inverse power method*.

Here we often restrict our attention to the case of a real *symmetric* matrix ($A^T = A$, or $A_{ij} = A_{ji}$), or a Hermitian matrix ($A^T = A^*$), for which many things are a bit simpler:

- all eigenvalues are real,
- for symmetric matrices, all eigenvectors are also real,
- there is a complete set of orthogonal eigenvectors $\vec{v}_k$, $1 \leq k \leq n$ that form a basis for all vectors, and so on.

However, the methods described here can be used more generally, or can be made to work with minor adjustments.

The eigenvalue are roots of the characteristic polynomial, $\det(A - \lambda I)$; repeated roots are possible, and they will all be named, so there are always values λ_i , $1 \leq i \leq n$. Here, these eigenvalues will be enumerated in decreasing order of magnitude:

$$|\lambda_1| \geq |\lambda_2| \cdots \geq |\lambda_n|.$$

Generically, all the magnitudes are different, which makes things works more easily, so that will sometimes be assumed while developing the intuition of the method.

3.9.1 The Power Method

The basic tool is the *Power Method*, which will usually *but not always* succeed in computing the eigenvalue of largest magnitude, λ_1 , and a corresponding eigenvector $\vec{v}_1$. Its success mainly involves assuming there being a unique largest eigenvalue: $\lambda_1 > \lambda_i$ for $i > 1$.

In its simplest form, one starts with a unit-length vector $\vec{x}^0$, so $\|\vec{x}^0\| = 1$, constructs the successive multiples $\vec{y}^k = A^k \vec{x}^0$ by successive multiplications, and rescales at each stage to the unit vectors $\vec{x}^k = \vec{y}^k / \|\vec{y}^k\|$.

Note that $\vec{y}^{k+1} = A \vec{x}^k$, so that once $\vec{x}^k$ is approximately an eigenvector for eigenvalue λ , $\vec{y}^{k+1} \approx \lambda \vec{x}^k$, leading to the eigenvalue approximation

$$r^{(k)} := \langle \vec{y}^{k+1}, \vec{x}^k \rangle \approx \langle \lambda \vec{x}^k, \vec{x}^k \rangle \approx \lambda$$

Remark 2.9.1 (dot products in Python)

Here and below, I use $\langle \vec{a}, \vec{b} \rangle$ to denote the inner product (a.k.a. *dot product*) of two vectors; with Numpy arrays this is given by `a . dot (b)`.

Algorithm 2.9.1 (A basic version of the power method)

Choose initial vector $\vec{y}_0$, maybe with a random number generator.

Normalize to $\vec{x}^0 = \vec{y}^0 / \|\vec{y}^0\|$.

for k from 0 to k_{max}

$$\vec{y}^{k+1} = A \vec{x}^k$$

$$r^{(k)} = \langle \vec{y}^{k+1}, \vec{x}^k \rangle$$

$$\vec{x}^{k+1} = \vec{y}^{k+1} / \|\vec{y}^{k+1}\|$$

end

The final values of $r^{(k)}$ and $\vec{x}^k$ approximate λ_1 and $\vec{v}_1$ respectively.

Exercise 1

Implement this algorithm and test it on the real, symmetric matrix

$$A = \begin{bmatrix} 3 & 1 & 1 \\ 1 & 8 & 1 \\ 1 & 1 & 4 \end{bmatrix}$$

This all real eigenvalues, all within 2 of the diagonal elements (this claim should be explained as part of the project write-up), so start with it.

As a debugging strategy, you could replace all those off-diagonal ones by a small value δ :

$$A_\delta = \begin{bmatrix} 3 & \delta & \delta \\ \delta & 8 & \delta \\ \delta & \delta & 4 \end{bmatrix}$$

Then the [Gershgorin circle theorem](#) ensures that each eigenvalue is within 2δ of an entry on the main diagonal. Furthermore, if δ is small enough that the circles of radius 2δ centered on the diagonal elements do not overlap, then there is one eigenvalue in each circle.

You could even start with $\delta = 0$, for which you know exactly the eigenvalues: they are the diagonal elements.

Here and below you could check your work with Numpy, using function `numpy.linalg.eig(A)`.

However, that is almost cheating, so note that there is also a backward error check: see how small $\|Av - \lambda v\|/\|v\|$ is.

```
import numpy as np
import numpy.linalg as la
```

```
help(la.eig)
```

```
delta = 0.01
A = np.array([[3, delta, delta], [delta, 8, delta], [delta, delta, 4]])
[eigenvalues, eigenvectors] = la.eig(A)
```

```
print(f"With {delta=}, so that A is")
print(A)
lambda_0 = eigenvalues[0]
lambda_1 = eigenvalues[1]
lambda_2 = eigenvalues[2]
# The eigenvectors are the columns of the output array `eigenvectors`, so:
v_0 = list(eigenvectors[:,0]) # Lists print more nicely than arrays!
v_1 = list(eigenvectors[:,1])
v_2 = list(eigenvectors[:,2])
print(f"The eigenvalues are {eigenvalues}")
print("and the eigenvalue-eigenvector pairs are")
print(f"{lambda_0=}, \t {v_0=}")
print(f"{lambda_1=}, \t {v_1=}")
print(f"{lambda_2=}, \t {v_2=}")
```

```
With delta=0.01, so that A is
[[3.  0.01 0.01]
 [0.01 8.  0.01]
 [0.01 0.01 4.  ]]
The eigenvalues are [2.99988041 8.0000451 4.00007449]
```

(continues on next page)

(continued from previous page)

```

and the eigenvalue-eigenvector pairs are
lambda_0=2.9998804099870666,          v_0=[-0.9999482535404538, 0.
↪0019798921714429545, 0.009978490285907787]
lambda_1=8.00004509976119,          v_1=[0.002004981562986897, 0.999994852570506, ↪
↪0.0025049713419308117]
lambda_2=4.000074490251736,          v_2=[0.009973479349182978, -0.
↪0025248484075824345, 0.9999470760246215]

```

Refinement: deciding the iteration count

Some details are omitted above; above all, how to decide the number of iterations.

One approach is to use the fact that an eigenvector-eigenvalue pair satisfies $A\vec{v} - \lambda\vec{v} = 0$, so the “residual norm”

$$\frac{\|A\vec{x}^k - r^{(k)}\vec{x}^k\|}{\|\vec{x}^k\|}, = \|\vec{y}^{k+1} - r^{(k)}\vec{x}^k\| \text{ since } \|\vec{x}^k\| = 1$$

is a measure of “relative backward error”.

Thus one could replace the above `for` loop by a `while` loop based on a condition like stopping when

$$\|\vec{y}^{k+1} - r^{(k)}\vec{x}^k\| \leq \epsilon.$$

Alternatively, keep the `for` loop, but exit early (with `break`) if this condition is met.

I generally recommend this `for-if-break` form for implementing iterative methods, because it makes avoidance of infinite loops simpler, and avoids the common `while` loop issue that you do not yet have an error estimate when the loop starts.

Exercise 2

Modify your code from Exercise 1 to implement this accuracy control.

3.9.2 The Inverse Power Method

The next step is to note that if A is nonsingular, its inverse $B = A^{-1}$ has the same eigenvectors, but with eigenvalues $\mu_i = 1/\lambda_i$.

Thus we can apply the power method to B in order to compute its largest eigenvalue, which is $\mu_n = 1/\lambda_n$, along with the corresponding eigenvector $\vec{v}_n$.

The main change to the above is that

$$\vec{y}^{k+1} = A^{-1}\vec{x}_k.$$

However, as usual one can (and should) avoid actually computing the inverse. Instead, express the above as the system of equations.

$$A\vec{y}^{k+1} = \vec{x}_k.$$

Here is an important case where the LU factorization method can speed things up greatly: a single LU factorization is needed, after which for each k one only has to do the far quicker forward and backward substitution steps: $O(n^2)$ cost for each iteration instead of $O(n^3/3)$.

Algorithm 2.9.2 (A basic version of the inverse power method)

Choose initial vector $\vec{y}_0$, maybe with a random number generator.

Normalize to $\vec{x}^0 = \vec{y}^0 / \|\vec{y}^0\|$.

Compute an LU factorization $LU = A$.

for k from 0 to k_{max}

 Solve $L\vec{z}^{k+1} = \vec{x}^k$

 Solve $U\vec{y}^{k+1} = \vec{z}^{k+1}$

$r^{(k)} = \langle \vec{y}^{k+1}, \vec{x}^k \rangle$

$\vec{x}^{k+1} = \vec{y}^{k+1} / \|\vec{y}^{k+1}\|$

end

(If all goes well) the final values of $r^{(k)}$ and $\vec{x}^k$ approximate λ_n and $\vec{v}_n$ respectively.

Exercise 3

Implement this basic algorithm (with a fixed iteration count, as in Example 1), and then create a second version that imposes an accuracy target (as in Example 2).

3.9.3 Getting other eigenvalues with the Shifted Inverse Power Method

The inverse power method computes the eigenvalue closest to 0; by *shifting*, we can compute the eigenvalue closest to any chosen value s . Then by searching various values of s , we can hope to find all the eigenvectors. As a variant, once we have λ_1 and λ_n , we can search nearby for other large or small eigenvalues: often the few largest and/or the few smallest are most important.

With a symmetric (or Hermitian) matrix, once the eigenvalue of largest magnitude, λ_1 is known, the rest are known to be real values in the interval $[-|\lambda_1|, |\lambda_1|]$, so we know roughly where to seek them.

The main idea here is that for any number s , matrix $A - sI$ has eigenvalues $\lambda_i - s$, with the same eigenvectors as A :

$$(A - sI)\vec{v}_i = (\lambda_i - s)\vec{v}_i$$

Thus, applying the inverse power method to $A - sI$ computes its largest eigenvalue γ , and then $\lambda = 1/(\gamma + s)$ is the eigenvalue of A closest to s .

Exercise 4

As above, implement this, probably starting with a fixed iteration count version.

For the test case above, some plausible initial choices for the shifts are each if the entries on the main diagonal, and as above, testing with A_s

3.9.4 Further topics: getting all the eigenvalues with the QR Method, etc.

The above methods are not ideal when many or all of the eigenvalues of a matrix are wanted; then a variety of more advanced methods have been developed, starting with the QR (factorization) Method.

We will not address the details of that method in this course, but one way to think about it for a symmetric matrix is that:

- The eigenvectors are orthogonal.
- Thus, if after computing λ_1 and $\vec{v}_1$, one uses the power iteration starting with $\vec{x}^{0,2}$ orthogonal to $\vec{v}_1$, then all the new iterates $\vec{x}^{k,2}$ will stay orthogonal, and one will get the eigenvector corresponding to the largest remaining eigenvalue: you get $\vec{v}_2$ and λ_2 .
- Continuing likewise, one can get the eigenvalues in descending order of magnitude.
- As a modification, one can do all these almost in parallel: at iteration k , have an approximation $\vec{x}^{k,i}$ for each λ_i and at each stage, got by adjusting these new approximations so that $\vec{x}^{k,i}$ is orthogonal to all the approximations $\vec{x}^{k,j}$, $j < i$, for all the previous (larger) eigenvalues. This uses a variant of the [Gram-Schmidt](#) method for orthogonalizing a set of vectors.

3.10 Solving Nonlinear Systems of Equations by generalizations of Newton's Method — a brief introduction

References:

- Section 2.7 *Nonlinear Systems of Equations* of [Sauer, 2022]; in particular Sub-section 2.7.1 *Multivariate Newton's Method*.
- Chapter 10 *Numerical Solution of Nonlinear Systems of Equations* of [Burden *et al.*, 2016]; in particular Sections 10.1 and 10.2.

3.10.1 Background

A system of simultaneous nonlinear equations

$$F(x) = 0, \quad F : \mathbb{R}^n \rightarrow \mathbb{R}^n$$

can be solved by a variant of Newton's method.

Remark (Some notation)

For the sake of emphasise analogies between results for vector-valued quantities and what we have already seen for real- or complex-valued quantities, several notational choices are made in these notes:

- Using the same notation for vectors as for real or complex numbers (no over arrows or bold face).
- Often denoting the derivative of function f as Df rather than f' , and higher derivatives as $D^p f$ rather than $f^{(p)}$. Partial derivatives of $f(x, y, \dots)$ are $D_x f$, $D_y f$, etc., and for vector arguments $x = (x_1, \dots, x_j, \dots, x_n)$, they are $D_{x_1} f, \dots, D_{x_j} f, \dots, D_{x_n} f$, or more concisely, $D_1 f, \dots, D_j f, \dots, D_n f$. (This also fits better with Julia code notation — even for partial derivatives.)
- Subscripts will mostly be reserved for components of vectors, labelling the terms of a sequence with superscripts, $x^{(k)}$ and such.
- Explicit division is avoided.

However, I use capital letters for vector-valued functions, for analogy to the use of capital letter for matrices.

Rewriting Newton's method according to this new style, $x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{Df(x^{(k)})}$

or to avoid explicit division and introducing the useful increment $\delta^{(k)} := x^{(k+1)} - x^{(k)}$,

$$Df(x^{(k)})(\delta^{(k)}) = f(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + \delta^{(k)}.$$

3.10.2 Newton's method iteration formula for systems

For vector valued functions, we will see in a while that an analogous result is true:

$$(D\mathbf{F}(x^{(k)}))(\delta^{(k)}) = \mathbf{F}(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + \delta^{(k)}$$

where $DF(x)$ is the $n \times n$ matrix of all the partial derivatives $(D_{x_j} F_i)(x)$ or $(D_j F_i)(x)$, where $x = (x_1, x_2, \dots, x_n)$.

Justification: linearization for function of several variables

To justify the above result, we need at least a case of Taylor's Theorem for functions of several variables, for both $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $F : \mathbb{R}^n \rightarrow \mathbb{R}^n$; just for linear approximations. This material from multi-variable calculus will be reviewed when we need it.

Warning: although mathematically this can be written with matrix inverses as

$$\mathbf{X}^{(k+1)} = \mathbf{X}^{(k)} - (D\mathbf{F}(\mathbf{X}^{(k)}))^{-1}(\mathbf{F}(\mathbf{X}^{(k)})),$$

evaluation of the inverse is in general about three times slower than solving the linear system, so is best avoided. (We have seen a good compromise; *Solving $Ax = b$ with LU factorization*, $A = LU$ the LU factorization of a matrix.)

Even avoiding matrix inversion, this involves repeatedly solving systems of n simultaneous linear equations in n unknowns, $Ax = b$, where the matrix A is $D\mathbf{F}(x^{(k)})$, and that will be seen to involve about $n^3/3$ arithmetic operations.

It also requires computing the new values of these n^2 partial derivatives at each iteration, also potentially with a cost proportional to n^3 .

When n is large, as is common with differential equations problems, this factor of n^3 indicates a potentially very large cost per iteration, so various modifications have been developed to reduce the computational cost of each iteration (with the trade-off being that more iterations are typically needed): so-called *quasi-Newton methods*.

POLYNOMIAL COLLOCATION AND APPROXIMATION

References:

- Chapter 3 *Interpolation* of [Sauer, 2022].
- Chapter 3 *Interpolation and Polynomial Approximation* of [Burden *et al.*, 2016].
- Chapter 4 of [Kincaid and Cheney, 1990].

4.1 Polynomial Collocation (Interpolation/Extrapolation)

References:

- Section 3.1 *Data and Interpolating Functions* in [Sauer, 2022].
- Section 3.1 *Interpolation and the Lagrange Polynomial* in [Burden *et al.*, 2016].
- Section 4.1 in [Cheney and Kincaid, 2012].

4.1.1 Introduction

Numerical methods for dealing with functions require describing them, at least approximately, using a finite list of numbers, and the most basic approach is to approximate by a polynomial. (Other important choices are rational functions and “trigonometric polynomials”: sums of multiples of sines and cosines.) Such polynomials can then be used to approximate derivatives and integrals.

The simplest idea for approximating $f(x)$ on domain $[a, b]$ is to start with a finite collection of **node** values $x_i \in [a, b]$, $0 \leq i \leq n$ and then seek a polynomial p which **collocates** with f at those values: $p(x_i) = f(x_i)$ for $0 \leq i \leq n$. Actually, we can put the function aside for now, and simply seek a polynomial that passes through a list of points (x_i, y_i) ; later we will achieve collocation with f by choosing $y_i = f(x_i)$.

In fact there are infinitely many such polynomials: given one, add to it any polynomial with zeros at all of the $n + 1$ nodes. So to make the problem well-posed, we seek the collocating polynomial of *lowest degree*.

Theorem 3.1.1 (Existence and uniqueness of a collocating polynomial)

Given $n + 1$ distinct values x_i , $0 \leq i \leq n$, and corresponding y -values y_i , there is a unique polynomial P of degree at most n with $P(x_i) = y_i$ for $0 \leq i \leq n$.

(*Note:* although the degree is typically n , it can be less; as an extreme example, if all y_i are equal to c , then $P(x)$ is that constant c .)

Historically there are several methods for finding P_n and proving its uniqueness, in particular, the *divided difference* method introduced by Newton and the *Lagrange polynomial* method. However for our purposes, and for most modern needs, a different method is easiest, and it also introduces a strategy that will be of repeated use later in this course: the **Method of Undertermined Coefficients** or **MUC**.

In general, this method starts by assuming that the function wanted is a sum of unknown multiples of a collection of known functions. Here, $P(x) = c_n x^n + c_{n-1} x^{n-1} + \dots + c_1 x + c_0 = \sum_{j=0}^n c_j x^j$.

(Note: any of the c_i could be zero, including c_n , in which case the degree is less than n .)

The unknown factors ($c_0 \dots c_n$) are the *undetermined coefficients*.

Next one states the problem as a system of equations for these undetermined coefficients, and solves them.

Here, we have $n + 1$ conditions to be met:

$$P(x_i) = \sum_{j=0}^n c_j x_i^j = y_i, \quad 0 \leq i \leq n$$

This is a system of $n + 1$ simultaneous linear equations in $n + 1$ unknowns, so the question of existence and uniqueness is exactly the question of whether the corresponding matrix is singular, and so is equivalent to the case of all $y_i = 0$ having only the solution with all $c_i = 0$.

Back in terms of polynomials, this is the claim that the only polynomial of degree at most n with distinct zeros $x_0 \dots x_n$ is the zero function. And this is true, because any non-trivial polynomial with those $n + 1$ distinct roots is of degree at least $n + 1$, so the only “degree n ” polynomial fitting this data is $P(x) \equiv 0$. The theorem is proven.

The proof of this theorem is completely constructive; it gives the only numerical method we need, and which is the one implemented in Numpy through the pair of functions `numpy.polyfit` and `numpy.polyval`. (Aside: here as in many places, Numpy mimics the names and functionality of corresponding Matlab tools.)

Briefly, the algorithm is this (indexing from 0 !)

- Create the $n + 1 \times n + 1$ matrix V with elements

$$v_{i,j} = x_i^j, \quad 0 \leq i \leq n, \quad 0 \leq j \leq n$$

and the $n + 1$ -element column vector y with elements y_i as above.

- Solve $Vc = y$ for the vector of coefficients c_j as above.

I use the name V because this is called the **Vandermonde Matrix**.

```
from numpy import array, linspace, zeros, zeros_like, exp
from matplotlib.pyplot import figure, plot, title, grid, legend
```

Example 3.1.1

As usual, I concoct a first example with known correct answer, by using a polynomial as f :

$$f(x) = 4 + 7x - 2x^2 - 5x^3 + 2x^4$$

using the nodes $x_0 = 1, x_1 = 2, x_2 = 0, x_3 = 3.3$ and $x_4 = 4$ (They do not need to be in order.)

```
def f(x):
    return 4 + 7*x - 2*x**2 - 5*x**3 + 2*x**4
```

```

xnodes = array([1., 2., 7., 5., 4.]) # They do not need to be in order
nnodes = len(xnodes)
n = nnodes-1
print(f"The x nodes 'x_i' are {xnodes}")
ynodes = zeros_like(xnodes)
for i in range(nnodes):
    ynodes[i] = f(xnodes[i])
print(f"The y values at the nodes are {ynodes}")

```

```

The x nodes 'x_i' are [1. 2. 7. 5. 4.]
The y values at the nodes are [ 7. 18. 5443. 1239. 448.]

```

The Vandermonde matrix:

```

V = zeros([nnodes, nnodes])
for i in range(nnodes):
    for j in range(nnodes):
        V[i,j] = xnodes[i]**j

```

Solve, using our functions seen in earlier sections and gathered in *Notebook for generating the module numericalMethods*

```

from numericalMethods import rowReduce, backwardSubstitution

```

```

(U, z) = rowReduce(V, ynodes)
c = backwardSubstitution(U, z)
print(f"The coefficients of P are {c}")

```

```

The coefficients of P are [ 4.  7. -2. -5.  3.]

```

We can also check the resulting values of the polynomial:

```

P = c[0] + c[1]*xnodes + c[2]*xnodes**2 + c[3]*xnodes**3 + c[4]*xnodes**4

```

```

for (x, y, P_i) in zip(xnodes, ynodes, P):
    print(f"P({x}) should be {y}; it is {P_i}")

```

```

P(1.0) should be 7.0; it is 7.0
P(2.0) should be 18.0; it is 18.0
P(7.0) should be 5443.0; it is 5443.0
P(5.0) should be 1239.0; it is 1239.0
P(4.0) should be 448.0; it is 448.0

```

4.1.2 Functions for computing the coefficients and evaluating the polynomials

We will use this procedure several times, so it time to put it into a functions — and add a pretty printer for polynomials.

```
def fitPolynomial(x, y):
    """Compute the coefficients  $c_i$  of the polynomial of lowest degree that
    collocates the points  $(x[i], y[i])$ .
    These are returned in an array  $c$  of the same length as  $x$  and  $y$ , even if the
    degree is less than the normal  $\text{length}(x)-1$ ,
    in which case the array has some trailing zeroes.
    The polynomial is thus  $p(x) = c[0] + c[1]x + \dots c[d] x^d$  where  $n = \text{length}(x)-1$ ,
    the nominal degree.
    """
    nnodes = len(x)
    n = nnodes - 1
    V = zeros([nnodes, nnodes])
    for i in range(nnodes):
        for j in range(nnodes):
            V[i,j] = x[i]**j
    (U, z) = rowReduce(V, y)
    c = backwardSubstitution(U, z)
    return c
```

```
def evaluatePolynomial(x, coeffs):
    # Evaluate the polynomial with coefficients in coeffs at the points in x.
    npoints = len(x)
    ncoeffs = len(coeffs)
    n = ncoeffs - 1
    powers = linspace(0, n, n+1)
    y = zeros_like(x)
    for i in range(npoints):
        y[i] = sum(coeffs * x[i]**powers)
    return y
```

```
def showPolynomial(c):
    print("P(x) = ", end="")
    n = len(c)-1
    print(f"{c[0]:.4}", end="")
    if n > 0:
        coeff = c[1]
        if coeff > 0:
            print(f" + {coeff:.4}x", end="")
        elif coeff < 0:
            print(f" - {-coeff:.4}x", end="")
    if n > 1:
        for j in range(2, len(c)):
            coeff = c[j]
            if coeff > 0:
                print(f" + {coeff:.4}x^{j}", end="")
            elif coeff < 0:
                print(f" - {-coeff:.4}x^{j}", end="")
    print()
```

While debugging, redo the first example:

```
c_new = fitPolynomial(xnodes, ynodes)
print(c_new)
```

```
[ 4.  7. -2. -5.  3.]
```

```
P_i_new = evaluatePolynomial(xnodes, c_new)
print(P_i_new)
```

```
[ 7.  18. 5443. 1239. 448.]
```

```
print(f"The values of P(x_i) are {P_i_new}")
```

```
The values of P(x_i) are [ 7.  18. 5443. 1239. 448.]
```

```
showPolynomial(c_new)
```

```
P(x) = 4.0 + 7.0x - 2.0x^2 - 5.0x^3 + 3.0x^4
```

Example 3.1.2 ($f(x)$ not a polynomial of degree $\leq n$)

Make an exact fit impossible by using the same function but using only four nodes and reducing the degree of the interpolating P to three: $x_0 = 1, x_1 = 2, x_2 = 3$ and $x_3 = 4$

```
# Reduce the degree of $P$ to at most 3:
n = 3
xnodes = array([-2.0, 0., 1.0, 2.])
ynodes = zeros_like(xnodes)
for i in range(len(xnodes)):
    ynodes[i] = f(xnodes[i])
print(f"n is now {n}, the nodes are now {xnodes}, with f(x_i) values {ynodes}")
c = fitPolynomial(xnodes, ynodes)
print(f"The coefficients of P are now {c}")
showPolynomial(c)
print(f"The values of P at the nodes are now {evaluatePolynomial(xnodes, c)}")
```

```
n is now 3, the nodes are now [-2.  0.  1.  2.], with f(x_i) values [70.  4.  7.  18.]
The coefficients of P are now [ 4. -5. 10. -2.]
P(x) = 4.0 - 5.0x + 10.0x^2 - 2.0x^3
The values of P at the nodes are now [70.  4.  7. 18.]
```

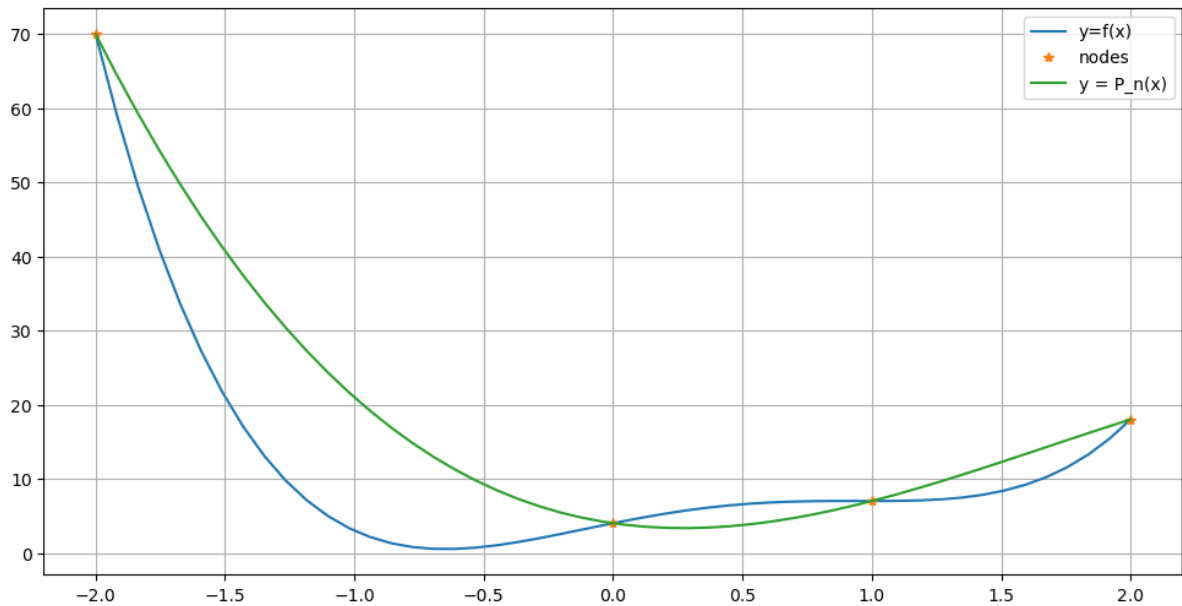
There are several ways to assess the accuracy of this fit; we start graphically, and later consider the maximum and root-mean-square (RMS) errors.

```
x = linspace(min(xnodes), max(xnodes)) # defaulting to 50 points, for graphing
```

```

figure(figsize=[12,6])
plot(x, f(x), label="y=f(x)")
plot(xnodes, ynodes, "*", label="nodes")
P_n_x = evaluatePolynomial(x, c)
plot(x, P_n_x, label="y = P_n(x)")
legend()
grid(True);

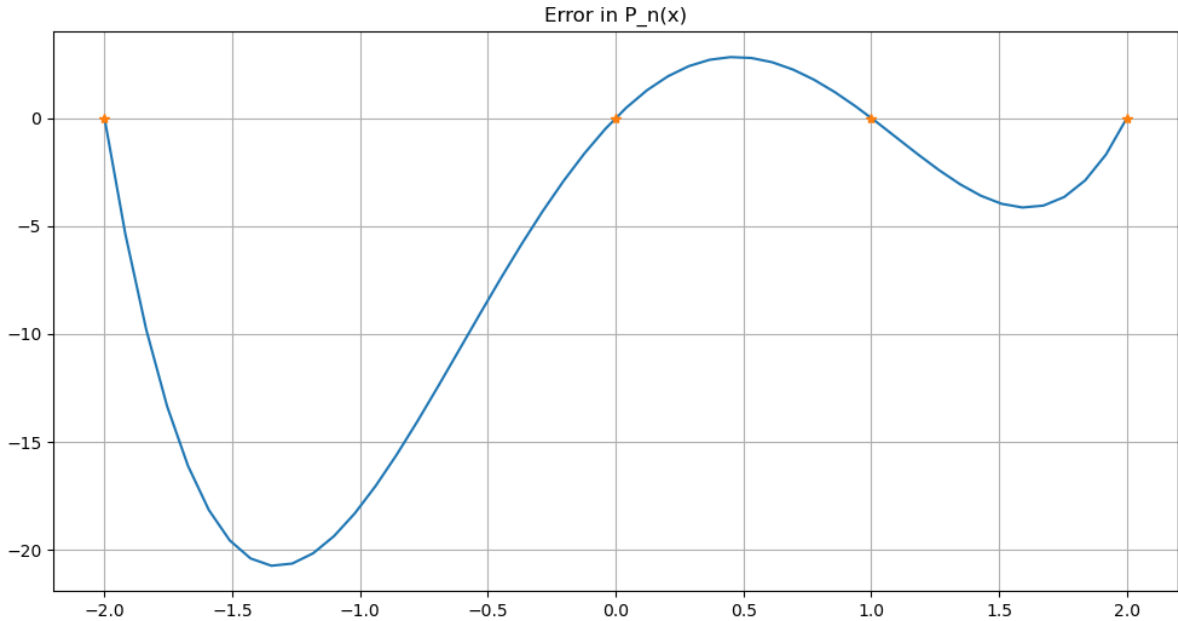
```



```

P_error = f(x) - P_n_x
figure(figsize=[12,6])
title("Error in P_n(x)")
plot(x, P_error, label="y=f(x)")
plot(xnodes, zeros_like(xnodes), "*")
grid(True);

```


**Example 3.1.3 ($f(x)$ not a polynomial at all)**

$f(x) = e^x$ with five nodes, equally spaced from -1 to 1

```
def g(x): return exp(x)
a_g = -1.0
b_g = 1.0
```

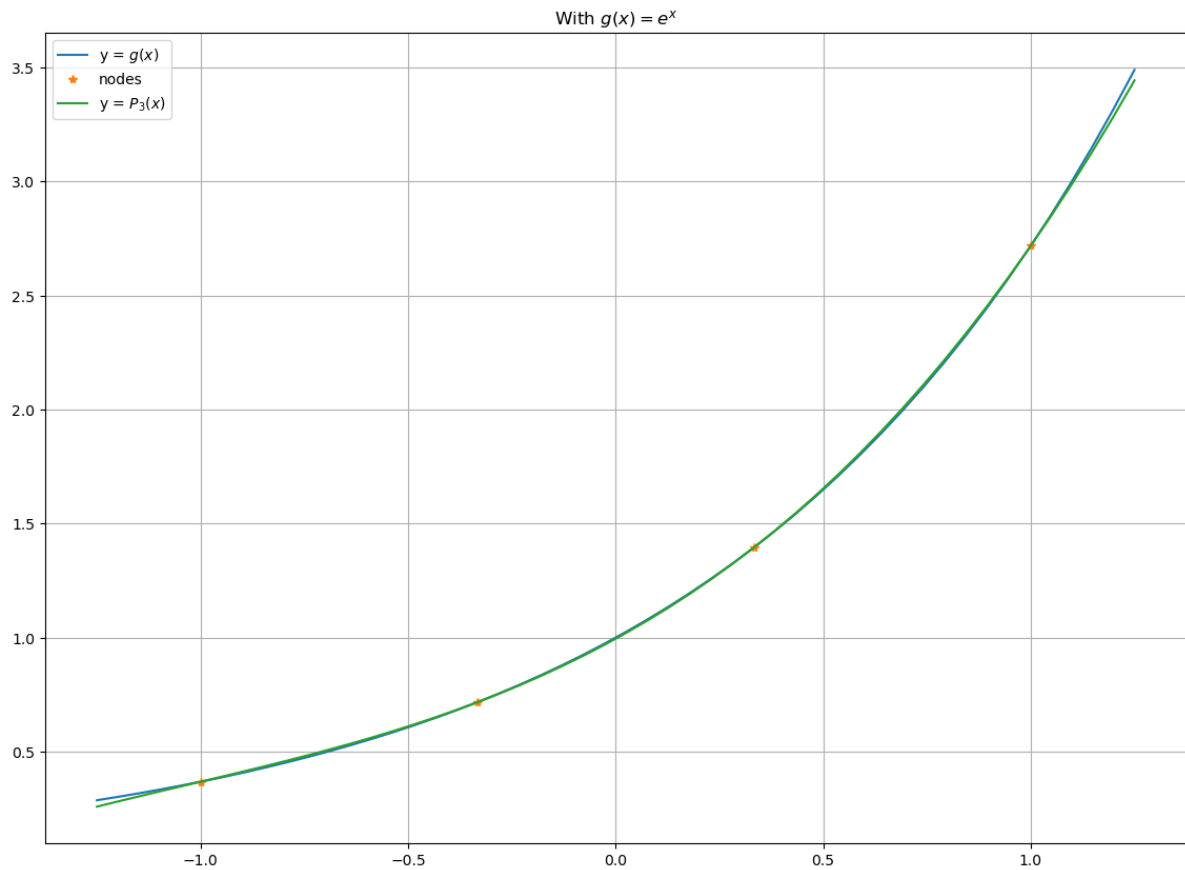
```
n = 3
xnodes_g = linspace(a_g, b_g, n+1)
ynodes_g = zeros_like(xnodes_g)
for i in range(len(xnodes_g)):
    ynodes_g[i] = g(xnodes_g[i])
print(f"{n=}")
print(f"node x values {xnodes_g}")
print(f"node y values {ynodes_g}")
c_g = fitPolynomial(xnodes_g, ynodes_g)
print(f"The coefficients of P are {c_g}")
showPolynomial(c_g)
P_values = evaluatePolynomial(c_g, xnodes_g)
print(f"The values of P(x_i) are {P_values}")
```

```
n=3
node x values [-1.          -0.33333333  0.33333333  1.          ]
node y values [0.36787944 0.71653131 1.39561243 2.71828183]
The coefficients of P are [0.99519577 0.99904923 0.54788486 0.17615196]
P(x) = 0.9952 + 0.999x + 0.5479x^2 + 0.1762x^3
The values of P(x_i) are [-0.01593727 -0.00316622 -0.91810613 -1.04290824]
```

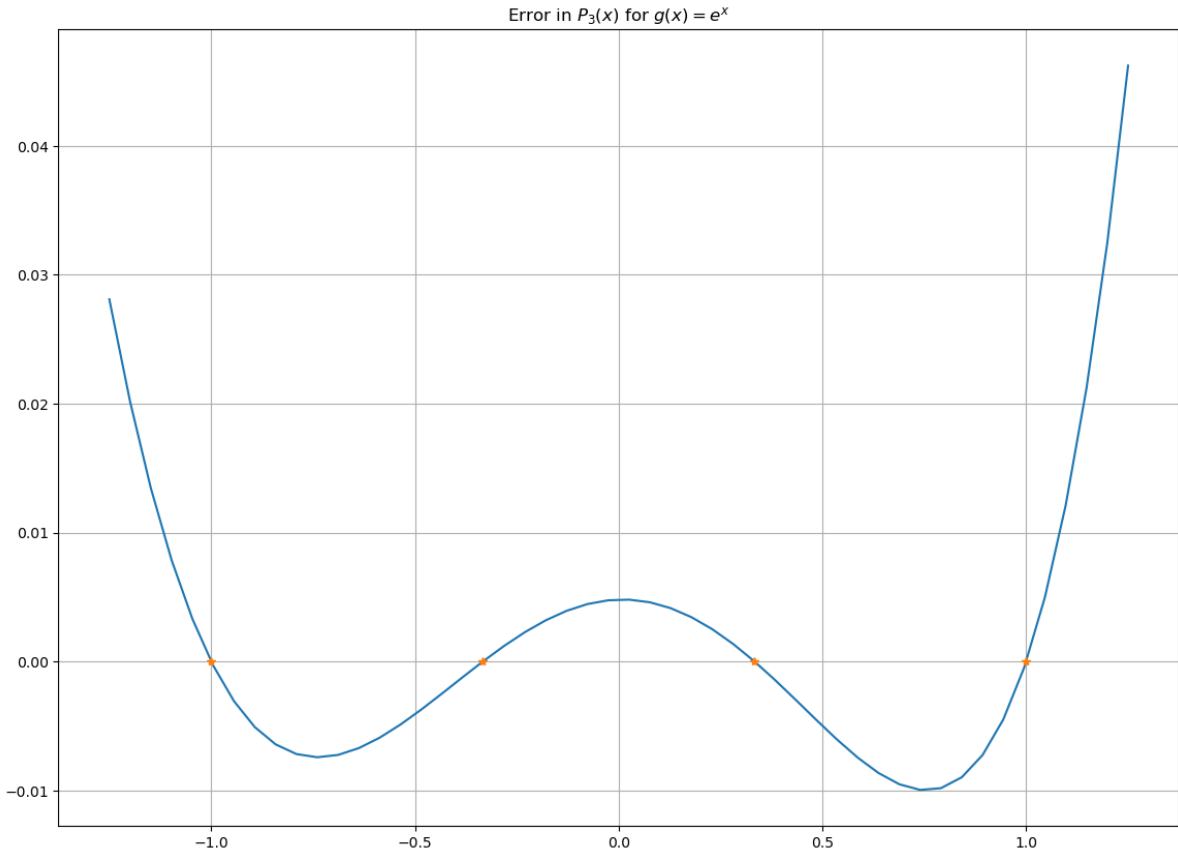
There are several ways to assess the accuracy of this fit. We start graphically, and later consider the maximum and root-mean-square (RMS) errors.

```
x_g = linspace(a_g - 0.25, b_g + 0.25) # Go a bit beyond the nodes in each direction
```

```
figure(figsize=[14,10])
title("With  $g(x) = e^x$ ")
plot(x_g, g(x_g), label="y =  $g(x)$ ")
plot(xnodes_g, ynodes_g, "*", label="nodes")
P_g = evaluatePolynomial(x_g, c_g)
plot(x_g, P_g, label=f"y =  $P_{\{n\}}(x)$ ")
legend()
grid(True);
```



```
P_error = g(x_g) - P_g
figure(figsize=[14,10])
title(f"Error in  $P_{\{n\}}(x)$  for  $g(x) = e^x$ ")
plot(x_g, P_error)
plot(xnodes_g, zeros_like(xnodes_g), "*")
grid(True);
```



This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

4.2 Error Formulas for Polynomial Collocation

References:

- Section 3.2.1 *Interpolation error formula* in [Sauer, 2022].
- Section 3.1 *Interpolation and the Lagrange Polynomial* in [Burden et al., 2016].
- Section 4.2 *Errors in Polynomial Interpolation* in [Kincaid and Cheney, 1990].

4.2.1 Introduction

When a polynomial P_n is given by collocation to $n + 1$ points (x_i, y_i) , $y_i = f(x_i)$ on the graph of a function f , one can ask how accurate it is as an approximation of f at points x other than the nodes: what is the error $E(x) = f(x) - P(x)$?

As is often the case, the result is motivated by considering the simplest “non-trivial case”: f a polynomial of degree one too high for an exact fit, so of degree $n + 1$. The result is also analogous to the familiar error formula for Taylor polynomial approximations.

```
import numpy as np
from matplotlib.pyplot import figure, plot, title, grid, legend
from numericalMethods import fitPolynomial, evaluatePolynomial
# showPolynomial
```

4.2.2 Error in $P_n(x)$ when collocating with a sufficiently differentiable function

Theorem 3.2.1

For a function f with continuous derivative of order $n + 1$ $D^{n+1}f$, the polynomial P_n of degree at most n that fits the points $(x_i, f(x_i))$ $0 \leq i \leq n$ differs from f by

$$E_n(x) = f(x) - P_n(x) = \frac{D^{n+1}f(\xi_x)}{(n+1)!} \prod_{i=0}^n (x - x_i) \quad (4.1)$$

for some value of ξ_x that is amongst the $n + 2$ points $x_0, \dots, x_n$ and x .

In particular, when $a \leq x_0 < x_1 \dots < x_n \leq b$ and $x \in [a, b]$, then also $\xi_x \in [a, b]$.

Observation 3.2.1

This is rather similar to the error formula for the Taylor polynomial p_n with center x_0 :

$$e_n(x) = f(x) - p_n(x) = \frac{D^{n+1}f(\xi_x)}{(n+1)!} (x - x_0)^{n+1}, \text{ some } \xi_x \text{ between } x_0 \text{ and } x. \quad (4.2)$$

This is effectively the limit of Equation (4.1) when all the x_i congeal to x_0 .

4.2.3 Error bound with equally spaced nodes is $O(h^{n+1})$, but ...

An important special case is when there is a single parameter h describing the spacing of the nodes; when they are the equally spaced values $x_i = a + ih$, $0 \leq i \leq n$, so that $x_0 = a$ and $x_n = b$ with $h = \frac{b-a}{n}$. Then there is a somewhat more practically usable error bound:

Theorem 3.2.2

For $x \in [a, b]$ and the above equally spaced nodes in that interval $[a, b]$,

$$|E_n(x)| = |f(x) - P_n(x)| \leq \frac{M_{n+1}}{n+1} h^{n+1}, = O(h^{n+1}), \quad (4.3)$$

where $M_{n+1} = \max_{x \in [a, b]} |D^{n+1}f(x)|$.

4.2.4 Possible failure of convergence

A major practical problem with this error bound is that it does not in general guarantee convergence $P_n(x) \rightarrow f(x)$ as $n \rightarrow \infty$ with fixed interval $[a, b]$, because in some cases M_{n+1} grows too fast.

A famous example is the “Witch of Agnesi” (so-called because it was introduced by Maria Agnesi, author of the first textbook on differential and integral calculus).

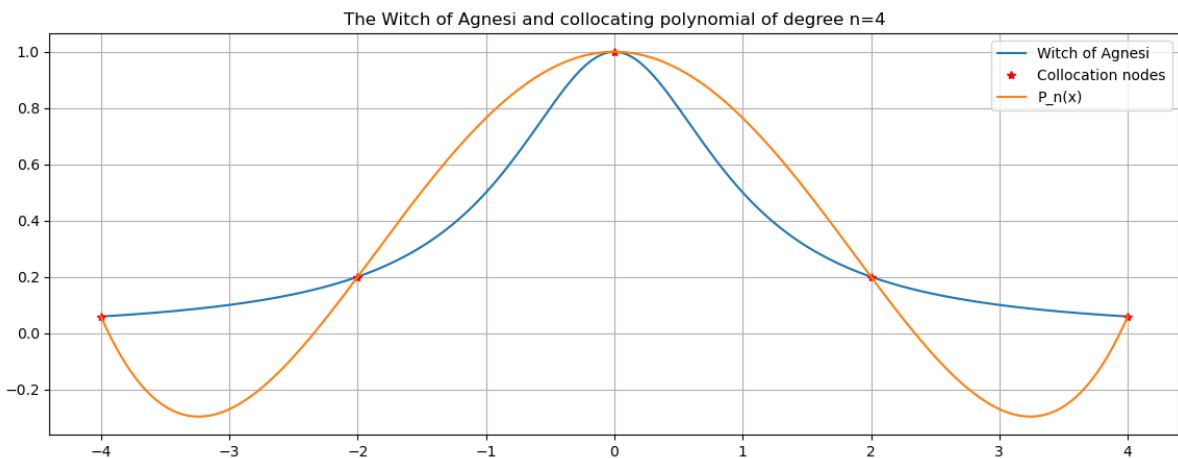
```
def agnesi(x):
    return 1/(1+x**2)
```

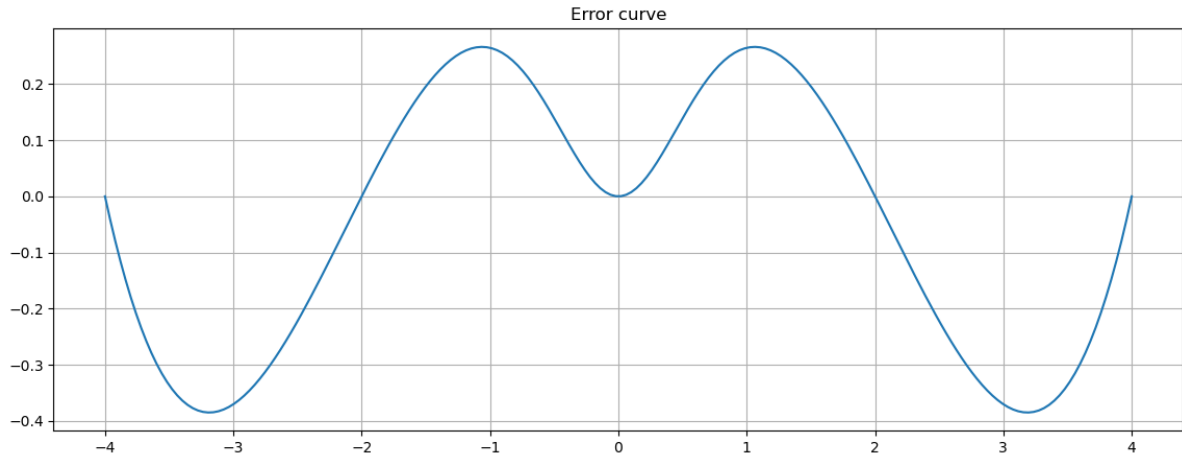
```
def graph_agnesi_collocation(a, b, n):
    figure(figsize=[14, 5])
    title(f"The Witch of Agnesi and collocating polynomial of degree {n}")
    x = np.linspace(a, b, 200) # Plot 200 points instead of the default 50, as some-
    ↪ fine detail is needed
    agnesi_x = agnesi(x)
    plot(x, agnesi_x, label="Witch of Agnesi")
    xnodes = np.linspace(a, b, n+1)
    ynodes = agnesi(xnodes)
    c = fitPolynomial(xnodes, ynodes)
    P_n = evaluatePolynomial(x, c)
    plot(xnodes, ynodes, 'r*', label="Collocation nodes")
    plot(x, P_n, label="P_n(x)")
    legend()
    grid(True)

    figure(figsize=[14, 5])
    title(f"Error curve")
    E_n = P_n - agnesi_x
    plot(x, E_n)
    grid(True);
```

Start with $n = 4$, which seems somewhat well-behaved:

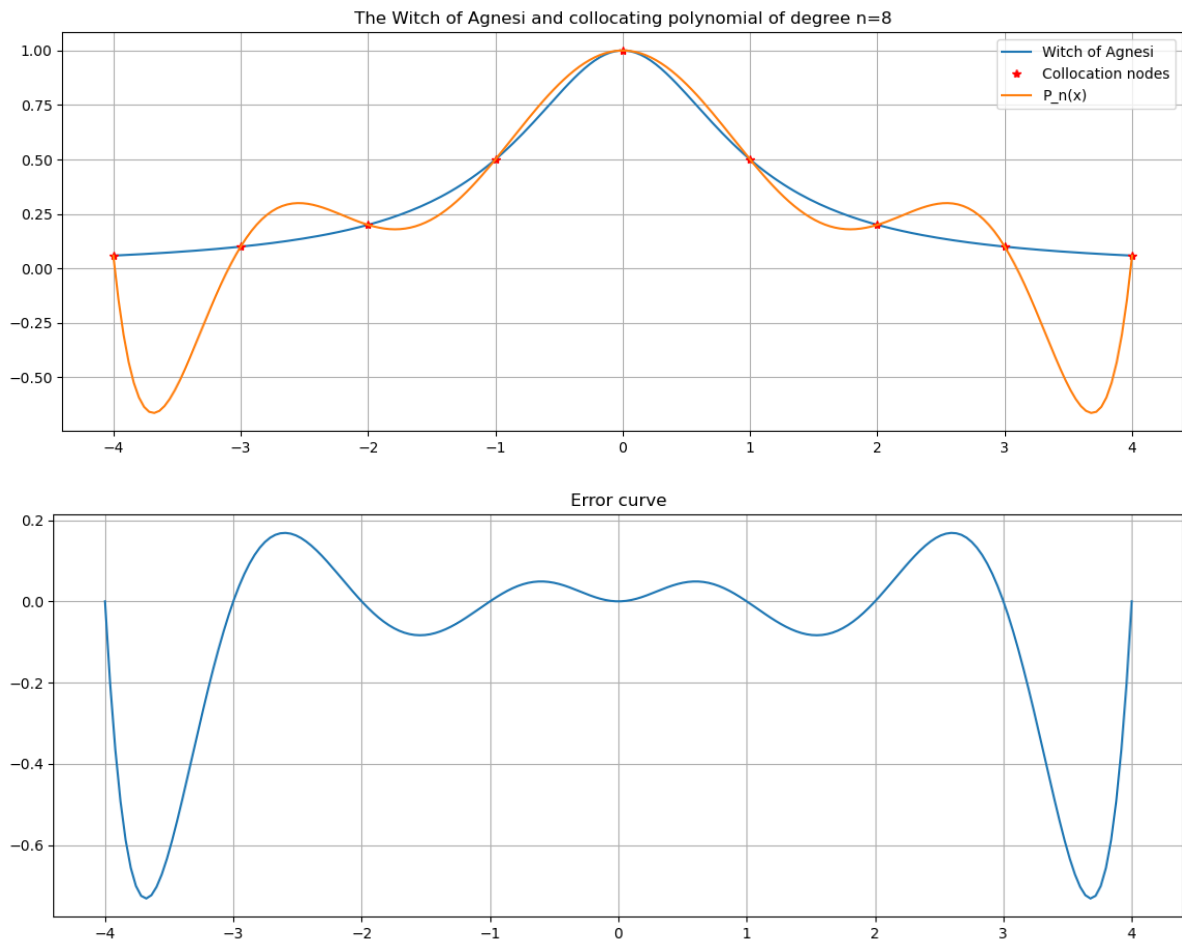
```
graph_agnesi_collocation(a=-4, b=4, n=4)
```





Now increase the number of intervals, doubling each time.

```
graph_agnesi_collocation(a=-4, b=4, n=8)
```

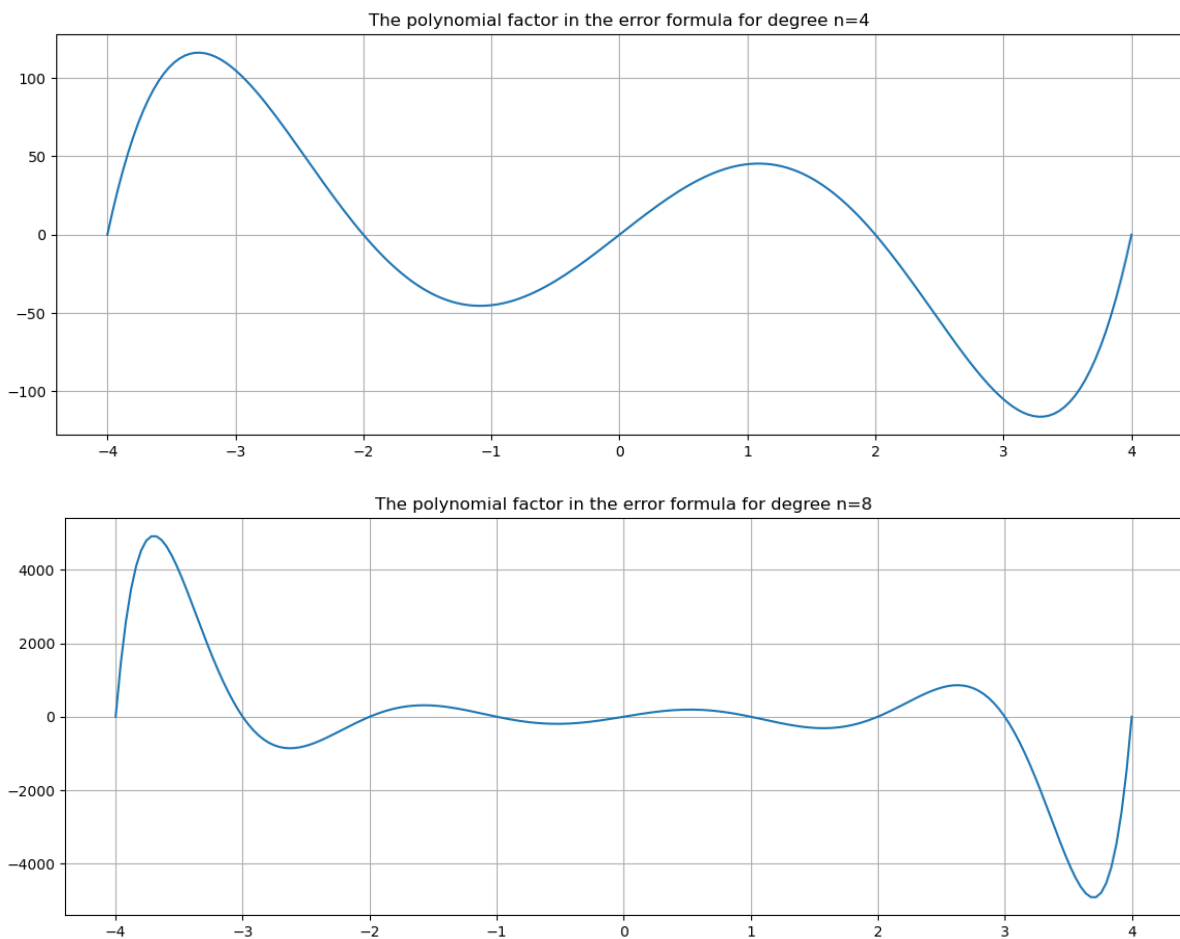


The curve fits better in the central part, but gets worse towards the ends!

One hint as to why is to plot the polynomial factor in the error formula above:

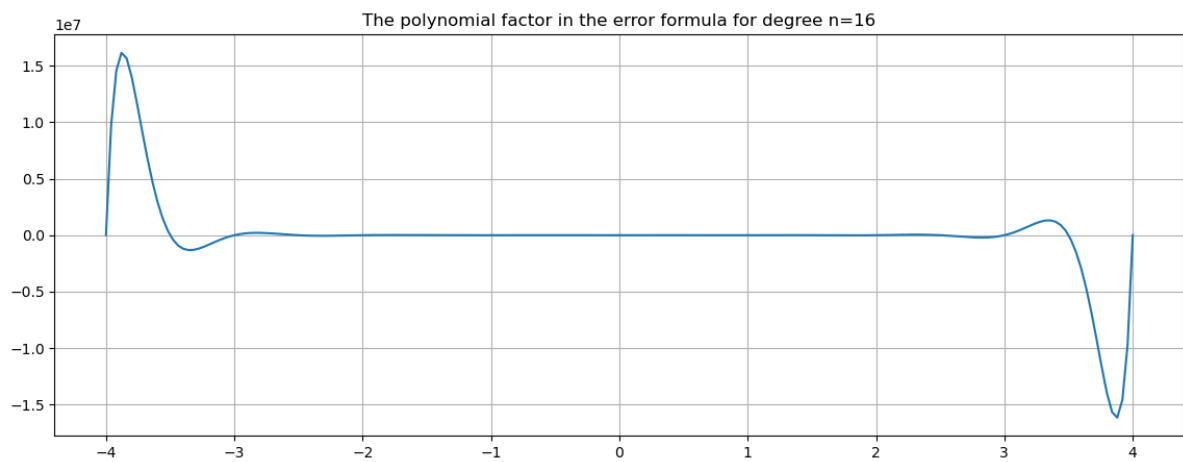
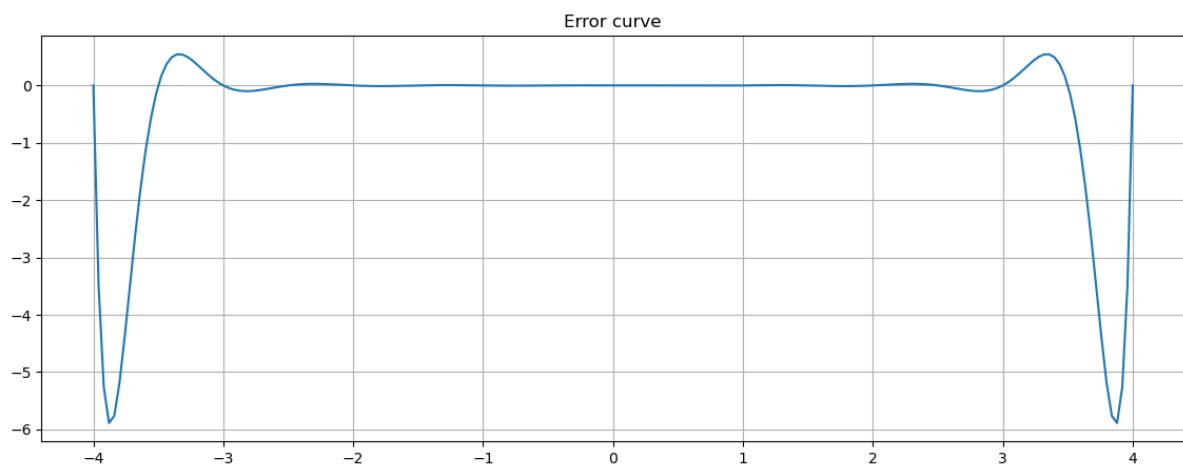
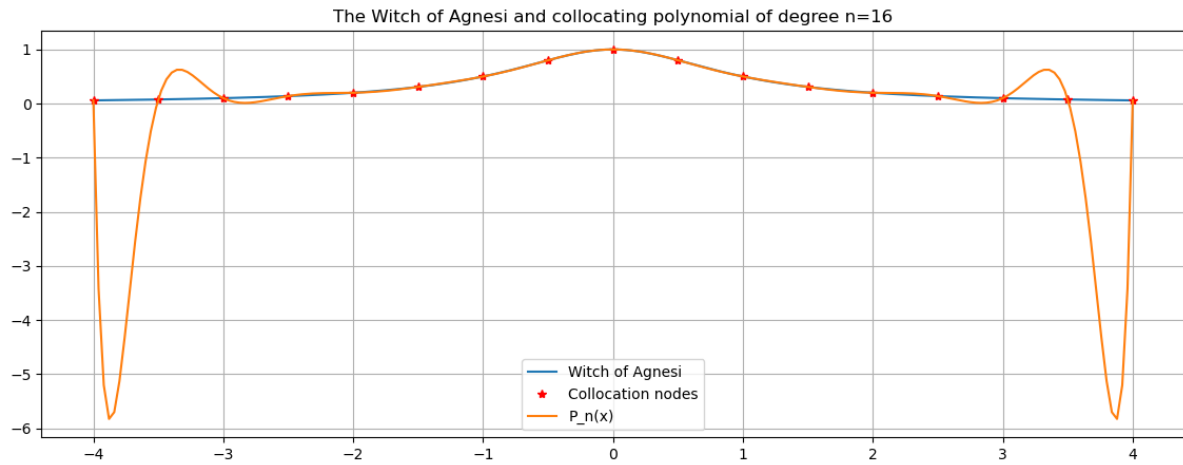
```
def graph_error_formula_polynomial(a, b, n):
    figure(figsize=[14, 5])
    title(f"The polynomial factor in the error formula for degree {n}")
    x = np.linspace(a, b, 200) # Plot 200 points instead of the default 50, as some
    ↪ fine detail is needed
    xnodes = np.linspace(a, b, n+1)
    polynomial_factor = np.ones_like(x)
    for x_node in xnodes:
        polynomial_factor *= (x - x_node)
    plot(x, polynomial_factor)
    grid(True);
```

```
graph_error_formula_polynomial(a=-4, b=4, n=4)
graph_error_formula_polynomial(a=-4, b=4, n=8)
```



As n increases, it just gets worse:

```
graph_agnesi_collocation(a=-4, b=4, n=16)
graph_error_formula_polynomial(a=-4, b=4, n=16)
```



4.2.5 Two solutions: piecewise interpolation and least squares approximation

The approach of least squares approximation is introduced in the next section *Least-squares Fitting to Data*; that can be appropriate when the original data is not exact (due to measurement error in an experiment, for example) so a good approximation at each node can be more appropriate than exact collocation at each but with implausible behavior between the nodes.

When instead exact collocation is sought, piecewise interpolation is typically used. This involves collocation with multiple polynomials of a fixed degree, each on a part of the domain. Then for each such polynomial M_{m+1} in the above error formula is independent of the number N of nodes and with the nodes on interval $[a, b]$ at equal spacing $h = (b-a)/(N-1)$, one has the convergence result

$$|E_m(x)| \leq \frac{M_{m+1}}{m+1} h^{m+1} = O(h^{m+1}) = O\left(\frac{1}{N^{m+1}}\right), \rightarrow 0 \text{ as } N \rightarrow \infty.$$

This only requires that f has a continuous derivatives up to order $m+1$.

The simplest case of this — quite often used in computer graphics, including `matplotlib.pyplot.plot` — is to divide the domain into $N-1$ sub-intervals of equal width separated by nodes $x_i = a + ih$, $0 \leq i \leq N$, and then approximate $f(x)$ linearly on each sub-interval by using the two surrounding nodes x_i and x_{i+1} determined by having $x_i \leq x \leq x_{i+1}$: this is **piecewise linear interpolation**.

This gives the approximating function $L_N(x)$, and the above error formula, now with $m=1$, says that the worst absolute error anywhere in the interval $[a, b]$ is

$$|E_2(x)| = |f(x) - L_N(x)| \leq \frac{M_2}{2} h^2, \quad M_2 = \max_{x \in [a, b]} |f''(x)|.$$

Thus for any f that is twice continuously differentiable the error at each x -value converges to zero as $N \rightarrow \infty$. Further, it is *uniform convergence*: the maximum error over all points in the domain goes to zero.

Preview: definite integrals (en route to solving differential equations)

Integrating this piecewise linear approximation over interval $[a, b]$ gives the Compound Trapezoid Rule approximation of $\int_a^b f(x) dx$. As we will soon see, this also has error at worst $O(h^2)$, $= O(1/N^2)$: each doubling of effort reduces errors by a factor of about four.

Also, you might have heard of Simpson's Rule for approximating definite integrals (and anyway, you will soon!): that uses piecewise quadratic interpolation and we will see that this improves the errors to $O(h^4)$, $= O(1/N^4)$: each doubling of effort reduces errors by a factor of about 16.

Remark 3.2.1 (Computer graphics and smoother approximating curves)

As mentioned, computer graphics often draws graphs from data points this way, most often with either piecewise linear or piecewise cubic ($m=3$) approximation.

However, this can give sharp “corners” at the nodes, so many nodes are needed to make this visually acceptable. That is unavoidable with piecewise linear curves, but for higher degrees there are modifications of this strategy that give smoother curves: piecewise cubics turn out to work very nicely for that, and these are introduced in the section *Piecewise Polynomial Approximating Functions and Spline Interpolation*.

4.3 Choosing the collocation points: the Chebyshev method

Co-authored with Stephen Roberts of the Australian National University.

References:

- Section 3.1 *Data and Interpolating Functions* in [Sauer, 2022].
- Section 3.1 *Interpolation and the Lagrange Polynomial* in [Burden *et al.*, 2016].
- Section 4.2 of [Chenney and Kincaid, 2012].
- Section 6.1 of [Kincaid and Chenney, 1990].

In some situations, one can choose the points x_i to use in polynomial collocations (these points are also called the **nodes**) and a natural objective is to minimise the worst case error over some interval $[a, b]$ on which the approximation is to be used. As discussed previously, the best one can do in most cases is to minimise the maximum absolute value of the polynomial $w_{n+1}(x) := \prod_{i=0}^n (x - x_i)$ arising in the error formula.

The intuitive idea of using equally spaced points is not optimal as w_{n+1} reaches considerably larger values between the outermost pairs of nodes than elsewhere, as seen with the example of the *Witch of Agnesi* in section [Error Formulas for Polynomial Collocation](#). Better intuition suggests that moving the nodes a bit closer in these regions of large error will reduce the maximum error there while not increasing it too much elsewhere, and reduce the maximum error. Further, it would seem that this strategy is possible so long as the maximum amplitude in some of the intervals between the nodes is larger than others: the endpoints a and b need not be nodes so there are $n + 2$ such intervals.

This suggests the conjecture that the smallest possible maximum amplitude of $w_{n+1}(x)$ on an interval $[a, b]$ will be obtained for a set of nodes such that $|w_{n+1}(x)|$ takes its maximum value $n + 2$ times, once in each of the interval separated by the nodes. Indeed this is true, and the nodes achieving this result are the so called **Chebyshev points** or **Chebyshev nodes**, given by the simple formula

$$x_i = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad 0 \leq i \leq n \quad (4.4)$$

To understand this result, consider the case where the interval of interest is $[-1, 1]$, so that these special nodes are $\cos\left(\frac{2i+1}{2n+2}\pi\right)$. The general case then follows by using the change of variables $x = (a+b)/2 + t(b-a)/2$. The reason that this works is that these are the roots of the function

$$T_{n+1}(x) := \cos((n+1)\cos^{-1}x)$$

which turns out to be a polynomial of degree $n + 1$ that takes its maximum absolute value of 1 at the $n + 2$ points $\cos\left(\frac{i}{n+1}\pi\right)$, $0 \leq i \leq n + 1$.

There are a number of claims here: most are simple consequences of the definition and what is known about the roots and extreme values of cosine. The one surprising fact is that $T_n(x)$ is a polynomial of degree n , known as a **Chebyshev polynomial**. The notation comes from an alternative transliteration, *Tchebyshev*, of this Russian name.

This can be checked by induction. The first few cases are easy to check: $T_0(x) = 1$, $T_1(x) = x$ and $T_2(x) = \cos 2\theta = 2\cos^2\theta - 1 = 2x^2 - 1$. In general, let $\theta = \cos^{-1}x$ so that $\cos\theta = x$. Then trigonometric identities give

$$\begin{aligned} T_{n+1}(x) &= \cos(n+1)\theta \\ &= \cos n\theta \cos \theta - \sin n\theta \sin \theta \\ &= T_n(x)x - \sin n\theta \sin \theta \end{aligned}$$

and similarly

$$\begin{aligned} T_{n-1}(x) &= \cos(n-1)\theta \\ &= \cos n\theta \cos \theta + \sin n\theta \sin \theta \\ &= T_n(x)x + \sin n\theta \sin \theta \end{aligned}$$

Thus $T_{n+1}(x) + T_{n-1}(x) = 2xT_n(x)$ or

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$$

Since T_0 and T_1 are known to be polynomials, the same follows for each successive n from this formula. The induction also shows that

$$T_n(x) = 2^{n-1}x^n + \text{terms involving lower powers of } x$$

so in particular the degree is n .

With this information, the error formula can be written in a special form. Firstly w_{n+1} is then a polynomial of degree $n + 1$ with the same roots as T_{n+1} , so is a multiple of the latter function. Secondly, the leading coefficient of w_{n+1} is 1, compared to 2^{n+1} for the Chebyshev polynomial, so $w_{n+1} = T_{n+1}/2^n$. Finally, the maximum of w_{n+1} is seen to be $1/2^n$ and we have the result that

Theorem 3.3.1

When a polynomial approximation $p(x)$ to a function $f(x)$ on the interval $[-1, 1]$ is constructed by collocation at the roots of T_{n+1} , the error is bounded by

$$|f(x) - p(x)| \leq \frac{1}{2^n(n+1)!} \max_{-1 \leq t \leq 1} |f^{(n+1)}(t)|$$

When the interval is $[a, b]$ and the collocation points are the appropriately rescaled Chebyshev points as given in (4.4).

$$|f(x) - p(x)| \leq \frac{(b-a)^{n+1}}{2^{2n+1}(n+1)!} \max_{a \leq x \leq b} |f^{(n+1)}(x)|$$

This method works well in many cases. Further, it is known that any continuous on any interval $[a, b]$ can be approximated arbitrarily well by polynomials, in the sense that the maximum error over the whole interval can be made as small as one likes [this is the *Weierstrass Approximation Theorem*]. However, collocation at these Chebyshev nodes will not work for all continuous functions: indeed no choice of points will work for all cases, as is made precise in theorem 6 on page 288 of [Kincaid and Cheney, 1990]. One way to understand the problem is that the error bound relies on derivatives of ever higher order, so does not even apply to some continuous functions.

This suggests a new strategy: break the interval $[a, b]$ into smaller interval, approximate on each interval by a polynomial of some small degree, and join these polynomials together. Hopefully, the errors will only depend on a few derivatives, and so will be more controllable, while using enough nodes and small enough intervals will allow the errors to be made as small as desired. This fruitful idea is dealt with next.

4.4 Piecewise Polynomial Approximating Functions and Spline Interpolation

Co-authored with Stephen Roberts of the Australian National University.

References:

- Sections 3.6, 6.2, 6.4 of [Kincaid and Cheney, 1990].
- Section 3.4 *Cubic Splines* in [Sauer, 2022].
- Sections 3.5 *Cubic Spline Interpolation* and 3.4 *Hermite Interpolation* of [Burden et al., 2016].
- Sections 6.1 and 6.2 of Chapter 6 *Spline Functions* [Cheney and Kincaid, 2012].

The idea of approximating a function (or interpolating between a set of data points) with a function that is piecewise polynomial takes its simplest form using *continuous piecewise linear functions*. Indeed, this is the method most commonly used to produce a graph from a large set of data points: for example, the command `plot` from `matplotlib.pyplot` (for Python) or `PyPlot` (for Julia) does it.

The idea is simply to draw straight lines between each successive data point. It is worth analysing this simple method before considering more accurate approaches.

Consider a set of $n + 1$ points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$ again, this time requiring the x values to be in increasing order. Then define the linear functions

$$L_i(x) = y_i + (x - x_i) \frac{y_{i+1} - y_i}{x_{i+1} - x_i}, \quad x_i \leq x \leq x_{i+1}, \quad 0 \leq i < n$$

These can be joined together into a continuous function

$$L(x) = L_i(x) \text{ for } x_i \leq x \leq x_{i+1}$$

with the values $L(x_i) = y_i$ at all nodes, so that the definition is consistent at the points where the domains join, also guaranteeing continuity.

4.4.1 Spline Interpolation

Reference: Section 6.4 of [Kincaid and Cheney, 1990].

If a piecewise linear approximation is approximated that passes through a given set of $n + 1$ points or **knots**

$$(t_0, y_0), \dots, (t_n, y_n)$$

and is linear in each of the n interval between them, the “smoothest” curve that one can get is the continuous one given by using linear interpolation between each consecutive pair of points. Less smooth functions are possible, for example the piecewise constant approximation where $L(x) = y_i$ for $x_i \leq x < x_{i+1}$.

The general strategy of spline interpolation is to approximate with a piecewise polynomial function, with some fixed degree k for the polynomials, and is as smooth as possible at the joins between different polynomials. Smoothness is measured by the number of continuous derivatives that the function has, which is only in question at the knots of course.

The traditional and most important case is that of **cubic splines interpolants**, which have the form

$$S(x) = S_i(x), \quad t_i \leq x \leq t_{i+1}, \quad 0 \leq i < n$$

where each $S_i(x)$ is a cubic and the interpolation conditions are

$$S_i(t_i) = y_i, \quad S_i(t_{i+1}) = y_{i+1}, \quad 0 \leq i < n$$

These conditions automatically give continuity, but leave many degrees of freedom to impose more smoothness. Each cubic is described by four coefficients and so there are $4n$ in all, and the interpolation conditions give only $2n$ conditions. There are $n - 1$ knots where different cubics join, so requiring S to have continuous first and second derivatives imposes $2(n - 1)$ further conditions for a total of $4n - 2$. This is the best smoothness possible without $S(x)$ becoming a single cubic, and leaves two degrees of freedom. These will be dealt with later, but one approach is imposing zero second derivatives at each end of the interval.

Thus we have the equations

$$S'_{i-1}(t_i) = S'(t_i)$$

and

$$S''_{i-1}(t_i) = S''(t_i),$$

$$1 \leq i \leq n-1.$$

The brute force method would be to write something like

$$S_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i$$

which would leave to a set of $4n$ simultaneous linear equations for these $4n$ unknowns once the two missing conditions have been chosen.

This could then be solved numerically, but the size and cost of the problem can be considerably reduced, to a tridiagonal system of $n-1$ equations.

Start by considering the second derivative of $S(x)$, which must be continuous and piecewise linear. Its values at the knots can be called $x_i = S''_i(t_i)$ and the lengths of the interval called $h_i = x_{i+1} - x_i$ so that

$$S''_i(x) = \frac{z_i}{h_i}(t_{i+1} - x) + \frac{z_{i+1}}{h_i}(x - t_i)$$

Integrating twice,

$$S_i(x) = \frac{z_i}{6h_i}(t_{i+1} - x)^3 + \frac{z_{i+1}}{6h_i}(x - t_i)^3 + C_i(t_{i+1} - x) + D_i(x - t_i)$$

The interpolation conditions then determine C_i and D_i :

$$S_i(x) = \frac{z_i}{6h_i}(t_{i+1} - x)^3 + \frac{z_{i+1}}{6h_i}(x - t_i)^3 + \left(\frac{y_i}{h_i} - \frac{z_i h_i}{6}\right)(t_{i+1} - x) + \left(\frac{y_{i+1}}{h_i} - \frac{z_{i+1} h_i}{6}\right)(x - t_i) \quad (4.5)$$

In effect, three quarters of the equations have been solved explicitly, leaving only the z_i to be determined using the remaining condition of the continuity of $S'(x)$.

Differentiating the above expression and evaluating at the appropriate points gives the expressions

$$S'_i(t_i) = -\frac{h_i}{3}z_i - \frac{h_i}{6}z_{i+1} - \frac{y_i}{h_i} + \frac{y_{i+1}}{h_i} \quad (4.6)$$

$$S'_{i-1}(t_i) = -\frac{h_{i-1}}{6}z_{i-1} + \frac{h_{i-1}}{3}z_i - \frac{y_{i-1}}{h_{i-1}} + \frac{y_i}{h_{i-1}} \quad (4.7)$$

Equating these at the internal knots (and simplifying a bit) gives

$$h_{i-1}z_{i-1} + 2(h_i + h_{i-1})z_i + h_i z_{i+1} = \frac{6}{h_i}(y_{i+1} - y_i) - \frac{6}{h_{i-1}}(y_i - y_{i-1}) \quad (4.8)$$

These are $n-1$ linear equations in the $n+1$ unknowns z_i , so various different cubic spline interpolants can be constructed by adding two extra conditions in the form of two more linear equations. The traditional way is the one mentioned above: require the second derivative to vanish at the two endpoints. That is

$$S''(t_0) = S''(t_n) = 0$$

which gives a **natural spline**.

In terms of the z_i this gives the trivial equations $z_0 = z_n = 0$. Thus these two unknowns can be eliminated from the equations in (4.8) giving the following tridiagonal system:

$$= \begin{bmatrix} 2(h_0 + h_1) & h_1 & & & \\ h_1 & 2(h_1 + h_2) & \ddots & & \\ & \ddots & \ddots & h_{n-2} & \\ & & h_{n-2} & 2(h_{n-2} + h_{n-1}) & \\ & & & & \end{bmatrix} \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_{n-1} \end{bmatrix}$$

$$= \begin{bmatrix} 6((y_2 - y_1)/h_1 - (y_1 - y_0)/h_0) \\ 6((y_3 - y_2)/h_2 - (y_2 - y_1)/h_1) \\ \vdots \\ 6((y_n - y_{n-1})/h_{n-1} - (y_{n-1} - y_{n-2})/h_{n-2}) \end{bmatrix}$$

Solving tridiagonal systems is far more efficient if it can be done without pivoting by the method seen earlier, and this is a good method if the matrix is diagonally dominant.

That is true here: recalling that the t_i are in increasing order, each h_i is positive, so each diagonal element is at least twice the sum of the absolute values of all other elements in the same row. This result incidentally also shows that the equations have a unique solution, which means that the natural cubic spline exists and is determined uniquely by the data, requiring about $O(n)$ operations.

Evaluation of $S(x)$ is then done by finding the i such that $t_i \leq x < t_{i+1}$ and then evaluating the appropriate case in (4.5).

4.4.2 Clamped Splines and Error Bounds

Reference: Section 3.6 of [Kincaid and Cheney, 1990].

Though the algorithm for natural cubic spline interpolation is widely available in software [TO DO: add Numpy/Julia references] it is worth knowing the details. In particular, it is then easy to consider minor changes, like different conditions at the end points.

Recall that the **natural** or **free** spline has the boundary conditions

$$S''(t_0) = S''(t_n) = 0 \quad (4.9)$$

When the spline is to be used to approximate a function $f(x)$ one useful alternative choice of boundary conditions is to specify the derivative of the spline function to match that of f at the endpoints:

$$S'(t_0) = f'(t_0), \quad S'(t_n) = f'(t_n) \quad (4.10)$$

This is called a **clamped spline**.

When the function f or its derivatives are not known, they can be approximated from the data itself. Thus a generalisation of the last condition is

$$S'(t_0) = d_0, \quad S'(t_n) = d_n \quad (4.11)$$

for some approximations of the derivatives.

The subject of approximating a function's derivative using a finite collection of values of the function will be taken up soon in more detail, but the simplest approach is to use the difference quotient from the definition of the derivative. This gives

$$\begin{aligned} d_0 &:= \frac{y_1 - y_0}{h_0} = \frac{f(t_1) - f(t_0)}{t_1 - t_0} \\ d_n &:= \frac{y_n - y_{n-1}}{h_{n-1}} = \frac{f(t_n) - f(t_{n-1})}{t_n - t_{n-1}} \end{aligned}$$

as one choice for the approximate derivatives.

The cubic splines given by using some such approximate derivatives will be called **modified clamped spline**.

These new conditions require a revision of the previous algorithm, but one benefit is that there is a better result guaranteeing the accuracy of the approximation.

To derive the new equations and algorithm for [modified] clamped splines return to the equations (4.6) and (4.7) used to derive the equation (4.8) that defines the tridiagonal system of $n - 1$ equations for the second derivatives $z_1, \dots, z_{n-1}$.

Instead of eliminating the two unknowns z_0 and z_n , we can add two more linear equations by using those equations (4.6) and (4.7) respectively at t_0 and t_n [i.e. for $i = 0$ and $i = n$] and equating to the values to whatever d_0 and d_n we are

using:

$$\begin{aligned}
 S'(t_0) &= S'_0(t_0) \\
 &= -\frac{h_0}{3}z_0 - \frac{h_0}{6}z_1 - \frac{y_0}{h_0} + \frac{y_1}{h_0} \\
 &= d_0 \\
 S'(t_n) &= S'_{n-1}(t_n) \\
 &= \frac{h_{n-1}}{6}z_{n-1} + \frac{h_{n-1}}{3}z_n - \frac{y_{n-1}}{h_{n-1}} + \frac{y_n}{h_{n-1}} \\
 &= d_n
 \end{aligned}$$

In conjunction with equation (4.8), this gives the new tridiagonal system

$$\begin{aligned}
 &\begin{bmatrix} 2h_0 & h_0 & & & \\ h_0 & 2(h_0 + h_1) & h_1 & & \\ & \ddots & \ddots & \ddots & \\ & & h_{n-2} & 2(h_{n-2} + h_{n-1}) & h_{n-1} \\ & & & h_{n-1} & 2h_{n-1} \end{bmatrix} \begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_{n-1} \\ z_n \end{bmatrix} \\
 &= \begin{bmatrix} 6((y_1 - y_0)/h_0 - d_0) \\ 6((y_2 - y_1)/h_1 - (y_1 - y_0)/h_1) \\ \vdots \\ 6((y_n - y_{n-1})/h_{n-1} - (y_{n-1} - y_{n-2})/h_{n-2}) \\ 6(d_n - (y_n - y_{n-1})/h_{n-1}) \end{bmatrix}
 \end{aligned}$$

As in the case of the tridiagonal system for natural splines, the rows of the matrix also satisfy the condition of diagonal dominance, so again this system has a unique solution that can be computed accurately with only $O(n)$ operations and no pivoting.

4.4.3 Error Bounds for Approximation with Clamped Splines

If the exact derivatives mentioned in (4.10) are available, the errors are bounded as follows

Theorem 3.4.1

Suppose that $f(x)$ is four times continuously differentiable on the interval $[a, b]$, with $\max_{a \leq x \leq b} |f^{(4)}(x)| \leq M$. Then the clamped cubic spline approximation $S(x)$ using the points $a = t_0 < t_1 < \dots < t_n = b$ and $y_i = f(t_i)$ satisfies

$$|f(x) - S(x)| \leq M \frac{5}{384} \left(\max_{0 \leq i \leq n-1} h_i \right)^4$$

for every point $x \in [a, b]$.

There is also an error bound of the same “fourth order” form for the natural cubic spline: that is, one of the form of some constant depending on f times the fourth power of $\max_{0 \leq i \leq n-1} h_i$. However it is far more complicated to describe: see page 138 of [Burden *et al.*, 2016] for more comments on this.

When we have studied methods for approximating derivatives, it will be possible to establish error bounds for modified clamped splines with various approximations for the derivatives at the endpoints, so that they depend only on the values of f at the knots. With care, these more practical approximations can also be made fourth order accurate.

4.4.4 Hermite Cubic Approximation

Reference: Section 6.2 of [Kincaid and Cheney, 1990].

Hermite interpolation in general consists in finding a polynomial $H(x)$ to approximate a function $f(x)$ by giving a set of points $t_0, \dots, t_n$ and requiring that the value of the polynomial and its first few derivatives match that of the original function.

The simplest case that is not simply polynomial interpolation or Taylor polynomial approximation is where there are two points, and first derivatives are required to match. This gives four conditions

$$\begin{aligned} H(t_0) &= f(t_0) = y_0, H'(t_0) = f'(t_0) = y'_0 \\ H(t_1) &= f(t_1) = y_1, H'(t_1) = f'(t_1) = y'_1 \end{aligned}$$

and counting constants suggests that there should be a unique cubic h with these properties. From now on, I will use “cubic” to include the degenerate cases that are actually quadratics and so on.

To determine this cubic it is convenient to put it in the form

$$H(x) = a + b(x - t_0) + (x - t_0)^2[c + d(x - t_{i+1})]$$

and let $h = t_1 - t_0$; then applying the four conditions in turn gives

$$\begin{aligned} a &= y_0, & b &= y'_0 \\ c &= \frac{y_1 - y_0}{h^2} - \frac{y'_0}{h}, & d &= \frac{y'_1 - y'_0}{3h^2} - \frac{2(y_1 - y_0)}{3h^3} \end{aligned}$$

With more points, one could look for higher order polynomials, but it is useful in some cases to construct a piecewise cubic approximation, with the cubic between each consecutive pair of nodes determined only by the value of the function and its derivative at those nodes. Thus the piecewise Hermite cubic approximation to f on the interval $[a, b]$ for the points $a = t_0 < t_1 < \dots < t_n$ is given by a set of n cubics

$$H(x) = H_i(x) = a_i + b_i(x - t_i) + (x - t_i)^2[c_i + d_i(x - t_{i+1})], \quad t_i \leq x < t_{i+1}$$

with

$$\begin{aligned} a_i &= y_i, & b_i &= y'_i \\ c_i &= \frac{y_{i+1} - y_i}{h_i^2} - \frac{y'_i}{h_i} \\ d_i &= \frac{y'_{i+1} - y'_i}{3h_i^2} - \frac{2(y_{i+1} - y_i)}{3h_i^3} \end{aligned}$$

where $y_i := f(t_i)$, $y'_i := f'(t_i)$ and $h_i := t_{i+1} - t_i$. Most often, the points are equally spaced so that

$$h_i = h := (b - a)/n.$$

There is an error formula for this (which is also an error formula for a clamped spline in the case $n = 1$)

Theorem 3.4.2

For $x \in [t_i, t_{i+1}]$

$$f(x) - H(x) = \frac{f^{(4)}(\xi)}{4!} [(x - t_i)(x - t_{i+1})]^2$$

where $\xi \in [t_i, t_{i+1}]$. Thus if $|f^{(4)}(x)| \leq M_i$ for $x \in [t_i, t_{i+1}]$,

$$|f(x) - H(x)| \leq \frac{M_i}{384} h_i^4$$

Proof. See page 311 of [Kincaid and Cheney, 1990].

Thus the accuracy is about as good as for clamped splines: the trade off is that the Hermite approximation is less smooth (only one continuous derivative at the nodes), but the error is “localised”. That is, if the fourth derivative of f is large or non-existent in one interval, the accuracy of the Hermite approximation only suffers in that interval, not over the whole domain.

However this comparison is a bit unfair, as the Hermite approximation uses the extra information about the derivatives of f . This is also often impractical: either the derivatives are not known, or there is no known function f but only a collection of values y_i .

To overcome this problem, the derivatives needed in the above formulas can be approximated from the y_i as was done for modified clamped splines. To do this properly, it is worth taking a thorough look at methods for approximating derivatives and bounding the accuracy of such approximations.

4.5 Least-squares Fitting to Data

References:

- Chapter 4 *Least Squares* of [Sauer, 2022], sections 1 and 2.
- Section 8.1 *Discrete Least Squares Approximation* of [Burden *et al.*, 2016].

```
import numpy as np
from matplotlib.pyplot import figure, plot, title, legend, grid, loglog
from numpy.random import random
from numericalMethods import solveLinearSystem
from numericalMethods import evaluatePolynomial
```

```
import numericalMethods as nm
```

We have seen that when trying to fit a curve to a large collection of data points, fitting a single polynomial to all of them can be a bad approach. This is even more so if the data itself is inaccurate, due for example to measurement error.

Thus an important approach is to find a function of some simple form that is close to the given points but not necessarily fitting them exactly: given N points

$$(x_i, y_i), \quad 1 \leq i \leq N \quad (\text{or } 0 \leq i < N \text{ with Python!})$$

we seek a function $f(x)$ so that the errors at each point,

$$e_i = y_i - f(x_i),$$

are “small” overall, in some sense.

Two important choices for the function $f(x)$ are

- polynomials of low degree, and
- periodic sinusoidal functions;

we will start with the simplest case of fitting a straight line.

4.5.1 Measuring “goodness of fit”: several options

The first decision to be made is how to measure the overall error in the fit, since the error is now a vector of values $e = \{e_i\}$, not a single number. Two approaches are widely used:

- *Min-Max*: minimize the maximum of the absolute errors at each point, $\|e\|_{max}$ or $\|e\|_{\infty} = \max_{1 \leq i \leq n} |e_i|$
- *Least Squares*: Minimize the sum of the squares of the errors, $\sum_1^n e_i^2$

4.5.2 What doesn't work

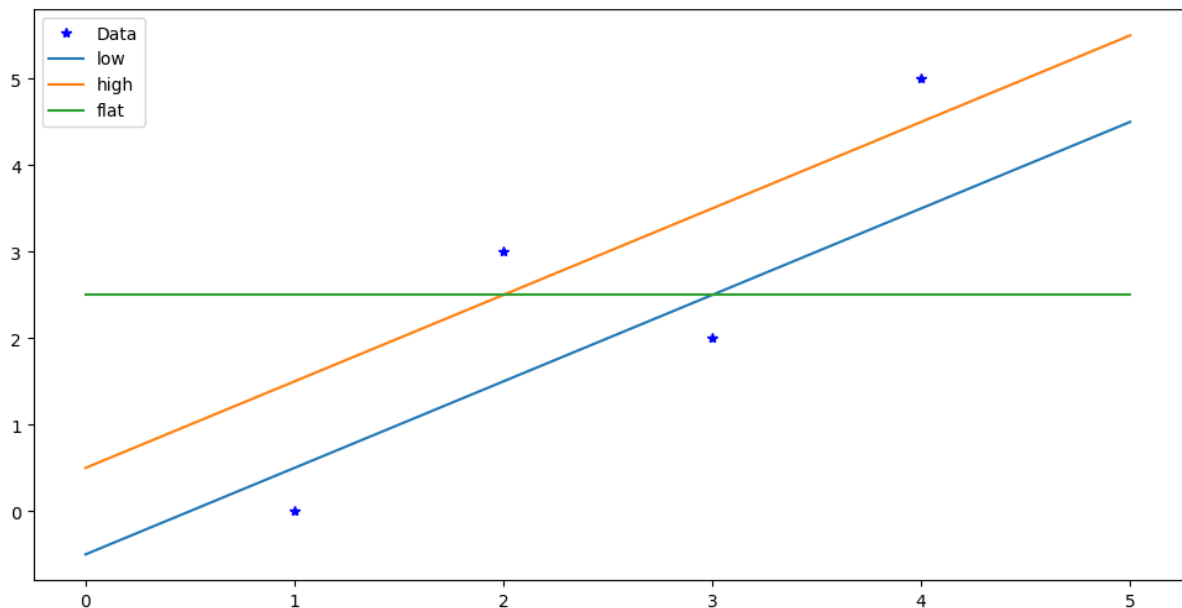
Another seemingly natural approach is:

- Minimize the sum of the absolute errors, $\|e\|_1 = \sum_1^n |e_i|$

but this often fails completely. In the following example, all three lines minimize this measure of error, along with infinitely many others: any line that passes below half of the points and above the other half.

```
xdata = [1., 2., 3., 4.]
ydata = [0., 3., 2., 5.]

figure(figsize=[12,6])
plot(xdata, ydata, 'b*', label="Data")
xplot = np.linspace(0.,5.)
ylo = xplot - 0.5
yhi = xplot + 0.5
yflat = 2.5*np.ones_like(xplot)
plot(xplot, ylo, label="low")
plot(xplot, yhi, label="high")
plot(xplot, yflat, label="flat")
legend(loc="best");
```



The Min-Max method is important and useful, but computationally difficult. One hint is the presence of absolute values in the formula, which get in the way of using calculus to get equations for the minimum.

Thus the easiest and most common approach is **Least Squares**, or equivalently, minimizing the root-mean-square error, which is just the Euclidean length $\|e\|_2$ of the error vector e . That “geometrical” interpretation of the goal can be useful. So we start with that.

4.5.3 Linear least squares

The simplest approach is to seek the straight line $y = f(x) = c_0 + c_1x$ that minimizes the total square sum error,

$$E(c_0, c_1) = \sum_i e_i^2 = \sum_i (c_0 + c_1x_i - y_i)^2.$$

Note well that the unknowns here are just the two values c_0 and c_1 , and E is a fairly simple polynomial function of them. The minimum error must occur at a critical point of this function, where both partial derivatives are zero:

$$\frac{\partial E}{\partial c_0} = 2 \sum_i (c_0 + c_1x_i - y_i) = 0,$$

$$\frac{\partial E}{\partial c_1} = 2 \sum_i (c_0 + c_1x_i - y_i)x_i = 0.$$

These are just simultaneous linear equations, which is the secret of why the least squares approach is so much easier than any alternative. The equations are:

$$\begin{bmatrix} \sum_i 1 & \sum_i x_i \\ \sum_i x_i & \sum_i x_i^2 \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \end{bmatrix} = \begin{bmatrix} \sum_i y_i \\ \sum_i x_i y_i \end{bmatrix}$$

where of course $\sum_i 1$ is just N .

It will help later to introduce the notation

$$m_j = \sum_i x_i^j, \quad p_j = \sum_i x_i^j y_i$$

so that the equations are

$$Mc = p$$

with

$$M = \begin{bmatrix} m_0 & m_1 \\ m_1 & m_2 \end{bmatrix}, p = \begin{bmatrix} p_0 \\ p_1 \end{bmatrix}, c = \begin{bmatrix} c_0 \\ c_1 \end{bmatrix}.$$

(0-based indexing wins this time.)

```
def linefit(x, y):
    m0 = len(x)
    m1 = sum(x)
    m2 = sum(x**2)
    M = np.array([[m0, m1], [m1, m2]])
    p = np.array([sum(y), sum(x*y)])
    c = solveLinearSystem(M, p)
    return c
```

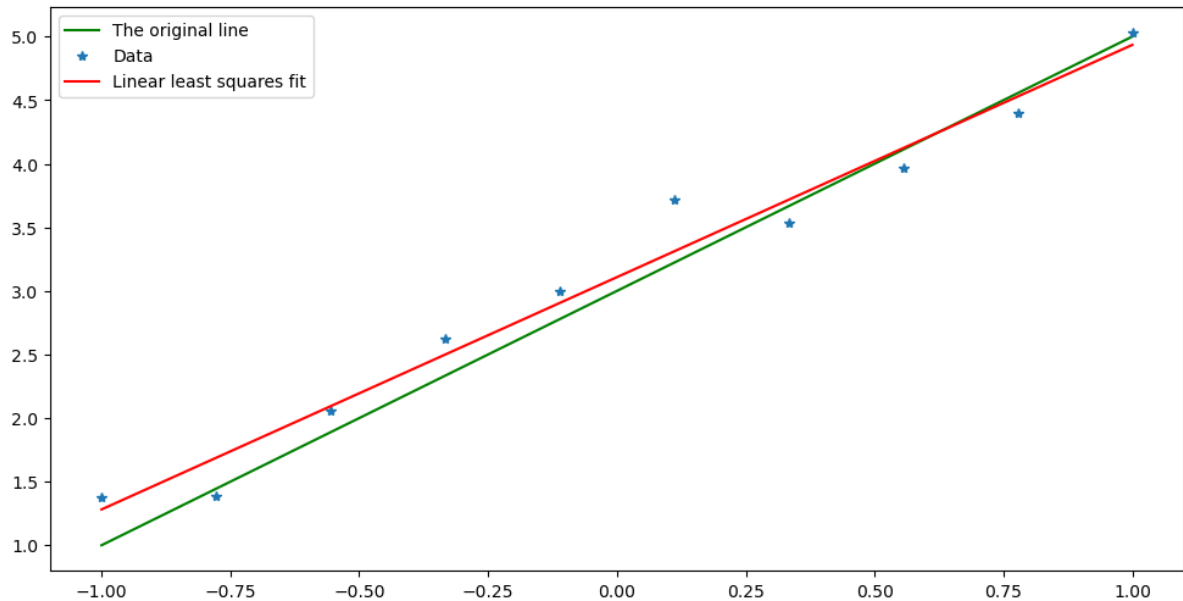
```

N = 10
x = np.linspace(-1, 1, N)
# Emulate a straight line with measurement errors:
# random(N) gives N values uniformly distributed in the range [0,1],
# and so with mean 0.5.
# Thus subtracting 1/2 simulates more symmetric "errors", of mean zero.
yline = 2*x + 3
y = yline + (random(N) - 0.5)

figure(figsize=[12,6])
plot(x, yline, 'g', label="The original line")
plot(x, y, '*', label="Data")
c = linefit(x, y)
print("The coefficients are", c)
xplot = np.linspace(-1, 1, 100)
plot(xplot, evaluatePolynomial(xplot, c), 'r', label="Linear least squares fit")
legend(loc="best");

```

The coefficients are [3.1081582 1.82658183]



4.5.4 Least squares fitting to higher degree polynomials

The method above extends to finding a polynomial

$$p(x) = c_0 + c_1x + \cdots + c_nx^n$$

that gives the best least squares fit to data

$$(x_1, y_1), \dots, (x_N, y_N)$$

in that the coefficients c_k given the minimum of

$$E(c_0, \dots, c_n) = \sum_i (p(x_i) - y_i)^2 = \sum_i \left(y_i - \sum_k c_k x_i^k \right)^2$$

Note that when $N = n + 1$, the solution is the interpolating polynomial, with error zero.

The necessary conditions for a minimum are that all $n + 1$ partial derivatives of E are zero:

$$\frac{\partial E}{\partial c_j} = 2 \sum_i \left(y_i - \sum_k c_k x_i^k \right) x_i^j = 0, \quad 0 \leq j \leq n.$$

This gives

$$\sum_i \sum_k (c_k x_i^{j+k}) = \sum_k \left(\sum_i x_i^{j+k} \right) c_k = \sum_i y_i x_i^j, \quad 0 \leq j \leq n,$$

or with the notation $m_k = \sum_i x_i^k$, $p_k = \sum_i x_i^k y_i$ introduced above,

$$\sum_k m_{j+k} c_k = p_j, \quad 0 \leq j \leq n.$$

That is, the equations are again $Mc = p$, but now with

$$M = \begin{bmatrix} m_0 & m_1 & \dots & m_n \\ m_1 & m_2 & \dots & m_{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ m_n & m_{n+1} & \dots & m_{2n} \end{bmatrix}, \quad p = \begin{bmatrix} p_0 \\ p_1 \\ \vdots \\ p_n \end{bmatrix}, \quad c = \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix}.$$

Remark 3.5.1 (Alternative geometrical derivation)

In the next section *Least-squares Fitting to Data: Appendix on The Geometrical Approach*, another way to derive this result is given, using geometry and linear algebra instead of calculus.

```
def fitPolynomialLeastSquares(x, y, n):
    """Compute the coefficients c_i of the polynomial of degree n that give the best
    least squares fit to data (x[i], y[i])."""
    N = len(x)
    m = np.zeros(2*n+1)
    for k in range(2*n+1):
        m[k] = sum(x**k)
    M = np.zeros([n+1, n+1])
    for i in range(n+1):
        for j in range(n+1):
            M[i, j] = m[i+j]
    p = np.zeros(n+1)
    for k in range(n+1):
        p[k] = sum(x**k * y)
    c = solveLinearSystem(M, p)
    return c
```

```
N = 10
n = 3
xdata = np.linspace(0, np.pi/2, N)
ydata = np.sin(xdata)

figure(figsize=[14, 5])
```

(continues on next page)

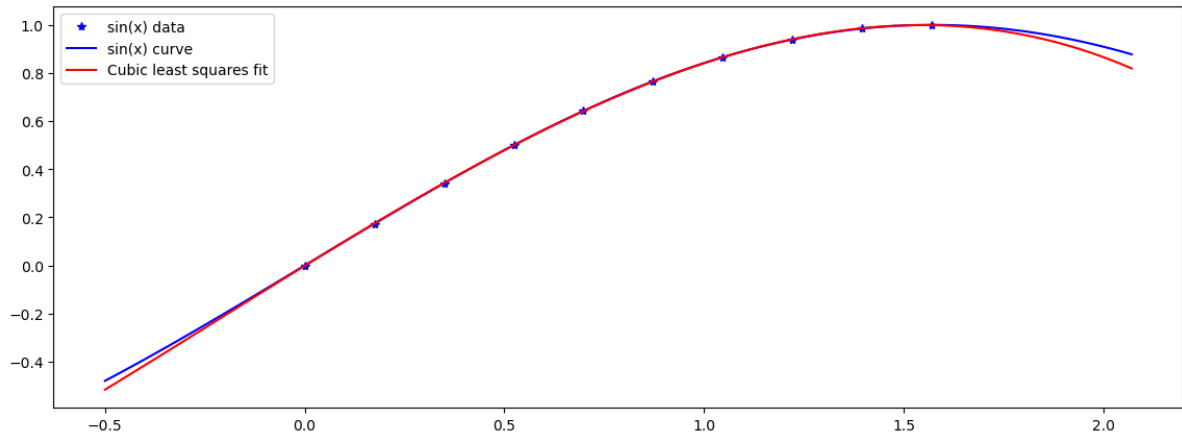
(continued from previous page)

```

plot(xdata, ydata, 'b*', label="sin(x) data")
xplot = np.linspace(-0.5, np.pi/2 + 0.5, 100)
plot(xplot, np.sin(xplot), 'b', label="sin(x) curve")
c = fitPolynomialLeastSquares(xdata, ydata, n)
print("The coefficients are", c)
plot(xplot, evaluatePolynomial(xplot, c), 'r', label="Cubic least squares fit")
legend(loc="best");

```

The coefficients are [-0.00108728 1.02381733 -0.06859178 -0.11328717]

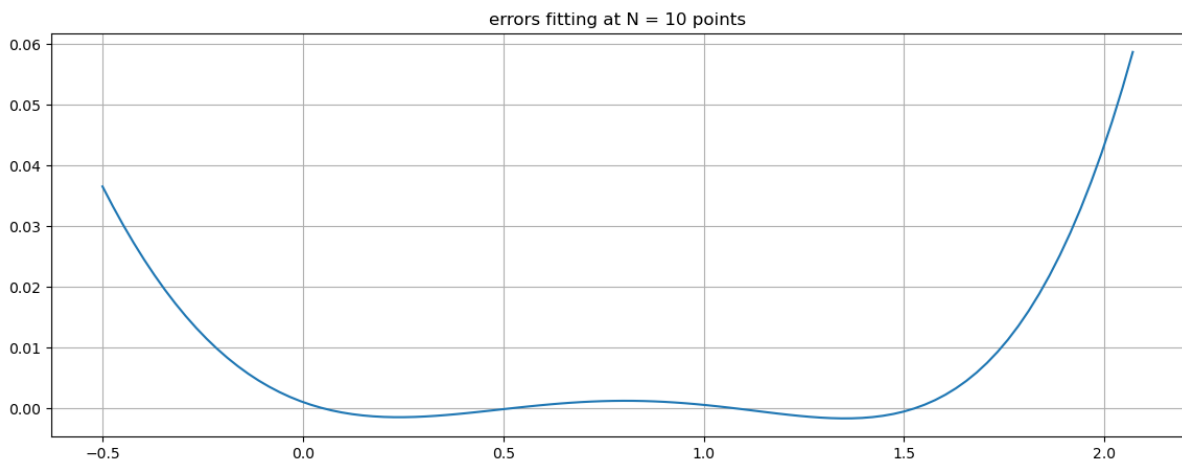


The discrepancy between the original function $\sin(x)$ and this cubic is:

```

figure(figsize=[14,5])
plot(xplot, np.sin(xplot) - evaluatePolynomial(xplot, c))
title("errors fitting at N = 10 points")
grid(True);

```



What if we fit at more points?

```

N = 50
xdata = np.linspace(0, np.pi/2, N)
ydata = np.sin(xdata)

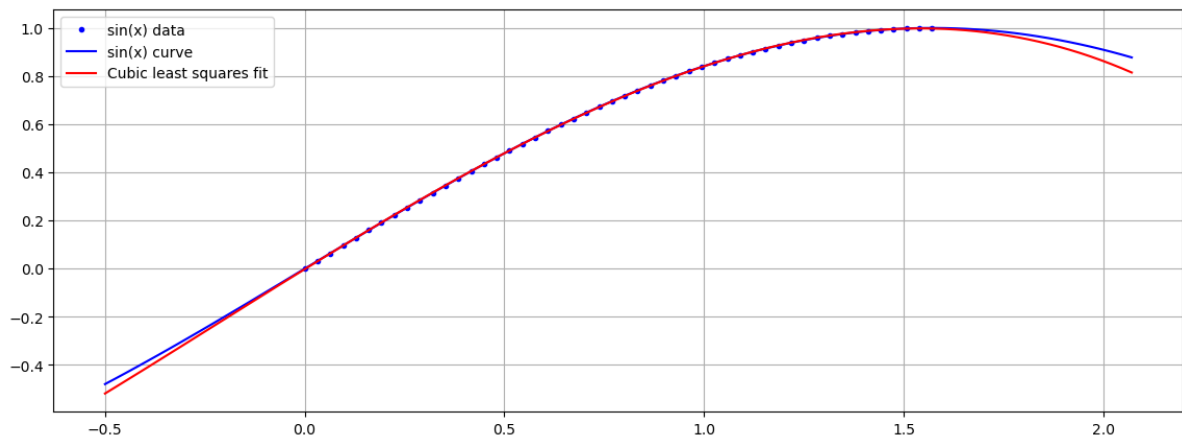
```

(continues on next page)

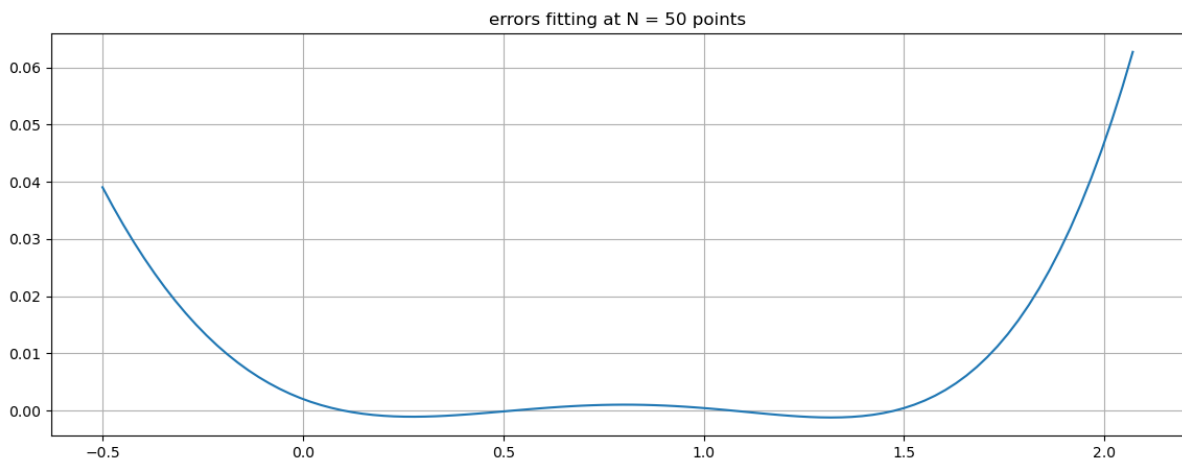
(continued from previous page)

```
figure(figsize=[14,5])
plot(xdata, ydata, 'b.', label="sin(x) data")
plot(xplot, np.sin(xplot), 'b', label="sin(x) curve")
c = fitPolynomialLeastSquares(xdata, ydata, n)
print("The coefficients are", c)
plot(xplot, evaluatePolynomial(xplot, c), 'r', label="Cubic least squares fit")
legend(loc="best")
grid(True);
```

The coefficients are [-0.00200789 1.02647568 -0.06970463 -0.1137182]



```
figure(figsize=[14,5])
plot(xplot, np.sin(xplot) - evaluatePolynomial(xplot, c))
title("errors fitting at N = 50 points")
grid(True);
```



Not much changes!

This hints at another use of least squares fitting: fitting a simpler curve (like a cubic) to a function (like $\sin(x)$), rather than to discrete data.

4.5.5 Nonlinear fitting: power-law relationships

When data (x_i, y_i) is inherently positive, it is often natural to seek an approximate power law relationship

$$y_i \approx cx_i^p$$

That is, one seeks the power p and scale factor c that minimizes error in some sense.

When the magnitudes and the data y_i vary greatly, it is often appropriate to look at the relative errors

$$e_i = \left| \frac{cx_i^p - y_i}{y_i} \right|$$

and this can be shown to be very close to looking at the absolute errors of the logarithms

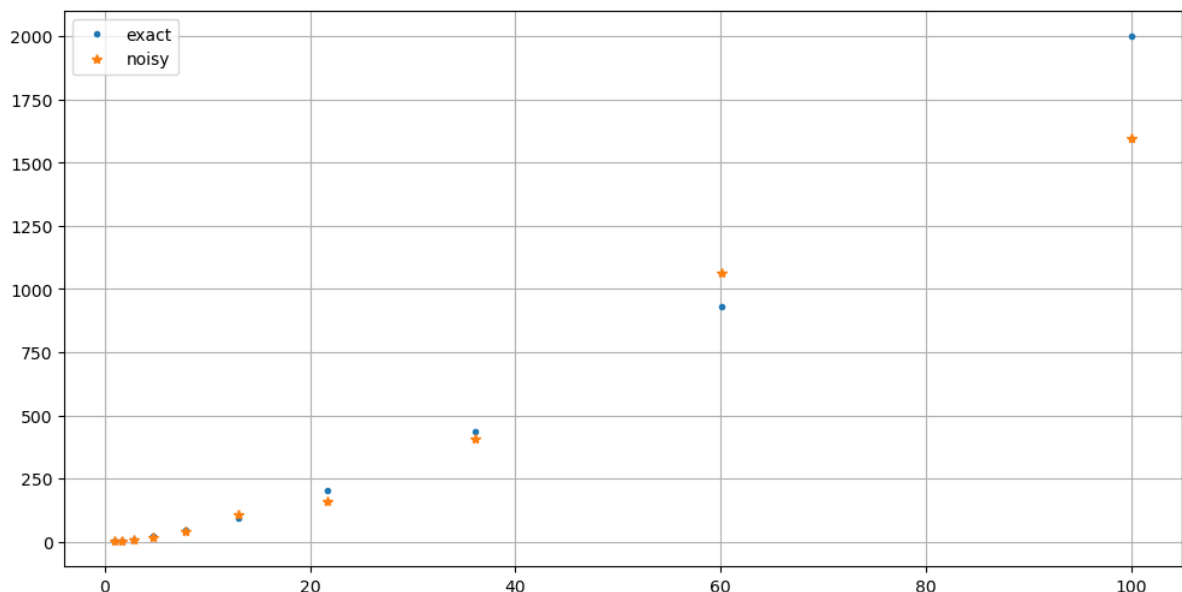
$$|\ln(cx_i^p) - \ln(y_i)| = |\ln(c) + p \ln(x_i) - \ln(y_i)|$$

Introducing the new variables $X_i = \ln(x_i)$, $Y_i = \ln(Y_i)$ and $C = \ln(c)$, this becomes the familiar problem of finding a linear approximation of the data Y_i by $C + pX_i$.

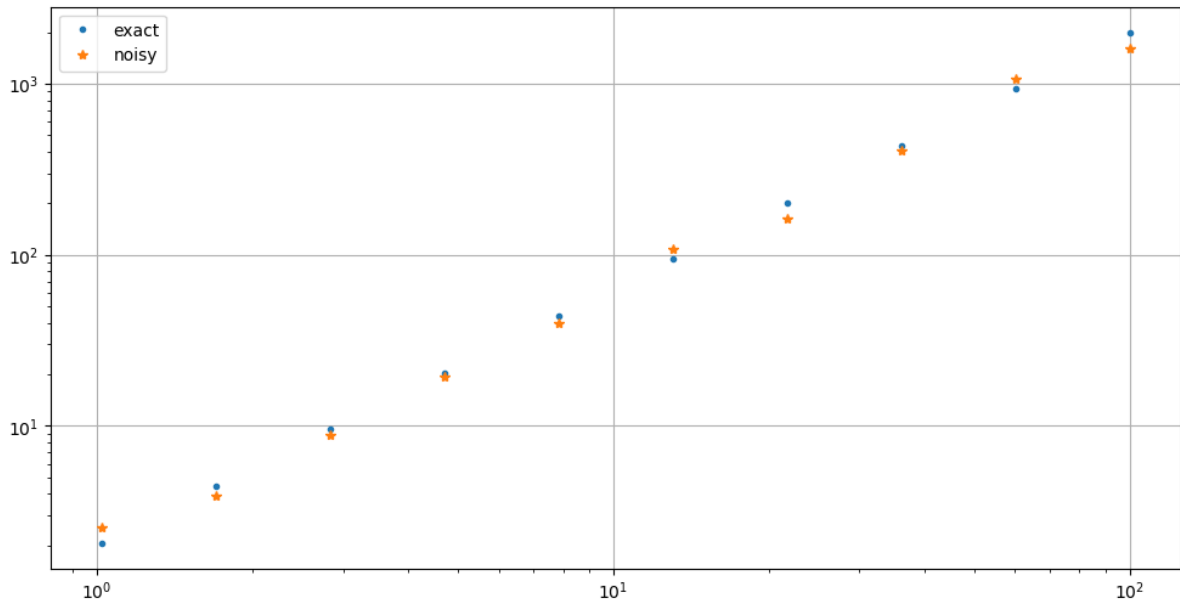
4.5.6 A simulation

```
cexact = 2.0
pexact = 1.5
x = np.logspace(0.01, 2.0, 10)
xplot = np.logspace(0.01, 2.0) # For graphs later
yexact = cexact * x**pexact
y = yexact * (1.0 + (random(len(yexact)) - 0.5)/2)
```

```
figure(figsize=[12,6])
plot(x, yexact, '.', label='exact')
plot(x, y, '*', label='noisy')
legend()
grid(True);
```



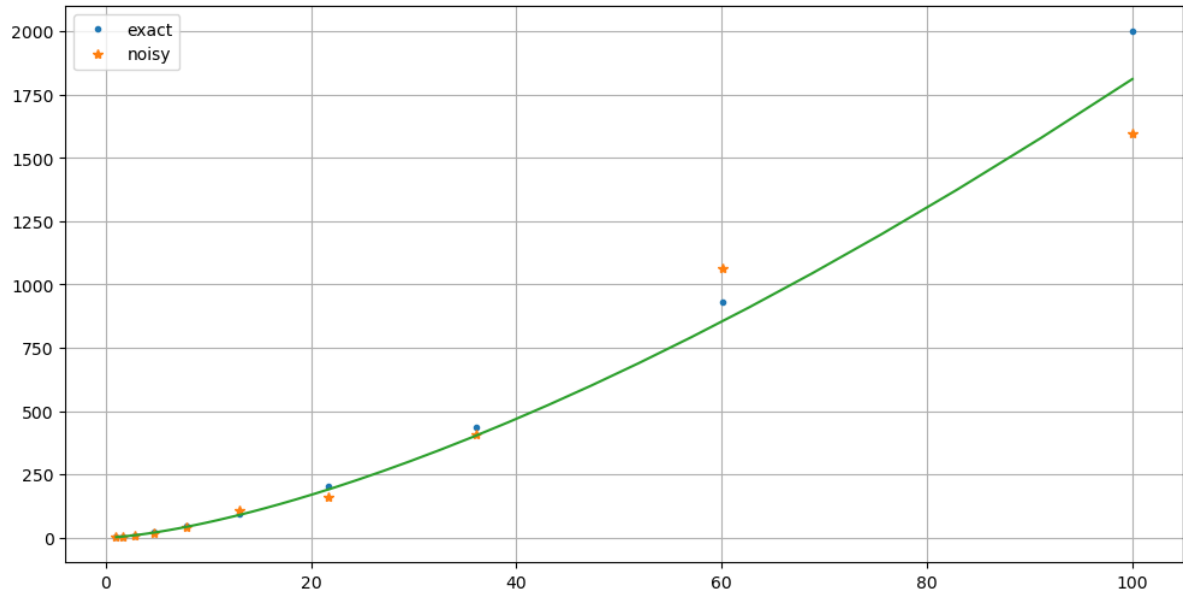

```
figure(figsize=[12,6])
loglog(x, yexact, '.', label='exact')
loglog(x, y, '*', label='noisy')
legend()
grid(True)
```



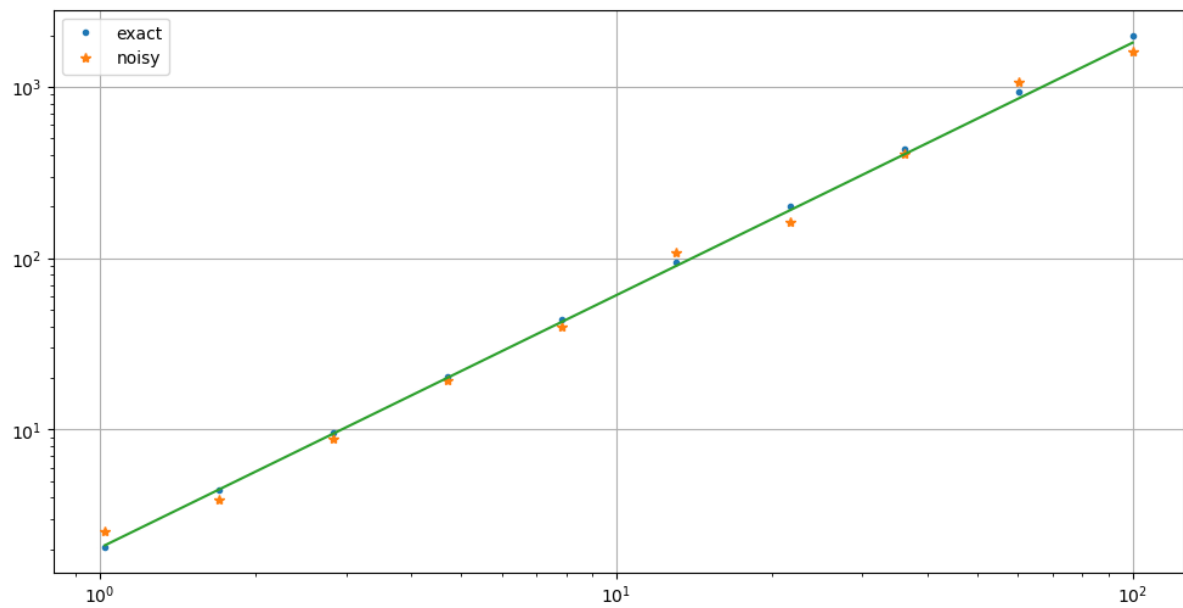
```
X = np.log(x)
Y = np.log(y)
Cp = fitPolynomialLeastSquares(X, Y, 1)
C = np.exp(Cp[0])
p = Cp[1]
print(f"C={C}, p={p}")
```

```
C=2.0413344277069996, p=1.474126908463272
```

```
figure(figsize=[12,6])
plot(x, yexact, '.', label='exact')
plot(x, y, '*', label='noisy')
plot(xplot, C * xplot**p)
legend()
grid(True);
```



```
figure(figsize=[12,6])
loglog(x, yexact, '.', label='exact')
loglog(x, y, '*', label='noisy')
loglog(xplot, C * xplot**p)
legend()
grid(True);
```



4.6 Least-squares Fitting to Data: Appendix on The Geometrical Approach

References:

- Chapter 4 *Least Squares* of [Sauer, 2022], sections 1 and 2.
- Section 8.1 *Discrete Least Squares Approximation* of [Burden et al., 2016].

4.6.1 Introduction

We have seen that one common and important approach to approximating data

$$(x_i, y_i), \quad 1 \leq i \leq N$$

by a polynomial $y = p(x) = c_0 + \dots + c_n x^n$ of degree at most n is to minimize the “average” of the errors

$$e_i = y_i - f(x_i),$$

in the sense of the *root-mean-square error* $E_{RMS} = \sqrt{\sum_{i=1}^N e_i^2}$. Equivalently, we will avoid the square root and just minimize the sum of the squares of the errors:

$$E(c_0, c_1, \dots, c_n) = \sum_{i=1}^N e_i^2$$

4.6.2 Linear least squares: minimizing RMS error using calculus

One way to derive the needed formulas is by seeking the critical point of the above function via the $n + 1$ equations

$$\frac{\partial E}{\partial c_i} = 0, \quad 0 \leq i \leq n$$

Fortunately these give a system of linear equations, and it has a unique solution, thus giving the desired global minimum.

However, there is another “geometrical” approach, that is also relevant as an introduction to strategies also used for other minimization problems, for example with application to the numerical solutions of boundary value problems for differential equations.

4.6.3 Linear least squares: minimizing RMS error by minimizing “Euclidean” distance with geometry

For approximation by a polynomial $y = p(x) = c_0 + \dots + c_n x^n$, we can think of the data $y_i, 1 \leq i \leq N$ as giving a point in N -dimensional space $(R)^N$, and the approximations as giving another point with coordinates $\tilde{y}_i := p(x_i)$.

Then the least squares problem is to minimize the Euclidean distance $\|y - \tilde{y}\|_2$.

One way to think of this is that we attempt unsuccessfully to solve the collocation equations $p(x_i) = y_i$ as an *over-determined* system of N equations in $n + 1$ unknowns $Ac = y$, where

$$A = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & \dots & x_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_i & x_i^2 & \dots & x_i^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^n \end{bmatrix}$$

so that Ac evaluates the polynomial at all the x_i values.

Now we introduce a key geometrical idea: the possible values of $\tilde{y} = Ac$ lie in an $(n + 1)$ -dimensional sub-space or “hyperplane” within $\mathbb{R}^N$, and the point in this hyper-plane closest to $y \in \mathbb{R}^N$ is the perpendicular projection of the latter point onto this hyper-plane: that is, the error vector $e = y - \tilde{y}$ is perpendicular to every vector in the subspace of vectors Ac' . Thus, $e \perp Ac'$ for every $c' \in \mathbb{R}^{n+1}$.

Writing this in terms of inner products,

$$(y - \tilde{y}, Ac') = 0 \quad \text{for every } c' \in \mathbb{R}^{n+1}.$$

Recall that $(x, Ay) = (A^T x, y)$ where A^T is the transpose of A : the mirror image with $a_{i,j}^T = a_{j,i}$.

Using this gives

$$(A^T(y - \tilde{y}), c') = 0 \quad \text{for every } c' \in (\mathbb{R})^{n+1}.$$

and so the vector at left must be zero: $A^T(y - \tilde{y}) = 0$.

Inserting $\tilde{y} = Ac$ gives $A^T y = A^T Ac$, so

$$Mc = A^T y$$

where $M := A^T A$

Since here A is $N \times (n + 1)$, A^T is $(n + 1) \times N$, and the product M is an $(n + 1) \times (n + 1)$ square matrix.

Further calculation shows that in fact

$$M = \begin{bmatrix} m_0 & m_1 & \dots & m_n \\ m_1 & m_2 & \dots & m_{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ m_n & m_{n+1} & \dots & m_{2n} \end{bmatrix}, \quad m_k = \sum_{i=1}^N x_i^k$$

and the right-hand side is

$$A^T y = p = [p_0, p_1, \dots, p_n]^T, \quad p_k = \sum_{i=1}^N x_i^k y_i$$

so these equations are the same ones $Mc = p$ given by the previous calculus derivation.

DERIVATIVES AND DEFINITE INTEGRALS

5.1 Approximating Derivatives by the Method of Undetermined Coefficients

Updated 2023-11-12, with some rephrasing and corrections.

References:

- Section 5.1 *Numerical Differentiation* of [Sauer, 2022].
- Section 4.1 *Numerical Differentiation* of [Burden et al., 2016].
- Section 4.2 *Estimating Derivatives and Richardson Extrapolation* of [Chenney and Kincaid, 2012].

We have seen several formulas for approximating a derivative $Df(x)$ or higher derivative $D^k f(x)$ in terms of several values of the function f , such as

$$Df(x) \approx D_h f(x) := \frac{f(x+h) - f(x)}{h} \quad (5.1)$$

and

$$D^2 f(x) \approx \delta^2 f(x) := \frac{f(x-h) - 2f(x) + f(x+h)}{h^2}. \quad (5.2)$$

For the first case we can use the Taylor formula for $n = 1$,

$$f(x+h) = f(x) + Df(x)h + \frac{1}{2}D^2 f(\xi_x)h^2 \quad \text{where } \xi_x \text{ is between } x \text{ and } x+h$$

(see Equations (2.5) or (2.8) in the section *Taylor's Theorem and the Accuracy of Linearization*); this gives

$$D_h f(x) = \frac{f(x+h) - f(x)}{h} = Df(x) + \frac{D^2 f(\xi_x)}{2}h$$

leading to the error formula

$$D_h f(x) - Df(x) = \frac{1}{2}D^2 f(\xi_x)h, = O(h) \quad \text{or equivalently, } o(1).$$

The approximations in equations (5.1) and (5.2) of k -th derivatives ($k = 1$ or 2 so far) are linear combinations of values of f at various points, with the denominator scaling with the k -th power of the mode spacing scale h ; this makes sense given the linearity of derivatives and the way that the k -th derivative scales when one rescales $f(x)$ to $f(cx)$.

Thus we will make the *Ansatz* that the k -th derivative $D^k f(x)$ can be approximated using values at the $r-l+1$ equally spaced points

$$x+lh, x+(l+1)h, \dots, x+rh$$

where the integers l and r can be negative, positive or zero. The assumed form then is

$$D^k f(x) \approx D_h^k f(x) = \frac{C_l f(x+lh) + C_{l+1} f(x+(l+1)h) + \dots + C_r f(x+rh)}{h^k} + O(h^p)$$

(The reason for the power k in the denominator will be seen soon.)

So we seek to determine the values of the initially undetermined coefficients C_i , by the criterion of giving an error $O(h^p)$ with the highest possible order p . With $r-l+1$ coefficients to choose, we generally get $p = r-l+1-k$, but with symmetry $l = -r$ and k even we get one better, $p = r-l+2-k$, because the order p must then be even. Thus we need the number of points $r-l+1$ to be more than k : for example, at least two for a first derivative as already seen.

Example 4.1.1 (The basic forward difference approximation)

$$Df(x) = \frac{f(x+h) - f(x)}{h} + O(h)$$

has $k = 1, l = 0, r = 1, p = 1$.

Example 4.1.2 (A three-point one-sided difference approximation of the first derivative)

This is the case $k = 1$ and can be sought with $l = 0, r = 2$, as

$$Df(x) = \frac{C_0 f(x) + C_1 f(x+h) + C_2 f(x+2h)}{h} + O(h^p)$$

and the most accurate choice is $C_0 = -3/2, C_1 = 2, C_2 = -1/2$, again of second order, which is exactly $p = r-l+1-k$, with no “symmetry bonus”:

$$Df(x) \approx \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} + O(h^2).$$

One can use Taylor’s Theorem to *check* an approximation like this, and also get information about its accuracy. To do this, insert a Taylor series formula with center x , like

$$f(x+h) = f(x) + Df(x)h + \frac{D^2 f(x)}{2} h^2 + \frac{D^3 f(x)}{6} h^3 + \dots$$

If you are not sure how accurate the result is, you might need to initially be vague about how many terms are needed, so I will do it that way and then go back and be more specific once we know more.

A series for $f(x+2h)$ is also needed:

$$\begin{aligned} f(x+2h) &= f(x) + Df(x)(2h) + \frac{D^2 f(x)}{2} (2h)^2 + \frac{D^3 f(x)}{6} (2h)^3 + \dots \\ &= f(x) + 2Df(x)h + \frac{D^2 f(x)}{2} 4h^2 + \frac{D^3 f(x)}{6} 8h^3 + \dots \\ &= f(x) + 2Df(x)h + 2D^2 f(x)h^2 + \frac{4D^3 f(x)}{3} h^3 + \dots \end{aligned}$$

Insert these into the above three-point formula, and see how close it is to the exact derivative:

$$\begin{aligned} &\frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} \\ &= \frac{-3f(x) + 4[f(x) + Df(x)h + \frac{D^2 f(x)}{2} h^2 + \frac{D^3 f(x)}{6} h^3 + \dots] - [f(x) + 2Df(x)h + 2D^2 f(x)h^2 + \frac{4D^3 f(x)}{3} h^3 + \dots]}{2h} \end{aligned}$$

Now gather terms with the same power of h (which is also gathering terms with the same order of derivative):

$$\begin{aligned}\frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} &= f(x)\frac{-3+4-1}{2h} + Df(x)\frac{4-2}{2} + D^2f(x)\frac{4/4-2/2}{h} + D^3f(x)\frac{4/12-4/6^2}{h} + \dots \\ &= Df(x) - \frac{D^3f(x)}{3}h^2 + \dots\end{aligned}$$

and it is clear that the omitted terms have higher power of h : h^3 and up. That is, they are $O(h^3)$, or more conveniently $o(h^2)$.

Thus we have confirmed that the error in this approximation is

$$Df(x) - \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} = \frac{D^3f(x)}{3}h^2 + o(h^2) = O(h^2).$$

Example 4.1.3 (A three-point centered difference approximation of $D^2f(x)$)

This has $k = 2$, $l = -1$, $r = 1$ and so

$$D^2f(x) \approx \frac{C_{-1}f(x-h) + C_0f(x) + C_1f(x+h)}{h^2}$$

and it can be found (as discussed below) that the coefficients $C_{-1} = C_1 = 1$, $C_0 = -2$ give the highest order error: $p = 2$; one better than $p = r - l + 1 - k = 1$ due to symmetry:

$$D^2f(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} + O(h^2).$$

5.1.1 Method 1: use Taylor polynomials in h of degree $p+k-1$

(so with error terms $O(h^{p+k})$.)

Each of the terms $f(x+ih)$ in the above formula for the approximation $D_h^k f(x)$ of the k -th derivative $D_k f(x)$ can be expanded with the Taylor Formula up to order $p+k$,

$$f(x+ih) = f(x) + (ih)Df(x) + (ih)^2/2D^2f(x) + \dots + (ih)^j/j!D^j f(x) + \dots + (ih)^{p+k}/(p+k)!D^{p+k} f(x) + o(h^{p+k})$$

Then these can be rearranged, putting the terms with the same derivative $D^j f(x)$ together — all of which have the same factor h^j in the numerator, and so the same factor h^{j-p} overall:

$$\begin{aligned}D_h^k f(x) &= (C_l + \dots + C_r)f(x)h^{-k} \\ &\quad + (lC_l + \dots + rC_r)Df(x)h^{1-k} \\ &\quad + (l^2C_l + \dots + r^2C_r)D^2f(x)h^{2-k} \\ &\quad \vdots \\ &\quad + (l^jC_l + \dots + r^jC_r)D^j f(x)h^{j-k} \\ &\quad \vdots \\ &\quad + (l^{p+k}C_l + \dots + p+k^jC_r)D^{p+k} f(x)h^p \\ &\quad + o(h^p)\end{aligned}$$

The final “small” term $o(h^p)$ comes from the terms $o(h^{p+k})$ in each Taylor’s formula term, each divided by h^k .

We want this whole thing to be approximately $D^k f(x)$, and the strategy is to match the coefficients of the derivatives:

- Matching the coefficients of $D_h^k f(x)$,

$$(l^k C_l + \dots + r^k C_r) D^k f(x) h^{k-k} = (l^k C_l + \dots + r^k C_r) D^k f(x) = D^k f(x)$$

so

$$l^k C_l + \dots + r^k C_r = 1 =$$

- On the other hand, there should be no term with factor $f(x)h^{-k}$, so

$$C_l + \dots + C_r = 0$$

- More generally, for any j other than k the coefficients should vanish, so

$$l^j C_l + \dots + r^j C_r = 0, \quad 0 \leq j \leq p+k \text{ except for } j = k$$

This last line gives $p+k$ linear equations in the $p+k+1$ coefficients $C_1, \dots, C_{p+k}$, and then the previous equation gives us a total of $p+k+1$ equations — as needed for the existence of a unique solution.

$$C_l + \dots + C_r = 0 \tag{5.3}$$

$$l^j C_l + \dots + r^j C_r = 0, j \neq k \tag{5.4}$$

$$l^k C_l + \dots + r^k C_r = 1 \tag{5.5}$$

$$\tag{5.6}$$

And indeed it can be verified that the resulting matrix for this system of equations is non-singular, and so there is a unique solution for the coefficients $C_l \dots C_r$.

Exercise A

A) Derive the formula in [Example 4.1.1](#).

Do this by setting up the three equations as above for the coefficients C_0 , C_1 and C_2 , and solving them. Do this “by hand”, to get exact fractions as the answers; use the two Taylor series formulas, but now take advantage of what we saw above: that the error starts at the terms in $D^3 f(x)$. So use the forms

$$f(x+h) = f(x) + Df(x)h + \frac{D^2 f(x)}{2}h^2 + \frac{D^3 f(x)}{6}h^3 + O(h^4)$$

and

$$f(x+2h) = f(x) + 2Df(x)h + 2D^2 f(x)h^2 + \frac{4D^3 f(x)}{3}h^3 + O(h^4)$$

B) Verify the result in [Example 4.1.3](#).

Again, do this by hand, and exploit the symmetry. Note that it works a bit better than expected, due to the symmetry.

5.1.2 Degree of Precision and testing with monomials

This concept relates to a simpler way of determining the coefficients.

The **degree of precision** of an approximation formula (of a derivative or integral) is the highest degree d such that the formula is exact for all polynomials of degree up to d . For example it can be checked that in the examples above, the degrees of precision are 1, 2, and 3 respectively. All three conform to a general pattern:

Theorem 4.1.1

The degree of precision is $d = p + k - 1$, so in the typical case with no “symmetry bonus” $d = r - l$

This is confirmed by the above derivation: for f any polynomial of degree $p + k - 1$ or less, the Taylor polynomials of degree at most $p + k - 1$ used there have no error.

Thus for example, the minimal symmetric approximation of a fourth derivative, which must have even order $p = 2$, will have degree of precision 5.

5.1.3 Method 2: use monomials of degree up to $p+k-1$

From the above degree of precision result, one can determine the coefficients by requiring degree of precision $p + k - 1$, and for this it is enough to require exactness for each of the simple monomial functions $1, x, x^2$, and so on up to x^{p+k-1} .

Also, this only needs to be tested at $x = 0$, since “translating” the variables does not effect the result.

This is probably the simplest method in practice.

Example 4.1.4

Let us revisit [Example 4.1.2](#). The goal is to get exactness in

$$\frac{C_0 f(x) + C_1 f(x+h) + C_2 f(x+2h)}{h} = Df(x)$$

for the monomials $f(x) = 1, f(x) = x$, and so on, to the highest power possible, and this only needs to be checked at $x = 0$.

First, $f(x) = 1$, so $Df(0) = 0$:

$$\frac{C_0 \times 1 + C_1 \times 1 + C_2 \times 1}{h} = 0,$$

so

$$C_0 + C_1 + C_2 = 0$$

Next, $f(x) = x$, so $Df(0) = 1$:

$$\frac{C_0 f(0) + C_1 f(h) + C_2 f(2h)}{h} = \frac{C_0 0 + C_1 h + C_2 2h}{h} = C_1 + 2C_2 = 1$$

so

$$C_1 + 2C_2 = 1$$

We need at least three equations for the three unknown coefficients, so continue with $f(x) = x^2, Df(0) = 0$:

$$\frac{C_0 f(0) + C_1 f(h) + C_2 f(2h)}{h} = \frac{C_0 0 + C_1 h^2 + C_2 (2h)^2}{h} = (C_1 + 4C_2)h = 0$$

so

$$C_1 + 4C_2 = 0$$

We can solve these by elimination; for example:

- The last equation gives $C_1 = -4C_2$
- The previous one then gives $-4C_2 + 2C_2 = 1$, so $C_2 = -1/2$ and thus $C_1 = -4C_2 = 2$.

- The first equation then gives $C_0 = -C_1 - C_2 = -3/2$ all as claimed above.

So far the degree of precision has been shown to be at least 2. In some cases it is better, so let us check by looking at $f(x) = x^3$:

$Df(x) = 0$, whereas

$$\frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} = \frac{-30 + 4h^3 - (2h)^3}{2h} = \frac{-2h^3}{2h} = -4h^2, \neq 0$$

So, no luck this time (that typically requires some symmetry), but this calculation does indicate in a relatively simple way that the error is $O(h^2)$.

Remark 4.1.1

If you want to verify more rigorously the order of accuracy of a formula devised by this method, one can use the “checking” procedure with Taylor polynomials and their error terms as done in [Example 4.1.2](#) above.

Exercise B: like Exercise A, but using Method 2

A) Verify the result in [Example 4.1.1](#), this time by Method 2.

That is, impose the condition of giving the exact value for the derivative at $x = 0$ for the monomial $f(x) = 1$, then the same for $f(x) = x$, and so on until there are enough equations to determine a unique solution for the coefficients.

B) Verify the result in [Example 4.1.3](#), by Method 2.

5.2 Richardson Extrapolation

References:

- Section 5.1.3 *Extrapolation* in [Sauer, 2022].
- Section 4.2 *Richardson Extrapolation* in [Burden *et al.*, 2016].
- Section 4.2 *Estimating Derivatives and Richardson Extrapolation* in [Chenney and Kincaid, 2012].

5.2.1 Motivation

With derivative approximations like

$$\Delta_h f(x) := \frac{f(x+h) - f(x)}{h} = Df(x) + \frac{D^2 f(x)}{2} h + O(h^2) = Df(x) + O(h)$$

and

$$\delta_h^2 f(x) := \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} = D^2 f(x) + \frac{D^4 f(x)}{12} h^2 + O(h^4) = D^2 f(x) + O(h^2)$$

there are limits on the ability to improve accuracy by simply using a smaller value of h : one is that rounding error become problematic.

Another is that when we are approximating the derivative at a collection of points in an interval $[a, b]$, $x_i = a + ih$, $0 \leq i \leq n$, $h = \frac{b-a}{n}$, reducing h requires increasing the number of points $n+1$, and so increases the “cost” (time and other resources needed) of the calculation.

Thus we would like to produce new approximation formulas of higher order p ; that is, with error $O(h^p)$ for p greater than the values $p = 1$ for $\Delta_h f(x)$ or $p = 2$ for $\delta_h^2 f(x)$.

5.2.2 Procedure

The general framework for this is an exact quantity Q_0 for which we have an approximation formula $Q(h)$ with

$$Q(h) = Q_0 + c_p h^p + O(h^q), \quad q > p$$

and we wish to achieve adequate accuracy while keeping h as large as possible.

The kernel of the idea is to initially ignore the smaller part of the error, $O(h^q)$ and just consider

$$Q(h) \approx Q_0 + c_p h^p,$$

and evaluate for two values of h ; most often either h and $2h$ (or h and $h/2$, which is more or less equivalent.)

That gives

$$Q(2h) \approx Q_0 + c_p (2h)^p = Q_0 + c_p 2^p h^p,$$

and with only Q_0 and c_p unknown, this is two (approximate) linear equations in two unknowns, so we can solve for the desired quantity Q_0 by basic Gaussian elimination. This gives

$$Q_0 \approx \frac{2^p Q(h) - Q(2h)}{2^p - 1} =: Q_q(h).$$

But is this new approximation any better than the original? Using the more complete error formula above for $Q(h)$ and its version with h replaced by $2h$,

$$Q(2h) = Q_0 + c_p (2h)^p + O((2h)^q), \quad Q(h) = Q_0 + c_p h^p + O(h^q),$$

one gets

$$Q_q(h) = \frac{2^p Q(h) - Q(2h)}{2^p - 1} = Q_0 + O(h^q),$$

so indeed an improvement, since $q > p$.

Rewriting to get an error estimate

We can get a useful practical error estimate by rewriting the above result as

$$Q_0 \approx Q(h) + \frac{Q(h) - Q(2h)}{2^p - 1} \tag{5.7}$$

so that the quantity

$$E_h := \frac{Q(h) - Q(2h)}{2^p - 1} \approx Q_0 - Q(h) \tag{5.8}$$

is approximately the error in $Q(h)$. Thus,

1. Richardson extrapolation can be viewed as “correcting” Q_h by subtracting of this estimated error:

$$Q_0 \approx Q_q(h) = Q_h + E_h$$

2. This magnitude $|E_h|$ of this error estimate can be used as a (typically pessimistic!) estimate of the error in the corrected result Q_q . Sometimes makes sense to use an even more cautious error estimate, by discarding the denominator $2^p - 1$: using $|Q(h) - Q(2h)|$ as an estimate of the error in the extrapolated value Q_q .

Either way, these follow the pervasive pattern of using the change between the two most recent approximations as an error estimate.

Note the analogy to Newton's method for solving $f(x) = 0$, which can be broken into the two steps

- estimate the error in approximate root x_n as $E_n := -f(x_n)/f'(x_n)$
- update the approximation to $x_{n+1} = x_n + E_n$.

Finally, note that this is always *extrapolation*, in the sense of “going beyond”: the new approximation is on the opposite side of the better of the original approximations from the less accurate of them.

Example 4.2.1

For the basic forward difference approximation above, this process give a three-point method of second order accuracy ($q = 2$):

$$\begin{aligned} \frac{2\Delta_h f(x) - \Delta_{2h} f(x)}{2 - 1} &= 2 \frac{f(x+h) - f(x)}{h} - \frac{f(x+2h) - f(x)}{2h} \\ &= \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} \\ &= Df(x) + O(h^2). \end{aligned}$$

Exercise 1(a)

Apply Richardson extrapolation to the standard three-point, second order accurate approximation $Q(h) := \delta_h^2 f(x)$ of the second derivative $Q_0 := D^2 f(x)$ as given above, and verify that it gives a fourth-order accurate five-point approximation formula.

Exercise 1(b)

As a supplementary exercise, one could verify the order of accuracy directly with Taylor polynomials, or verify that the new formula has degree of precision $d = 5$, and hence is of order $p = 4$ due to the formula $d = p + k - 1$ for approximations of k -th derivatives, given in the notes for Day 11.

One could also derive the same formula “from scratch” using the Method of Undetermined Coefficients.

Exercise 2

Apply Richardson extrapolation to the above one-sided three-point, second order accurate approximation of the derivative $Df(x)$, and verify that it gives a third-order accurate four-point approximation formula.

But note something strange about this new formula: it skips $f(x + 3h)$.

Here, instead of extrapolating, one is probably better off applying the Method of Undetermined Coefficients directly with data $f(x)$, $f(x + h)$, $f(x + 2h)$, $f(x + 3h)$ and $f(x + 4h)$: what order of accuracy does that give?

5.2.3 A variant, more useful for integration and ODE boundary value problems: parameter n

A slight variant of the above is approximation with an integer parameter n , such as approximations of integrals by the (composite) trapezoid rule with n intervals, T_n , or the approximate solution of an ordinary differential equation at the above-described collection of $n + 1$ equally spaced values in domain $[a, b]$. Then a more natural notation of the approximation formula is Q_n instead of $Q(h)$.

The errors of the form $c_p h^p + O(h^q)$ become

$$Q_n = Q_0 + O\left(\frac{1}{n^p}\right) = Q_0 + \frac{c_p}{n^p} + O\left(\frac{1}{n^q}\right).$$

The main difference is that to work with integer values of n , it must be the quantity that is doubled, whereas doubling of h would correspond to halving of n .

The extrapolation formula becomes

$$Q_0 = \frac{2^p Q_{2n} - Q_n}{2^p - 1} + O\left(\frac{1}{n^q}\right). \quad (5.9)$$

Remark 4.2.1

For the slightly more general case of increasing from n to kn , one gets

$$Q_0 = \frac{k^p Q_{kn} - Q_n}{k^p - 1} + O\left(\frac{1}{n^q}\right).$$

A common verbal description for both forms

This can be summarized with the same verbal form as the original formula:

- 2^p times the more accurate approximation,
- minus the less accurate approximation,
- all divided by $(2^p - 1)$

Also

The error in the more accurate approximation is approximated by the difference between the two approximations, divided by $(2^p - 1)$

Rewriting to get an error estimate, again

As with the “ h ” form above, this extrapolation can be broken into two steps

$$E_{2n} := \frac{Q_{2n} - Q_n}{2^p - 1},$$

$$Q_0 = Q_{2n} + E_{2n} + O\left(\frac{1}{n^q}\right).$$

so E_{2n} estimates the error in Q_{2n} , and the improved approximation can be expressed as

$$Q_{2n} + E_{2n}.$$

5.2.4 Repeated Richardson extrapolation

The new improved approximation formulas have the same sort of error formula, but for order q instead of order p , so we could extrapolate again to get an even higher order method, and this can be done numerous times if there is a suitable power series in h or $1/n$ for the errors.

That is not so useful for derivative approximations, where one can get the same or better results with the method of underdetermined coefficients, but can be very useful for integration methods, and for the related task of solving boundary value problems for ordinary differential equations.

For example, it can be applied to the composite trapezoid rule, giving the composite Simpson's rule at the first step, and then a succession of approximations of ever higher order – this is known as the **Romberg method**.

Repeated Richardson extrapolation can also be applied to the approximate solution of differential equations; we might explore that later.

5.3 Definite Integrals, Part 1: The Building Blocks

Updated 2023-11-07, correcting some typos.

References:

- Sections 5.2.1 and 5.2.4 of Chapter 5 *Numerical Differentiation and Integration* in [Sauer, 2022].
- Sections 4.3 *Elements of Numerical Integration* of [Burden *et al.*, 2016].

5.3.1 Introduction

The objective of this and several subsequent sections is to develop methods for approximating a definite integral

$$I = \int_a^b f(x) \, dx$$

This is arguably even more important than approximating derivatives, for several reasons; in particular, because there are many functions for which antiderivative formulas cannot be found, so that the result of the Fundamental Theorem of Calculus, that

$$\int_a^b f(x) \, dx = F(b) - F(a), \text{ for } F \text{ any antiderivative of } f$$

does not help us.

One core idea is to approximate the function f by a polynomial (or several), and use its integral as an approximation. The two simplest possibilities here are approximating by a constant and by a straight line; here we explore the latter; the former will be visited soon.

```
from numpy import array, linspace, exp
from matplotlib.pyplot import figure, plot, title, grid, legend
```

5.3.2 Approximating with a single linear function: the Trapezoid Rule

The idea is to approximate $f : [a, b] \rightarrow \mathbb{R}$ by collocation at the end points of this interval:

$$f(x) \approx L(x) := \frac{f(a)(b-x) + f(b)(x-a)}{b-a}, = f_{ave}(b-a)$$

Then the approximation — which will be called T_1 , for reasons that will become clear soon — is

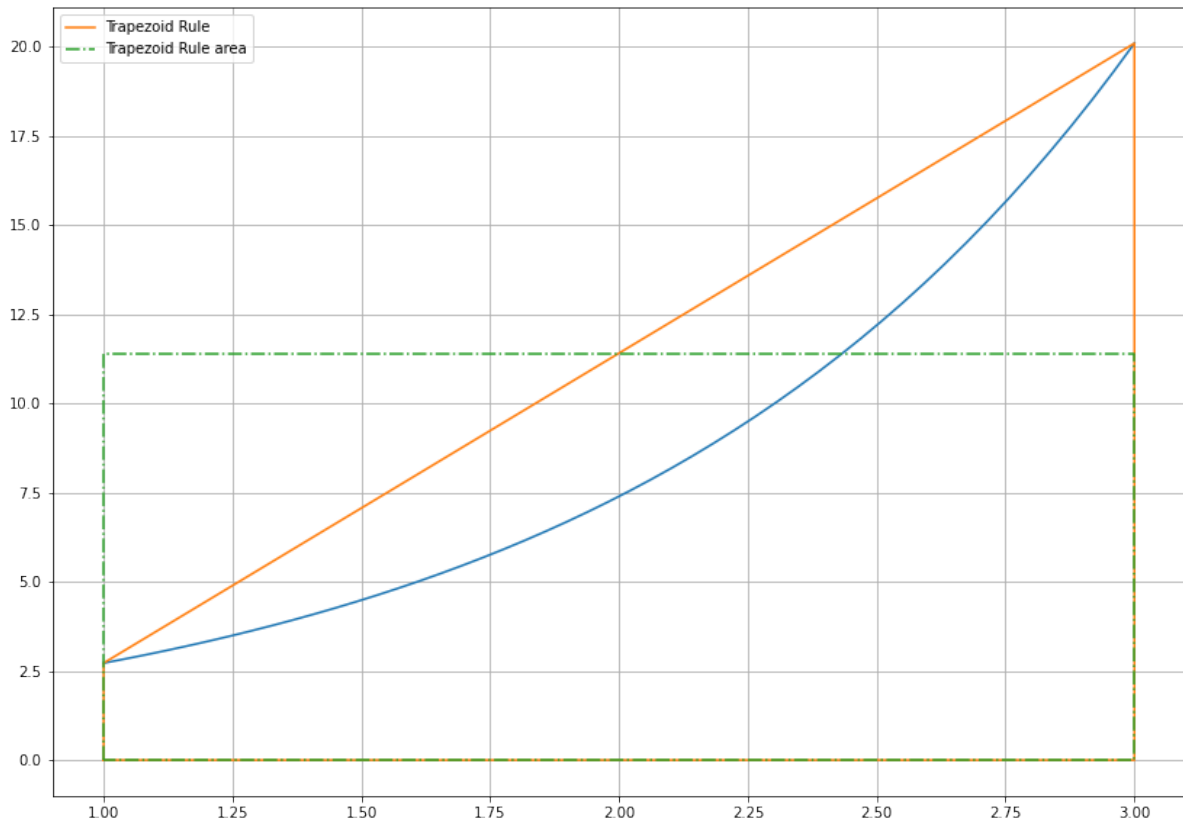
$$I \approx T_1 = \int_a^b L(x) dx = \frac{f(a) + f(b)}{2} (b-a)$$

This can be interpreted as replacing $f(x)$ by f_{ave} the average of the value at the end points, and integrating that simple function.

For the example $f(x) = e^x$ on $[-1, 3]$

```
a = 1
b = 3
def f(x): return exp(x)
```

```
f_ave = (f(a) + f(b))/2
x = linspace(a, b)
figure(figsize=[14,10])
plot(x, f(x))
plot([a, a, b, b, a], [0, f(a), f(b), 0, 0], label="Trapezoid Rule")
plot([a, a, b, b, a], [0, f_ave, f_ave, 0, 0], '-.', label="Trapezoid Rule area")
legend()
grid(True)
```



The approximation T_1 is the area of the orange trapezoid (hence the name!) which is also the area of the green rectangle.

5.3.3 Approximating with a constant: the Midpoint Rule

The idea here is to approximate $f : [a, b] \rightarrow \mathbb{R}$ by its value at the midpoint of the interval, like the building blocks in a Riemann sum with the middle being the intuitive best choice of where to put the rectangle.

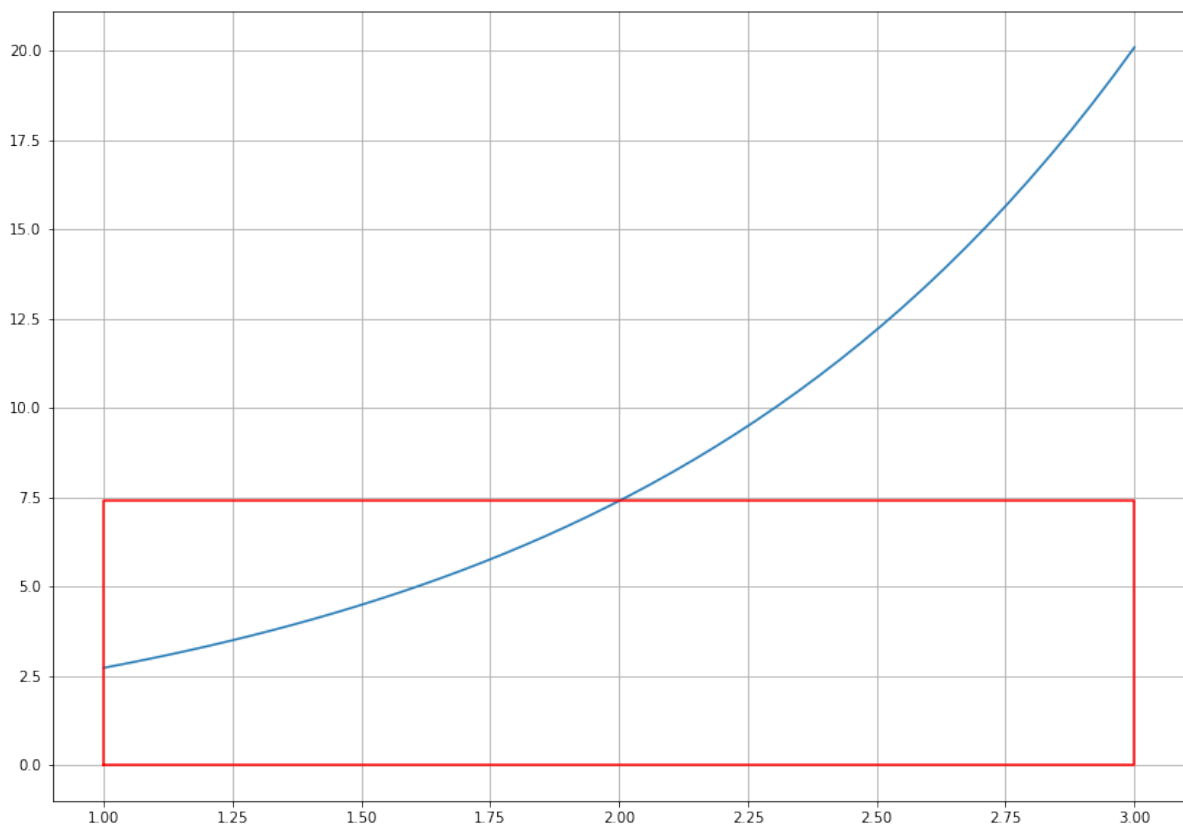
$$f(x) \approx f_{mid} := f\left(\frac{a+b}{2}\right)$$

Then the approximation — which will be called M_1 — is

$$I \approx M_1 = \int_a^b f_{mid} dx = f\left(\frac{a+b}{2}\right)(b-a)$$

For the same example $f(x) = e^x$ on $[-1, 3]$

```
f_mid = f((a+b)/2)
figure(figsize=[14,10])
plot(x, f(x))
plot([a, a, b, b, a], [0, f_mid, f_mid, 0, 0], 'r', label="Midpoint Rule")
grid(True)
```



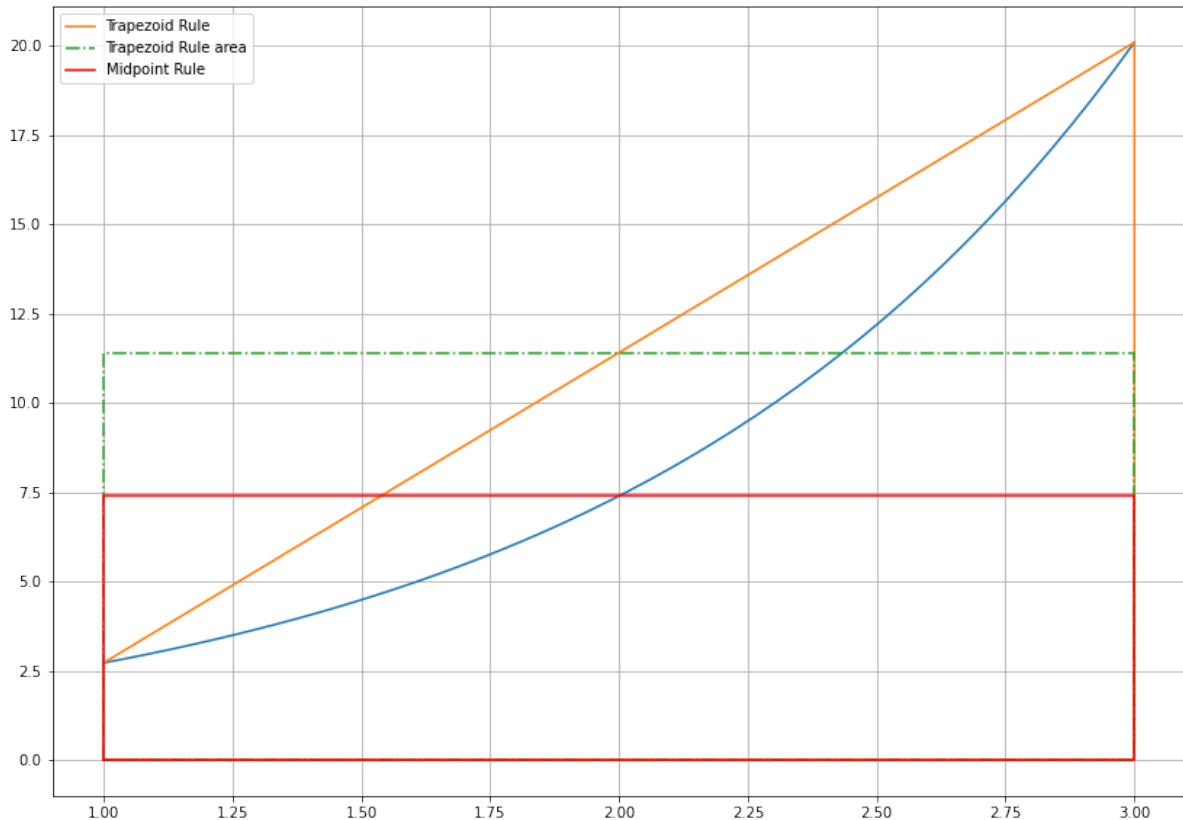
The approximation M_1 is the area of the red rectangle.

The two methods can be compared by combining these graphs:


```

f_mid = f((a+b)/2)
figure(figsize=[14,10])
plot(x, f(x))
plot([a, a, b, b, a], [0, f(a), f(b), 0, 0], label="Trapezoid Rule")
plot([a, a, b, b, a], [0, f_ave, f_ave, 0, 0], '-.', label="Trapezoid Rule area")
plot([a, a, b, b, a], [0, f_mid, f_mid, 0, 0], 'r', label="Midpoint Rule")
legend()
grid(True)

```



5.3.4 Error Formulas

These graphs indicate that the trapezoid rule will over-estimate the error for this and any function that is convex up on the interval $[a, b]$. With closer examination it can perhaps be seen that the Midpoint Rule will instead underestimate in this situation, because its “overshoot” at left is less than its “undershoot” at right.

We can derive error formulas that confirm this, and which are the basis for both practical error estimates and for deriving more accurate approximation methods.

The first such method will be to use multiple small intervals instead of a single bigger one (using piecewise polynomial approximation) and for that, it is convenient to define $h = b - a$ which will become the parameter that we reduce in order to improve accuracy.

Theorem 4.3.1 (Error in the Trapezoid Rule, T_1)

For a function f that is twice differentiable on interval $[a, b]$, the error in the Trapezoid Rule is

$$\int_a^b f(x)dx - T_1 = -\frac{(b-a)^3}{12}f''(\xi) \quad \text{for some } \xi \in [a, b]$$

It will be convenient to define $h := b - a$ so that this becomes

$$\int_a^b f(x)dx - T_1 = -\frac{h^3}{12}f''(\xi) \quad \text{for some } \xi \in [a, b].$$

Theorem 4.3.2 (Error in the Midpoint Rule, M_1)

For a function f that is twice differentiable on interval $[a, b]$ and again with $h = b - a$, the error in the Midpoint Rule is

$$\int_a^b f(x)dx - M_1 = \frac{h^3}{24}f''(\xi) \quad \text{for some } \xi \in [a, b]$$

These will be verified below, using the error formulas for Taylor polynomials and collocation polynomials.

For now, note that:

- The results confirm that for a function that is convex up, the Trapezoid Rule overestimates and the Midpoint Rule underestimates.
- The ratio of the errors is approximately -2 . This will be used to get a better result by using a weighted average: *Simpson's Rule*.
- The errors are $O(h^3)$. This opens the door to Richardson Extrapolation, as will be seen soon in the method of *Romberg Integration*.

Proofs of these error results

One side benefit of the following verifications is that they also offer illustrations of how the two fundamental error formulas help us: Taylor's Formula and its cousin the error formula for polynomial collocation.

To help prove the above formulas, we introduce a result that also helps in various places later:

Theorem 4.3.3 (The Integral Mean Value Theorem)

In an integral

$$\int_a^b f(x)w(x) dx$$

with f continuous and the “weight function” $w(x)$ positive valued (actually, it is enough that $w(x) \geq 0$ and it is not zero everywhere), there is a point $\xi \in [a, b]$ that gives a “weighted average value” for $f(x)$ in the sense that

$$\int_a^b f(x)w(x) dx = \int_a^b f(\xi)w(x) dx = f(\xi) \int_a^b w(x) dx$$

Proof. As f is continuous on the closed, bounded interval $[a, b]$, the **Extreme Value Theorem** from calculus says that f has a minimum L and a maximum H on this interval: $L \leq f(x) \leq H$. Since $w(x) \geq 0$, this gives

$$Lw(x) \leq f(x)w(x) \leq Hw(x)$$

and by integrating,

$$L \int_a^b w(x) dx \leq \int_a^b f(x)w(x) dx \leq H \int_a^b w(x) dx$$

Dividing by $\int_a^b w(x) dx$ (which is positive),

$$L \leq \frac{\int_a^b f(x)w(x) dx}{\int_a^b w(x) dx} \leq H$$

and the Mean Value Theorem says that f attains this value for some $\xi \in [L, H]$:

$$f(\xi) = \frac{\int_a^b f(x)w(x) dx}{\int_a^b w(x) dx} \quad (5.10)$$

Clearing the denominator gives the claimed result.

Proof. (of *Theorem 4.3.1, the trapezoid rule error formula*)

The function integrated to get the Trapezoid Rule is the linear collocating polynomial $L(x)$, and from the section *Error Formulas for Polynomial Collocation*, we have

$$f(x) - L(x) = \frac{f''(\xi_x)}{2}(x-a)(x-b)$$

Integrating each side gives

$$\int_a^b (f(x) - L(x)) dx = I - T_1 = \int_a^b \frac{f''(\xi_x)}{2}(x-a)(x-b) dx$$

To get around the complication that ξ_x depends on x in an unknown way, use the Integral Mean Value Theorem with weight function $w(x) = (x-a)(b-x)$, ≥ 0 for $a \leq x \leq b$. Then with $-f''$ as the function f in (5.10):

$$I - T_1 = - \int_a^b \frac{f''(\xi_x)}{2}(x-a)(b-x) dx = - \frac{f''(\xi)}{2} \int_a^b (x-a)(b-x) dx$$

A bit of calculus gives $\int_a^b (x-a)(b-x) dx = \frac{(b-a)^3}{6}$, so

$$I - T_1 = - \frac{f''(\xi)}{2} \frac{(b-a)^3}{6} = - \frac{f''(\xi)}{12} (b-a)^3 = - \frac{f''(\xi)}{12} h^3,$$

as advertised.

Proof. (of *Theorem 4.3.2, the midpoint rule error formula*)

For this, we can use Taylor's Theorem for the linear approximation

$$f(x) = f(c) + f'(c)(x-c) + \frac{f''(\xi_x)}{2}(x-c)^2$$

with $c = (a+b)/2$, the midpoint. That is,

$$f(x) - f(c) = f'(c)(x-c) + \frac{f''(\xi_x)}{2}(x-c)^2$$

and integrating each side gives

$$\int_a^b f(x) - f(c) dx = I - M_1 = \int_a^b \left[f'(c)(x-c) + \frac{f''(\xi_x)}{2}(x-c)^2 \right] dx$$

Here symmetry helps, by eliminating the first (potentially biggest) term in the error: we use the fact that $a = c - h/2$ and $b = c + h/2$

$$\int_a^b f'(c)(x-c) dx = f'(c) \int_{c-h/2}^{c+h/2} x - c dx = [(x-c)^2/2]_{c-h/2}^{c+h/2} = (h/2)^2 - (h/2)^2 = 0$$

Thus the error simplifies to

$$I - M_1 = \int_a^b \frac{f''(\xi_x)}{2}(x-c)^2 dx$$

and much as above, the Integral Mean Value Theorem can be used, this time with weight function $w(x) = (x-c)^2, \geq 0$:

$$I - M_1 = \frac{f''(\xi)}{2} \int_a^b (x-c)^2 dx$$

Another calculus exercise: $\int_a^b (x-c)^2 dx = \int_{-h/2}^{h/2} x^2 dx = [x^3/3]_{-h/2}^{h/2} = h^3/12$, so indeed,

$$I - M_1 = \frac{f''(\xi)}{24} h^3$$

5.3.5 Appendix: Approximating a Definite Integral With the Left-hand Endpoint Rule

An even simpler approximation of $\int_a^b f(x) dx$ is the *Left-hand Endpoint Rule*, probably seen in a calculus course. For a single interval, this uses the approximation

$$f(x) \approx f(a)$$

leading to

$$I := \int_a^b f(x) dx \approx L_1 := \int_a^b f(a) dx = f(a)(b-a)$$

The corresponding composite rule with n sub-intervals of equal width $h = (b-a)/n$ is

$$L_n = \sum_{i=0}^{n-1} f(x_i)h, = \sum_{i=0}^{n-1} f(a+ih)h$$

with $x_i = a + ih$ as before.

Theorem 4.3.4 (Error in the Left-hand Endpoint Rule, L_1)

For a function f that is differentiable on interval $[a, b]$, the error in the Left-hand Endpoint Rule is

$$\int_a^b f(x) dx - L_1 = \frac{(b-a)^2}{2} f'(\xi), = \frac{h^2}{2} f'(\xi) \quad \text{for some } \xi \in [a, b]$$

Proof. This time use Taylor's Theorem just for the constant approximation with center a :

$$f(x) = f(a) + f'(\xi_x)(x - a)$$

That is,

$$f(x) - f(a) = f'(\xi_x)(x - a)$$

so integrating each side gives

$$\int_a^b f(x) - f(a) dx = I - L_1 = \int_a^b f'(\xi_x)(x - a) dx$$

Using the Integral Mean Value Theorem again, now with weight $w(x) = x - a$ gives

$$\int_a^b f'(\xi_x)(x - a) dx = f'(\xi) \int_a^b (x - a) dx = f'(\xi) \frac{(b - a)^2}{2} = \frac{h^2}{2} f'(\xi) \text{ for some } \xi \in [a, b]$$

and inserting this into the previous formula gives the result.

5.4 Definite Integrals, Part 2: The Composite Trapezoid and Midpoint Rules

References:

- Section 5.2.3 and 5.2.4 of Chapter 5 *Numerical Differentiation and Integration* in [Sauer, 2022].
- Section 4.4 *Composite Numerical Integration* of [Burden et al., 2016].

5.4.1 Introduction

The “elementary” integral approximations of the definite integral

$$I = \int_a^b f(x) dx$$

seen in the previous section the Trapezoid Rule

$$T_1 = \int_a^b L(x) dx = \frac{f(a) + f(b)}{2} (b - a)$$

and the Midpoint Rule

$$M_1 = f\left(\frac{a + b}{2}\right) (b - a)$$

are of course of very low accuracy in themselves. They are however central building blocks for various more accurate methods and also for some good methods for numerical solution of differential equations.

The basic strategy for improving accuracy is to derive the domain of integration $[a, b]$ into numerous smaller intervals, and use these rules on each such sub-interval: the *composite* rules.

In turn, the most straightforward way to do this is to use n sub-intervals of equal width $h = (b - a)/n$, so that the sub-interval endpoints are $x_0 = a + ih$, $0 \leq i \leq n$: that is

sub-intervals $[x_{i-1}, x_i]$, $1 \leq i \leq n$ separated by the nodes.

$$a, a + h, a + 2h, \dots, b - h, b$$

The Composite Midpoint Rule

Using the Midpoint Rule on each interval and summing gives a formula that could be familiar:

$$\begin{aligned} M_n &:= f\left(\frac{x_0 + x_1}{2}\right)h + f\left(\frac{x_1 + x_2}{2}\right)h + \dots + f\left(\frac{x_{n-1} + x_n}{2}\right)h \\ &= f\left(\frac{a + (a + h)}{2}\right)h + f\left(\frac{(a + h) + (a + 2h)}{2}\right)h + \dots + f\left(\frac{(b - h) + b}{2}\right)h \\ &= [f(a + h/2) + f(a + 3h/2) + \dots + f(b - h/2)]h \end{aligned}$$

This is a *Riemann Sum* as used in the definition of the definite integral; possibly the best and natural one in most situations, by using the midpoints of each interval. The theory of definite integrals also guarantees that $M_n \rightarrow I$ as $n \rightarrow \infty$ so long as the function f is continuous — the next question for us will be “how fast?”

The Composite Trapezoid Rule

Using the Trapezoid Rule on each interval instead gives

$$\begin{aligned} T_n &:= \frac{f(x_0) + f(x_1)}{2}h + \frac{f(x_1) + f(x_2)}{2}h + \dots + \frac{f(x_{n-1}) + f(x_n)}{2}h \\ &:= \frac{f(a) + f(a + h)}{2}h + \frac{f(a + h) + f(a + 2h)}{2}h + \dots + \frac{f(b - h) + f(b)}{2}h \\ &= \left[\frac{f(a)}{2} + f(a + h) + f(a + 2h) + \dots + f(b - h) + \frac{f(b)}{2} \right]h \end{aligned}$$

This is also a Riemann sum, with intervals of length $h/2$ at each end, using value at the ends of those intervals, and the rest of width h , with the Midpoint Rule used. So again, we know that $T_n \rightarrow I$ as $n \rightarrow \infty$ and next want to know “how fast?”

Accuracy and Error Formulas

In brief, the errors for each of these rules is the sum of the errors for each of the pieces; I will just state them for now.

Firstly,

$$I - M_n = \sum_{i=1}^n \frac{h^3}{24} f''(\xi_i), \quad \text{for some } \xi_i \in [x_{i-1}, x_i]$$

This can be rewritten as

$$I - M_n = \frac{h^3}{24} \sum_{i=1}^n f''(\xi_i)$$

and as we will see, this sum can have each $f''(\xi_i)$ replace by an “average value” $f''(\xi)$, $\xi \in [a, b]$:

$$I - M_n = \frac{h^3}{24} \sum_{i=1}^n f''(\xi) = \frac{h^3}{24} n f''(\xi) = \frac{h^2}{24} (b - a) f''(\xi)$$

and the most important conclusion for now is that

$$I - M_n = O(h^2)$$

Similarly,

$$I - T_n = -\frac{h^2}{12}(b-a)f''(\xi) = O(h^2)$$

again with $\xi \in [a, b]$, but note well: these two ξ values are probably not the same!

5.4.2 Cancelling Some Error Terms: The Composite Simpson's Rule

Ignoring the ξ values being different, this suggests again that we can cancel some of the errors with a weighted average:

$$S_{2n} := \frac{2M_n + T_n}{3}$$

Indeed we will see that the main, $O(h^2)$, errors cancel out, and also due to symmetry, the error is even in h , so that

$$I - S_{2n} = O(h^4)$$

The name is because this is the *Composite Simpson's Rule*, and the interleaving of the different x values used by M_n and T_n means that it uses $2n + 1$ nodes, and so $2n$ sub-intervals.

The Missing Step: A Generalized Mean Value Theorem

A key step in getting more useful error formulas for approximations of integrals is the following result:

Theorem 4.4.1 (Generalized Mean Value Theorem)

For any continuous function f on an interval $[a, b]$ and any collection of points $x_i \in [a, b]$, $1 \leq i \leq n$, there is a point $\xi \in [a, b]$ for which

$$f(c) = \frac{\sum_{i=1}^n f(x_i)}{n}, \text{ so } \sum_{i=1}^n f(x_i) = nf(c)$$

That is, the value of the function at c is the average of its values at those other points.

Proof. The proof is rather similar to that of [The Integral Mean Value Theorem](#) in the previous section; essentially replacing the integral there by a sum:

As f is continuous on the closed, bounded interval $[a, b]$, the **Extreme Value Theorem** from calculus says that f has a minimum L and a maximum H on this interval. Each of the values $f(x_i)$ is in interval $[L, H]$ so their average is also:

$$f(x_i) \in [L, H] \quad \text{and thus} \quad \frac{\sum_{i=1}^n f(x_i)}{n} \in [L, H]$$

The **Mean Value Theorem** then says that f attains this mean value for some $\xi \in [L, H]$.

Completing the derivation of the error formulas for these composite rules

I will spell this out for the Composite Trapezoid Rule; it works very similarly for the “midpoint” case.

First, break the exact integral up as

$$I = \int_a^b f(x) dx = \sum_{i=1}^n I^{(i)}, \quad \text{where} \quad I^{(i)} = \int_{x_{i-1}}^{x_i} f(x) dx$$

Similarly,

$$T_n = \sum_{i=1}^n T^{(i)}$$

where each $T^{(i)}$ is the Trapezoid Rule approximation of $I^{(i)}$:

$$T^{(i)} = \frac{f(x_{i-1}) + f(x_i)}{2} h$$

The error in T_n is the sum of the errors in each piece:

$$\begin{aligned} I - T_n &= \sum_{i=1}^n I^{(i)} - \sum_{i=1}^n T^{(i)} \\ &= \sum_{i=1}^n (I^{(i)} - T^{(i)}) \\ &= \sum_{i=1}^n -\frac{h^3}{12} f''(\xi_i), \quad x_i \in [x_{i-1}, x_i] \\ &= -\frac{h^3}{12} \sum_{i=1}^n f''(\xi_i) \end{aligned}$$

Now we can use the above mean value result (with f'' in place of f) to replace the last sum above by $nf''(\xi)$, some $\xi \in [a, b]$, so that as claimed,

$$I - T_n = -\frac{h^3}{12} n f''(\xi) = -\frac{h^2}{12} (b - a) f''(\xi) = O(h^2),$$

using $hn = b - a$.

Another error formula, useful for Richardson Extrapolation

Starting from

$$I - T_n = -\frac{h^3}{12} \sum_{i=1}^n f''(\xi_i) = -\frac{h^2}{12} \sum_{i=1}^n (f''(\xi_i) h)$$

note that the sum in the second version is a Riemann sum for approximating the integral

$$I'' := \int_a^b f''(x) dx = [f'(x)]_a^b = f'(b) - f'(a),$$

so it seems that

$$I - T_n \approx -\frac{f'(b) - f'(a)}{12} h^2 = O(h^2)$$

A virtue of this form is that now we have a good chance of evaluating the coefficient of h^2 , so this gives a “practical error formula” when $f'(x)$ is known.

Another useful fact (not proven in these notes) is that the error for the basic Trapezoid rule can be computed with the help of Taylor’s Theorem in a series:

$$T_1 = \int_a^b f(x) dx = B_2 D^2 f(\xi_2) h^3 + B_4 D^4(\xi_4) h^5 + \dots$$

(where $B_2 = 1/12$ as seen above).

Putting the higher power terms into the above argument one can get

$$\begin{aligned} T_n &= \int_a^b f(x) dx + B_2 [Df(b) - Df(a)] h^2 + B_4 [D^3 f(b) - D^3 f(a)] h^4 + \dots + B_{2k} [D^{2k-1} f(b) - D^{2k-1} f(a)] h^{2k} + \dots \\ &= O(h^2) + O(h^4) + \dots + O(h^{2k}) \end{aligned}$$

so that

$$T_n = \int_a^b f(x) dx + \frac{Df(b) - Df(a)}{12} h^2 + O(h^4)$$

The last form is the setup for Richardson extrapolation — and the previous one with a succession of “big-O” terms is the setup for *repeated* Richardson extrapolation, to get a succession of approximations with errors $O(h^2)$, then $O(h^4)$, then $O(h^6)$, and so on: [Definite Integrals, Part 4: Romberg Integration](#).

There are similar formulas for the Composite Midpoint Rule, like

$$I - M_n = \frac{h^2}{24} (b-a) f''(\xi) = \frac{Df(b) - Df(a)}{24} h^2 + O(h^4)$$

but we will see why the Composite Trapezoid Rule is far more useful for Richardson extrapolation.

5.4.3 Appendix: The Composite Left-hand Endpoint Rule, and its Error

The Composite Left-hand Endpoint Rule with n sub-intervals of equal width $h = (b-a)/n$ is

$$L_n = \sum_{i=0}^{n-1} f(x_i) h, \quad = \sum_{i=0}^{n-1} f(a + ih) h$$

To study its errors, start as with the Compound Trapezoid Rule: break the integral up as

$$I = \int_a^b f(x) dx = \sum_{i=1}^n I^{(i)}, \quad \text{where} \quad I^{(i)} = \int_{x_{i-1}}^{x_i} f(x) dx$$

and the approximation as

$$L_n = \sum_{i=1}^n L^{(i)}$$

where each $L^{(i)}$ is the Left-hand Endpoint Rule approximation of $I^{(i)}$:

$$L^{(i)} = f(x_{i-1}) h$$

Then the error in L_n is again the sum of the errors in each piece:

$$\begin{aligned}
 I - L_n &= \sum_{i=1}^n I^{(i)} - \sum_{i=1}^n L^{(i)} \\
 &= \sum_{i=1}^n (I^{(i)} - L^{(i)}) \\
 &= \sum_{i=1}^n \frac{h^2}{2} f'(\xi_i), \quad x_i \in [x_{i-1}, x_i] \\
 &= \frac{h^2}{2} \sum_{i=1}^n f'(\xi_i)
 \end{aligned}$$

The Generalized Mean Value Theorem — now with f' in place of f — allows us to replace the last sum above by $nf'(\xi)$, some $\xi \in [a, b]$, so that as claimed,

$$I - L_n = \frac{h^2}{2} nf'(\xi) = \frac{h}{2} (b-a) f'(\xi) = O(h)$$

Remark 4.4.1

As with the Composite Trapezoid Rule, one can also get

$$L_n = \int_a^b f(x) dx + \frac{f(b) - f(a)}{2} h + O(h^2)$$

5.5 Definite Integrals, Part 3: The (Composite) Simpson's Rule and Richardson Extrapolation

References:

- Sections 5.2.2 and 5.2.3 of Chapter 5 *Numerical Differentiation and Integration* in [Sauer, 2022].
- Sections 4.3 and 4.4 of Chapter 5 *Numerical Differentiation and Integration* in [Burden et al., 2016].

5.5.1 Introduction

The Composite Simpson's Rule can be derived in several ways. The traditional approach is to devise Simpson's Rule by approximating the integrand function with a collocating quadratic (using three equally spaced nodes) and then “compounding”, as seen with the Trapezoid and Midpoint Rules.

We have already seen another approach: using a 2:1 weighted average of the Trapezoid and Midpoint Rules with the goal of cancelling their $O(h^2)$ error terms.

This section will show a third approach, based on Richardson extrapolation: this will set us up for Romberg Integration.

5.5.2 The Basic Simpson's Rule by Richardson Extrapolation

From the section on *The Composite Trapezoid and Midpoint Rules*, we have

$$T_n = \int_a^b f(x) dx + \frac{Df(b) - Df(a)}{12} h^2 + O(h^4), = I + c_2 h^2 + O(h^4)$$

where I is the integral to be approximated (the “Q” in the section on *Richardson Extrapolation*, and $c_2 = (Df(b) - Df(a))/12$.

Thus the “n form” of Richardson Extrapolation with $p = 2$ gives a new approximation that I will call S_{2n} :

$$S_{2n} = \frac{4T_{2n} - T_n}{4 - 1}$$

To start, look at the simplest case of this:

$$S_2 = \frac{4T_2 - T_1}{3}$$

Defining $h = (b - a)/2$, the ingredients are

$$T_1 = \frac{f(a) + f(b)}{2} (b - a) = \frac{f(a) + f(b)}{2} 2h = (f(a) + f(b))h$$

and

$$T_2 = \left[\frac{f(a)}{2} + f(a + h) + \frac{f(b)}{2} \right] h$$

so

$$S_2 = \frac{[2f(a) + 4f(a + h) + 2f(b)] - [f(a) + f(b)]}{3} h, = \frac{f(a) + 4f(a + h) + f(b)}{3} h$$

which is the basic Simpson's Rule. The subscript “2” is because this uses two intervals, with $h = (b - a)/2$

5.5.3 Accuracy and Order of Precision of Simpson's Rule

Rather than derive this the traditional way — by fitting a quadratic to the function values at $x = a$, $a + h$ and b — this can be confirmed “a posteriori” by showing that the degree of precision is at least 2, so that it is exact for all quadratics. And actually we get a bonus, thanks to some symmetry.

For $f(x) = 1$, the exact integral is $I = b - a, = 2h$, and also

$$S_2 = \frac{1 + 4 \times 1 + 1}{3} h, = 2h$$

For $f(x) = x$, the exact integral is $I = \int_a^b x dx = [x^2/2]_a^b = (b^2 - a^2)/2 = (b - a)(b + a)/2 = (a + b)h$

and

$$S_2 = \frac{a + 4(a + b)/2 + b}{3} h = \frac{a + 2(a + b) + b}{3} h = (a + b)h$$

However, it is sufficient to translate the domain to the symmetric interval $[-h, h]$, so redo the $f(x) = x$ case this easier way:

The exact integral is $\int_{-h}^h x dx = 0$ (because the function is odd)

$$S_2 = \frac{-h + 4 \times 0 + h}{3} h = 0$$

For $f(x) = x^2$, again do it just on the symmetric interval $[-h, h]$: the exact integral is $\int_{-h}^h x^2 dx = [x^3/3]_{-h}^h = 2h^3/3$ and

$$S_2 = \frac{(-h)^2 + 4 \times 0^2 + h^2}{3} h = 2h^3/3$$

So the degree of precision is *at least* 2, as expected.

What about cubics? Check with $f(x) = x^3$, again on interval $[-h, h]$.

Almost no calculation is needed: symmetry does it all for us:

- on one hand, the exact integral is zero due to the function being odd on a symmetric interval: $\int_{-h}^h x^3 dx = [x^4/4]_{-h}^h = 0$
- on the other hand,

$$S_2 = \frac{(-h)^3 + 4 \times 0^3 + h^3}{3} h = 0$$

The degree of precision is at least 3.

Our luck ends here, but looking at $f(x) = x^4$ is informative:

For $f(x) = x^4$,

- the exact integral is $\int_{-h}^h x^4 dx = [x^5/5]_{-h}^h = 2h^5/5$
- on the other hand

$$S_2 = \frac{(-h)^4 + 4 \times 0^4 + h^4}{3} h = 2h^5/3$$

So there is a discrepancy of $(4/15)h^5 = O(h^5)$.

This Simpson's Rule has degree of precision 3: it is exact for all cubics, but not for all quartics.

The last result also indicate the order of error:

$$S_2 - I = O(h^5)$$

Just as for the composite Trapezoid and Midpoint Rules, when we combine multiple simple Simpson's Rule approximations with $2n$ intervals each of width $h = (b - a)/(2n)$, the error is roughly multiplied by n , so h^5 goes to $nh^5 = (b - a)h^4$, leading to

$$S_{2n} - I = O(h^4)$$

5.5.4 Appendix: Deriving Simpson's Rule by the Method of Undetermined Coefficients

We wish to determine the most accurate approximation of the form

$$\int_a^b f(x) dx \approx [C_1 f(a) + C_2 f(c) + C_3 f(b)] h$$

where c is the midpoint, $c = (a + b)/2$

This will be done by the first, "hardest" method: inserting Taylor polynomial and error terms, but to make it a bit less hard, we can consider just the symmetric case $a = -h$, $b = h$, $h = (b - a)/2$ by making the change of variables $x \rightarrow x - c$.

As we now know that this will be exact for cubics, use third order Taylor polynomials:

$$f(\pm h) = f(0) \pm f'(0)h + \frac{f''(0)}{2}h^2 \pm \frac{f'''(0)}{6}h^3 + \frac{f^{(4)}(\xi_{\pm})}{24}h^4$$

(Note that the special values $\xi_{\pm}$ are in general different for the “+h” and “-h” cases.)

As usual, gather terms with the same power of h :

$$\begin{aligned} S_2 &= hf(0)(C_1 + C_2 + C_3) \\ &\quad + h^2 f^{(1)}(0)(-C_1 + C_3) \\ &\quad + h^3 f^{(2)}(0)(C_1/2 + C_3/2) \\ &\quad + h^4 f^{(3)}(0)(-C_1/6 + C_3/6) \\ &\quad + h^5 (C_1 f^{(4)}(\xi_-) + C_3 f^{(4)}(\xi_+))/24 \end{aligned}$$

The exact integral can also be computed with Taylor's formula:

$$\begin{aligned} I &= \int_{-h}^h f(x) dx = \int_{-h}^h \left[f(0) + Df(0)x + \frac{D^2 f(0)}{2}x^2 + \frac{D^3 f(0)}{6}x^3 + \frac{D^4 f(0)}{24}x^4 + \frac{D^5 f(\xi_x)}{120}x^5 \right] dx \\ &= 2hf(0) + \frac{D^2 f(0)}{3}h^3 + \frac{D^3 f(0)}{12}h^5 + O(h^6) \end{aligned}$$

(Symmetry causes all the odd power integrals to vanish.)

so the error is

$$\begin{aligned} S_2 - I &= hf(0)(C_1 + C_2 + C_3 - 2) \\ &\quad + h^2 Df(0)(-C_1 + C_3) \\ &\quad + h^3 D^2 f(0)(C_1/2 + C_3/2 - 1/3) \\ &\quad + O(h^5) \end{aligned}$$

The best possibility is setting the coefficients of h , h^2 and h^3 to zero:

$$\begin{aligned} C_1 + C_2 + C_3 &= 2 \\ -C_1 + C_3 &= 0 \\ C_1/2 + C_3/2 &= 1/3 \end{aligned}$$

Symmetry helps, as the “ h^2 ” equation $-C_1 + C_3 = 0$ gives $C_3 = C_1$, leaving

$$C_1 = 1/3, \quad 2C_1 + C_2 = 2$$

and thus

$$C_1 = C_3 = 1/3, \quad C_2 = 4/3$$

as claimed above.

5.6 Definite Integrals, Part 4: Romberg Integration

References:

- Section 5.3 *Romberg Integration* of [Sauer, 2022].
- Section 4.5 *Romberg Integration* of [Burden et al., 2016].

5.6.1 Introduction

Romberg Integration is based on repeated Richardson extrapolation from the composite trapezoidal rule, starting with one interval and repeatedly doubling. Our notation starts with

$$R_{i,0} = T_{2^i}, \quad i = 0, 1, 2, \dots$$

where

$$T_n = \left(\frac{f(a)}{2} + \sum_{k=1}^{n-1} f(a + kh) + \frac{f(b)}{2} \right) h, \quad h = \frac{b-a}{n}$$

and the second index will indicate the number of extrapolation steps done (none so far!)

Actually we only need this T_n formula for the single trapezoidal rule, to get

$$R_{0,0} = T_1 = \frac{f(a) + f(b)}{2} (b-a),$$

because the most efficient way to get the other values is recursively, with

$$T_{2n} = \frac{T_n + M_n}{2}$$

where M_n is the composite midpoint rule,

$$M_n = h \sum_{k=1}^n f(a + (k-1/2)h), \quad h = \frac{b-a}{n}$$

Extrapolation is then done with the formula

$$R_{i,j} = \frac{4^j R_{i,j-1} - R_{i-1,j-1}}{4^j - 1}, \quad j = 1, 2, \dots, i$$

which can also be expressed as

$$R_{i,j} = R_{i,j-1} + E_{i,j-1}, \quad \text{where } E_{i,j-1} = \frac{R_{i,j-1} - R_{i-1,j-1}}{4^j - 1} \quad \text{is an error estimate.}$$

$$R_{i,1} = S_{2n} = \frac{4T_{2n} - T_n}{4 - 1}, \quad n = 2^{i-1}$$

5.6.2 An algorithm, in pseudocode

The above can now be arranged into a basic algorithm. It does a fixed number M of levels of extrapolation so using 2^M intervals; a refinement would be to use the above error estimate $E_{i,j-1}$ as the basis for a stopping condition.

Algorithm 4.6.1 (Romberg Integration)

$n \leftarrow 1$

$h \leftarrow b - a$

$$R_{0,0} = \frac{f(a) + f(b)}{2} h$$

for i from 1 to M :

$$R_{i,0} = (R_{i-1,0} + h \sum_{k=1}^n f(a + (i-1/2)h)) / 2$$

```
for j from 1 to i:  
    
$$R_{i,j} = \frac{4^j R_{i,j-1} - R_{i-1,j-1}}{4^j - 1}$$
  
end for  
 $n \leftarrow 2n$   
 $h \leftarrow h/2$   
end for
```

MINIMIZATION

6.1 Finding the Minimum of a Function of One Variable Without Using Derivatives — A Brief Introduction

References:

- Section 13.1 *Unconstrained Optimization Without Derivatives* of [Sauer, 2022], in particular sub-section 13.1.1 *Golden Section Search*.
- Section 11.1, *One-Variable Case* in Chapter 11 *Optimization* of [Chenney and Kincaid, 2012].

6.1.1 Introduction

The goal of this section is to find the minimum of a function $f(x)$ and more specifically to find its location: the argument p such that $f(p) \leq f(x)$ for all x in the domain of f .

Several features are similar to what we have seen with zero-finding:

- Some restrictions on the function f are needed:
 - with zero-finding, to guarantee *existence* of a solution, we needed at least an interval $[a, b]$ on which the function is continuous and with a sign change between the endpoints;
 - for minimization, the criterion for existence is simply an interval $[a, b]$ on which the function is continuous.
- With zero-finding, we needed to compare the values of the function at three points $a < c < b$ to determine a new, smaller interval containing the root; with minimization, we instead need to compare the values of the function at four points $a < c < d < b$ to determine a new, smaller interval containing the minimum.
- There are often good reasons to be able to do this without using derivatives.

As is often the case, a guarantee of a *unique* solution helps to devise a robust algorithm:

- to guarantee uniqueness of a zero in interval $[a, b]$, we needed an extra condition like the function being *monotonic*;
- to guarantee uniqueness of a minimum in interval $[a, b]$, the condition we use is being *monomodal*: The function is decreasing near a , increasing near b , and changes between decreasing and increasing only once (which must therefore happen at the minimum.)

So we assume from now on that the function is *monomodal* on the interval $[a, b]$.

6.1.2 Step 1: finding a smaller interval within $[a, b]$ that contains the minimum

As claimed above, three points are not enough: even if for $a < c < b$ we have $f(a) > f(c)$ and $f(c) < f(b)$, the minimum could be either to the left or the right of c .

So instead, choose two internal points c and d , $a < c < d < b$.

- if $f(c) < f(d)$, the function is increasing on at least part of the interval $[c, d]$, so the transition from decreasing to increasing is to the left of d : the minimum is in $[a, d]$;
- if instead $f(c) > f(d)$, the “mirror image” argument shows that the minimum is in $[c, b]$.

What about the borderline case when $f(c) = f(d)$? The monomodal function cannot be either increasing or decreasing on all of $[c, d]$ so must first decrease and then increase: the minimum is in $[c, d]$, and so is in either of the above intervals.

So we almost have a first algorithm, except for the issue of *choosing*; given an interval $[a, b]$ on which function f is monomodal:

1. Choose two internal points c and d , with $a < c < d < b$
2. Evaluate $f(c)$ and $f(d)$.
3. If $f(c) < f(d)$, replace the interval $[a, b]$ by $[a, d]$; else replace it by $[c, b]$.
4. If the new interval is short enough to locate the minimum with sufficient accuracy (e.g. its length is less than twice the error tolerance) stop; its midpoint is a sufficiently accurate approximate answer; otherwise, repeat from step (1).

6.1.3 Step 2: choosing the internal points so that the method is guaranteed to converge

There are a couple of details that need to be resolved:

(A) Deciding how to choose the internal points c and d .

(B) Verifying that the interval does indeed shrink to arbitrarily small length after enough iterations, so that the algorithm succeeds.

Once we have done that and got a working algorithm, there will be the issue of speed:

(C) Amongst the many ways that we could choose the internal points, finding one that (typically at least) is fastest, in the sense of minimizing the number of functions evaluations needed.

For now, I will just describe one “naive” approach that works, but is not optimal for speed; **Trisection**:

Take c and d to divide the interval $[a, b]$ into three equal-width sub-intervals: $c = (2a + b)/3$, $d = (a + 2b)/3$, so that each of $[a, c]$, $[c, d]$ and $[d, b]$ are of length $(b - a)/3$.

Then the new interval is $2/3$ as long as the previous one, and the errors shrink by a factor of $(2/3)^k$ after k steps, eventually getting as small as one wishes.

6.1.4 Step 3: choosing the internal points so that the method converges as fast as possible

Coming soon: this leads to the [Golden Section Search](#) ...

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

6.2 Finding the Minimum of a Function of Several Variables — Coming Soon

References:

- Chapter 13 *Optimization* of [Sauer, 2022], in particular sub-sections 13.2.2 *Steepest Descent* and 13.1.3 *Nelder-Mead*.
- Chapter 11 *Optimization* of [Chenney and Kincaid, 2012].

6.2.1 Introduction

This future section will focus on two methods for computing the minimum (and its location) of a function $f(x, y, \dots)$ of several variables:

- *Steepest Descent* where the gradient is used iteratively to find the direction in which to search for a new approximate location where f has a lower value.
- The method of Nelder and Mead, which does not use derivatives.

INITIAL VALUE PROBLEMS FOR ORDINARY DIFFERENTIAL EQUATIONS

7.1 Basic Concepts and Euler's Method

References:

- Sections 6.1.1 *Euler's Method* in [Sauer, 2022].
- Section 5.2 *Euler's Method* in [Burden *et al.*, 2016].
- Sections 7.1 and 7.2 in [Chenney and Kincaid, 2012].

```
import numpy as np
#from matplotlib import pyplot as plt
# Shortcuts for some favorite commands:
from numpy import linspace
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend
```

7.1.1 The Basic ODE Initial Value Problem

We consider the problem of solving (approximately) the ordinary differential equation

$$\frac{du}{dt} = f(t, u(t)), a \leq t \leq b$$

with the *initial condition*

$$u(a) = u_0$$

I will follow the common custom of referring to the independent variable as “time”.

For now, $u(t)$ is real-valued, but little will change when we later let it be vector-valued (and/or complex-valued).

Notation for the solution of an initial value problem

Sometimes, we need to be more careful and explicit in describing the function that solves the above initial value problem; then the input *parameters* a and $u_0 = u(a)$ will be included of the function's formula:

$$u(t) = u(t; a, u_0)$$

(It is standard mathematical convention to separate *parameters* like a and u_0 from *variables* like t by putting the former after a semicolon.

7.1.2 Examples

A lot of useful intuition comes from these four fairly simple examples.

Example (Integration)

If the derivative depends only on the independent variable t , so that

$$\frac{du}{dt} = f(t), a \leq t \leq b$$

the solution is given by integration:

$$u(t) = u_0 + \int_a^t f(s) ds.$$

In particular, with $u_0 = 0$ the value at b is

$$u(t) = \int_a^b f(t) dt,$$

and this gives us a back-door way to use numerical methods for solving ODEs to evaluate definite integrals.

Example (Integration)

The simplest case with u present in f is $f(t, u) = f(u) = u$. But it does not hurt to add a constant, so:

$$\frac{du}{dt} = ku, \quad k \text{ a constant.}$$

The solution is

$$u(t) = u_0 e^{k(t-a)}$$

We will see that this simple example contains the essence of ideas relevant far more generally.

Example (A nonlinear equation, with solutions existing only for a finite time)

In the previous examples, $f(t, u)$ is linear in u (consider t as fixed); nonlinearities can lead to more difficult behavior. The equation

$$\frac{du}{dt} = u^2, \quad u(a) = u_0$$

can be solved by separation of variables — or for now you can just verify the solution

$$u(t) = \frac{1}{T-t}, \quad T = a + 1/u_0.$$

Note that if $u_0 > 0$, the only exists for $t < T$. (The solution is also valid for $T > 0$, but that part has no connection to the initial data at $t = a$.)

This example warns us that the IVP might not be **well-posed** when we set the interval $[a, b]$ in advance: all we can guarantee in general is that a solution exists up to some time $b, b > a$.

Example (A “stiff” equation with disparate time scales)

One common problem in practical situations is differential equations where some phenomena happen on a very fast time scale, but only ever at very small amplitudes, so they have very little relevance to the overall solution. One example is descriptions of some chemical reactions, where some reaction products (like free radicals) are produced in tiny quantities and break down very rapidly, so they change on a very fast time scale but are scarcely relevant to the overall solution.

This disparity of time-scales is called *stiffness*, from the analogy of a mechanical system in which some components are very stiff and so vibrate at very high frequencies, but typically only at very small amplitudes, or very quickly damped away, so that they can often be safely described by assuming that those stiff parts are completely rigid — do not move at all.

One equation that illustrates this feature is

$$\frac{du}{dt} = -\sin t - k(u - \cos t)$$

where k is large and positive. Its family of solutions is

$$u(t) = \cos t + ce^{-k(t-a)}$$

with $c = u_0 - \cos(a)$ for the initial value problem $u(a) = u_0$.

These all get close to $\cos t$ quickly and then stay nearby, but with a rapid and rapidly decaying “transient” ce^{-kt} .

Many of the most basic and widely use numerical methods (including Euler’s Method that we meet soon) need to use very small time steps to handle that fast transient, even when it is very small because $u_0 \approx 1$.

On the other hand there are methods that “supress” these transients, allowing use of larger time steps while still getting an accurate description of the main, slower, phenomena. The simplest of these is the *Backward Euler Method* that we will see in a later section.

7.1.3 The Tangent Line Method, a.k.a. Euler’s Method

Once we know $u(t)$ (or a good approximation) at some time t , we also know the value of $u'(t) = f(t, u(t))$ there; in particular, we know that $u(a) = u_0$ and so $u'(a) = f(a, u_0)$.

This allows us to approximate u for slightly larger values of the argument (which I will call “time”) using its tangent line:

$$u(a+h) \approx u(a) + u'(a)h = u_0 + f(a, u_0)h \text{ for "small" } h$$

and more generally

$$u(t+h) \approx u(t) + f(t, u(t))h \text{ for "small" } h$$

This leads to the simplest approximation: choose a step size h determining equally spaced times $t_i = a + ih$ and define — recursively — a sequence of approximations $U_i \approx u(t_i)$ with

$$U_0 = u_0$$

$$U_{i+1} = U_i + hf(t_i, U_i)$$

If we choose a number of time steps n and set $h = (b-a)/n$ for $0 \leq i \leq n$, the second equation is needed for $0 \leq i < n$, ending with $U_n \approx u(t_n) = u(b)$.

This “two-liner” does not need a pseudo-code description; instead, we can go directly to a rudimentary Python function for Euler’s Method:

```
def eulerMethod(f, a, b, u_0, n):
    """Solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0"""
    h = (b-a)/n
    t = linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for t.
    u = np.empty_like(t)
    u[0] = u_0
    for i in range(n):
        u[i+1] = u[i] + f(t[i], u[i])*h
    return (t, u)
```

Exercise 1

Show that for the integration case $f(t, u) = f(t)$, Euler’s method is the same as the composite left-hand endpoint rule, as in the section *Definite Integrals, Part 2*.

Solving for Example 6.1.1, an integration

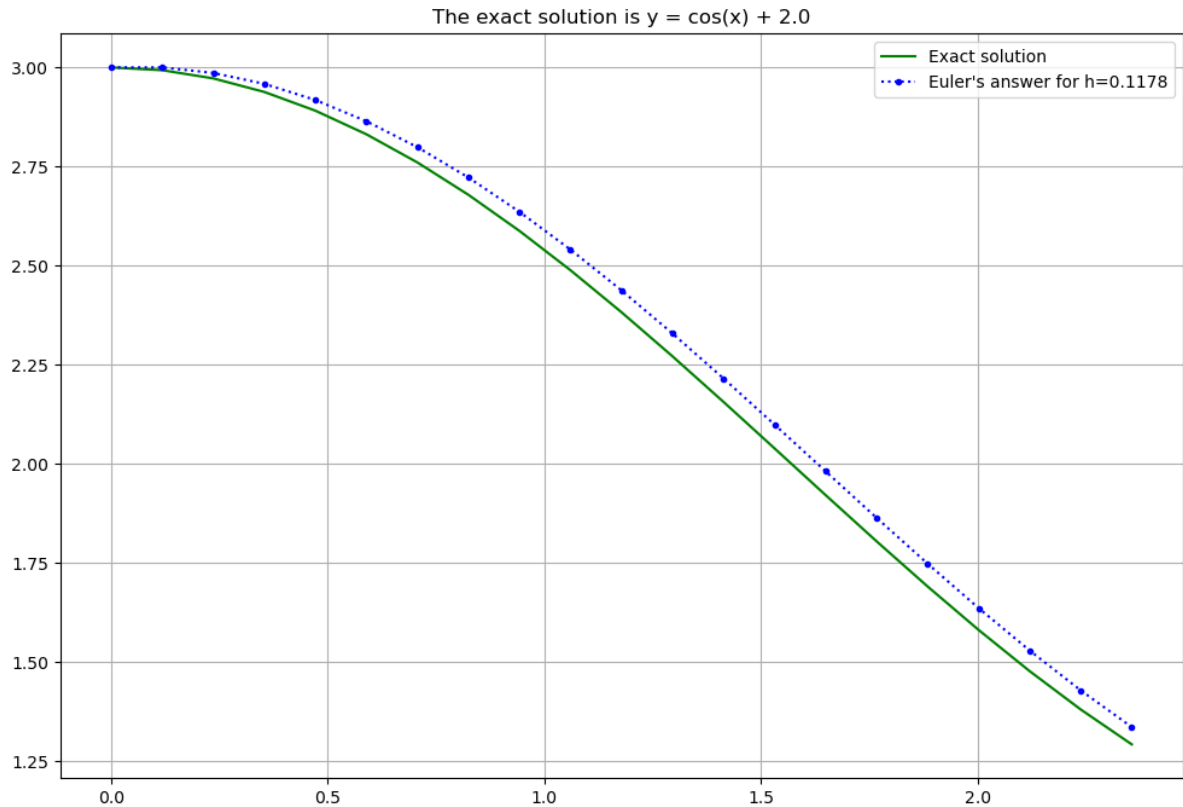
```
def f1(t, u):
    """For integration of -sin(x)
    The general solution is
    u(t) = cos(x) + C,
    C = u_0 - cos(a)
    """
    return -np.sin(t)

def u1(t, a, u_0):
    return np.cos(t) + (u_0 - np.cos(a));
```

```
a = 0.
b = 3/4*np.pi
u_0 = 3.
n = 20

(t, U) = eulerMethod(f1, a, b, u_0, n)
u = u1(t, a, u_0)

figure(figsize=[12,8])
title(f"The exact solution is y = cos(x) + {u_0 - np.cos(a)}")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b", label=f"Euler's answer for h={ (b-a)/n:0.4g}")
legend()
grid(True);
```

Solving for Example 6.1.2, some exponential functions

```
def f2(t, u):
    """For solving  $du/dt = k u$ .
    The variable  $k$  may be defined later, so long as that is done before this function
    is used.
    """
    return k*u

def u2(t, a, u_0, k):
    return u_0 * np.exp(k*(t-a));
```

```
# You could experiment by changing these values here;
# for now I instead redefine them below.
k = 1.
u_0 = 0.8
a = 0.
b = 2.
n = 40

(t, U) = eulerMethod(f2, a, b, u_0, n)
u = u2(t, a, u_0, k)

figure(figsize=[12,8])
title(f"The exact solution is  $u = \{u_0\} \cdot \exp(\{k\} \cdot t)$ ")
plot(t, u, "g", label="Exact solution")
```

(continues on next page)

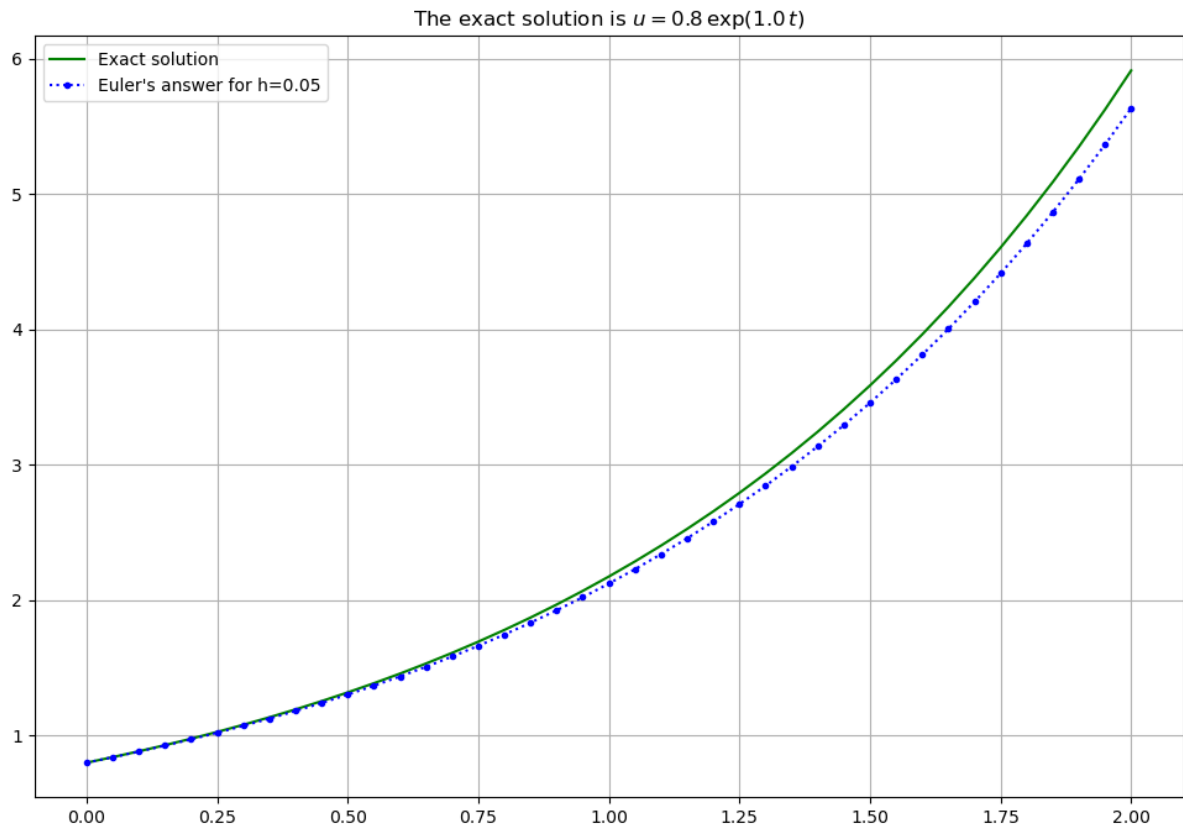
(continued from previous page)

```

plot(t, U, ".*b", label=f"Euler's answer for h={(b-a)/n:0.4g}")

legend()
grid(True);

```



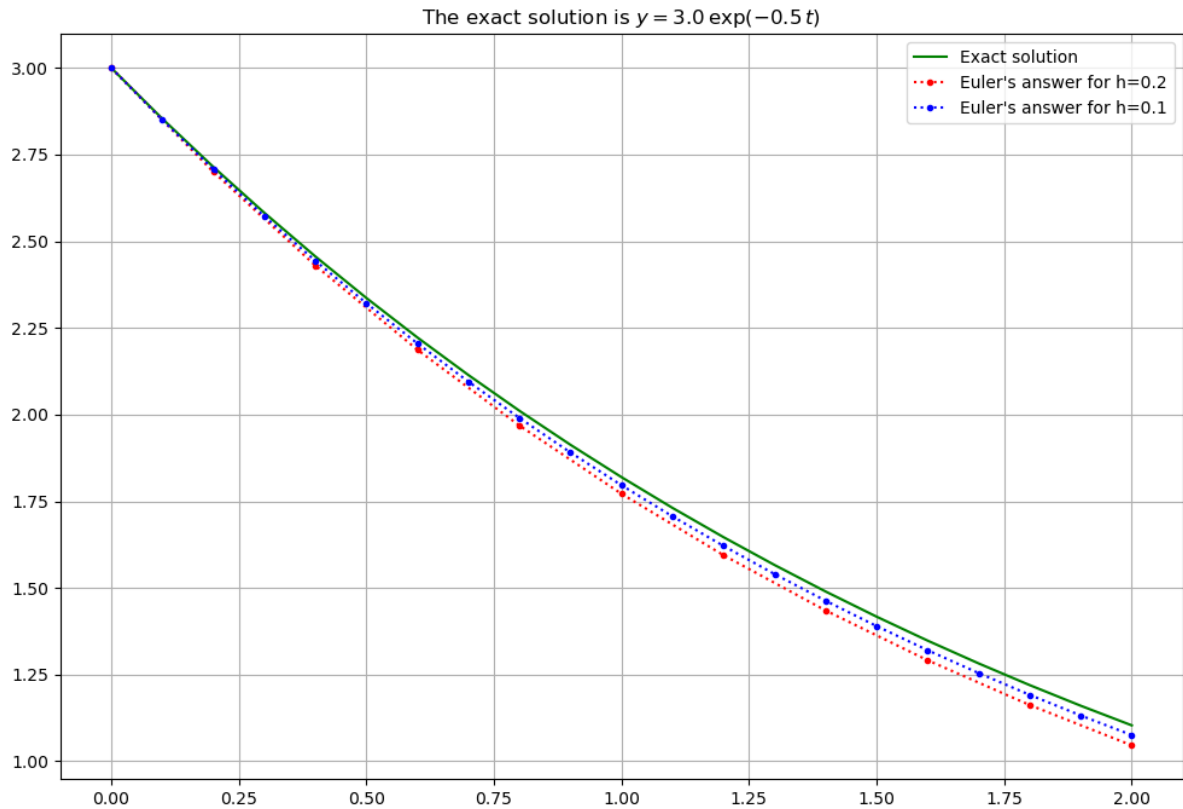
```

# You could experiment by changing these values here.
k = -0.5
u_0 = 3.
a = 0.
b = 2.

(t10, U10) = eulerMethod(f2, a, b, u_0, 10)
(t20, U20) = eulerMethod(f2, a, b, u_0, 20)
t = t20
u = u2(t, a, u_0, k)

figure(figsize=[12,8])
title(f"The exact solution is $y = \{u_0\} \backslash, \exp(\{k\} \backslash, t)$")
plot(t, u, "g", label="Exact solution")
plot(t10, U10, ".*r", label=f"Euler's answer for h={(b-a)/10:0.4g}")
plot(t20, U20, ".*b", label=f"Euler's answer for h={(b-a)/20:0.4g}")
legend()
grid(True);

```



Solving for Example 6.1.3: solutions that blow up

```
def f3(t, u):
    """For solving du/dt = u^2
    # The general solution is u(t) = 1/((a + 1/u_0) - t), = 1/(T-t) with T = a + 1/u_0
    """
    return u**2

def u3(t, a, u_0):
    T = a + 1./u_0
    return 1./(T - t);
```

```
a = 0.
b = 0.9
u_0 = 1.

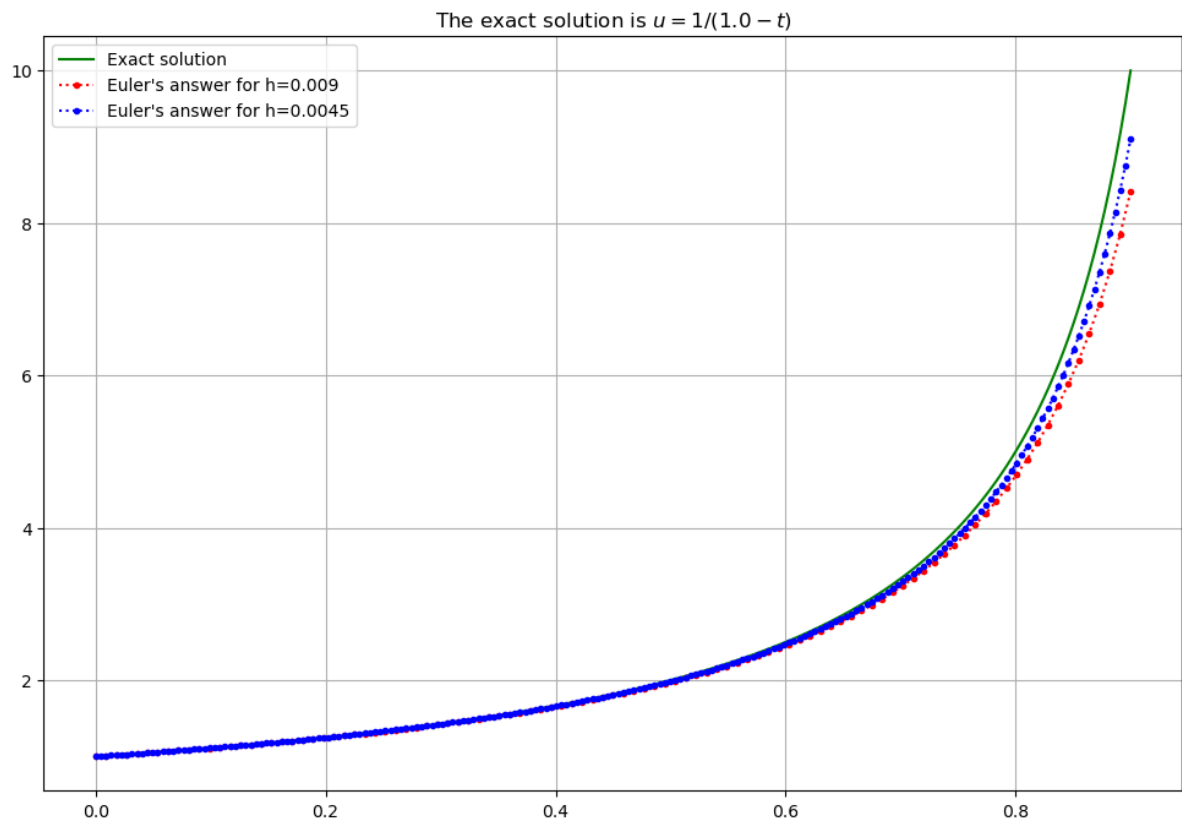
(t100, U100) = eulerMethod(f3, a, b, u_0, 100)
(t200, U200) = eulerMethod(f3, a, b, u_0, 200)
t = t200
u = u3(t, a, u_0)

figure(figsize=[12,8])
title(f"The exact solution is $u = 1/({a} + 1/{u_0}) - t)$")
plot(t, u, "g", label=f"Exact solution")
plot(t100, U100, "r", label=f"Euler's answer for h={b-a}/100:0.2g}")
plot(t200, U200, "b", label=f"Euler's answer for h={b-a}/200:0.2g}")
```

(continues on next page)

(continued from previous page)

```
legend()
grid(True);
```

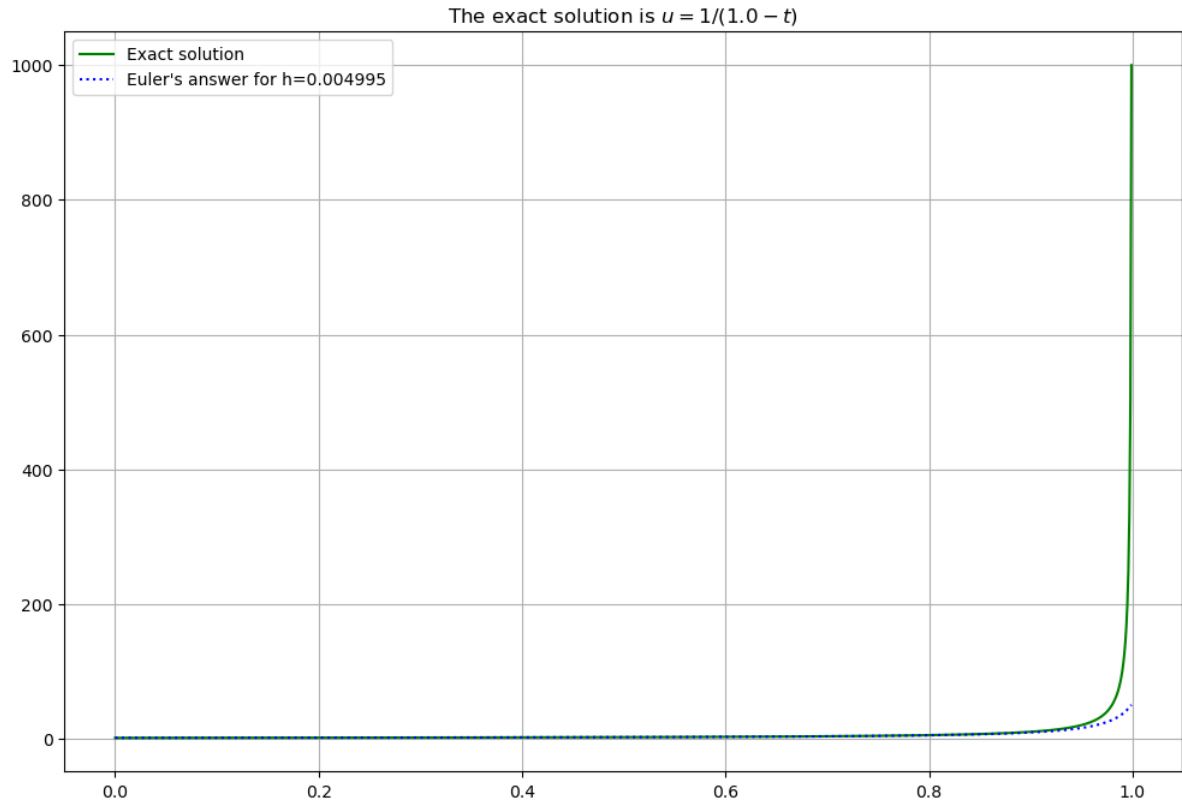


There is clearly a problem when t reaches 1; let us explore that:

```
a = 0.
b = 0.999
u_0 = 1.
n = 200

(t, U) = eulerMethod(f3, a, b, u_0, n)
T = a + 1/u_0
tplot = linspace(a, b, 1000) # More t values are needed to get a good graph near the
    ↪ vertical asymptote
u = u3(tplot, a, u_0)

figure(figsize=[12,8])
title(f"The exact solution is $u = 1/({T} - t)$")
plot(tplot, u, "g", label=f"Exact solution")
plot(t, U, ":b", label=f"Euler's answer for h={ (b-a)/n:0.4g}")
legend()
grid(True);
```

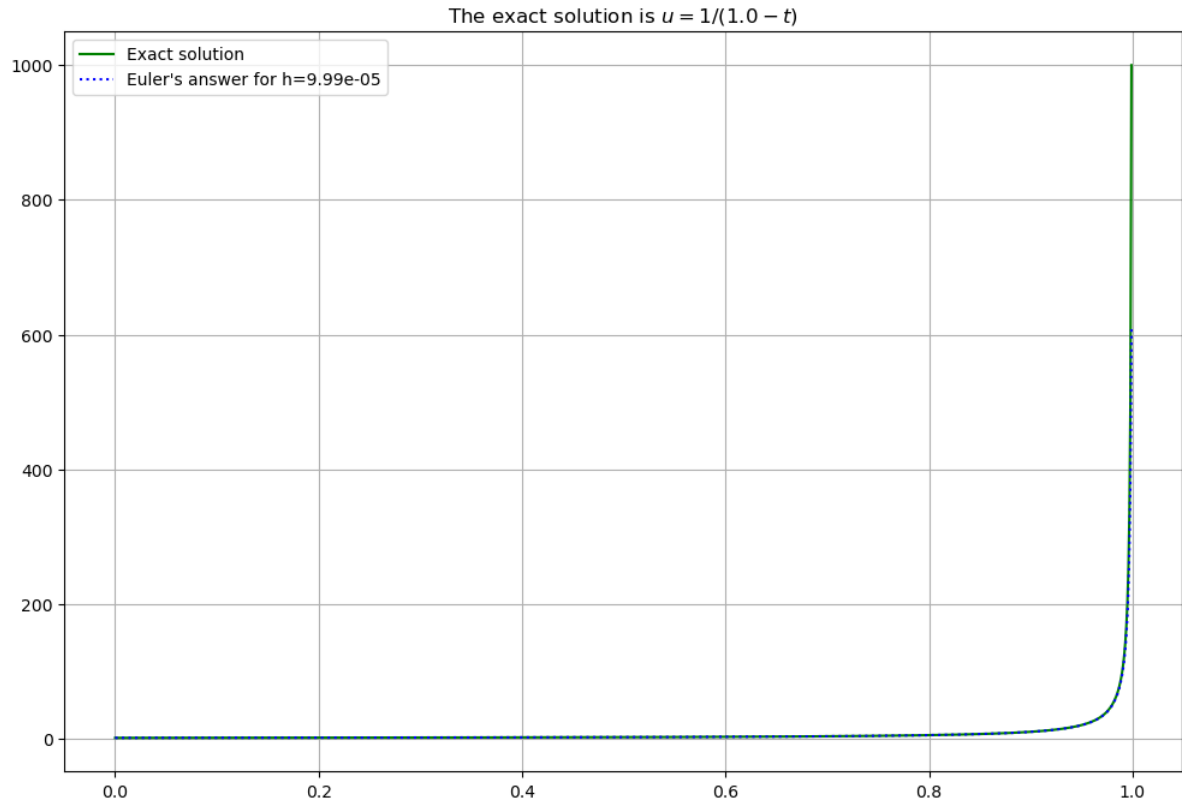


Clearly Euler's method can never produce the vertical asymptote. The best we can do is improve accuracy by using more, smaller time steps:

```
n = 10000

(t, U) = eulerMethod(f3, a, b, u_0, n)
T = a + 1/u_0
tplot = linspace(a, b, 1000) # More t values are needed to get a good graph near the
    ↪ vertical asymptote
u = u3(tplot, a, u_0)

figure(figsize=[12,8])
title(f"The exact solution is $u = 1/({T} - t)$")
plot(tplot, u, "g", label="Exact solution")
plot(t, U, "b", label=f"Euler's answer for h={b-a}/n:0.4}")
legend()
grid(True);
```



Solving for Solving for Example 6.1.4, a stiff ODE

```
def f4(t, u):
    """The variable k may be defined later, so long as that is done before this_
    ↪function is used.
    The general solution is  $u(t) = u(t; a, u_0, k) = \cos t + (u_0 - \cos(a)) e^{-K(t-a)}$ 
    ↪a) }
    """
    return -np.sin(t) - k*(u - np.cos(t))

def u4(t, a, u_0, k):
    return np.cos(t) + (u_0 - np.cos(a)) * np.exp(k*(a-t))
```

With enough steps (small enough step size h), all is well:

```
a = 0.
b = 2 * np.pi # One period
u_0 = 2.
k = 40.
n = 400

(t, U) = eulerMethod(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)

figure(figsize=[12,8])
title(f"The exact solution is  $u = \cos t + \{u_0-1:0.4g\} \exp(-\{k\} t)$ ")
plot(t, u, "g", label=f"Exact solution for  $\{k=\}$ ")
```

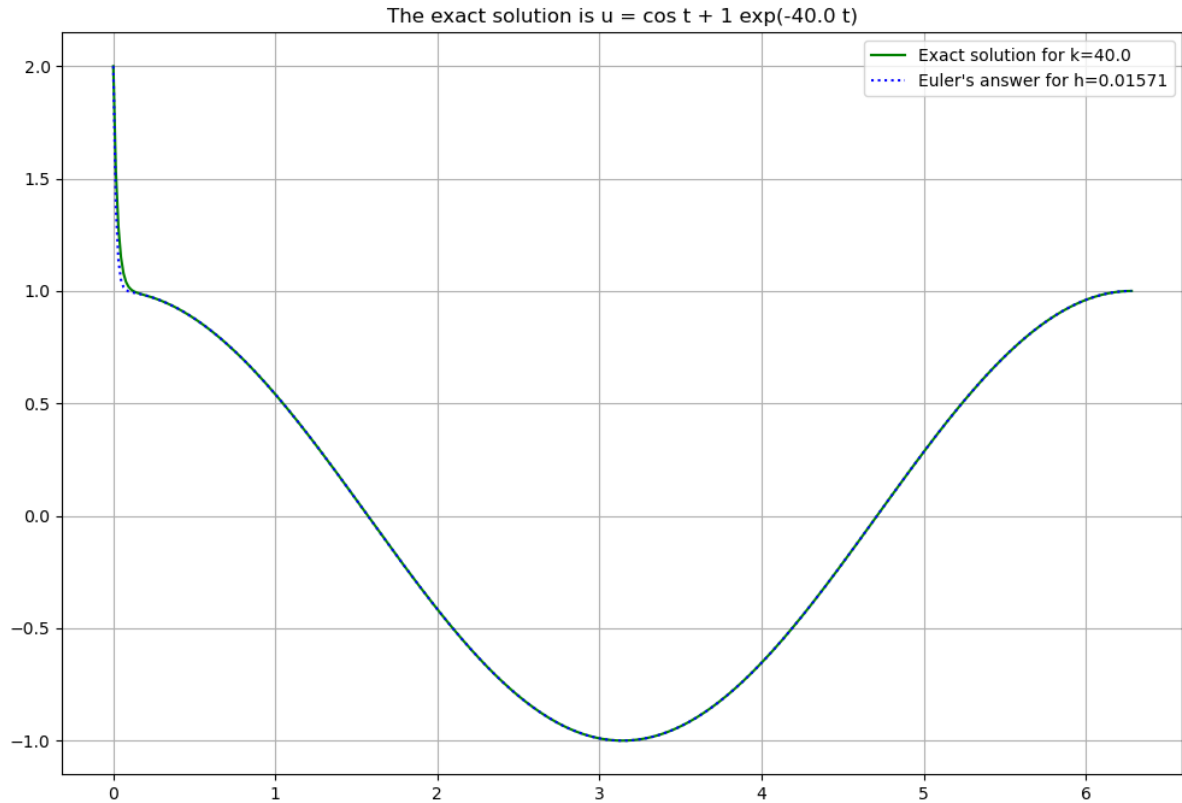
(continues on next page)

(continued from previous page)

```

plot(t, U, "b", label=f"Euler's answer for h={ (b-a)/n:0.4g}")
legend()
grid(True);

```



However, with large steps (still small enough to handle the $\cos t$ part), there is a catastrophic failure, with growing oscillations that, as we will see, are a characteristic feature of *instability*.

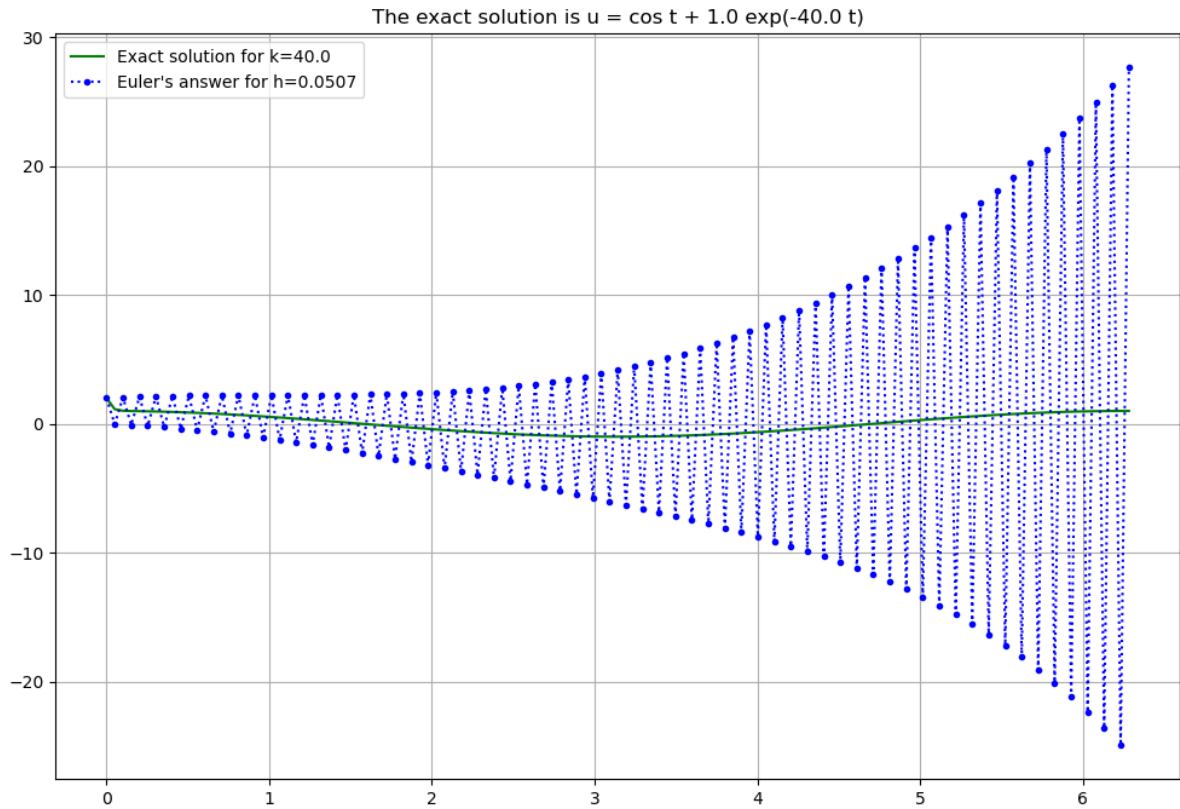
```

n = 124

(t, U) = eulerMethod(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)

figure(figsize=[12,8])
title(f"The exact solution is  $u = \cos t + \{u_0-1:0.3\} \exp(-\{k\} t)$ ")
plot(t, u, "g", label=f"Exact solution for {k=}")
plot(t, U, "b", label=f"Euler's answer for h={ (b-a)/n:0.3}")
legend()
grid(True);

```

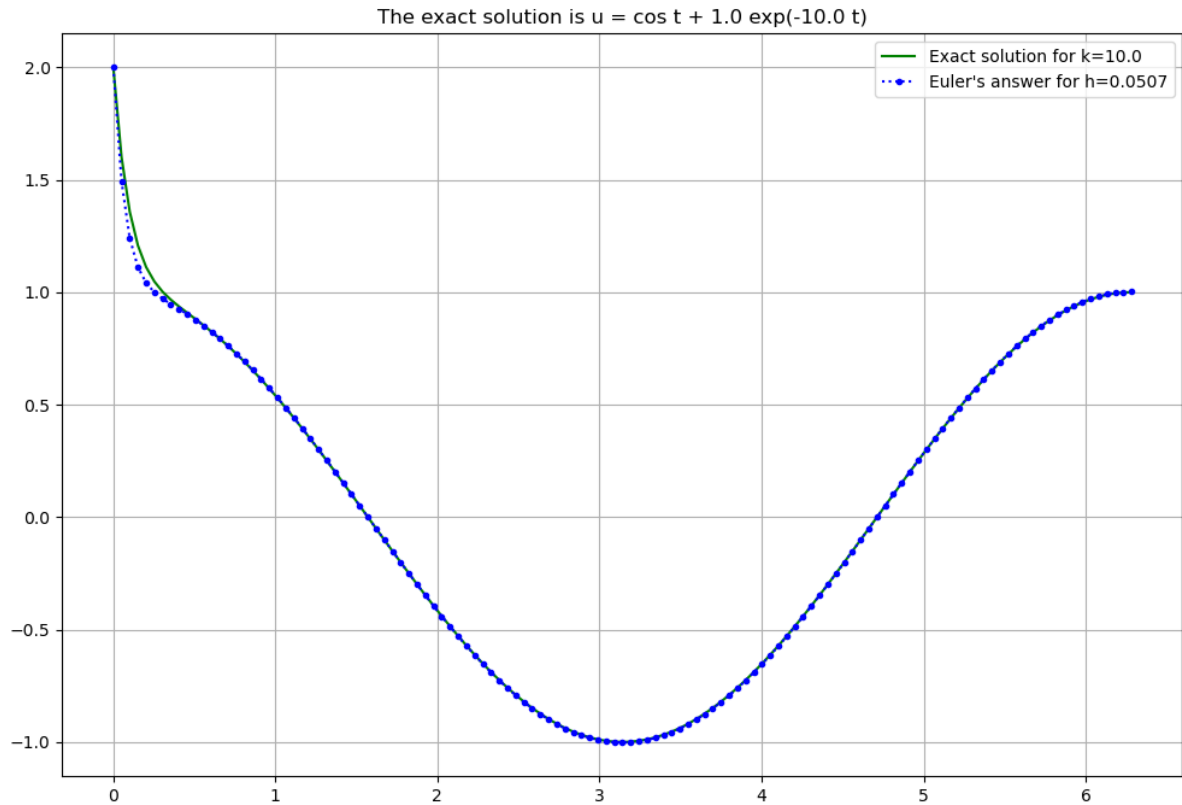


To show that the k part is the problem, reduce k while leaving the rest unchanged:

```
k = 10.

(t, U) = eulerMethod(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)

figure(figsize=[12,8])
title(f"The exact solution is  $u = \cos t + \{u_0-1:0.3\} \exp(-\{k\} t)$ ")
plot(t, u, "g", label=f"Exact solution for  $\{k=\}$ ")
plot(t, U, '.:b', label=f"Euler's answer for  $h=\{(b-a)/n:0.3\}$ ")
legend()
grid(True);
```

Variable Time Step Sizes h_i (just a preview for now)

It is sometime useful to adjust the time step size; for example reducing it when the derivative is larger, (as happens in Example 3 above). This gives a slight variant, now expressed in pseudo-code:

Input: f, a, b, n

$t_0 = a$

$U_0 = u_0$

for i in $[0, n)$:

 Choose h_i somehow

$t_{i+1} = t_i + h_i$

$U_{i+1} = U_i + h_i f(t_i, U_i)$

end for

In a later section, we will see how to estimate errors within an algorithm, and then how to use such error estimates to guide the choice of step size.

Error Analysis for the Canonical Test Case, $u' = ku$.

A great amount of intuition about numerical methods for solving ODE IVPs comes from that “simplest nontrivial example”, number 2 above. We can solve it with constant step size h , and thus study its errors and accuracy. The recursion relation is now

$$U_{i+1} = U_i + hkU_i = U_i(1 + hk),$$

with solution

$$U_i = u_0(1 + hk)^i.$$

For comparison, the exact solution of this ODE IVP is

$$u(t_i) = u_0 e^{k(t_i - a)} = u_0 e^{k i h} = u_0 (e^{kh})^i$$

So each is a geometric series: the difference is that the *growth factor* is $G = (1 + hk)$ for Euler’s method, vs $g = e^{kh} = 1 + hk + (hk)^2/2 + \dots = 1 + hk + O(h^2)$ for the ODE.

This deviation at each time step is $O(h^2)$, suggesting that by the end $t = b$, at step n , the error will be $O(nh^2) = O\left(\frac{b-a}{h}h^2\right) = O(h)$.

This is in fact what happens, but to verify that, we must deal with the challenge that once an error enters at one step, it is potentially amplified at each subsequent step, so the errors introduced at each step do not simply get summed like they did with definite integrals.

Global Error and Local (Truncation) Error

Ultimately, the error we need to understand is the **global error**: at step i ,

$$E_i = u(t_i) - U_i$$

We will approach this by first considering the new error added at each step, the **local truncation error** (or **discretization error**).

At the first step this is the same as above:

$$e_1 = u(t_1) - U_1 = u(a + h) - U_1$$

However at later steps we compare the results U_{i+1} to what the solution would be if it were exact at the start of that step: that is, if U_i were exact.

Using the notation $u(t; t_i, U_i)$ introduced above for the solution of the ODE with initial condition $u(t_i) = U_i$, the **location truncation error** at step i is the discrepancy at time t_{i+1} between what Euler’s method and the exact solution give when both start at that point (t_i, U_i) :

$$e_i = u(t; t_i, U_i) - U_{i+1}$$

Error propagation in $u' = ku$, $k \geq 0$.

After one step, $E_1 = u(t_1) - U_1 = e_1$.

At step 2,

$$E_2 = u(t_2) - U_2 = (u(t_2) - u(t_2, t_1, U_1)) + (u(t_2, t_1, U_1) - U_2) = (u(t_2) - u(t_2, t_1, U_1)) + e_2$$

The first term is the difference at $t = t_2$ of two solutions with values at $t = t_1$ being $u(t_1)$ and U_1 respectively. As the ODE is linear and homogeneous, this is the solution of the same ODE with value at $t = t_1$ being $u(t_1) - U_1$, which is e_1 : that solution is $e_1 e^{y(t-t_1)}$, so at $t = t_2$ it is $e_1 e^{kh}$. Thus the global error after two steps is

$$E_2 = e_2 + (e^{kh})e_1 :$$

the error from the previous step has been amplified by the growth factor $g = e^{kh}$:

$$E_2 = e_2 + ge_1 :$$

This continues, so that

$$E_3 = e_3 + gE_1 = e_3 + g(e_2 + ge_1) = e_3 + ge_2 + g^2e_1$$

and so on, leading to

$$E_i = e_i + ge_{i-1} + g^2e_{i-2} + \dots + g^{i-1}e_1.$$

Bounding the local truncation errors ...

To get a bound on the global error from the formula above, we first need a bound on the local truncation errors e_i .

Taylor's theorem gives $e^{kh} = 1 + kh + e^{k\xi}(kh)^2/2$, $0 < \xi < kh$, so

$$e_i = U_i e^{kh} - U_i(1 + kh) = U_i(e^{k\xi}h^2/2)$$

and thus

$$|e_i| \leq |U_i| \frac{e^{kh}}{2} h^2$$

Also, since $1 + kh < e^{kh}$, $|U_i| < |u(t_i)| = |u_0|e^{k(t_i-a)}$, and we only need this to the beginning of the last step, $i \leq n-1$, for which

$$|U_i| < |u_0|e^{k(b-h-a)}$$

Thus

$$|e_i| \leq \frac{|u_0|e^{k(b-h-a)}e^{kh}}{2} h^2 = \frac{|u_0|e^{k(b-a)}}{2} h^2$$

That is,

$$|e_i| \leq Ch^2 \text{ where } C := \frac{|u_0|e^{k(b-a)}}{2}$$

... and using this to complete the bound on the global truncation error

Using this bound on the local errors e_i in the above sum for the global error E_i ,

$$|E_i| \leq Ch^2(1 + g + \dots + g^{i-1}) = C \frac{g^i - 1}{g - 1} h^2$$

Since $g^i = e^{khi} = e^{k(t_i-a)}$ and the denominator $g - 1 = e^{kh} - 1 > kh$, we get

$$|E_i| \leq C \frac{e^{k(t_i-a)} - 1}{kh} h^2 \leq \frac{|u_0|e^{k(b-a)}}{2} \frac{e^{k(t_i-a)} - 1}{k} h, = O(h)$$

Note that this global error formula is built from three factors:

- The first is the constant $\frac{|u_0 e^{k(b-a)}|}{2}$ which is roughly half of the maximum value of the exact solution over the interval $[a, b]$.
- The second $\frac{e^{k(t_i-a)} - 1}{k}$ depends on t , and
- The third is h , showing the overall order of accuracy: first order: the overall absolute error is $O(h)$.

A more general error bound

A very similar result applies to the solution $u(t; a, u_0)$ of the more general initial value problem

$$\frac{du}{dt} = f(t, u), \quad u(a) = u_0$$

so long as the function f is “somewhat well-behaved” in that it satisfies a so-called *Lipschitz Condition*: that there is some constant K such that

$$\left| \frac{\partial f}{\partial u}(t, u) \right| \leq K$$

for the relevant time values $a \leq t \leq b$.

(**Aside:** As you might have seen in a course on differential equations, such a Lipschitz condition is necessary to even guarantee that the initial value problem has a unique solution, so it is a quite reasonable requirement.)

Then this constant K plays the part of the exponential growth factor k above:

first one shows that the local truncation error is bounded by

$$|e_i| \leq Ch^2 \text{ where now } C := \frac{|u_0 e^{K(b-a)}|}{2};$$

then calculating as above bounds the global truncation error with

$$|E_i| \leq \frac{|u_0 e^{K(b-a)}|}{2} \frac{e^{K(t_i-a)} - 1}{k} h, = O(h)$$

There is much room for improvement

As with definite integrals, this is not very impressive, so in the next section on *Runge-Kutta Methods* we will explore several widely used methods that improve to second order and then fourth order accuracy. Later, we will see how to get even higher orders.

But first, we can illustrate how this exponential growth of errors looks in some examples, and compare the better behaved errors in definite integrals.

This will be done by looking at the effect of a small change in the initial value, to simulate an error that arises there.

Error propagation for Example 6.1.1

```
a = 0.
b = 2*np.pi
u_0 = 1. # Original value
n = 100

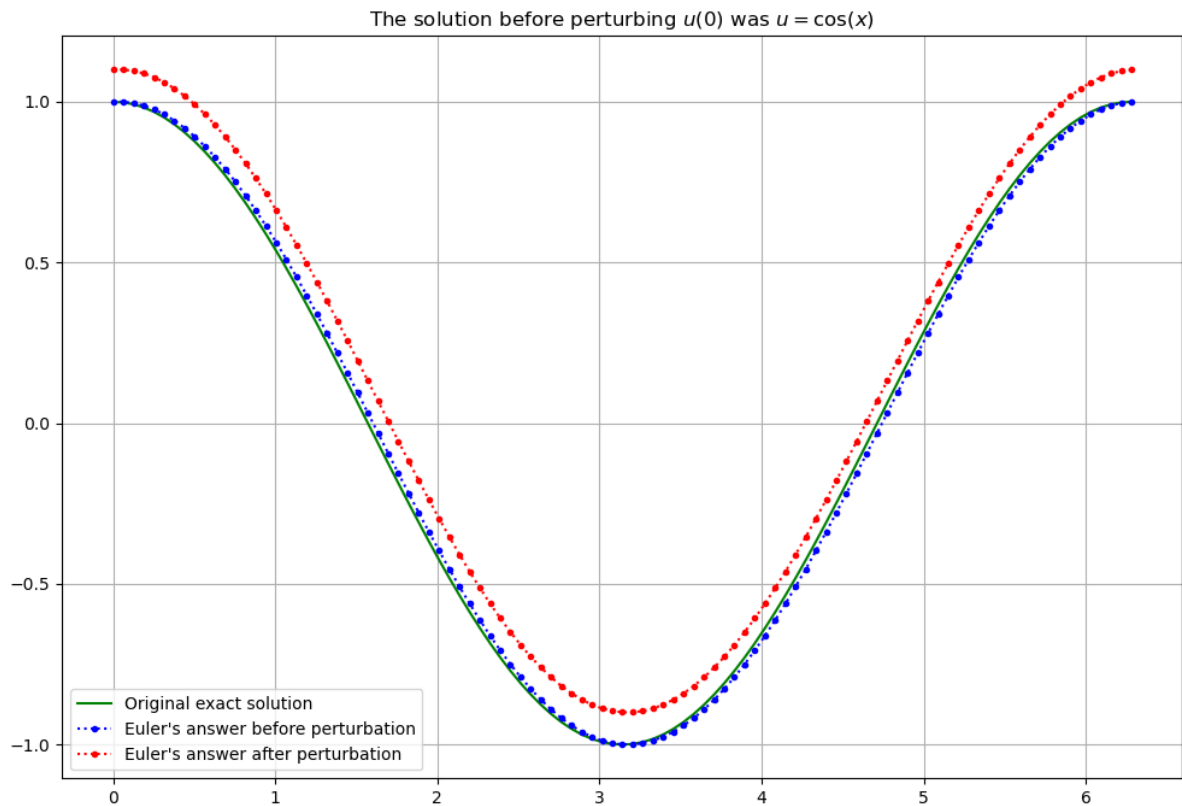
(t, U) = eulerMethod(f1, a, b, u_0, n)
u = u1(t, a, u_0)
```

But now “perturb” the initial value in all cases by this much:

```
delta_u_0 = 0.1

(x, U_perturbed) = eulerMethod(f1, a, b, u_0+delta_u_0, n)
```

```
figure(figsize=[12,8])
title("The solution before perturbing  $u(0)$  was  $u = \cos(x)$ ")
plot(t, u, "g", label="Original exact solution")
plot(t, U, "b:", label="Euler's answer before perturbation")
plot(t, U_perturbed, "r:", label="Euler's answer after perturbation")
legend()
grid(True);
```



This just shifts all the u values up by the perturbation of u_0 .

Error propagation for Example 6.1.2

```
k = 1.
a = 0.
b = 2.
u_0 = 1. # Original value
delta_u_0 = 0.1
n = 100

(t, U) = eulerMethod(f2, a, b, u_0, n)
```

(continues on next page)

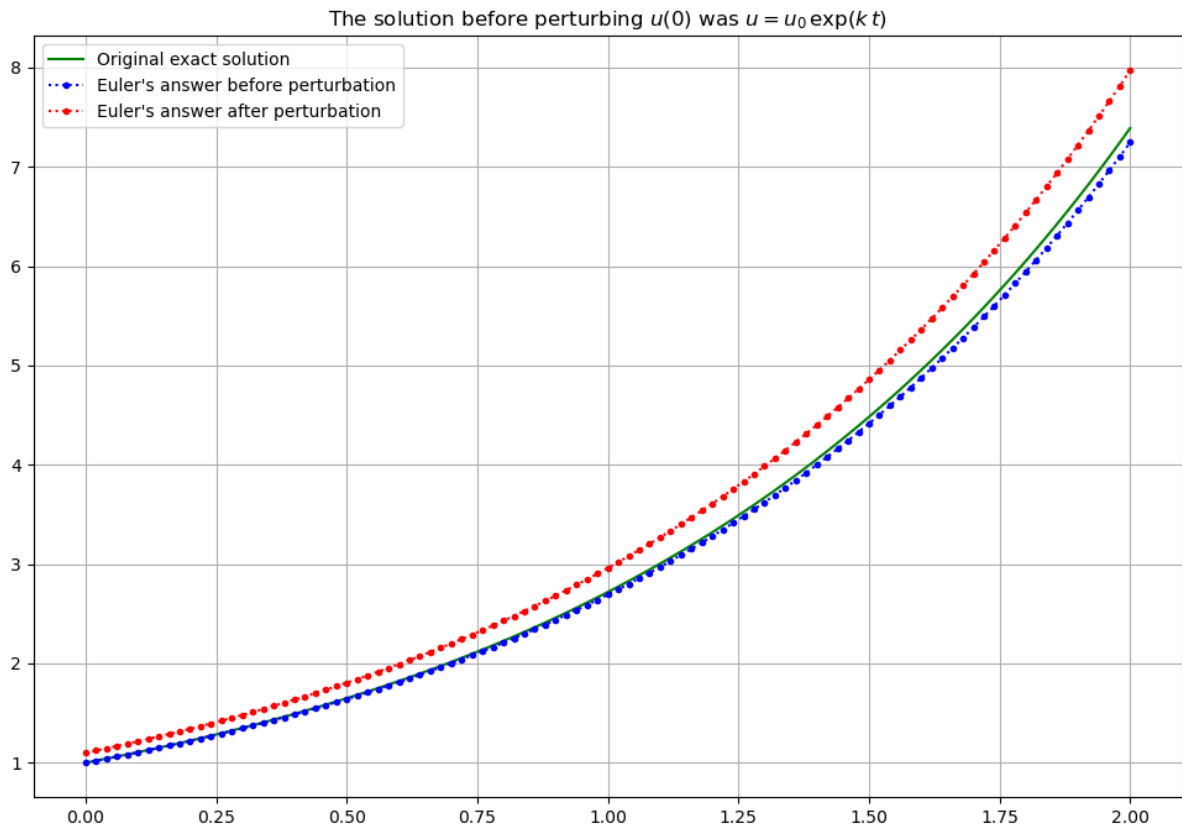
(continued from previous page)

```

(t, U_perturbed) = eulerMethod(f2, a, b, u_0+delta_u_0, n)
u = u2(t, a, u_0, k)

figure(figsize=[12,8])
title("The solution before perturbing $u(0)$ was $u = \{u_0\} \backslash, \backslash \exp(\{k\} \backslash, t)$")
plot(t, u, "g", label="Original exact solution")
plot(t, U, "b:", label="Euler's answer before perturbation")
plot(t, U_perturbed, "r:", label="Euler's answer after perturbation")
legend()
grid(True);

```

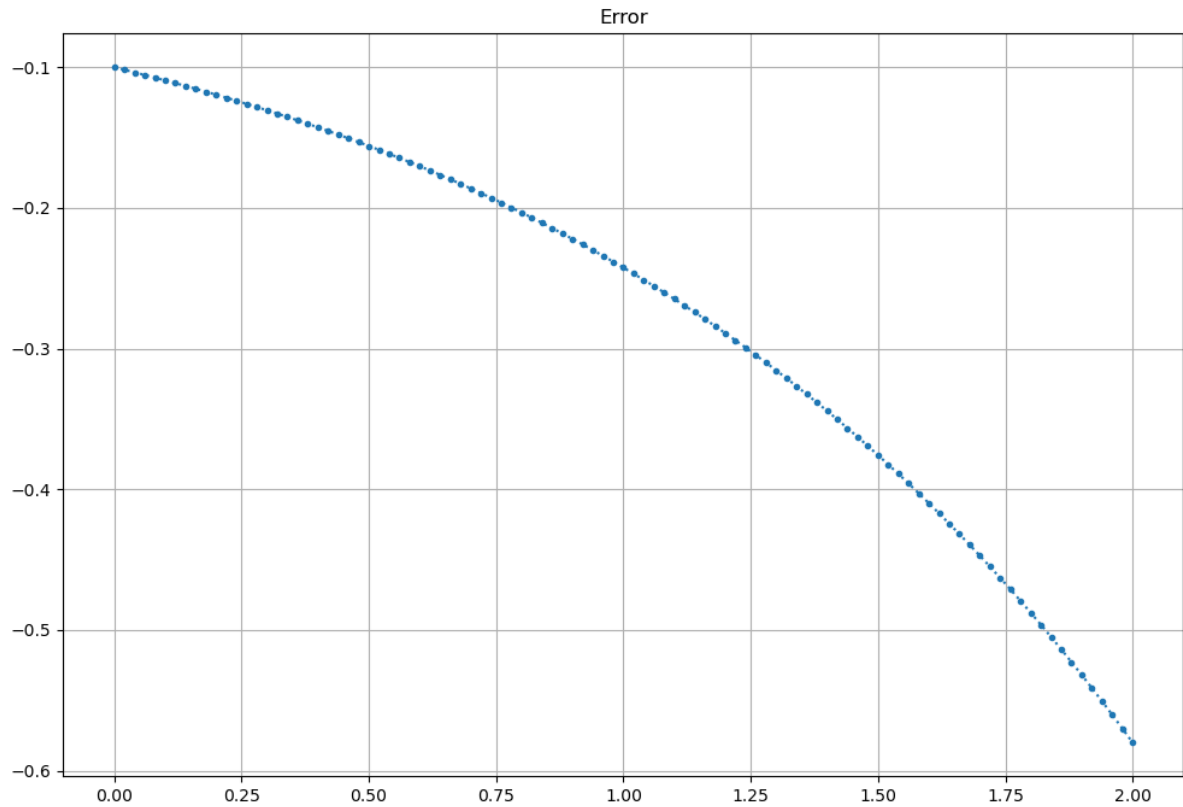


Graphing the error shows its exponential growth:

```

figure(figsize=[12,8])
title("Error")
plot(t, u - U_perturbed, 'r:')
U_perturbed
grid(True);

```



This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

7.2 Runge-Kutta Methods

Remark 6.2.1 (TO DO)

Improve the presentation of examples

References:

- Sections 6.4 *Runge-Kutta Methods and Applications* in [Sauer, 2022].
- Section 5.4 *Runge-Kutta Methods* in [Burden *et al.*, 2016].
- Sections 7.1 and 7.2 in [Chenney and Kincaid, 2012].

```
import numpy as np
# Shortcuts for some favorite commands:
from numpy import linspace
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend
```

7.2.1 Introduction

The original Runge-Kutta method is the fourth order accurate one to be described below, which is still used a lot, though with some modifications.

However, the name is now applied to a variety of methods based on a similar strategy, so first, here are a few simpler methods, all of some value, at least for small, low precision calculations.

7.2.2 Note: the methods described below are

- *the Explicit Trapezoid Method*
- *the Explicit Midpoint Method*, and
- *The classical Runge-Kutta Method*,

with comparisons back to *Euler's Method* seen in *Basic Concepts and Euler's Method*.

7.2.3 Euler's Method as a Runge-Kutta method

The simplest of all methods of this general form is Euler's method. To set up the notation to be used below, rephrase it this way:

To get from (t, u) to an approximation of $(t + h, u(t + h))$, use the approximation

$$K_1 = hf(t, u) \\ u(t + h) \approx u + K_1$$

7.2.4 Second order Runge-Kutta methods

We have seen that the global error of Euler's method is $O(h)$: it is only first order accurate. This is often insufficient, so it is more common even for small, low precision calculation to use one of several second order methods:

The Explicit Trapezoid Method (a.k.a. the Improved Euler method or Huen's method)

One could try to adapt the trapezoid method for integrating $f(t)$ to solve $du/dt = f(t)$

$$u(t + h) = u(t) + \int_t^{t+h} f(s)ds \approx u(t) + h \frac{f(t) + f(t + h)}{2}$$

to solving the ODE $du/dt = f(t, u)$ but there is a problem that needs to be overcome:

we get

$$u(t + h) \approx u(t) + h \frac{f(t, u(t)) + hf(t + h, u(t + h))}{2}$$

and inserting the values $U_i \approx u(t_i)$ and so on gives

$$U_{i+1} \approx U_i + h \frac{f(t_i, U_i) + f(t_{i+1}, U_{i+1})}{2}$$

This is known as the **Implicit Trapezoid Rule**, because the value U_{i+1} that we seek appears at the right-hand side too: we only have an *implicit* formula for it.

On one hand, one can in fact use this formula, by solving the equation at each time step for the unknown U_{i+1} ; for example, one can use methods seen in earlier sections such as fixed point iteration or the secant method.

We will return to this in a later section; however, for now we get around this more simply by inserting an approximation at right — the only one we know so far, given by Euler's Method. That is:

- replace $u(t + h)$ at right by the tangent line approximation $u(t + h) \approx u(t) + hf(t, u(t))$, giving

$$u(t + h) \approx u(t) + h \frac{f(t, u(t)) + f(t + h, u(t) + hf(t, u(t)))}{2}$$

and for the formulas in terms of the U_i , replace U_{i+1} at right by $U_i + hf(t_i, U_i)$, giving

$$U_{i+1} = U_i + h \frac{f(t_i, U_i) + f(t_{i+1}, U_i + hf(t_i, U_i))}{2}$$

This is the **Explicit Trapezoid Rule**.

It is convenient to break this down into two stages, one for each evaluation of $f(t, u)$:

$$\begin{aligned} K_1 &= hf(t, u) \\ K_2 &= hf(t + h, u + K_1) \\ u(t + h) &\approx u + \frac{1}{2}(K_1 + K_2) \end{aligned}$$

For equal sized time steps, this leads to

Algorithm 6.2.1 (The Explicit Trapezoid Method)

$$\begin{aligned} U_0 &= u_0 \\ U_{i+1} &= U_i + \frac{1}{2}(K_1 + K_2), \\ &\text{where} \\ K_1 &= hf(t_i, U_i) \\ K_2 &= hf(t_{i+1}, U_i + K_1) \end{aligned}$$

We will see that, despite the mere first order accuracy of the Euler approximation used in getting K_2 , this method is second order accurate; the key is the fact that any error in the approximation used for $f(t + h, u(t + h))$ gets multiplied by h .

Exercise 1

- Verify that for the simple case where $f(t, u) = f(t)$, this gives the same result as the composite trapezoid rule for integration.
- Do one step of this method for the canonical example $du/dt = ku$, $u(t_0) = u_0$. It will have the form $U_1 = GU_0$ where the growth factor G approximates the factor $g = e^{kh}$ for the exact solution $u(t_1) = gu(t_0)$ of the ODE.
- Compare to $G = 1 + kh$ seen for Euler's method.
- Use the previous result to express U_i in terms of $U_0 = u_0$, as done for Euler's method.

```
def explicitTrapezoid(f, a, b, u_0, n):
    """Use the Explicit Trapezoid Method (a.k.a Improved Euler)
    to solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for t.
    u = np.empty_like(t)
    u[0] = u_0
    for i in range(n):
        K_1 = f(t[i], u[i])*h
        K_2 = f(t[i]+h, u[i]+K_1)*h
        u[i+1] = u[i] + (K_1 + K_2)/2.
    return (t, u)
```

As always, this function can now also be imported from `numericalMethods`, with

```
from numericalMethods import explicitTrapezoid
```

Examples

For all methods in this section, we will solve for versions of Example 2 and 4 in the section *Basic Concepts and Euler's Method*.

$$\frac{du}{dt} = f_1(t, u) = ku$$

with solution

$$u(t) = u_1(t; a, u_0) = u_0 e^{t-a}$$

and

$$\frac{du}{dt} = k(\cos(t) - u) - \sin(t)$$

with solution

$$u(t) = u_2(t; a, u_0, k) = \cos t + ce^{-k(t-a)}, \quad c = u_0 - \cos(a)$$

For comparison, the same examples are *done below with Euler's method*.

```
def f2(t, u):
    """The simplest "genuine" ODE, (not just integration)
    The solution is u(t) = u(t; a, u_0) = u_0 exp(k(t-a))
    """
    return k*u
def u2(t, a, u_0, k):
    return u_0 * np.exp(k*(t-a))
```

```
a = 1.
b = 3.
u_0 = 2.
k = 1.5
n = 40
```

(continues on next page)

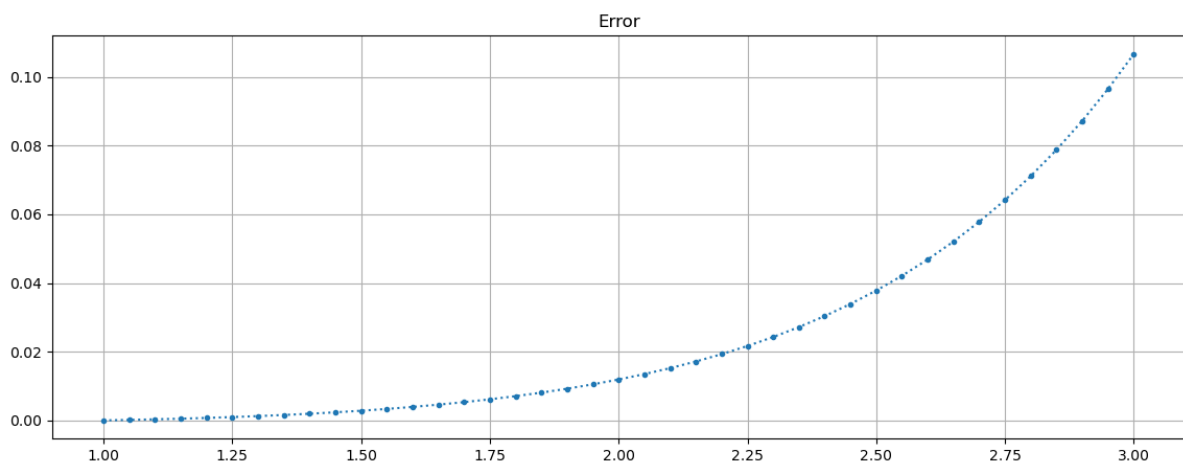
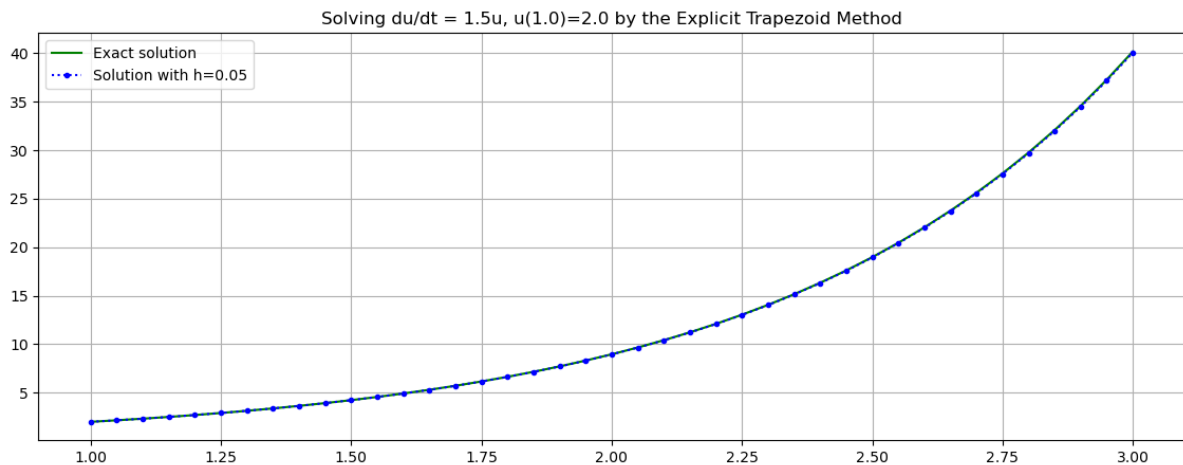
(continued from previous page)

```

(t, U) = explicitTrapezoid(f2, a, b, u_0, n)
u = u2(t, a, u_0, k)
h = (b-a)/n
figure(figsize=[14,5])
title(f"Solving du/dt = {k}u, u({a})={u_0} by the Explicit Trapezoid Method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b:", label=f"Solution with h={h:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
plot(t, u - U, 'b:')
grid(True);

```



```

def f4(t, u):
    """A simple more "generic" test case, with f(t, u) depending on both variables.
    The general solution is
        u(t) = u(t; a, u_0) = cos t + C e^(-k t),
        C = (u_0 - cos(a)) exp(k a)
    """

```

(continues on next page)

(continued from previous page)

```

    return k*(np.cos(t) - u) - np.sin(t)

def u4(t, a, u_0, k):
    return np.cos(t) + (u_0 - np.cos(a)) * np.exp(k*(a-t))

```

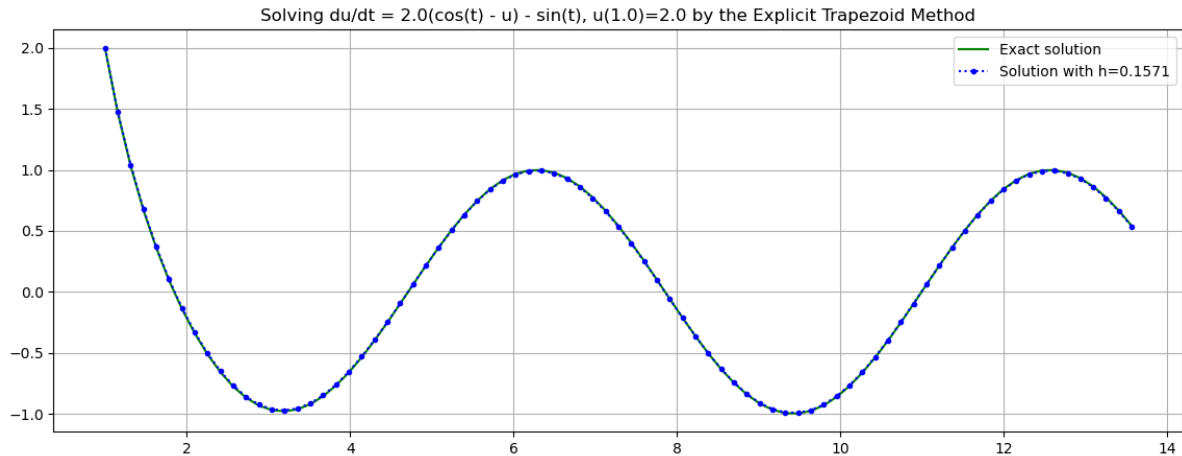
```

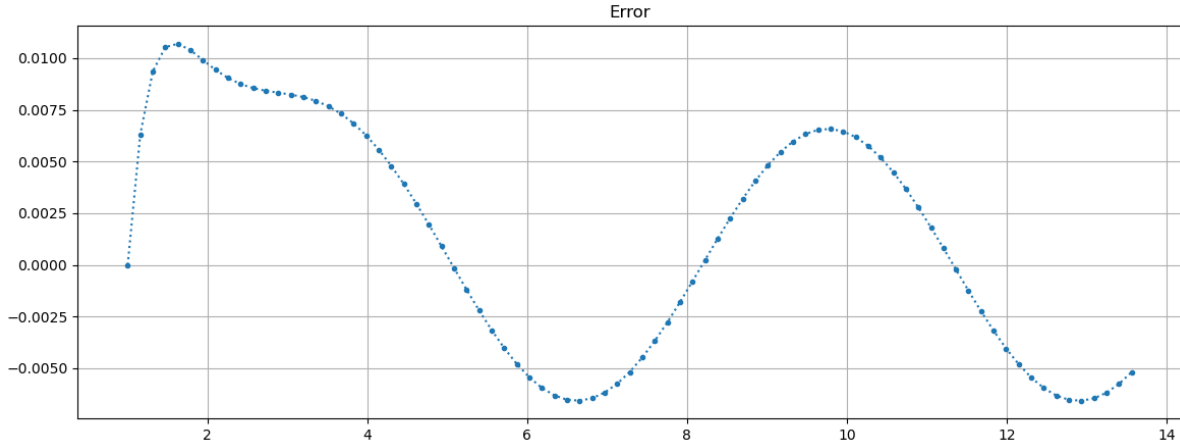
a = 1.
b = a + 4 * np.pi # Two periods
u_0 = 2.
k = 2.
n = 80

(t, U) = explicitTrapezoid(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)
#h = (b-a)/n
figure(figsize=[14,5])
title(f"Solving du/dt = {k}(cos(t) - u) - sin(t), u({a})={u_0} by the Explicit_
    Trapezoid Method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b:", label=f"Solution with h={(b-a)/n:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
plot(t, U - u, 'b:')
grid(True);

```





The Explicit Midpoint Method (a.k.a. Modified Euler)

If we start with the Midpoint Rule for integration in place of the Trapezoid Rule, we similarly get an approximation

$$u(t+h) \approx u(t) + hf(t+h/2, u(t+h/2))$$

This has the slight extra complication that it involves three values of u including $u(t+h/2)$ which we are not trying to evaluate. We deal with that by making yet another approximation, using an average of u values:

$$u(t+h/2) \approx \frac{u(t) + u(t+h)}{2}$$

leading to

$$u(t+h) \approx u(t) + hf\left(t+h/2, \frac{u(t) + u(t+h)}{2}\right)$$

and in terms of $U_i \approx u(t_i)$, the **Implicit Midpoint Rule**

$$U_{i+1} = U_i + hf\left(t+h/2, \frac{U_i + U_{i+1}}{2}\right)$$

We will see later that this is a particularly useful method in some situations, such as long-time solutions of ODEs that describe the motion of physical systems with conservation of momentum, angular momentum and kinetic energy.

However, for now we again seek a more straightforward *explicit* method; using the same tangent line approximation strategy as above gives the **Explicit Midpoint Rule**

$$\begin{aligned} K_1 &= hf(t, u) \\ K_2 &= hf(t+h/2, u + K_1/2) \\ u(t+h) &\approx u + K_2 \end{aligned}$$

and thus for equal-sized time steps

Algorithm 6.2.2 (The Explicit Midpoint Method)

$$\begin{aligned} U_0 &= u_0 \\ U_{i+1} &= U_i + K_2 \\ \text{where} \\ K_1 &= hf(t_i, U_i) \\ K_2 &= hf(t_i + h/2, U_i + K_1/2) \end{aligned}$$

Exercise 2 (a lot like Exercise 1)

- A) Verify that for the simple case where $f(t, u) = f(t)$, this give the same result as the composite midpoint rule for integration (same comment as above).
- B) Do one step of this method for the canonical example $du/dt = ku$, $u(t_0) = u_0$. It will have the form $U_1 = GU_0$ where the growth factor G approximates the factor $g = e^{kh}$ for the exact solution $u(t_1) = gu(t_0)$ of the ODE.
- C) Compare to the growth factors G seen for previous methods, and to the growth factor g for the exact solution.

Exercise 3

- A) Apply Richardson extrapolation to one step of Euler's method, using the values given by step sizes h and $h/2$.
- B) This should give a second order accurate method, so compare it to the above two methods.

```
def explicitMidpoint(f, a, b, u_0, n):
    """Use the Explicit Midpoint Method (a.k.a Modified Euler)
    to solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for_
    ↪ t.
    u = np.empty_like(t)
    u[0] = u_0
    for i in range(n):
        K_1 = f(t[i], u[i])*h
        K_2 = f(t[i]+h/2, u[i]+K_1/2)*h
        u[i+1] = u[i] + K_2
    return (t, u)
```

Again, available for import with

```
from numericalMethods import explicitMidpoint
```

Examples

```
a = 1.
b = 3.
u_0 = 2.
k = 1.5
n = 40

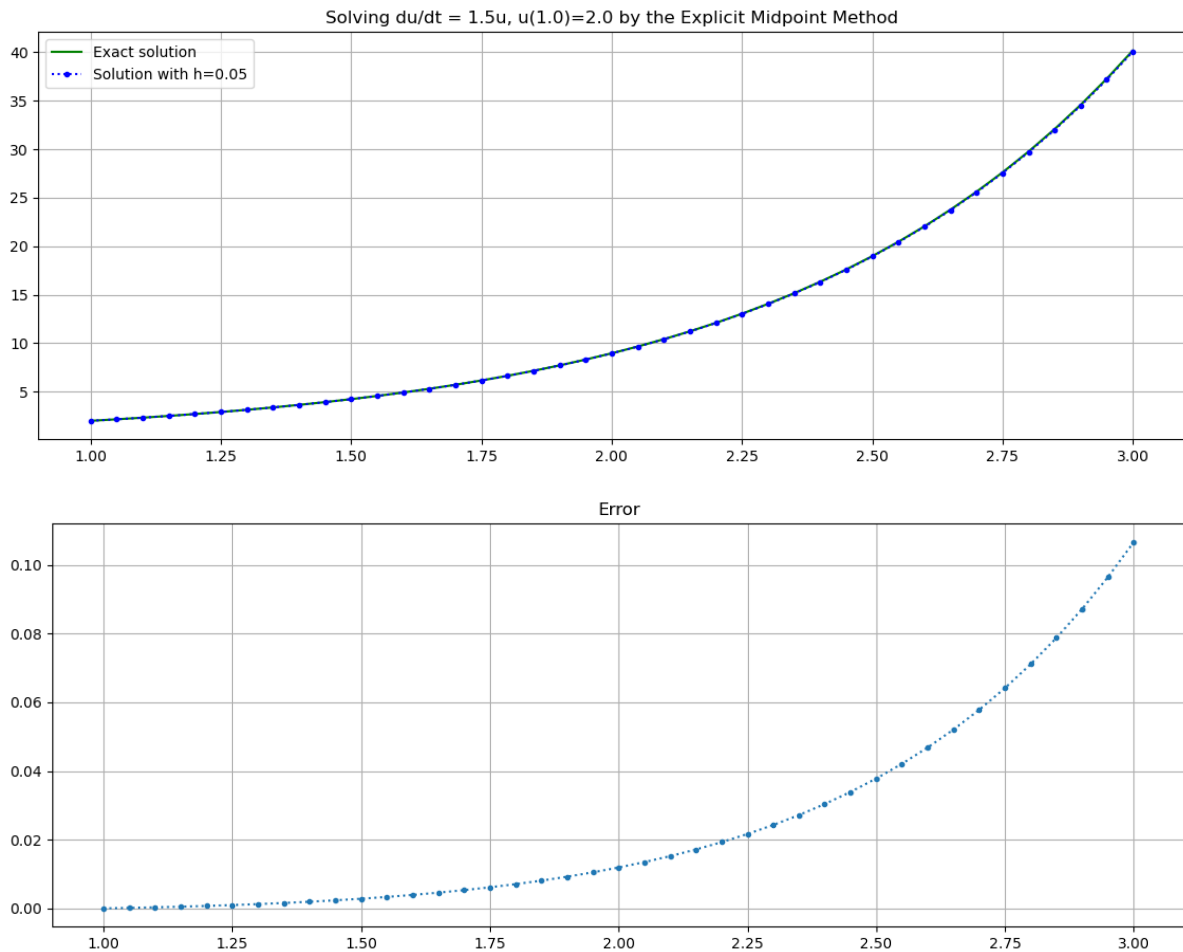
(t, U) = explicitMidpoint(f2, a, b, u_0, n)
u = u2(t, a, u_0, k)
figure(figsize=[14,5])
title(f"Solving du/dt = {k}u, u({a})={u_0} by the Explicit Midpoint Method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b", label=f"Solution with h={ (b-a)/n:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
```

(continues on next page)

(continued from previous page)

```
plot(t, u - U, '.:')
grid(True);
```



Observation: The errors are very similar to those for the Explicit Trapezoid Method, not “half as much and of opposite sign” as seen with integration; the exercises give a hint as to why this is so.

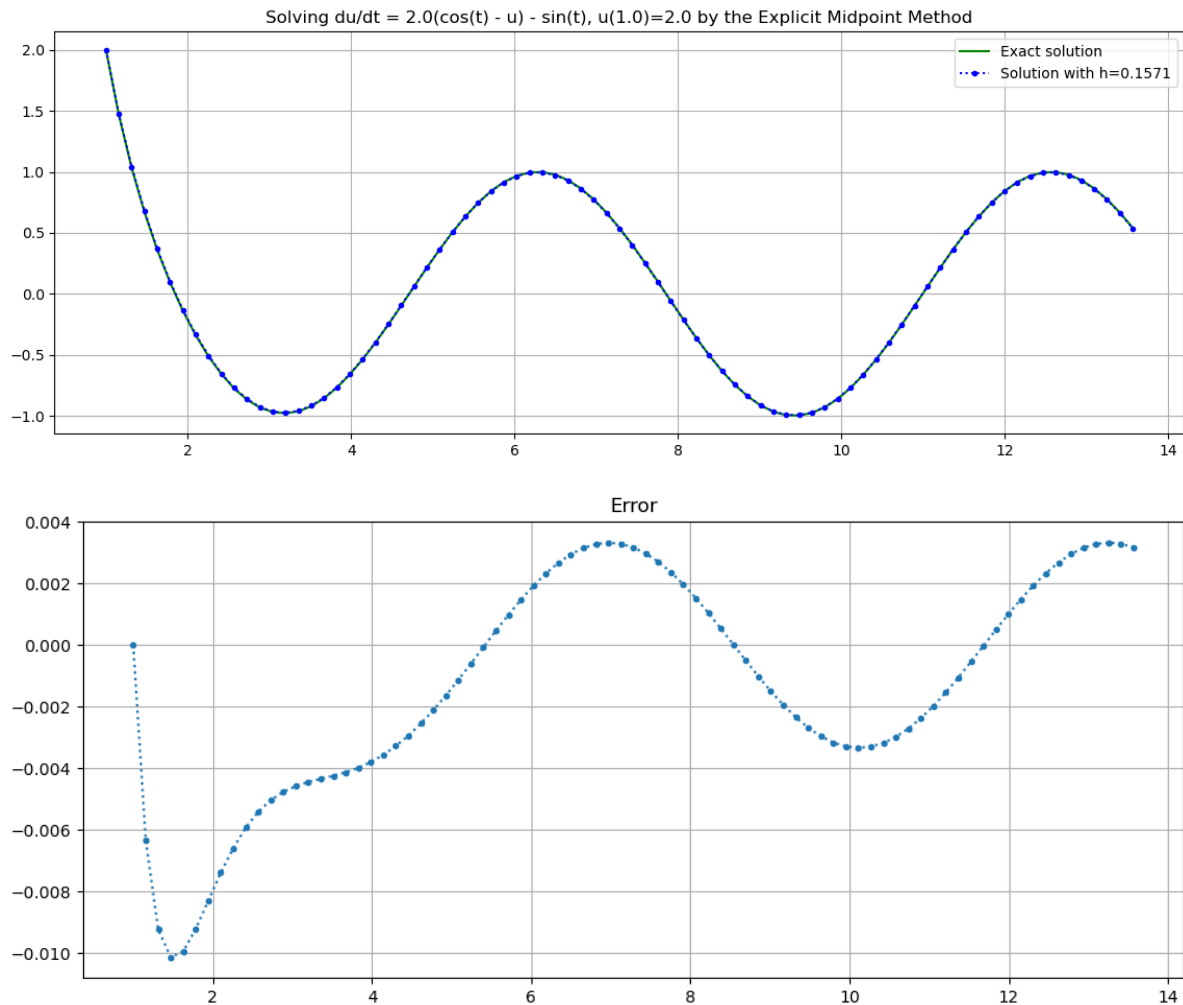
```
a = 1.
b = a + 4 * np.pi # Two periods
u_0 = 2.
k = 2.
n = 80

(t, U) = explicitMidpoint(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)
h = (b-a)/n
figure(figsize=[14,5])
title(f"Solving du/dt = {k}(cos(t) - u) - sin(t), u({a})={u_0} by the Explicit_
↵Midpoint Method")
plot(t, u, "g", label="Exact solution")
plot(t, U, ".:b", label=f"Solution with h={(b-a)/n:0.4}")
legend()
grid(True)
```

(continues on next page)

(continued from previous page)

```
figure(figsize=[12,5])
title(f"Error")
plot(t, u - U, 'b.:')
grid(True);
```



Observation: This time, the errors are slightly better than for the Explicit Trapezoid Method but still not “half as much” as seen with integration; this is because this equation has a mix of integration (the “sin” and “cos” parts) and exponential growth (the “Ku” part.)

7.2.5 The “Classical”, Fourth Order Accurate, Runge-Kutta Method

This is the original Runge-Kutta method:

Algorithm 6.2.3 (Runge-Kutta)

$$\begin{aligned} K_1 &= hf(t, u) \\ K_2 &= hf(t + h/2, u + K_1/2) \\ K_3 &= hf(t + h/2, u + K_2/2) \\ K_4 &= hf(t + h, u + K_3) \\ u(t + h) &\approx u + \frac{1}{6}(K_1 + 2K_2 + 2K_3 + K_4) \end{aligned}$$

The derivation of this is far more complicated than those above, and is omitted. For now, we will instead assess its accuracy “*a posteriori*”, through the next exercise and some examples.

Exercise 4

- A) Verify that for the simple case where $f(t, u) = f(t)$, this gives the same result as the composite Simpson’s Rule for integration.
- B) Do one step of this method for the canonical example $du/dt = ku$, $u(t_0) = u_0$. It will have the form $U_1 = GU_0$ where the growth factor G approximates the factor $g = e^{kh}$ for the exact solution $u(t_1) = gu(t_0)$ of the ODE.
- C) Compare to the growth factors G seen for previous methods, and to the growth factor g for the exact solution.

```
def rungeKutta(f, a, b, u_0, n):
    """Use the (classical) Runge-Kutta Method
    to solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for_
    ↪ t.
    u = np.empty_like(t)
    u[0] = u_0
    for i in range(n):
        K_1 = f(t[i], u[i])*h
        K_2 = f(t[i]+h/2, u[i]+K_1/2)*h
        K_3 = f(t[i]+h/2, u[i]+K_2/2)*h
        K_4 = f(t[i]+h, u[i]+K_3)*h
        u[i+1] = u[i] + (K_1 + 2*K_2 + 2*K_3 + K_4)/6
    return (t, u)
```

Yet again, available for import with

```
from numericalMethods import rungeKutta
```

Examples

```

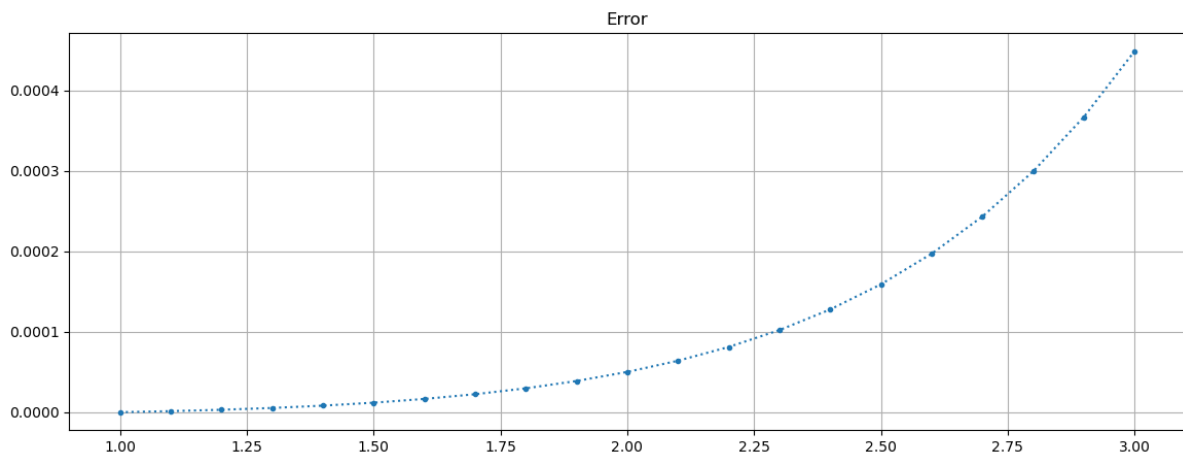
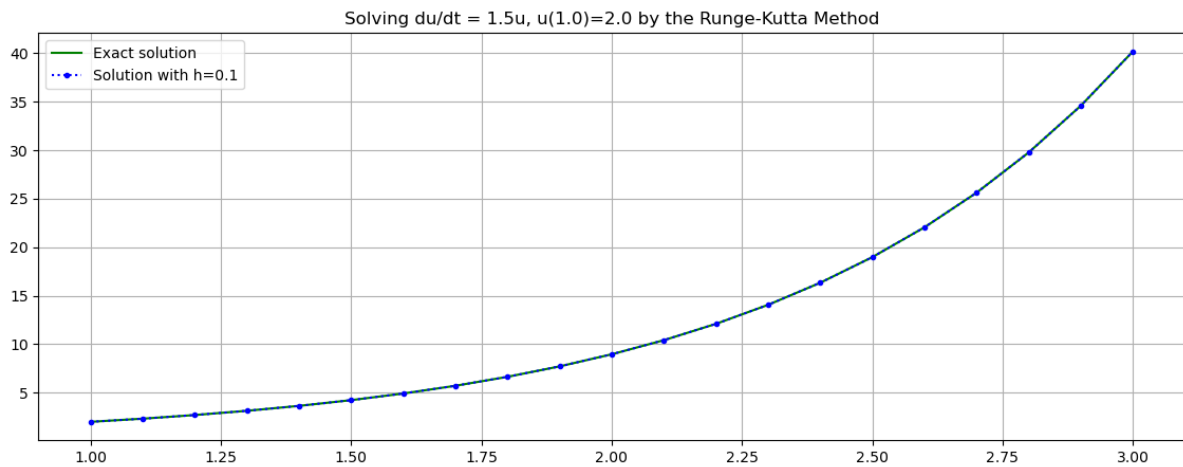
a = 1.
b = 3.
u_0 = 2.
k = 1.5
n = 20

(t, U) = rungeKutta(f2, a, b, u_0, n)
u = u2(t, a, u_0, k)

figure(figsize=[14,5])
title(f"Solving du/dt = {k}u, u({a})={u_0} by the Runge-Kutta Method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b:", label=f"Solution with h={(b-a)/n:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
plot(t, u - U, "b:")
grid(True);

```



```

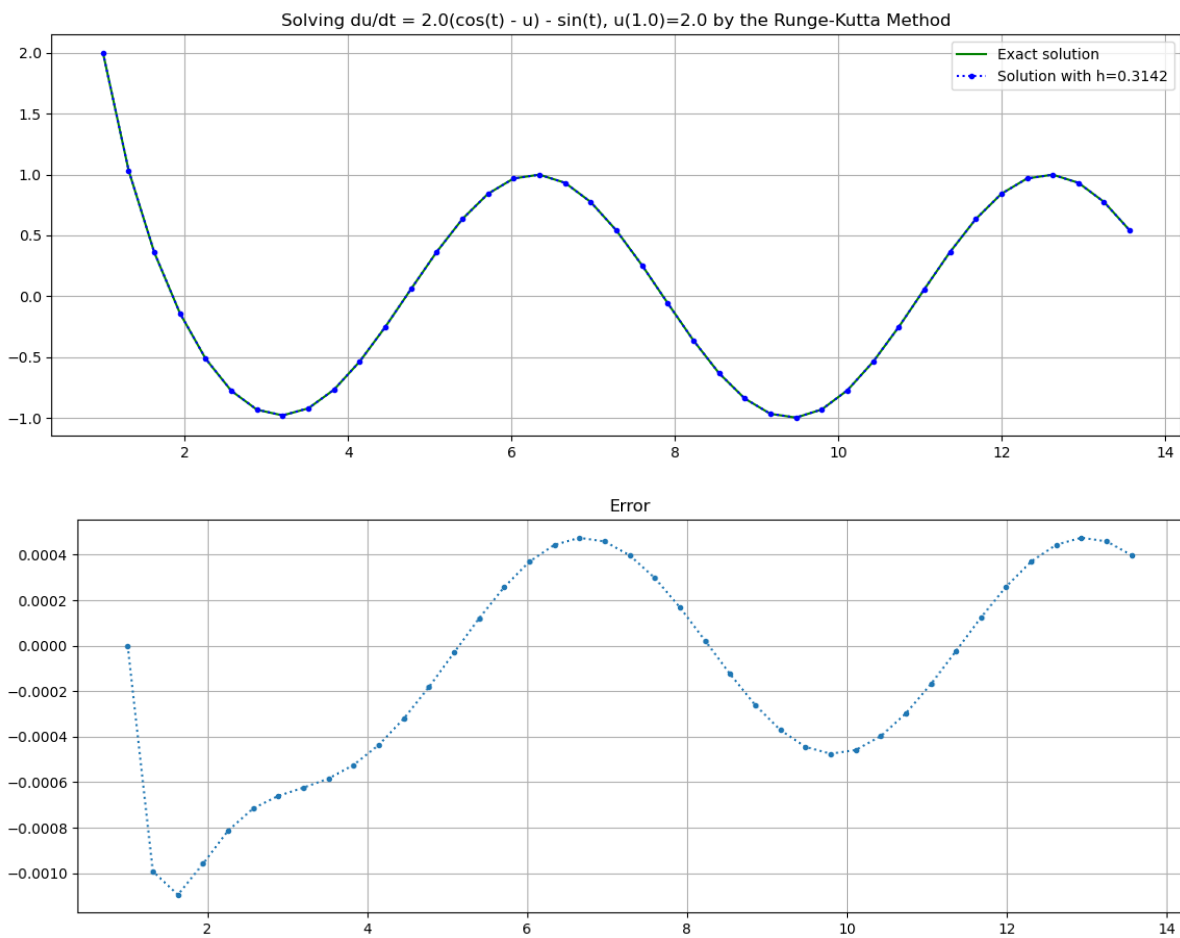
a = 1.
b = a + 4 * np.pi # Two periods
u_0 = 2.
k = 2.
n = 40

(t, U) = rungeKutta(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)

figure(figsize=[14,5])
title(f"Solving du/dt = {k}(cos(t) - u) - sin(t), u({a})={u_0} by the Runge-Kutta_
Method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b:", label=f"Solution with h={(b-a)/n:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
plot(t, u - U, "b:")
grid(True);

```



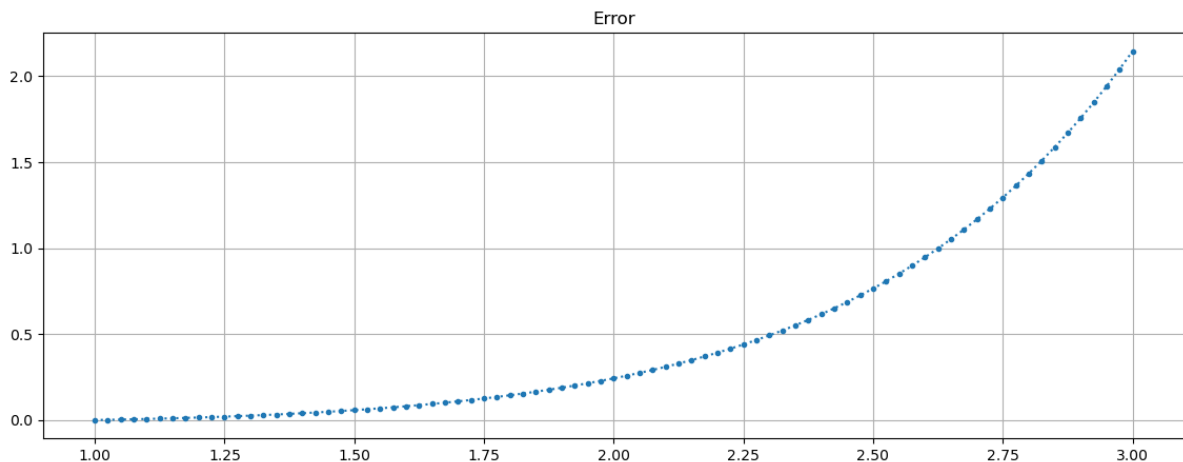
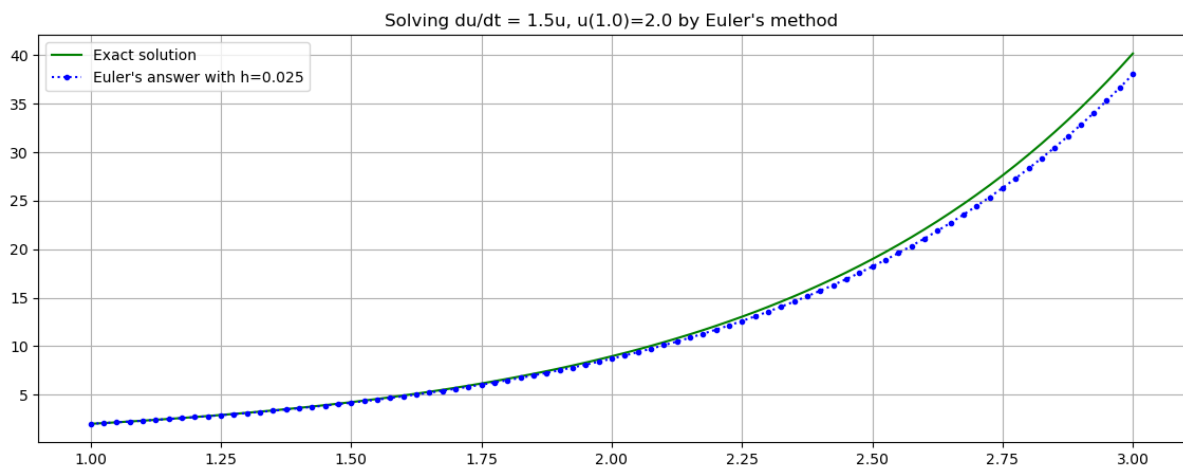
7.2.6 For comparison: the above examples done with Euler's Method

```
from numericalMethods import eulerMethod
```

```
a = 1.
b = 3.
u_0 = 2.
k = 1.5
n = 80

(t, U) = eulerMethod(f2, a, b, u_0, n)
u = u2(t, a, u_0, k)
figure(figsize=[14,5])
title(f"Solving du/dt = {k}u, u({a})={u_0} by Euler's method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b.", label=f"Euler's answer with h={(b-a)/n:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
plot(t, u - U, "b.")
grid(True);
```



```

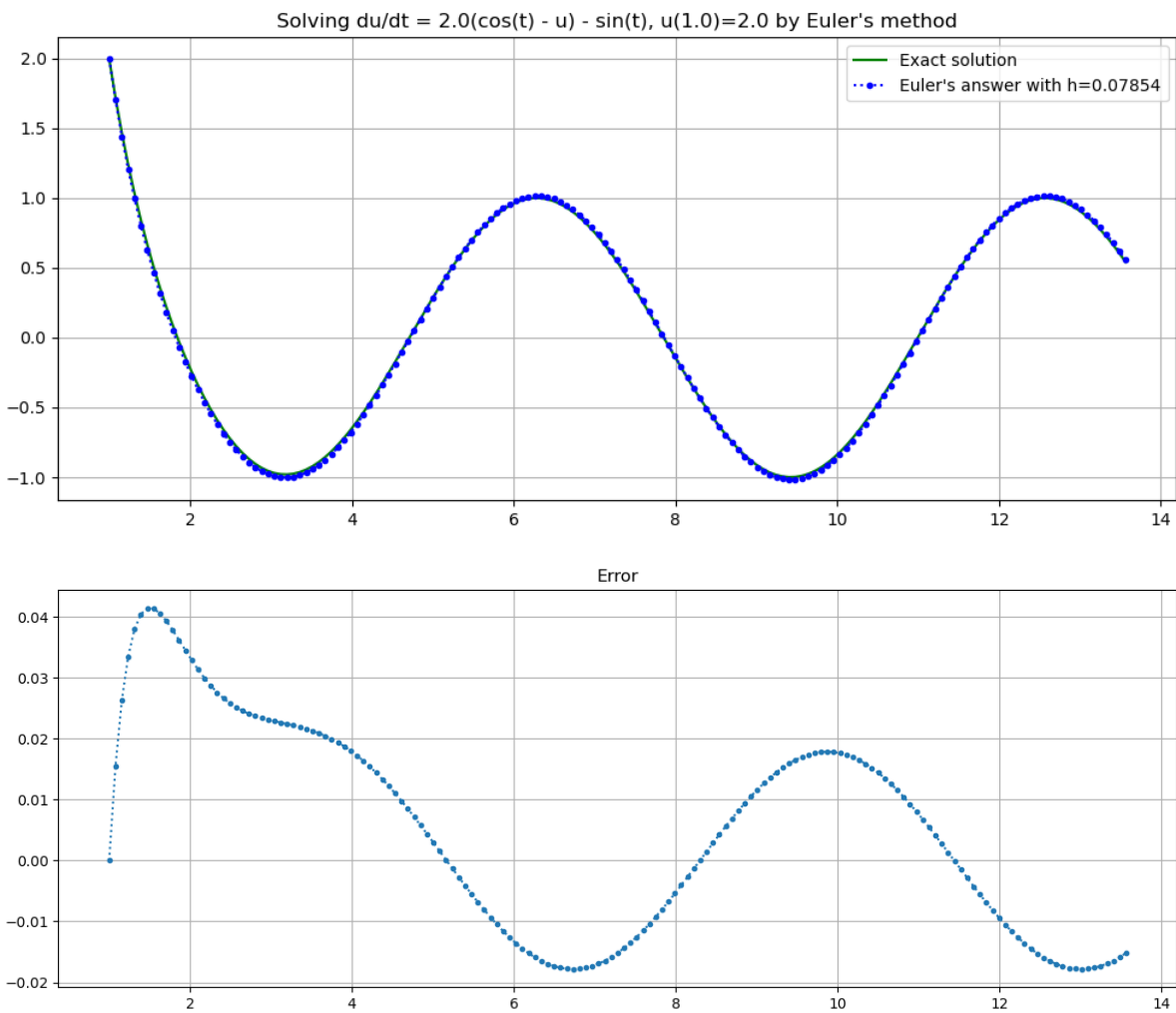
a = 1.
b = a + 4 * np.pi # Two periods
u_0 = 2.
k = 2.
n = 160

(t, U) = eulerMethod(f4, a, b, u_0, n)
u = u4(t, a, u_0, k)

figure(figsize=[12,5])
title(f"Solving du/dt = {k}(cos(t) - u) - sin(t), u({a})={u_0} by Euler's method")
plot(t, u, "g", label="Exact solution")
plot(t, U, "b:", label=f"Euler's answer with h={ (b-a)/n:0.4}")
legend()
grid(True)

figure(figsize=[14,5])
title(f"Error")
plot(t, u - U, "b:")
grid(True);

```



7.3 A Global Error Bound for One Step Methods

References:

- Subsection 6.2.1 *Local and global truncation error* in [Sauer, 2022].
- Section 5.2 *Euler's Method* in [Burden *et al.*, 2016].
- Section 8.5 of [Kincaid and Cheney, 1990]

All the methods seen so far for solving ODE IVP's are *one-step methods*: they fit the general form

$$U_{i+1} = F(t_i, U_i, h)$$

For example, Euler's Method has

$$F(t, U, h) = U + hf(t, U),$$

the Explicit Midpoint Method (Modified Euler) has

$$F(t, U, h) = U + hf(t + h/2, U + hf(t, U)/2)$$

and even the Runge-Kutta method has a similar form, but it is long and ugly.

For these, there is a general result that gives a bound on the global truncation error ("GTE") once one has a suitable bound on the local truncation error ("LTE"). This is very useful, because bounds on the LTE are usually far easier to derive.

Theorem 6.3.1

When solving the ODE IVP

$$du/dt = f(t, u), \quad u(a) = u_0$$

on interval $t \in [a, b]$ by a one step method, one has a bound on the local truncation error

$$|e_i| = |U_{i+1} - u(t_i + h; t_i, U_i)| = |F(t_i, U_i, h) - u(t_i + h; t_i, U_i)| \leq Ch^{p+1} = O(h^{p+1})$$

and the ODE itself satisfies the *Lipschitz Condition* that for some constant K ,

$$\left| \frac{\partial F}{\partial u}(t, u) \right| \leq K$$

then there is a bound on the global truncation error:

$$|E_i| = |U_i - u(t_i; a, u_0)| \leq C \frac{e^{K(t_i-a)} - 1}{K} h^p, = O(h^p)$$

So yet again, there is a loss of one factor of h in going from local to global error, as first seen with the composite rules for definite integrals.

We saw a glimpse of this for Euler's method, in the section [Basic Concepts and Euler's Method](#) where the Taylor's Theorem error formula can be used to get the LTE bound

$$|e_i| \leq Ch^2 \text{ where } C = \frac{|u_0 e^{K(b-a)}|}{2}$$

and this leads to the GTE bound

$$|E_i| \leq \frac{|u_0 e^{K(b-a)}|}{2} \frac{e^{K(t_i-a)} - 1}{K} h.$$

7.3.1 Order of accuracy for the basic Runge-Kutta type methods

- For Euler’s method, it was stated in section *Basic Concepts and Euler’s Method*, (and verified for the test case of $du/dt = ku$) that the global truncation error is of first order in step-size h :
- The Explicit (and Implicit) Trapezoid and Midpoint rules, the local truncation error is $O(h^3)$ and so their global truncation error is $O(h^2)$ — they are second order accurate, just as for the corresponding approximate integration rules.
- The classical Runge-Kutta method, has local truncation error $O(h^5)$ and so its global truncation error is $O(h^4)$ — just as for the composite Simpson’s Rule, to which it corresponds for the “integration” case $dy/dt = f(t)$.

7.4 Systems of ODEs and Higher Order ODEs

References:

- Section 6.3 *Systems of Ordinary Differential Equations* in [Sauer, 2022], to Sub-section 6.3.1 *Higher order equations*.
- Section 5.9 *Higher Order Equations and Systems of Differential Equations* in [Burden et al., 2016].

The short version of this section is that the numerical methods and algorithms developed so far for the initial value problem

$$\begin{aligned}\frac{du}{dt} &= f(t, u(t)), \quad a \leq t \leq b \\ u(a) &= u_0\end{aligned}$$

all also work for system of first order ODEs by simply letting u and f be vector-valued, and for that, the Python code requires only one small change.

Also, higher order ODE’s (and systems of them) can be converted into systems of first order ODEs.

7.4.1 Converting a second order ODE to a first order system

To convert

$$y'' = f(t, y, y')$$

with initial conditions

$$y(a) = y_0, \quad y'(a) = v_0$$

to a first order system, introduce the two functions

$$\begin{aligned}u_1(t) &= y(t) \\ u_2(t) &= \frac{dy}{dt} = u_1'(t)\end{aligned}$$

Then

$$y'' = u_1' = f(t, u_0, u_1)$$

and combining with the definition of u_1 gives the system

$$\begin{aligned}u_0' &= u_1 \\ u_1' &= f(t, u_0, u_1) \\ &\text{with initial conditions} \\ u_0(a) &= y_0 \\ u_1(a) &= v_0\end{aligned}$$

Next this can be put into vector form. Defining the vector-valued functions

$$\begin{aligned}\tilde{u}(t) &= \langle u_1(t), u_2(t) \rangle \\ \tilde{f}(t, \tilde{u}(t)) &= \langle u_1(t), f(t, u_2(t), u_2(t)) \rangle\end{aligned}$$

and initial data vector

$$\tilde{u}_0 = \langle u_{0,1}, u_{0,2} \rangle = \langle y_0, v_0 \rangle$$

puts the equation into the form

$$\begin{aligned}\frac{d\tilde{u}}{dt} &= \tilde{f}(t, \tilde{u}(t)), \quad a \leq t \leq b \\ \tilde{u}(a) &= \tilde{u}_0 \\ \frac{d\tilde{u}}{dt} &= \tilde{f}(t, \tilde{u}(t)), \quad a \leq t \leq b \\ \tilde{u}(a) &= \tilde{u}_0\end{aligned}$$

7.4.2 Test Cases

In this and subsequent sections, numerical methods for higher order equations and systems will be compared using several test cases:

Test Case A: Motion of a (Damped) Mass-Spring System in One Dimension

A simple mathematical model of a damped mass-spring system is

$$\begin{aligned}M \frac{d^2 y}{dt^2} &= -Ky - D \frac{dy}{dt} \\ &\text{with initial conditions} \\ y(a) &= y_0 \\ \left. \frac{dy}{dt} \right|_{t=a} &= v_0\end{aligned} \tag{7.1}$$

where K is the spring constant and D is the coefficient of friction, or drag.

The first order system form can be left in terms of y and y' as

$$\frac{d}{dt} \begin{bmatrix} y \\ y' \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -K & -D \end{bmatrix} \begin{bmatrix} y \\ y' \end{bmatrix}$$

Exact solutions

For testing of numerical methods in this and subsequent sections, here are the exact solutions.

They depend on whether

- $D < D_0 := 2\sqrt{KM}$: *underdamped*,
- $D > D_0$: *overdamped*, or
- $D = D_0$: *critically damped*.

We will mostly explore the first two more “generic” cases.

For the underdamped case, the general solution is

$$y(t) = e^{-(D/(2M))(t-a)} [A \cos(\omega(t-a)) + B \sin(\omega(t-a))], \quad \omega = \frac{\sqrt{4KM - D^2}}{2M}$$

For the above initial conditions, $A = y_0$ and $B = (v_0 + y_0 D/(2M))/\omega$.

An important special case of this is the undamped system $M \frac{d^2 y}{dt^2} = -Ky$ for which the solutions become

$$y(t) = A \cos(\omega(t-a)) + B \sin(\omega(t-a)), \quad \omega = \sqrt{K/M}$$

and it can be verified that the “energy”

$$E(t) = \frac{M}{2} (y'(t))^2 + \frac{K}{2} (y(t))^2 = \frac{1}{2} (Ku_1^2 + Mu_2^2)$$

is conserved: $dE/dt = 0$. Conserved quantities can provide a useful check of the accuracy of numerical method, so we will look at this below.

For the overdamped case, the general solution is

$$y(t) = Ae^{\lambda_+(t-a)} + Be^{\lambda_-(t-a)}, \quad \lambda_{\pm} = \frac{-D \pm \Delta}{2M}, \quad \Delta = \sqrt{D^2 - 4KM}$$

For the above initial conditions, $A = M(v_0 - \lambda_- y_0)/\Delta$ and $B = y_0 - A$.

Remark 6.4.1 (Stiffness)

Fixing M and scaling $K = D \rightarrow \infty$, $\Delta = D\sqrt{1 - 4M/D} \approx D - 2M$ so

$$\lambda_- \approx -\frac{D}{M} + 1 \rightarrow -\infty, \quad \lambda_+ \approx -1.$$

Thus the time scales of the two exponential decays become hugely different, with the fast term $Be^{\lambda_-(t-a)}$ becoming negligible compared to the slower decaying $Ae^{\lambda_+(t-a)}$.

This is a simple example of **stiffness**, and influences the choice of a good numerical method for solving such equations.

The variable can be rescaled to the case $K = M = 1$, so that will be done from now on, but of course you can easily experiment with other parameter values by editing copies of the Jupyter notebooks.

Test Case B: A “Fast-Slow” Equation

The equation

$$y'' + (K+1)y' + Ky = 0, \quad y(0) = y_0, y'(0) = v_0$$

has first order system form

$$\frac{d}{dt} \begin{bmatrix} y \\ y' \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -K & -(K+1) \end{bmatrix} \begin{bmatrix} y \\ y' \end{bmatrix}$$

and the general solution

$$y(t) = Ae^{-t} + Be^{-Kt}$$

so for large K , it has two very disparate time scales, with only the slower scale of much significance after an initial transient.

This is a convenient “toy” example for testing two refinements to algorithms:

- Variable time step sizes, so that they can be short during the initial transient and longer later, when only the e^{-t} behavior is significant.
- Implicit methods that can effectively suppress the fast but extremely small e^{-kt} terms while handling the larger, slower terms accurately.

The examples below will use $K = 100$, but as usual, you are free to experiment with other values.

Test Case C: The Freely Rotating Pendulum

Both the above equations are constant coefficient linear, which is convenient for the sake of having exact solution to compare with, but one famous nonlinear example is worth exploring too.

A pendulum with mass m concentrated at a distance L from the axis of rotation and that can rotate freely in a vertical plane about that axis and with possible friction proportional to D , can be modeled in terms of its angular position θ and angular velocity $\omega = \theta'$ by

$$ML\theta'' = -Mg \sin \theta - DL\theta', \quad \theta(0) = \theta_0, \theta'(0) = \omega_0$$

or in system form

$$\frac{d}{dt} \begin{bmatrix} \theta \\ \omega \end{bmatrix} = \begin{bmatrix} \omega \\ -\frac{g}{L} \sin \theta - \frac{D}{M} \omega \end{bmatrix}$$

These notes will mostly look at the frictionless case $D = 0$, which has conserved energy

$$E(\theta, \omega) = \frac{ML}{2} \omega^2 - Mg \cos \theta$$

For this, the solution falls into three qualitatively different cases depending on whether the energy is less than, equal to, or greater than the “critical energy” Mg , which is the energy of the unstable stationary solutions $\theta(t) = \pi \pmod{2\pi}$, $\omega(t) = 0$: “balancing at the top”:

- For $E < Mg$, a solution can never reach the top, so the pendulum rocks back and forth, reaching maximum height at $\theta = \pm \arccos(-E/(Mg))$
- For $E > Mg$, solutions have angular speed $|\omega| \geq \sqrt{E - Mg} > 0$ so it never drops to zero, and so the direction of rotation can never reverse: solutions rotate in one direction for ever.
- For $E = Mg$, one special type of solution is those up-side down stationary ones. Any other solution always has $|\omega| = \sqrt{E - Mg \cos \theta} > 0$, and so never stops or reverses direction but instead approaches the above stationary point asymptotically both as $t \rightarrow \infty$ and $t \rightarrow -\infty$. To visualize concretely, the solution starting at the bottom with $\theta(0) = 0$, $\omega(0) = \sqrt{2g/L}$ has $\theta(t) \rightarrow \pm\pi$ and $\omega(t) \rightarrow 0$ as $t \rightarrow \pm\infty$.

Remark 6.4.2 (Separatrices)

This last kind of special solution is known as a **separatrix** due to separating the other two qualitatively different sorts of solution. They are also known as **heteroclinic orbits**, for “asymptotically” starting and ending at different stationary solutions in each time direction — or **homoclinic** if you consider the angle as a “mod 2π ” value describing a position, so that $\theta = \pm\pi$ are the same location and the solutions start and end at the same stationary point.

```
import numpy as np
# Shortcuts for some favorite mathematical functions and numbers:
from numpy import sqrt, sin, cos, pi
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend, show
```

The Euler’s method code from before does not quite work, but only slight modification is needed; that “scalar” version

```
def eulerMethod(f, a, b, u_0, n):
    h = (b-a)/n
    t = np.linspace(a, b, n+1)
    u = np.empty_like(t)
    u[0] = u_0
    for i in range(n):
        u[i+1] = u[i] + f(t[i], u[i])*h
    return (t, u)
```

becomes

```
def eulerMethodSystem(f, a, b, u_0, n):
    """Use Euler's Method to solve du/dt = f_mass_spring(t, u) for t in [a, b], with
    initial value u(a) = u_0
    Modified from function eulermethod to handle systems.
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1)

    # Only the following three lines change for the system version
    n_unknowns = len(u_0)
    u = np.zeros([n+1, n_unknowns])
    u[0] = np.array(u_0) # In case u_0 is a single number (the scalar case)

    for i in range(n):
        u[i+1] = u[i] + f(t[i], u[i])*h
    return (t, u)
```

7.4.3 Solving the Mass-Spring System

```
def f_mass_spring(t, u):
    return np.array([ u[1], -(K/M)*u[0] - (D/M)*u[1] ])
```

```
def E_mass_spring(y, Dy):
    return (K * y**2 + M * Dy**2)/2
```

```
def y_mass_spring(t, t_0, u_0, K, M, D):
    (y_0, v_0) = u_0
    discriminant = D**2 - 4*K*M
    if discriminant < 0: # underdamped
        omega = sqrt(4*K*M - D**2)/(2*M)
        A = y_0
        B = (v_0 + y_0*D/(2*M))/omega
        return exp(-D/(2*M)*(t-t_0)) * ( A*cos(omega*(t-t_0)) + B*sin(omega*(t-t_0)) )
    elif discriminant > 0: # overdamped
        Delta = sqrt(discriminant)
        lambda_plus = (-D + Delta)/(2*M)
        lambda_minus = (-D - Delta)/(2*M)
        A = M*(v_0 - lambda_minus * y_0)/Delta
        B = y_0 - A
        return A*exp(lambda_plus*(t-t_0)) + B*exp(lambda_minus*(t-t_0))
    else:
        q = -D/(2*M)
```

(continues on next page)

(continued from previous page)

```

A = y_0
B = v_0 - A * q
return (A + B*t)*exp(q*(t-t_0))

```

```

def damping(K, M, D):
    if D == 0:
        print("Undamped")
    else:
        discriminant = D**2 - 4*K*M
        if discriminant < 0:
            print("Underdamped")
        elif discriminant > 0:
            print("Overdamped")
        else:
            print("Critically damped")

```

The above functions are available in module `numericalMethods`; they will be used in later sections.

First solve without damping, so the solutions have sinusoidal solutions

Note: the orbits go clockwise for undamped (and underdamped) systems.

```

M = 1.0
K = 1.0
D = 0.0
y_0 = 1.0
Dy_0 = 0.0
u_0 = [y_0, Dy_0]
a = 0.0
periods = 4
b = 2 * pi * periods

stepsperperiod = 1000
n = stepsperperiod * periods

(t, U) = eulerMethodSystem(f_mass_spring, a, b, u_0, n)
Y = U[:,0]
DY = U[:,1]

figure(figsize=[14,7])
title(f"y and dy/dt with {K/M=}, {D=} by Euler's method with {stepsperperiod} steps_
↳per period")
plot(t, Y, label="y")
plot(t, DY, label="dy/dt")
legend()
xlabel("t")
grid(True)

# Phase plane diagram; for D=0 the exact solutions are ellipses (circles if M = k)

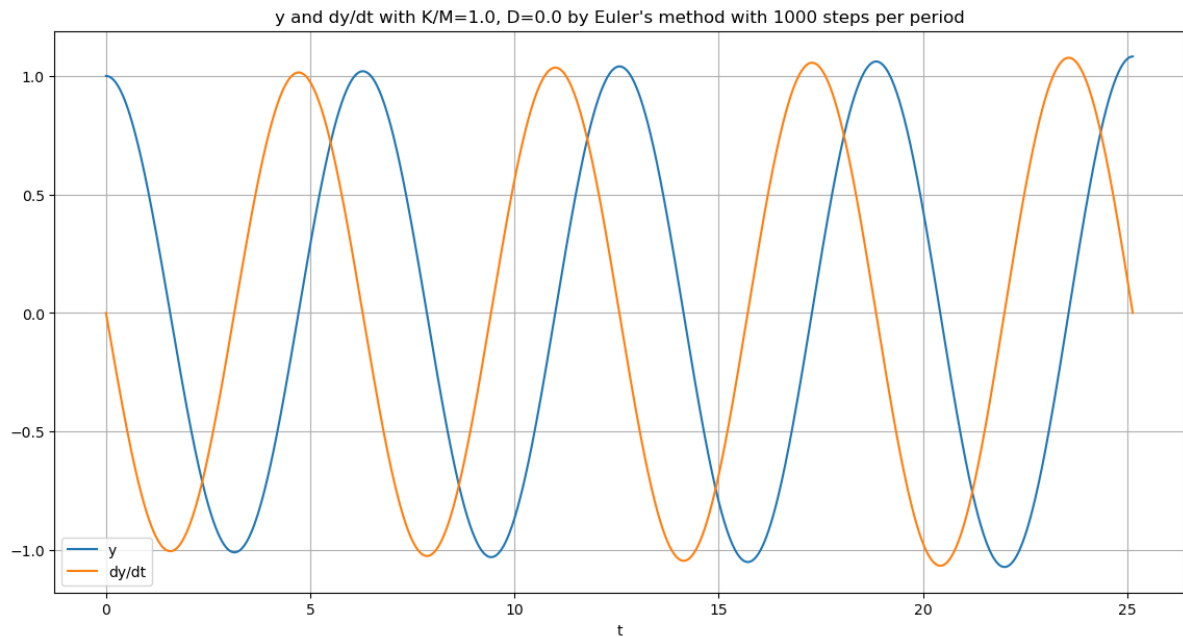
figure(figsize=[8,8]) # Make axes equal length; orbits should be circular or
↳"circular spirals"
title(f"The orbits of the mass-spring system, {K/M=}, {D=} by Euler's method with
↳{stepsperperiod} steps per period")

```

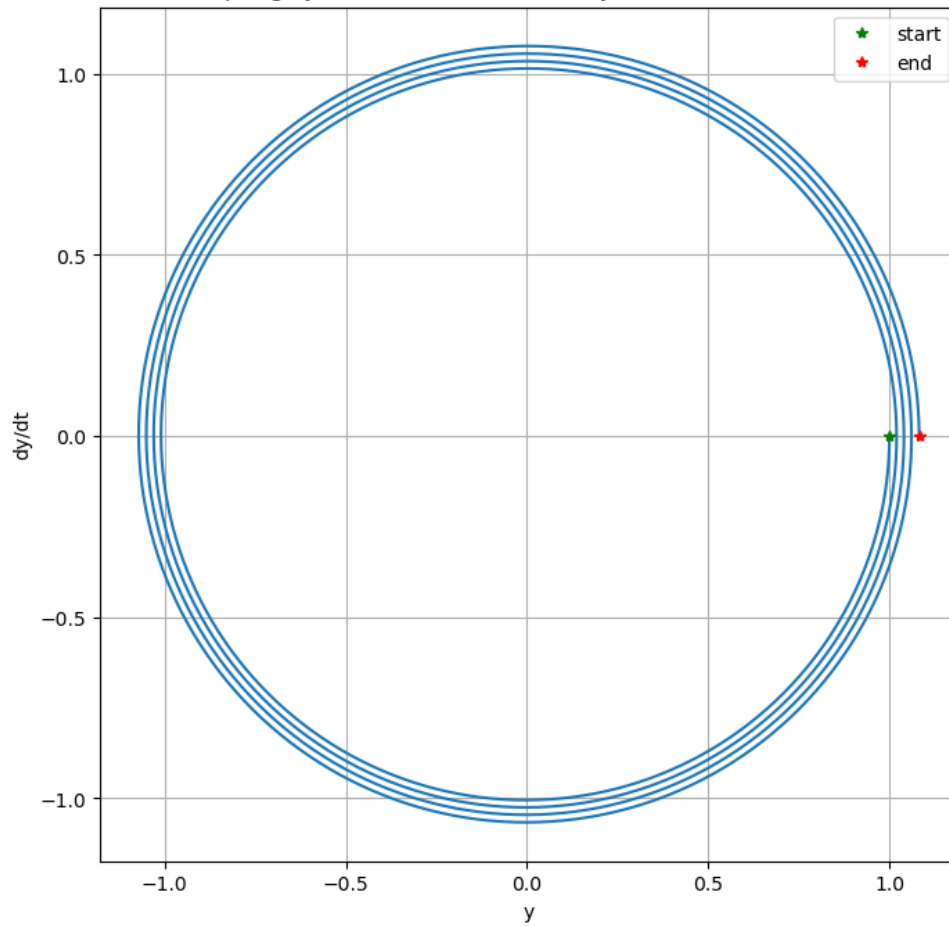
(continues on next page)

(continued from previous page)

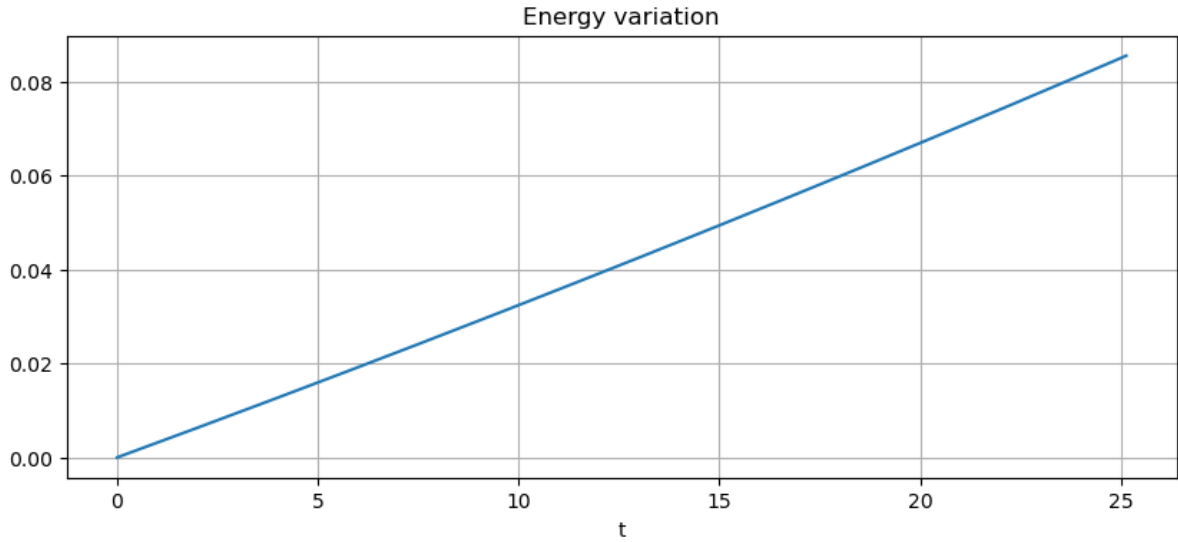
```
plot(Y, DY)
xlabel("y")
ylabel("dy/dt")
plot(Y[0], DY[0], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)
```



The orbits of the mass-spring system, $K/M=1.0$, $D=0.0$ by Euler's method with 1000 steps per period



```
figure(figsize=[10,4])
E_0 = E_mass_spring(y_0, Dy_0)
E = E_mass_spring(Y, DY)
title("Energy variation")
plot(t, E - E_0)
xlabel("t")
grid(True)
```



Damped

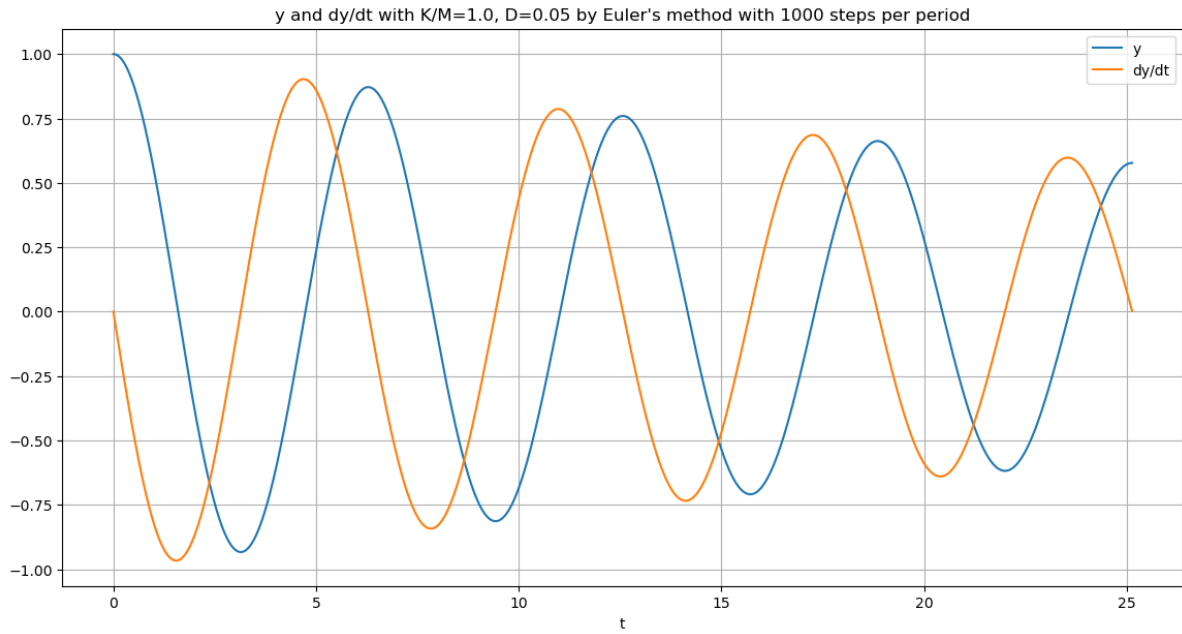
```

D = 0.05 # Underdamped: decaying oscillations
#D = 1.1 # Overdamped: exponential decay
(t, U) = eulerMethodSystem(f_mass_spring, a, b, u_0, n)
Y = U[:,0]
DY = U[:,1]

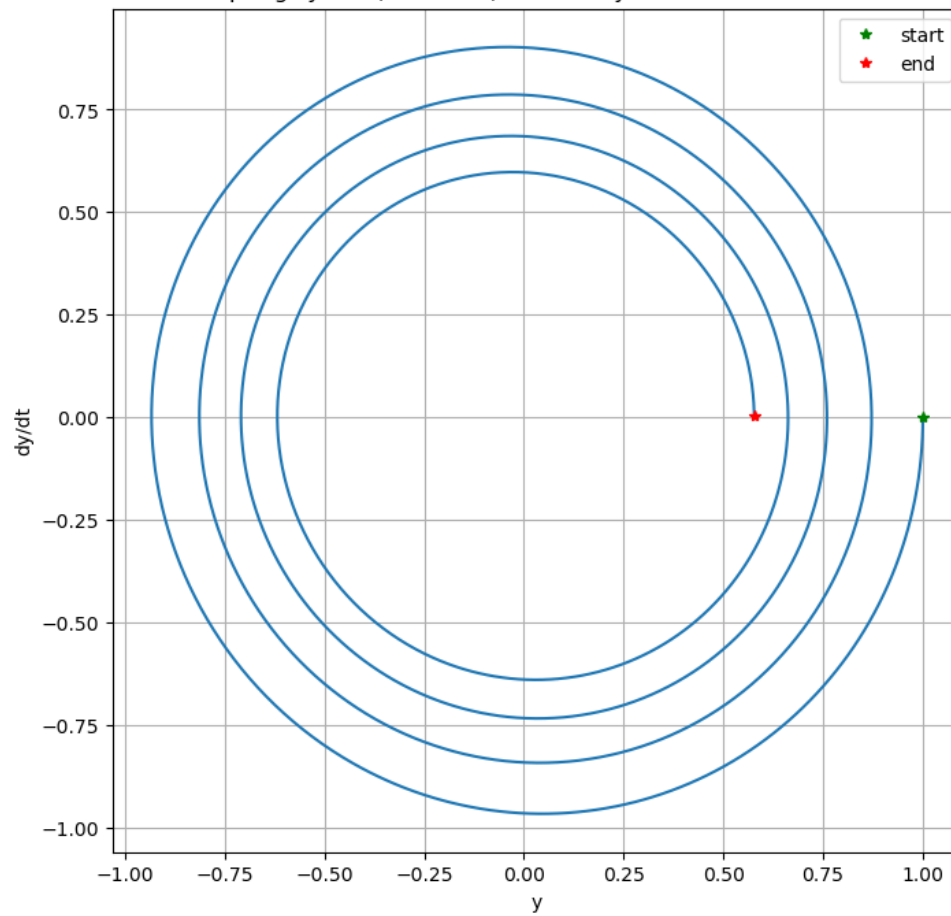
figure(figsize=[14,7])
title(f"y and dy/dt with {K/M=}, {D=} by Euler's method with {stepsperperiod} steps_
↳per period")
plot(t, Y, label="y")
plot(t, DY, label="dy/dt")
legend()
xlabel("t")
grid(True)

figure(figsize=[8,8]) # Make axes equal length
title(f"The orbits of the mass-spring system, {K/M=}, {D=} by Euler's method with
↳{stepsperperiod} steps per period")
plot(Y, DY)
xlabel("y")
ylabel("dy/dt")
plot(Y[0], DY[0], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```



The orbits of the mass-spring system, $K/M=1.0$, $D=0.05$ by Euler's method with 1000 steps per period



7.4.4 The “Classical” Runge-Kutta Method, Extended to Systems of Equations

As above, the previous “scalar” function for this method needs just three lines of code modified.

Before:

```
def rungeKutta(f, a, b, u_0, n):
    """Use the (classical) Runge-Kutta Method
    to solve  $du/dt = f_{\text{mass\_spring}}(t, u)$  for  $t$  in  $[a, b]$ , with initial value  $u(a) = u_0$ 
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for t.
    u = np.empty_like(t)
    u[0] = u_0
    for i in range(n):
        K_1 = f_mass_spring(t[i], u[i])*h
        K_2 = f_mass_spring(t[i]+h/2, u[i]+K_1/2)*h
        K_3 = f_mass_spring(t[i]+h/2, u[i]+K_2/2)*h
        K_4 = f_mass_spring(t[i]+h, u[i]+K_3)*h
        u[i+1] = u[i] + (K_1 + 2*K_2 + 2*K_3 + K_4)/6
    return (t, u)
```

After:

```
def rungeKuttaSystem(f, a, b, u_0, n):
    """Use the (classical) Runge-Kutta Method
    to solve system  $du/dt = f_{\text{mass\_spring}}(t, u)$  for  $t$  in  $[a, b]$ , with initial value  $u(a) = u_0$ 
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for t.

    # Only the following three lines change for the system version.
    n_unknowns = len(u_0)
    u = np.zeros([n+1, n_unknowns])
    u[0] = np.array(u_0)

    for i in range(n):
        K_1 = f_mass_spring(t[i], u[i])*h
        K_2 = f_mass_spring(t[i]+h/2, u[i]+K_1/2)*h
        K_3 = f_mass_spring(t[i]+h/2, u[i]+K_2/2)*h
        K_4 = f_mass_spring(t[i]+h, u[i]+K_3)*h
        u[i+1] = u[i] + (K_1 + 2*K_2 + 2*K_3 + K_4)/6
    return (t, u)
```

```
M = 1.0
k = 1.0
D = 0.0
y_0 = 1.0
Dy_0 = 0.0
u_0 = [y_0, Dy_0]
a = 0.0
periods = 4
b = 2 * pi * periods
```

(continues on next page)

(continued from previous page)

```

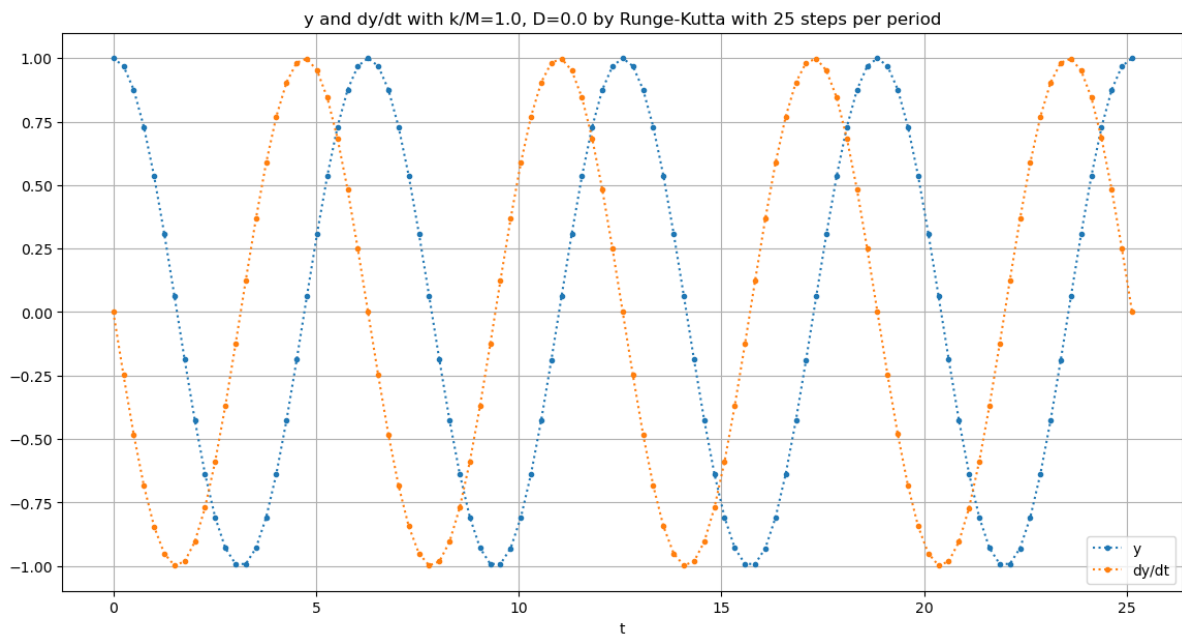
stepsperperiod = 25
n = stepsperperiod * periods

(t, U) = rungeKuttaSystem(f_mass_spring, a, b, u_0, n)
y = U[:,0]
Dy = U[:,1]

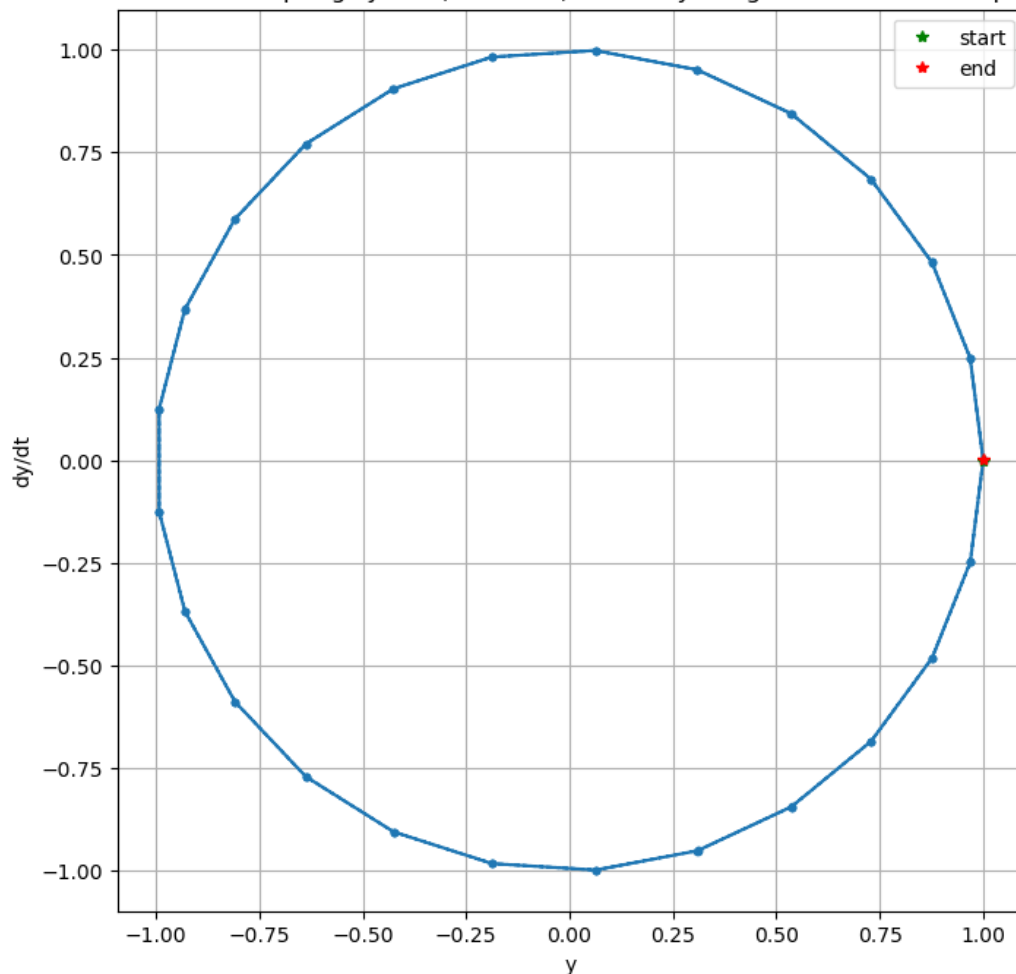
figure(figsize=[14,7])
title(f"y and dy/dt with {k/M=}, {D=} by Runge-Kutta with {stepsperperiod} steps per_
↵period")
plot(t, y, ":", label="y")
plot(t, Dy, ":", label="dy/dt")
legend()
xlabel("t")
grid(True)

figure(figsize=[8,8]) # Make axes equal length
title(f"The orbits of the mass-spring system, {k/M=}, {D=} by Runge-Kutta with
↵{stepsperperiod} steps per period")
plot(y, Dy, ":")
xlabel("y")
ylabel("dy/dt")
plot(y[0], Dy[0], "g*", label="start")
plot(y[-1], Dy[-1], "r*", label="end")
legend()
grid(True)

```



The orbits of the mass-spring system, $k/M=1.0$, $D=0.0$ by Runge-Kutta with 25 steps per period



7.4.5 Appendix: the Explicit Trapezoid and Midpoint Methods for systems

Yet again, the previous functions for these methods need just three lines of code modified.

The demos are just for the non-dissipative case, where the solution is known to be $y = \cos t$, $dt/dt = -\sin t$.

For a fairer comparison of “accuracy vs computational effort” to the Runge-Kutta method, twice as many time steps are used so that the same number of function evaluations are used for these three methods.

```
def explicitTrapezoidSystem(f, a, b, u_0, n):
    """Use the Explicit Trapezoid Method (a.k.a Improved Euler)
    to solve system  $du/dt = f_{\text{mass\_spring}}(t, u)$  for  $t$  in  $[a, b]$ , with initial value_
    ↪  $u(a) = u_0$ 
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1)

    # Only the following three lines change for the systems version
    n_unknowns = len(u_0)
    u = np.zeros([n+1, n_unknowns])
    u[0] = np.array(u_0)
```

(continues on next page)

(continued from previous page)

```

for i in range(n):
    K_1 = f_mass_spring(t[i], u[i])*h
    K_2 = f_mass_spring(t[i]+h, u[i]+K_1)*h
    u[i+1] = u[i] + (K_1 + K_2)/2.
return (t, u)

```

```

M = 1.0
k = 1.0
D = 0.0
y_0 = 1.0
Dy_0 = 0.0
u_0 = [y_0, Dy_0]
a = 0.0
periods = 4
b = 2 * pi * periods

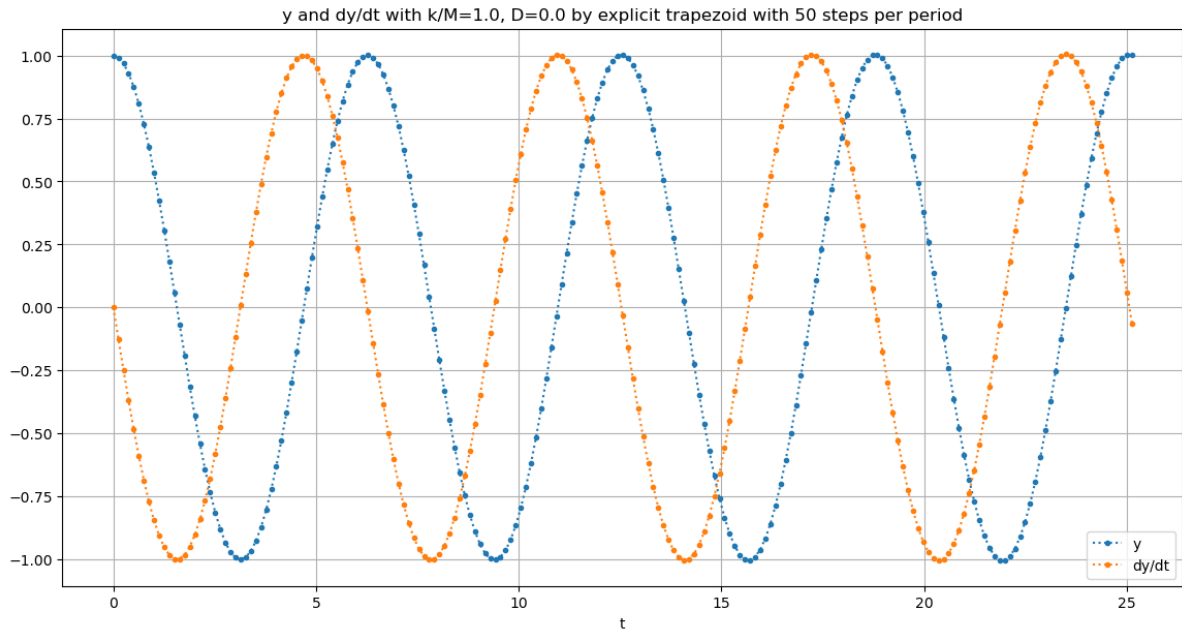
stepsperperiod = 50
n = stepsperperiod * periods

(t, U) = explicitTrapezoidSystem(f_mass_spring, a, b, u_0, n)
y = U[:,0]
Dy = U[:,1]

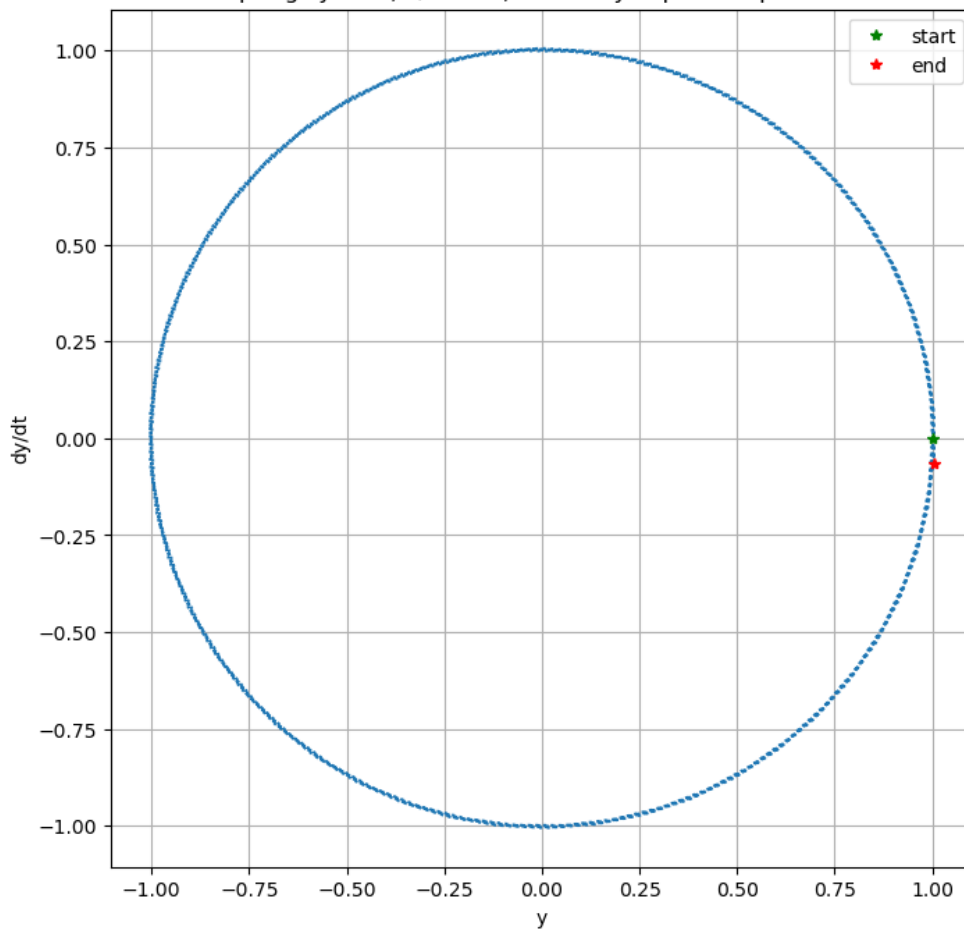
figure(figsize=[14,7])
title(f"y and dy/dt with {k/M=}, {D=} by explicit trapezoid with {stepsperperiod}
↳steps per period")
plot(t, y, ":", label="y")
plot(t, Dy, ":", label="dy/dt")
legend()
xlabel("t")
grid(True)

figure(figsize=[8,8]) # Make axes equal length
title(f"The orbits of the mass-spring system, {k/M=}, {D=} by explicit trapezoid with
↳{stepsperperiod} steps per period")
plot(y, Dy, ":")
xlabel("y")
ylabel("dy/dt")
plot(y[0], Dy[0], "g*", label="start")
plot(y[-1], Dy[-1], "r*", label="end")
legend()
grid(True)

```



The orbits of the mass-spring system, $k/M=1.0$, $D=0.0$ by explicit trapezoid with 50 steps per period



At first glance this is doing well, keeping the orbits circular. However note the discrepancy between the start and end points: these should be the same, as they are (visually) with the Runge-Kutta method.

```

def explicitMidpointSystem(f, a, b, u_0, n):
    """Use the Explicit Midpoint Method (a.k.a Modified Euler)
    to solve system  $du/dt = f_{\text{mass\_spring}}(t, u)$  for  $t$  in  $[a, b]$ , with initial value
    ↪  $u(a) = u_0$ 
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1)

    # Only the following three lines change for the systems version.
    n_unknowns = len(u_0)
    u = np.zeros([n+1, n_unknowns])
    u[0] = np.array(u_0)

    for i in range(n):
        K_1 = f_mass_spring(t[i], u[i])*h
        K_2 = f_mass_spring(t[i]+h/2, u[i]+K_1/2)*h
        u[i+1] = u[i] + K_2
    return (t, u)

```

```

M = 1.0
k = 1.0
D = 0.0
y_0 = 1.0
Dy_0 = 0.0
u_0 = [y_0, Dy_0]
a = 0.0
periods = 4
b = 2 * pi * periods

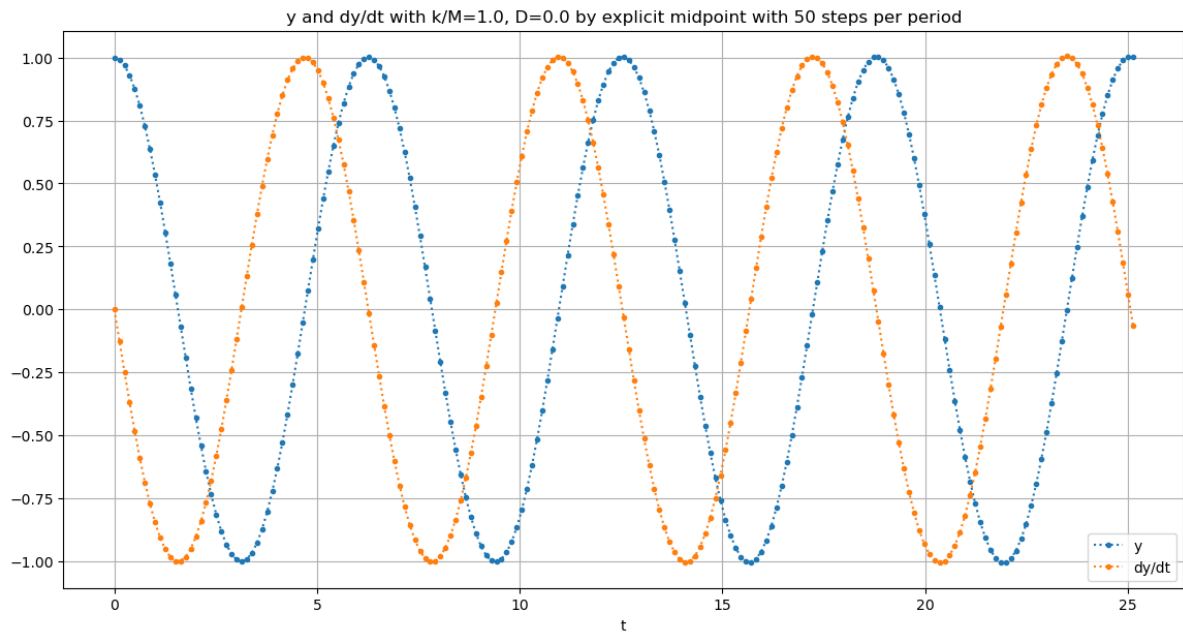
stepsperperiod = 50
n = stepsperperiod * periods

(t, U) = explicitMidpointSystem(f_mass_spring, a, b, u_0, n)
y = U[:,0]
Dy = U[:,1]

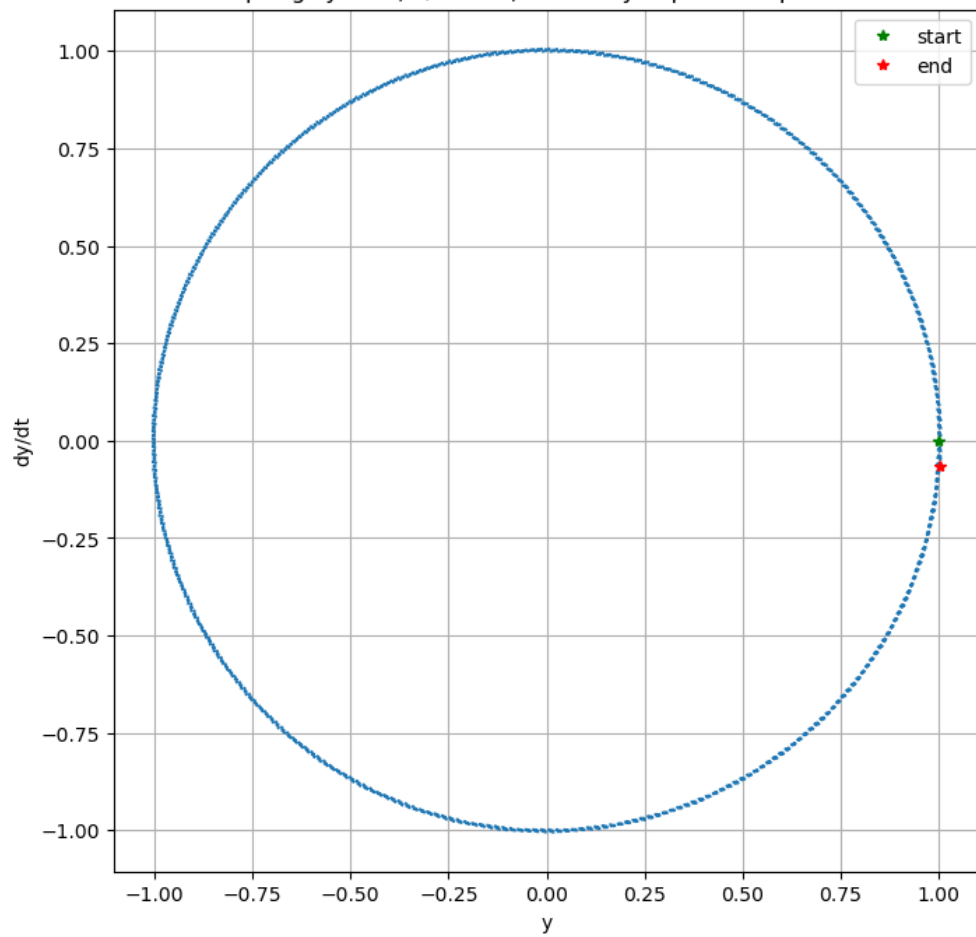
figure(figsize=[14,7])
title(f"y and dy/dt with {k/M=}, {D=} by explicit midpoint with {stepsperperiod} ↪
    ↪ steps per period")
plot(t, y, ":", label="y")
plot(t, Dy, ":", label="dy/dt")
legend()
xlabel("t")
grid(True)

figure(figsize=[8,8]) # Make axes equal length
title(f"The orbits of the mass-spring system, {k/M=}, {D=} by explicit midpoint with
    ↪ {stepsperperiod} steps per period")
plot(y, Dy, ":")
xlabel("y")
ylabel("dy/dt")
plot(y[0], Dy[0], "g*", label="start")
plot(y[-1], Dy[-1], "r*", label="end")
legend()
grid(True)

```



The orbits of the mass-spring system, $k/M=1.0$, $D=0.0$ by explicit midpoint with 50 steps per period



7.5 Error Control and Variable Step Sizes

References:

- Section 6.5 *Variable Step-Size Methods* in [Sauer, 2022].
- Section 5.5 *Error Control and the Runge-Kutta-Fehlberg Method* in [Burden *et al.*, 2016].
- Section 7.3 in [Chenney and Kincaid, 2012].

7.5.1 The Basic ODE Initial Value Problem

We consider again the initial value problem

$$\frac{du}{dt} = f(t, u) \quad a \leq t \leq b, \quad u(a) = u_0$$

We now allow the possibility that u and f are vector-valued as in the section on *Systems of ODEs and Higher Order ODEs*, but omitting the tilde notation $\tilde{u}$, $\tilde{f}$.

7.5.2 Error Control by Varying the Time Step Size h_i

Recall the variable step-size version of Euler's method:

Algorithm 6.5.1

Input: f , a , b , n

$t_0 = a$

$U_0 = u_0$

$h = (b - a)/n$

for i in $[0, n)$:

 Choose step size h_i somehow!

$t_{i+1} = t_i + h_i$

$U_{i+1} = U_i + h_i f(t_i, U_i)$

end

We now consider how to choose each step size, by estimating the error in each step, and aiming to have error per unit time below some limit like $\epsilon/(b - a)$, so that the global error is no more than about ϵ .

As usual, the theoretical error bounds like $O(h_i^2)$ for a single step of Euler's method are not enough for quantitative tasks like choosing h_i , but they do motivate more practical estimates.

7.5.3 A crude error estimate for Euler's Method: Richardson Extrapolation

Starting at a point $(t, u(t))$, we can estimate the error in Euler's method approximation at a slightly later time $t_i + h$ by using two approximations of $U(t + h)$:

- The value given by a step of Euler's method with step size h : call this U^h
- The value given by taking two steps of Euler's method each with step size $h/2$: call this $U_2^{h/2}$, because it involves 2 steps of size $h/2$.

The first order accuracy of Euler's method gives $e_h = u(t + h) - U^h \approx 2(u(t + h) - U_2^{h/2})$, so that

$$e_h \approx \frac{U_2^{h/2} - U^h}{2}$$

Step size choice

What do we do with this error information?

The first obvious ideas are:

- Accept this step if e_h is small enough, taking $h_i = h$, $t_{i+1} = t_i + h_i$, and $U_{i+1} = U^h$, but
- reject it and try again with a smaller h value otherwise; maybe halving h ; but there are more sophisticated options too.

Exercise A

Write a formula for U_h and e_h if one starts from the point (t_i, U_i) , so that $(t_i + h, U^h)$ is the proposed value for the next point (t_{i+1}, U_{i+1}) in the approximate solution — but only if e_h is small enough!

Error tolerance

One simple criterion for accuracy is that the estimated error in this step be no more than some overall upper limit on the error in each time step, T . That is, accept the step size h if

$$|e_h| \leq T$$

A crude approach to reducing the step size when needed

If this error tolerance is not met, we must choose a new step size h' , and we can predict roughly its error behavior using the known order nature of the error in Euler's method: scaling down to $h' = sh$, the error in a single step scales with h^2 (in general it scales with h^{p+1} for a method of order p), and so to reduce the error by the needed factor $\frac{e_h}{T}$ one needs approximately

$$s^2 = \frac{T}{|e_h|}$$

and so using $e_h \approx \tilde{e}_h = |U^{h/2} - U^h|$ suggests using

$$s = \left(\frac{T}{|U^{h/2} - U^h|} \right)^{1/2}$$

However this new step size might have error that is still slightly too large, leading to a second failure. Another is that one might get into an infinite loop of step size reduction.

So refinements of this choice must be considered.

Increasing the step size when desirable

If we simply follow the above approach, the step size, once reduced, will never be increased. This could lead to great inefficiency, through using an unnecessarily small step size just because at an earlier part of the time domain, accuracy required very small steps.

Thus, after a successful time step, one might consider increasing h for the next step. This could be done using exactly the above formula, but again there are risks, so again refinement of this choice must be considered.

One problem is that if the step size gets too large, the error estimate can become unreliable; another is that one might need some minimum “temporal resolution”, for nice graphs and such.

Both suggest imposing an upper limit on the step size h .

7.5.4 Another strategy for getting error estimates: two (related) Runge-Kutta methods

The recurring strategy of estimating errors by the difference of two different approximations — one expected to be far better than the other — can be used in a nice way here. I will first illustrate with the simplest version, using Euler’s Method and the Explicit Trapezoid Method.

Recall that the increment in Euler’s Method from time t to time $t + h$ is

$$K_1 = hf(t, U)$$

whereas for the Explicit Trapezoid Method it is $(K_1 + K_2)/2$, as given by

$$\begin{aligned} K_1 &= hf(t, U) \\ K_2 &= hf(t + h, U + K_1) \end{aligned}$$

Thus we can use the difference, $|K_1 - (K_1 + K_2)/2| = |(K_1 - K_2)/2|$ as an error estimate. In fact to be cautious, one often drops the factor of $1/2$, so using approximation $\tilde{e}_h = |K_1 - K_2|$.

One has to be careful: this estimates the error in Euler’s Method, and one has to use it that way: using the less accurate value K_1 as the update.

A basic algorithm for the time step starting with t_i, U_i is

Algorithm 6.5.2

$K_1 \leftarrow hf(t_i, U_i)$

$K_2 \leftarrow hf(t_i + h, U_i + K_1)$

$e_h \leftarrow |K_1 - K_2|$

$s \leftarrow \sqrt{T/e_h}$

if $e_h < T$

$U_{i+1} = U_i + K_1$

$t_{i+1} = t_i + h$

 Increase h for the *next* time step:

$h \leftarrow sh$

else: (not good enough: reduce h and try again)

$h \leftarrow sh$

Start again from $K_1 = \dots$
end

However, in practice one needs:

- An upper limit h_{max} on the step size h , partly because error estimates become unreliable if h gets too large, and also because subsequent use of the results (like graphs) might need sufficiently “fine” data.
- A lower limit h_{min} on h , to avoid infinite loops and such.
- Since we are using only an approximation $\tilde{e}_h$ of e_h , and out of general caution, it is typical to include a “safety factor” of about 0.8 or 0.9, when computing the *next* time step: reducing the step size scale factor to $S = 0.9\sqrt{T/e_h}$.

Incorporating these refinements:

Algorithm 6.5.3

```
 $K_1 = hf(t_i, U_i)$   
 $K_2 = hf(t_i + h, U_i + K_1)$   
 $e_h = |K_1 - K_2|$   
 $s = 0.9\sqrt{T/e_h}$   
if  $e_h < T$   
     $U_{i+1} = U_i + K_1$   
     $t_{i+1} = t_i + h$   
    Increase  $h$  for the next time step:  
     $h \leftarrow \min(0.9sh, h_{max})$   
else: (not good enough; reduce  $h$  and try again)  
     $h \leftarrow \max(0.9sh, h_{min})$   
    Start again from  $K_1 = \dots$   
end
```

Exercise B

Implement the above, and test on the two familiar examples

$$\begin{aligned} du/dt &= Ku \\ \text{and} \\ du/dt &= K(\cos(t) - u) - \sin(t) \end{aligned}$$

($K = 1$ is enough.)

Partial Solution to Exercise B

```

import numpy as np
from matplotlib.pyplot import figure, plot, title, grid

# Get a "stop watch"
# This gives the "wall clock" time in seconds since a reference moment, called "the_
epoch".
# (For macOS and UNIX, the epoch is the beginning of 1970.)
from time import time

def euler_error_control(f, a, b, u_0, errorTolerance=1e-3, h_min=1e-6, h_max=0.1,
steps_max=1000, demoMode=False):
    # Use Python lists rather than Numpy arrays; they are easier to "increment"
    # Initialize variables holding the current values of t and U
    steps = 0
    t_i = a
    U_i = u_0
    t = [t_i]
    U = [U_i]
    h = h_max # Start optimistically!
    while t_i < b and steps < steps_max:
        K_1 = h*f(t_i, U_i)
        K_2 = h*f(t_i + h/2, U_i + K_1/2)
        errorEstimate = abs(K_1 - K_2)
        s = 0.9 * np.sqrt(errorTolerance/errorEstimate)
        if errorEstimate <= errorTolerance: # Success!
            t_i += h
            U_i += K_1
            t.append(t_i)
            U.append(U_i)
            # Adjust step size up, but not too big
            h = min(s*h, h_max)
        else: # Inaccurate; reduce step size and try again
            h = max(s*h, h_min)
            if demoMode: print(f"{t_i=}: Decreasing step size to {h:0.3e} and trying_
again.")
            # A refinement not mentioned above; the next step should not overshoot t=b:
            if t_i + h > b:
                h = b - t_i
            steps += 1
    # Convert out to Numpy arrays, so that they can be used as input to numerical_
functions and such:
    t = np.array(t)
    U = np.array(U)
    return (t, U)
    # Note: if the step count ran out, this does not reach t=b, but at least it is_
correct as far as it goes

```

```

def f(t, u):
    return k*u

```

```

k = 1.
a = 1.
b = 3.

```

(continues on next page)

(continued from previous page)

```

u_0 = 2.
def u(t):
    """Note: the input "t" must be a numpy array or a number; not a Python list."""
    return u_0*np.exp(k*(t-a))

```

```

errorTolerance = 1e-2
time_start = time()
(t, U) = euler_error_control(f, a, b, u_0, errorTolerance, demoMode=True)
time_end = time()
time_elapsed = time_end - time_start

steps = len(U) - 1
h_ave = (b-a)/steps
U_exact = u(t)
U_error = U-U_exact
U_max = max(abs(U_error))
print()
print(f"With {errorTolerance=}, this took {steps} time steps, of average length {h_ave:0.3}")
print(f"The maximum absolute error is {U_max:0.3}")
print(f"The maximum absolute error per time step is {U_max/steps:0.3}")
print(f"The time taken to solve was {time_elapsed:0.3} seconds")

figure(figsize=[14,5])
title(f"Solution to du/dt={k}u, u({a})={u_0}")
plot(t, U, ".:")
grid(True)

figure(figsize=[14,5])
title(f"Error in the above")
plot(t, U_error, ".:")
grid(True);

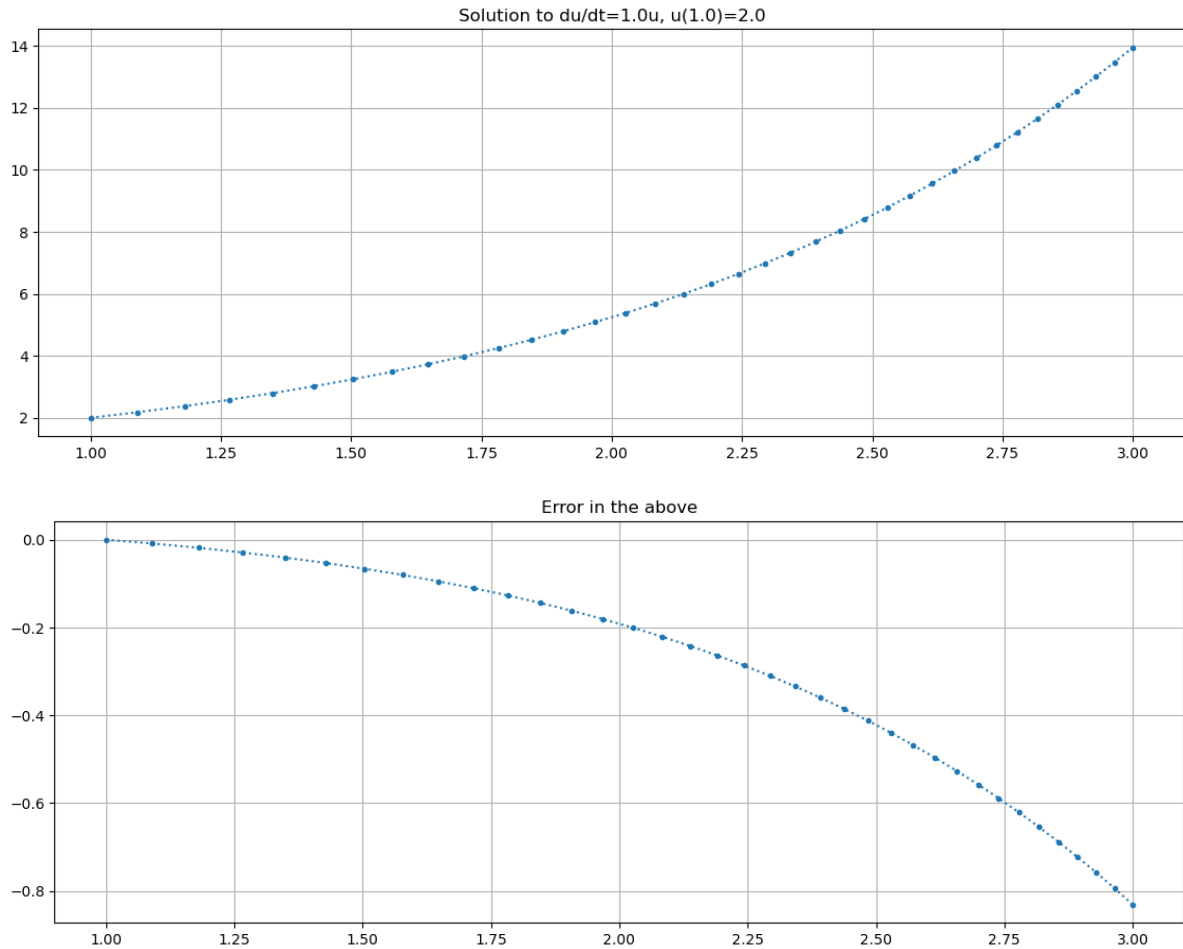
```

t_i=1.0: Decreasing step size to 9.000e-02 and trying again.

```

With errorTolerance=0.01, this took 36 time steps, of average length 0.0556
The maximum absolute error is 0.831
The maximum absolute error per time step is 0.0231
The time taken to solve was 0.000254 seconds

```



```

errorTolerance = 1e-3
time_start = time()
(t, U) = euler_error_control(f, a, b, u_0, errorTolerance, demoMode=True)
time_end = time()
time_elapsed = time_end - time_start

steps = len(U) - 1
h_ave = (b-a)/steps
U_exact = u(t)
U_error = U-U_exact
U_max = max(abs(U_error))
print()
print(f"With {errorTolerance=}, this took {steps} time steps, of average length {h_ave:0.3}")
print(f"The maximum absolute error is {U_max:0.3}")
print(f"The maximum absolute error per time step is {U_max/steps:0.3}")
print(f"The time taken to solve was {time_elapsed:0.3} seconds")

figure(figsize=[14,5])
title(f"Solution to du/dt={k}u, u({a})={u_0}")
plot(t, U, ".:")
grid(True)

figure(figsize=[14,5])

```

(continues on next page)

(continued from previous page)

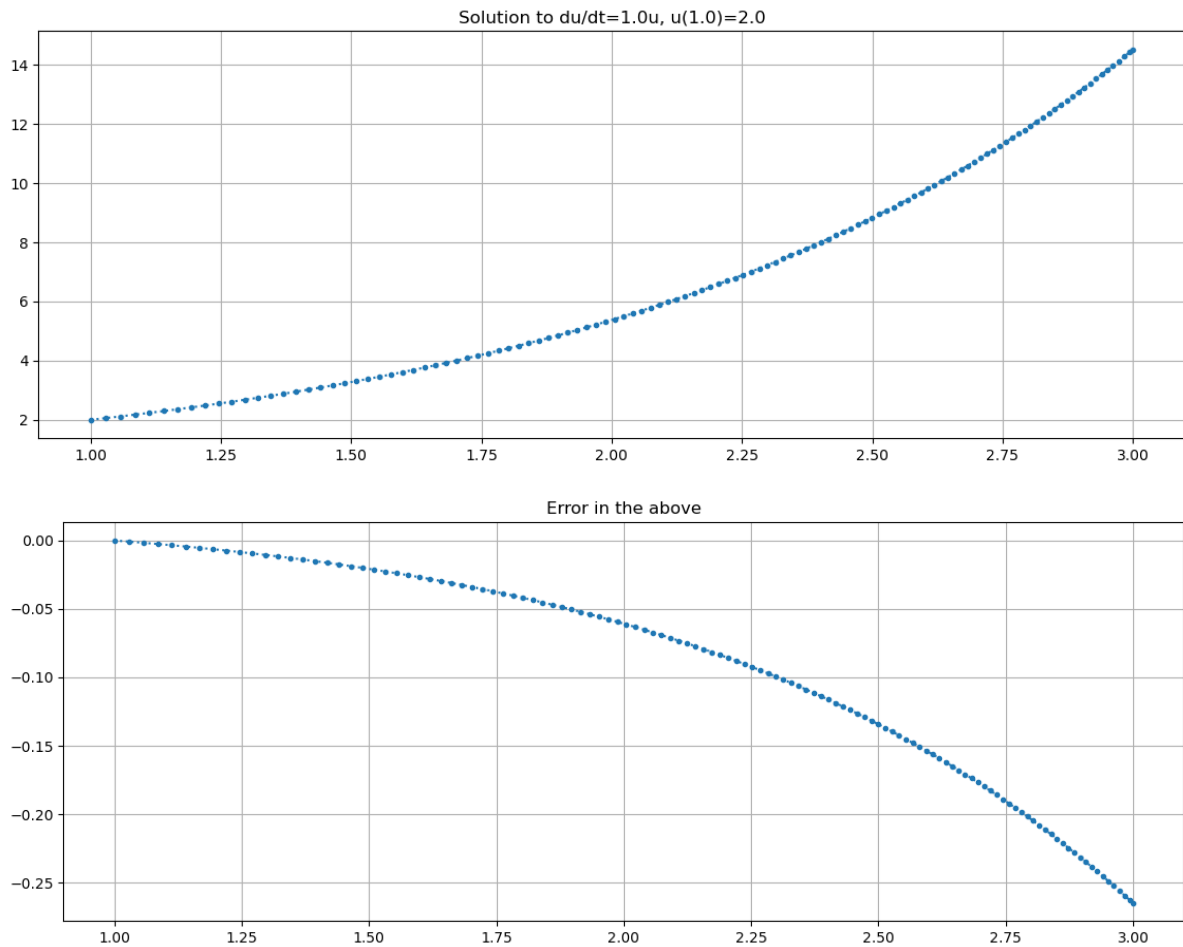
```

title(f"Error in the above")
plot(t, U_error, ".:")
grid(True);

```

t_i=1.0: Decreasing step size to 2.846e-02 and trying again.

With errorTolerance=0.001, this took 119 time steps, of average length 0.0168
 The maximum absolute error is 0.265
 The maximum absolute error per time step is 0.00223
 The time taken to solve was 0.000433 seconds



```

errorTolerance = 1e-4
time_start = time()
(t, U) = euler_error_control(f, a, b, u_0, errorTolerance, demoMode=True)
time_end = time()
time_elapsed = time_end - time_start

steps = len(U) - 1
h_ave = (b-a)/steps
U_exact = u(t)
U_error = U-U_exact

```

(continues on next page)

(continued from previous page)

```

U_max = max(abs(U_error))
print()
print(f"With {errorTolerance=}, this took {steps} time steps, of average length {h_
ave:0.3}")
print(f"The maximum absolute error is {U_max:0.3}")
print(f"The maximum absolute error per time step is {U_max/steps:0.3}")
print(f"The time taken to solve was {time_elapsed:0.3} seconds")

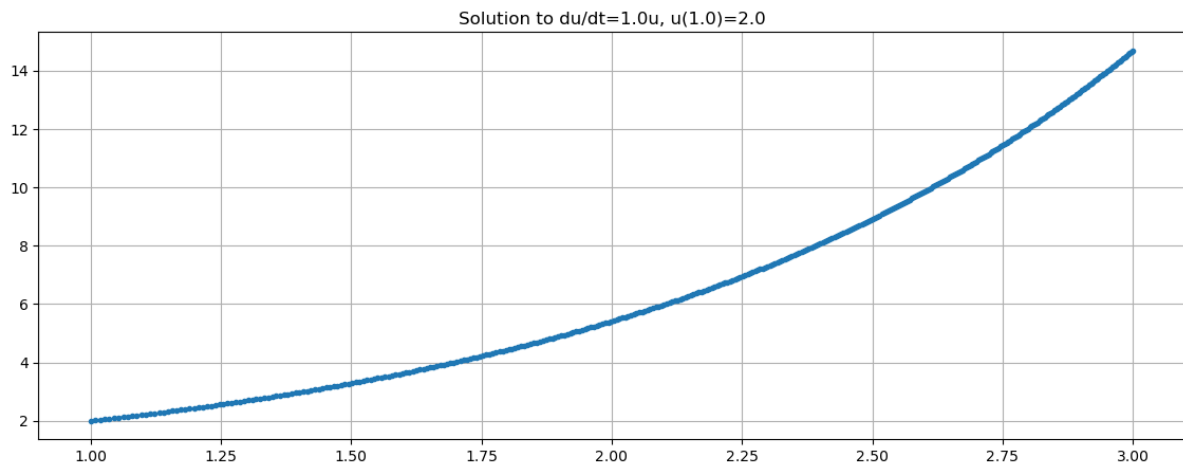
figure(figsize=[14,5])
title(f"Solution to du/dt={k}u, u({a})={u_0}")
plot(t, U, ".:")
grid(True)

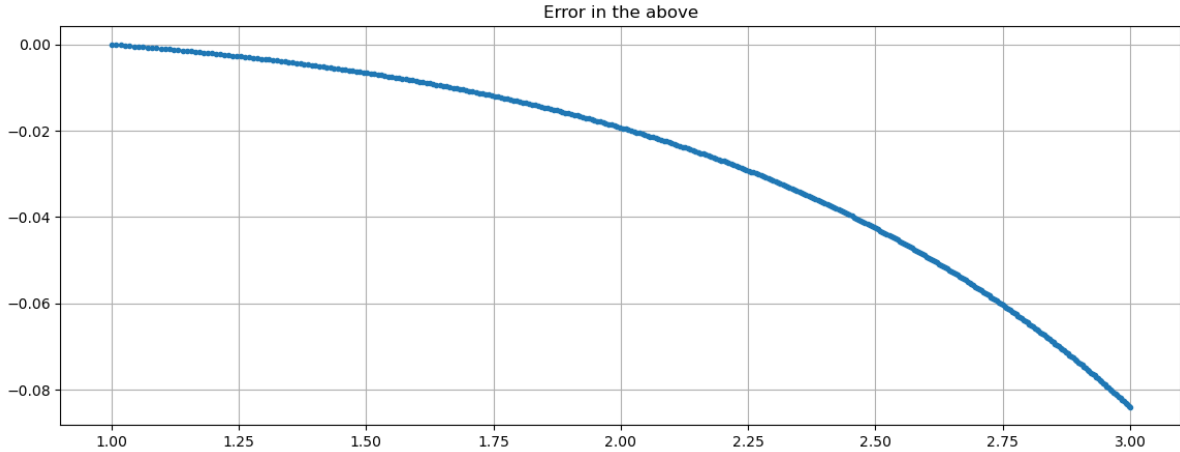
figure(figsize=[14,5])
title(f"Error in the above")
plot(t, U_error, ".:")
grid(True);

```

t_i=1.0: Decreasing step size to 9.000e-03 and trying again.

With errorTolerance=0.0001, this took 380 time steps, of average length 0.00526
 The maximum absolute error is 0.084
 The maximum absolute error per time step is 0.000221
 The time taken to solve was 0.00127 seconds





7.5.5 The explicit trapezoid method with error control

In practice, one usually needs at least second order accuracy, and one approach to that is using computing a “candidates” for the next time step with a second order accurate Runge-Kutta method and also a third order accurate one, the latter used only to get an error estimate for the former.

Perhaps the simplest of these is based on adding error estimation to the Explicit Trapezoid Rule. Omitting the step size adjustment for now, the main ingredients are:

Algorithm 6.5.4

$$K_1 = hf(t, U)$$

$$K_2 = hf(t + h, U + K_1)$$

(So far, as for the explicit trapezoid method)

$$K_3 = hf(t + h/2, U + (K_1 + K_2)/4)$$

(a midpoint approximation, using the above)

$$\delta_2 = (K_1 + K_2)/2$$

(The order 2 increment as for the explicit trapezoid method)

$$\delta_3 = (K_1 + 4K_3 + K_2)/6$$

(An order 3 increment — note the resemblance to Simpson’s Rule for integration. This is only used to get the final error estimate below)

$$e_h = |\delta_2 - \delta_3|, = |K_1 - 2K_3 + K_2|/3$$

Again, if this step is accepted, one uses the explicit trapezoid rule step: $U_{i+1} = U_i + \delta_2$.

Step size adjustment

The scale factor s for step size adjustment must be modified for a method order p (with $p = 2$ now):

- Changing step size by a factor s will change the error e_h in a single time step by a factor of about s^{p+1} .
- Thus, we want a new step with this rescaled error of about $s^{p+1}e_h$ roughly matching the tolerance T . Equating would give $s^{p+1}e_h = T$, so $s = (T/e_h)^{1/(p+1)}$, but as noted above, since we are using only an approximation $\tilde{e}_h$ of e_h it is typical to include a “safety factor” of about 0.9, so something like

$$s = 0.9 \left(\frac{T}{|\tilde{e}_h|} \right)^{1/(p+1)}$$

Thus for this second order accurate method, we then get

$$s = 0.9 \left(\frac{3T}{|K_1 - 2K_3 + K_2|} \right)^{1/3}$$

A variant: relative error control

One final refinement: it is more common in software to impose a *relative error* bound: aiming for $|e_h/u(t)| \leq T$, or $|e_h| \leq T|u(t)|$. Approximating $u(t)$ by U_i , this changes the step size rescaling guideline to

$$s = 0.9 \left| \frac{TU_i}{\tilde{e}_h} \right|^{1/(p+1)}$$

Exercise C

Implement *The explicit trapezoid method with error control*, and test on the two familiar examples

$$\begin{aligned} du/dt &= Ku \\ \text{and} \\ du/dt &= K(\cos(t) - u) - \sin(t) \end{aligned}$$

($K = 1$ is enough.)

7.5.6 Fourth order accurate methods with error control: Runge-Kutta-Felberg and some newer refinements

The details involve some messy coefficients; see the references above for those.

The basic idea is to devise a fifth order accurate Runge-Kutta method such that we can also get a fourth order accurate method from the same collection of *stages* K_i values. One catch is that any such fifth order method requires six stages (not five as you might have guessed).

The first such method, still widely used, is the [Runge-Kutta-Felberg Method](#) published by Erwin Fehlberg in 1970:

Algorithm 6.5.5 (Runge-Kutta-Fehlberg)

$$\begin{aligned} K_1 &= hf(t, U) \\ K_2 &= f(t + \tfrac{1}{4}h, U + K_1/4) \\ K_3 &= f(t + \tfrac{3}{8}h, U + \tfrac{3}{32}K_1 + \tfrac{9}{32}K_2) \\ K_4 &= f(t + \tfrac{12}{13}h, U + \tfrac{1932}{2197}K_1 - \tfrac{7200}{2197}K_2 + \tfrac{7296}{2197}K_3) \end{aligned}$$

$$K_5 = f(t + h, U + \frac{439}{216}K_1 - 8K_2 + \frac{3680}{2565}K_3 - \frac{845}{4104}K_4)$$

$$K_6 = f(t + \frac{1}{2}h, U - \frac{8}{27}K_1 + 2K_2 - \frac{3544}{513}K_3 + \frac{1859}{4104}K_4 - \frac{11}{40}K_5)$$

$$\delta_4 = \frac{25}{216}K_1 + \frac{1408}{2565}K_3 + \frac{2197}{4104}K_4 - \frac{1}{5}K_5$$

(The order 4 increment that will actually be used)

$$\delta_5 = \frac{16}{135}K_1 + \frac{6656}{12825}K_3 + \frac{28561}{56430}K_4 - \frac{9}{50}K_5 + \frac{2}{55}K_6$$

(The order 5 increment, used only to get the following error estimate)

$$\tilde{e}_h = \frac{1}{360}K_1 - \frac{128}{4275}K_3 + \frac{2197}{75240}K_4 + \frac{1}{50}K_5 + \frac{2}{55}K_6$$

This method is typically used with the relative error control mentioned above, and since the order is $p = 4$, the recommended step-size rescaling factor is

$$s = 0.9 \left| \frac{TU_i}{\tilde{e}_h} \right|^{1/5}, = 0.9 \left| \frac{TU_i}{\frac{1}{360}K_1 - \frac{128}{4275}K_3 + \frac{2197}{75240}K_4 + \frac{1}{50}K_5 + \frac{2}{55}K_6} \right|^{1/5},$$

7.5.7 ODE solvers in Python package SciPy

Newer software often uses a variant called the [Dormand–Prince method](#) published in 1980; for example this is the default method in the module `scipy.integrate` within Python package SciPy. This is similar in form to “R-K-F”, but has somewhat smaller errors.

The basic usage is

```
ode_solution = scipy.integrate.solve_ivp(f, [a, b], y_0)
```

because the output is an “object” containing many items; the ones we need for now are `t` and `y`, extracted with

```
t = ode_solution.t
y = ode_solution.y
```

This default usage is synonymous with

```
ode_solution = scipy.integrate.solve_ivp(f, [a, b], y_0, method="RK45")
```

where “RK45” refers to the Dormand–Prince method. Other options include `method="RK23"`, which is second order accurate, and very similar to the above [The explicit trapezoid method with error control](#)

Notes:

- SciPy’s notation is $dy/dt = f(t, y)$, so the result is called `y`, not `u`
- The initial data `y_0` must be in a list or numpy array, even if it is a single number.
- The output `y` is a 2D array, even if it is a single equation rather than a system.
- This might output very few values; for more output times (for better graphs?), try something like

```
t_plot = np.linspace(a, b)
[t, y] = solve_ivp(f, [a, b], y_0, t_eval=t_plot)
```

Example

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Get an ODE IVP solve function, from module "integrate" within package "scipy"
from scipy.integrate import solve_ivp
```

To read more about this SciPy function `scipy.integrate.solve_ivp`, run the following help command in the notebook version of this section:

```
help(solve_ivp)
```

```
def f(t, u):
    return u
```

```
a = 0.0
b = 2.0
y_0 = [1.0]
```

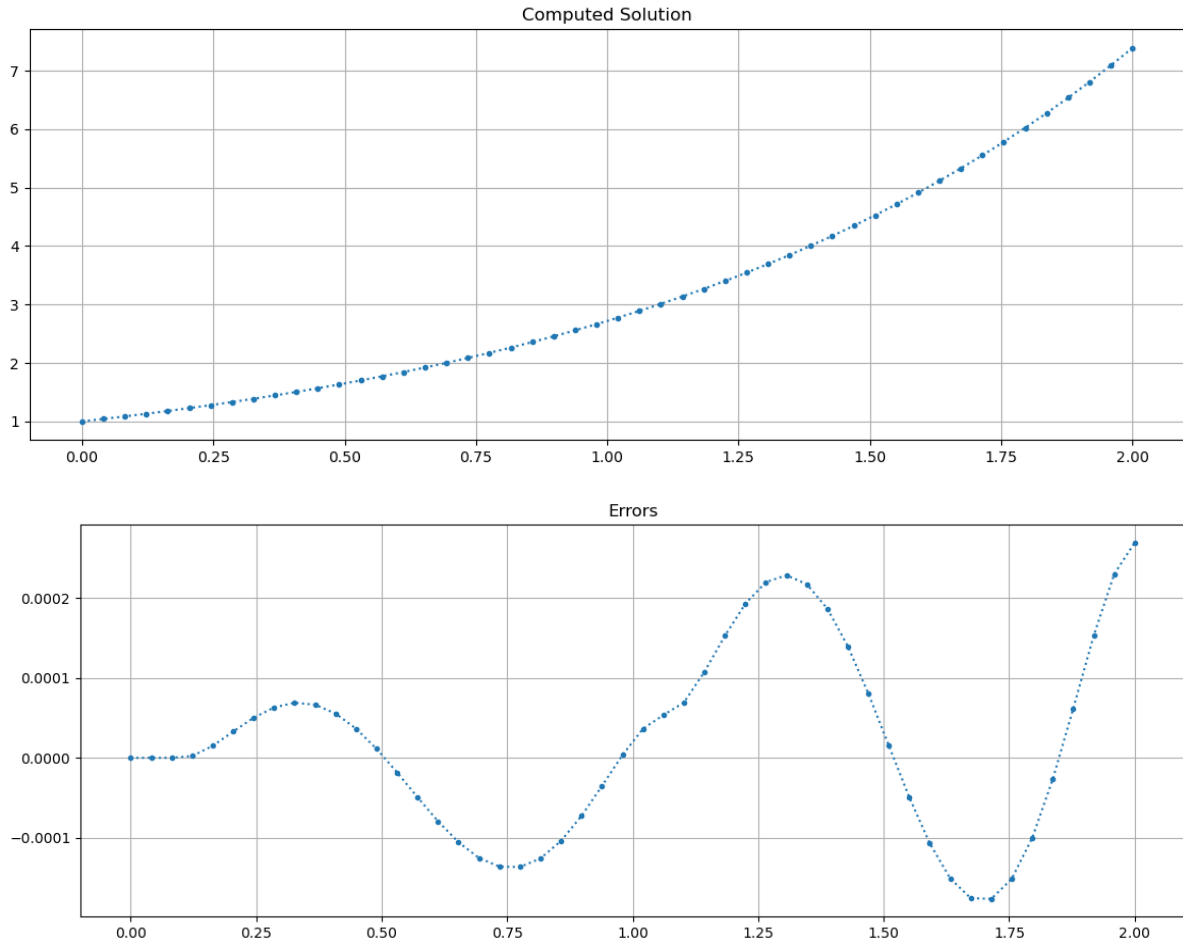
```
t_plot = np.linspace(a, b)
time_start = time()
ode_solution = solve_ivp(f, [a, b], y_0, t_eval=t_plot)
time_end = time()
time_elapsed = time_end - time_start
print(f"Time take to solve: {time_elapsed:0.3} seconds")

# The output is an "object" containing many items; the ones we need for now are t and y.
# More precisely "y" is a 2D array (just as y_0 is always an array)
# though with only a single row for this example, so we just want the 1D array y[0]
# These are extracted as follows:
t = ode_solution.t
y = ode_solution.y[0]

figure(figsize=[14,5])
plt.title("Computed Solution")
plt.plot(t, y, ".:")
grid(True)

y_exact = np.exp(t)
errors = y - y_exact
figure(figsize=[14,5])
plt.title("Errors")
plt.plot(t, errors, ".:")
grid(True);
```

```
Time take to solve: 0.00107 seconds
```



```
# Increase accuracy requirement.
# "rtol" is a relative error tolerance, defaulting to 1e-3
# "atol" is an absolute error tolerance, defaulting to 1e-3
# It solve to the less demanding of these two,
# so both must be specified to increase accuracy.

t_plot = np.linspace(a, b)

time_start = time()
ode_solution = solve_ivp(f, [a, b], y_0, t_eval=t_plot, rtol=1e-12, atol=1e-12)
time_end = time()
time_elapsed = time_end - time_start
print(f"Time take to solve: {time_elapsed:0.3} seconds")

t = ode_solution.t
y = ode_solution.y[0]
figure(figsize=[14,5])
plt.title("Computed Solution")
plt.plot(t, y, ".:")
grid(True)

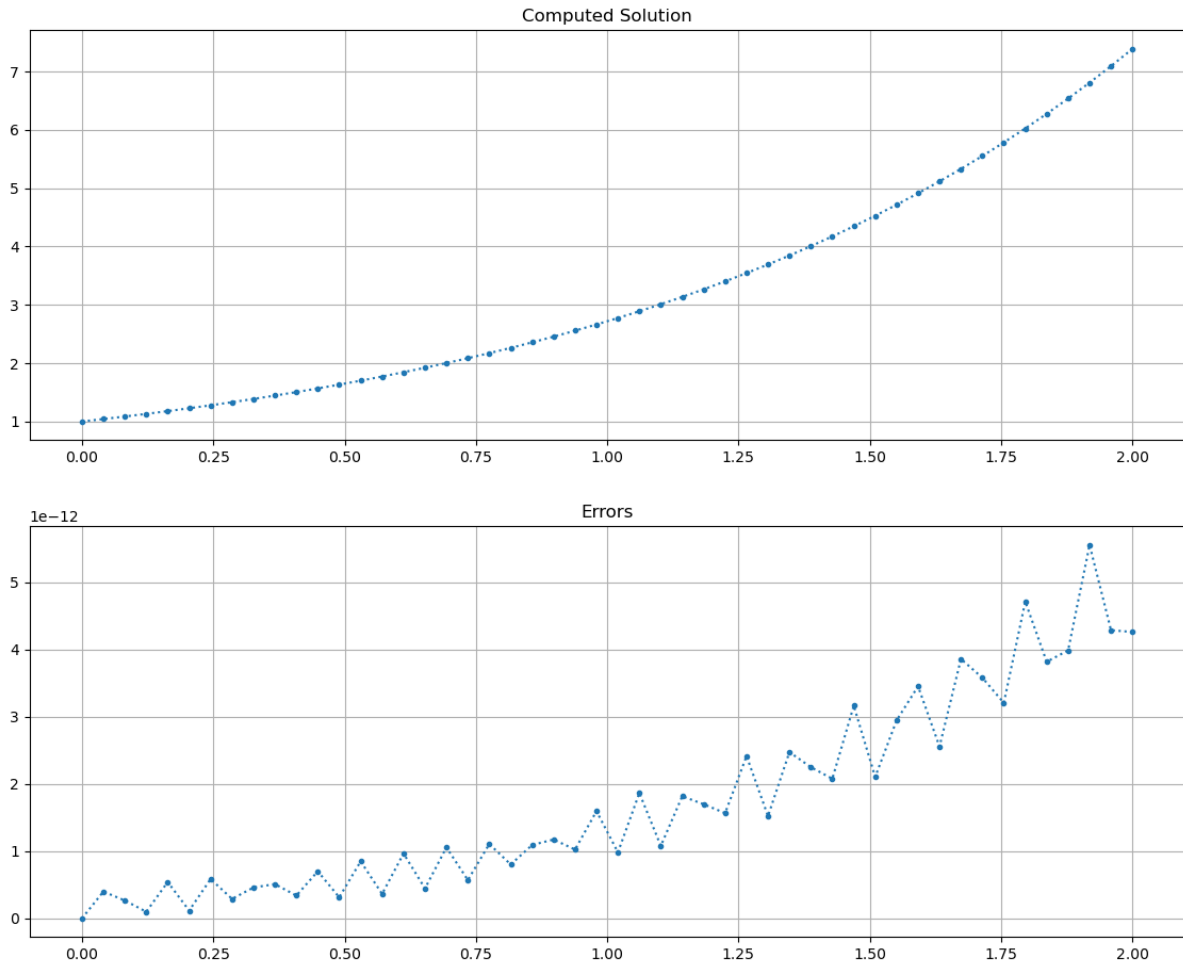
y_exact = np.exp(t)
errors = y - y_exact
figure(figsize=[14,5])
```

(continues on next page)

(continued from previous page)

```
plt.title("Errors")
plt.plot(t, errors, ".:")
grid(True);
```

Time take to solve: 0.00872 seconds



7.6 An Introduction to Multistep Methods: Leap-frog

References:

- Section 6.7 *Multistep Methods* of [Sauer, 2022].
- Section 5.6 *Multistep Methods* of [Burden *et al.*, 2016].

7.6.1 Introduction

When approximating derivatives we saw that there is a distinct advantage in accuracy to using the centered difference approximation

$$\frac{df}{dt}(t) \approx \delta_h f(t) := \frac{f(t+h) - f(t-h)}{2h}$$

(with error $O(h^2)$) over the forward difference approximation

$$\frac{df}{dt}(t) \approx \Delta_h f(t) := \frac{f(t+h) - f(t)}{h}$$

(which has error $O(h)$).

However Euler's method used the latter, and all ODE methods seen so far avoid using values at "previous" times like $t - h$. There is a reason for this, as using data from previous times introduces some complications, but sometimes those are worth overcoming, so let us look into this.

In this section, we look at one simple multi-step method, based on the above centered-difference derivative approximation.

Future sections will look at higher order methods such as the Adams-Bashforth and Adams-Moulton methods.

7.6.2 The Leap-frog method

Inserting the above centered difference approximation of the derivative into the ODE $du/dt = f(t, u)$ gives

$$\frac{u(t+h) - u(t-h)}{h} \approx f(t, u(t))$$

which leads to the *leap-frog method*

$$\frac{U_{i+1} - U_{i-1}}{2h} = f(t_i, U_i)$$

or

$$U_{i+1} = U_{i-1} + 2hf(t_i, U_i)$$

This is the first example of a

Definition 6.6.1 (Multistep Method)

A **multistep method** for numerical solution of an ODE IVP $du/dt = f(t, u)$, $u(t_0) = u_0$ is one of the form

$$U_{i+1} = \phi(U_i, \dots, U_{i-s+1}, h), \quad s > 1$$

so that the new approximate value of $u(t)$ depends on approximate values at multiple previous times.

More specifically, this is called an s -step method.

This jargon is consistent with all methods seen in earlier sections being called *one-step methods*. For example, Euler's method can be written as

$$U_{i+1} = \phi_E(U_i, h) := U_i + hf(t_i, U_i)$$

and the explicit midpoint method can be written as the one-liner

$$U_{i+1} = \phi_{EMP}(U_i, h) := U_i + hf(t_i + h/2, U_i + hf(t_i, U_i)/2)h$$

The leap-frog method already illustrates two of the complications that arise with multistep methods:

- The initial data $u(a) = u_0$ gives U_0 , but then the above formula gives U_1 in terms of U_0 and the non-existent value U_{-1} ; a different method is needed to get U_1 . More generally, with an s -step methods, one needs to compute the first $s - 1$ steps, up to U_{s-1} , by some other method.
- Leap-frog needs a constant step size h ; the strategy of error estimation and error control using variable step size is still possible with some multistep methods, but that is distinctly more complicated than we have seen with one-step methods, and is not addressed in these notes.

Fortunately, many differential equations can be handled well by choosing a suitable fixed step size h . Thus, in these notes we work only with equal step sizes, so that our times are $t_i = a + ih$ and we aim for approximations $U_i \approx u(a + ih)$.

7.6.3 Second order accuracy of the leap-frog method

Using the fact that the centered difference approximation is second order accurate, one can verify that

$$\frac{u(t_{i+1}) - u(t_{i-1}))}{2h} - f(t, u(t_i)) = O(h^2)$$

(Alternatively one can get this by inserting quadratic Taylor polynomials centered at t_i , and their error terms.)

The definition of local truncation error needs to be extended slightly: it is the error $U_{i+1} - u(t_{i+1})$ when one starts with exact values for all previous steps; that is, assuming $U_j = u(t_j)$ for all $j \leq i$.

The above results then shows that the local truncation error in each step is $U_{i+1} - u(t_{i+1}) = O(h^3)$, so that the “local truncation error per unit time” is

$$\frac{U_{i+1} - u(t_{i+1})}{h} = O(h^2).$$

says that when a one-step methods has local truncation error per unit time of $O(h^p)$ it also has global truncation error of the same order. The situation is a bit more complicated with multi-step methods, but loosely:

if the errors in a multistep method has local truncation $O(h^p)$ **and** it converges (i.e. the global error goes to zero at $h \rightarrow 0$) then it does so at the expected rate of $O(h^p)$.

But multi-step methods can fail to converge, even if the local truncation error is of high order! This is dealt with via the concept of **stability**; not considered here, but addressed in both references above, and a topic for future expansion of these notes.

In particular, when the leap-frog method converges it is second order accurate, just like the centered difference approximation of du/dt that it is built upon.

7.6.4 The speed advantage of multi-step methods like the leapfrog method

This second order accuracy illustrates a major potential advantage of multi-step methods: whereas any one-step Runge-Kutta method that is second order accurate (such as the explicit trapezoid or explicit midpoint methods) require at least two evaluations of $f(t, u)$ for each time step, the leapfrog methods requires only one.

More generally, for every s , there are s -step methods with errors $O(h^s)$ that require only one evaluation of $f(t, u)$ per time step — for example, the Adams-Bashforth methods, as seen at

- Section *Adams-Bashforth Multistep Methods*
- https://en.wikipedia.org/wiki/Linear_multistep_method#Adams-Bashforth_methods
- https://en.m.wikiversity.org/wiki/Adams-Bashforth_and_Adams-Moulton_methods
- [Sauer, 2022] Section 6.7.1 and 6.7.2
- [Burden *et al.*, 2016] Section 5.6

In comparison, any *explicit* one-step method order p require at least p evaluations of $f(t, u)$ per time step.

(See the *Implicit Methods: Adams-Moulton* for the distinction between explicit and implicit methods.)

```
import numpy as np
# Shortcuts for some favorite mathematical functions and numbers:
from numpy import sqrt, sin, cos, pi, exp
# Shortcuts for some favorite graphics commands:
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend, show
import numericalMethods as nm
```

```
def leapfrog(f, a, b, U_0, U_1, n):
    nUnknowns = len(U_0)
    t = np.linspace(a, b, n+1)
    u = np.zeros([n+1, nUnknowns])
    u[0] = U_0
    u[1] = U_1
    h = (b-a)/n
    for i in range(1, n):
        u[i+1] = u[i-1] + 2*h*f(t[i], u[i])
    return (t, u)
```

Demo with the mass-spring system

As seen in the section *Systems of ODEs and Higher Order ODEs* the Damped Mass-Spring Equation (6.4.1) is

$$M \frac{d^2 y}{dt^2} = -Ky - D \frac{dy}{dt}$$

with initial conditions

$$y(a) = y_0$$

$$\left. \frac{dy}{dt} \right|_{t=a} = v_0$$

with first-order system form

$$\frac{du_0}{dt} = u_1$$

$$\frac{du_1}{dt} = -\frac{K}{M}u_0 - \frac{D}{M}u_1$$

with initial conditions

$$u_0(a) = y_0$$

$$u_1(a) = v_0$$

The right-hand side f is given by

```
def f_mass_spring(t, u):
    return np.array([ u[1], -(K/M)*u[0] - (D/M)*u[1] ])
```

and the solutions seen in that section are given by function `y_mass_spring`

```
def y_mass_spring(t, t_0, u_0, K, M, D):
    (y_0, v_0) = u_0
    discriminant = D**2 - 4*K*M
```

(continues on next page)

(continued from previous page)

```

if discriminant < 0: # underdamped
    omega = np.sqrt(4*K*M - D**2)/(2*M)
    A = y_0
    B = (v_0 + y_0*D/(2*M))/omega
    return exp(-D/(2*M)*(t-t_0)) * (A*cos(omega*(t-t_0)) + B*sin(omega*(t-t_0)))
elif discriminant > 0: # overdamped
    Delta = sqrt(discriminant)
    lambda_plus = (-D + Delta)/(2*M)
    lambda_minus = (-D - Delta)/(2*M)
    A = M*(v_0 - lambda_minus * y_0)/Delta
    B = y_0 - A
    return A*exp(lambda_plus*(t-t_0)) + B*exp(lambda_minus*(t-t_0))
else:
    q = -D/(2*M)
    A = y_0
    B = v_0 - A * q
    return (A + B*t)*exp(q*(t-t_0))

```

which could alternatively be imported from module numericalMethods.

```

M = 1.
K = 1.
D = 0.
U_0 = [1., 0.]
a = 0.
periods = 4
b = 2 * pi * periods

# Note: In the notes on systems, the second order methods were tested with 50 is per_
↳period
#stepsperperiod = 50 # As for the second order accurate explicit trapezoid and_
↳midpoint methods
stepsperperiod = 100 # Equal cost per unit time as for the explicit trapezoid and_
↳midpoint and Runge-Kutta methods
n = stepsperperiod * periods

# We need U_1, and get it with the Runge-Kutta method;
# this is overkill for accuracy, but since only one step is needed, the time cost is_
↳negligible.
h = (b-a)/n
(t_1step, U_1step) = nm.rungekutta_system(f_mass_spring, a, a+h, U_0, 1)
U_1 = U_1step[-1]
(t, U) = leapfrog(f_mass_spring, a, b, U_0, U_1, n)

Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[14,7])
title(f"y and dy/dt with {K/M=}, {D=} by leap-frog with {periods} periods,
↳{stepsperperiod} steps per period")
plot(t, Y, ":", label="y")
plot(t, DY, ":", label="dy/dt")
legend()
xlabel("t")
grid(True)

```

(continues on next page)

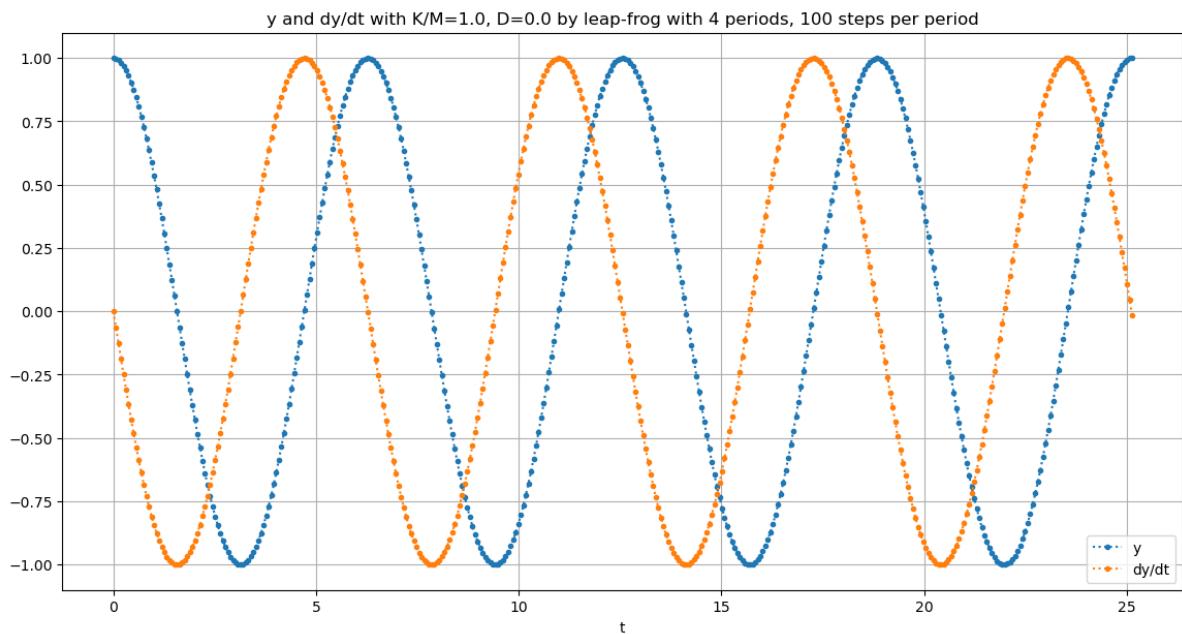
(continued from previous page)

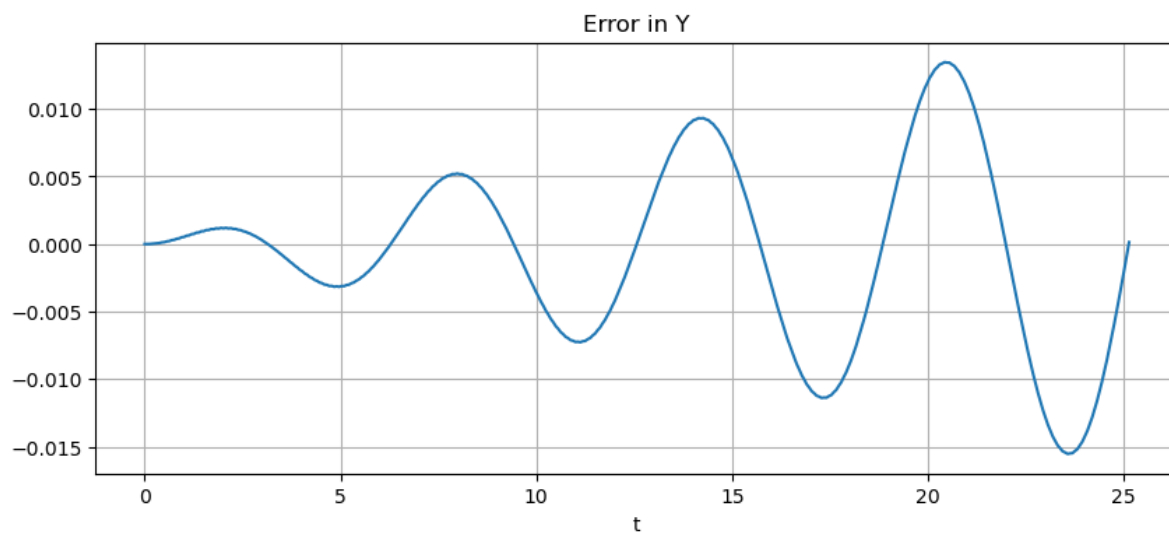
```

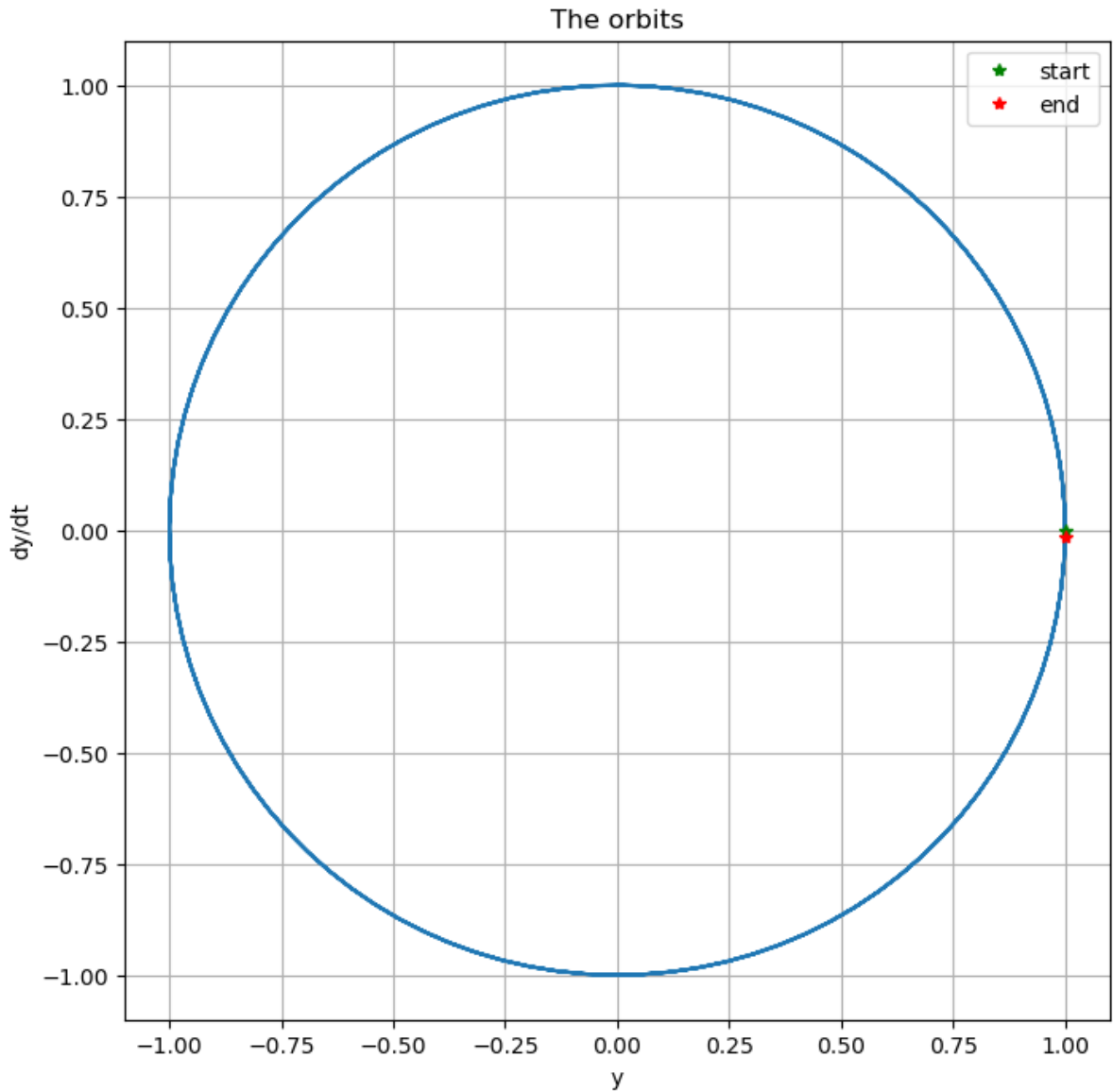
figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[8,8]) # Make axes equal length; orbits should be circular or
    ↪ "circular spirals"
title(f"The orbits")
plot(Y, DY)
xlabel("y")
ylabel("dy/dt")
plot(y[0], DY[0], "g*", label="start")
plot(y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```







```
D = 0.

periods = 32
b = 2 * pi * periods

# Note: In the notes on systems, the second order methods were tested with 50 steps.
#       ↳ per period
# stepsperperiod = 50 # As for the second order accurate explicit trapezoid and
#       ↳ midpoint methods
stepsperperiod = 100 # Equal cost per unit time as for the explicit trapezoid and
#       ↳ midpoint and Runge-Kutta methods
n = stepsperperiod * periods

# We need U_1, and get it with the Runge-Kutta method;
# this is overkill for accuracy, but since only one step is needed, the time cost is
#       ↳ negligible.
```

(continues on next page)

(continued from previous page)

```

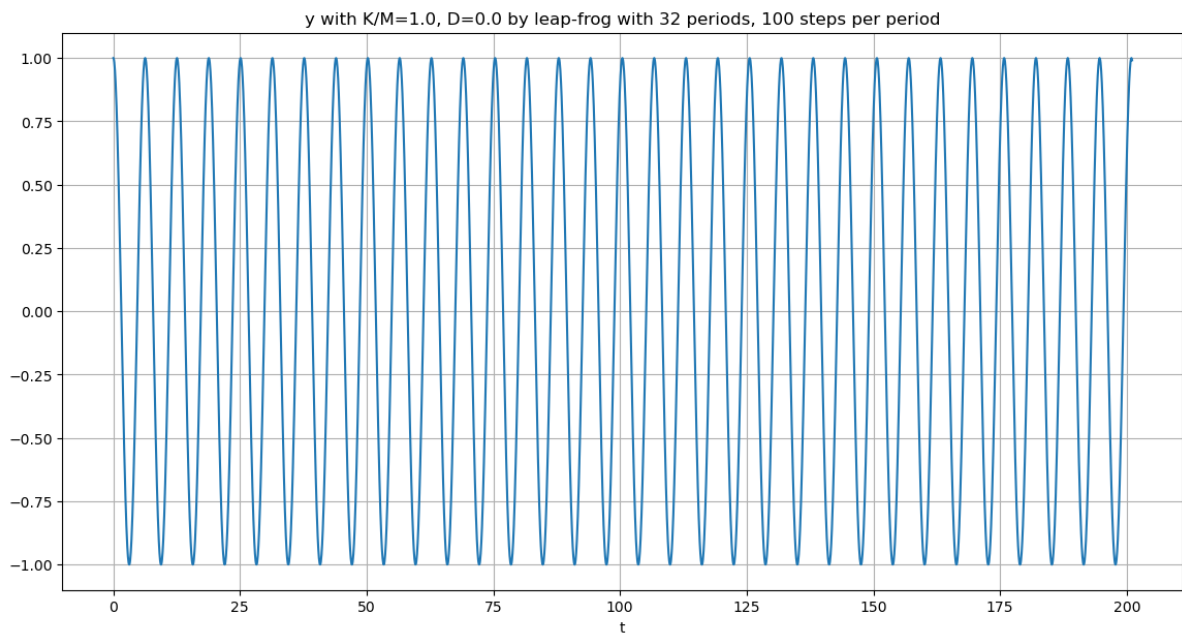
h = (b-a)/n
(t_1step, U_1step) = nm.rungekutta_system(f_mass_spring, a, a+h, U_0, 1)
U_1 = U_1step[-1]
(t, U) = leapfrog(f_mass_spring, a, b, U_0, U_1, n)

y = U[:,0]
Dy = U[:,1]

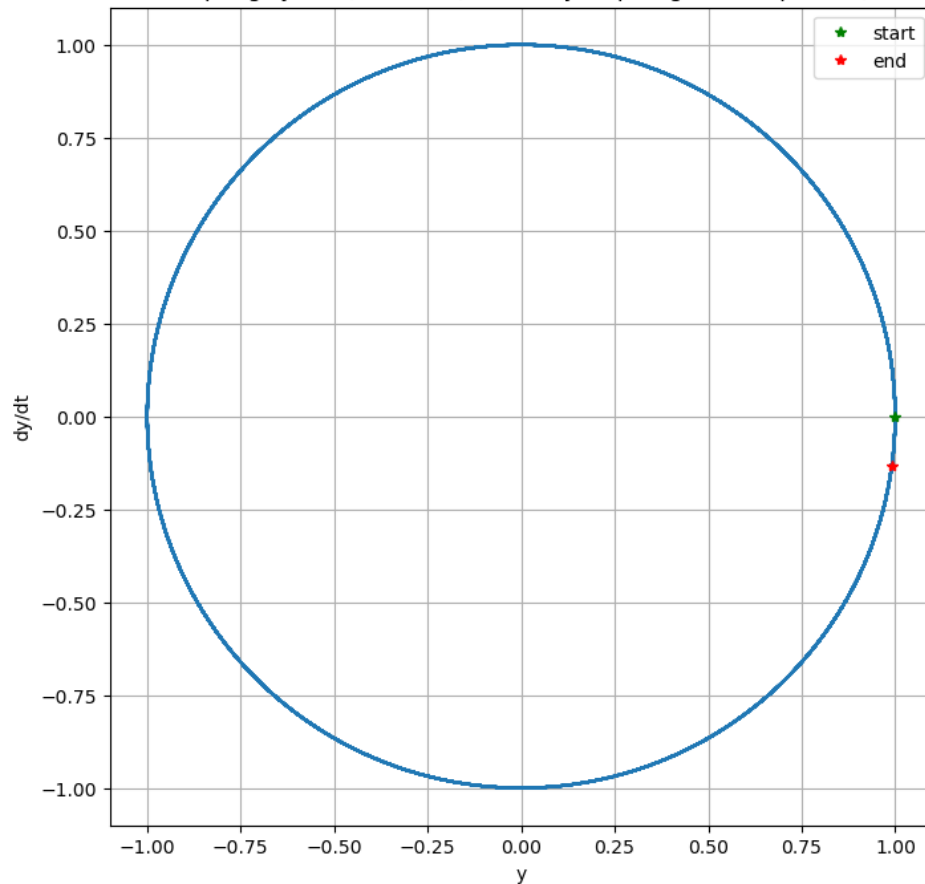
figure(figsize=[14,7])
title(f"y with {K/M=}, {D=} by leap-frog with {periods} periods, {stepsperperiod}↵
↵steps per period")
plot(t, y, label="y")
xlabel("t")
grid(True)

figure(figsize=[8,8]) # Make axes equal length; orbits should be circular or
↵"circular spirals"
title(f"The orbits of the mass-spring system, {K/M=}, {D=} by leap-frog with {periods}↵
↵periods, {stepsperperiod} steps per period")
plot(y, Dy)
xlabel("y")
ylabel("dy/dt")
plot(y[0], Dy[0], "g*", label="start")
plot(y[-1], Dy[-1], "r*", label="end")
legend()
grid(True)

```



The orbits of the mass-spring system, $K/M=1.0$, $D=0.0$ by leap-frog with 32 periods, 100 steps per period



But with damping, things eventually go wrong!

This is an example in **instability**: reducing the step-size only postpones the problem, but does not avoid it.

In future sections it will be seen that the leap-frog method is stable (and a good choice) for “conservative” systems like the undamped mass-spring system, but unstable otherwise, such as for the damped case.

```
D = 0.1

periods = 32
b = 2 * pi * periods

# Note: In the notes on systems, the second order methods were tested with 50 steps
# per period
# stepsperperiod = 50 # As for the second order accurate explicit trapezoid and
# midpoint methods
stepsperperiod = 100 # Equal cost per unit time as for the explicit trapezoid and
# midpoint and Runge-Kutta methods
n = stepsperperiod * periods

# We need U_1, and get it with the Runge-Kutta method;
# this is overkill for accuracy, but since only one step is needed, the time cost is
# negligible.
h = (b-a)/n
```

(continues on next page)

(continued from previous page)

```

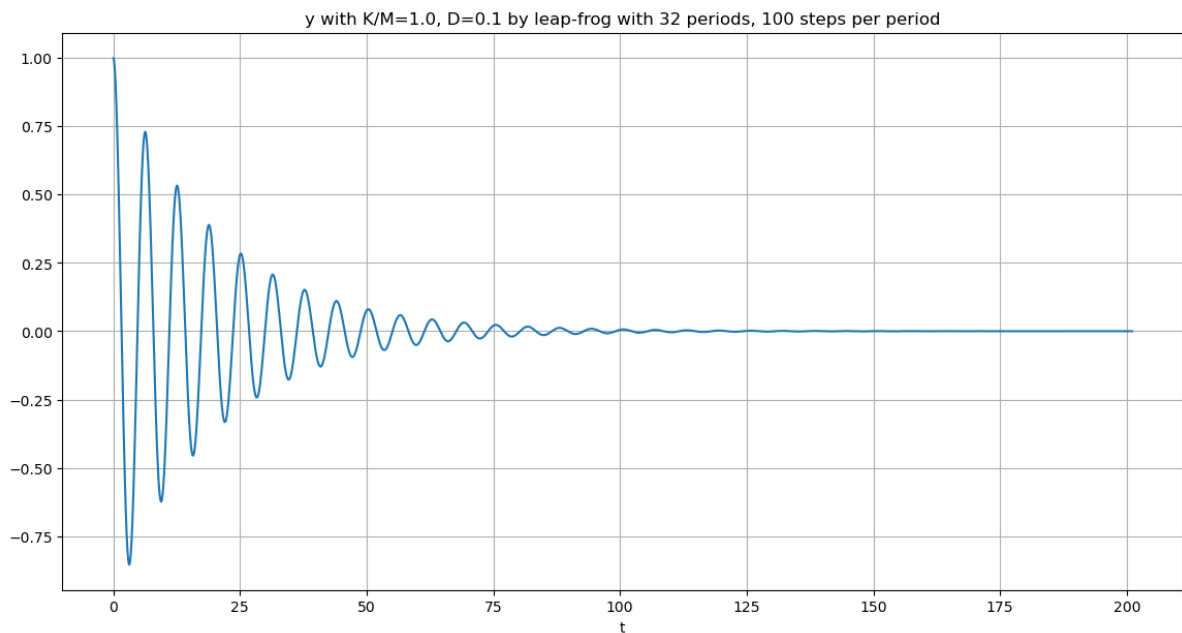
(t_1step, U_1step) = nm.rungekutta_system(f_mass_spring, a, a+h, U_0, 1)
U_1 = U_1step[-1]
(t, U) = leapfrog(f_mass_spring, a, b, U_0, U_1, n)

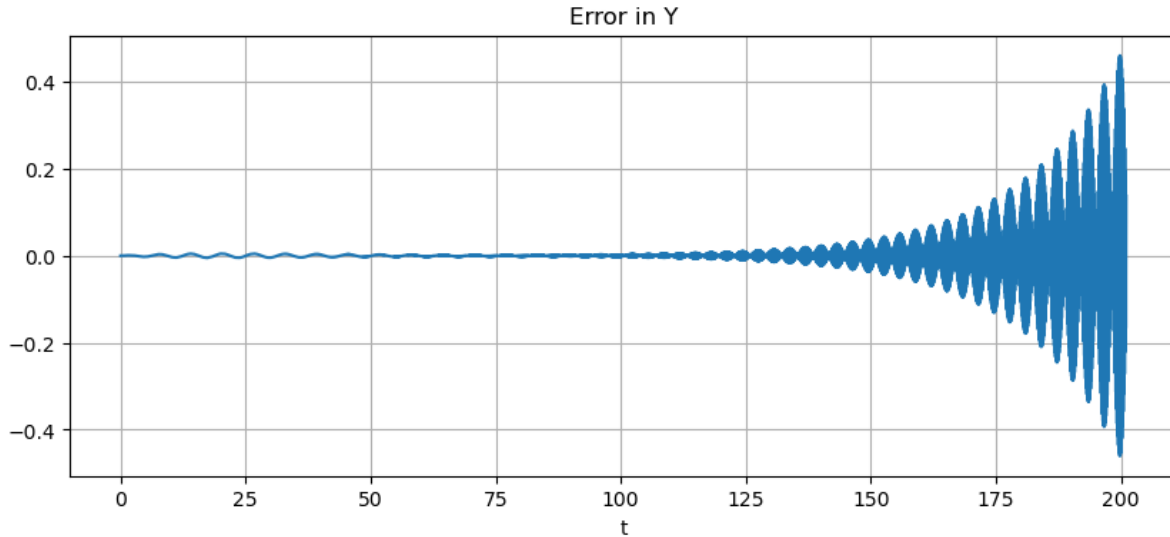
Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[14,7])
title(f"y with {K/M=}, {D=} by leap-frog with {periods} periods, {stepsperperiod}↵
↵steps per period")
plot(t, y, label="y")
xlabel("t")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

```





7.7 Adams-Bashforth Multistep Methods

References:

- Section 6.7 *Multistep Methods* in [Sauer, 2022].
- Section 5.6 *Multistep Methods* in [Burden *et al.*, 2016].

7.7.1 Introduction

Recall from *An Introduction to Multistep Methods: Leap-frog*:

Definition 6.7.1 (Multistep Method)

A **multistep method** for numerical solution of an ODE IVP $du/dt = f(t, u)$, $u(t_0) = u_0$ is one of the form

$$U_i = \phi(U_{i-1}, \dots, U_{i-s}, h), \quad s > 1$$

so that the new approximate value of $u(t)$ depends on approximate values at multiple previous times. (The shift of indexing to describe “present” in terms of “past” will be convenient here.)

This is called an s -step method: the Runge-Kutta family of methods are all one-step.

We will be more specifically interested in what are called **linear multistep methods**, where the function at right is a linear combination of value of $u(t)$ and $f(t, u(t))$.

So for now we look at

$$U_i = a_0 U_{i-s} + \dots + a_{s-1} U_{i-1} + h(b_0 f(t_{i-s}, U_{i-s}) + \dots + b_{s-1} f(t_{i-1}, U_{i-1}))$$

The **Adams-Bashforth** methods are a case of this with the only a_i term being $a_{s-1} = 1$:

$$U_i = U_{i-1} + h(b_0 f(t_{i-s}, U_{i-s}) + \dots + b_{s-1} f(t_{i-1}, U_{i-1}))$$

As will be verified later, the s -step version of this is accurate to order s , so one can get arbitrarily high order of accuracy by using enough steps.

Aside. The case $s = 1$ is Euler's method, now written as

$$U_i = U_{i-1} + hf(t_{i-1}, U_{i-1})$$

The Adams-Bashforth methods are probably the most commonly used **explicit, one-stage** multi-step methods; we will see more about the alternatives of **implicit** and **multi-stage** options in future sections. (Note that all Runge-Kutta methods (except Euler's) are multi-stage: the explicit trapezoid and midpoint methods are 2-stage; the classical Runge-Kutta method is 4-stage.)

The most basic **Adams-Bashforth** multi-step method is the 2-step method, which can be thought of this way:

1. Start with the two most recent values, $U_{i-1} \approx u(t_{i-1})$ and $U_{i-2} \approx u(t_{i-2})$
2. Use the derivative approximations $F_{i-1} := f(t_{i-1}, U_{i-1}) \approx u'(t_{i-1})$ and $F_{i-2} := f(t_{i-2}, U_{i-2}) \approx u'(t_{i-2})$ and linear extrapolation to "predict" the value of $u'(t_i - h/2)$; one gets: $u'(t_i - h/2) \approx \frac{3}{2}u'(t_{i-1}) - \frac{1}{2}u'(t_{i-2}) \approx \frac{3}{2}F_{i-1} - \frac{1}{2}F_{i-2}$
3. Use this in the centered difference approximation

$$\frac{u(t_i) - u(t_{i-1}))}{h} \approx u'(t_i - h/2)$$

to get

$$\frac{U_i - U_{i-1}}{h} \approx \frac{3}{2}F_{i-1} - \frac{1}{2}F_{i-2}$$

That is,

$$U_i = U_{i-1} + \frac{h}{2} (3F_{i-1} - F_{i-2}), = U_{i-1} + \frac{h}{2} (3f(t_{i-1}, U_{i-1}) - f(t_{i-2}, U_{i-2})) \quad (7.2)$$

Equivalently, one can

1. Find the collocating polynomial $p(t)$ through (t_{i-1}, F_{i-1}) and (t_{i-2}, F_{i-2}) [so just a line in this case]
2. Use this on the interval (t_{i-1}, t_i) (*extrapolation*) as an approximation of $u'(t) = f(t, u(t))$ in that interval.
3. Use

$$u(t_i) = u(t_{i-1}) + \int_{t_{i-1}}^{t_i} u'(\tau) d\tau \approx u(t_{i-1}) + \int_{t_{i-1}}^{t_i} p(\tau) d\tau,$$

where the latter integral is easy to evaluate exactly.

One does not actually do this in each case; it is enough to verify that the integral gives $(\frac{3}{2}F_{i-1} - \frac{1}{2}F_{i-2})h$.

See [Exercise A](#).

To code this algorithm, it is convenient to shift the indices, to get a formula for U_i . Also, note that what is $F_i = f(t_i, U_i)$ at one step is reused as $F_{i-1} = f(t_{i-1}, U_{i-1})$ at the next, so to avoid redundant evaluations of $f(t, u)$, these quantities should also be saved, at least till the next step:

$$U_i = U_{i-1} + \frac{h}{2} (3F_{i-1} - F_{i-2})$$

```
import numpy as np
# Shortcuts for some favorite mathematical functions and numbers:
from numpy import sqrt, sin, cos, pi, exp
# Shortcuts for some favorite graphics commands:
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend, show
```

```
#import numericalMethods as nm
```

```
from numericalMethods import rungekutta_system
```

```
def adamsbashforth2(f, a, b, U_0, U_1, n):
    n_unknowns = len(U_0)
    t = np.linspace(a, b, n+1)
    u = np.zeros([n+1, n_unknowns])
    u[0,:] = U_0
    u[1,:] = U_1
    F_i_2 = f(a, U_0) # F_0 to start when computing U_2
    h = (b-a)/n
    for i in range(n):
        F_i_1 = f(t[i], u[i,:])
        u[i+1,:] = u[i,:] + (3*F_i_1 - F_i_2) * (h/2)
        F_i_2 = F_i_1
    return (t, u)
```

Demonstrations with the mass-spring system

```
def f_mass_spring(t, u): return np.array([ u[1], -(K/M)*u[0] - (D/M)*u[1] ])
```

```
from numericalMethods import y_mass_spring
```

```
M = 1.0
K = 1.0
D = 0.0
y_0 = 1.0
v_0 = 0.0
U_0 = [y_0, v_0]
a = 0.0
periods = 4
b = 2*pi * periods

# Using the same time step size as with the leapfrog method in the previous section.
stepsperperiod = 100
n = int(stepsperperiod * periods)

# We need U_1, and get it with the Runge-Kutta method;
# this is overkill for accuracy, but since only one step is needed, the time cost is
# negligible.
h = (b-a)/n
(t_1step, U_1step) = rungekutta_system(f_mass_spring, a, a+h, U_0, 1)
U_1 = U_1step[-1,:]
(t, U) = adamsbashforth2(f_mass_spring, a, b, U_0, U_1, n)

Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 2-step Adams-Bashforth with $periods periods, {stepsperperiod}
      ↪ steps per period")
```

(continues on next page)

(continued from previous page)

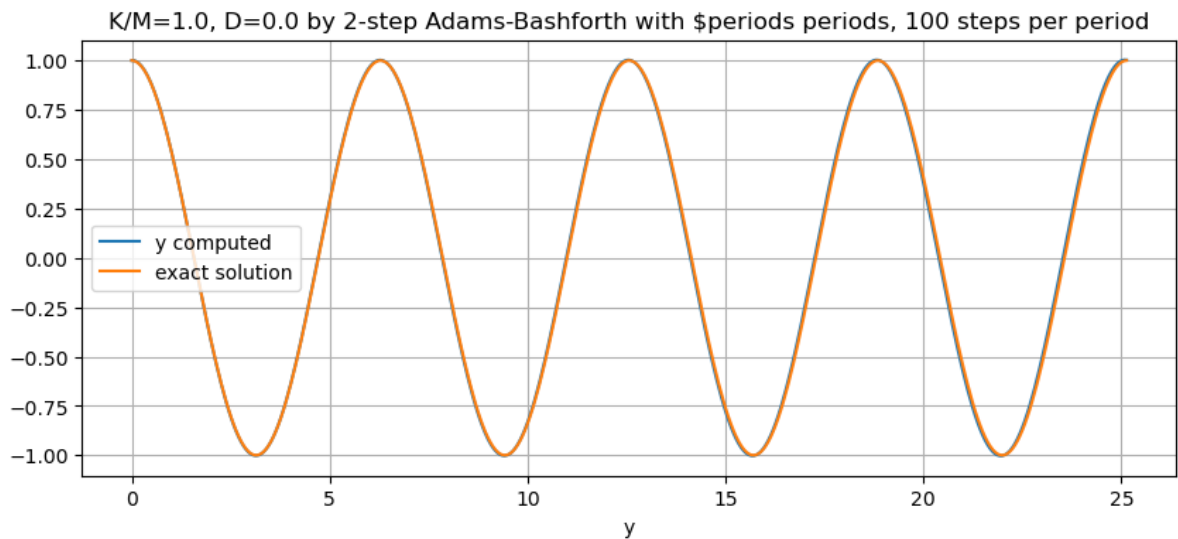
```

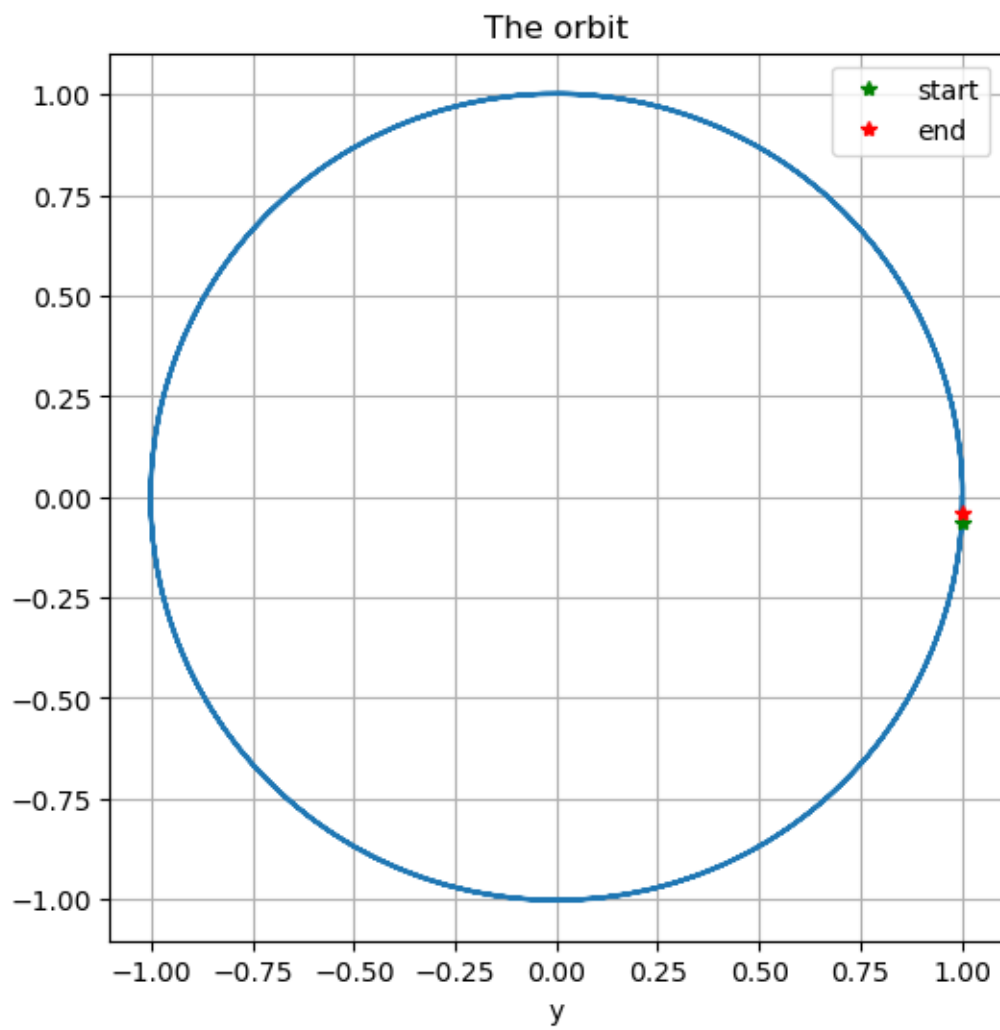
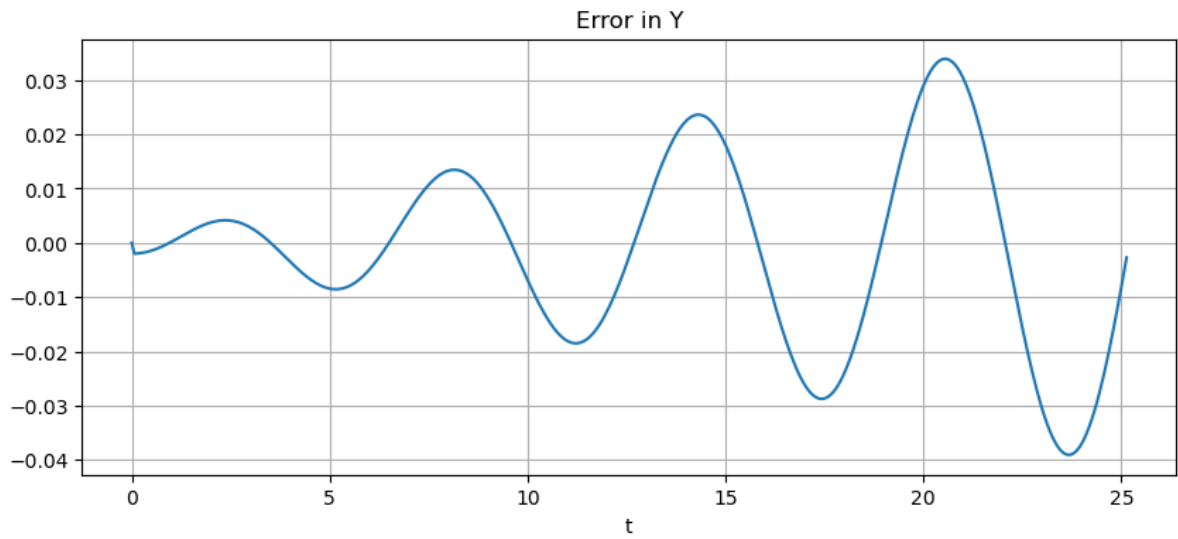
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
xlabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[6,6]) # Make axes equal length; orbits should be circular or
    ↪ "circular spirals"
title("The orbit")
plot(Y, DY)
xlabel("y")
plot(Y[1], DY[1], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```





```
D = 0.0
```

(continues on next page)

(continued from previous page)

```

periods = 16
b = 2*pi * periods

# Using the same time step size as with the leapfrog method in the previous section.
stepsperperiod = 100
n = int(stepsperperiod * periods)

# We need U_1, and get it with the Runge-Kutta method;
# this is overkill for accuracy, but since only one step is needed, the time cost is
↪negligible.
h = (b-a)/n
(t_1step, U_1step) = rungekutta_system(f_mass_spring, a, a+h, U_0, 1)
U_1 = U_1step[-1,:]
(t, U) = adamsbashforth2(f_mass_spring, a, b, U_0, U_1, n)

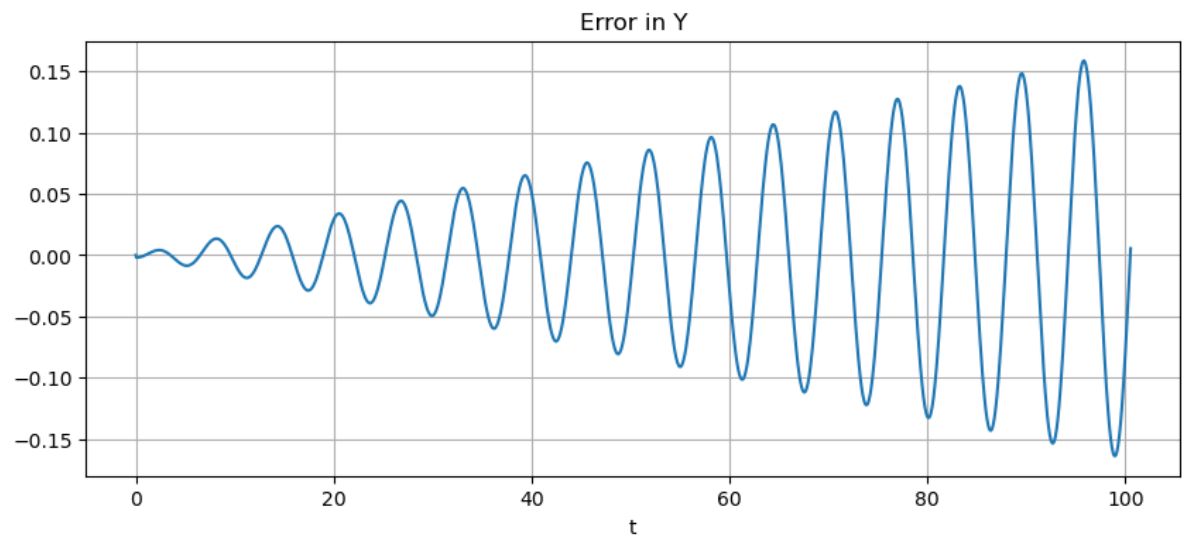
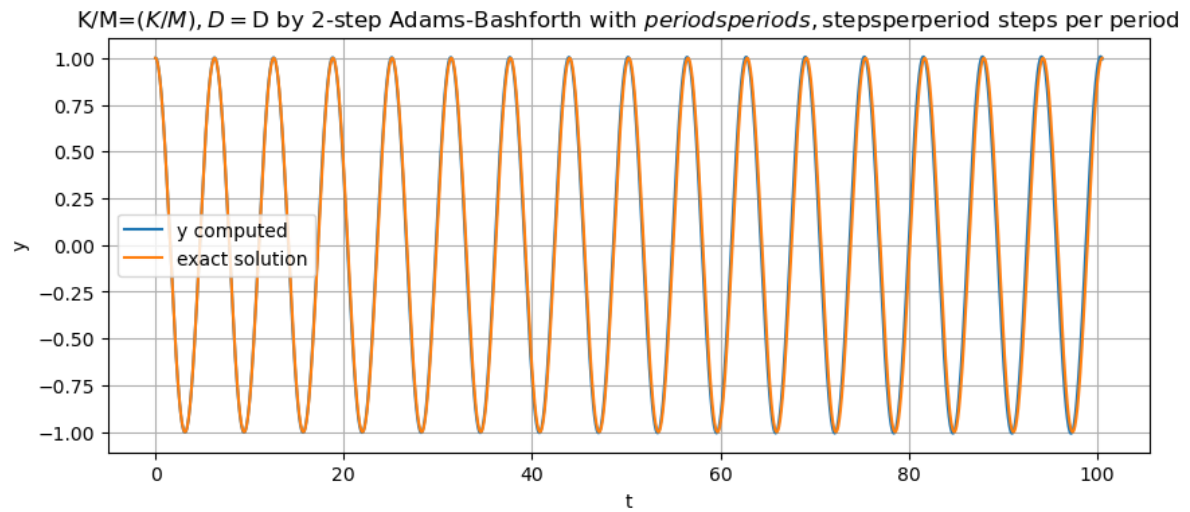
Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

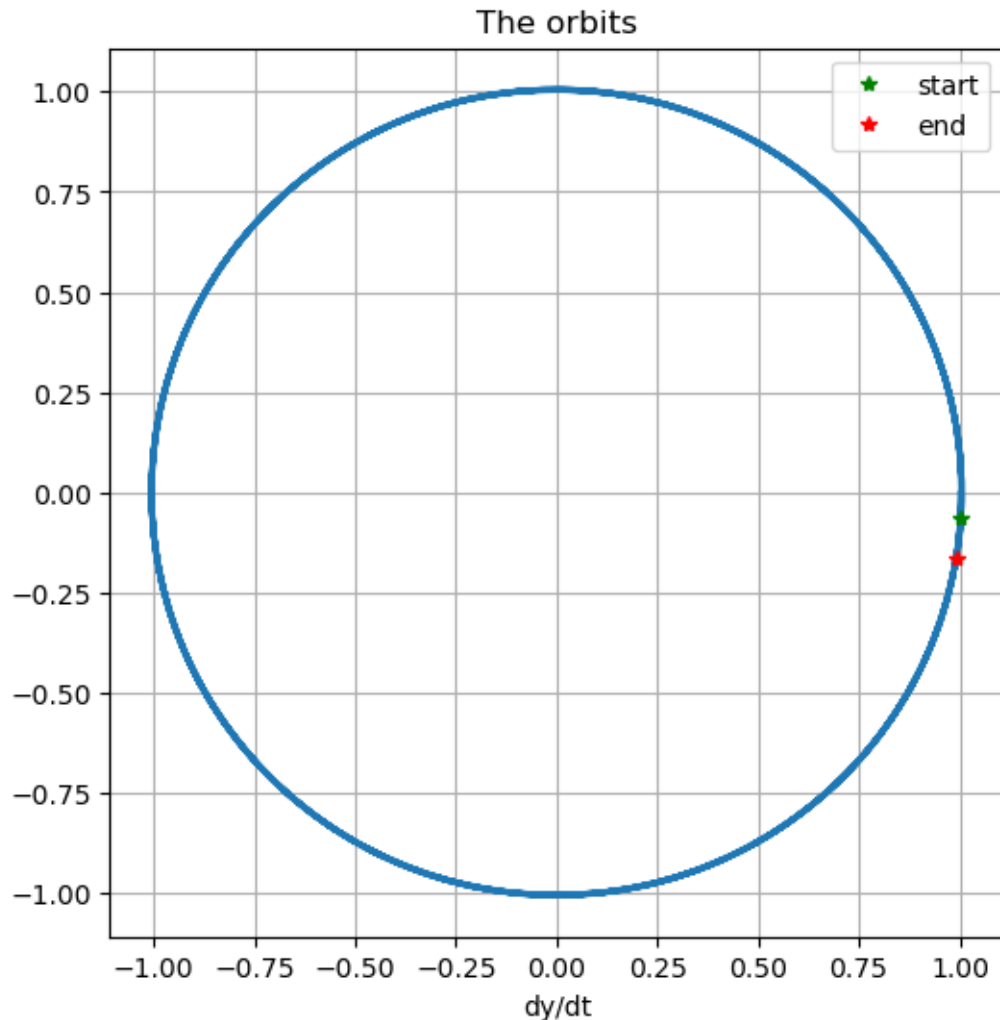
figure(figsize=[10,4])
title("K/M=$(K/M), D=$D by 2-step Adams-Bashforth with $periods periods,
↪$stepsperperiod steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[6,6]) # Make axes equal length; orbits should be circular or
↪"circular spirals"
title("The orbits")
plot(Y, DY)
xlabel("y")
xlabel("dy/dt")
plot(Y[1], DY[1], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```





In comparison to the (also second order accurate) leap-frog method, this is distinctly worse; the errors are more than twice as large, and the solution fails to stay on the circle; unlike leapfrog, the energy $E(t) = \frac{1}{2}(y^2(t) + Dy^2(t))$ is not conserved.

On the other hand ...

This time with damping, nothings goes wrong!

This is an example in **stability**; in future sections it will be seen that the the Adams-Bashforth methods are all stable for these equations for small enough step size h , and so converge to the correct solution as $h \rightarrow 0$.

Looking back, this suggests (correctly) that while the leapfrog method is well-suited to *conservative* equations, Adams-Bashforth methods are much preferable for more general equations.

```
D = 0.5

periods = 4
b = 2*pi * periods

# Using the same time step size as with the leapfrog method in the previous section.
```

(continues on next page)

(continued from previous page)

```

stepsperperiod = 100
n = int(stepsperperiod * periods)

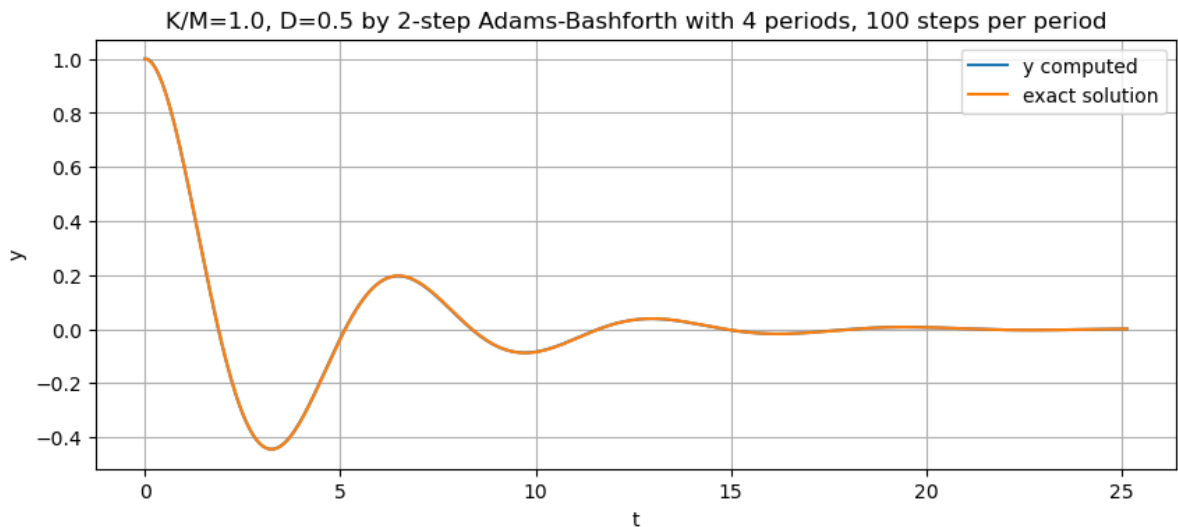
# We need U_1, and get it with the Runge-Kutta method;
# this is overkill for accuracy, but since only one step is needed, the time cost is
# negligible.
h = (b-a)/n
(t_1step, U_1step) = rungekutta_system(f_mass_spring, a, a+h, U_0, 1)
U_1 = U_1step[-1,:]
(t, U) = adamsbashforth2(f_mass_spring, a, b, U_0, U_1, n)

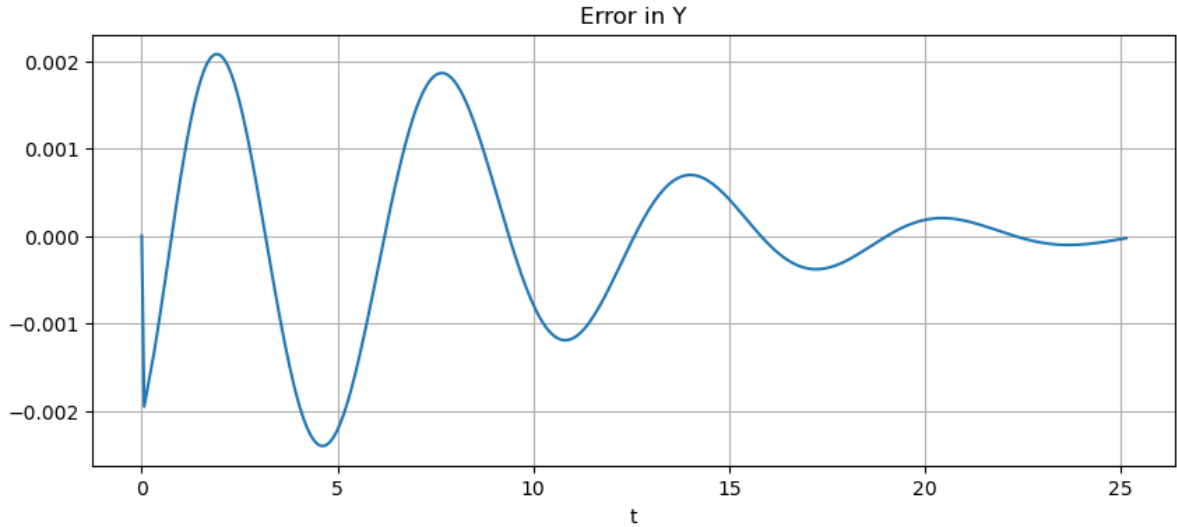
Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 2-step Adams-Bashforth with {periods} periods,
      {stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

```





7.7.2 Higher order Adams-Bashforth methods

The strategy described above of polynomial approximation, extrapolation, and integration can be generalized to get the s step Adams-Bashforth method, of order s ; to get the approximation U_i of $u(t_i)$ from data at the s most recent previous times t_{i-1} to t_{i-s} :

1. Find the collocating polynomial $p(t)$ of degree $s - 1$ through $(t_{i-1}, F_{i-1}) \dots (t_{i-s}, F_{i-s})$
2. Use this on the interval (t_{i-1}, t_i) (extrapolation) as an approximation of $u'(t) = f(t, u(t))$ in that interval.
3. Use $u(t_i) = u(t_{i-1}) + \int_{t_{i-1}}^{t_i} u'(\tau) d\tau \approx u(t_{i-1}) + \int_{t_{i-1}}^{t_i} p(\tau) d\tau$, where the latter integral can be evaluated exactly.

Again, one does not actually evaluate this integral; it is enough to verify that the resulting form will be

$$U_i = U_{i-1} + h(b_0 F_{i-s} + b_1 F_{i-s+1} + \dots b_{s-1} F_{i-1})$$

with the coefficients being the same for any $f(t, u)$ and any h .

In fact, the polynomial fitting and integration can be skipped: the coefficients can be derived by the method of undetermined coefficients as seen in [Approximating Derivatives by the Method of Undetermined Coefficients](#) and this also established that the local truncation error is $O(h^s)$:

- insert Taylor polynomial approximations of $u(t_{i-k}) = u(t_i) - kh$ and $f(t_{i-k}, u(t_{i-k})) = u'(t_{i-k}) = u'(t_i - kh)$ into $U_i = U_{i-1} + h(b_0 f(t_{i-s}, U_{i-s}) + \dots + b_{s-1} f(t_{i-1}, U_{i-1}))$
- solve for the s coefficients $b_0 \dots b_{s-1}$ that give the highest power for the residual error: the terms in the first s powers of h (from $h^0 = 1$ to h^{s-1}) can be cancelled, leaving an error $O(h^s)$.

The first few Adams-Bashforth formulas are:

- $s = 1$: $b_0 = 1$, $U_i = U_{i-1} + hF_{i-1} = U_{i-1} + hf(t_{i-1}, U_{i-1})$ (Euler's method)
- $s = 2$: $b_0 = -1/2$, $b_1 = 3/2$, $U_i = U_{i-1} + \frac{h}{2} (3F_{i-1} - F_{i-2})$ (as above)
- $s = 3$: $b_0 = 5/12$, $b_1 = -16/12$, $b_2 = 23/12$, $U_i = U_{i-1} + \frac{h}{12} (23F_{i-1} - 16F_{i-2} + 5F_{i-3})$
- $s = 4$: $b_0 = -9/24$, $b_1 = 37/24$, $b_2 = -59/24$, $b_3 = 55/24$, $U_i = U_{i-1} + \frac{h}{24} (55F_{i-1} - 59F_{i-2} + 37F_{i-3} - 9F_{i-4})$

```

def adamsbashforth3(f, a, b, U_0, U_1, U_2, n):
    n_unknowns = len(U_0)
    h = (b-a)/n
    t = np.linspace(a, b, n+1)
    u = np.zeros([n+1, n_unknowns])
    u[0,:] = U_0
    u[1,:] = U_1
    u[2,:] = U_2
    F_i_3 = f(a, U_0) # F_0 to start when computing U_3
    F_i_2 = f(a+h, U_1) # F_1 to start when computing U_3
    for i in range(2,n):
        F_i_1 = f(t[i], u[i,:])
        u[i+1,:] = u[i,:] + (23*F_i_1 - 16*F_i_2 + 5*F_i_3) * (h/12)
        (F_i_2, F_i_3) = (F_i_1, F_i_2)
    return (t, u)

D = 0.0

periods = 16
b = 2*pi * periods

# Using the same time step size as for leapfrog method in the previous section.
stepsperperiod = 100
n = int(stepsperperiod * periods)

# We need U_1 and U_2, and get them with the Runge-Kutta method;
# this is overkill for accuracy, but since only two steps are needed, the time cost
# is negligible.
h = (b-a)/n
(t_2step, U_2step) = rungekutta_system(f_mass_spring, a, a+2*h, U_0, 2)
U_1 = U_2step[1,:]
U_2 = U_2step[2,:]
(t, U) = adamsbashforth3(f_mass_spring, a, b, U_0, U_1, U_2, n)

Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 3-step Adams-Bashforth with {periods} periods,
      {stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[6,6]) # Make axes equal length; orbits should be circular or
# "circular spirals"

```

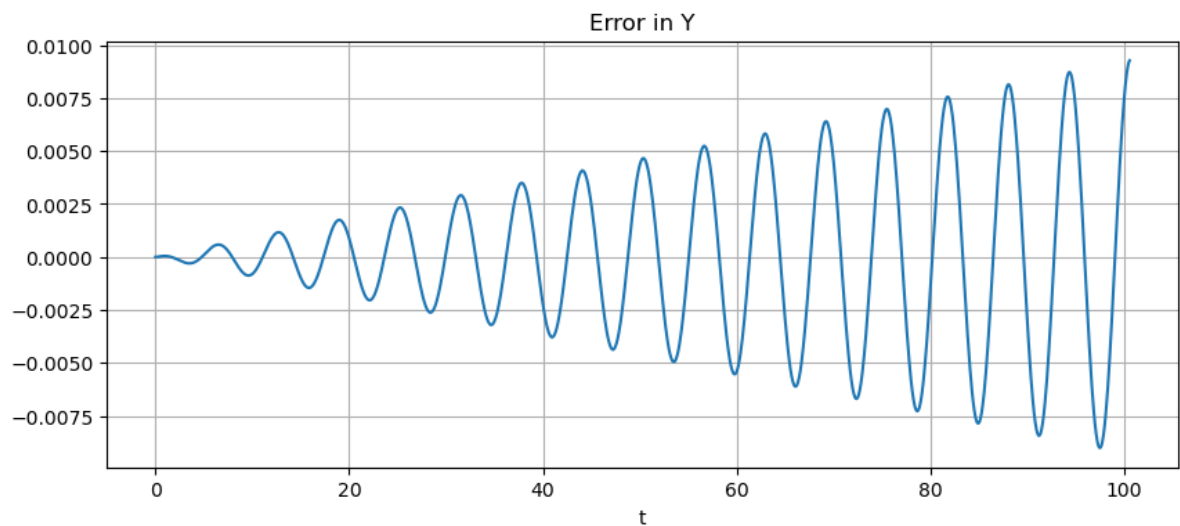
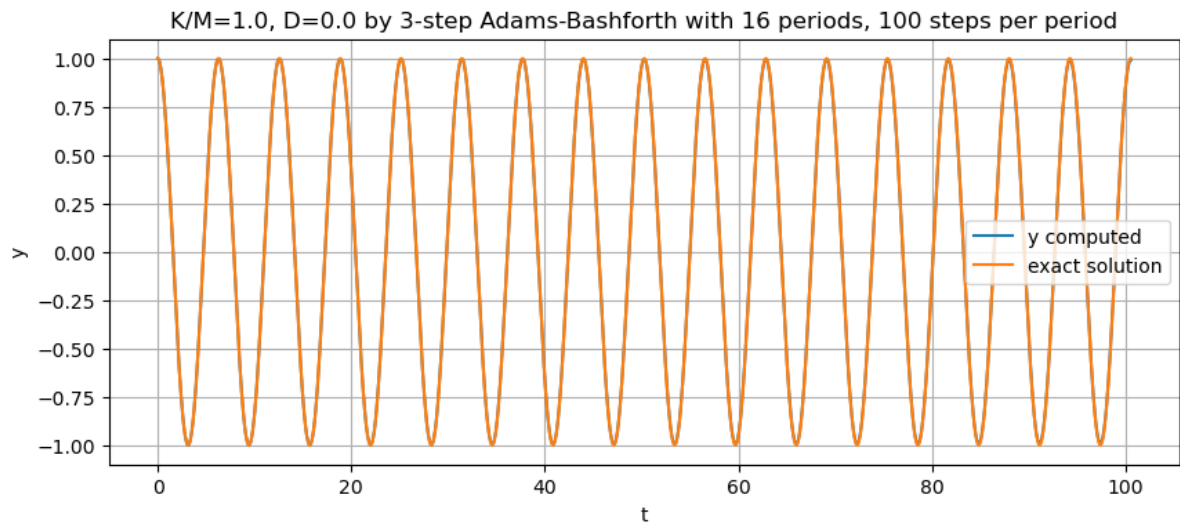
(continues on next page)

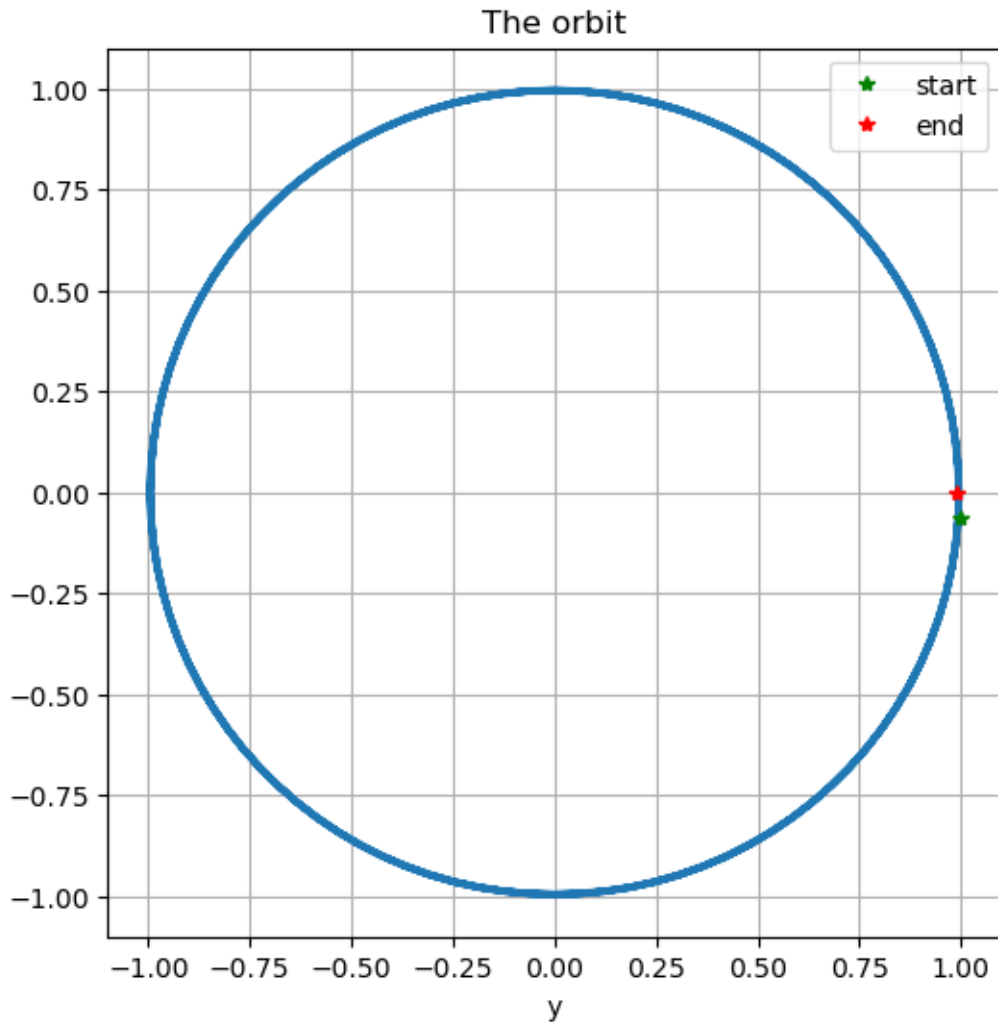
(continued from previous page)

```

title("The orbit")
plot(Y, DY)
xlabel("y")
plot(Y[1], DY[1], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```





Comparing to the leap-frog method, this higher order method at last has smaller errors (and they can be got even smaller by increasing the number of steps) but the leapfrog method is still better at keeping the solutions on the circle.

```
D = 0.5

periods = 4
b = 2*pi * periods

# Note: In the notes on systems, the second order Runge-Kutta methods were tested
# with 50 steps per period
# stepsperperiod = 50 # As for the second order accurate explicit trapezoid and
# midpoint methods
stepsperperiod = 100 # Equal cost per unit time as for the explicit trapezoid and
# midpoint and Runge-Kutta methods
n = int(stepsperperiod * periods)

# We need U_1 and U_2, and get them with the Runge-Kutta method;
# this is overkill for accuracy, but since only two steps are needed, the time cost
# is negligible.
h = (b-a)/n
(t_2step, U_2step) = rungekutta_system(f_mass_spring, a, a+2*h, U_0, 2)
```

(continues on next page)

(continued from previous page)

```

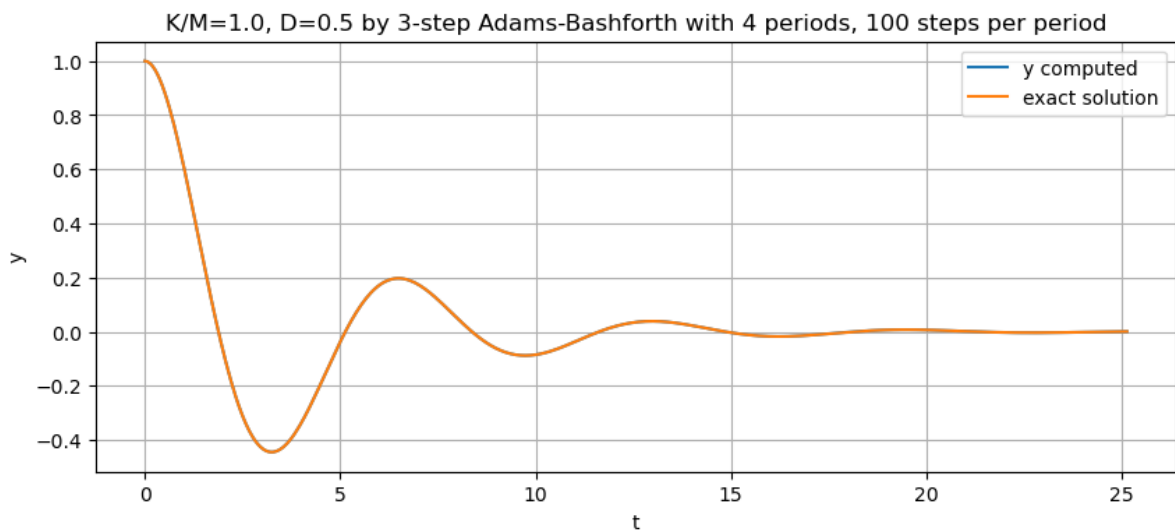
U_1 = U_2step[1,:]
U_2 = U_2step[2,:]
(t, U) = adamsbashforth3(f_mass_spring, a, b, U_0, U_1, U_2, n)

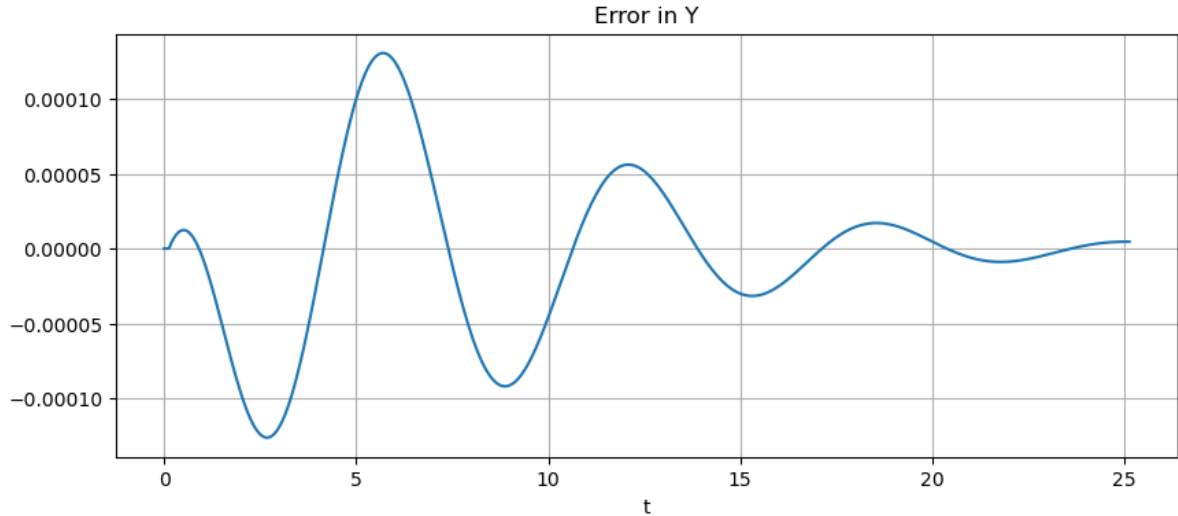
Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 3-step Adams-Bashforth with {periods} periods,
      ↪{stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

```





The fourth-order, four step method does at last appear to surpass leap-frog on the conservative case:

```
def adamsbashforth4(f, a, b, U_0, U_1, U_2, U_3, n):
    n_unknowns = len(U_0)
    h = (b-a)/n
    t = np.linspace(a, b, n+1)
    u = np.zeros([n+1, n_unknowns])
    u[0,:] = U_0
    u[1,:] = U_1
    u[2,:] = U_2
    u[3,:] = U_3
    F_i_4 = f(a, U_0) # F_0 to start when computing U_4
    F_i_3 = f(a+h, U_1) # F_1 to start when computing U_4
    F_i_2 = f(a+2*h, U_2) # F_1 to start when computing U_4
    h = (b-a)/n
    for i in range(3,n):
        F_i_1 = f(t[i], u[i,:])
        u[i+1,:] = u[i,:] + (55*F_i_1 - 59*F_i_2 + 37*F_i_3 - 9*F_i_4) * (h/24)
        (F_i_2, F_i_3, F_i_4) = (F_i_1, F_i_2, F_i_3)
    return (t, u)
```

```
D = 0.0

periods = 16
b = 2*pi * periods

# Using the same time step size as for leapfrog method in the previous section.
stepsperperiod = 100
n = int(stepsperperiod * periods)

# We need U_1, U_2 and U_3, and get them with the Runge-Kutta method;
# this is overkill for accuracy, but since only three steps are needed, the time cost
# is negligible.
h = (b-a)/n
(t_3step, U_3step) = rungekutta_system(f_mass_spring, a, a+3*h, U_0, 3)
U_1 = U_3step[1,:]
U_2 = U_3step[2,:]
U_3 = U_3step[3,:]
```

(continues on next page)

(continued from previous page)

```

(t, U) = adamsbashforth4(f_mass_spring, a, b, U_0, U_1, U_2, U_3, n)

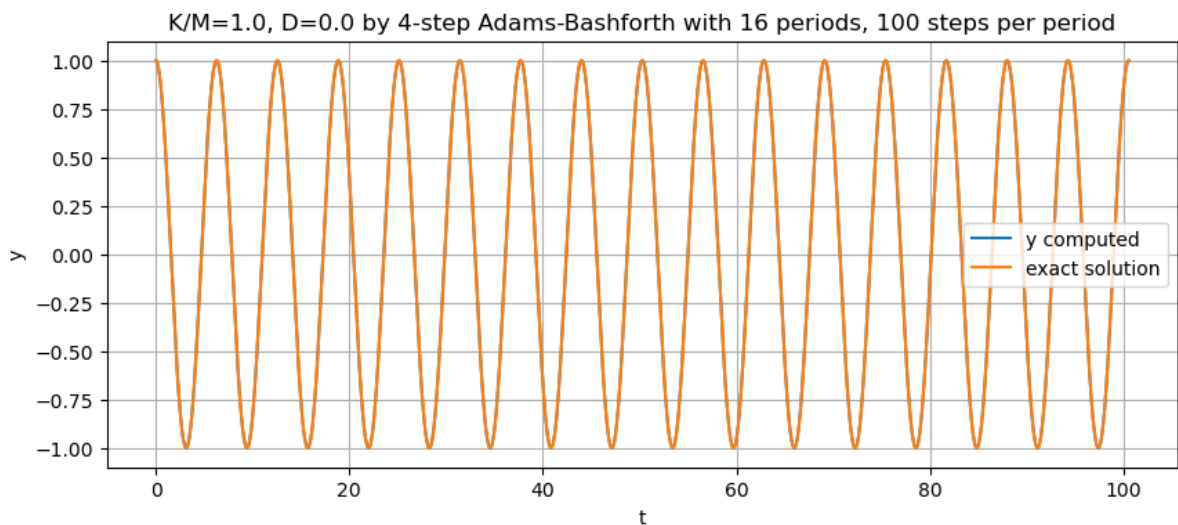
Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

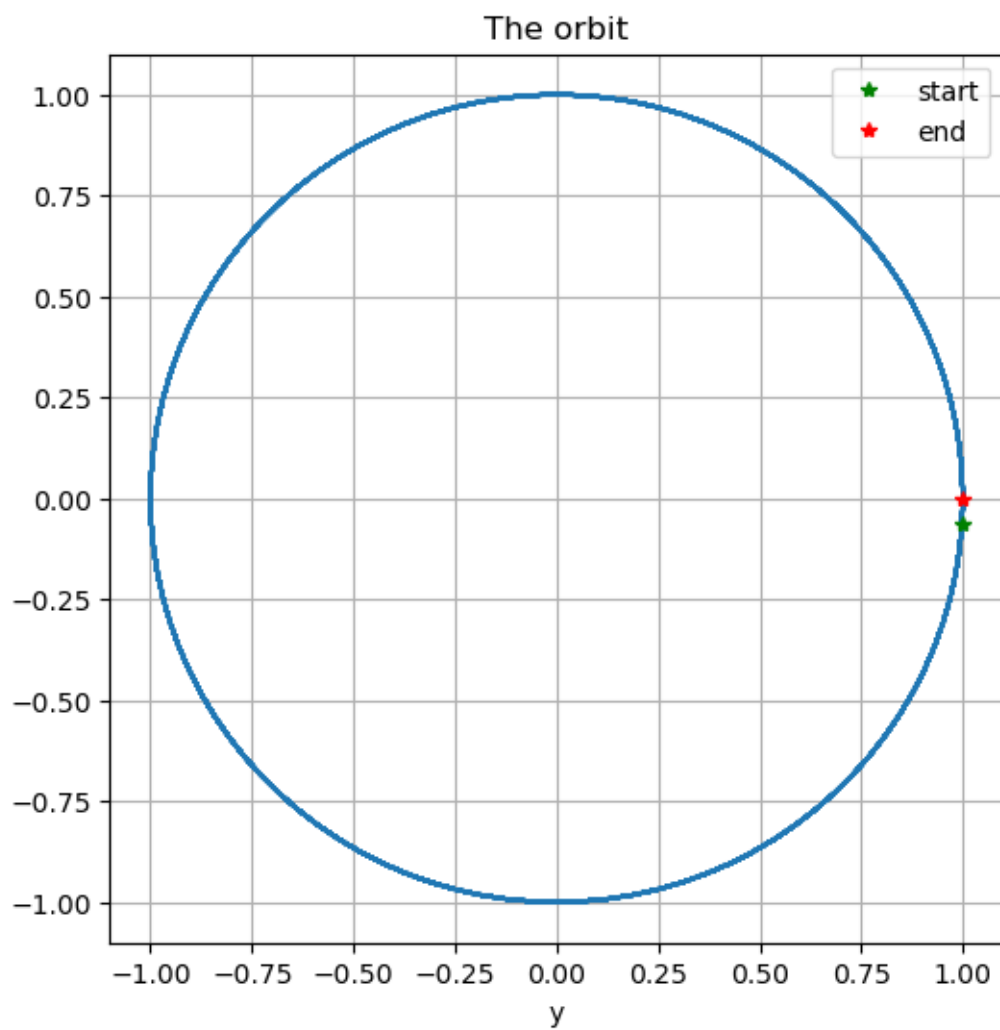
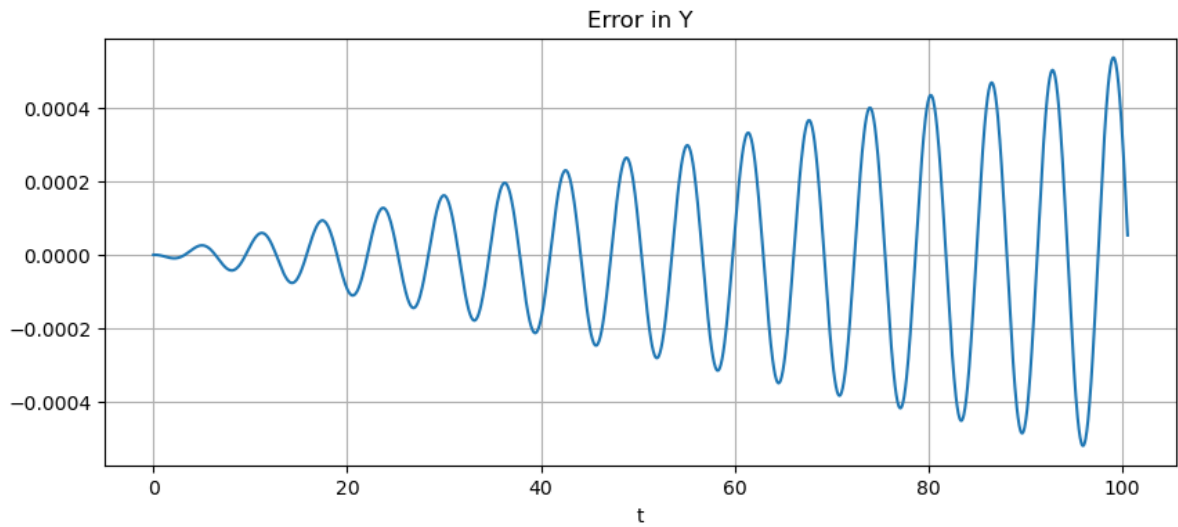
figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 4-step Adams-Bashforth with {periods} periods,
      ↳{stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[6,6]) # Make axes equal length; orbits should be circular or
↳"circular spirals"
title("The orbit")
plot(Y, DY)
xlabel("y")
plot(Y[1], DY[1], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```





D = 0.5

(continues on next page)

(continued from previous page)

```

periods = 4
b = 2*pi * periods

# Using the same time step size as for leapfrog method in the previous section.
stepsperperiod = 50
n = int(stepsperperiod * periods)

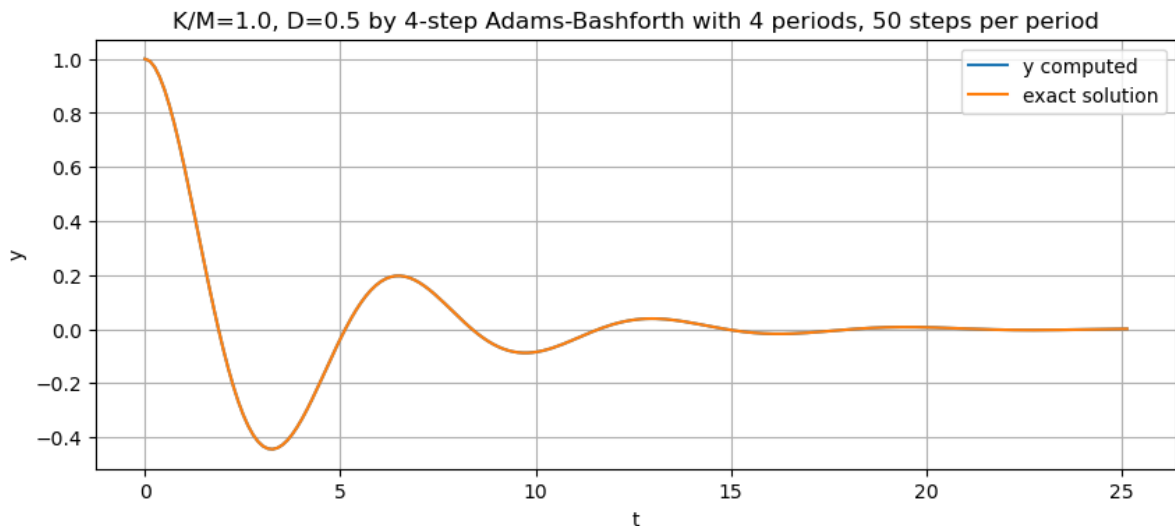
# We need U_1, U_2 and U_3, and get them with the Runge-Kutta method.
h = (b-a)/n
(t_3step, U_3step) = rungekutta_system(f_mass_spring, a, a+3*h, U_0, 3)
U_1 = U_3step[1,:]
U_2 = U_3step[2,:]
U_3 = U_3step[3,:]
(t, U) = adamsbashforth4(f_mass_spring, a, b, U_0, U_1, U_2, U_3, n)

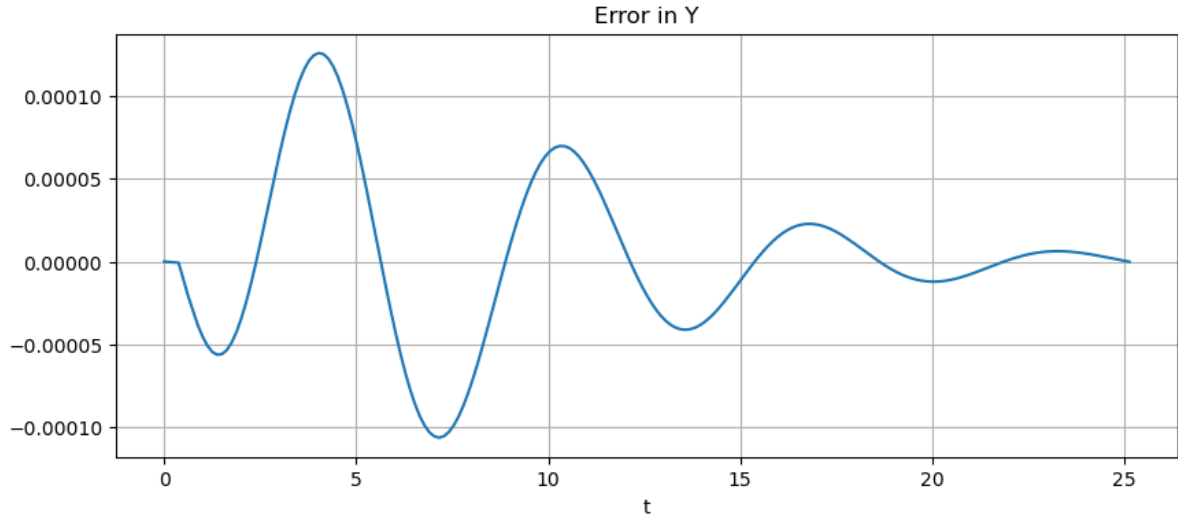
Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 4-step Adams-Bashforth with {periods} periods,
      ↪ {stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

```





Finally, an “equal cost” comparison to the fourth order Runge-Kutta method results in section *Systems of ODEs and Higher Order ODEs* with four times as many steps per unit time: the fourth order Adams-Bashforth method come out ahead in these two test cases.

```
D = 0.0

periods = 16
b = 2*pi * periods

stepsperperiod = 100
n = int(stepsperperiod * periods)

# We need U_1, U_2 and U_3, and get them with the Runge-Kutta method;
# this is overkill for accuracy, but since only three steps are needed, the time cost
# is negligible.
h = (b-a)/n
(t_3step, U_3step) = rungekutta_system(f_mass_spring, a, a+3*h, U_0, 3)
U_1 = U_3step[1,:]
U_2 = U_3step[2,:]
U_3 = U_3step[3,:]
(t, U) = adamsbashforth4(f_mass_spring, a, b, U_0, U_1, U_2, U_3, n)

Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 4-step Adams-Bashforth with {periods} periods,
      {stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
```

(continues on next page)

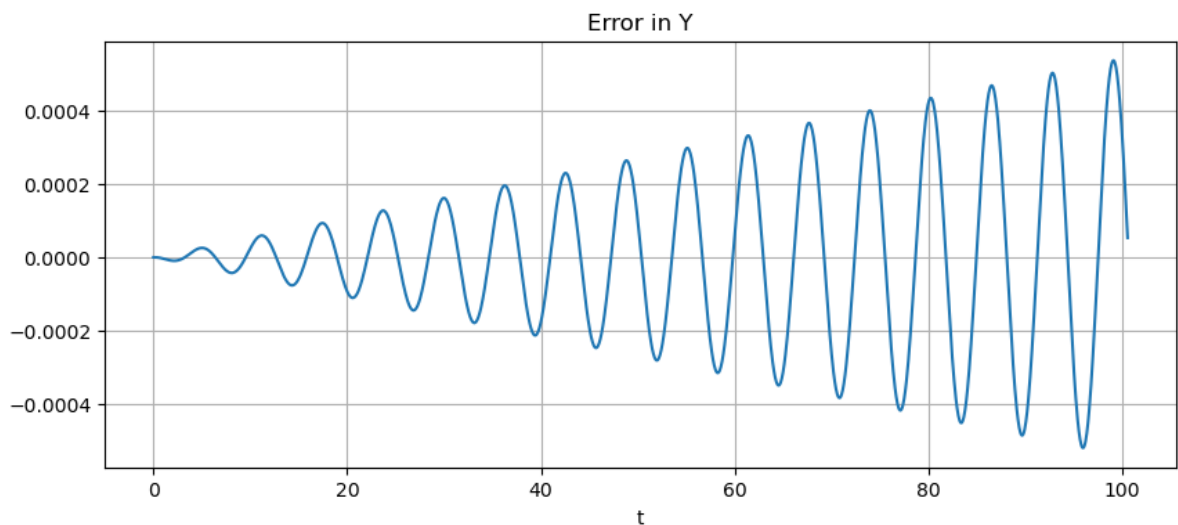
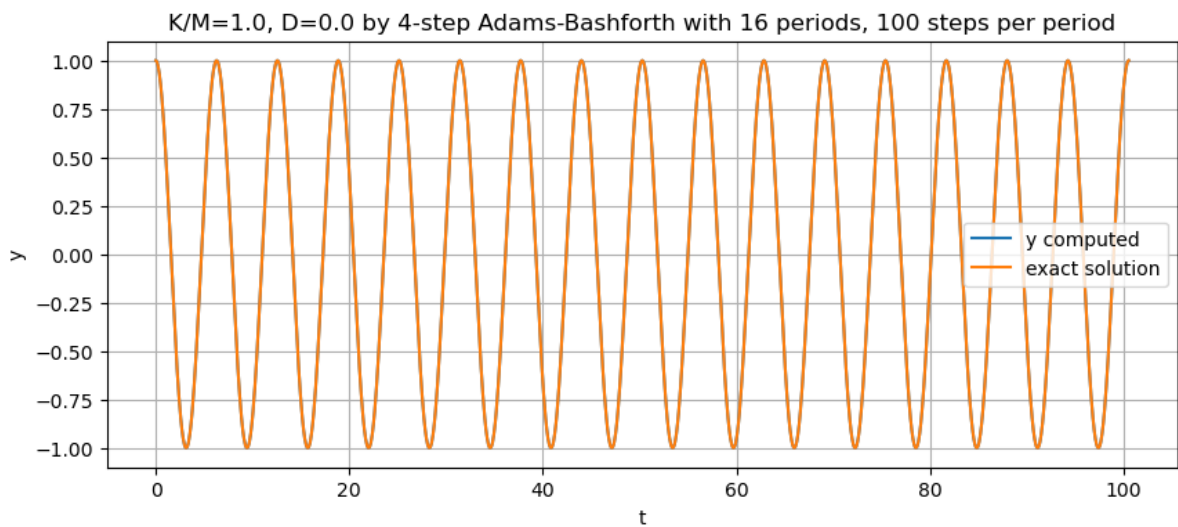
(continued from previous page)

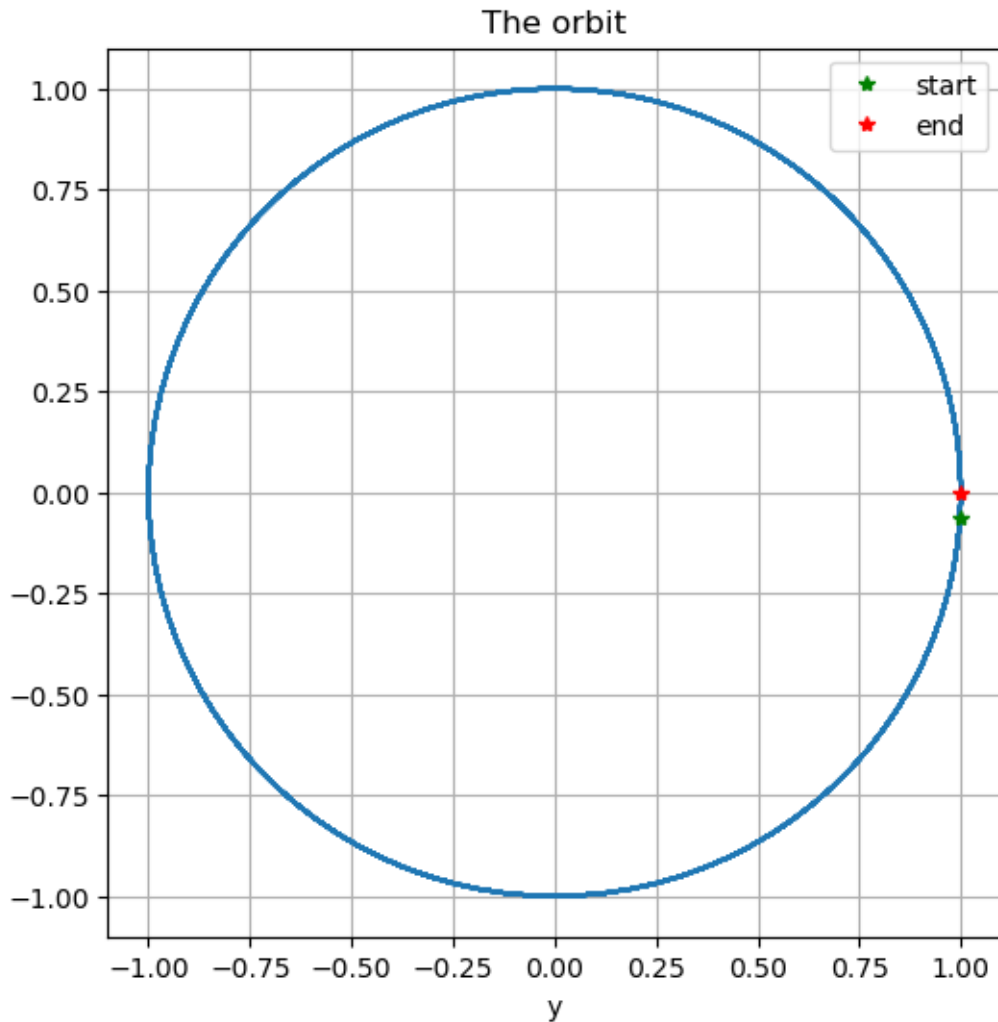
```

plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[6,6]) # Make axes equal length; orbits should be circular or
    ↪ "circular spirals"
title("The orbit")
plot(Y, DY)
xlabel("y")
plot(Y[1], DY[1], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```





```

D = 0.5

periods = 4
b = 2*pi * periods

stepsperperiod = 100
n = int(stepsperperiod * periods)

# We need U_1, U_2 and U_3, and get them with the Runge-Kutta method;
# this is overkill for accuracy, but since only three steps are needed, the time cost
# is negligible.
h = (b-a)/n
(t_3step, U_3step) = rungekutta_system(f_mass_spring, a, a+3*h, U_0, 3)
U_1 = U_3step[1,:]
U_2 = U_3step[2,:]
U_3 = U_3step[3,:]
(t, U) = adamsbashforth4(f_mass_spring, a, b, U_0, U_1, U_2, U_3, n)

Y = U[:,0]
DY = U[:,1]
y = y_mass_spring(t, t_0=a, u_0=U_0, K=K, M=M, D=D) # Exact solution

```

(continues on next page)

(continued from previous page)

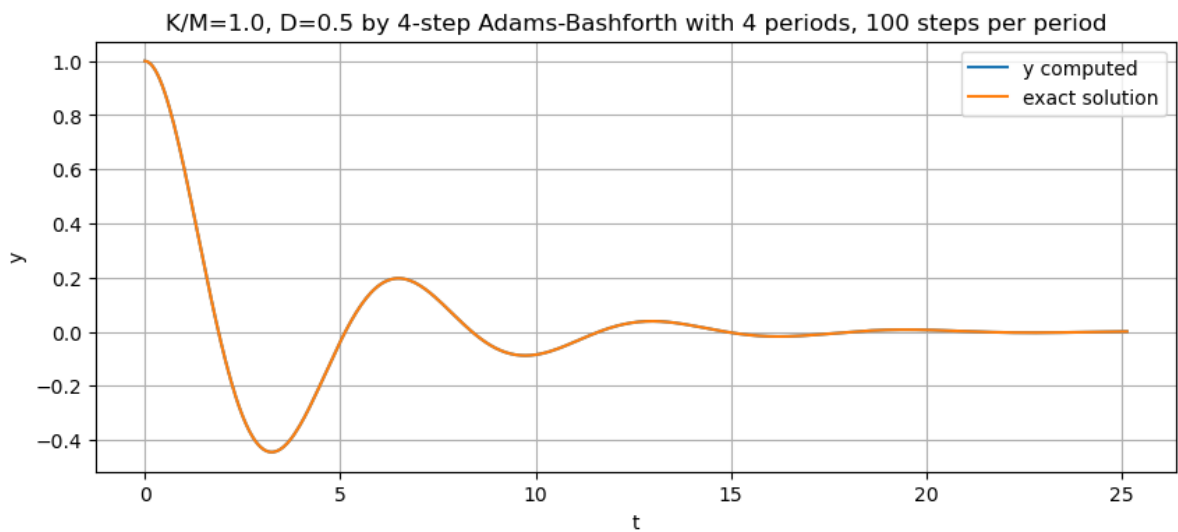
```

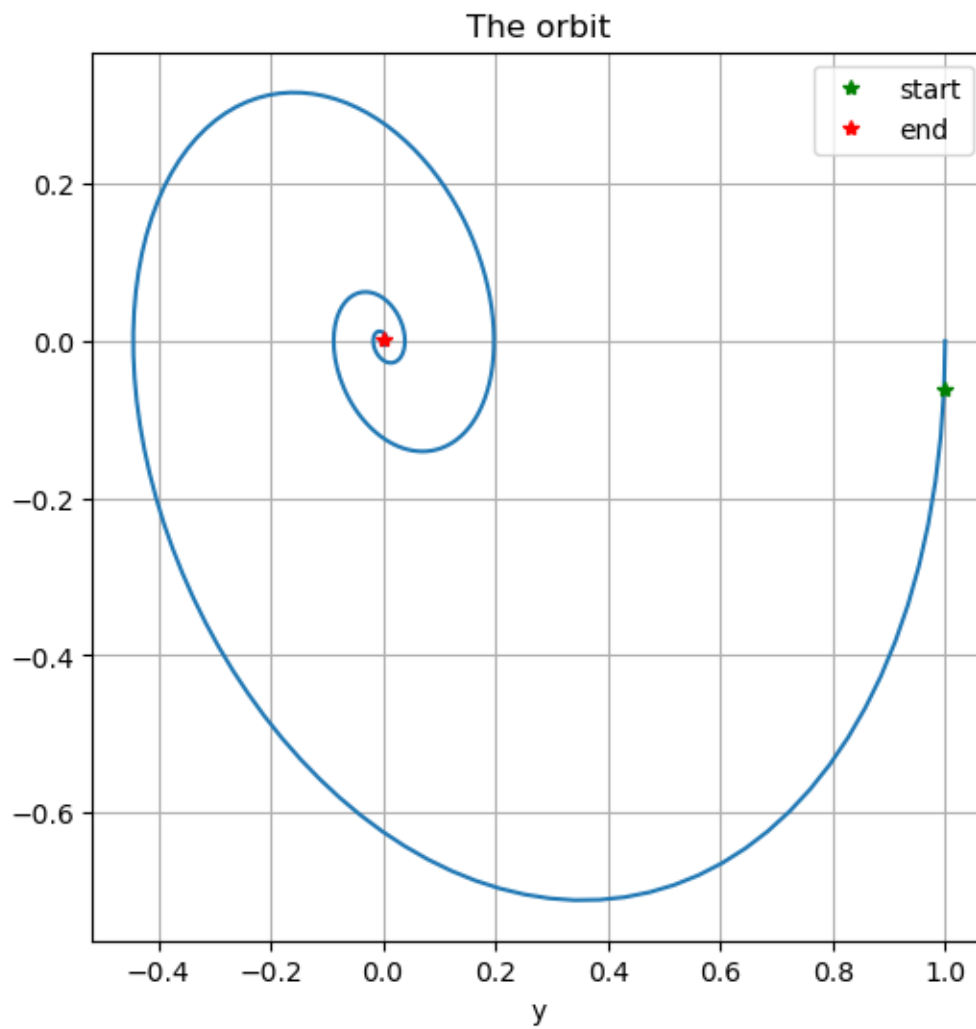
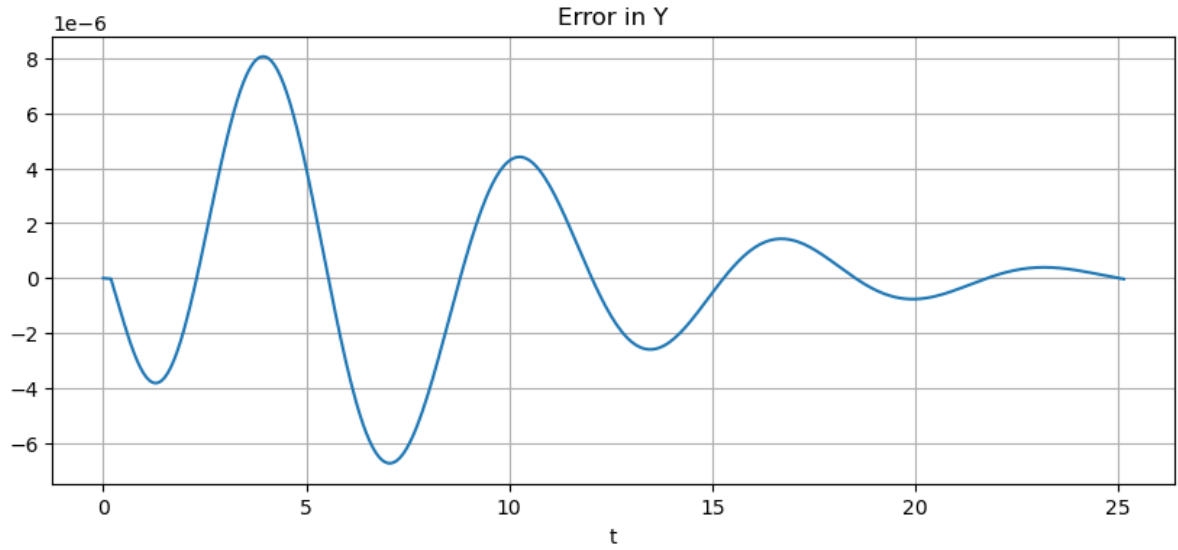
figure(figsize=[10,4])
title(f"{K/M=}, {D=} by 4-step Adams-Bashforth with {periods} periods,
      ↪ {stepsperperiod} steps per period")
plot(t, Y, label="y computed")
plot(t, y, label="exact solution")
legend()
xlabel("t")
ylabel("y")
grid(True)

figure(figsize=[10,4])
title("Error in Y")
plot(t, y-Y)
xlabel("t")
grid(True)

figure(figsize=[6,6]) # Make axes equal length; orbits should be circular or
      ↪ "circular spirals"
title("The orbit")
plot(Y, DY)
xlabel("y")
plot(Y[1], DY[1], "g*", label="start")
plot(Y[-1], DY[-1], "r*", label="end")
legend()
grid(True)

```





7.7.3 Exercises

Exercise A

Verify the derivation of Equation (7.2) for the second order Adams-Bashforth method, via polynomial collocation and integration.

Exercise B

Verify the above result for $s = 3$ by the method of undetermined coefficients.

7.8 Implicit Methods: Adams-Moulton

References:

- Section 6.7 *Multistep Methods* in [Sauer, 2022].
- Section 5.6 *Multistep Methods* in [Burden *et al.*, 2016].

7.8.1 Introduction

So far, most methods we have seen give the new approximation value with an explicit formula for it in terms of previous (and so already known) values; the general **explicit s-step method** seen in *Adams-Bashforth Multistep Methods* was

$$U_i = \phi(U_{i-1}, \dots, U_{i-s}, h), \quad s > 1$$

However, we briefly saw two **implicit methods** back in *Runge-Kutta Methods*, in the process of deriving the explicit trapezoid and explicit midpoint methods: the **implicit trapezoid method** (or just the **trapezoid method**, as this is the real thing, before the further approximations were used to get an explicit formula)

$$U_{i+1} = U_i + h \frac{f(t_i, U_i) + f(t_{i+1}, U_{i+1}))}{2}$$

and the **Implicit Midpoint Method**

$$U_{i+1} = U_i + hf \left(t + h/2, \frac{U_i + U_{i+1}}{2} \right)$$

These are clearly not as simple to work with as explicit methods, but the equation solving can often be done. In particular for linear differential equations, these give linear equations for the unknown U_{i+1} , so even for systems, they can be solved by the method seen earlier in these notes.

Another strategy is noting that these are fixed point equations so that fixed point iteration can be used. The factor h at right helps; it can be shown that for small enough h (how small depends on the function f), these are contraction mappings and so fixed point iteration works.

This idea can be combined with linear multistep methods, and one important case is modifying the Adams-Bashforth method by allowing $F_i = f(t_i, U_i)$ to appear at right: this gives the Adams-Moulton form

$$U_i = U_{i-1} + h(b_0 f(t_{i-s}, U_{i-s}) + \dots + b_s f(t_i, U_i))$$

where the only change from Adams-Bashforth methods is that b_s term.

The coefficients can be derived much as for those, by the method of undetermined coefficients; one valuable difference is that there are now $s + 1$ undetermined coefficients, so all error terms up to $O(h^s)$ can be cancelled and the error made $O(h^{s+1})$: one degree higher.

The $s = 1$ case is familiar:

$$U_i = U_{i-1} + h(b_0 f(t_{i-1}, U_{i-1}) + b_1 f(t_i, U_i))$$

and as symmetry suggests, the solution is $b_0 = b_1 = 1/2$, giving

$$U_i = U_{i-1} + h \frac{f(t_{i-1}, U_{i-1}) + f(t_i, U_i)}{2}$$

which is the (implicit) trapezoid rule in the new shifted indexing.

This is much used for numerical solution of partial differential equations of evolution type (after first approximating by large system of ordinary differential equations). In that context it is often known as the **Crank-Nicholson method**.

We can actually start at $s = 0$; the first few Adams-Moulton methods are:

$$s = 0 : b_0 = 1$$

$$U_i - h f(t_i, U_i) = U_{i-1}$$

The backward Euler method

$$s = 1 : b_0 = b_1 = 1/2$$

$$U_i - \frac{h}{2} f(t_i, U_i) = U_{i-1} + \frac{h}{2} (F_{i-1})$$

The (implicit) trapezoid method

$$s = 2 : b_0 = -1/12, b_1 = 8/12, b_2 = 5/12$$

$$U_i - \frac{5h}{12} f(t_i, U_i) = U_{i-1} + \frac{h}{12} (-F_{i-2} + 8F_{i-1})$$

$$s = 3 : b_0 = 1/24, b_1 = -5/24, b_2 = 19/24, b_3 = 9/24$$

$$U_i - \frac{9h}{24} f(t_i, U_i) = U_{i-1} + \frac{h}{24} (F_{i-3} - 5F_{i-2} + 19F_{i-1})$$

The use of F_{i-k} notation emphasizes that these earlier values of $F_{i-k} = f(t_{i-k}, U_{i-k})$ are known from a previous step, so can be stored for reuse.

The backward Euler method has not been mentioned before; it comes from using the backward counterpart of the forward difference approximation of the derivative:

$$u'(t) \approx \frac{u(t) - u(t-h)}{h}$$

Like Euler's method it is only first order accurate, but it has excellent *stability* properties, which makes it useful in some situations.

Rather than implementing any of these, the next section introduces a strategy for deriving explicit methods of comparable accuracy, much as in [Runge-Kutta Methods](#) Euler's method (Adams-Bashforth $s = 1$) was combined with the trapezoid method (Adams-Moulton $s = 1$) to get the explicit trapezoid method: an explicit method with the same order of accuracy as the latter of this pair.

7.8.2 Exercises

Coming soon.

7.9 Predictor-Corrector Methods — Coming Soon

References:

- Section 6.7 *Multistep Methods* in [Sauer, 2022].
- Section 5.6 *Multistep Methods* in [Burden *et al.*, 2016].

7.9.1 Introduction

We have seen one predictor-corrector method already: the explicit trapezoid method, which uses Euler’s method to *predict* a first approximation of the solution, and then *corrects* this using an approximation of the implicit trapezoid method.

We will look at combining the s -step Adams-Bashforth and Adams-Moulton methods to achieve an two-stage, s -step method of same order of accuracy as the latter while being explicit — the explicit trapezoid method is this in the simplest case $s = 1$.

```
import numpy as np
from matplotlib import pyplot as plt
# Shortcuts for some favorite commands:
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend
```

7.10 Introduction to Implicit Methods and Stiff Equations

References:

- Section 6.6 *Implicit Methods and Stiff Equations* in [Sauer, 2022].
- Section 5.11 *Stiff Equations* in [Burden *et al.*, 2016].

```
import numpy as np
from matplotlib import pyplot as plt
# Shortcuts for some favorite commands:
from matplotlib.pyplot import figure, plot, grid, title, xlabel, ylabel, legend
```

7.10.1 Introduction

Coming soon ...

Part III

Exercises

EXERCISES ON THE BISECTION METHOD

8.1 A test case

As a first test case, we will solve $x - \cos(x) = 0$, which can be shown to have a unique root that lies in the interval $[0, 1]$. Then other equations can be tried.

8.2 Exercise 1

Create a Python function implementing the first, simplest algorithm from the section on *Root finding by interval halving*, which performs a fixed number of iterations, `max_iterations`. (This was called “N” there, but in code I encourage using more descriptive names for variables.)

This be used as: `root = bisection1(f, a, b, max_iterations)`

Test it with the above example, and then try solving at least one other equation.

The main task is to create a Python function whose input specifies a function `f`, the interval end-points `a` and `b`, and an upper limit `tol` on the allowable absolute error in the result; and whose output is both an approximate root `c` and a bound `errorBound` on its absolute error.

That is, we require that there is an exact root r near c , in that

$$|r - c| \leq \text{errorBound} \leq \text{TOL}.$$

The definition of this Python function will be of the form

```
def bisection(f, a, b, TOL):  
    . . .  
    return c, errorBound
```

and the usage will be something like

```
(root, errorBound) = bisection(f, a, b, TOL)  
print(f'The approximate root is {root} with absolute error at most {errorBound}')
```

I give a definition for the test function f . Note that I get the cosine function from the module `numpy` rather than the standard Python module `math`, because `numpy` will be far more useful for us, and so I encourage you to **avoid module `math` as much as possible!**

```
from numpy import cos
def f(x):
    return x - cos(x)
```

This helps with the readability of large collections of code, avoiding the need to look further up the file to see where an object like `cos` comes from. (It is also essential if you happen to use two functions of the same name from different modules, though in the current example, one is unlikely to want both `math.cos` and `numpy.cos`.)

8.3 The bisection method algorithm in “pseudocode”

Here is a description of the bisection method algorithm in *pseudocode*, as used in our text book and these notes: a mix of notations from mathematics and computer code, whatever makes the ideas clearest.

Input: f (a continuous function from and to the real numbers), a and b (real numbers, $a < b$ with $f(a)$ and $f(b)$ of opposite sign) $errorTolerance$ (the maximum allowable absolute error)

Output will be: r (an approximation of a solution of $f(r) = 0$) $errorBound$ (an upper limit on the absolute error in that approximation).

$c = \frac{a+b}{2}$ $errorBound = c - a$ while $errorBound > errorTolerance$: if $f(a)f(c) > 0$: $a \leftarrow c$ else: $b \leftarrow c$
end if $c = \frac{a+b}{2}$ $errorBound = c - a$ end while $r = c$

Output: r , $errorBound$

8.4 Exercise 2

Create Python/Numpy code for the more refined algorithm `mabvm`, involving to a specified maximum allowable absolute error, so with usage `(root, errorBound) = bisection(f, a, b, TOL)`

Again test by solving $x - \cos x = 0$, using the fact that there is a solution in the interval $(-1, 1)$, but this time solve accurate to within 10^{-4} , and output the final error bound as well as the approximate root.

8.5 Exercise 3

Consider the equation $x^5 = x^2 + 10$.

- Find an interval $[a, b]$ of length one in which there is guaranteed to be a root.
- Compute the next two improved approximations given by the bisection method.
- Determine how many iterations of the bisection method would then be needed to approximate the root with an absolute error of at most 10^{-10} .

Do this without actually computing those extra iterations or computing the root!

EXERCISES ON FIXED POINT ITERATION

9.1 Exercise 1

The equation $x^3 - 2x + 1 = 0$ can be written as a fixed point equation in many ways, including

1. $x = \frac{x^3 + 1}{2}$

and

2. $x = \sqrt[3]{2x - 1}$

For each of these options:

- (a) Verify that its fixed points do in fact solve the above cubic equation.
- (b) Determine whether fixed point iteration with it will converge to the solution $r = 1$. (assuming a “good enough” initial approximation).

Note: computational experiments can be a useful start, but prove your answers mathematically!

EXERCISES ON ERROR MEASURES AND CONVERGENCE**10.1 Exercise 1**

- a) Find the multiplicity of the root $r = 0$ of the function $f(x) = 1 - \cos x$.
- b) Evaluate the forward and backward errors of the approximate root $\tilde{r} = 0.001$.

EXERCISES ON NEWTON'S METHOD

11.1 Exercise 1

a) Show that Newton's method applied to

$$f(x) = x^k - a$$

leads to fixed point iteration with function

$$g(x) = \frac{(k-1)x + \frac{a}{x^{k-1}}}{k}.$$

b) Then verify mathematically that the iteration $x_{k+1} = g(x_k)$ has super-linear convergence.

11.2 Exercise 2

a) Create a Python function for Newton's method, with usage

```
(root, errorEstimate, iterations, functionEvaluations) = newton(f, Df, x_0, ↵  
↵errorTolerance, maxIterations)
```

(The last input parameter `maxIterations` could be optional, with a default like `maxIterations=100`.)

b) based on your function `bisection2` create a third (and final!) version with usage

```
(root, errorBound, iterations, functionEvaluations) = bisection(f, a, b, ↵  
↵errorTolerance, maxIterations)
```

c) Use both of these to solve the equation

$$f_1(x) = 10 - 2x + \sin(x) = 0$$

i) with [estimated] absolute error of no more than 10^{-6} , and then

ii) with [estimated] absolute error of no more than 10^{-15} .

Note in particular how many iterations and how many function evaluations are needed.

Graph the function, which will help to find a good starting interval $[a, b]$ and initial approximation x_0 .

d) Repeat, this time finding the unique real root of

$$f_2(x) = x^3 - 3.3x^2 + 3.63x - 1.331 = 0$$

Again graph the function, to find a good starting interval $[a, b]$ and initial approximation x_0 .

e) This second case will behave differently than for f_1 in part (c): describe the difference. (We will discuss the reasons in class.)

EXERCISES ON ROOT-FINDING WITHOUT DERIVATIVES

12.1 Exercise 1: Comparing Root-finding Methods

Note: This builds on the previous exercise comparing the Bisection and Newton's Methods; just adding the Secant Method.

A) Write a Python function implementing the secant method with usage

```
(root, errorEstimate, iterations, functionEvaluations) = secant(f, a, b,   
↪errorTolerance, maxIterations)
```

Update your previous implementations of the bisection method and Newton's method to mimic this interface:

```
(root, errorEstimate, iterations, functionEvaluations) = bisection(f, a, b,   
↪errorTolerance, maxIterations)  
  
(root, errorEstimate, iterations, functionEvaluations) = newton(f, x_0,   
↪errorTolerance, maxIterations)
```

Aside: the last parameter `maxIterations` could be optional, with a default like `maxIterations=100`.

B) Use these to solve the equation

$$10 - 2x + \sin(x) = 0$$

i) with [estimated] absolute error of no more than 10^{-8} , and then

ii) with [estimated] absolute error of no more than 10^{-15} .

Note in particular how many iterations and how many function evaluations are needed.

C) Discuss: rank these methods for speed, as indicated by these experiments, and explain your ranking.

EXERCISES ON MACHINE NUMBERS, ROUNDING ERROR AND ERROR PROPAGATION

13.1 Exercise 1

Verify that when *dividing* two numbers, the relative error in the quotient is no worse than slightly more than the sum of the relative errors in the numbers divided. (Mimic the argument for the corresponding result on products.)

13.2 Exercise 2

For $x = 8.024$ and $y = 8.006$,

- Round each to three significant figures, giving x_a and y_a .
- Compute the absolute errors in each of these approximations, and in their difference as an approximation of $x - y$.
- Compute the relative errors in each of these three approximations.

Then look at rounding to only two significant digits!

13.3 Exercise 3

(a) Illustrate why computing the roots of the quadratic equation $ax^2 + bx + c = 0$ with the standard formula

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

can sometimes give poor accuracy when evaluated using machine arithmetic such as IEEE-64 floating-point arithmetic. This is not always a problem, so identify specifically the situations when this could occur, in terms of a condition on the coefficients a , b , and c . (It is sufficient to consider real value of the coefficients. Also as an aside, there is no loss of precision, when the roots are non-real, so you only need consider quadratics with real roots.)

(b) Then describe a careful procedure for always getting accurate answers. State the procedure first with words and mathematical formulas, and then express it in pseudo-code.

EXERCISES ON SOLVING SIMULTANEOUS LINEAR EQUATIONS

14.1 Exercise 1

A) Solve the system of equations

$$\begin{bmatrix} 4. & 2. & 1. \\ 9. & 3. & 1. \\ 25. & 5. & 1. \end{bmatrix} x = \begin{bmatrix} 0.693147 \\ 1.098612 \\ 1.609438 \end{bmatrix}$$

by naive Gaussian elimination. Do this *by hand*, rounding each intermediate result to four significant digits, and write down each intermediate version of the system of equations.

B) Compute the residual vector $r := b - Ax_a$ and residual maximum norm $\|r\|_{\max} = \|b - Ax_a\|_{\max}$ for your approximation. Residual calculations must be done to high precision, so I recommend that you do this part with Python in a notebook.

14.2 Exercise 2

Repeat Exercise 1, except using *maximal element partial pivoting*. Then compare the residuals for the two methods (with and without pivoting), and comment.

14.3 Exercise 3

A) Compute the Doolittle LU factorization of the matrix

$$A = \begin{bmatrix} 4. & 2. & 1. \\ 9. & 3. & 1. \\ 25. & 5. & 1. \end{bmatrix}$$

as in the above exercises.

As above, do this by hand, rounding values to four significant digits, and describing the intermediate steps. (Feel free to then corroborate your hand working with Python code, but the results will not be exactly the same!)

B) Use this LU factorization to solve $Ax = b$ for x where

$$b = \begin{bmatrix} 0.693147 \\ 1.098612 \\ 1.609438 \end{bmatrix}$$

as above.

14.4 Some Relevant Algorithms

14.4.1 Row reduction

The basic algorithm for row reduction is

```
for k from 1 to n-1:      Step k: get zeros in column k below row k:
    for i from k+1 to n:  update only the rows that change: from k + 1 on:
        The multiple of row k to subtract from row i:

$$l_{i,k} = a_{i,k}^{(k-1)} / a_{k,k}^{(k-1)} \quad \text{If } a_{k,k}^{(k-1)} \neq 0!$$

        Subtract ( $l_{i,k}$  times row k) from row i in matrix A, in the columns that are not automatically zero:
        for j from k+1 to n:

$$a_{i,j}^{(k)} = a_{i,j}^{(k-1)} - l_{i,k} a_{k,j}^{(k-1)}$$

        end for
        and at right, subtract ( $l_{i,k}$  times  $b_k$ ) from  $b_i$ :

$$b_i^{(k)} = b_i^{(k-1)} - l_{i,k} b_k^{(k-1)}$$

    end for
```

14.4.2 Backward substitution with an upper triangular matrix

The basic algorithm for backward substitution is

```

$$x_n = c_n / u_{n,n}$$

for i from n-1 down to 1

$$x_i = \frac{c_i - \sum_{j=i+1}^n u_{i,j} x_j}{u_{i,i}}$$

end for
```

EXERCISES ON APPROXIMATING DERIVATIVES, THE METHOD OF UNDETERMINED COEFFICIENTS AND RICHARDSON EXTRAPOLATION

15.1 Exercise 1

Show that for a three-point one-sided difference approximation of the first derivative

$$Df(x) = \frac{C_0 f(x) + C_1 f(x+h) + C_2 f(x+2h)}{h} + O(h^p)$$

the most accurate choice is $C_0 = -3/2$, $C_1 = 2$, $C_2 = -1/2$, giving

$$Df(x) \approx \frac{-3f(x) + 4f(x+h) - f(x+2h)}{2h} + O(h^2).$$

and verify that this is of second order

Do this by setting up the three equations as above for the coefficients C_0 , C_1 and C_2 , and solving them. Do this “by hand”, to get exact fractions as the answers; use the two Taylor series formulas, but now take advantage of what we saw above, which is that the error starts at the terms in $D^3 f(x)$, so use the forms

$$f(x+h) = f(x) + Df(x)h + \frac{D^2 f(x)}{2}h^2 + \frac{D^3 f(x)}{6}h^3 + O(h^4)$$

and

$$f(x+2h) = f(x) + 2Df(x)h + 2D^2 f(x)h^2 + \frac{4D^3 f(x)}{3}h^3 + O(h^4)$$

15.2 Exercise 2

Repeat Exercise 1, but using the degree of precision method.

That is, impose the condition of giving the exact value for the derivative at $x = 0$ for the monomial $f(x) = 1$, then the same for $f(x) = x$, and so on until there are enough equations to determine a unique solution for the coefficients.

15.3 Exercise 3

Verify that the most accurate three-point centered difference approximation of $D^2 f(x)$, form

$$D^2 f(x) \approx \frac{C_{-1}f(x-h) + C_0f(x) + C_1f(x+h)}{h^2}$$

is given by the coefficients $C_{-1} = C_1 = 1$, $C_0 = -2$ in that this is of the highest order; $p = 2$.

That is

$$D^2 f(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2} + O(h^2).$$

Do this by hand, and exploit the symmetry.

Note that it works a bit better than expected, due to the symmetry.

15.4 Exercise 4

Repeat Exercise 3, but using the degree of precision method.

15.5 Exercise 5

Derive a symmetric five-point approximation of the second derivative, using the Method of Undetermined Coefficients; I recommend that you use the simpler second, “monomials” approach.

Note: try to exploit symmetry to reduce the number of equations that need to be solved.

15.6 Exercise 6

Use the symmetric centered difference approximation of the second derivative and Richardson extrapolation to get another more accurate approximation of this derivative.

Then compare to the result in Exercise 5.

Part IV

Python Tutorial

INTRODUCTION

This is a selection of notes on Python, particularly about using the packages [Numpy](#) and [Matplotlib](#) which might not have been encountered in a first course on Python programming.

These are excerpts from the Jupyter book *Python for Scientific Computing*, which might also be useful for review and reference.

GETTING PYTHON SOFTWARE FOR SCIENTIFIC COMPUTING

17.1 Anaconda, featuring JupyterLab, Spyder, and IPython

I suggest that you get [Anaconda](#) for your own computers, even if you also have access to it via computers on-campus. Get a sufficiently recent version: at least version 3.9; we will use some of the newer features, especially for working more easily with matrices and vectors.

Anaconda is a free download from <https://www.anaconda.com/products/individual>

Once Anaconda is installed, you access its components by opening the *Anaconda Navigator*. The most important of these for us will be *JupyterLab*, for working with Jupyter notebooks; see the [Project Jupyter](#) site.

Other Anaconda components of possible interest are:

- [Spyder](#) an Integrated Development Environment (like IDLE, but far fancier!) for writing and running Python code files (suffix `.py`). This has more advanced editing and debugging tools than JupyterLab, so some readers might prefer to develop Python code in Spyder (or even with IDLE) and then copy the code into a notebook for final presentation. Aside: “Spyder” is a portmanteau for **ScientificPY**thon **DE**velopment envi**RON**ment.)
- [IPython console](#) provides a command line for executing Python code, akin to what you might be familiar with when using IDLE. (Note that within Spyder, one pane is an iPython console, along-side the editing pane.) (Aside: “IPython” is short for **I**nteractive **P**ython; the “I” is capitalized; it is not an Apple product!)

17.2 Colab (a.k.a. Colaboratory); a purely online alternative for Jupyter notebooks

An alternative is to use the online resource [Colab](#) provided by Google. This works entirely with Jupyter notebooks, stored in the Colab website. Colab does not support directly running Python code files. However, it does support uploading your own modules in Python code files, from which a notebook can import stuff. (Never mind if you do not yet know about creating modules and importing from them; we will review that later.)

SUGGESTIONS AND NOTES ON PYTHON AND JUPYTER NOTEBOOK USAGE

Brenton LeMesurier lemesurierb@cofc.edu

Version of September 25, 2022

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

1. Start every document (Notebook, Python code file, etc.) with:
 - A title (e.g. “Assignment 1”)
 - Your name (and maybe email address)
 - The date, of the latest version — so update this date after any major changes to the file.See the first cell above for an example!
2. Put all `import` statements together in the first code cell of the notebook. Similarly, in Python code files (suffix `.py`) make the import statements the first code, straight below any opening comments.
3. Put each function definition in its own cell, and only define a function once per notebook.
4. More generally, divide both text and Python code into many, small cells; that makes it easier to edit, preview, debug and make changes. For example, start a new cell for each new section at any level (`# ...`, `## ...`, etc.) so that each section heading is the first line of a Markdown cell.
5. Whenever practical, use different names for similar but different items. For example, name sample functions `f_1`, `f_2` and so on, rather than reusing the generic name `f`. Likewise for variants of a function, as with `bisection1`, `bisection2` ...
6. Remember that Python is case-sensitive, so that `ErrorBound`, `errorBound` and `errorbound` are three different variables!
7. On the last item, I suggest avoiding capital letters almost always, using them only when that style is “dictated” by standard mathematical notation. For example it makes sense to call a matrix `A`, but amongst the above three names, I would use only the latter or otherwise `error_bound` with an underscore to improve readability. The underscore `_` is an “honorary letter”, often used as a fake space to separate words in a name. (Note: the underscore `_` is not a dash `-`; it is typed as “shift-dash”).
8. One very common mistake is not reducing the indentation at the end of a block (the lines that go with an `if`, `for`, `while` or such).
9. Another thing to check for: that wherever a variable is used (at right in an assignment, in the input parameters of a function, etc.) it already has a value — and an up-to-date value at that.

10. Only use the Markdown section heading syntax `# ...`, `## ...` and so on for items that are legitimately the titles of sections, sub-sections, and so on. Otherwise, emphasize with the `*...*` and `**... **` syntax for *italics* and **boldface** respectively.
11. When the result of a numerical calculation is only an approximation (almost always in this course!) aim to also provide an upper bound on the absolute error — and if that is not feasible, then at least provide a plausible *estimate* of the error, or a measure of *backward error*.
12. Avoid infinite loops! (In JupyterLab, the disk at top-right is black while code is running; goes white when finished; in case of execution never finishing, you can stop it with the square “Stop” button above.) One strategy is to avoid `while` loops, in particular `while True` or `while 1`; I will illustrate the following approach in sample codes for iterative methods:
 - set a maximum number of iterations `maxiterations`
 - use a `for` loop `for iteration in range(maxiterations):` instead of `while not weAreDone:`
 - Implement the stopping condition with `if we_are_done: break` somewhere inside the `for` loop.
13. The last step before you submit a notebook or otherwise share it is to **do a “clean run”**, restarting the kernel and then running every cell, and then read it through, checking that the Python code has run successfully, its output is correct, and so on.

Usually the easiest way to do this in JupyterLab is with the “fast forward” double right-arrow button at the top; alternatively, there is a menu item `Kernel > Restart Kernel and Run All Cells ...`

If you get Python errors but still want to share the file (for example, to ask for debugging help), this run will stop at the first error, so scan down to that first error, use menu item `Run > Run Selected Cell and All Below`, and repeat at each error till the run gets to the last cell.

PYTHON BASICS

Last Revised September 26, 2022

19.1 Using Python interactively

For some tasks, Python can be used interactively, much like a scientific calculator (and later, like a graphing scientific calculator) and that is how we will work for this section.

There are multiple ways of doing this, and I invite you to experiment with them all at some stage:

- With the “bare” IPython console; this is available in many ways, including from the Anaconda Navigator by opening *Qt Console*
- With Spyder, which can be accessed from within the Anaconda Navigator by opening *Spyder*; within that, there is an interactive Python console frame at bottom-right.
- In Jupyter notebooks (like this one), which can be accessed from within the Anaconda Navigator by opening *JupyterLab*.

I suggest learning one tool at a time, starting with this Jupyter notebook; however if you wish to also one of both of the alternatives above, go ahead.

19.2 Running some or all cells in a notebook

A Jupyter notebook consists of a sequence of cells, and there are two types that we care about:

- **Code** cells, which contain Python code
- **Markdown** cells (like all the ones so far), which contain text and mathematics, using the [Markdown](#) markup notation for text combined with LaTeX markup notation for mathematical formulas.

Running a Code cell executes its Python code; running a Markdown cell formats its content for nicer output.

There are several ways to run a cell:

- The “Play” (triangle icon) button above
- Typing “Shift-Return” (or “Shift-Enter”)

Both of these run the current cell and then move down to the next cell as “current”.

Also, the menu “Run” above has various options — hopefully self-explanatory, at least after some experimentation.

To get a Markdown cell back from nicely formatted “display mode” back to “edit mode”, double click in it.

It will be convenient to see the value of an expression by simply making it the last line of a cell.

```
2 + 2
```

```
4
```

To see several values from a cell, put them on that last line, separated by commas; the results will appear also separated by commas, and in parentheses:

```
2 + 2, 6 - 1
```

```
(4, 5)
```

(As we will see in the next section, these are `tuples`; a very convenient way of grouping information.)

19.3 Numbers and basic arithmetic

Python, like most programming languages distinguishes integers from floating point (“real”) numbers; even “1.0” is different from ‘1’, the decimal point showing that the former is a floating point number.

One difference is that there is a maximum possible floating point number of about 10^{300} , whereas Python integers can be arbitrarily large — see below with exponentiation.

Addition and subtraction with “+” and “-” are obvious, as seen above.

```
2 + 2, 6 - 2
```

```
(4, 4)
```

However, there are a few points to note with multiplication, exponentiation, and division.

19.3.1 Multiplication and exponentiation

Firstly, the usual multiplication sign not in the standard character set on standard keyboards, so an asterisk “*” is used instead:

```
2 * 3
```

```
6
```

Likewise exponentiation needs a special “keyboard-friendly” notation, and it is a double asterisk “**” (not “^”):

```
2 ** 3
```

```
8
```

Numbers with exponents can be expressed using this exponential notation, but note how Python outputs them:

```
5.3*10**21
```

```
5.3e+21
```

```
7*10**-8
```

```
7e-08
```

This “e” notation is the standard way to describe numbers with exponents, and you can input them that way too.

When printing numbers, Python decides if the number is big enough or small enough to be worth printing with an exponent:

```
2e-10, 3E-5, 3e-4, 5e+4, 10000000000, 6e15, 6e16
```

```
(2e-10, 3e-05, 0.0003, 50000.0, 10000000000, 60000000000000000.0, 6e+16)
```

Aside: note that either “e” or “E” can be used in input of exponents. However in most contexts, Python is case sensitive; we will see an example soon.

19.3.2 Division

Division is normally denoted by “/”

```
6/3
```

```
2.0
```

```
5/3
```

```
1.6666666666666667
```

but there are a few things to note with integers.

Firstly as you see above, dividing two integers always gives a floating point number result, not an integer: note the decimal point in “2.0”.

To do integer division, use “//”

```
6//3
```

```
2
```

```
5//3
```

```
1
```

and to get the remainder, use “%”

```
6%3
```

```
0
```

```
5%3
```

```
2
```

19.3.3 Complex numbers

Python uses `j` for the square root of -1 rather than `i`, and complex numbers are written as `a + bj` or just `bj` for purely imaginary numbers. (Here ‘a’ and ‘b’ must be literal numbers, not names of variables.)

The coefficient `b` in the imaginary part is always needed, even if it is 1.

```
2 + 2j, (2+2j)**2, 1j, 1j**2
```

```
((2+2j), 8j, 1j, (-1+0j))
```

but

```
j
```

```
-----
NameError                                Traceback (most recent call last)
Input In [16], in <cell line: 1>()
----> 1 j

NameError: name 'j' is not defined
```

19.4 Boolean values (True-False)

These are `True` and `False`, capitalized.

Many other options are allowed, like using 0 for `False` and 1 for `True`, but for the sake of readability, I discourage that.

The basic logical operations are done with the usual English words: “and”, “or” and “not”

```
True and True, True and False, False and False
```

```
(True, False, False)
```

```
True or True, True or False, False or False
```

```
(True, True, False)
```

Both “and” and “or” use lazy or “short-circuiting” evaluation: if the T/F at left determines the overall truth value answer (it is `False` with `and`, `True` with `or`) then the term at right is not evaluated.

For example, the following avoids division by zero (the syntax will be explained later, but hopefully it is mostly clear.)


```

if q != 0 and -1 < p/q < 1:
    print(f"{p}/{q} is a proper fraction")
else:
    print(f"{p}/{q} is not a proper fraction")

```

19.5 Comparisons

Comparisons of numbers and tests for equality exist, with two catches:

- There are no keyboard symbols for $\leq$, $\geq$ or $\neq$, so `<=`, `>=` and `!=` are used
- Equality is indicated with a *double* equals sign `==` because a single equals sign has another job, as seen below under *Naming quantities, and displaying information with print*.

So in all:

```
< <= == != >= >
```

One convenient difference from most programming languages is that comparisons can be chained, as seen in the example above:

```
-1 < p/q < 1
```

is equivalent to

```
-1 < p/q and p/q < 1
```

but is both more readable and more efficient, because the middle term is only evaluated once.

Like `and` and `or`, this is short-circuiting.

Chaining can even be done in cases where usual mathematical style forbids, with reversing of the direction of the inequalities:

```
2 < 4 > 3
```

```
True
```

19.6 Character strings

Chunks of text can be described by surrounding them either with double quotes (“real quotation marks”) or so-called ‘single quotes’, (which are actually apostrophes).

```
"I recommend using 'double quote' characters, except perhaps when quote marks must
appear within a string, like here."
```

```
"I recommend using 'double quote' characters, except perhaps when quote marks must
appear within a string, like here."
```

However, using apostrophes (single right quotes) is also allowed by Python, perhaps because they are slightly easier to type.

19.6.1 String concatenation and duplication

To concatenate two strings, “add” them with +.

Also, consistent with that, making multiple copies of a string is done by “multiplication”, with *

```
greeting = "Hello"
audience = "world"
sentence = greeting + " " + audience + "."
print(sentence)
print(3*(greeting+' '))
```

```
Hello world.
Hello Hello Hello
```

19.7 Naming quantities, and displaying information with print

It often helps to give names to quantities, which is done with assignment statements. For example, to solve for x in the very simple equation $ax + b = 0$ for given values of a and b , let us name all three quantities:

```
a = 3
b = 7
x = -b/a
```

(Note: this is why testing for equality uses == instead of =.)

Running the above cell just computes the three values with no output. To see the values, a more flexible alternative to the “last line of a cell” method is the function `print`:

```
print(a, b, x)
```

```
3 7 -2.3333333333333335
```

That output does not explain things very clearly; we can add explanatory text to the printed results in several ways. Most basically:

```
print("For a =", a, "and b =", b, "the solution of ax + b = 0 is", x)
```

```
For a = 3 and b = 7 the solution of ax + b = 0 is -2.3333333333333335
```

The above prints six items — three text strings (each quoted), three numbers — with the input items separated by commas.

There is an alternative notation for such output, called “f-strings”, which use braces to specify that the value of a variable be inserted into a string of text to display:

```
print(f"For a = {a} and b = {b} the solution of ax + b = 0 is {x}")
```

```
For a = 3 and b = 7 the solution of ax + b = 0 is -2.3333333333333335
```

In this example each pair of braces inserts the value of a variable; one can instead put an expression in braces to have it evaluated and that value displayed. So in fact we could skip the variable x entirely:

```
print(f"For a = {a} and b = {b} the solution of ax + b = 0 is {-b/a}")
```

```
For a = 3 and b = 7 the solution of ax + b = 0 is -2.3333333333333335
```

We can also display the equation more directly:

```
print(f"The solution of {a} x + {b} = 0 is {-b/a}")
```

```
The solution of 3 x + 7 = 0 is -2.3333333333333335
```

A final shortcut with `print`: if an expression in braces ends with `=`, both the expression and its value are displayed:

```
print(f"For {a=} and {b=} the solution of ax + b = 0 is {-b/a=}")
```

```
For a=3 and b=7 the solution of ax + b = 0 is -b/a=-2.3333333333333335
```

19.8 Some mathematical functions and constants: module `math`

Python includes many **modules** and **packages** that add useful definitions of functions, constants and such. For us, the most fundamental is `math`, which is a standard part of Python.

Aside: we will soon learn about another package `numpy` and then mostly use that instead of `math`. I mention `math` for now because it is a core part of Python whereas `numpy` is a separate “add-on”, and you should be aware of `math` if only because many references on doing mathematical stuff with Python will refer to it.

We can access specific items with a `from ... import` command; to start with, two variables containing famous values:

```
from math import pi, e
```

```
print(f'pi is approximately {pi}')
```

```
pi is approximately 3.141592653589793
```

```
print(f'e is approximately {e}')
```

```
e is approximately 2.718281828459045
```

Each is accurate to about 16 significant digits; that is the precision of the 64-bit number system standard in most modern computers.

19.8.1 Names (like “e” and “pi”) are case-sensitive

We can now return to the comment above about case sensitivity. Compare the results of the following two input lines:

```
print(f'e = {e}')
```

```
e = 2.718281828459045
```

```
print(f'E = {E}')
```

```
-----  
NameError                                Traceback (most recent call last)  
Input In [33], in <cell line: 1>()  
----> 1 print(f'E = {E}')
```

NameError: name 'E' is not defined

19.8.2 Some math functions

Module math also provides a lot of familiar mathematical functions

```
from math import sin, cos, cosh
```

which can be used like this:

```
sin(pi/4)
```

```
0.7071067811865475
```

```
cos(pi)
```

```
-1.0
```

```
cosh(0)
```

```
1.0
```

Notes

- All Python functions need parentheses around their arguments; no lazy shortcuts with trig. functions and such.
- Trig. functions use radians, not degrees.

19.8.3 Importing all of a module

With the command

```
import math
```

we get access to many functions such as `tan`, but need to address them by fully qualified name, like `math.tan`:

```
print(f"tan(pi/4) = {math.tan(pi/4)}")
```

```
tan(pi/4) = 0.9999999999999999
```

If we had not already imported `pi` above, its full name `math.pi` would also be needed, as in:

```
print(f"tan(pi/4) = {math.tan(math.pi/4)}")
```

```
tan(pi/4) = 0.9999999999999999
```

Aside: The imperfection of computer arithmetic becomes clear: the exact value is 1 of course. However, the precision of about 16 decimal places is far greater than for any experimental measurement (so far), so this is usually not a problem in practice.

19.8.4 Wild imports: avoid, usually

There is another way of importing all the contents of a module so that they are available on a “first name basis”, called a **wild import**; it is like the single-item imports above with `from math import ...` but now with an asterisk (often called the *wild-card character* in this context) as the name of the items to import:

```
from math import *
```

```
log(10)
```

```
2.302585092994046
```

Wild imports can be convenient when working interactively (using Python like a scientific calculator) through saving a bit of typing, but I strongly recommend using the previous more specific approaches when creating files and programs.

One reason is that it is then unambiguous where a given item came from, even if multiple `import` commands are used. For example, we will see later that there are both `math.cos` and `numpy.cos`, and they behave differently in some situations.

Another reason for explicit imports is for internal efficiency in storage and execution time; wild imports can potentially load thousands of items from a large module even if only a few are used.

19.9 Logarithmic functions

We just saw that there is a function `log` in module `math`, but which base does it use?

```
log(10)
```

```
2.302585092994046
```

```
log(e)
```

```
1.0
```

Evaluating these two expressions reveals that “`log()`” is the *natural logarithm*, base e ; what mathematicians usually call “ $\ln$ ”.

For the base ten version “ $\log_{10}$ ” (sometimes just called “ $\log$ ”) use `log10`:

```
log10(100)
```

```
2.0
```

19.9.1 Powers and roots

A square root can of course be computed using the $1/2$ power, as with

```
16**0.5
```

```
4.0
```

but this function is important enough to have a named form provided by module `math`:

```
sqrt(2)
```

```
1.4142135623730951
```

19.10 Notes on organization and presentation of course work

If you are doing the exercises in these notes for a course, record both your input and the resulting output, in a document that you will submit for feedback and then grading, such as a modified copy of this notebook.

I also encourage you to make notes of other things that you learn, beyond what must be submitted — they will help later in the course.

Also, every document that you submit, hand-written or electronic, should start with:

- A *title*,
- *your name* (possibly also with your email address), and

- the *date*, of the **current version**.

(See the *Title Cell* above for example.)

NOTES ON PYTHON CODING STYLE (UNDER CONSTRUCTION)

20.1 Restrictions on characters used in names

TL;DR: *Mostly letters and digits.*

- The names of **Python variables** (including the names of functions) must start with a letter and only contain *letters*, *digits*, and the *underscore* `_` (typed as “shift-dash”). That is, no **dashes** (`-`), **spaces** or other punctuation.
- The names of **files containing Python modules** must follow the above restrictions (apart from the period in the suffix `.py` of course) because these names are used as variable names in `import` statements.
- The names of **notebook files** have just a little more flexibility: they can also include *hyphens*. That is, they should only contain letters, digits, underscores and hyphens; no **spaces** or other punctuation. (Again, apart from the period in the suffix `.ipynb`.)

Aside: this is roughly the same rule as for web-site addresses.

Although you can get away with some other characters in some situations, that will run into problems in situations like cross-referencing to a notebook from another notebook, posting on a web-site, and using a notebook as a section in a Jupyter Book.

20.2 Naming style

In this book names are sometimes a description formed from several words, and since spaces are forbidden, this is generally done by using *camelCase*; for example, an error estimate might be in a variable `errorEstimate`. (Another popular style is to use the underscore as a proxy for a space, as with `error_estimate`.)

One place where underscores are used in names is where the corresponding mathematical notation would have a subscript; for example, the mathematical name x_0 becomes `x_0`. (This mimics the LaTeX notation for subscripts.)

PYTHON VARIABLES, INCLUDING LISTS AND TUPLES, AND ARRAYS FROM PACKAGE NUMPY

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

21.1 Foreword

With this and all future sections, start by creating your own Jupyter notebook; perhaps by copying relevant cells from this notebook and then adding your work.

If you also wish to start practicing with the Spyder IDE, then in addition use it to create a Python code file with that code, and run the commands there too.

Later you might find it preferable to develop code in Spyder and then copy the working code and notes into a notebook for final presentation — Spyder has better tools for debugging.

21.2 Numerical variables

The first step beyond using Python merely as a calculator is storing value in variables, for reuse later in more elaborate calculations. For example, to find both roots of a quadratic equation

$$ax^2 + bx + c = 0$$

we want the values of each coefficient and are going to use each of them twice, which we might want to do without typing in each coefficient twice over.

21.2.1 Example

We can solve the specific equation

$$2x^2 - 8x + 6 = 0$$

using the quadratic formula. But first we need to get the square root function:

```
from math import sqrt
```

Then the rest looks almost like normal mathematical notation:

```
a = 2
b = -10
c = 8
```

```
root0 = (-b - sqrt(b**2 - 4 * a * c)) / (2 * a)
root1 = (-b + sqrt(b**2 - 4 * a * c)) / (2 * a)
```

(Aside: why did I number the roots 0 and 1 instead of 1 and 2? The answer is coming up soon.)

Where are the results? They have been stored in variables rather than printed out, so to see them, use the print function:

```
print('The smaller root is', root0, 'and the larger root is', root1)
```

```
The smaller root is 1.0 and the larger root is 4.0
```

Aside: This is the first mention of the function `print`, for output to the screen (or to files). You can probably learn enough about its usage from examples in this and subsequent units of the course, but for more information see also the notes on *Formatted Output and Some Text String Manipulation*

Formatted Output and Some Text String Manipulation

A short-cut for printing the value of a variable is to simply enter its name on the last line of a cell:

```
root0
```

```
1.0
```

You can also do this for multiple variables, as with:

```
root0, root1
```

```
(1.0, 4.0)
```

Note that the output is parenthesized: this, as will be explained below, is a *tuple*

21.3 Text variables

Other information can be put into variables, such as strings of text:

```
LastName = "LeMesurier"
FirstName = 'Brenton'
print("Hello, my name is", FirstName, LastName)
```

```
Hello, my name is Brenton LeMesurier
```

Note that either ‘single quotes’ or “double quotes” can be used to surround text, but one must be consistent within each piece of text. I recommend always using double quotes (which are true quotation characters) not single quotes (which are actually apostrophes: for one thing many other languages require this, so it could help to get into the habit).

21.4 Lists

Python has several ways of grouping together information into one variable. We first look at *lists*, which can collect all kinds of information together:

```
coefficients = [2, -10, 8]
name = ["LeMesurier", "Brenton"]
phone = [9535917]
print(coefficients, name, phone)
```

```
[2, -10, 8] ['LeMesurier', 'Brenton'] [9535917]
```

Lists can be combined by “addition”, which is concatenation:

```
name + phone
```

```
['LeMesurier', 'Brenton', 9535917]
```

Individual entries (“elements”) can be extracted from lists; note that **Python always counts from 0**, and indices go in [brackets], not (parentheses) or {braces}:

```
LastName = name[0]
FirstName = name[1]
print(FirstName, LastName)
```

```
Brenton LeMesurier
```

and we can modify list elements this way too:

```
name[1] = 'John'
print(name[1])
print(name)
```

```
John
['LeMesurier', 'John']
```

We can use the list of coefficients to specify the quadratic, and store both roots in a new list.

But let’s shorten the name first, by making “q” a synonym for “coefficients”:

```
q = coefficients
print('The list of coefficients is', q)
```

```
The list of coefficients is [2, -10, 8]
```

```
roots = [(-q[1] - sqrt(q[1]**2 - 4 * q[0] * q[2]))/(2 * q[0]),
         (-q[1] + sqrt(q[1]**2 - 4 * q[0] * q[2]))/(2 * q[0])]
print('The list of roots is', roots)
print('The individual roots are', roots[0], 'and', roots[1])
```

```
The list of roots is [1.0, 4.0]
The individual roots are 1.0 and 4.0
```

See now why I enumerated the roots from 0 previously?

For readability, you might want to “unpack” the coefficients by copying into individual variables, and then use the more familiar formulas above:

```
a = q[0]
b = q[1]
c = q[2]
```

Alternatively one can unpack the elements of a list into separate variables with

```
(a, b, c) = q
```

```
roots = [(-b - sqrt(b**2 - 4 * a * c))/(2 * a),
          (-b + sqrt(b**2 - 4 * a * c))/(2 * a)]
print('The list of roots is again', roots)
```

```
The list of roots is again [1.0, 4.0]
```

21.4.1 The equals sign = creates *synonyms* for lists; not copies

Note that it says above that the statement `q = coefficients` makes `q` is a *synonym* for `coefficients`, not a copy of its values. To see this, note that when we make a change to `q` it also applies to `coefficients` (and vice versa):

```
print("q is", q)
print("coefficients is", coefficients)
q[0] = 4
print("q is now", q)
print("coefficients is now", coefficients)
```

```
q is [2, -10, 8]
coefficients is [2, -10, 8]
q is now [4, -10, 8]
coefficients is now [4, -10, 8]
```

To avoid confusion below, let’s change the value back:

```
coefficients[0] = 2
```

21.4.2 Looking at the end of a list, with negative indices

Python allows you to count backwards from the end of a list, by using negative indices:

- index -1 refers to the last element
- index -k refers to the element k from the end.

For example:

```
digits = [1, 2, 3, 4, 5, 6, 7, 8, 9]
print('The last digit is', digits[-1])
print('The third to last digit is', digits[-3])
```

```
The last digit is 9
The third to last digit is 7
```

This also works with the *Numpy arrays: for vectors, matrices, and beyond* and *Tuples* introduced below.

21.5 Tuples

One other useful kind of Python collection is a **tuple**, which is a lot like a list except that it is **immutable**: you cannot change individual elements. Tuples are denoted by surrounding the elements with parentheses “(…)” in place of the brackets “[...]” used with lists:

```
qtuple = (2, -10, 8)
qtuple
```

```
(2, -10, 8)
```

```
print(qtuple)
```

```
(2, -10, 8)
```

```
qtuple[2]
```

```
8
```

Actually, we have seen tuples before without the name being mentioned: when a list of expressions is put on one line separated by commas, the result is a tuple. This is because when creating a tuple, the surrounding parentheses can usually be omitted:

```
name = "LeMesurier", "Brenton"
print(name)
```

```
('LeMesurier', 'Brenton')
```

Tuples can be concatenated by “addition”, as for lists:

```
name_and_contact_info = name + ('843-953-5730', 'RSS 344')
print(name_and_contact_info)
```

```
('LeMesurier', 'Brenton', '843-953-5730', 'RSS 344')
```

21.6 Naming rules for variables

There are some rules limiting which names can be used for variables:

- The first character must be a letter.
- All characters must be “alphanumeric”: only letters of digits.
- However, the underscore “_” (typed with “shift dash”) is an honorary letter: it can be used where you are tempted to have a space.

Note well: no dashes “-” or spaces, or any other punctuation.

When you are tempted to use a space in a name, such as when the name is a descriptive phrase, it is recommended to use an underscore. (Another option is to capitalize the first letter of each new word: so-called *camelCase* or *UpperCamelCase*.)

21.6.1 Exercise A

It will soon be convenient to group the input data to and output values from a calculation in tuples.

Do this by rewriting the quadratic solving exercise using a tuple “coefficients” containing the coefficients (a, b, c) of a quadratic $ax^2 + bx + c$ and putting the roots into a tuple named “roots”.

Break this up into three steps, each in its own code cell (an organizational pattern that will be important later):

1. Input: create the input tuple.
2. Calculation: use this tuple to compute the tuple of roots.
3. Output: print the roots.

This is only a slight variation of what is done above with lists, but the difference will be important later.

21.7 The immutability of tuples (and also of text strings)

As mentioned above, a major difference from lists is that tuples are **immutable**; their contents cannot be changed: I cannot change the lead coefficient of the quadratic above with

```
qtuple[0] = 4
```

```
-----
TypeError                                Traceback (most recent call last)
Input In [25], in <cell line: 1>()
----> 1 qtuple[0] = 4

TypeError: 'tuple' object does not support item assignment
```


This difference between **mutable** objects like *lists* and **immutable** ones like *tuples* comes up in multiple places in Python. The one other case that we are most likely to encounter in this course is strings of text, which are in some sense “tuples of characters”. For example, the characters of a string can be addressed with indices, and concatenated:

```
language = "Python"
print(f"The initial letter of '{language}' is '{language[0]}')
print(f"The first three letters are '{language[0:3]}')
languageversion = language + ' 3'
print(f"We are using version '{languageversion}')
```

```
The initial letter of 'Python' is 'P'
The first three letters are 'Pyt'
We are using version 'Python 3'
```

Aside: Here a new feature of printing and string manipulation is used, “f-string formatting” (new in Python version 3.6). For details, see the notes on *formatted output and some text string manipulation* mentioned above.

Also as with tuples, one cannot change the entries via indexing; we cannot “lowercase” that name with

```
language[0] = "p"
```

```
-----
TypeError                                Traceback (most recent call last)
Input In [27], in <cell line: 1>()
----> 1 language[0] = "p"

TypeError: 'str' object does not support item assignment
```

21.8 Numpy arrays: for vectors, matrices, and beyond

Many mathematical calculations involve vectors, matrices and other arrays of numbers. At first glance, Python lists and tuples look like vectors, but as seen above, “addition” of such objects does not do what you want with vectors.

Thus we need a type of object that is specifically an *array* of numbers of the same type that can be manipulated like a vector or matrix. There is not a suitable entity for this in the core Python language, but Python has a method to add features using **modules** and **packages**, and the most important one for us is **Numpy**: this provides for suitable numerical arrays through objects of type `ndarray`, and provides tools for working with them, like the function `array()` for creating arrays from lists or tuples. (Numpy also provides a large collection of other tools for numerical computing, as we will see later.)

21.8.1 Importing modules

One way to make Numpy available is to *import* it with just

```
import numpy
```

Then the function `array` is accessed by its “fully-qualified name” `numpy.array`, and we can create an `ndarray` that serves for storing a vector:

```
u = numpy.array([1, 2, 3])
```

```
u
```

```
array([1, 2, 3])
```

```
print(u)
```

```
[1 2 3]
```

Note: As you might have noticed above, displaying the value of a variable by simply typing its name describes it in more detail than the `print` function; sometimes it is a description that could be used to create the object. Thus I will sometimes use both display methods below, as a reminder of the syntax and semantics of Numpy arrays.

As seen above, if we just want that one function, we can import it specifically with the command

```
from numpy import array
```

and then it can be referred to by its short name alone:

```
v = array([4, 5, 6, 7])
```

```
print(v)
```

```
[4 5 6 7]
```

21.8.2 Notes

1. Full disclosure: Python’s core collection of resources does provide another kind of object called an *array*, but we will **never** use that in this course, and I advise you to avoid it: the Numpy `ndarray` type of array is far better for what we want to do! The name “ndarray” refers to the possibility of creating n-dimensional arrays — for example, to store matrices — which is one of several important advantages.
2. There is another add-on package *Pylab*, which contains most of Numpy plus some stuff for graphics (from package *Matplotlib*, which we will meet later, in [Section 8](#)) That is intended to reproduce a Matlab-like environment, especially when used in Spyder, which is deliberately Matlab-like. So you could instead use `from pylab import *`, and that will sometimes be more convenient. However, when you search for documentation, you will find it by searching for `numpy`, not for `pylab`. For example the full name for function `array` is `numpy.array` and once we import Numpy with `import numpy` we can get help on that with the command `help(numpy.array)`.

Beware: this `help` information is sometimes very lengthy, and “expert-friendly” rather than “beginner-friendly”.

Thus, now is a good time to learn that when the the up-array and down-array keys get to the top or bottom of a cell in a notebook, they keep moving to the previous or next cell, skipping past the output of any code cell.

```
help(numpy.array)
```

The function `help` can also give information about a type of object, such as an `ndarray`. Note that `ndarray` is referred to as a *class*; if that jargon is unfamiliar, you can safely ignore it for now, but if curious you can look at the brief notes on [classes, objects, attributes and methods](#)

Beware: this `help` information is even more long-winded, and tells you far more about numpy arrays than you need to know for now! So make use of that down-arrow key.

```
help(numpy.ndarray)
```

21.8.3 Creating arrays (from lists and otherwise)

Numpy arrays (more pedantically, objects of type `ndarray`) are in some ways quite similar to lists, and as seen above, one way to create an array is to convert a list:

```
list0 = [1, 2, 3]
list1 = [4, 5, 6]
array0 = array(list0)
array1 = array(list1)
```

```
list0
```

```
[1, 2, 3]
```

```
array0
```

```
array([1, 2, 3])
```

```
print(list0)
```

```
[1, 2, 3]
```

```
print(array0)
```

```
[1 2 3]
```

We can skip the intermediate step of creating lists and instead create arrays directly:

```
array0 = array([1, 2, 3])
array1 = array([4, 5, 6])
```

Printing makes these seem very similar, though an array is displayed without commas between elements. Note that this is like the style for a single-row matrix.

```
print('list0 =', list0)
print('array0 =', array0)
```

```
list0 = [1, 2, 3]
array0 = [1 2 3]
```

Also, we can extract and modify elements in the same way:

```
print('The first element of list0 is', list0[0])
print('The last element of array1 is', array1[-1])
```

(continues on next page)

(continued from previous page)

```
array0[1] = 7
print('The value of array0 is now', array0)
```

```
The first element of list0 is 1
The last element of array1 is 6
The value of array0 is now [1 7 3]
```

21.8.4 Numpy arrays understand vector arithmetic

Addition and other arithmetic reveal some important differences:

```
print(list0 + list1)
```

```
[1, 2, 3, 4, 5, 6]
```

```
print(array0 + array1)
```

```
[ 5 12  9]
```

```
print(2 * list0)
```

```
[1, 2, 3, 1, 2, 3]
```

```
print(2 * array0)
```

```
[ 2 14  6]
```

Note what multiplication does to lists!

21.8.5 Describing matrices as 2D arrays, or as “arrays of arrays of numbers”

A list can have other lists as its elements, and likewise an array can be described as having other arrays as its elements, so that a matrix can be described as a succession of rows. First, a list of lists can be created:

```
listoflists = [list0, list1]
```

```
print(listoflists)
```

```
[[1, 2, 3], [4, 5, 6]]
```

```
listoflists[1][-1]
```

```
6
```

Then this can be converted to a two dimensional array:

```
matrix = array(listoflists)
```

```
print(matrix)
```

```
[[1 2 3]
 [4 5 6]]
```

```
matrix*3
```

```
array([[ 3,  6,  9],
       [12, 15, 18]])
```

We can also combine arrays into new arrays directly:

```
anothermatrix = array([array1, array0])
```

```
anothermatrix
```

```
array([[4, 5, 6],
       [1, 7, 3]])
```

```
print(anothermatrix)
```

```
[[4 5 6]
 [1 7 3]]
```

Note that we must use the notation `array(...)` to do this; without the function `array()` we would get a *list of arrays*, which is a different animal, and much less fun for doing mathematics with:

```
listofarrays = [array1, array0]
listofarrays*3
```

```
[array([4, 5, 6]),
 array([1, 7, 3]),
 array([4, 5, 6]),
 array([1, 7, 3]),
 array([4, 5, 6]),
 array([1, 7, 3])]
```

21.8.6 Referring to array elements with double indices, or with successive single indices

The elements of a multi-dimensional array can be referred to with multiple indices:

```
matrix[1,2]
```

```
6
```

but you can also use a single index to extract an “element” that is a row:

```
matrix[1]
```

```
array([4, 5, 6])
```

and you can use indices successively, to specify first a row and then an element of that row:

```
matrix[1][2]
```

```
6
```

This ability to manipulate rows of a matrix can be useful for linear algebra. For example, in row reduction we might want to subtract four times the first row from the second row, and this is done with:

```
print('Before the row operation, the matrix is:')
print(matrix)
matrix[1] -= 4 * matrix[0]  # Remember, this is short-hand for matrix[1] = matrix[1] -
↪ 4 * matrix[0]
print('After the row operation, it is:')
print(matrix)
```

```
Before the row operation, the matrix is:
[[1 2 3]
 [4 5 6]]
After the row operation, it is:
[[ 1  2  3]
 [ 0 -3 -6]]
```

Note well the effect of Python indexing starting at zero: the indices used with a vector or matrix are all one less than you might expect based on the notation seen in a linear algebra course.

21.8.7 Higher dimensional arrays

Arrays with three or more indices are possible, though we will not see much of them in this course:

```
arrays_now_in_3D = array([matrix, anothermatrix])
```

```
arrays_now_in_3D
```

```
array([[[ 1,  2,  3],
        [ 0, -3, -6]],

       [[ 4,  5,  6],
        [ 1,  7,  3]]])
```

```
print(arrays_now_in_3D)
```

```
[[[ 1  2  3]
   [ 0 -3 -6]]

 [[ 4  5  6]
   [ 1  7  3]]]
```

Exercise B

Create two arrays, containing the matrices $A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$, $B = \begin{bmatrix} 3 & 0 \\ 2 & 1 \end{bmatrix}$. Then look at what is given by the formula

```
C = A * B
```

and what you get instead with the strange notation

```
D = A @ B
```

Explain in words what is going on in each case!

DECISION MAKING WITH IF, ELSE, AND ELIF

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

22.1 Introduction

To move beyond using Python to execute a simple sequence of commands, two new algorithmic features are needed: *decision making* and *repetition (with variation)*. In this unit we look at decision making, using *conditional statements*; `if` statements.

We first see the simplest case of either doing something or not, depending on some condition; here, avoiding division by zero. The code we will use first is:

```
if b == 0:
    print("Do you really want to try dividing by zero?!")
print(f"{a}/{b} = {a/b}")
```

Now to test it with various values of `a` and `b`.

Initially I will duplicate the code for each test case, but later we will see how to avoid repeating yourself, by either editing the code in the notebook or using commands for repetition.

It is all fine with

```
a = 4
b = 3
```

```
if b == 0:
    print("Do you really want to try dividing by zero?!")
print(f"{a}/{b} = {a/b}")
```

```
4/3 = 1.3333333333333333
```

but not with

```
a = 4
b = 0
```

```
if b == 0:
    print("Do you really want to try dividing by zero?!")
print(f"a/b = {a/b}")
```

Do you really want to try dividing by zero?!

```
-----
ZeroDivisionError                                Traceback (most recent call last)
Input In [5], in <cell line: 3>()
      1 if b == 0:
      2     print("Do you really want to try dividing by zero?!")
----> 3 print(f"a/b = {a/b}")

ZeroDivisionError: division by zero
```

There are several points to note here:

1. Testing for equality is done with a **double equal sign**, to distinguish from assigning a value to the variable at the left, which is what `b = 0` does.
2. Statement like the above `if ...` that control the execution of a subsequent list or *block* of statements ends with a colon, and statements in the following block are indented by **four spaces**. *Avoid using tabs for indentation!* (The JupyterLab and Spyder editors do this indentation automatically when you press “return” after a line of code that ends with a colon.)
3. The end of the collection of controlled statements is indicated simply by the end of that indentation; unlike some other programming languages, there is no `line end`, or `end if` or such. So the second `print` statement is not in the “if block”, and is executed regardless of what happens in the `if` statement.
4. Thus we see that in Python, *indentation has meaning*; it is not just for readability. (Aside: this and the previous point mimic typical English language style for lists, as indeed in the current cell!)
5. This code illustrates a classic bit of bad programming: detecting a problem but then not doing anything about it!

On that last point, the next form of conditional statement specifies actions for both the `True` and `False` cases, using an `else` clause for the latter:

```
a = 4
b = 0
```

```
if b == 0:
    print("You cannot divide by zero! Try changing the value of b and rerun this cell.
    ↵")
else:
    print(f"{a}/{b} = {a/b}")
```

You cannot divide by zero! Try changing the value of b and rerun this cell.

We can break this into pieces, to see first what `b == 0` does:

```
answer = (b == 0)
print(f"The answer is {answer}")
```

The answer is True

Note that logical statements like `b == 0` have value either `True` or `False` — and remember that these are case sensitive; “true” and “false” have no special meaning.

```
if answer:
    print("You cannot divide by zero! Try changing the value of b and rerunning this_
↪cell.")
else:
    print(f"{a}/{b} = {a/b}")
```

You cannot divide by zero! Try changing the value of b and rerunning this cell.

Next, exercise the other “else” option.

Aside: it is good code development practice to use a collection of tests which ensure that every piece of code is executed.

```
a = 4
b = 3
```

```
answer = (b == 0)
print(f"The answer is {answer}")
```

The answer is False

```
if answer:
    print("You cannot divide by zero! Try changing the value of b and rerun this cell.
↪")
else:
    print(f"{a}/{b} = {a/b}")
```

4/3 = 1.3333333333333333

Aside: avoiding code duplication, with the `for` statement

We will learn about commands for repetition in a later unit, but just as a preview, here is an example of how to do a succession of similar things without duplicating lines of code.

Also note the two levels of indentation, each with four spaces.

```
a = 4
for b in (3, 0):
    print(f"With a = {a} and b = {b},")
    if b == 0:
        print("b is 0; you cannot divide by zero!")
    else:
        print(f"{a}/{b} = {a/b}")
```

With a = 4 and b = 3,
4/3 = 1.3333333333333333
With a = 4 and b = 0,
b is 0; you cannot divide by zero!

Here is another way to do the same thing, this time putting the more “normal” case first, which tends to improve readability:

```
a = 4
b = 3
```

```
if b != 0: # Note: This is the notation for "is not equal to".
    print(f"{a}/{b} = {a/b}")
else:
    print("b is 0; you cannot divide by zero!")
```

```
4/3 = 1.3333333333333333
```

22.2 Handling multiple possibilities

More than two possibilities can be handled, using an `elif` clause (short for “else, if”). Let’s see this, while also introducing inequality comparisons:

```
x = 4
```

```
if x > 0:
    print("x is positive")
elif x < 0:
    print("x is negative")
else: # By process of elimination ...
    print("x is zero")
```

```
x is positive
```

22.2.1 Exercise A

Test the above by changing to various other values of `x` and rerunning. As mentioned above, ensure that you exercise all three possibilities.

While experimenting in a notebook (or Python code file), one way to do this is to edit in new values for `x` and then re-run, but for final presentation and submission of your work, do this by one of the methods seen above:

- make multiple copies of the code, one for each chosen value of `x`.
- be more adventurous by working out how to use a `for` statement to run a list of test cases. This is looking ahead to *Iteration with for*.

22.3 There can be as many `elif` clauses as you want.

In the following, remember that `a % b` is the remainder after integer division of `a` by `b`:

```
n = 36
```

```

if n % 10 == 0:
    print(f"{n} is a multiple of ten")
elif n % 5 == 0:
    print(f"{n} is an odd multiple of five")
elif n % 2 == 0:
    print(f"{n} is even, but not a multiple of five.")
else:
    print(f"{n} has no factor of either two or five.")

```

```

36 is even, but not a multiple of five.

```

22.3.1 Exercise B

Again, test all four possibilities by using a suitable collection of values for n .

22.4 Plan before you code!

Start with written preparation and planning of your procedure, and check that with me before creating any Python code.

You can do this on paper or a text file, but as a goal for how best to do things, you could also try to present this planing in a Jupyter notebook; then you could append the Python work to that notebook later.

As part of this planning, select a collection of test cases that explore all the possibilities: keep in mind the goal that each line of code is executed in a least one test case (bearing in mind that `if` statements can skip some lines of code, depending on whether various statements are true or false).

22.4.1 Exercise C. Planning for robust handling of all possible “quadratics” $ax^2 + bx + c = 0$

In the next exercise, we will create Python code that correctly handles the task of finding all the real roots of a quadratic equation $ax^2 + bx + c = 0$ for the input of any real numbers a , b and c .

Before writing any code, we need a written plan, distinguishing all qualitatively different cases for the input, and deciding how to handle each of them. (This planning and decision making will be an important requirement in many later exercises.)

Produce quadratic solving code that is **robust**; handling all possible triples of real numbers a , b and c for the coefficients in equation $ax^2 + bx + c = 0$.

Not all choices of those coefficients will give two distinct real roots, so work out all the possibilities, and try to handle them all.

DEFINING AND USING PYTHON FUNCTIONS

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

23.1 Introduction

Our main objective is to learn how to define our own Python *functions*, and see how these can help us to deal with sub-tasks done repeatedly (with variation) within a larger task.

Typically, core mathematical tasks (like evaluating a function $f(x)$ or solving an equation $f(x) = 0$) will be done within Python functions designed to communicate with other parts of the Python code, rather than getting their input interactively or returning results by displaying them on the screen.

In this course, the notebook approach is prioritized; that makes it easier to produce a single document that combines text and mathematical information, Python code, and computed results, making for a more “literate”, understandable presentation.

Example A. A very simple function for computing the mean of two numbers

To illustrate the syntax, let’s start with a very simple function: computing the mean of two numbers.

```
def mean(a, b):  
    mean_of_a_and_b = (a + b) / 2  
    return mean_of_a_and_b
```

This can be used as follows:

```
mean_of_2_and_6 = mean(2, 6)  
print('The mean of 2 and 6 is', mean_of_2_and_6)
```

```
The mean of 2 and 6 is 4.0
```

23.1.1 Notes

1. The definition of a function begins with the command `def`
2. This is followed by:
 - a name for the function,
 - a parenthesized, comma-separated list of variable names, which are called the input *arguments*,
 - and finally a **colon**, which as always introduces an indented list or *block* of statements; in this case the statements that describe the actions performed by the function.
 - Indentation is done with **four spaces**. (Tabs are also legal but advised against, so please forget that I mentioned them.)
 - The end of the block of code for the function is indicated simply by the end of the indentation; there is no `end` line as in some other programming languages. The last line of the definition is often a `return` statement, but there can also be multiple `return` statements at various places in the code block, or no `return` at all.
3. When using the function, it is given a list of expressions (variable names or values or more elaborate formulas) and the values of these are *copied* to the corresponding internal variables given by the list of input arguments mentioned in the function's `def` line. (However, with lists and arrays, the idea of *copying* will require clarification!)
4. As soon as the function gets to a statement beginning with `return`, it evaluates the expression on that line, and ends, sending back this as the value of the function.
5. The name used for a variable into which the value of a function is assigned (here, `mean_of_2_and_6`) does not have to be the same as the name used internally in the `return` statement (here, `mean_of_a_and_b`).

In general, what follows `return` is an expression (formula) that is evaluated to get the value that is then sent back; listing the name of one or more variables is just one simple way to do this. For example, the above function `mean` could instead be defined with

```
def mean(a, b):  
    return (a + b)/2
```

Also, multiple values can be given in a `return` line; they are then output as a *tuple*.

```
def mean_and_difference(a, b):  
    """Compute the mean and difference of two numbers."""  
    return (a + b)/2, b - a
```

```
mean_and_difference(2, 5)
```

```
(3.5, 3)
```

23.2 Variables “created” inside a function definition are local to it

A more subtle point: all the variables appearing in the function's `def` line (here `a` and `b`) and any created inside with an assignment statement (here just `mean_of_a_and_b`) are purely *local* to the function; they do not exist outside the function. For that reason, when you call a function, you have to do something with the return value, like assign it to a variable (as done here with `mean_of_a_and_b`) or use it as input to another function (see below).

Aside: there is a way that a variable can be shared between a function and the rest of the file in which the function definition appears; so-called *global variables*, using `global` statements. However, it is generally good practice to avoid them as much as possible, so I will do so in these notes.

To illustrate this point about local variables, let us look for the values of variables `a` and `mean_of_a_and_b` in the code after the function is called:

```
mean310 = mean(3, 10)
```

```
print('After using function mean:')
print(f'a = {a}')
```

After using function mean:

```
-----
NameError                                Traceback (most recent call last)
Input In [6], in <cell line: 2>()
      1 print('After using function mean:')
----> 2 print(f'a = {a}')
```

NameError: name 'a' is not defined

```
print('mean_of_a_and_b =', mean_of_a_and_b)
```

```
-----
NameError                                Traceback (most recent call last)
Input In [7], in <cell line: 1>()
----> 1 print('mean_of_a_and_b =', mean_of_a_and_b)
```

NameError: name 'mean_of_a_and_b' is not defined

```
print(f'mean310 = {mean310}')
```

```
mean310 = 6.5
```

The same name can even be used locally in a function and also outside it in the same file; they are different objects with independent values:

```
def double(a):
    print(f"At the start of function 'double', a = {a}")
    a = 2 * a
    print(f"A bit later in that function, a = {a}")
    return a
a = 1
print(f"Before calling function 'double', a = {a}")
b = double(a)
print(f"After calling function 'double', b = {b}, but a = {a} again.")
```

```
Before calling function 'double', a = 1
At the start of function 'double', a = 1
A bit later in that function, a = 2
After calling function 'double', b = 2, but a = 1 again.
```

Warning about keeping indentation correct: The line `a = 1` is after the function definition, not part of it, as indicated by the reduced indentation; however Python code editing tools like JupyterLab and Spyder will default to indenting each new line as much as the previous one when you end a line by typing “return”. Thus, when typing in a function `def`, it is important to manually reduce the indentation (“dedent”) at the end of the definition. The same is true for all statements that end with a colon and so control a following block of code, like `if`, `else`, `elif` and `for`.

Example B. More on multiple output values, with tuples

Often, a function computes and returns several quantities; one example is a function version of our quadratic equation solver, which takes three input parameters and computes a pair of roots. Here is a very basic function for this, ignoring for now possible problems like division by zero:

```
from math import sqrt

def solve_quadratic(c_2, c_1, c_0):
    """Compute the roots of a quadratic,
    where c_i is the coefficient of x^i
    """
    discriminant = c_1**2 - 4 * c_0 * c_2
    root0 = (-c_1 + sqrt(discriminant))/(2 * c_2)
    root1 = (-c_1 - sqrt(discriminant))/(2 * c_2)
    return root0, root1
```

Now what is returned is a *tuple*, and it can be stored into a single variable:

```
roots = solve_quadratic(2, -10, 8)
print(f"Variable 'roots' has value {roots}")
print(f"One root is {roots[0]} and the other is {roots[1]}")
```

```
Variable 'roots' has value (4.0, 1.0)
One root is 4.0 and the other is 1.0
```

However, it is often convenient to store each returned value into a separate variable, using tuple notation at left in the assignment statement:

```
(rootA, rootB) = solve_quadratic(2, -10, 8)
print(f"One root is {rootA} and the other is {rootB}")
```

```
One root is 4.0 and the other is 1.0
```

23.3 Note: With tuples, parentheses are optional

When tuples were introduced in the section on *Python variables, etc.*, they were described as “a parenthesized list of values separated by commas”, but note that above, no parentheses were used: the `return` line was

```
return root0, root1
```

not

```
return (root0, root1)
```

though the latter version would also work.

In fact, you can always omit the parentheses when specifying a tuple, just giving a comma-separated list of values, even on the left side of an assignment statement. Thus, the above example could also be done like this:

```
rootA, rootB = solve_quadratic(2, -10, 8)
print(f"One root is {rootA} and the other is {rootB}")
```

```
One root is 4.0 and the other is 1.0
```

23.4 Single-member tuples: not an oxymoron

Tuples can have a single member, but then to make it clear to Python that it is a tuple, there must always be a comma after that sole element. Compare:

```
tuple_a = (1,)
print(tuple_a)
```

```
(1,)
```

```
tuple_b = 2,
print(tuple_b)
```

```
(2,)
```

```
not_a_tuple_c = (3)
print(not_a_tuple_c)
```

```
3
```

23.5 Documenting functions with triple quoted comments, and `help`**

Note that the code blocks for some of the functions above start with a comment surrounded by a triplet of quote characters at each end, and this sort of comment can continue over multiple lines.

In addition to making it easier to have long comments, this sort of comment provides some *self-documentation* for the function — not just in the code for the function, but also in the Python `help` system:

```
help(mean_and_difference)
```

```
Help on function mean_and_difference in module __main__:
```

```
mean_and_difference(a, b)
    Compute the mean and difference of two numbers.
```

```
help(solve_quadratic)
```

```
Help on function solve_quadratic in module __main__:
```

```
solve_quadratic(c_2, c_1, c_0)
    Compute the roots of a quadratic,
    where c_i is the coefficient of x^i
```

About the reference to module `__main__`: that is the standard name for code that is not explicitly part of any other module, such as anything defined in the current file rather than imported from elsewhere. Compare to what we get when a function comes from another module:

```
help(sqrt)
```

```
Help on built-in function sqrt in module math:
```

```
sqrt(x, /)
    Return the square root of x.
```

As you might expect, all objects provided by standard modules like `math` and `numpy` have some documentation provided; `help` is useful in cases like this where you cannot see the Python code that defines the function.

When a function `def` lacks such a *self-documentation* comment, `help` still tells us something; the syntax for using it, and where it come from:

```
help(mean)
```

```
Help on function mean in module __main__:
```

```
mean(a, b)
```

23.6 Exercise A. A robust function for solving quadratics

Refine the above function `solve_quadratic` for solving quadratics, and make it **robust**; handling all possible input triples of real numbers in a “reasonable” way. (If you have done Exercise C of the section {doc}(decisions-with-if-else-elif, this is essentially just integrating that code into a function.)

Not all choices of those coefficients will give two distinct real roots, so work out all the possibilities, and try to handle them all.

1. Its input arguments are three real (floating point) numbers, giving the coefficients a , b , and c of the quadratic equation $ax^2 + bx + c = 0$.
2. It always **returns** a result (usually a pair of numerical values for the roots) for any “genuine” quadratic equation, meaning one with $a \neq 0$.
3. If the quadratic has real roots, these are output with a `return` statement — no `print` commands in the function.
4. In cases where it is not a genuine quadratic, or there are no real roots, return the special value `None`.
5. As an **optional** further refinement: in the “prohibited” case $a = 0$, have it produce a custom error message, by “raising an exception”. We will learn more about handling “exceptional” situations later, but for now you could just use the command:

```
raise ZeroDivisionError("The coefficient 'a' of x^2 cannot have the value zero.")
```

or

```
raise ValueError("The coefficient 'a' of x^2 cannot have the value zero.")
```

Ultimately put this in a cell in a Jupyter notebook (suggested name: “exercises_on_functions.ipynb”); if you prefer to develop it with Spyder, I suggest the filename “quadratic_solver_c.py”

23.6.1 Testing

Test and demonstrate this function with a list of test cases, including:

1. $2x^2 - 10x + 8 = 0$
2. $x^2 - 2x + 1 = 0$
3. $x^2 + 2 = 0$
4. $x^2 + 6x + 25 = 0$
5. $4x - 10 = 0$

23.6.2 Notebook organization

To start developing good notebook organizational practice:

- As always, start with a *title cell* giving a title, your name and the date of *most recent revision*.
- Follow this by an *introduction cell* giving a brief description of what the notebook does — see above; follow the links here!
- Put any `import` statements straight after the introduction, before any other Python code.
- Define the function in one cell (more generally, define each function in its own cell).
- Follow the function definition[s] with the test cases, each in its own cell.
- If anything noteworthy arises in a test case, add comments on this in a Markdown cell after that test case cell.

23.7 Keyword arguments: specifying input values by name

Sometimes a function has numerous input arguments, and then it might be hard to remember what order they go in.

Even with just a few arguments, there can be room for confusion; for example, in the above function `solve_quadratic` do we give the coefficients in order `c_2`, `c_1`, `c_0` as for $c_2x^2 + c_1x + c_0 = 0$, or in order `c_0`, `c_1`, `c_2` as for $c_0 + c_1x + c_2x^2 = 0$?

To improve readability and help avoid errors, Python has a nice optional feature of specifying input arguments *by name*; they are then called *keyword arguments*, and can be given in any order.

For example:

```
moreroots = solve_quadratic(c_1=3, c_2=1, c_0=-10)
print(moreroots)
```

```
(2.0, -5.0)
```

When you are specifying the parameters by name, there is no need to have them in any particular order. For example, if you like to write polynomials “from the bottom up”, as with $-10 + 3x + x^2$, which is $c_0 + c_1x + c_2x^2$, you could do this:

```
sameroots = solve_quadratic(c_0=-10, c_1=3, c_2=1)
print(sameroots)
```

```
(2.0, -5.0)
```

23.8 Functions as input to other functions

In mathematical computing, we often wish to define a (Python) function that does something with a (mathematical) function. A simple example is implementing the basic difference quotient approximation of the derivative

$$f'(x) = Df(x) \approx \frac{f(x+h) - f(x)}{h}$$

with a function `Df_approximation`, whose input will include the function f as well as the two numbers x and h .

Python makes this fairly easy, since Python functions, like numbers, can be the values of variables, and given as input to other functions in the same way: in fact the statement

```
def f(...): ...
```

creates variable `f` with this function as its value.

So here is a suitable definition:

```
def Df_approximation(f, x, h):
    return (f(x+h) - f(x))/h
```

and one way to use it is as follows:

```
def p(x):
    return 2*x**2 - 10*x + 8
x0 = 1
h = 1e-4

Df_x0_h = Df_approximation(p, x0, h)
print(f'Df({x0}) is approximately {Df_x0_h}')
```

```
Df(1) is approximately -5.999800000004996
```

A bit more about keyword arguments: they can be mixed with positional arguments, but once an argument is given in keyword forms all later ones must be also. Thus it works to do this:

```
Dfxh = Df_approximation(p, x=x0, h=1e-6)
print(f'Df({x0}) is approximately {Dfxh}')
```

```
Df(1) is approximately -5.99999799888451
```

but the following fails:

```
Dfxh = Df_approximation(p, x=2, 0.000001)
print(f'Df(2) is approximately {Dfxh}')
```

```
Input In [27]
    Dfxh = Df_approximation(p, x=2, 0.000001)
                                     ^
SyntaxError: positional argument follows keyword argument
```

23.9 Optional input arguments and default values

Sometimes it makes sense for a function to have default values for arguments, so that not all argument values need to be specified. For example, the value $h = 10^{-8}$ is in some sense close to “ideal”, so let us make that the default, by giving h a “suggested” value as part of the function’s (new, improved) definition:

```
def Df_approximation(f, x, h=1e-8):
    return (f(x+h) - f(x))/h
```

The value for input argument h can now optionally be omitted when the function is used, getting the same result as before:

```
Df_x0_h = Df_approximation(p, x0)
print(f'Df({x0}) is approximately {Df_x0_h}')
```

```
Df(1) is approximately -5.999999963535174
```

or we can specify a value when we want to use a different one:

```
big_h = 0.01
Df_x0_h = Df_approximation(p, x0, big_h)
print(f'Using h={big_h}, Df({x0}) is approximately {Df_x0_h}')
```

```
Using h=0.01, Df(1) is approximately -5.979999999999919
```

23.9.1 Arguments with default values must come after all others in the `def`

When default values are given for some arguments but not all, these must appear in the function definition after all the arguments without default values, as is the case with $h=1e-8$ above.

23.9.2 Exercise B. A function with function inputs (and exceptions)

A usually more accurate formula for approximating derivatives is the *Centered Difference Rule* $Df(x) \approx \frac{f(x+h)-f(x-h)}{2h}$

1. Write a function that uses this, with usage

```
Dfxh = Df_CD_approximation(f, x, h)
```

2. Give h the default value 10^{-12}
3. Raise an exception if the forbidden value $h = 0$ is used.
4. Choose and implement a few test cases.

You may do this all in the same notebook as above.

If you wish to also do this in a Python code file, use a different file than the quadratic solving exercise; maybe named “centered_differences.py”.

23.10 Optional topic: anonymous functions, a.k.a. lambda functions

Note: Some students might be interested in “anonymous functions”, also known as “lambda functions”, so here is a brief introduction. However, this topic is not needed for this course; it is only a convenience, and if you are new to computer programming, I suggest that you skip this section for now.

One inconvenience in the above example with `Df_approximation` is that we had to first put the values of each input argument into three variables. Sometimes we would rather skip that step, and indeed we have seen that we could put the numerical argument values in directly:

```
Df_x_h = Df_approximation(p, 4, 1e-4)
print(f'The derivative is approximately {Df_x_h}')
```

```
The derivative is approximately 6.000200000002565
```

However, we still needed to define the function first, and give it a name, `p`.

If the function is only ever used this one time, we can avoid this, specifying the function directly as an input argument value to the function `Df_approximation`, without first naming it.

This is done with what is called an *anonymous function*, or for mysterious historical reasons, a *lambda function*.

For the example above, we can do this:

```
Df_x_h = Df_approximation(lambda x: 2*x**2 - 10*x + 8, 4, 1e-4)
print(f'The derivative is approximately {Df_x_h}')
```

```
The derivative is approximately 6.000200000002565
```

We can even do it all in a single line by composing two functions, `print` and `Df_approximation`:

```
print(f'The derivative is approximately {Df_approximation(lambda x: 2*x**2 - 10*x + 8,
↪ 4, 1e-4)}')
```

```
The derivative is approximately 6.000200000002565
```

Here, the expression


```
lambda x: 2*x**2 - 10*x + 8
```

creates a function that is mathematically the same as function `p` above; it just has no name.

In general, the form is a single-line expression with four elements:

- It starts with `lambda`
- next is a list of input argument names, separated by commas if there are more than one (but no parentheses!?)
- then a colon
- and finally, a formula involving the input variables.

We can, if we want, assign a lambda function to a variable, so we could have defined `p` as

```
p = lambda x: 2*x**2 - 10*x + 8
```

though I am not sure if that has any advantage over doing this with `def`:

```
def p(x): return 2*x**2 - 10*x + 8
```

As an example of that, and also of having a lambda function that returns multiple values, here is yet another quadratic equation solver:

```
solve_quadratic = lambda a, b, c: ( (-b + sqrt(b**2 - 4*a*c))/(2 * a), (-b -  
sqrt(b**2 - 4*a*c))/(2 * a) )
```

```
print(f'The roots of 2*x**2 - 10*x + 8 are {solve_quadratic(2, -10, 8)}')
```

```
The roots of 2*x**2 - 10*x + 8 are (4.0, 1.0)
```

Anonymous functions have most of the fancy features of functions created with `def`, with the big exception that they must be defined on a single line. For example, they also allow the use of keyword arguments, allowing the input argument values to be specified by keyword in any order. It is also possible to give default values to some arguments at the end of the argument list.

To show off a few of these refinements:

```
print(solve_quadratic(b=-10, a=2, c=8))
```

```
(4.0, 1.0)
```

```
Df_approximation = lambda f, x, h=1e-8: (f(x+h) - f(x))/h
```

```
Df_approximation(sqrt, 4.0)
```

```
0.24999997627617176
```

```
Df_approximation(sqrt, 4.0, 0.01)
```

```
0.249843945007866
```

ITERATION WITH FOR

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

24.1 Introduction

The last fundamental tool for describing algorithms is *iteration* or “looping”: tasks that repeat the same sequence of actions repeatedly, with possible variation like using different input values at each repetition.

In Python — as in most programming languages — there are two versions:

- when the number of iterations to be done is determined before we start — done with `for` loops;
- when we must decide “on the fly” whether we are finished by checking some conditions as part of each repetition — done with `while` loops.

This Unit covers the first case, of `for` loops; the more flexible `while` loops will be introduced in the section on *Iteration with while*.

24.2 Repeating a predetermined number of times, with `for` loops

We can apply the same sequence of commands for each of a list of values by using a `for` statement, followed by an indented list of statements.

24.2.1 Example A

We can compute the square roots of several numbers. Here I give those numbers in a tuple (i.e., in parentheses). They could just as well be in a list [i.e., in brackets.]

```
import math
```

```
for x in (1, 2, 4, 9, 20, 1000000, 3141592653589793, 121):  
    square_root_of_x = math.sqrt(x)  
    print(f'The square root of {x:g} is {square_root_of_x:g}')
```

```
The square root of 1 is 1
The square root of 2 is 1.41421
The square root of 4 is 2
The square root of 9 is 3
The square root of 20 is 4.47214
The square root of 1e+06 is 1000
The square root of 3.14159e+15 is 5.60499e+07
The square root of 121 is 11
```

Aside A: Using “import module” instead of “from module import object”

Note the way that I handled importing from a module this time: the `import` statement just specifies that the module is wanted, and then I refer to each item used from it by its “full name”, prefixed by the module name, connected with a period.

This is often the recommended way to do things in larger collections of code, because it makes immediately clear where the object (here `sqrt`) comes from, without having to look up to the top of the file to check the `import` statement. On the other hand, the `from module import item` syntax is slightly more compact, and makes things read more like usual mathematical notation. So especially with mathematical objects like `sqrt` or `pi` where the name is fairly unambiguous, I typically use this shorter form.

Aside B: Controlling the appearance of printed output

Note also the new way that I inserted the values of the variables `x` and `square_root_of_x` into the string of text to be printed. Each `:g` appended in the `{...}` specified using the general (“g”) format for a real number: with this, large enough values (like 1000000) get displayed in scientific notation (with an exponent) to save space.

Later we will see refinements of this output format control, like specifying how many significant digits or correct decimal places are displayed, and ensuring that values on successive lines appear neatly aligned in columns: for more information, see the [supplementary notes on formatted output and text string manipulation](#).

24.3 Repeating for a range of equally spaced integers with `range()`

One very common choice for the values to use in a `for` loop is a range of consecutive integers, or more generally, equally spaced integers. The first n natural numbers are given by `range(n)` — remember that for Python, the natural numbers start with 0, so this range is the *semi-open interval of integers* $[0, n) = \{i : 0 \leq i < n\}$, and so n is the first value *not* in the range!

24.3.1 Example B

Let’s combine this with a previous example:

```
for n in range(8):
    if n % 4 == 0:
        print(f'{n} is a multiple of 4')
    elif n % 2 == 0:
        # A multiple of 2 but not of 4, or we would have stopped with the above "match
        ↪ ".
        print(f'{n} is an odd multiple of 2')
    else:
        print(f'{n} is odd')
```

```

0 is a multiple of 4
1 is odd
2 is an odd multiple of 2
3 is odd
4 is a multiple of 4
5 is odd
6 is an odd multiple of 2
7 is odd

```

24.4 Ranges that start elsewhere

To specify a range of integers starting at integer m instead of at zero, use

```
range(m, n)
```

Again, the terminal value n is the first value *not* in the range, so this gives $[m, n) = \{i : m \leq i < n\}$

24.4.1 Example C

```

for i in range(10, 15):
    print(f'{i} cubed is {i**3}')

```

```

10 cubed is 1000
11 cubed is 1331
12 cubed is 1728
13 cubed is 2197
14 cubed is 2744

```

Aside C: yet more on formatting printed output

Note that in the stuff to insert into the text string for printing, each item can be an *expression* (a formula), which gets evaluated to produce the value to be printed.

24.5 Ranges of equally-spaced integers

The final, most general use of the function `range` is generating integers with equal spacing other than 1, by giving it three arguments:

```
range(start, stop, increment)
```

So to generate just even integers, specify a spacing of two:

24.5.1 Example D

```
for even in range(0, 10, 2):  
    print(f'2^{even} = {2**even}')
```

```
2^0 = 1  
2^2 = 4  
2^4 = 16  
2^6 = 64  
2^8 = 256
```

24.5.2 Brief Exercise A

Even though the first argument has the “default” of zero, I did not omit it here: what would happen if I did so?

24.6 Decreasing sequences of integers

We sometimes want to count down, which is done by using a negative value for the increment in function `range`.

24.6.1 Example E

Before running the following code, work out what it will do — this might be a bit surprising at first.

```
for n in range(10, 0, -1):  
    print(f'{n} seconds')
```

```
10 seconds  
9 seconds  
8 seconds  
7 seconds  
6 seconds  
5 seconds  
4 seconds  
3 seconds  
2 seconds  
1 seconds
```

24.6.2 Brief Exercise B

What is the last output, and why?

24.6.3 Example F: The first n factorials

I will illustrate two methods; with and without using an array to store the values.

```
# We first need a value for n.
# It will be the input argument to the function for each method.
n = 5
```

```
"""
Method 1: producing a numpy array of the factorials after displaying them one at a
time.

We need to create an array of the desired length before we compute the values that go
into it,
and one strategy is to create an initially empty array.
By default array elements are real numbers rather than integers,
so here see how to specify integers, with optional argument `dtype=int`)
Aside: another option is complex numbers: `dtype=complex`.
"""

import numpy

# Create a numpy array of n integers, values not yet specified:
factorials = numpy.empty(n, dtype=int)

factorials[0] = 1
print(f"0! = {factorials[0]}")
for i in range(1,n):
    factorials[i] = factorials[i-1] * i
    print(f"{i}! = {factorials[i]}")
print(f"The whole array is {factorials}")
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
The whole array is [ 1  1  2  6 24]
```

Note that if we just want to print them one at a time inside the `for` loop, we do not need the array; we can just keep track of the most recent value:

```
"""
Method 2: storing just the most recent value[s] needed.
"""
i_factorial = 1
print(f"0! = {i_factorial}")
for i in range(1,n):
    i_factorial *= i # Note: this "*" notation gives a short-hand for "i_factorial
    = i_factorial * i"
    print(f"{i}! = {i_factorial}")
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
```

Aside C: The shorthands +=, *=, -= etc.

In the above code, the notation

```
`i_factorial *= i`
```

means

```
`i_factorial = i_factorial * i`.
```

The same pattern works for many arithmetic operators, so that for example

```
sum += f(x)*h
```

means

```
sum = sum + f(x)*h
```

which could be useful in computing a sum to approximate a definite integral.

Apart from the slight space-saving, this is an example of the **Don't Repeat Yourself** principle, and can both improve readability and help to avoid mistakes. For example, with a more complicated expression like

```
A[2*i-1, 3*j+i+4] = A[2*i-1, 3*j+i+4] + f(i, j)
```

you might have to look carefully to check that the array reference on each side is the same, whereas that is made clear by saying

```
A[2*i-1, 3*j+i+4] += f(i, j)
```

Also, if that weird array indexing has to be changed, say to

```
A[2*i-1, 4*j+i+3] += f(i, j)
```

one only has to make that change in one place.

Exercise C

Write a Python function that inputs a natural number n , and with the help of a `for` loop, computes and prints the first n Fibonacci numbers. Note that these are defined by the formulas

$$F_0 = 1$$

$$F_1 = 1$$

$$F_i = F_{i-1} + F_{i-2} \text{ for } i \geq 2$$

where I count from zero, because Python.

For now it is fine for your function to deliver its output to the screen with function `print()` and not use any `return` line; we will come back to this issue below.

Follow method 2 above, without the need for an array.

Plan before you code. Before you create any Python code, work out the mathematical, algorithmic details of this process and write down your plan in a mix of words and mathematical notation — and then check that with me before you proceed. My guideline here is that this initial written description should make sense even to someone who knows little or nothing about Python or any other programming language.

One issue in particular is how to deal with the first two values; the ones not given by the recursion formula $F_i = F_{i-1} + F_{i-2}$. In fact, this *initialization* is often an important first step to deal with when designing an iterative algorithm.

ITERATION WITH WHILE

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

25.1 Introduction

The last fundamental tool for describing algorithms is *iteration* or “looping”: tasks that repeat the same sequence of actions repeatedly, with possible variation like using different input values at each repetition.

The section on *Iteration with for* covered the easier case where the number of iterations to be done is determined before we start; now we consider the case where we must decide “on the fly” whether the iteration is finished, by checking some conditions as part of each repetition; this is usually done with `while` loops.

25.2 Repeating an initially unknown number of times, with `while` loops

Often, calculating numerical approximate solutions follows a pattern of iterative improvement, like

1. Get an initial approximation.
2. Use the best current approximation to compute a new, hopefully better one.
3. Check the accuracy of this new approximation.
4. If the new approximation is good enough, stop — otherwise, repeat from step 2.

For this, a `while` loop can be used. Its general meaning is:

```
while "some logical statement is True":  
    repeat this  
    and this  
    and so on  
    when done with the last indented line, go back to the "while" line and check again  
This less indented line is where we continue after the "while" iteration is finished.
```

25.2.1 Example A: Computing cube roots, quickly

We are now ready for illustrations that do something more mathematically substantial: computing cube roots using only a modest amount of basic arithmetic. For now this is just offered as an example of programming methods, and the rapid success might be mysterious, but is explained in a numerical methods course like Math 245. Also, the phrase “backward error” should be familiar to students of numerical methods.

Note how the backward error allows us to check accuracy without relying on the fact that — in this easy case — we already know the answer. Change from ‘a=8’ to ‘a=20’ to see the advantage!

```
# We are going to approximate the cube root of a:
a = 8

# A first very rough approximation:
root = 1

# I will tolerate an error of this much:
error_tolerance = 1e-8
# Aside to students in a numerical course: this is a _backward error_ specification.

# The answer "root" should satisfy root**3 - a = 0, so check how close we are:
while abs(root**3 - a) > error_tolerance:
    root = (2*root + a/root**2)/3
    print(f'The new approximation is {root:20.16g}, with backward error of
    ↪{abs(root**3 - a):e}')
print('Done!')
print(f'The cube root of {a:g} is approximately {root:20.16g}')
print(f'The backward error in this approximation is {abs(root**3 - a):.2e}')
```

```
The new approximation is      3.333333333333333, with backward error of 2.903704e+01
The new approximation is      2.462222222222222, with backward error of 6.927316e+00
The new approximation is      2.081341247671579, with backward error of 1.016332e+00
The new approximation is      2.003137499141287, with backward error of 3.770908e-02
The new approximation is      2.000004911675504, with backward error of 5.894025e-05
The new approximation is      2.000000000012062, with backward error of 1.447429e-10
Done!
The cube root of 8 is approximately      2.000000000012062
The backward error in this approximation is 1.45e-10
```

Aside D: I have thrown in some more refinements of output format control, “:20.16g”, “:e” and “:.2e”. If you are curious, you could try to work out what they do from these examples, or read up on this, for example in the [notes on formatted output](#). But that is not essential, at least for now.

25.2.2 Example B: computing and printing the factorials less than N

As a variant of *Example F: The first n factorials* in the previous section, if we want to compute the all factorials that are less than N, we do not know in advance how many there are, which is a problem with a `for` loop.

Thus, in place of the `for` loop used there, we can do this:

```
N = 1000
```

```

"""
Compute and print all factorials less than N
"""
i = 0
i_factorial = 1
print(f"{i}! = {i_factorial}")
while i_factorial < N:
    i += 1
    i_factorial *= i
    if i_factorial < N: # We have to check again, in case the latest value overshoots
        print(f"{i}! = {i_factorial}")

```

```

0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720

```

If we want to store all the values, we cannot create an array of the correct length in advance, as was done in Example F. This is one place where Python *lists* have an advantage over Numpy *arrays*; lists can be extended incrementally. Also, the way we do this introduces a new kind of Python programming tool: a *method* for transforming an *object*.

25.3 Appending to lists, and our first use of Python *methods*

In an exercise like the above, it might be nice to accumulate a list of all the results, but the number of them is not known in advance, so the array creation strategy seen in Example F cannot be used.

This is one place where Python lists have an advantage over Numpy arrays; lists can be extended incrementally. Also, the way we do this introduces a new kind of Python programming tool: a *method* for transforming an object. The general syntax for methods is

```
object.method(...)
```

which has the effect of transforming the object, and can take a tuple of arguments, or none. Thus, it is sort of like

```
object = method(object, ...)
```

but for one thing, avoids repetition of the object name.

Aside E: A taste of object-oriented programming. This is our first encounter with the notation and concepts of *object-oriented programming*, which is so important in languages like **Java** and **C++**. A *method* is a special kind of function [here `append()`] which does things like transforming an *object* [here the list `factorials`].

This course will use only little bits of this object-oriented programming style, but Python has a full collection of tools for it, which CSCI students in particular will probably appreciate.

25.3.1 Example C: creating and appending to a list

We start with an empty list and then append values with the method `.append()`.

```
listOfPrimes = [] # An empty list
print(listOfPrimes)
listOfPrimes.append(2)
print(listOfPrimes)
listOfPrimes.append(3)
print(listOfPrimes)
```

```
[]
[2]
[2, 3]
```

25.3.2 Example D: Storing a list of the values of the factorials less than N

Now we use this new list manipulation tool to create the desired list of factorial values: creating a list of all values $i!$ with $i! < N$.

```
"""
Collecting a Python list of all the factorials less than N.
"""
factorials = [] # Start with an empty list
i = 0
i_factorial = 1
print(f"{i}! = {i_factorial}")
factorials.append(i_factorial)
while i_factorial < N:
    i += 1
    i_factorial *= i
    if i_factorial < N: # We have to check again, in case the latest value overshoots
        print(f"{i}! = {i_factorial}")
        factorials.append(i_factorial)
print()
print(f"The list of all factorials less than {N} is {factorials}")
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
```

```
The list of all factorials less than 1000 is [1, 1, 2, 6, 24, 120, 720]
```

Note: Of course, the list could then be converted to an array with

```
factorials = numpy.array(factorials)
```

if having an array is useful later.

25.3.3 Exercise A: Fibonacci numbers less than N

Write a Python function that inputs a natural number N , and with the help of a `while` loop, computes and prints in turn each Fibonacci number less than or equal to N .

For now the values are only printed, and so one does not need to store them all; only a couple of the most recent ones.

Note well; this is all $F_i \leq N$, not the Fibonacci numbers up to F_N . Thus we do not know how many there are initially: this is the scenario where `while` loops are more natural than `for` loops.

Written planning. Again, start by working out and writing down your mathematical plan, and check it with me before creating any Python code.

25.3.4 Exercise B: all output via a `return` statement; no `print` to screen in the function

Modify your function from the previous exercise to cumulate the needed Fibonacci numbers in a Python list, and `return` this list. This time, your function itself should not `print` anything: instead, your file will display the results with a single `print` function after invoking the function.

NOTE: This approach of separating the **calculations** in a function from subsequent **display of results** is the main way that we will arrange things from now on.

CODE FILES, MODULES, AND AN INTEGRATED DEVELOPMENT ENVIRONMENT

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

26.1 Introduction

So far, we have worked on each topic within a single Jupyter notebook.

However, in some situations it is better to put some Python code into a separate file, sometimes called a *module*, and then either execute that code directly, or access it from a notebook or from another code file. We have already used a few modules like `math` in a way that illustrates one advantage: gathering Python functions and variables that can then be reused by various notebooks without reproducing the same code in each one.

That will be useful when you have functions that you wish to use repeatedly within different tasks and from different notebooks or different Python code files; for example if you build a collection of Python functions for common linear algebra calculations.

26.2 Integrated Development Environments: Spyder et al

To work with files of Python code and create modules to use from within notebooks, it is convenient to use a software tool that supports creating and editing such files and then running the code: a so-called *Integrated Development Environment*, or “IDE”.

For that task, these notes describe the use of [Spyder](#), [from “Scientific PYthon Development enviRonment”] which is included in Anaconda and can be opened from the Anaconda Navigator. However there are several other widely used alternatives, such as the fairly rudimentary [IDLE](#) and the more advanced [PyCharm](#). One reason for preferring Spyder over IDLE is that Spyder integrates the “Scientific Python” packages Numpy, Matplotlib and Scipy that we will be using.

26.2.1 Exercise A. testing and demonstrations in a Python code file

To start learning the use of the Spyder IDE, we will reproduce Exercise A of the section on *functions*.

1. Open Spyder (from Anaconda-Navigator).
2. In Spyder, create a new file, using menu “File”, item “New File ...”.
3. Save this file into the folder numerical-methods” (or whatever you named it) for this course, using the name “`exerciseA.py`”.
4. Spyder puts some stuff in the newly created “empty” file, in particular a “triple quoted comment” near the top. Edit this comment to include a **title**, **your name**, the **date** and some **description**, and maybe some contact information. Mine would look something like:

```
""" For Exercise A @author: Brenton LeMesurier <lemesurierb@cofc.edu> Last
revised January 0, 1970 """
```

This special triple quoting notation is used for introductory comments and documentation for a file, as well as for functions; we will see one nice use of this soon, with the function help.

5. Immediately after these opening comments, copy in any `import` statements needed by the function `solve_quadratic` in Exercise 3A.

Note: It is good practice for all imports to be near the top of a code file or notebook, straight after any introductory text.

6. Copy the code for the function `solve_quadratic` developed in Exercise 3A into this file, below the `import` statement (if any).
7. Then copy in the code for the test cases after the function definition.
8. Run the code in the folder, using the “Run” button atop Spyder (it looks like a triangular “Play” icon).

Aside: For more substantial Python code development, it can be better to first develop the code in a file like this, using Spyder (or another IDE), and then copy the working code into a notebook for presentation; that allows you to make use of more powerful testing and debugging tools.

On the other hand, there is a potential drawback with putting the function definition and test cases in one Python file: if an error (“exception”) occurs for any test case, the rest of the file is not executed. For now, avoid this by having at most one such exceptional case, and putting it last.

We will later learn several better ways of handling this sort of situation — one seen already is the above approach of testing from separate cells in a notebook.

26.2.2 Exercise B. using code from a Python code file with command `import`

Functions (and variables) defined in a code file can be used from a notebook or from another code file by importing: effectively we have created a *module* `exercise_A`, and can access the function `solve_quadratic` defined there from a notebook by using the command `from exercise_A import solve_quadratic`

Note that the base-name of the file is now being used as a variable name — that is why code file names should follow the same naming rules as variable names.

To experiment a bit more with defining and using your own modules:

1. Create a notebook `modules.ipynb` — you can cut and paste from this notebook, but leave out irrelevant stuff: copy mainly the headings and the statements of exercises.
2. Make a copy of the file `exercise_A.py` named `functions.py`.

3. In that file `functions_exercise.py`, remove the test cases for Exercise A, and add the definition of function `Df_CD_approximation` that was created in Exercise A of the section *Functions*. So now this new file defines a module `functions` which just defines these functions, not running any test cases using them or producing any output when run.
4. In notebook `modules.ipynb`, import these two functions, and below the `import` command, copy in the test case cells used in section *Functions* for both Exercises A and B.
5. Run the notebook.

This combination of notebook with importing from a module can give a notebook presentation that is more concise and less cluttered; this will be a distinct advantage at times later in the course where the collection of function definitions is far longer.

26.2.3 Exercise C. Start building a module of functions you create in this course.

Make a copy of the module file `modules.py` with a name like `numerical_methods.py`.

As the course progresses, you will create some more useful functions, such as one for solving equations using Newton's method: gather those in this module `numerical_methods`.

RECURSION (VS ITERATION)

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

27.1 Introduction

Although we have now seen all the essential tools for describing mathematical calculations and working with functions, there is one more algorithmic tool that can be very convenient, and is also of fundamental importance in the study of functions in both mathematics and theoretical computer science.

This is **recursively defined functions**: where the definition of the function refers to other values of the same function. To avoid infinite loops, the other values are typically for input values that are in some sense “earlier” or “smaller”, such as lower values of a natural number.

We have already seen two familiar examples in Unit 5; the factorial:

$$\begin{aligned}0! &= 1 \\ n! &= n \cdot (n-1)! \text{ for } n \geq 1\end{aligned}$$

and the Fibonacci numbers

$$\begin{aligned}F_0 &= 1 \\ F_1 &= 1 \\ F_n &= F_{n-1} + F_{n-2} \text{ for } n \geq 2\end{aligned}$$

27.2 Iterative form

These can be implemented using iteration, as seen in Unit 5; here are two versions:

```
def factorial_iterative(n):  
    n_factorial = 1  
    for i in range(2, n+1): # n+1 to include the factor n  
        n_factorial *= i  
    return n_factorial
```

```
n = 5  
print(f'{n}! = {factorial_iterative(n)}')  
print('Test some edge cases:')  
print('0! =', factorial_iterative(0))  
print('1! =', factorial_iterative(1))
```

```
5! = 120
Test some edge cases:
0! = 1
1! = 1
```

```
def first_n_factorials(n):
    factorials = [1]
    for i in range(1, n):
        factorials.append(factorials[-1]*i)
    return factorials
```

```
n = 10
print(f'The first {n} factorials (so up to {n-1}!) are {first_n_factorials(n)}')
```

```
The first 10 factorials (so up to 9!) are [1, 1, 2, 6, 24, 120, 720, 5040, 40320,
↵362880]
```

27.3 Recursive form

However we can also use a form that is closer to the mathematical statement.

First, let us put the factorial definition in more standard mathematical notation for functions

$$f(0) = 1$$
$$f(n) = n \cdot f(n-1) \text{ for } n \geq 1$$

Next to make it more algorithmic and start the segue towards Python code, distinguish the two cases with an `if`

```
if n == 0:
    f(n) = 1
else:
    f(n) = n * f(n-1)
```

Here that is in Python code:

```
def factorial_recursive(n):
    if n == 0:
        return 1
    else:
        return n * factorial_recursive(n-1)
```

```
n = 9
print(f'{n}! = {factorial_recursive(n)}')
```

```
9! = 362880
```

Yes, Python functions are allowed to call themselves, though one must beware that this could lead to an infinite loop.

27.3.1 Exercise A

Can you see how you could cause that problem with this function?

Try, and you will see that Python has some defences against this kind of infinite loop.

It can be illuminating to *trace* the steps with some extra print commands:

```
def factorial_recursive_with_tracing(n, trace=False):
    if trace:
        print(f'n = {n}')
    if n == 0:
        factorial = 1
    else:
        factorial = n * factorial_recursive_with_tracing(n-1, trace)
    if trace:
        print(f'{n}! = {factorial}')
    return factorial
```

```
n = 5
nfactorial = factorial_recursive_with_tracing(n, trace=True)
print('The final result is', nfactorial)
```

```
n = 5
n = 4
n = 3
n = 2
n = 1
n = 0
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
The final result is 120
```

Experiment: Try sending this *tracing* version into an infinite loop. (In JupyterLab, note the square “stop” button to the right of the triangular “play” button!)

27.3.2 Exercise B: Fibonacci numbers by recursion

Write and test a recursive function to evaluate the n -th Fibonacci number F_n , mimicking the first, simplest recursive version for the factorial above.

Do this without using either lists, arrays, or special variables holding the two previous values!

If you have access to Spyder (or another Python IDE) develop and test this in a Python file “`exercise7b.py`” and submit that as well as a final notebook for this unit.

Test for n at least 5.

27.3.3 Exercise C: tracing the recursion

Write and test a second recursive function to evaluate the first n Fibonacci numbers, adding the option of tracing output, as in the second recursive example above.

Again test for n at least 5.

Again, develop and test this in a Python file “`exercise7c.py`” initially if you have access to a suitable IDE.

Comment on why this illustrates that, although recursive implementations can be very concise and elegant, they are sometimes very inefficient compared to expressing the calculation as an iteration with `for` or `while` loops.

27.3.4 Exercise D: present your work in a Jupyter notebook

If you have not done so already, put all your code for the above exercises into a single Jupyter notebook.

Make sure that, like all documents produced in this course, the notebook and any other files submitted have an appropriate title, your name, and the correct date at the top!

Note the way I express the date as “Last modified”; keep this up-to-date when you revise.

27.4 Bonus Material: Tail Recursion

Some recursive algorithms are so-called *tail recursive*, which means that when a function calls itself, the “calling” invocation of the function has nothing more to do; the task is handed off entirely to the new invocation. This means that it can be possible to “clean up” by getting rid of all memory and such associated with the calling invocation of the function, eliminating that nesting seen in the above tracing and potentially improving efficiency by a lot.

Some programming languages do this “clean up” of so-called “tail calls”; indeed *functional programming languages* forbid variables to have their values changed within a function (so that functions in such a language are far more like real mathematical functions), and this rules out many `while` loop algorithms, like those above. Then recursion is a central tool, and there is a high priority on implementing recursion in this efficient way.

For example, here is a tail recursive approach to the factorial:

```
def factorial_tail_recursive(n):
    '''For convenience, we wrap the actual "working" function inside one with simpler_
    ↪input:
    '''
    def tail_factorial(result_so_far, n):
        print(f'result_so_far = {result_so_far}, n = {n}')
        if n == 0:
            return result_so_far
        else:
            return tail_factorial(result_so_far*n, n-1)
    result_so_far = 1
    return tail_factorial(result_so_far, n)
```

```
n = 9
print(f'factorial_tail_recursive gives {n}! = {factorial_tail_recursive(n)}')
print('\nFor comparison,')
print(f'factorial_recursive gives {n}! = {factorial_recursive(n)}')
print(f'factorial_iterative gives {n}! = {factorial_iterative(n)}')
```



```

result_so_far = 1, n = 9
result_so_far = 9, n = 8
result_so_far = 72, n = 7
result_so_far = 504, n = 6
result_so_far = 3024, n = 5
result_so_far = 15120, n = 4
result_so_far = 60480, n = 3
result_so_far = 181440, n = 2
result_so_far = 362880, n = 1
result_so_far = 362880, n = 0
factorial_tail_recursive gives 9! = 362880

```

```

For comparison,
factorial_recursive gives 9! = 362880
factorial_iterative gives 9! = 362880

```

However, tail recursion is in general equivalent to iteration with a `while` loop, with the input and output of the tail recursive function instead being variables that are updated in the loop. Thus it is mostly a matter of preference as to how one expresses the algorithm.

For example, the above can be rather straightforwardly translated to the following:

```
print(f'factorial_tailless gives {n}! = {factorial_tailless(n)}')
```

```
factorial_tailless gives 9! = 362880
```

27.5 A final challenge (optional, and maybe hard!)

Write a tail-recursive Python function for computing the Fibonacci number F_n .

PLOTTING GRAPHS WITH MATPLOTLIB

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

28.1 Introduction: Matplotlib and Pyplot

Numerical data is often presented with graphs, and the tools we use for this come from the module `matplotlib.pyplot` which is part of the Python *package* `matplotlib`. (A Python *package* is essentially a module that also contains other modules.)

28.2 Sources on Matplotlib

Matplotlib is a huge collection of graphics tools, of which we see just a few here. For more information, the home site for Matplotlib is <http://matplotlib.org> and the section on pyplot is at http://matplotlib.org/1.3.1/api/pyplot_api.html

However, another site that I find easier as an introduction is <https://scipy-lectures.org/intro/matplotlib/>

In fact, that whole site <https://scipy-lectures.org/> is quite useful a a reference on Python, Numpy, and so on.

Note: the descriptions here are for now about working in notebooks: see the note below on *differences when using Spyder and IPython*.

28.3 Choosing where the graphs appear

In a notebook, we can choose between having the figures produced by Matplotlib appear “inline” (that is, within the notebook window) or in separate windows. For now we will use the inline option, which is the default, but can also be specified explicitly with the command

```
%matplotlib inline
```

To activate that, uncomment the line below; that is, remove the leading hash character “#”

```
%matplotlib inline
```

This is an IPython *magic command*, indicated by starting with the percent character “%” — you can read more about them at <https://ipython.org/ipython-doc/dev/interactive/magics.html>

Alternatively, one can have figures appear in separate windows, which might be useful when you want to save them to files, or zoom and pan around the image. That can be chosen with the magic command

```
%matplotlib tk
```

```
##matplotlib tk
```

As far as I know, this magic works for Windows and Linux as well as Mac OS; let me know if it does not!

We need some Numpy stuff, for example to create arrays of numbers to plot.

Note that this is Numpy only: Python lists and tuples do not work for this, and nor do the versions of functions like `sin` from module `math`!

```
# Import a few favorites, and let them be known by their first names:  
from numpy import linspace, sin, cos, pi
```

And for now, just the one main matplotlib graphics function, `plot`

```
from matplotlib.pyplot import plot
```

To access all of pyplot, add its common nickname `plt`:

```
import matplotlib.pyplot as plt
```

28.4 Producing arrays of “x” values with the numpy function `linspace`

To plot the graph of a function, we first need a collection of values for the abscissa (horizontal axis). The function `linspace` gives an array containing a specified number of equally spaced values over a specified interval, so that

```
tenvalues = linspace(1., 7., 10)
```

gives ten equally spaced values ranging from 1 to 7:

```
print(f"Array 'tenvalues' is:\n{tenvalues}")
```

```
Array 'tenvalues' is:  
[1.          1.66666667 2.33333333 3.          3.66666667 4.33333333  
 5.          5.66666667 6.33333333 7.          ]
```

Not quite what you expected? To get values with ten *intervals* in between them, you need 11 values:

```
tenintervals = linspace(1., 7., 11)  
print(f"Array 'tenintervals' is: \n {tenintervals}")
```

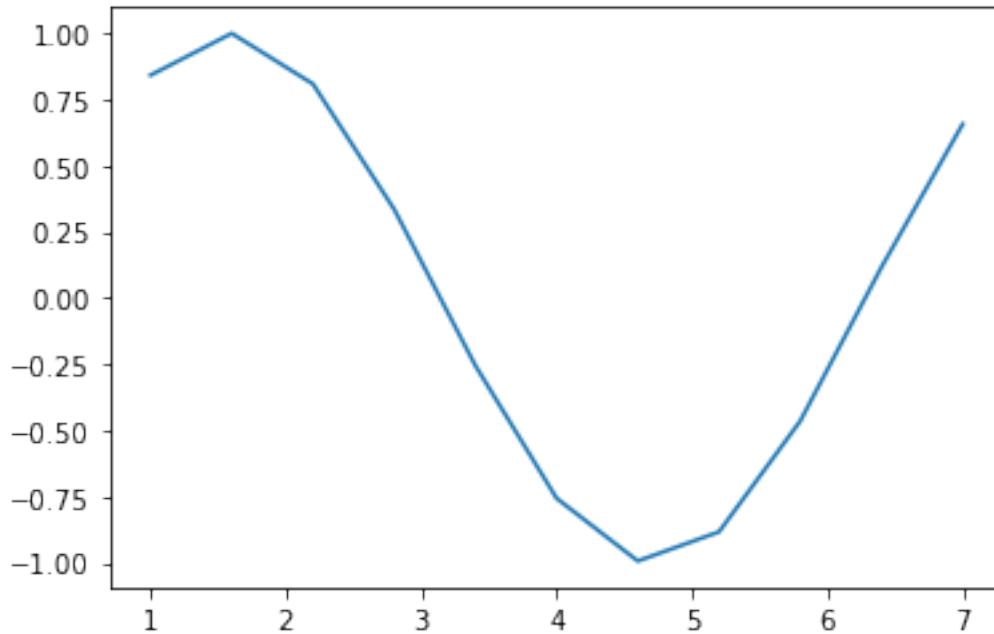
```
Array 'tenintervals' is:  
[1.  1.6 2.2 2.8 3.4 4.  4.6 5.2 5.8 6.4 7. ]
```

28.5 Basic graphs with `plot`

We could use these 11 values to graph a function, but the result is a bit rough, because the given points are joined with straight line segments:

```
plot(tenintervals, sin(tenintervals))
```

```
[<matplotlib.lines.Line2D at 0x7fa69a063d30>]
```

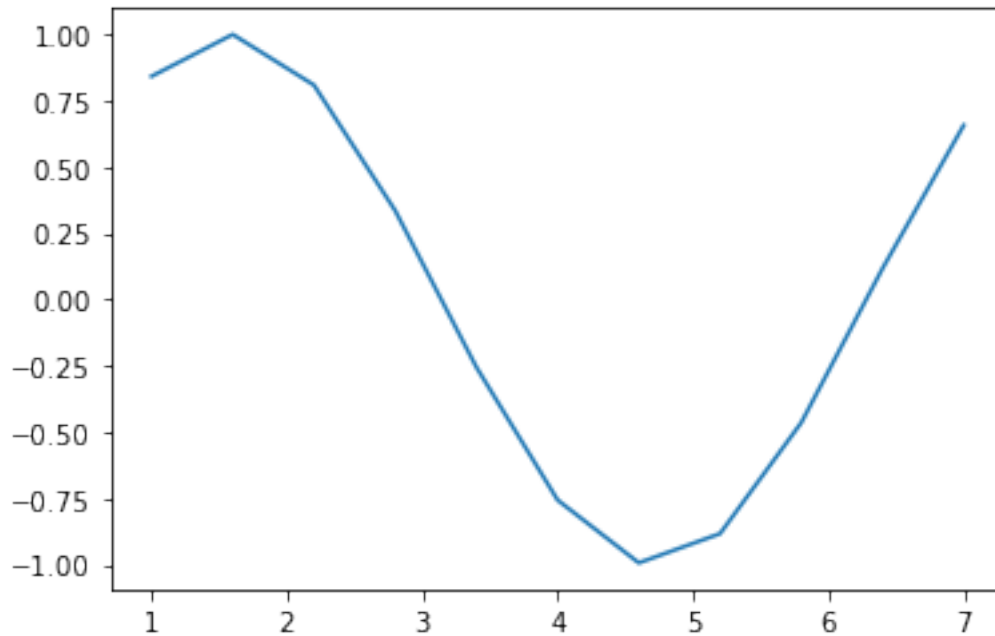


Here we see the default behavior of joining the given points with straight lines.

Aside: That text output above the graph is a message returned as the output value of function `plot`; that is what happens when you execute a function but do not “use” its return value by either saving its result into a variable or making it input to another function.

You might want to suppress that, and that can be done by saving its `return` value into a variable (which you can then ignore).

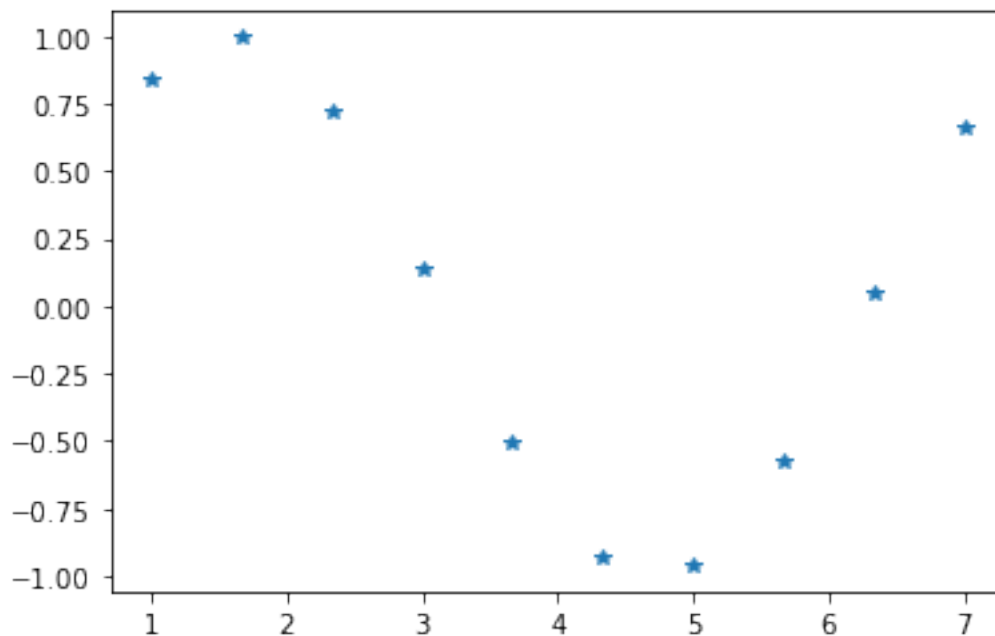
```
plotmessage = plot(tenintervals, sin(tenintervals))
```



(More on this output clean-up below.)

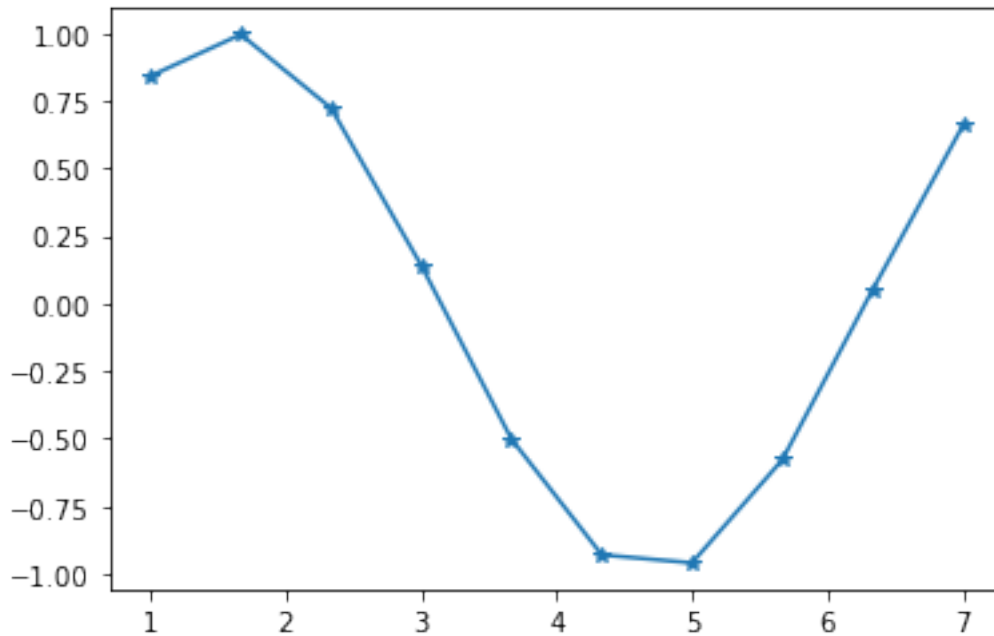
For discrete data it might be better to mark each point, unconnected. This is done by adding a third argument, a text string specifying a marker, such as a star:

```
plotmessage = plot(tenvalues, sin(tenvalues), '*')
```



Or maybe both lines and markers:

```
plotmessage = plot(tenvalues, sin(tenvalues), '-*')
```



28.6 Smoother graphs

It turns out that 50 points is often a good choice for a smooth-looking curve, so the function `linspace` has this as a *default input parameter*: you can omit that third input value, and get 50 points.

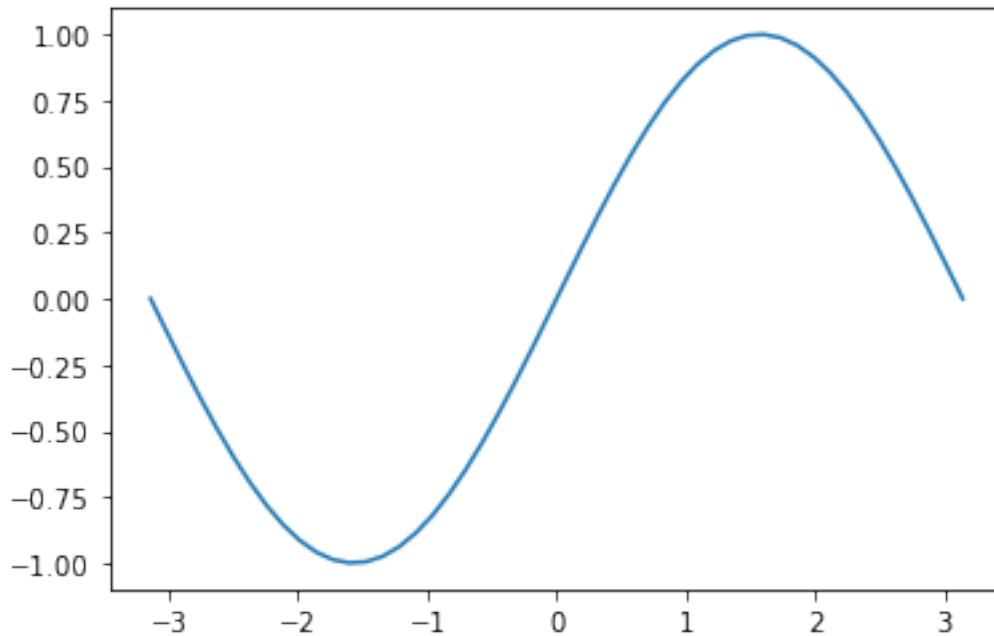
Let's use this to plot some trig. functions.

```
x = linspace(-pi, pi)
```

```
print(x)
```

```
[-3.14159265 -3.01336438 -2.88513611 -2.75690784 -2.62867957 -2.5004513
 -2.37222302 -2.24399475 -2.11576648 -1.98753821 -1.85930994 -1.73108167
 -1.60285339 -1.47462512 -1.34639685 -1.21816858 -1.08994031 -0.96171204
 -0.83348377 -0.70525549 -0.57702722 -0.44879895 -0.32057068 -0.19234241
 -0.06411414  0.06411414  0.19234241  0.32057068  0.44879895  0.57702722
  0.70525549  0.83348377  0.96171204  1.08994031  1.21816858  1.34639685
  1.47462512  1.60285339  1.73108167  1.85930994  1.98753821  2.11576648
  2.24399475  2.37222302  2.5004513   2.62867957  2.75690784  2.88513611
  3.01336438  3.14159265]
```

```
# With a line through the points
plotmessage = plot(x, sin(x), '-')
```

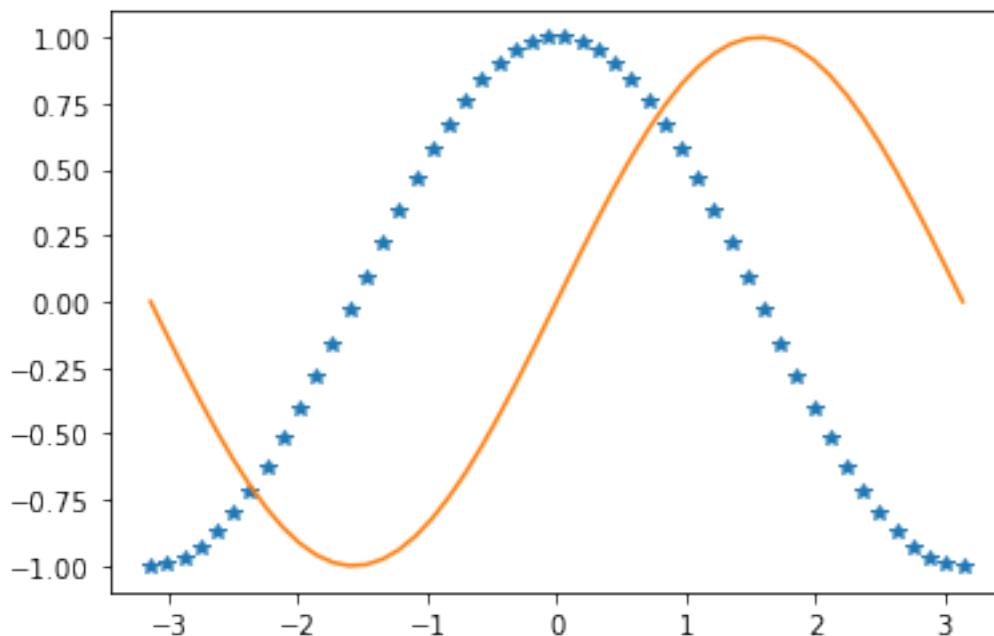


28.7 Multiple curves on a single figure

As we have seen when using `plot` to produce inline figures in a Jupyter notebook, `plot` commands in different cells produce separate figures.

To combine curves on a single graph, one way is to use successive `plot` commands within the same cell:

```
plot(x, cos(x), '*')  
plotmessage = plot(x, sin(x))
```



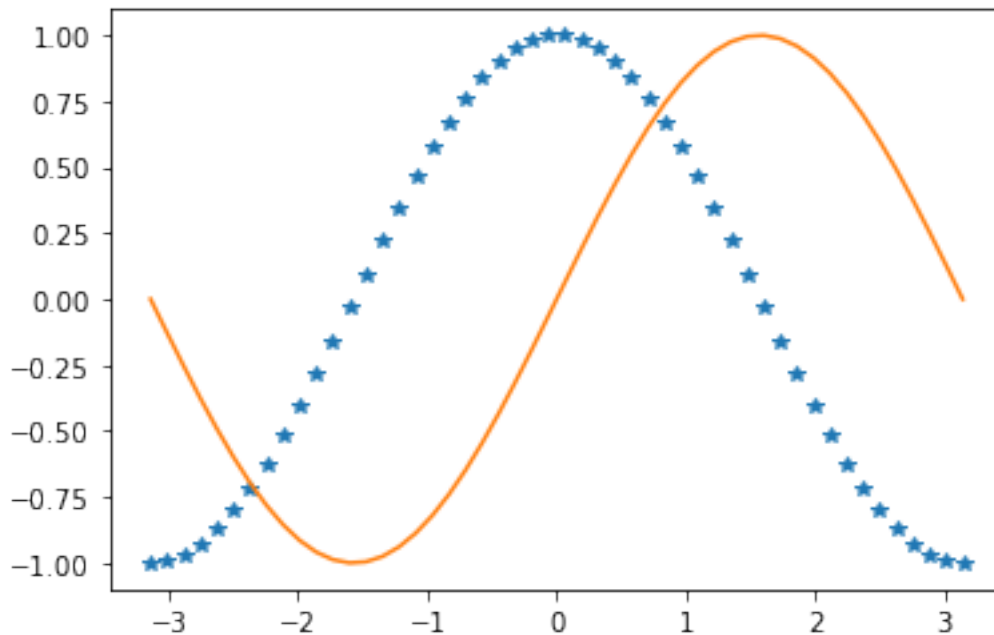
On the other hand, when plotting externally, or from a Python script file or the IPython command line, successive `plot` commands keep adding to the same figure until you explicitly specify otherwise, with the function `figure` introduced below.

Aside on message clean up: a Jupyter cell only displays the output of the last function invoked in the cell (along with anything explicitly output with a `print` function), so I only needed to intercept the message from the last `plot` command.

28.8 Two curves with a single `plot` command

Several curves can be specified in a single `plot` command (which also works with external figure windows of course.)

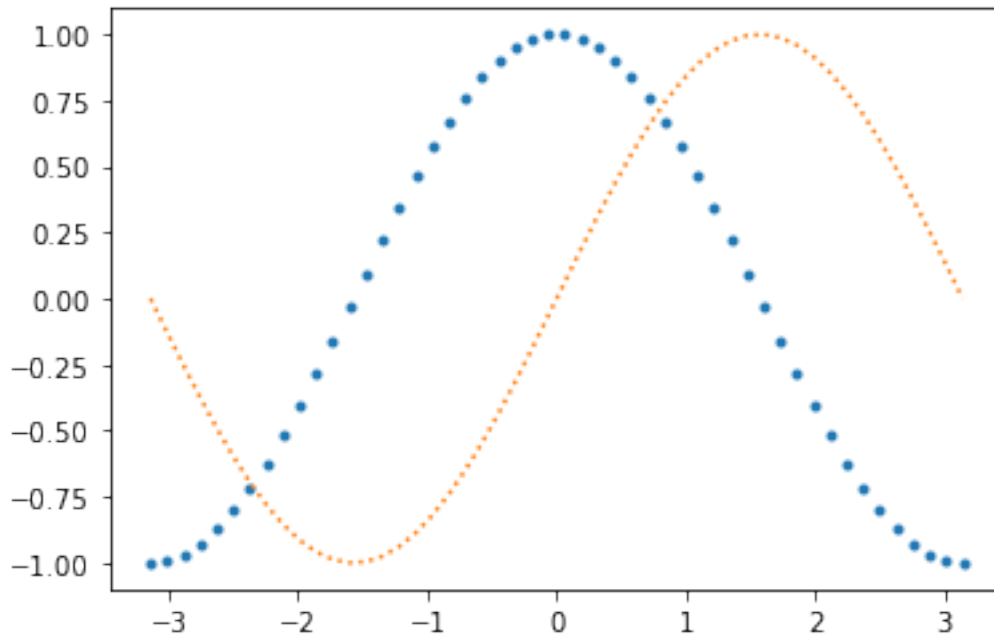
```
plotmessage = plot(x, cos(x), '*', x, sin(x))
```



Note that even with multiple curves in a single `plot` command, markers can be specified on some, none or all: Matplotlib uses the difference between an array and a text string to recognize which arguments specify markers instead of data.

Here are some other marker options — particularly useful if you need to print in back-and-white.

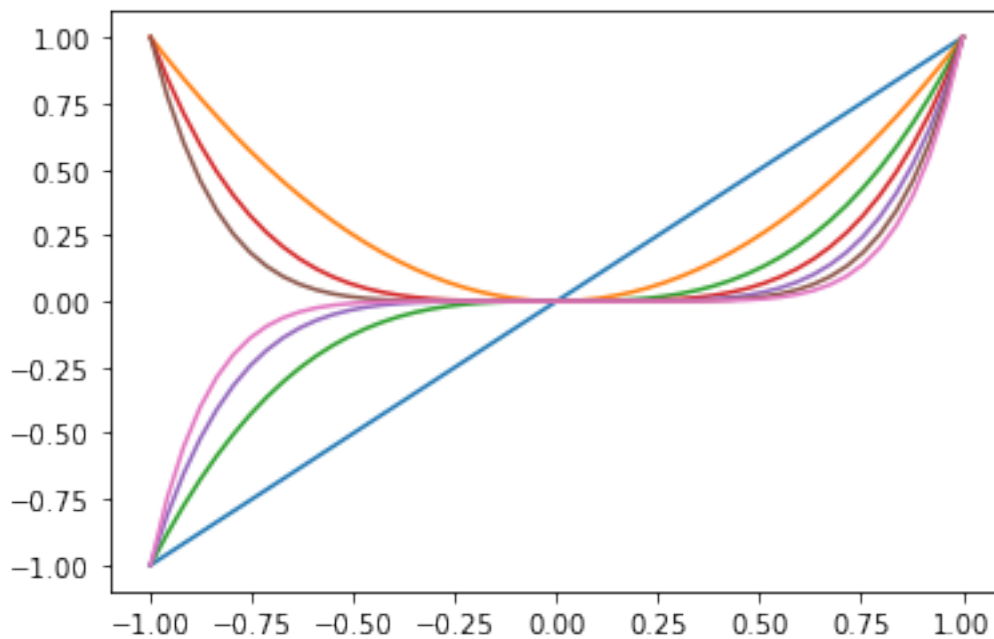
```
plotmessage = plot(x, cos(x), '.', x, sin(x), ':')
```



28.9 Multiple curves in one figure

There can be any number of curves in a single `plot` command:

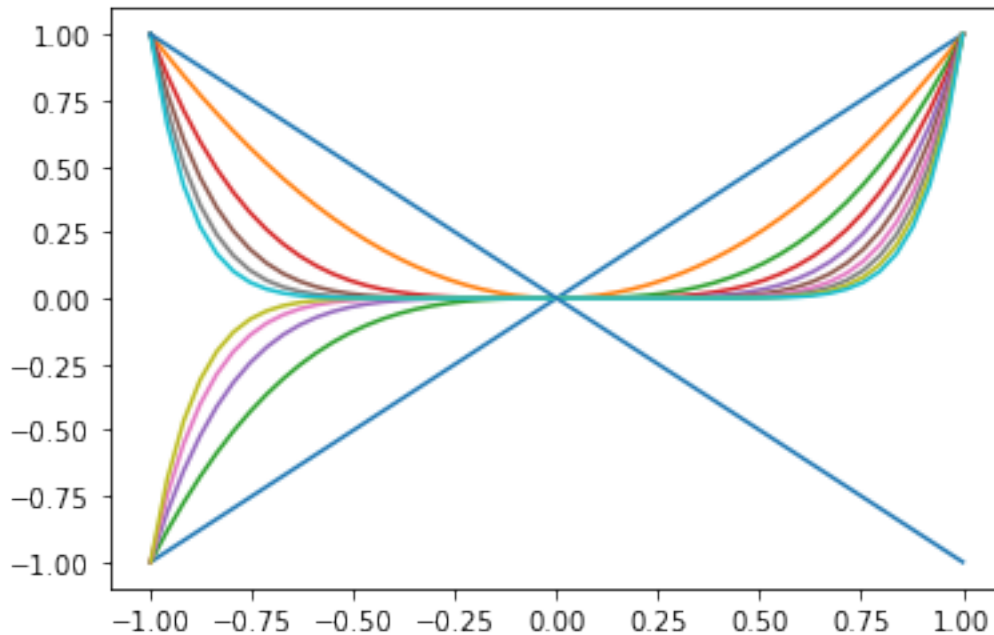
```
x = linspace(-1,1)
plotmessage = plot(x, x, x, x**2, x, x**3, x, x**4, x, x**5, x, x**6, x, x**7)
```



Note the color sequence.

With enough curves (more than ten? It depends on the version of matplotlib in use) the color sequence eventually repeats – but you probably don’t want that many curves on one graph.

```
x = linspace(-1,1)
plotmessage = plot(x, x, x, x**2, x, x**3, x, x**4, x, x**5,
                  x, x**6, x, x**7, x, x**8, x, x**9, x, x**10,
                  x, -x)
```



Aside on long lines of code: The above illustrates a little Python coding hack: one way to have a long command continue over several lines is simply to have parentheses wrapped around the part that spans multiple lines—when a line ends with an opening parenthesis not yet matched, Python knows that something is still to come.

28.9.1 Aside: using IPython magic commands in Spyder and with the IPython command line

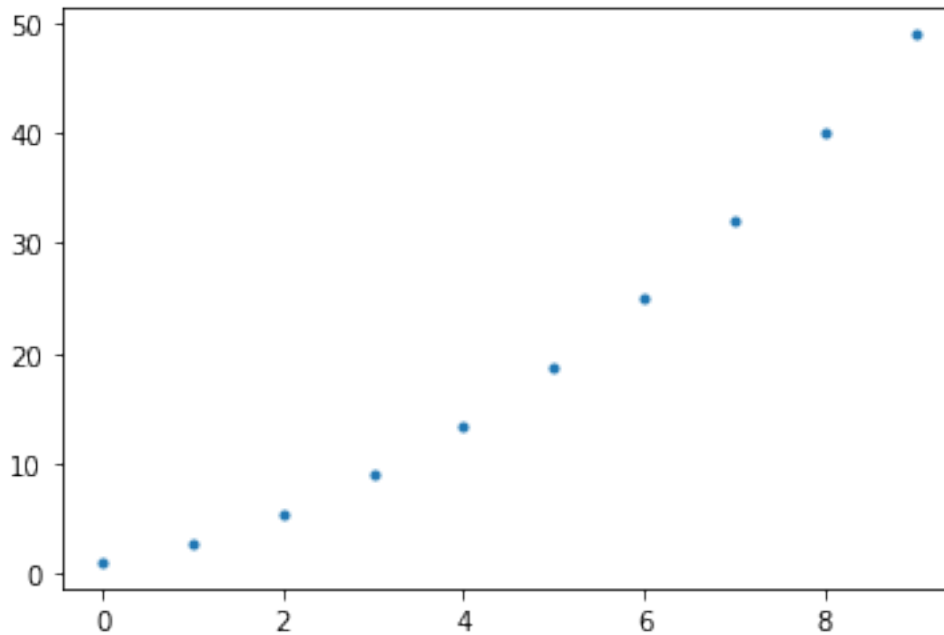
If using Spyder and the IPython command line, there is a similar choice of where graphs appear, but with a few differences to note:

- With the “inline” option (which is again the default) figures then appear in a pane within the Spyder window.
- The “tk” option works exactly as with notebooks, with each figure appearing in its own window.
- **Note:** Any such IPython magic commands must be entered at the IPython interactive command line, not in a Python code file.

28.10 Plotting sequences

A curve can also be specified by a single array of numbers: these are taken as the values of a sequence, indexed Pythonically from zero, and plotted as the ordinates (vertical values):

```
plotmessage = plot(tenvalues**2, '.')
```



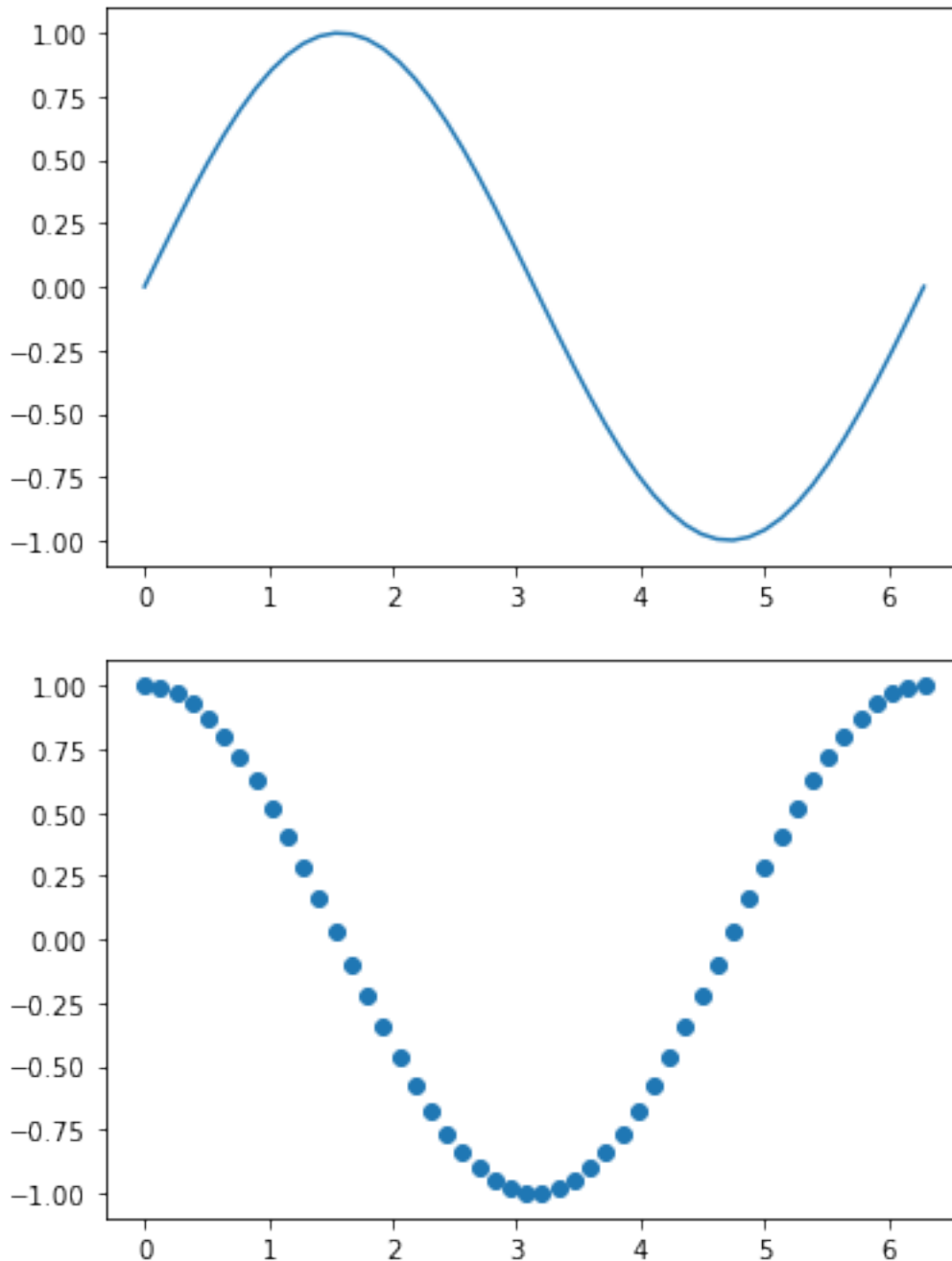
28.11 Plotting curves in separate figures (from a single cell)

From within a single Jupyter cell, or when working with Python files or in the IPython command window (as used within Spyder), successive `plot` commands keep adding to the previous figure. To instead start the next `plot` in a separate figure, first create a new “empty” figure, with the function `matplotlib.pyplot.figure`.

With a full name as long as that, it is worth importing so that it can be used on a first name basis:

```
from matplotlib.pyplot import figure
```

```
x = linspace(0, 2*pi)
plot(x, sin(x))
figure()
plotmessage = plot(x, cos(x), 'o')
```



The `figure` command can also do other things, like attach a name or number to a figure when it is displayed externally, and change from the default size.

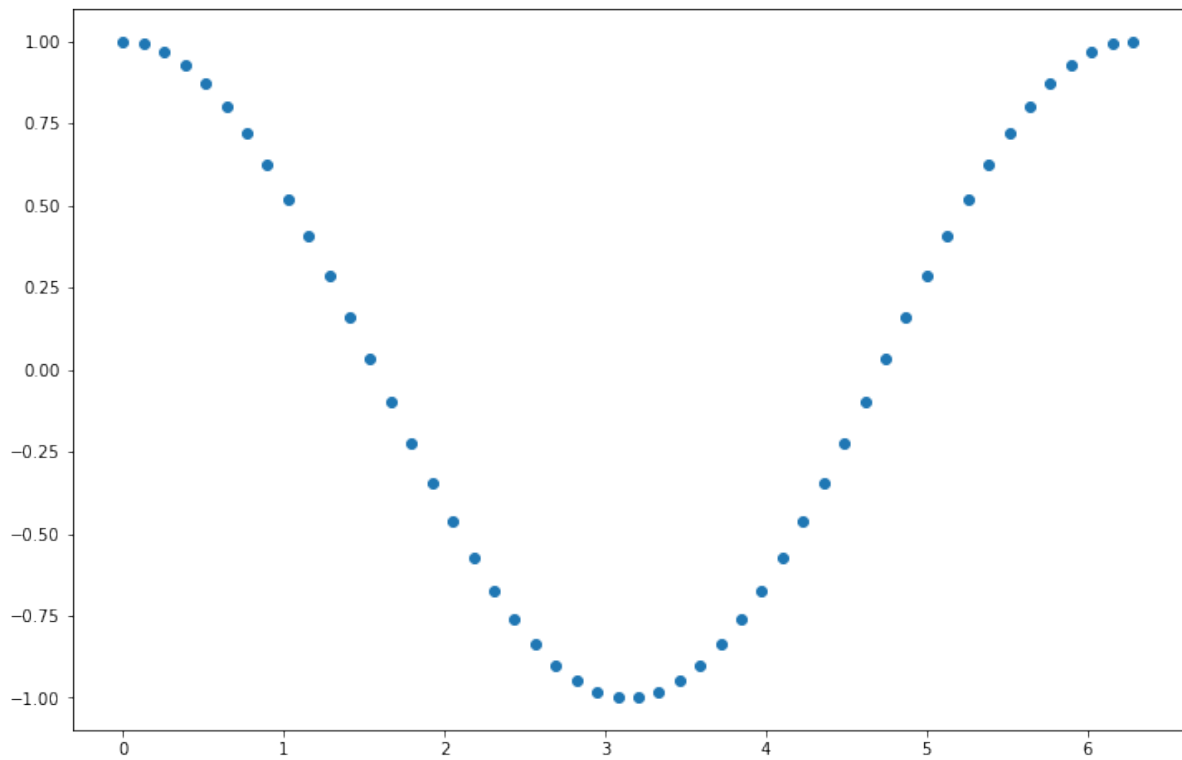
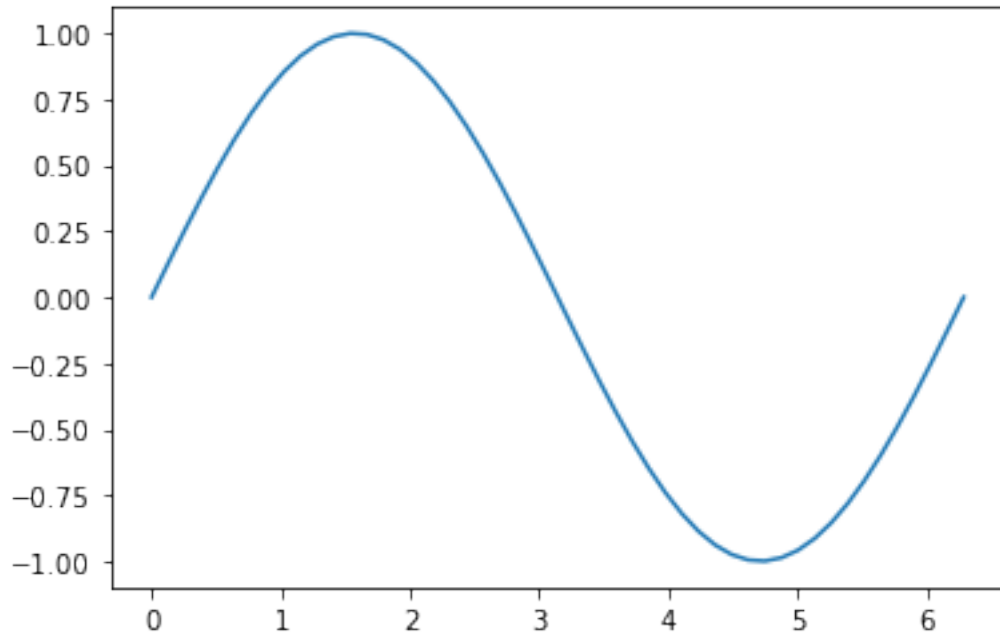
So even though this is not always needed in a notebook, from now on each new figure will get an explicit `figure` command. Revisiting the last example:

```
x = linspace(0, 2*pi)
figure(99)
# What does 99 do?
# See with external "tk" display of figures,
# as with `%matplotlib tk`
```

(continues on next page)

(continued from previous page)

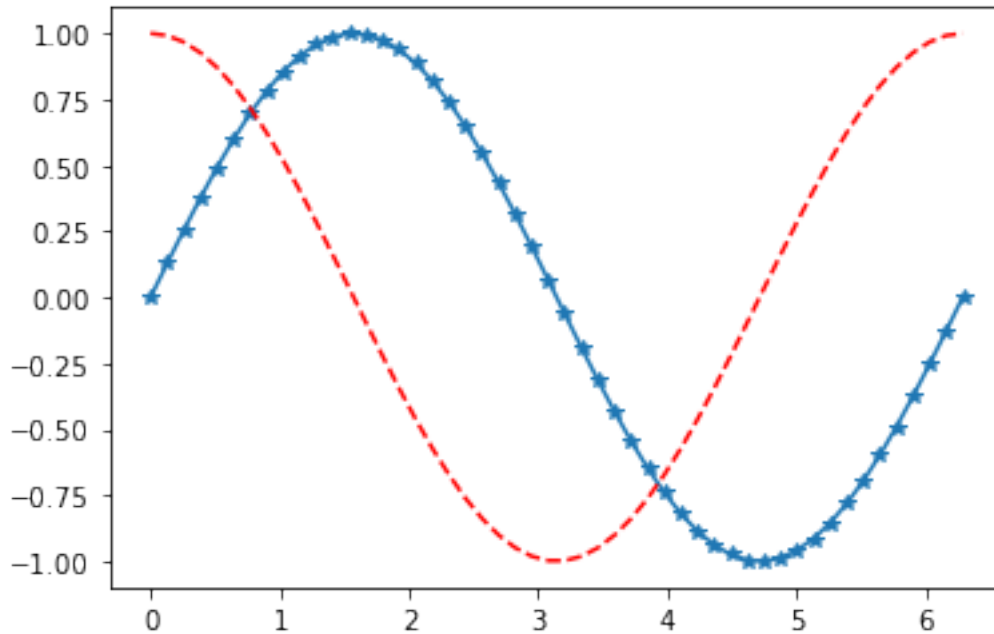
```
plot(x, sin(x))  
figure(figsize=(12,8))  
plotmessage = plot(x, cos(x), 'o')
```



28.12 Decorating the Curves

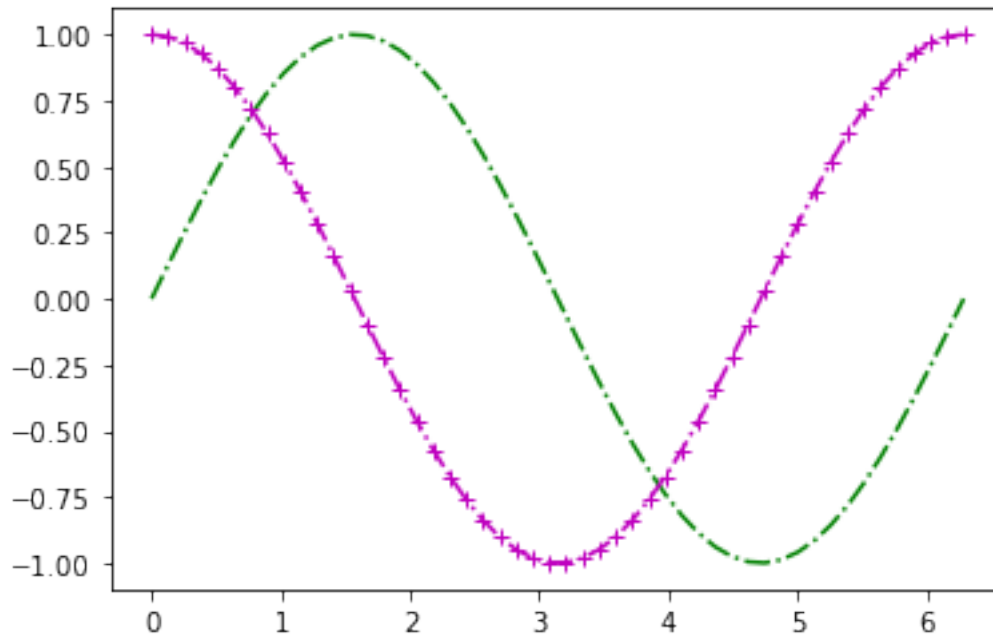
Curves can be decorated in different ways. We have already seen some options, and there are many more. One can specify the color, line styles like dashed or dash-dot instead of solid, many different markers, and to have both markers and lines. As seen above, this can be controlled by an optional text string argument after the arrays of data for a curve:

```
figure()
plot(x, sin(x), '*-')
plotmessage = plot(x, cos(x), 'r--')
```



These three-part curve specifications can be combined: in the following, `plot` knows that there are two curves each specified by three arguments, not three curves each specified by just an “x-y” pair:

```
figure()
plotmessage = plot(x, sin(x), 'g-.', x, cos(x), 'm+-.')
```



28.13 Exercises

28.13.1 Exercise A: Explore ways to refine your figures

There are many commands for refining the appearance of a figure after its initial creation with `plot`. Experiment yourself with the commands `title`, `xlabel`, `ylabel`, `grid`, and `legend`.

Using the functions mentioned above, produce a refined version of the above sine and cosine graph, with:

- a *title* at the top
- *labels* on both axes
- a *legend* identifying each curve
- a *grid* or “graph paper” background, to make it easier to judge details like where a function has zeros.

28.13.2 Exercise B: Saving externally displayed figures to files

Then work out how to save this figure to a file (probably in format PNG), and turn that in, along with the file used to create it.

This is most readily done with externally displayed figures; that is, with `%matplotlib tk`. Making that change to `tk` in a notebook requires then restarting the kernel for it to take effect; use the menu *Kernel* above and select “Restart Kernel and Run All Cells ...”

For your own edification, explore other features of externally displayed figures, like zooming and panning: this cannot be done with inline figures.

28.14 Getting help from the documentation

For some of these, you will probably need to read up. For simple things, there is a function `help`, which is best used in the IPython interactive input window (within Spyder for example), but I will illustrate it here.

The entry for `plot` is unusually long! It provides details about all the options mentioned above, like marker styles. So this might be a good time to learn how to clear the output in a cell, to unclutter the view: either use the above menu “Edit*” or open the menu with Control-click or right-click on the code cell; then use “Clear Outputs” to remove the output of just the current cell.

```
help(plot)
```

The jargon used in `help` can be confusing at first; fortunately there are other online sources that are more readable and better illustrated, like <http://scipy-lectures.github.io/intro/matplotlib/matplotlib.html> mentioned above.

However, that does not cover everything; the official pyplot documentation at http://matplotlib.org/1.3.1/api/pyplot_api.html is more complete: explore its search feature.

28.15 P. S. A shortcut revealed: the IPython “magic” command `pylab`

So far I have encourage you to use explicit, specific import commands, because this is good practice when developing larger programs. However, for quick interactive work in the IPython command window and Jupyter notebooks, there is a sometimes useful shortcut: the IPython “magic” command

```
%pylab
```

adds everything from Numpy and the main parts of Matplotlib, including all the items imported above. (This creates the so-called **pylab** environment: that name combines “Python” with “Matlab”, as its goal is to produce an environment very similar to Matlab.)

Note that such “magic” commands are part of the IPython interactive interface, not Python language commands, so they must be used either in a IPython notebook or in the IPython command window (within Spyder), not in a Python “.py” file.

However, there is a way to access magics in python scripts; the above can be achieved in such a file with:

```
get_ipython().run_line_magic('pylab', '')
```

and the magic

```
%matplotlib inline
```

is achieved with

```
get_ipython().run_line_magic('matplotlib', 'inline')
```


NUMPY ARRAY OPERATIONS AND LINEAR ALGEBRA

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

Note: We will see more tools for linear algebra in the section on *Scipy Tools*, which introduces the package *Scipy*.

As of version 3.5, Python can handle matrix multiplication on Numpy arrays, using the at sign “@”:

```
C = A @ B
```

Why this strange notation? Because

```
D = A * B
```

is instead “point-wise” multiplication of arrays, with $D[i, j] = A[i, j] * B[i, j]$ instead of

$$C_{i,j} = \sum_k a_{ik} b_{kj}$$

Aside 1: Numpy `matrix` is now redundant!

Previously, Numpy supported matrix calculations with the variables of type `matrix`, a special sub-type of `array`. However, with the addition of “@” for matrix multiplication of arrays, that option is now un-needed, and I advise you to avoid it.

Aside 2: Floating point numbers vs integers

Though Python is generally good at understanding when an integer like 7 is to be used as a floating point (real) number, it is sometimes best to make this distinction explicitly when working with module `numpy`; otherwise sometimes division done within numpy functions returns an integer answer, like $7/2 = 3$.

Thus from now on, when I mean an integer to be used as a floating point number, I give it a decimal point: `7./2.` will reliably be `3.5`

First we explore some basic features offered by Numpy; then we will look at some more advanced tools from package *Scipy*, and specifically its module `scipy.linalg` for linear algebra.

```
import numpy as np
```

```
A = np.array([[1, 2, 3], [4, 5, 6]])
B = np.array([[1, 2], [3, 4], [5, 6]])
C = np.array([[10, 9, 8], [7, 6, 5]])
```

```
print(f"A is\n{A}")
print(f"B is\n{B}")
print(f"C is\n{C}")
```

```
A is
[[1 2 3]
 [4 5 6]]
B is
[[1 2]
 [3 4]
 [5 6]]
C is
[[10 9 8]
 [ 7 6 5]]
```

```
print(f"The matrix product A times B is:\n {A @ B}")
print(f"The matrix product B times A is:\n {B @ A}")
```

```
The matrix product A times B is:
[[22 28]
 [49 64]]
The matrix product B times A is:
[[ 9 12 15]
 [19 26 33]
 [29 40 51]]
```

```
print(f"The array product A * B fails:\n {A * B}")
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-4-3bb26336abf9> in <module>
----> 1 print(f"The array product A * B fails:\n {A * B}")

ValueError: operands could not be broadcast together with shapes (2,3) (3,2)
```

On the other hand:

```
print(f"Array product A times C is\n {A * C}\n")
```

```
Array product A times C is
[[10 18 24]
 [28 30 30]]
```

```
print(f"Matrix product A times C fails:\n {A @ C}")
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-6-55b834c682c2> in <module>
----> 1 print(f"Matrix product A times C fails:\n {A @ C}")

ValueError: matmul: Input operand 1 has a mismatch in its core dimension 0, with
expected signature (n?,k),(k,m?)->(n?,m?) (size 2 is different from 3)
```

29.1 The matrix transpose A^T is given by `A.T`

```
print(f"A-transpose is\n{A.T}")
```

```
A-transpose is
[[1 4]
 [2 5]
 [3 6]]
```

29.2 *Slicing*: Extracting rows, columns, and other rectangular chunks from matrices

This works with Python lists and Numpy arrays, and we have seen some of it before; I review it here because it will help with doing the row operations of linear algebra.

29.2.1 Index notation for slicing

For an index with n possible values, from 0 to $n-1$:

- $a:b$ means indices i in the usual semi-open interval $a \leq i < b$ or $[a, b)$
- $a:$ is short for $a:n$, so indices $a \leq i$, all the way to the maximum index value
- $:b$ is short for $0:b$, so all indices $i < b$
- $:$ combines both of the above, so is short for $0:n$; all possible indices
- index value -1 refers the last entry; the same as index $n-1$, but without needing to know n .
- index value $-m$ refers the “ m -th last” entry; the same as index $n-m$, again without needing to know n .

```
print(f"A:\n{A}")
print(f"The column of index 1 (presented as a row vector): {A[:,1]}")
print(f"The row of index 1: {A[1,:]}")
print(f"The first 2 elements of the row of index 1: {A[1,:2]}")
print(f"Another way to say the above: {A[1,0:2]}")
print(f"The bottom-right element: {A[-1,-1]}")
print(f"The 2x2 sub-matrix in the bottom-right corner:\n{A[-2:,-2:]}")
```

```
A:
[[1 2 3]
 [4 5 6]]
The column of index 1 (presented as a row vector): [2 5]
The row of index 1: [4 5 6]
The first 2 elements of the row of index 1: [4 5]
Another way to say the above: [4 5]
The bottom-right element: 6
The 2x2 sub-matrix in the bottom-right corner:
[[2 3]
 [5 6]]
```

29.2.2 Synonyms for array names with the equal sign (not copying!)

If we use the equal sign between two array names, that makes them synonyms, referring to the same values:

```
A = np.array([[1, 2, 3],[4, 5, 6]])
print(f"A is\n{A}")
Anickname = A
print(f"Anickname is\n{Anickname}")
Anickname[0,0] = 12
print(f"Anickname is now\n{Anickname}")
print(f"and so is A!\n{A}")
```

```
A is
[[1 2 3]
 [4 5 6]]
Anickname is
[[1 2 3]
 [4 5 6]]
Anickname is now
[[12  2  3]
 [ 4  5  6]]
and so is A!:
[[12  2  3]
 [ 4  5  6]]
```

29.2.3 Copying arrays with method .copy()

Thus if we want a separate new array or list with the same elements initially, we must make a copy with the method `.copy()`, not the equal sign:

```
A = np.array([[1, 2, 3],[4, 5, 6]])
print(f"A is\n{A}")
Acopy = A.copy()
print(f"Acopy is\n{Acopy}")
Acopy[0,0] = 54
print(f"Acopy is now\n{Acopy}")
print(f"A is still\n{A}")
```

```
A is
[[1 2 3]
 [4 5 6]]
Acopy is
[[1 2 3]
 [4 5 6]]
Acopy is now
[[54  2  3]
 [ 4  5  6]]
A is still
[[1 2 3]
 [4 5 6]]
```

Exercise A

Create a Numpy array (not a Numpy *matrix*; those are now mostly obsolete!) containing the matrix

$$A = \begin{bmatrix} 4. & 2. & 1. \\ 9. & 3. & 1. \\ 25. & 5. & 1. \end{bmatrix}$$

and one containing the vector

$$b = [0.693147, 1.098612, 1.386294]$$

Exercise B

Create arrays c and d containing respectively the last row of A and the middle column of A .

Note: Do this by manipulating the array A with indexing and slicing operations, without entering any numerical values for array entries explicitly.

PACKAGE SCIPY AND MORE TOOLS FOR LINEAR ALGEBRA

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

30.1 Introduction

The package *Scipy* provides a great array of functions for scientific computing; here we will just explore one part of it: some additional tools for linear algebra from module *linalg* within the package *Scipy*. This provides tools for solving simultaneous linear equations, for variations on the LU factorization seen in a numerical methods course, and much more.

This module has the standard standard nickname `la`, so import it using that:

```
import scipy.linalg as la
```

SciPy usually needs stuff from NumPy, so let's import that also:

```
import numpy as np
```

30.1.1 Exercise C

Use the SciPy function `solve` to solve $Ax = b$ for x , where

$$A = \begin{bmatrix} 4. & 2. & 1. \\ 9. & 3. & 1. \\ 25. & 5. & 1. \end{bmatrix}$$

and

$$b = [0.693147, 1.098612, 1.386294]$$

as in Exercise A of section *Numpy Array Operations and Linear Algebra*.

30.1.2 Exercise D

Check this result by computing the *residual vector* $r = Ax - b$.

Exercise D-bonus

Check further by computing the maximum error, or *maximum norm*, or *infinity norm* of this: $\|r\|_\infty = \|Ax - b\|_\infty$; that is, the maximum of the absolute values of the elements of r , $\max_{i=1}^n |r_i|$.

30.1.3 Exercise E

Next use the Scipy function `lu` to compute the $PA = LU$ factorization of A . Check your work by verifying that:

1. P is a permutation matrix (a one in each row and column; the rest all zeros)
2. PA is got by permutating the rows of A
3. L is (unit) lower triangular (all zeros above the main diagonal; ones on the main diagonal)
4. U is upper triangular (all zeros below the main diagonal)
5. The products PA and LU are the same (or very close; there might be rounding error).
6. The “residual matrix” $R = PA - LU$ is zero (or very close; there might be rounding error).

30.1.4 Exercise F

Use the above arrays b , P , L and U to solve $AX = b$ via $LUx = PAx = Pb$, as follows:

1. Solve $Lc = Pb$ for c .
2. Solve $Ux = c$ for x .

At each step verify by looking at the errors, or *residuals*,: $Lc - Pb$, $Ux - c$, $Ax - b$. Better yet, compute the maximum norm of each of these errors.

30.1.5 Optional Exercise G (on the matrix norm, seen in a numerical methods course)

Try to work out how to compute the matrix norm $\|A\|_\infty$.

Test your method on A , L and U , and compare $\|A\|_\infty$, $\|PA\|_\infty$, and $\|L\|_\infty\|U\|_\infty$.

SUMMATION AND INTEGRATION

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](https://creativecommons.org/licenses/by-sa/4.0/)

31.1 Introduction

This unit starts with the methods for approximating definite integrals seen in a calculus course like the (Composite) Midpoint Rule and (Composite) Trapezoidal Rule.

As bonus exercises, we will then work through some more advanced methods as seen in a numerical methods course; the algorithms will be stated below.

We will also learn more about working with modules: how to run tests while developing a module, and how to incorporate demonstrations into a notebook once a module is working.

31.2 Exercises

31.2.1 Exercise A: the Midpoint Rule, using a `for` loop

Write a Python function to be used as

```
M_n = midpoint0(f, a, b, n)
```

which returns the approximation of $I = \int_a^b f(x) dx$ given by the [Composite] Midpoint Rule with n intervals of equal width.

That is,

$$M_n = h (f(a + h/2) + f(a + 3h/2) + f(a + 5h/2) + \cdots + f(b - h/2)), \quad h = \frac{b - a}{n}.$$

In this first version, accumulate the sum using a `for` loop.

Test this with $I_1 := \int_1^e \frac{dx}{x}$ and several choices of n , such as 10 and 100.

Work out the exact value, and use this to display the size of the errors in each approximation.

31.2.2 Exercise B: the Midpoint Rule, using function `sum`

Express the above Midpoint Rule in summation notation “ Σ ” and reimplement as

```
M_n = midpoint1(f, a, b, n)
```

using the Python function `sum` to avoid any loops. (Doing it this way give a code that is more concise and closer to mathematical form, so hopefully more readable; it will also probably run faster.)

(Summation-and-Integration-Exercise-C)

31.2.3 Exercise C: the Trapezoidal Rule

Write a Python function to be used as

```
T_n = trapezoidal(f, a, b, n)
```

which returns the approximation of $I = \int_a^b f(x) dx$ given by the [Composite] Trapezoidal Rule with n intervals of equal width.

That is,

$$T_n = h \left(\frac{f(a)}{2} + f(a+h) + \cdots + f(b-h) + \frac{f(b)}{2} \right), \quad h = \frac{b-a}{n}.$$

Use the “`sum`” method as in [Exercise B](#) above; no loops allowed!

Again test this with $I_1 = \int_1^e \frac{dx}{x}$.

31.2.4 Testing and demonstrations within a module file

Create each of these three functions in a module `integration`.

Option A: using Spyder and IPython command line

Place the test cases below the function definitions within one or more `if` blocks starting

```
if __name__ == "__main__":
```

(Each of those long dashes is a *pair* of underscores.)

The contents of such an `if` block are executed when you run the file directly (as if it were a normal Python program file) but are ignored when the module is used with `import` from another file. Thus, these blocks can be used while developing the module, and later to provide a demonstration of the module’s capabilities.

Option B: using just notebooks and module files

As you may have seen, importing a module a second or subsequent time from another Python file or notebook does not get the updated version of the module's contents unless you restart the Python kernel. (Python does this to avoid duplicated effort when the same module is mentioned in several import statements in a file.) Thus while revising a module, it is more convenient to treat it like a normal Python file, testing from within by this method rather than by importing elsewhere.

31.3 Further Exercises

31.3.1 Exercise D: present your work so far with a notebook

Once the module is working for the above three methods, create a notebook which imports those functions and runs various examples with them. (That is, **copy** all the stuff from within the above `if __name__ == "__main__":` blocks to the notebook.)

The notebook should also describe the mathematical background; in particular, the formulas for all the methods.

(Summation-and-Integration-Exercise-E)

31.3.2 Bonus Exercise E: a more accurate result

Define another Python function which uses the above functions to compute the “weighted average” $S_n = \frac{2M_n + T_n}{3}$.

Yes, this is the Composite Simpson Rule, so make it

```
S_n = simpson(f, a, b, n)
```

Compare its accuracy to that of M_n and T_n .

Still do testing within the module file, but also add demonstrations to the above notebook once you are sure that this function is working.

31.3.3 Bonus Exercise F: The Romberg Method

The Romberg method generates a triangular array of approximations $R_{i,j}$, $j \leq i$ of an integral $I = \int_a^b f(x) dx$ with the end-of-row values $R_{0,0}, R_{1,1}, \dots, R_{n,n}$ being the main, successively better (usually) approximations.

It starts with the trapezoid rule $R_{0,0} := T_1 = \frac{f(a)+f(b)}{2}(b-a)$; the basic trapezoid rule.

Then using $T_{2n} = \frac{T_n + M_n}{2}$, one defines

$$R_{i,0} := T_{2^i} = \frac{T_{2^{i-1}} + M_{2^{i-1}}}{2} = \frac{T_{2^{i-1}} + M_{2^{i-1}}}{2}, \quad i \geq 1$$

Finally, Richardson extrapolation leads to

$$R_{i,1} := S_{2^i} = \frac{4T_{2^i} - T_{2^{i-1}}}{4-1} = \frac{4R_{i,0} - R_{i-1,0}}{4-1} = R_{i,0} + \frac{R_{i,0} - R_{i-1,0}}{4-1}, \quad i \geq 1$$

and with further extrapolations to the more general formula

$$R_{i,j} := \frac{4^j R_{i,j-1} - R_{i-1,j-1}}{4^j - 1} = R_{i,j-1} + \frac{R_{i,j-1} - R_{i-1,j-1}}{4^j - 1}, \quad 1 \leq j \leq i$$

Implement this, using these three formulas plus the above function for the composite midpoint rule.

One natural data structure is a 2D array with unused entries above the main diagonal. However, you might consider how to store this triangular collection of data as a list of lists, successively of lengths 1, 2 and so on up to n .

31.3.4 Final exercise: complete the notebook

Add further test cases; one interesting type of example is a periodic function.

RANDOM NUMBERS, HISTOGRAMS, AND A SIMULATION

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

32.1 Introduction

Random numbers are often useful both for simulation of physical processes and for generating a collection of test cases. Here we will do a mathematical simulation: approximating π on the basis that the unit circle occupies a fraction $\pi/4$ of the 2×2 square enclosing it.

32.1.1 Disclaimer

Actually, the best we have available is **pseudo-random** numbers, generated by algorithms that actually produce a very long but eventually repeating sequence of numbers.

32.2 Module `random` within package `numpy`

The pseudo-random number generator we use are provided by package Numpy in its module `random` – full name `numpy.random`. This module contains numerous random number generators; here we look at just a few.

We introduce the abbreviation “npr” for this, along with the standard abbreviations “np” for `numpy` and “plt” for module `matplotlib.pyplot` within package `matplotlib`

```
import numpy.random as npr
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

32.3 Uniformly distributed real numbers: `numpy.random.rand`

First, the function `rand` (full name `numpy.random.rand`) generates uniformly-distributed real numbers in the semi-open interval $[0, 1)$.

To generate a single value, use it with no argument:

```
n_samples = 4
for sample in range(n_samples):
    print(npr.rand())
```

```
0.6831120389814106
0.5548018157238266
0.8695626004658878
0.3752686546706374
```

32.3.1 Arrays of random values

To generate an array of values all at once, one can specify how many as the first and only input argument:

```
pseudorandomnumbers = npr.rand(n_samples)
print(pseudorandomnumbers)
```

```
[0.62144348 0.03793401 0.13891253 0.84806369]
```

However, the first method has an advantage in some situations: neither the whole list of integers from 0 to `n_samples - 1` nor the collection of random numbers is stored at any time: instead, just one value at a time is provided, used, and then “forgotten”. This can be beneficial or even essential when a very large number of random values is used; it is not unusual for a simulation to require more random values than the computer’s memory can hold.

32.3.2 Multi-dimensional arrays of random values

We can also generate multi-dimensional arrays, by giving the lengths of each dimension as arguments:

```
numbers2d = npr.rand(2,3)
print('A two-dimensional array of random numbers:\n', numbers2d)
numbers3d = npr.rand(2,3,4)
print('\nA three-dimensional array of random numbers:\n', numbers3d)
```

```
A two-dimensional array of random numbers:
[[0.18395217 0.71429007 0.68437103]
 [0.00242336 0.92888447 0.14129646]]

A three-dimensional array of random numbers:
[[[0.2511712  0.85518662 0.36048736 0.11214513]
  [0.00948166 0.91241279 0.07428897 0.18599767]
  [0.75078587 0.31421873 0.00762302 0.15589679]]

 [[0.78636402 0.48269234 0.08894184 0.33770037]
  [0.66866924 0.85747635 0.82185659 0.83088524]
  [0.98993293 0.10828591 0.78003614 0.53410965]]]
```


32.4 Normally distributed real numbers: `numpy.random.randn`

The function `randn` has the same interface, but generates numbers with the standard normal distribution of mean zero, standard deviation one:

```
print('Ten normally distributed values:\n', npr.randn(20))
```

```
Ten normally distributed values:
[ 1.01125557 -0.51308499  1.47690102 -0.1352893   0.03629515  1.06888055
 0.17867389  0.03297391  1.62309016  0.38036881 -1.45193537 -0.16739935
 1.52999647 -0.24956606  0.62335861 -0.23251452  0.4546426  -0.61267693
 1.24106299  0.52917226]
```

```
n_samples = 10**7
normf_samples = npr.randn(n_samples)
mean = sum(normf_samples)/n_samples
print('The mean of these', n_samples, 'samples is', mean)
standard_deviation = np.sqrt(sum(normf_samples**2)/n_samples - mean**2)
print('and their standard deviation is', standard_deviation)
```

```
The mean of these 10000000 samples is 0.00032428885458364956
and their standard deviation is 1.0002386370861691
```

Note The exact mean and standard deviation of the standard normal distribution are 0 and 1 respectively, so the slight variations above are due to these being only a sample mean and sample standard deviation.

32.5 Histogram plots with `matplotlib.random.hist`

`matplotlib.random` has a function `hist(x, bins, ...)` for plotting histograms, so we can check what this normal distribution actually looks like.

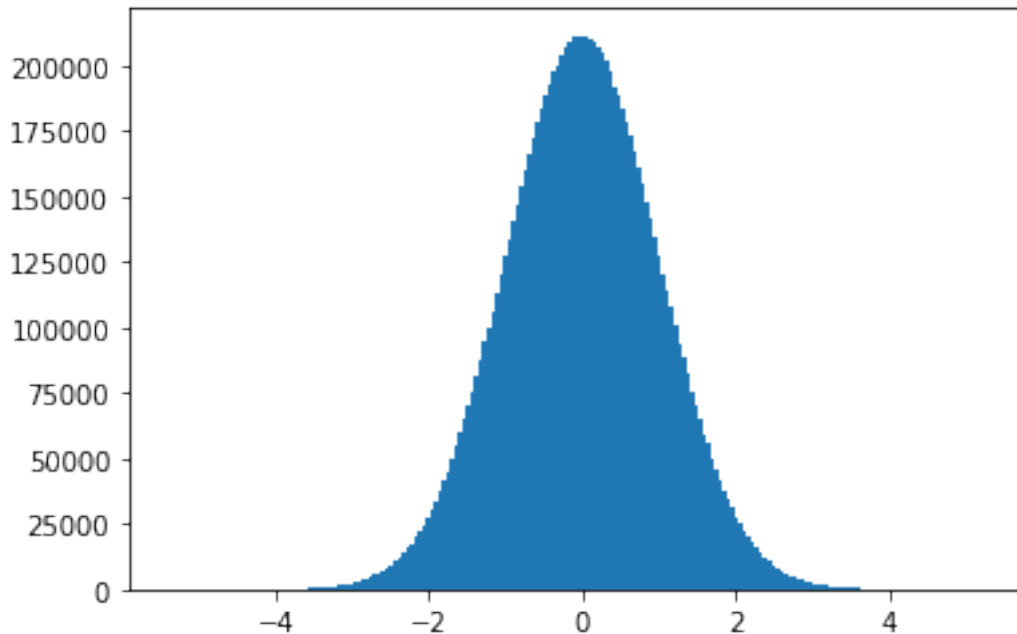
Input parameter `x` is the list of values, and when input parameter `bins` is given an integer value, the data is binned into that many equally wide intervals.

The function `hist` also returns three values:

- `n`, the number of values in each bin (the bar heights on the histogram)
- `bins` (which I prefer to call `bin_edges`), the list of values of the edges between the bins
- `patches`, which we can ignore!

It is best to assign this output to variables; otherwise the numerous values are sprayed over the screen.

```
# Note: the three output values must be assigned to variables, even though we do not
      ↪ need them here.
(n, bin_edges, patches) = plt.hist(normf_samples, 200)
```



32.6 Random integers: `numpy.random.randint`

One can generate pseudo-random integers, uniformly distributed between specified lower and upper values.

```
n_dice = 60
dice_rolls = npr.randint(1, 6+1, n_dice)
print(n_dice, 'random dice rolls:\n', dice_rolls)
# Count each outcome: this needs a list instead of an array:
dice_rolls_list = list(dice_rolls)
for value in (1, 2, 3, 4, 5, 6):
    count = dice_rolls_list.count(value)
    print(value, 'occured', count, 'times')
```

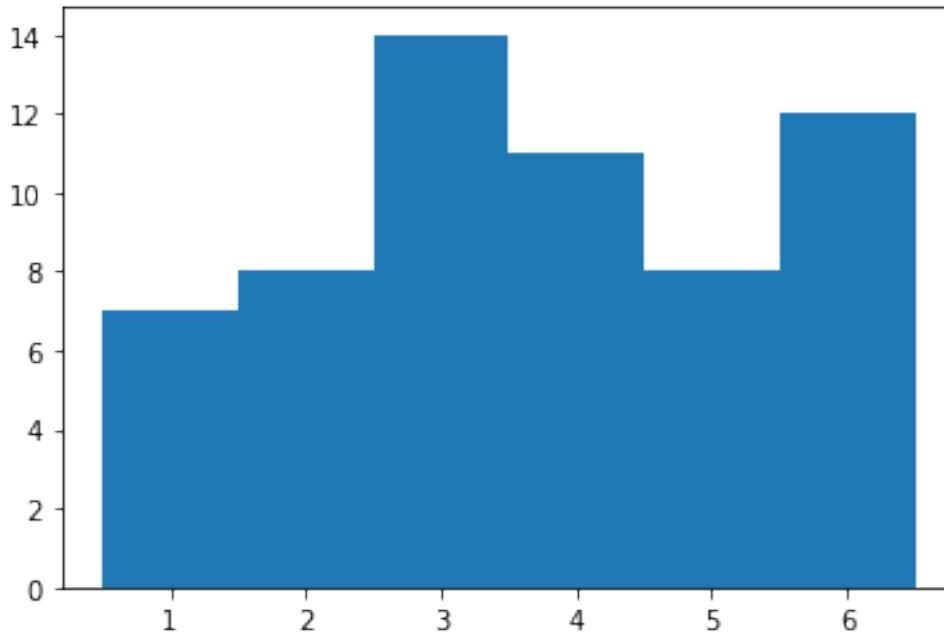
```
60 random dice rolls:
[4 3 2 3 3 3 3 4 1 6 5 4 4 6 6 3 2 6 5 2 5 4 3 1 4 4 1 1 2 2 3 4 3 4 1 6 1
 3 6 6 6 5 6 6 2 5 6 4 1 3 2 5 4 5 2 3 6 3 3 5]
1 occured 7 times
2 occured 8 times
3 occured 14 times
4 occured 11 times
5 occured 8 times
6 occured 12 times
```

32.6.1 Specifying bin edges for the histogram

This time, it is best to explicitly specify a list of the edges of the bins, by making the second argument `bins` a list. With six values, seven edges are needed, and it looks nicest if they are centered on the integers.

```
bin_edges = [0.5, 1.5, 2.5, 3.5, 4.5, 5.5, 6.5]
```

```
(n, bin_edges, patches) = plt.hist(dice_rolls, bin_edges)
```

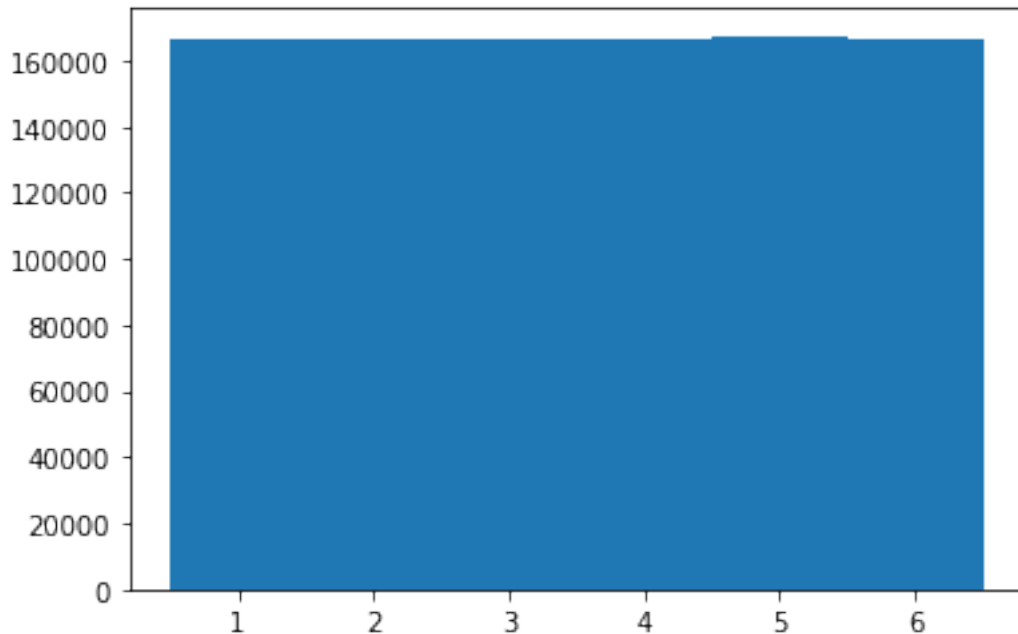


Run the above several times, redrawing the histogram each time; you should see a lot of variation.

Things average out with more rolls:

```
n_dice = 10**6
dice_rolls = npr.randint(1, 6+1, n_dice)
# Count each outcome: this needs a list instead of an array:
dice_rolls_list = list(dice_rolls)
for value in (1, 2, 3, 4, 5, 6):
    count = dice_rolls_list.count(value)
    print(value, 'occured a fraction', count/n_dice, 'of the time')
(n, bin_edges, patches) = plt.hist(dice_rolls, bins = bin_edges)
```

```
1 occured a fraction 0.166567 of the time
2 occured a fraction 0.166454 of the time
3 occured a fraction 0.166568 of the time
4 occured a fraction 0.16633 of the time
5 occured a fraction 0.167526 of the time
6 occured a fraction 0.166555 of the time
```



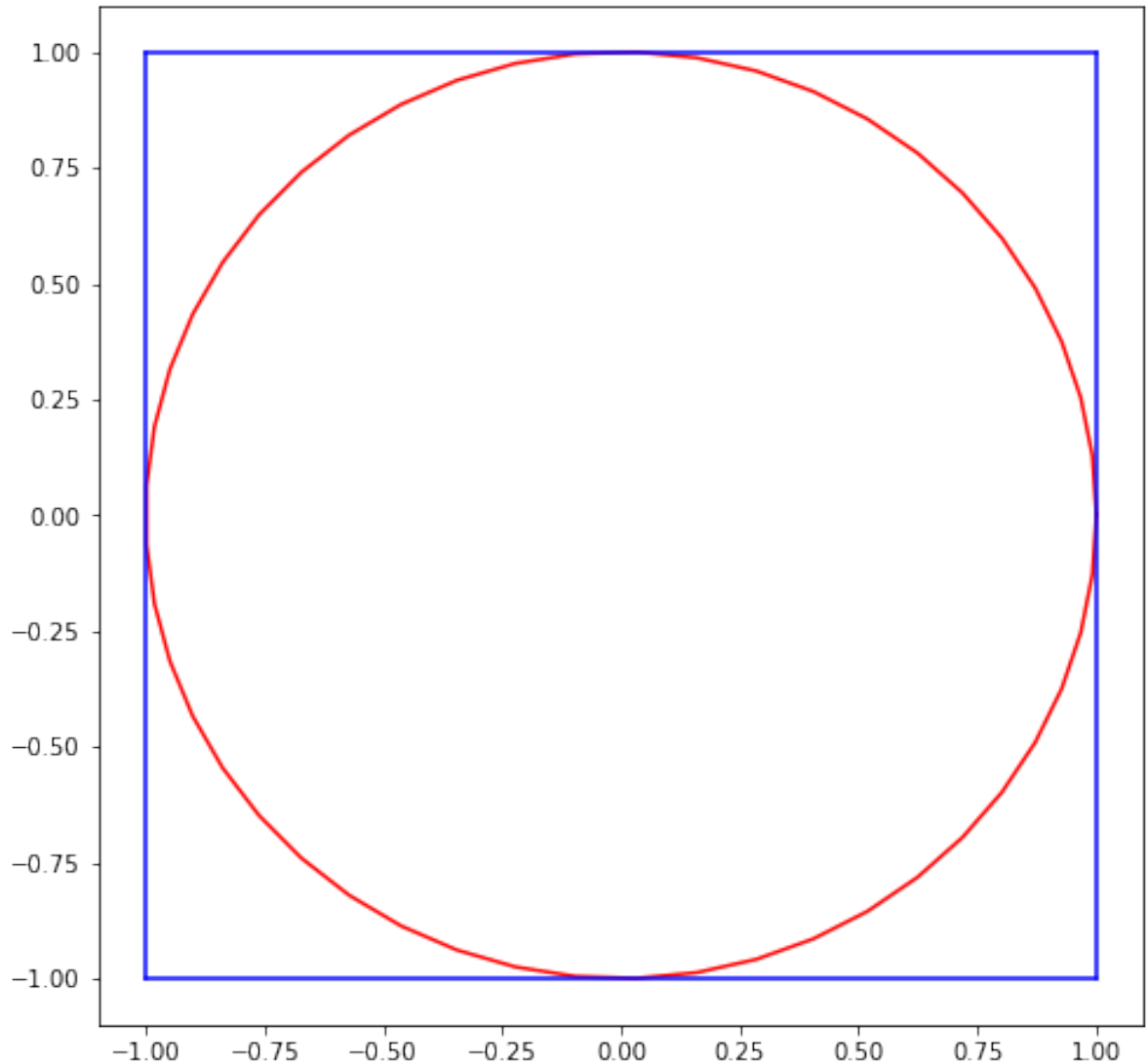
The histogram is now visually boring, but mathematically more satisfying.

Exercise A: approximating π

We can compute approximations of π by using the fact that the unit circle occupies a fraction $\pi/4$ of the circumscribed square:

```
plt.figure(figsize=[8, 8])
angle = np.linspace(0, 2*np.pi)
# Red circle
plt.plot(np.cos(angle), np.sin(angle), 'r')
# Blue square
plt.plot([-1,1], [-1,-1], 'b') # bottom side of square
plt.plot([1,1], [-1,1], 'b') # right side of square
plt.plot([1,-1], [1,1], 'b') # top side of square
plt.plot([-1,-1], [1,-1], 'b') # left side of square
```

```
[<matplotlib.lines.Line2D at 0x7fc7ea44be10>]
```



We can use this fact as follows:

- generate a list of N random points in the square $[-1, 1] \times [-1, 1]$ that circumscribes the unit circle, by generating successive uniformly distributed random values for both the x and y coordinates.
- compute the fraction of these that are inside the unit circle, which should be approximately $\pi/4$. (N needs to be fairly large; you could try $N = 100$ initially, but will increase it later.)
- Multiply by four and there you are!

Do multiple trials with the same number N of samples, to see the variation as an indication of accuracy.

Collect the various approximations of π in a list, compute the (sample) mean and (sample) standard deviation of the list, and illustrate with a histogram.

Exercise B

It takes a lot of samples to get decent accuracy, so after part (a) is working, experiment with successively more samples; increase the number N of samples per trial by big steps, like factors of 100.

For each choice of sample size N , compute the mean and standard deviation, and plot the histogram.

FORMATTED OUTPUT AND SOME TEXT STRING MANIPULATION

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

- *The function `print()`*
- *f-string formatting*
- *Formatting numbers*
- *Further reading*

33.1 The function `print()`

The basic tool for displaying information from Python to the screen is the function `print()`, as in

```
print("Hello world.")
```

```
Hello world.
```

What this actually does is convert one or more items given as its input arguments to strings of text, and then displays them on a line of output, each separated from the next by a single space. So it also handles numbers:

```
print(7)
print(1/3)
```

```
7
0.3333333333333333
```

and variables or expressions containing text strings, numbers or other objects:

```
x = 1
y = 2
print(x, "+", y, "=", x+y)
```

```
1 + 2 = 3
```

```
hours = [7, 11]
claim = "The hours of opening of our stores are"
print(claim, hours)
print("That is, from", hours[0], "am to", hours[1], "pm.")
```

```
The hours of opening of our stores are [7, 11]
That is, from 7 am to 11 pm.
```

33.2 F-string formatting

When assembling multiple pieces of information, the above syntax can get messy; also, it automatically inserts a blank space between items, so does not allow us to avoid the space before “am” and “pm”.

Python has several methods for finer string manipulation; my favorite is *f-strings*, introduced in Python version 3.6, so I will describe only it. (The two older approaches are the “%” operator and the `.format()` method; if you want to know about them — if only for a reading knowledge of code that uses them — there is a nice overview at <https://realpython.com/python-string-formatting/>)

The key idea of f-strings is that when an “f” is added immediately before the opening quote of a string, parts of that string within braces {...} are taken as input which is processed and the results inserted into a modified string. For example, the previous print command above could instead be done with precise control over blank spaces as follows:

```
print(f"That is, from {hours[0]}am to {hours[1]}pm.")
```

```
That is, from 7am to 11pm.
```

Actually, what happens here involves two steps:

1. The “f-string” is converted to a simple, literal string.
2. function `print(...)` then prints this literal string, as it would with any string.

Thus we can do things in two parts:

```
hours_of_operation = f"Our stores are open from {hours[0]}am till {hours[1]}pm."
print(hours_of_operation)
```

```
Our stores are open from 7am till 11pm.
```

Sometimes this is useful, because strings get used in other places besides `print()`, such as the titles and labels of a graph, and as we will see next, it can be convenient to assemble a statement piece at a time and then print the whole thing; for this, we use “addition” of strings, which is concatenation.

Note also the explicit insertion of spaces and such.

```
q1 = "When do stores open?"
a1 = f"Stores open at {hours[0]}am."
q2 = "When do stores close?"
a2 = f"Stores close at {hours[1]}pm."
faq = q1+' '+a1+' - '+q2+' '+a2
print(faq)
```



```
When do stores open? Stores open at 7am. – When do stores close? Stores close at 11pm.
```

33.3 Formatting numbers

The information printed above came out alright, but there are several reasons we might want finer control over the display, especially with real numbers (type `float`)

- Choosing the number of significant digits or correct decimals to display for a `float` (real number).
- Controlling the width a number's display (for example, in order to line up columns).

First, width control. The following is slightly ugly due to the shifts right.

```
for exponent in range(11):
    print(f"4^{exponent} = {4**exponent}")
```

```
4^0 = 1
4^1 = 4
4^2 = 16
4^3 = 64
4^4 = 256
4^5 = 1024
4^6 = 4096
4^7 = 16384
4^8 = 65536
4^9 = 262144
4^10 = 1048576
```

To line things up, we can specify that each output item has as many spaces as the widest of them needs: 2 and 7 columns respectively, with syntax `{quantity:width}`

```
for exponent in range(11):
    print(f"4^{exponent:2} = {4**exponent:7}")
```

```
4^ 0 =      1
4^ 1 =      4
4^ 2 =     16
4^ 3 =     64
4^ 4 =    256
4^ 5 =   1024
4^ 6 =   4096
4^ 7 =  16384
4^ 8 =  65536
4^ 9 = 262144
4^10 =1048576
```

That is still a bit strange with the exponents, because the output is right-justified. Left-justified would be better, and is done with a `<` before the width. (As you might guess, `>` can be used to explicitly specify right-justification).

```
for exponent in range(11):
    print(f"4^{exponent:<2} = {4**exponent:>7}")
```

```
4^0 = 1
4^1 = 4
4^2 = 16
4^3 = 64
4^4 = 256
4^5 = 1024
4^6 = 4096
4^7 = 16384
4^8 = 65536
4^9 = 262144
4^10 = 1048576
```

Next, dealing with `float` (real) numbers: alignment, significant digits, and scientific notation vs fixed decimal form.

Looking at:

```
for exponent in range(11):
    print(f"4^{exponent:<2} = {1/4**exponent}")
```

```
4^-0 = 1.0
4^-1 = 0.25
4^-2 = 0.0625
4^-3 = 0.015625
4^-4 = 0.00390625
4^-5 = 0.0009765625
4^-6 = 0.000244140625
4^-7 = 6.103515625e-05
4^-8 = 1.52587890625e-05
4^-9 = 3.814697265625e-06
4^-10 = 9.5367431640625e-07
```

... we see several things:

- real numbers are by default left justified, as opposed to the default right justification of integers.
- some automatic decision is made about when to use scientific notation, where the part after “e” is the exponent (base 10). [Roughly the shorter of the two options is used.]
- The number of significant digits can be excessive: the full 16 decimal place precision of 64-bit numbers is often not needed or wanted.

These can be adjusted. Justification was already mentioned, so the next new feature is an optional letter after the width value, to specify more about how to format the data:

- ‘d’ specifies an integer (but this is the default when the value is known to be of type `int`)
- ‘f’ specifies fixed width format for a `float`; no exponent
- ‘e’ specifies scientific notation format for a `float`; with exponent
- ‘g’ specifies that a `float` is formatted either ‘f’ or ‘e’, based on criteria like which conveys the most precision in a given amount of space.

All three format specifiers for type `float` also impose a lower limit on how many digits are displayed; about seven:

```
for exponent in range(11):
    print(f"4^{exponent:<2d} = {1/4**exponent:f}")
```

```

4^-0 = 1.000000
4^-1 = 0.250000
4^-2 = 0.062500
4^-3 = 0.015625
4^-4 = 0.003906
4^-5 = 0.000977
4^-6 = 0.000244
4^-7 = 0.000061
4^-8 = 0.000015
4^-9 = 0.000004
4^-10 = 0.000001

```

```

for exponent in range(11):
    print(f"4^-{exponent:<2} = {1/4**exponent:e}")

```

```

4^-0 = 1.000000e+00
4^-1 = 2.500000e-01
4^-2 = 6.250000e-02
4^-3 = 1.562500e-02
4^-4 = 3.906250e-03
4^-5 = 9.765625e-04
4^-6 = 2.441406e-04
4^-7 = 6.103516e-05
4^-8 = 1.525879e-05
4^-9 = 3.814697e-06
4^-10 = 9.536743e-07

```

```

for exponent in range(11):
    print(f"4^-{exponent:<2} = {1/4**exponent:g}")

```

```

4^-0 = 1
4^-1 = 0.25
4^-2 = 0.0625
4^-3 = 0.015625
4^-4 = 0.00390625
4^-5 = 0.000976562
4^-6 = 0.000244141
4^-7 = 6.10352e-05
4^-8 = 1.52588e-05
4^-9 = 3.8147e-06
4^-10 = 9.53674e-07

```

To control precision, the width specifier gains a “decimal” `p`, with `{...:w.p}` specifying `p` decimal places or significant digits, depending on context. Also, the width can be omitted if only precision matters, not spacing.

Let’s ask for 9 digits:

```

for exponent in range(11):
    print(f"4^-{exponent:<2d} = {1/4**exponent:.9f}")

```

```

4^-0 = 1.000000000
4^-1 = 0.250000000
4^-2 = 0.062500000

```

(continues on next page)

(continued from previous page)

```

4^-3 = 0.015625000
4^-4 = 0.003906250
4^-5 = 0.000976562
4^-6 = 0.000244141
4^-7 = 0.000061035
4^-8 = 0.000015259
4^-9 = 0.000003815
4^-10 = 0.000000954

```

```

for exponent in range(11):
    print(f"4^-{exponent:<2} = {1/4**exponent:.9e}")

```

```

4^-0 = 1.000000000e+00
4^-1 = 2.500000000e-01
4^-2 = 6.250000000e-02
4^-3 = 1.562500000e-02
4^-4 = 3.906250000e-03
4^-5 = 9.765625000e-04
4^-6 = 2.441406250e-04
4^-7 = 6.103515625e-05
4^-8 = 1.525878906e-05
4^-9 = 3.814697266e-06
4^-10 = 9.536743164e-07

```

Putting it all together, here are width and precision specified.

Note that text string pieces can also have width specifications; for example, to align columns:

```

print(f"{'':8}{'fixed point':15}{'scientific notation':16}")
for exponent in range(11):
    print(f"4^-{exponent:<2} = {1/4**exponent:11.9f}, = {1/4**exponent:0.9e}")

```

	fixed point	scientific notation
4^-0	= 1.000000000,	= 1.000000000e+00
4^-1	= 0.250000000,	= 2.500000000e-01
4^-2	= 0.062500000,	= 6.250000000e-02
4^-3	= 0.015625000,	= 1.562500000e-02
4^-4	= 0.003906250,	= 3.906250000e-03
4^-5	= 0.000976562,	= 9.765625000e-04
4^-6	= 0.000244141,	= 2.441406250e-04
4^-7	= 0.000061035,	= 6.103515625e-05
4^-8	= 0.000015259,	= 1.525878906e-05
4^-9	= 0.000003815,	= 3.814697266e-06
4^-10	= 0.000000954,	= 9.536743164e-07

Another observation: the last text string ‘scientific notation’ was too long for the specified 16 columns, and the “e” format for the second version of the power definitely needed more than the 0 columns requested. So they went over, rather than getting truncated — the width specification is just a minimum.

33.3.1 Further reading, and a P. S.:

- <https://realpython.com/python-f-strings/>
- As always, consider the function `help()`, as below

```
help(print)
```

Help on built-in function print in module builtins:

```
print(...)
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

    Prints the values to a stream, or to sys.stdout by default.
    Optional keyword arguments:
    file: a file-like object (stream); defaults to the current sys.stdout.
    sep: string inserted between values, default a space.
    end: string appended after the last value, default a newline.
    flush: whether to forcibly flush the stream.
```

By default a single space is inserted between items.

You can change that with the optional argument “sep”:

```
print(1, "+", 1, '=', 1+1, sep="")
```

```
1+1=2
```

```
print(1, "+", 1, '=', 1+1, sep=" ... ")
```

```
1 ... + ... 1 ... = ... 2
```

Also, by default the output line is ended at the end of a `print` command; technically, an “end of line” character “`\n`” is added at the end of the output string.

One can change this with the optional argument “end”, for example to allow a single line to be assembled in pieces, or to specify double spacing.

No new line after print:

```
for count_to_10 in range(1, 11):
    print(count_to_10, sep="", end="")
    if count_to_10 < 10: # there is more to come after this item
        print(" ... ", sep="", end="")
    else: print() # end the line at last
```

```
1 ... 2 ... 3 ... 4 ... 5 ... 6 ... 7 ... 8 ... 9 ... 10
```

Double spacing:

```
for count_to_10 in range(1, 11):
    print(count_to_10, end="\n\n")
```

1

2

3

4

5

6

7

8

9

10

CLASSES, OBJECTS, ATTRIBUTES, METHODS: VERY BASIC OBJECT-ORIENTED PROGRAMMING IN PYTHON

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

This is a very basic introduction to object-oriented programming in Python: defining *classes* and using them to create *objects* with *methods* (a cousin of functions) that act on those objects.

These are illustrated with objects that are vectors, and in particular 3-component vectors that have a cross product.

The first class will be for vectors with three components, labeled 'x', 'y' and 'z'.

On its own, this would be rather redundant, since numpy arrays could be used, but it serves to introduce some basic ideas, and then prepare for the real goal: 3-vectors with cross product.

34.1 Example A: Class `VeryBasic3Vector`

This first class `VeryBasic3Vector` illustrates some basic features of creating and using classes; however, it will be superseded soon!

Almost every class has a method with special name `__init__`, which is used to create objects of this class. In this case, `__init__` sets the three *attributes* on a `VeryBasic3Vector` — its x, y, and z components.

This class has just one other method, for the scalar (“dot”) product of two such vectors.

```
class VeryBasic3Vector():
    """A first, minimal class for 3-component vectors, offering just creation and the
    scalar ("dot") product."""
    def __init__(self, values):
        self.x = values[0]
        self.y = values[1]
        self.z = values[2]
    def dot(self, other):
        """a.dot(b) gives the scalar ("dot") product of a with b"""
        return self.x * other.x + self.y * other.y + self.z * other.z
```

Create a couple of `VeryBasic3Vector` objects:

```
a = VeryBasic3Vector([1, 2, 3])
print(f"The x, y, and z attributes of object a are {a.x}, {a.y} and {a.z}")
```

```
b = VeryBasic3Vector([4, 5, 2])
print(f"Object b contains the vector <{b.x}, {b.y}, {b.z}>")
```

This way of printing values one attribute at a time gets tiresome, so soon, an alternative will be introduced, in improved class `BasicNVector`

The attributes of an object can also be set directly:

```
a.y = 5
print(f"a is now <{a.x}, {a.y}, {a.z}>")
```

Methods are used as follows, with the variable before the “.” part being `self` and any parenthesized variables being the arguments to the method definition.

```
print(f'The scalar ("dot") product of a and b is {a.dot(b)}')
```

34.2 Example B: Class `BasicNVector`

A second class `BasicNVector` with some improvements over the above class `VeryBasic3Vector`:

- Allowing vectors of any length, “N”
- Methods for addition, subtraction and vector-by-scalar multiplication.
- The special method `__str__` (using this name is mandatory) to output an object’s values as a string — for using in `print`, for example.

Aside: here and below, the names of these special methods like `__str__` start and end with a *pair* of underscores, “__”.

```
class BasicNVector():
    """This improves on class VeryBasic3Vector by:
    - allowing any number of components,
    - adding several methods for vector arithmetic, and
    - defining the special method __str__() to help display the vector's value."""

    # First mimic the definitions in class VeryBasic3Vector:

    def __init__(self, list_of_components):
        self.list = list_of_components.copy()

    def dot(self, other):
        """a.dot(b) gives the scalar ("dot") product of a with b"""
        dot_product = 0
        for i in len(self):
            dot_product += self.list[i] * other.list[i]
        return dot_product

    # Next, some new stuff not in class VeryBasic3Vector

    def times(self, scale):
        """self.times(scale) gives the product of vector "self" by scalar "scale"""
        return BasicNVector([scale * component for component in self.list]) # This
        # uses a list comprehension; notes on comprehensions are coming soon!

    # The special method names wrapped in double underscores, __add__, __sub__ and __
    # __str__,
    # have special meanings, as will be revealed below.
```

(continues on next page)

(continued from previous page)

```

def __add__(self, other): # Vector addition - with definition of the "+"_
    ↪operation for pairs of BasicNVector objects
    return BasicNVector([ self.list[i] + other.list[i] for i in range(len(self.
    ↪list)) ])

def __sub__(self, other): # Vector subtraction - with definition of the "-"_
    ↪operation for pairs of BasicNVector objects
    return BasicNVector([ self.list[i] - other.list[i] for i in range(len(self.
    ↪list)) ])

def __str__(self):
    """How to convert the value to a text string.
    As above, this uses angle brackets <...> for vectors, to distinguish from_
    ↪lists [...] and tuples (...)"""
    string = '<'
    for component in self.list[:-1]:
        string += f'{component}, '
    string += f'{self.list[-1]}>'
    return string

```

We need to create new objects of class BasicNVector to use these new methods:

```

c = BasicNVector([1, 2, 3, 4])
d = BasicNVector([4, 5, 2, 3])

```

The new method “__str__” makes it easier to display the value of a BasicNVector, by just using print:

```

print("c = ", c)

```

And now we can try the other new methods:

```

c_times_3 = c.times(3)

```

```

print(f"{c} times 3 is {c_times_3}")

```

```

<1, 2, 3, 4> times 3 is <3, 6, 9, 12>

```

Here is the usual way of using the mysteriously-names method “__add__” ...

```

e = c.__add__(d)
print(f"{c} + {d} = {e}")

```

```

<1, 2, 3, 4> + <4, 5, 2, 3> = <5, 7, 5, 7>

```

... but that special name also means that it also specifies how the operation “+” works on a pair of BasicNVector objects:

```

f = c + d
print(f'{c} + {d} = {f}')

```

```

<1, 2, 3, 4> + <4, 5, 2, 3> = <5, 7, 5, 7>

```

Likewise for subtraction with __sub__:

```
print(f'{c} - {d} = {c - d}')
```

```
<1, 2, 3, 4> - <4, 5, 2, 3> = <-3, -3, 1, 1>
```

34.3 Inheritance: new classes that build on old ones

A new class can be defined by refining an existing one, by adding methods and such, to avoid defining everything from scratch. The basic syntax for creating class `ChildClass` based on an existing *parent class* named `ParentClass` is

```
class ChildClass(ParentClass):
```

Here we define class `Vector3`, which is restricted to vectors with 3 components, and uses that restriction to allow defining the vector cross product.

In addition, it makes the operator “*” do cross multiplication on such objects, by also defining the special method `__mul__`:

```
class Vector3(BasicNVector):
    """Restrict to BasicNVector objects of length 3, and then add the vector cross_
    product"""
    def __init__(self, list_of_components):
        if len(list_of_components) == 3:
            super().__init__(list_of_components)
            # Aside: function "super()" gives the parent class, so the above is_
            # equivalent to
            #BasicNVector.__init__(self, list_of_components)
        else: # Complain!
            raise ValueError('The length of a Vector3 object must be 3.')

    def cross_product(self, other): # the vector cross product
        (x1, y1, z1) = self.list
        (x2, y2, z2) = other.list
        return Vector3([ y1*z2 - y2*z1, z1*x2 - z2*x1, x1*y2 - x2*y1] )

    # Add a synonym: now it redefines the multiplication operator "*"
    # I could have just used the name `__mul__` instead of `cross_product` above;
    # I did it this was to also allow the more descriptive name `cross_product`, and_
    # to illustrate the use of synonyms for functions.
    __mul__ = cross_product
```

Again, we need some `Vector3` objects; the above `BasicNVector` objects do not know about the cross product.

But note that the previously-defined methods for class `BasicNVector` also work for `Vector3` objects, so for example we can still print with the help of method `__str__` from there.

```
u = Vector3([1, 2, 3])
v = Vector3([4, 5, 10])
print(f'The vector cross product of {u} with {v} is {u.cross_product(v)}')
print(f'The vector cross product of {v} with {u} is {v*u}')
```

```
The vector cross product of <1, 2, 3> with <4, 5, 10> is <5, 2, -3>
The vector cross product of <4, 5, 10> with <1, 2, 3> is <-5, -2, 3>
```

This is what happens with inappropriate input, thanks to that `raise` command:

```
w = Vector3([1, 2, 3, 4])
```

```
-----
ValueError                                Traceback (most recent call last)
<ipython-input-16-4dc0f24dc07a> in <module>
----> 1 w = Vector3([1, 2, 3, 4])

<ipython-input-14-a85930062fea> in __init__(self, list_of_components)
      7         #BasicNVector.__init__(self, list_of_components)
      8     else: # Complain!
----> 9         raise ValueError('The length of a Vector3 object must be 3.')
     10
     11     def cross_product(self, other): # the vector cross product

ValueError: The length of a Vector3 object must be 3.
```

Aside: That's ugly: as seen in the notes on *Exceptions and Exception Handling*, a more careful usage would be:

```
try:
    w = Vector3([1, 2, 3, 4])
except Exception as what_just_happened:
    print(f"Well, at least we tried, but: '{what_just_happened.args[0]}'.")
```

```
Well, at least we tried, but: 'The length of a Vector3 object must be 3.'
```


EXCEPTIONS AND EXCEPTION HANDLING

This work is licensed under [Creative Commons Attribution-ShareAlike 4.0 International](#)

35.1 Introduction

This unit addresses the almost inevitable occurrence of unanticipated errors in code, and methods to detect and handle *exceptions*.

Note: This “notebook plus modules” organization is useful when the collection of function definitions used in a project is lengthy, and would otherwise clutter up the notebook and hamper readability.

35.2 Handling and Raising Exceptions

Ideally, when we write a computer program for a mathematical task, we will plan in advance for every possibility, and design an algorithm to handle all of them. For example, anticipating the possibility of division by zero and avoiding it is a common issue in making a program **robust**.

However this is not always feasible; in particular while still developing a program, there might be situations that you have not yet anticipated, and so it can be useful to write a program that will detect problems that occur while the program is running, and handle them in a reasonable way.

We start by considering a very basic code for our favorite example, solving quadratic equations.

```
from math import sqrt
```

```
def quadratic_roots(a, b, c):  
    root_of_discriminant = sqrt(b**2 - 4*a*c)  
    return ( (-b - root_of_discriminant)/(2*a), (-b + root_of_discriminant)/(2*a) )
```

```
# An easy and familiar test case  
(a, b, c) = (2, -10, 8)  
print("Let's solve the quadratic equation %g*x**2 + %g*x + %g = 0" % (a, b, c))  
(root0, root1) = quadratic_roots(a, b, c)  
print("The roots are %g and %g" % (root0, root1))
```

```
Let's solve the quadratic equation 2*x**2 + -10*x + 8 = 0  
The roots are 1 and 4
```

Try it repeatedly, with some “destructive testing”: seek input choices that will cause various problems.

For this, it is useful to have an interactive loop to ask for test cases:

```
# Testing: add some hard cases, interactively
print("Let's solve some quadratic equations  $a*x^2 + b*x + c = 0$ ")
keepgoing = True
while keepgoing:
    a = float(input("a = "))
    b = float(input("b = "))
    c = float(input("c = "))
    print("Solving the quadratic equation  $%g*x^2 + %g*x + %g = 0$ " % (a, b, c))
    (root0, root1) = quadratic_roots(a, b, c)
    print("The roots are %g and %g" % (root0, root1))
    yesorno = input("Do you want to try another case? [Answer y/n]: ")
    keepgoing = yesorno[0] in {'y', 'Y'} # examine the first letter only!
```

Let me know what problems you found; we will work on detecting and handling all of them.

Some messages I get ended with these lines, whose meaning we will explore:

- ZeroDivisionError: float division by zero
- ValueError: math domain error
- ValueError: could not convert string to float ...

35.3 Catching any “exceptional” situation and handling it specially

Here is a minimal way to catch all problems, and at least apologize for failing to solve the equation:

```
# Exception handling, version 1
print("Let's solve some quadratic equations  $a*x^2 + b*x + c = 0$ ")
keepgoing = True
while keepgoing:
    try:
        a = float(input("a = "))
        b = float(input("b = "))
        c = float(input("c = "))
        print("Solving the quadratic equation  $%g*x^2 + %g*x + %g = 0$ " % (a, b, c))
        (root0, root1) = quadratic_roots(a, b, c)
        print("The roots are %g and %g" % (root0, root1))
    except:
        print("Something went wrong; sorry!")
    yesorno = input("Do you want to try another case? [Answer y/n]: ")
    keepgoing = yesorno[0] in {'y', 'Y'} # examine the first letter only, so
    ↪ "nevermore" means "no"
```

35.4 This try/except structure does two things:

- it first *tries* to run the code in the (indented) block introduced by the colon after the statement `try`
- if anything goes wrong (in Python jargon, if any **exception** occurs) it gives up on that `try` block and runs the code in the block under the statement `except`.

35.5 Catching and displaying the exception error message

One thing has been lost though: the messages like “float division by zero” as seen above, which say what sort of exception occurred.

We can regain some of that, by having `except` statement save that message into a variable:

```
# Exception handling, version 2: displaying the exception

print("Let's solve some quadratic equations  $a*x^2 + b*x + c = 0$ ")
keepgoing = True
while keepgoing:
    try:
        a = float(input("a = "))
        b = float(input("b = "))
        c = float(input("c = "))
        print("Solving the quadratic equation  $%g*x^2 + %g*x + %g = 0$ " % (a, b, c))
        (root0, root1) = quadratic_roots(a, b, c)
        print("The roots are %g and %g" % (root0, root1))
    except Exception as what_just_happened:
        print("Something went wrong; sorry!")
        print("The exception is: ", what_just_happened)
        print("The exception message is: ", what_just_happened.args[0])
        #if what_just_happened == "float division by zero":
        #if what_just_happened[:5] == "float":
        #    print("You cannot divide by zero.")
        yesorno = input("Do you want to try another case? [Answer y/n]: ")
        keepgoing = yesorno[0] in {'y', 'Y'} # examine the first letter only, so
        ↪ "nevermore" means "no"
```

This version detects every possible exception and handles them all in the same way, whether it be a problem in arithmetic (like the dreaded division by zero) or the user making a typing error in the input of the coefficients. Try answering “one” when asked for a coefficient!

35.6 Handling specific exception types

Python divides exceptions into many types, and the statement `except` can be given the name of an exception type, so that it then handles only that type of exception.

For example, in the case of division by zero, where we originally got a message

```
ZeroDivisionError: float division by zero
```

we can catch that particular exception and handle it specially:

```
# Exception handling, version 3: as above, but with special handling for division by
↪zero.

print("Let's solve some quadratic equations  $a*x^2 + b*x + c = 0$ ")
keepgoing = True
while keepgoing:
    try:
        a = float(input("a = "))
        b = float(input("b = "))
        c = float(input("c = "))
        print("Solving the quadratic equation  $%g*x^2 + %g*x + %g = 0$ " % (a, b, c))
        (root0, root1) = quadratic_roots(a, b, c)
        print("The roots are  $%g$  and  $%g$ " % (root0, root1))
    except ZeroDivisionError as what_just_happened: # Note the "CamelCase":↪
↪capitalization of each word
        print("Division by zero; the first coefficient cannot be zero!")
        print("Please try again.")
        yesorno = input("Do you want to try another case? [Answer y/n]: ")
        keepgoing = yesorno[0] in {'y', 'Y'} # examine the first letter only, so
↪"nevermore" means "no"
```

35.7 Handling multiple exception types

However, this still crashes with other errors, like typos in the input. To detect several types of exception, and handle each in an appropriate way, there can be a list of `except` statements, each with a block of code to run when that exception is detected. The type `Exception` was already seen above; it is just the totally generic case, so can be used as a catch-all after a list of exception types have been handled.

Experiment a bit, and you will see how these multiple `except` statements are used:

- the first `except` clause that matches is used, and any later ones are ignored;
- only if none matches does the code go back to simply “crashing”, as with version 0 above.

35.8 Summary: a while-try-except pattern for interactive programs

For programs with interactive input, a useful pattern for robustly handling errors or surprises in the input is a while-try-except pattern, with a form like:

```
try_again = True
while try_again:
    try:
        Get input
        Do stuff with it
        try_again = False
    except Exception as message:
        print("Exception", message, " occurred; please try again.")
        Maybe actually fix the problem, and if successful: try_again = False
```

One can refine this by adding `except` clauses for as many specific exception types as are relevant, with more specific handling for each.

35.8.1 Exercise D-A. Add handling for multiple types of exception

Copy your latest version of a “quadratic_solver” function here into your module name something like “math246” created for Unit 9) — or into a new file named something like `quadratic_solvers.py`. Then augment that function with multiple `except` clauses to handle all exceptions that we can get to occur.

First, read about the possibilities, for example in Section 5 of the official [Python 3 Standard Library Reference Manual](https://docs.python.org/3/library/exceptions.html) at <https://docs.python.org/3/library/exceptions.html>, or other sources that you can find.

Two exceptions of particular importance for us are `ValueError` and `ArithmeticError`, and sub-types of the latter like `ZeroDivisionError` and `OverflowError`. (Note the “CamelCase” capitalization of each word in an exception name: it is essential to get this right, since Python is case-sensitive.)

Aside: If you find a source on Python exceptions that you prefer to the above references, please let us all know!

35.9 Exercise D-B. Handling division by zero in Newton’s method

Using a basic code for Newton’s Method (such as the one I provide in module `root_finders`) experiment with exception handling for the possibility of division by zero.

(You could then do likewise with the Secant Method.)

Part V

Appendices

NOTEBOOK FOR GENERATING THE MODULE NUMERICMETHODS

Author: Brenton LeMesurier, lemesurierb@cofc.edu

A collection of functions implementing numerical algorithms from the Jupyter book *Introduction to Numerical Methods and Analysis with Python*, derived from material for the course *Elementary Numerical Analysis* at the University of Northern Colorado in Spring 2021.

Last updated on November 21, 2022

Recent additions; most recent first:

- `fitPolynomial`, `evaluatePolynomial`, `showPolynomial`
- a function `inverse` for computing the inverse of a matrix (but use this sparingly!)
- versions of Euler's Method and the Classical Runge-Kutta Method for systems.

36.1 Index

The three main sections (so far) are

- *Zero Finding*
- *Linear Algebra*
- *Solving Initial Value Problems for Ordinary Differential Equations*

```
import numpy as np
import matplotlib.pyplot as plt
```

36.2 Zero Finding: solving $f(x) = 0$ or $g(x) = x$

```
def bisection1(f, a, b, N, demoMode=False):
    '''Approximately solve equation f(x) = 0 in the interval [a, b] with N iterations
    of the Bisection Method.
    By the way, a triple-quoted multi-line comment like this provides convenient
    documentation about what a function does,
    so I encourage adding them. Try the command `help(bisection1)`
    '''
    c = (a + b)/2
    for iteration in range(N):
        if demoMode: print(f"\nIteration {iteration+1}:")
```

(continues on next page)

(continued from previous page)

```

    if f(a) * f(c) < 0:
        b = c
    else:
        a = c
    c = (a + b)/2
    if demoMode:
        print(f"The root is in interval [{a}, {b}]")
        print(f"The new approximation is {c}, with backward error {np.abs(f(c)):0.3}")
    root = c
    errorBound = (c-a)
    return (c, errorBound)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    def f(x): return x - np.cos(x)
    print("Solving by the Bisection Method.")
    (root, errorBound) = bisection1(f, a=-1, b=1, N=10, demoMode=True)
    print(f"\n{root=}, backward error {np.abs(f(root)):0.3}")

```

Solving by the Bisection Method.

```

Iteration 1:
The root is in interval [0.0, 1]
The new approximation is 0.5, with backward error 0.378

Iteration 2:
The root is in interval [0.5, 1]
The new approximation is 0.75, with backward error 0.0183

Iteration 3:
The root is in interval [0.5, 0.75]
The new approximation is 0.625, with backward error 0.186

Iteration 4:
The root is in interval [0.625, 0.75]
The new approximation is 0.6875, with backward error 0.0853

Iteration 5:
The root is in interval [0.6875, 0.75]
The new approximation is 0.71875, with backward error 0.0339

Iteration 6:
The root is in interval [0.71875, 0.75]
The new approximation is 0.734375, with backward error 0.00787

Iteration 7:
The root is in interval [0.734375, 0.75]
The new approximation is 0.7421875, with backward error 0.0052

Iteration 8:
The root is in interval [0.734375, 0.7421875]
The new approximation is 0.73828125, with backward error 0.00135

```

(continues on next page)

(continued from previous page)

```

Iteration 9:
The root is in interval [0.73828125, 0.7421875]
The new approximation is 0.740234375, with backward error 0.00192

Iteration 10:
The root is in interval [0.73828125, 0.740234375]
The new approximation is 0.7392578125, with backward error 0.000289

root=0.7392578125, backward error 0.000289

```

```

def fpi(g, x_0, errorTolerance=1e-6, maxIterations=20, demoMode=False):
    """Fixed point iteration for approximately solving  $x = f(x)$ ,
    x_0: the initial value
    """
    x = x_0
    for iteration in range(maxIterations):
        x_new = g(x)
        errorEstimate = np.abs(x_new - x)
        x = x_new
        if demoMode: print(f"x_{iteration} = {x}, {errorEstimate=:0.3}")
        if errorEstimate <= errorTolerance: break
    return (x, errorEstimate)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    def g(x): return np.cos(x)
    print("Fixed point iteration.")
    (root, errorEstimate) = fpi(g, 0, demoMode=True)
    print(f"\n{root=}, {errorEstimate=}")

```

```

Fixed point iteration.
x_0 = 1.0, errorEstimate=1.0
x_1 = 0.5403023058681398, errorEstimate=0.46
x_2 = 0.8575532158463933, errorEstimate=0.317
x_3 = 0.6542897904977792, errorEstimate=0.203
x_4 = 0.7934803587425655, errorEstimate=0.139
x_5 = 0.7013687736227566, errorEstimate=0.0921
x_6 = 0.7639596829006542, errorEstimate=0.0626
x_7 = 0.7221024250267077, errorEstimate=0.0419
x_8 = 0.7504177617637605, errorEstimate=0.0283
x_9 = 0.7314040424225098, errorEstimate=0.019
x_10 = 0.7442373549005569, errorEstimate=0.0128
x_11 = 0.7356047404363473, errorEstimate=0.00863
x_12 = 0.7414250866101093, errorEstimate=0.00582
x_13 = 0.7375068905132428, errorEstimate=0.00392
x_14 = 0.7401473355678757, errorEstimate=0.00264
x_15 = 0.7383692041223232, errorEstimate=0.00178
x_16 = 0.739567202212256, errorEstimate=0.0012
x_17 = 0.7387603198742114, errorEstimate=0.000807
x_18 = 0.7393038923969057, errorEstimate=0.000544
x_19 = 0.7389377567153446, errorEstimate=0.000366

root=0.7389377567153446, errorEstimate=0.00036613568156118603

```

```

def newtonMethod(f, Df, x0, errorTolerance=1e-15, maxIterations=20, demoMode=False):
    """Basic usage is:
    (rootApproximation, errorEstimate, iterations) = newton(f, Df, x0, errorTolerance)
    There is an optional input parameter "demoMode" which controls whether to
    - print intermediate results (for "study" purposes), or to
    - work silently (for "production" use).
    The default is silence.

    """
    x = x0
    for k in range(1, maxIterations+1):
        fx = f(x)
        Dfx = Df(x)
        # Note: a careful, robust code would check for the possibility of division by
        ↪ zero here,
        # but for now I just want a simple presentation of the basic mathematical
        ↪ idea.
        dx = fx/Dfx
        x -= dx # Aside: this is shorthand for "x = x - dx"
        errorEstimate = abs(dx)
        if demoMode:
            print(f"At iteration {k} x = {x} with estimated error {errorEstimate:0.3},
            ↪ backward error {abs(f(x)):0.3}")
            if errorEstimate <= errorTolerance:
                iterations = k
                return (x, errorEstimate, iterations)
        # If we get here, it did not achieve the accuracy target:
        iterations = k
    return (x, errorEstimate, iterations)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↪ importing [from] it
    def f(x): return x - np.cos(x)
    def Df(x): return 1. + np.sin(x)
    print("Solving by Newton's Method.")
    (root, errorEstimate, iterations) = newtonMethod(f, Df, x0=0.,
                                                    errorTolerance=1e-8, demoMode=True)

    print()
    print(f"The root is approximately {root}")
    print(f"The estimated absolute error is {errorEstimate}")
    print(f"The backward error is {abs(f(root)):0.3}")
    print(f"This required {iterations} iterations")

```

Solving by Newton's Method.

At iteration 1 x = 1.0 with estimated error 1.0, backward error 0.46

At iteration 2 x = 0.7503638678402439 with estimated error 0.25, backward error 0.0189

At iteration 3 x = 0.7391128909113617 with estimated error 0.0113, backward error 4.65e-05

At iteration 4 x = 0.739085133385284 with estimated error 2.78e-05, backward error 2.85e-10

At iteration 5 x = 0.7390851332151607 with estimated error 1.7e-10, backward error 0.0

(continues on next page)

(continued from previous page)

The root is approximately 0.7390851332151607
 The estimated absolute error is 1.7012340701403256e-10
 The backward error is 0.0
 This required 5 iterations

```
def falsePosition(f, a, b, errorTolerance=1e-15, maxIterations=15, demoMode=False):
    """Solve f(x)=0 in the interval [a, b] by the Method of False Position.
    This code also illustrates a few ideas that I encourage, such as:
    - Avoiding infinite loops, by using for loops sand break
    - Avoiding repeated evaluation of the same quantity
    - Use of descriptive variable names
    - Use of "camelCase" to turn descriptive phrases into valid Python variable names
    - An optional "demonstration mode" to display intermediate results.
    """
    fa = f(a)
    fb = f(b)
    for iteration in range(1, maxIterations+1):
        if demoMode: print(f"\nIteration {iteration}:")
        c = (a * fb - fa * b) / (fb - fa)
        fc = f(c)
        if fa * fc < 0:
            b = c
            fb = fc # N.B. When b is updated, so must be fb = f(b)
        else:
            a = c
            fa = fc
        errorBound = b - a
        if demoMode:
            print(f"The root is in interval [{a}, {b}]")
            print(f"The new approximation is {c}, with error bound {errorBound:0.3}, ↵
↵backward error {abs(fc):0.3}")
        if errorBound < errorTolerance:
            break
    # Whether we got here due to accuracy of running out of iterations,
    # return the information we have, including an error bound:
    root = c # the newest value is probably the most accurate
    return (root, errorBound)
```

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when ↵
↵importing [from] it
    def f(x): return x - np.cos(x)
    print("Solving by the Method of False Position.")
    (root, errorBound) = falsePosition(f, a=-1, b=1, demoMode=True)
    print(f"\nThe Method of False Position gave approximate root is {root},")
    print(f"with estimate error {errorBound:0.3}, backward error {abs(f(root)):0.3}")
```

Solving by the Method of False Position.

Iteration 1:
 The root is in interval [0.5403023058681398, 1]
 The new approximation is 0.5403023058681398, with error bound 0.46, backward error ↵
 ↵0.317

(continues on next page)

(continued from previous page)

```
Iteration 2:
The root is in interval [0.7280103614676171, 1]
The new approximation is 0.7280103614676171, with error bound 0.272, backward_
error 0.0185

Iteration 3:
The root is in interval [0.7385270062423998, 1]
The new approximation is 0.7385270062423998, with error bound 0.261, backward_
error 0.000934

Iteration 4:
The root is in interval [0.7390571666782676, 1]
The new approximation is 0.7390571666782676, with error bound 0.261, backward_
error 4.68e-05

Iteration 5:
The root is in interval [0.7390837322783136, 1]
The new approximation is 0.7390837322783136, with error bound 0.261, backward_
error 2.34e-06

Iteration 6:
The root is in interval [0.7390850630385933, 1]
The new approximation is 0.7390850630385933, with error bound 0.261, backward_
error 1.17e-07

Iteration 7:
The root is in interval [0.7390851296998365, 1]
The new approximation is 0.7390851296998365, with error bound 0.261, backward_
error 5.88e-09

Iteration 8:
The root is in interval [0.7390851330390691, 1]
The new approximation is 0.7390851330390691, with error bound 0.261, backward_
error 2.95e-10

Iteration 9:
The root is in interval [0.7390851332063397, 1]
The new approximation is 0.7390851332063397, with error bound 0.261, backward_
error 1.48e-11

Iteration 10:
The root is in interval [0.7390851332147188, 1]
The new approximation is 0.7390851332147188, with error bound 0.261, backward_
error 7.39e-13

Iteration 11:
The root is in interval [0.7390851332151385, 1]
The new approximation is 0.7390851332151385, with error bound 0.261, backward_
error 3.71e-14

Iteration 12:
The root is in interval [0.7390851332151596, 1]
The new approximation is 0.7390851332151596, with error bound 0.261, backward_
error 1.89e-15

Iteration 13:
```

(continues on next page)

(continued from previous page)

```

The root is in interval [0.7390851332151606, 1]
The new approximation is 0.7390851332151606, with error bound 0.261, backward_
error 1.11e-16

Iteration 14:
The root is in interval [0.7390851332151607, 1]
The new approximation is 0.7390851332151607, with error bound 0.261, backward_
error 0.0

Iteration 15:
The root is in interval [0.7390851332151607, 1]
The new approximation is 0.7390851332151607, with error bound 0.261, backward_
error 0.0

The Method of False Position gave approximate root is 0.7390851332151607,
with estimate error 0.261, backward error 0.0

```

```

def secantMethod(f, a, b, errorTolerance=1e-15, maxIterations=15, demoMode=False):
    """Solve f(x)=0 in the interval [a, b] by the Secant Method."""
    # Some more descriptive names
    x_older = a
    x_more_recent = b
    f_x_older = f(x_older)
    f_x_more_recent = f(x_more_recent)
    for iteration in range(1, maxIterations+1):
        if demoMode: print(f"\nIteration {iteration}:")
        x_new = (x_older * f_x_more_recent - f_x_older * x_more_recent) / (f_x_more_
recent - f_x_older)
        f_x_new = f(x_new)
        (x_older, x_more_recent) = (x_more_recent, x_new)
        (f_x_older, f_x_more_recent) = (f_x_more_recent, f_x_new)
        errorEstimate = abs(x_older - x_more_recent)
        if demoMode:
            print(f"The latest pair of approximations are {x_older} and {x_more_
recent},")
            print(f"where the function's values are {f_x_older:0.3} and {f_x_more_
recent:0.3} respectively.")
            print(f"The new approximation is {x_new}, with estimated error
{errorEstimate:0.3}, backward error {abs(f_x_new):0.3}")
            if errorEstimate < errorTolerance:
                break
        # Whether we got here due to accuracy of running out of iterations,
        # return the information we have, including an error estimate:
    return (x_new, errorEstimate)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when_
importing [from] it
    def f(x): return x - np.cos(x)
    print(f"Solving by the Secant Method.")
    (root, errorEstimate) = secantMethod(f, a=-1, b=1, demoMode=True)
    print(f"\nThe Secant Method gave approximate root is {root},")
    print(f"with estimated error {errorEstimate:0.3}, backward error {abs(f(root)):0.
3}")

```

Solving by the Secant Method.

Iteration 1:

The latest pair of approximations are 1 and 0.5403023058681398, where the function's values are 0.46 and -0.317 respectively.

The new approximation is 0.5403023058681398, with estimated error 0.46, backward error 0.317

Iteration 2:

The latest pair of approximations are 0.5403023058681398 and 0.7280103614676171, where the function's values are -0.317 and -0.0185 respectively.

The new approximation is 0.7280103614676171, with estimated error 0.188, backward error 0.0185

Iteration 3:

The latest pair of approximations are 0.7280103614676171 and 0.7396270126307336, where the function's values are -0.0185 and 0.000907 respectively.

The new approximation is 0.7396270126307336, with estimated error 0.0116, backward error 0.000907

Iteration 4:

The latest pair of approximations are 0.7396270126307336 and 0.7390838007832723, where the function's values are 0.000907 and -2.23e-06 respectively.

The new approximation is 0.7390838007832723, with estimated error 0.000543, backward error 2.23e-06

Iteration 5:

The latest pair of approximations are 0.7390838007832723 and 0.7390851330557806, where the function's values are -2.23e-06 and -2.67e-10 respectively.

The new approximation is 0.7390851330557806, with estimated error 1.33e-06, backward error 2.67e-10

Iteration 6:

The latest pair of approximations are 0.7390851330557806 and 0.7390851332151607, where the function's values are -2.67e-10 and 0.0 respectively.

The new approximation is 0.7390851332151607, with estimated error 1.59e-10, backward error 0.0

Iteration 7:

The latest pair of approximations are 0.7390851332151607 and 0.7390851332151607, where the function's values are 0.0 and 0.0 respectively.

The new approximation is 0.7390851332151607, with estimated error 0.0, backward error 0.0

The Secant Method gave approximate root is 0.7390851332151607, with estimated error 0.0, backward error 0.0

36.3 Linear Algebra

36.3.1 Basic row reduction elimination and backward substitution

```
def rowReduce(A, b):
    """To avoid modifying the matrix and vector specified as input,
    they are copied to new arrays, with the method .copy()
    Warning: it does not work to say "U = A" and "c = b";
    this makes these names synonyms, referring to the same stored data.
    Also: this does not modify the value of U below the main diagonal,
    since they are known to be zero, and will never be used;
    to clean this up, use function zeros_below_diagonal.
    """
    U = A.copy()
    c = b.copy()
    n = len(b)
    # The function zeros_like() is used to create L with the same size and shape as A,
    # and with all its elements zero initially.
    L = np.zeros_like(A)
    for k in range(n-1):
        for i in range(k+1, n):
            L[i,k] = U[i,k] / U[k,k]
            for j in range(k+1, n):
                U[i,j] = U[i,j] - L[i,k] * U[k,j]
            c[i] = c[i] - L[i,k] * c[k]
    return (U, c)
```

Immediately updated to the following, but I leave the first version for reference.

```
def rowReduce(A, b):
    """To avoid modifying the matrix and vector specified as input,
    they are copied to new arrays, with the method .copy()
    Warning: it does not work to say "U = A" and "c = b";
    this makes these names synonyms, referring to the same stored data.
    This code leaves "garbage values" below the main diagonal.
    """
    U = A.copy()
    c = b.copy()
    n = len(b)
    # The function zeros_like() is used to create L with the same size and shape as A,
    # and with all its elements zero initially.
    L = np.zeros_like(A)
    for k in range(n-1):
        # compute all the L values for column k:
        L[k+1:n,k] = U[k+1:n,k] / U[k,k] # Beware the case where U[k,k] is 0
        for i in range(k+1, n):
            U[i,k+1:n] -= L[i,k] * U[k,k+1:n] # Update row i
            c[k+1:n] -= L[k+1:n,k] * c[k] # update c values
    return (U, c)
```

```
def rowReduce(A, b, demoMode=False):
    """To avoid modifying the matrix and vector specified as input,
    they are copied to new arrays, with the method .copy()
    Warning: it does not work to say "U = A" and "c = b";
    this makes these names synonyms, referring to the same stored data.
```

(continues on next page)

(continued from previous page)

```

This code leaves "garbage values" below the main diagonal.
"""
U = A.copy()
c = b.copy()
n = len(b)
# The function zeros_like() is used to create L with the same size and shape as A,
# and with all its elements zero initially.
L = np.zeros_like(A)
for k in range(n-1):
    if demoMode: print(f"Step {k=}")
    # compute all the L values for column k:
    L[k+1:,k] = U[k+1:n,k] / U[k,k] # Beware the case where U[k,k] is 0
    if demoMode:
        print(f"The multipliers in column {k+1} are {L[k+1:,k]}")
    for i in range(k+1, n):
        U[i,k+1:n] -= L[i,k] * U[k,k+1:n] # Update row i
    c[k+1:n] -= L[k+1:n,k] * c[k] # update c values
    if demoMode:
        # insert zeros in U:
        U[k+1:, k] = 0.
        print(f"The updated matrix is\n{U}")
        print(f"The updated right-hand side is\n{c}")
return (U, c)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
#     A = np.array([[4. , 235. , 7.], [3. , 5. , -6.],[1. , -3. , 22.]])
    A = np.array([[1. , -3. , 15.], [5. , 200. , 7.], [4. , 11. , -6.]])
    print(f"A =\n{A}")
#     b = np.array([2. , 3. , 4.])
    b = np.array([4. , 3. , 2.])
    print(f"b = {b}")

    (U, c) = rowReduce(A, b, demoMode=True)
    #zeros_below_diagonal(U)
    print(f"U =\n{U}")
    print(f"c = {c}")

```

```

A =
[[ 1. -3. 15.]
 [ 5. 200.  7.]
 [ 4. 11. -6.]]
b = [4. 3. 2.]
Step k=0
The multipliers in column 1 are [5. 4.]
The updated matrix is
[[ 1. -3. 15.]
 [ 0. 215. -68.]
 [ 0. 23. -66.]]
The updated right-hand side is
[ 4. -17. -14.]
Step k=1
The multipliers in column 2 are [0.10697674]
The updated matrix is

```

(continues on next page)

(continued from previous page)

```

[[ 1.      -3.      15.      ]
 [ 0.     215.     -68.      ]
 [ 0.      0.     -58.7255814]]
The updated right-hand side is
[ 4.      -17.     -12.18139535]
U =
[[ 1.      -3.      15.      ]
 [ 0.     215.     -68.      ]
 [ 0.      0.     -58.7255814]]
c = [ 4.      -17.     -12.18139535]

```

```

def backwardSubstitution(U, c, demoMode=False):
    """Solve  $Ux = c$  for  $b$ .

    2021-03-07: added a demonstration mode.
    """
    n = len(c)
    x = np.zeros(n)
    x[-1] = c[-1]/U[-1,-1]
    if demoMode: print(f"x_{n} = {x[-1]}")
    for i in range(2, n+1):
        x[-i] = (c[-i] - sum(U[-i,1-i:] * x[1-i:])) / U[-i,-i]
        if demoMode: print(f"x_{n-i+1} = {x[-i]}")
    return x

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    x = backwardSubstitution(U, c, demoMode=True)
    print("")
    print(f"x = {x}")
    r = b - A@x
    print(f"The residual b - Ax = {r},")
    print(f"with maximum norm {max(abs(r)):.3}.")

```

```

x_3 = 0.20742911452558213
x_2 = -0.013464280057025185
x_1 = 0.8481704419451925

x = [ 0.84817044 -0.01346428  0.20742911]
The residual b - Ax = [ 0.0000000e+00 -8.8817842e-16 -4.4408921e-16],
with maximum norm 8.88e-16.

```

```

def solveLinearSystem(A, b):
    """Consolidate into a single function for solving  $Ax=b$ """
    return backwardSubstitution(*rowReduce(A, b));

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    x = solveLinearSystem(A, b)
    print("")
    print(f"x = {x}")

```

(continues on next page)

(continued from previous page)

```

r = b - A@x
print(f"The residual b - Ax = {r},")
print(f"with maximum norm {max(abs(r)).3}.")

```

```

x = [ 0.84817044 -0.01346428  0.20742911]
The residual b - Ax = [ 0.0000000e+00 -8.8817842e-16 -4.4408921e-16],
with maximum norm 8.88e-16.

```

36.3.2 LU factorization ...

```

def luFactorize(A, demoMode=False):
    """Compute the Doolittle LU factorization of A.
    Sums like  $\sum_{s=1}^{k-1} l_{k,s} u_{s,j}$  are done as matrix products;
    in the above case, row matrix  $L[k, 1:k-1]$  by column matrix  $U[1:k-1, j]$  gives the
    ↪sum for a give j,
    and row matrix  $L[k, 1:k-1]$  by matrix  $U[1:k-1, k:n]$  gives the relevant row vector.
    """
    n = len(A) # len() gives the number of rows in a 2D array.
    # Initialize U as the zero matrix;
    # correct below the main diagonal, with the other entries to be computed below.
    U = np.zeros_like(A)
    # Initialize L as the identity matrix;
    # correct on and above the main diagonal, with the other entries to be computed
    ↪below.
    L = np.identity(n)
    # Column and row 1 (i.e Python index 0) are special:
    U[0, :] = A[0, :]
    L[1:, 0] = A[1:, 0]/U[0, 0]
    if demoMode:
        print(f"After step k=0")
        print(f"U=\n{U}")
        print(f"L=\n{L}")
    for k in range(1, n-1):
        U[k, k:] = A[k, k:] - L[k, :k] @ U[:k, k:]
        L[k+1:, k] = (A[k+1:, k] - L[k+1:, :k] @ U[:k, k])/U[k, k]
        if demoMode:
            print(f"After step {k=}")
            print(f"U=\n{U}")
            print(f"L=\n{L}")
    # The last row (index "-1") is special: nothing to do for L
    U[-1, -1] = A[-1, -1] - sum(L[-1, :-1]*U[:-1, -1])
    if demoMode:
        print(f"After the final step, k={n-1}")
        print(f"U=\n{U}")
    return (L, U)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↪importing [from] it
    # A = np.array([[4. , 235. , 7.], [3. , 5. , -6.], [1. , -3. , 22.]])
    A = np.array([[1. , -3. , 15.], [5. , 200. , 7.], [4. , 11. , -6.]])
    print(f"A =\n{A}")

```

(continues on next page)

(continued from previous page)

```
# b = np.array([2. , 3. , 4.])
b = np.array([4. , 3. , 2.])
print(f"b = {b}")

(L, U) = luFactorize(A, demoMode=True)

print(f"A=\n{A}")
print(f"L=\n{L}")
print(f"U=\n{U}")
print(f"L times U is \n{L@U}")
print(f"The 'residual' A - LU is \n{A - L@U}")
```

36.3.3 ... and forward substitution ...

```
def forwardSubstitution(L, b, demoMode=False):
    """Solve  $Lc = b$  for  $c$ .
    """
    n = len(b)
    c = np.zeros(n)
    c[0] = b[0]
    if demoMode: print(f"c_1 = {c[0]}")
    for i in range(1, n):
        c[i] = b[i] - L[i,:i] @ c[:i]
        # Note: the above uses a row-by-column matrix multiplication to do the same as
        # c[i] = b[i] - sum(L[i,:i] * c[:i])
        if demoMode: print(f"c_{i+1} = {c[i]}")
    return c
```

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    c = forwardSubstitution(L, b, demoMode=True)
    print("")
    print(f"c = {c}")
```

36.3.4 ... and finally backward substitution again, to complete the LU factorization method example

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    x = backwardSubstitution(U, c, demoMode=True)
    print("")
    print(f"The residual  $c - Ux$  for the backward substitution step is {c - U@x}")
    print(f"\t with maximum norm {np.max(np.abs(c - U@x)):0.3}")
    print(f"The residual  $b - Ax$  for the whole solving process is {b - A@x}")
    print(f"\t with maximum norm {np.max(np.abs(b - A@x)):0.3}")
```

```

def rowReduceMaximalElementPivoting(A, b, demoMode=False):
    """Solve Ax=b by maximal element partial pivoting.
    This version actually swaps rows rather than keeping track of pivot row indices.
    """
    U = A.copy()
    c = b.copy()
    n = len(b)
    # The function zeros_like() is used to create L with the same size and shape as A,
    # and with all its elements zero initially.
    L = np.zeros_like(A)
    for k in range(n-1):
        if demoMode: print(f"Step {k=}")
        # Swap rows if necessary
        pivot_row = k
        size_U_pivot_row_k = abs(U[k,k])
        for i in range(k+1,n):
            size_U_i_k = abs(U[i,k])
            if size_U_i_k > size_U_pivot_row_k:
                pivot_row = i
                size_U_pivot_row_k = size_U_i_k
        # swap rows?
        if pivot_row > k:
            if demoMode:
                print(f"swap row {k} with row {pivot_row}")
            row_temp = U[pivot_row].copy()
            U[pivot_row] = U[k].copy()
            U[k] = row_temp.copy()
            #( U[k,:], U[pivot_row,:] ) = ( U[pivot_row,:], U[k,:] )
            ( c[k], c[pivot_row] ) = ( c[pivot_row], c[k] )
            if demoMode:
                print(f"After this row swap, the matrix is\n{U}")
                print(f"and the right-hand side is\n{c}")
        # compute all the L values for column k:
        L[k+1:,k] = U[k+1:n,k] / U[k,k]
        if demoMode:
            print(f"The multipliers in column {k+1} are {L[k+1:,k]}")
        for i in range(k+1, n):
            U[i,k+1:n] -= L[i,k] * U[k,k+1:n] # Update row i
        c[k+1:n] -= L[k+1:n,k] * c[k] # update c values
        if demoMode:
            # insert zeros in U:
            U[k+1:, k] = 0.
            print(f"The updated matrix is\n{U}")
            print(f"The updated right-hand side is\n{c}")
    return (U, c)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it
    A = np.array([[1. , -3. , 15.], [5. , 200. , 7.], [4. , 11. , -6.]])
    print(f"A =\n{A}")
    b = np.array([4. , 3. , 2.])
    print(f"b = {b}")

    (U, c) = rowReduceMaximalElementPivoting(A, b, demoMode=True)
    #zeros_below_diagonal(U)

```

(continues on next page)

(continued from previous page)

```
print(f"U =\n{U}")
print(f"c = {c}")
```

```
def inverse(A):
    """Use sparingly; there is usually a way to avoid computing inverses that is
    ↪faster and with less rounding error!"""
    n = len(A)
    A_inverse = np.zeros_like(A)
    (L, U) = luFactorize(A)
    for i in range(n):
        b = np.zeros(n)
        b[i] = 1.0
        c = forwardSubstitution(L, b)
        A_inverse[:,i] = backwardSubstitution(U, c)
    return A_inverse
```

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↪importing [from] it

    def hilbert(n):
        H = np.zeros([n,n])
        for i in range(n):
            for j in range(n):
                H[i,j] = 1.0/(1.0 + i + j)
        return H

    for n in range(2,4):
        H_n = hilbert(n)
        print(f"The Hilbert matrix H_{n} is")
        print(H_n)
        H_n_inverse = inverse(H_n)
        print("and its inverse is")
        print(H_n_inverse)
        print("to verify, their product is")
        print(H_n @ H_n_inverse)
        print()
```

36.4 Polynomial Collocation and Approximation

```
def fitPolynomial(x, y):
    """Compute the coefficients c_i of the polynomial of lowest degree that
    ↪collocates the points (x[i], y[i]).
    These are returned in an array c of the same length as x and y, even if the
    ↪degree is less than the normal length(x)-1,
    in which case the array has some trailing zeroes.
    The polynomial is thus p(x) = c[0] + c[1]x + ... c[d] x^d where n=length(x)-1,
    ↪the nominal degree.
    """
    nnodes = len(x)
    n = nnodes - 1
```

(continues on next page)

(continued from previous page)

```

V = np.zeros([nnodes, nnodes])
for i in range(nnodes):
    for j in range(nnodes):
        V[i,j] = x[i]**j
(U, z) = rowReduce(V, y)
c = backwardSubstitution(U, z)
return c

```

```

def evaluatePolynomial(x, coeffs):
    # Evaluate the polynomial with coefficients in coeffs at the points in x.
    npoints = len(x)
    ncoeffs = len(coeffs)
    n = ncoeffs - 1
    powers = np.linspace(0, n, n+1)
    y = np.zeros_like(x)
    for i in range(npoints):
        y[i] = sum(coeffs * x[i]**powers)
    return y

```

```

def showPolynomial(c):
    print("P(x) = ", end="")
    n = len(c)-1
    print(f"{c[0]:.4}", end="")
    if n > 0:
        coeff = c[1]
        if coeff > 0:
            print(f" + {coeff:.4}x", end="")
        elif coeff < 0:
            print(f" - {-coeff:.4}x", end="")
    if n > 1:
        for j in range(2, len(c)):
            coeff = c[j]
            if coeff > 0:
                print(f" + {coeff:.4}x^{j}", end="")
            elif coeff < 0:
                print(f" - {-coeff:.4}x^{j}", end="")
    print()

```

36.5 Solving Initial Value Problems for Ordinary Differential Equations

36.5.1 Basic implementations of some basic methods

(More to come.)

```

def eulerMethod(f, a, b, u_0, n=100):
    """Use Euler's Method to solve du/dt = f(t, u) for t in [a, b], with initial_
↪value u(a) = u_0"""
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for_
↪t.

```

(continues on next page)

(continued from previous page)

```

# Only the following two lines will need to change for the systems version
U = np.empty_like(t)
U[0] = u_0

for i in range(n):
    U[i+1] = U[i] + f(t[i], U[i])*h
return (t, U)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it.

    def f_1(t, u):
        """The simplest "genuine" ODE, (not just integration)
        The solution is  $u(t) = u(t; a, u_0) = u_0 \exp(t-a)$ 
        """
        return K*u
    def u_1(t): return u_0 * np.exp(K*(t-a))

    a = 1.
    b = 3.
    u_0 = 2.
    K = 0.5
    n = 10

    (t, U) = eulerMethod(f_1, a, b, u_0, n)
    h = (b-a)/n
    plt.figure(figsize=[12,6])
    plt.title(f"Solving  $du/dt = Ku$ , {a=} , {u_0=} – by Euler's method with {n} steps")
    plt.plot(t, u_1(t), 'g', label="Exact solution")
    plt.plot(t, U, 'b:', label=f"Euler's answer with h={h:0.3}")
    plt.legend()
    plt.grid(True)

    plt.figure(figsize=[12,4])
    plt.title(f"Error")
    plt.plot(t, U - u_1(t))
    plt.grid(True)

```

```

def eulerSystem(f, a, b, u_0, n=100):
    """Use Euler's Method to solve  $du/dt = f(t, u)$  for  $t$  in  $[a, b]$ , with initial
    value  $u(a) = u_0$ 
    Modified from function euler to handle systems."""
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for
    t.

    # Only the following three lines change for the systems version
    n_unknowns = len(u_0)
    U = np.zeros([n+1, n_unknowns])
    U[0] = np.array(u_0)

    for i in range(n):
        U[i+1] = U[i] + f(t[i], U[i])*h

```

(continues on next page)

(continued from previous page)

```
return (t, U)
```

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↪ importing [from] it.

    def f(t, u):
        return np.array([ u[1], -(k/M)*u[0] - (D/M)*u[1]])

    M = 1.0
    k = 1.0
    D = 0.1
    u_0 = [1.0, 0.0]
    a = 0.0
    b = 8 * np.pi # Four periods

    n=10000
    (t, U) = eulerSystem(f, a, b, u_0, n)
    y = U[:,0]
    Dy = U[:,1]

    plt.figure(figsize=[12,6])
    plt.title(f"U_0, = y with {k=}, {D=} - by Euler with {n} steps")
    plt.plot(t, y)
    plt.xlabel('t')
    plt.ylabel('y')
    plt.grid(True)

    plt.figure(figsize=[12,6])
    plt.title(f"U_1, = dy/dt with {k=}, {D=} - by Euler with {n} steps")
    plt.plot(t, Dy)
    plt.xlabel('t')
    plt.ylabel('dy/dt')
    plt.grid(True)

    plt.figure(figsize=[12,6])
    plt.title(f"U_0=y and U_1=dy/dt with {k=}, {D=} - by Euler with {n} steps")
    plt.plot(t, U)
    plt.xlabel('t')
    plt.ylabel('y and dy/dt')
    plt.grid(True)

    plt.figure(figsize=[8,8]) # Make axes equal length; orbits should be circular or
    ↪ "circular spirals"
    if D == 0.:
        plt.title(f"The orbits of the undamped mass-spring system, {k=} - by Euler
        ↪ with {n} steps")
    else:
        plt.title(f"The orbits of the damped mass-spring system, k={k}, D={D} - by
        ↪ Euler with {n} steps")
    plt.plot(y, Dy)
    plt.xlabel('y')
    plt.ylabel('dy/dt')
    plt.plot(y[0], Dy[0], "g*", label="start")
    plt.plot(y[-1], Dy[-1], "r*", label="end")
```

(continues on next page)

(continued from previous page)

```
plt.legend()
plt.grid(True)
```

```
def explicitTrapezoid(f, a, b, u_0, n=100):
    """Use the Explicit Trapezoid Method (a.k.a Improved Euler)
    to solve  $du/dt = f(t, u)$  for  $t$  in  $[a, b]$ , with initial value  $u(a) = u_0$ """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for
    ↪ t.

    # Only the following two lines will need to change for the systems version
    U = np.empty_like(t)
    U[0] = u_0

    for i in range(n):
        K_1 = f(t[i], U[i])*h
        K_2 = f(t[i]+h, U[i]+K_1)*h
        U[i+1] = U[i] + (K_1 + K_2)/2.
    return (t, U)
```

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↪ importing [from] it.

    a = 1.
    b = 3.
    u_0 = 2.
    K = 1.
    n = 10

    (t, U) = explicitTrapezoid(f_1, a, b, u_0, n)
    h = (b-a)/n
    plt.figure(figsize=[12,6])
    plt.title(f"Solving  $du/dt = Ku$ , {a=}, {u_0=} – by the Explicit Trapezoid Method
    ↪ with {n} steps")
    plt.plot(t, u_1(t), 'g', label="Exact solution")
    plt.plot(t, U, 'b:', label=f"Solution with h={h:0.3}")
    plt.legend()
    plt.grid(True)

    plt.figure(figsize=[12,4])
    plt.title(f"Error")
    plt.plot(t, U - u_1(t))
    plt.grid(True)
```

```
# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↪ importing [from] it.

    def f_2(t, u):
        """A simple more "generic" test case, with  $f(t, u)$  depending on both
        ↪ variables.
        The solution for  $a=0$  is  $u(t) = u(t; 0, u_0) = \cos t + (u_0 - 1) e^{(-Kt)}$ 
        The solution in general is  $u(t) = u(t; a, u_0) = \cos t + C e^{(-K t)}$ ,  $C = (u_0 -$ 
        ↪  $\cos(a)) \cdot \exp(K a)$ 
```

(continues on next page)

(continued from previous page)

```

    """
    return K*(np.cos(t) - u) - np.sin(t)
def u_2(t): return np.cos(t) + C * np.exp(-K*t)

a = 1.
b = a + 4 * np.pi # Two periods
u_0 = 2.
K = 2.
n = 50

(t, U) = explicitTrapezoid(f_2, a, b, u_0, n)
C = (u_0 - np.cos(a)) * np.exp(K*a)
h = (b-a)/n
plt.figure(figsize=[12,6])
plt.title(f"Solving du/dt = K(cos(t) - u) - sin(t), {K=}, {a=}, {u_0=} - by the_
↳Explicit Trapezoid Method with {n} steps")
plt.plot(t, u_2(t), 'g', label="Exact solution")
plt.plot(t, U, 'b:', label=f"Solution with h={h:0.3}")
plt.legend()
plt.grid(True)

plt.figure(figsize=[12,4])
plt.title(f"Error")
plt.plot(t, U - u_2(t))
plt.grid(True)

```

```

def explicitMidpoint(f, a, b, u_0, n=100):
    """Use the Explicit Midpoint Method (a.k.a Modified Euler)
    to solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0"""
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for_
    ↳t.

    # Only the following two lines will need to change for the systems version
    U = np.empty_like(t)
    U[0] = u_0

    for i in range(n):
        K_1 = f(t[i], U[i])*h
        K_2 = f(t[i]+h/2, U[i]+K_1/2)*h
        U[i+1] = U[i] + K_2
    return (t, U)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when_
    ↳importing [from] it.

    a = 1.
    b = 3.
    u_0 = 2.
    K = 1.
    n = 10

    (t, U) = explicitMidpoint(f_1, a, b, u_0, n)
    h = (b-a)/n

```

(continues on next page)

(continued from previous page)

```

plt.figure(figsize=[12,6])
plt.title(f"Solving du/dt = Ku, {a=}, {u_0=} - by the Explicit Midpoint Method_
↳with {n} steps")
plt.plot(t, u_1(t), 'g', label="Exact solution")
plt.plot(t, U, 'b:', label=f"Solution with h={h:0.3}")
plt.legend()
plt.grid(True)

plt.figure(figsize=[12,4])
plt.title(f"Error")
plt.plot(t, U - u_1(t))
plt.grid(True)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when_
↳importing [from] it.

    def f_2(t, u):
        """A simple more "generic" test case, with f(t, u) depending on both_
↳variables.
        The solution for a=0 is u(t) = u(t; 0, u_0) = cos t + (u_0 - 1) e^(-Kt)
        The solution in general is u(t) = u(t; a, u_0) = cos t + C e^(-K t), C = (u_0_
↳- cos(a)) exp(K a)
        """
        return K*(np.cos(t) - u) - np.sin(t)
    def u_2(t): return np.cos(t) + C * np.exp(-K*t)

    a = 1.
    b = a + 4 * np.pi # Two periods
    u_0 = 2.
    K = 2.
    n = 50

    (t, U) = explicitMidpoint(f_2, a, b, u_0, n)
    C = (u_0 - np.cos(a)) * np.exp(K*a)
    h = (b-a)/n
    plt.figure(figsize=[12,6])
    plt.title(f"Solving du/dt = K(cos(t) - u) - sin(t), {K=}, {a=}, {u_0=} - by the_
↳Explicit Midpoint Method with {n} steps")
    plt.plot(t, u_2(t), 'g', label="Exact solution")
    plt.plot(t, U, 'b:', label=f"Solution with h={h:0.3}")
    plt.legend()
    plt.grid(True)

    plt.figure(figsize=[12,4])
    plt.title(f"Error")
    plt.plot(t, U - u_2(t))
    plt.grid(True)

```

```

def rungeKutta(f, a, b, u_0, n=100):
    """Use the (classical) Runge-Kutta Method
    to solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0"""
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for_
↳t.

```

(continues on next page)

(continued from previous page)

```

# Only the following two lines will need to change for the systems version
U = np.empty_like(t)
U[0] = u_0

for i in range(n):
    K_1 = f(t[i], U[i])*h
    K_2 = f(t[i]+h/2, U[i]+K_1/2)*h
    K_3 = f(t[i]+h/2, U[i]+K_2/2)*h
    K_4 = f(t[i]+h, U[i]+K_3)*h
    U[i+1] = U[i] + (K_1 + 2*K_2 + 2*K_3 + K_4)/6
return (t, U)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it.

    a = 1.
    b = 3.
    u_0 = 2.
    K = 1.
    n = 10

    (t, U) = rungeKutta(f_1, a, b, u_0, n)
    h = (b-a)/n
    plt.figure(figsize=[12,6])
    plt.title(f"Solving du/dt = Ku, {a=}, {u_0=} - by the Runge-Kutta Method with {n}
    steps")
    plt.plot(t, u_1(t), 'g', label="Exact solution")
    plt.plot(t, U, 'b:', label=f"Solution with h={h:0.3}")
    plt.legend()
    plt.grid(True)

    plt.figure(figsize=[12,4])
    plt.title(f"Error")
    plt.plot(t, U - u_1(t))
    plt.grid(True)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it.

    def f_2(t, u):
        """A simple more "generic" test case, with f(t, u) depending on both
        variables.
        The solution for a=0 is u(t) = u(t; 0, u_0) = cos t + (u_0 - 1) e^(-Kt)
        The solution in general is u(t) = u(t; a, u_0) = cos t + C e^(-K t), C = (u_0
        cos(a)) exp(K a)
        """
        return K*(np.cos(t) - u) - np.sin(t)
    def u_2(t): return np.cos(t) + C * np.exp(-K*t)

    a = 1.
    b = a + 4 * np.pi # Two periods
    u_0 = 2.

```

(continues on next page)

(continued from previous page)

```

K = 2.
n = 50

(t, U) = rungeKutta(f_2, a, b, u_0, n)
C = (u_0 - np.cos(a)) * np.exp(K*a)
h = (b-a)/n
plt.figure(figsize=[12,6])
plt.title(f"Solving du/dt = K(cos(t) - u) - sin(t), {K=}, {a=}, {u_0=} - by the
↳Runge-Kutta Method with {n} steps")
plt.plot(t, u_2(t), 'g', label="Exact solution")
plt.plot(t, U, 'b:', label=f"Solution with h={h:0.3}")
plt.legend()
plt.grid(True)

plt.figure(figsize=[12,4])
plt.title(f"Error")
plt.plot(t, U - u_2(t))
plt.grid(True)

```

```

def rungeKuttaSystem(f, a, b, u_0, n=100):
    """Use the (classical) Runge-Kutta Method
    to solve du/dt = f(t, u) for t in [a, b], with initial value u(a) = u_0"""
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for
    ↳t.

    # Only the following three lines change for the systems version - the same lines
    ↳as for eulerSystem and so on.
    n_unknowns = len(u_0)
    U = np.zeros([n+1, n_unknowns])
    U[0] = np.array(u_0)

    for i in range(n):
        K_1 = f(t[i], U[i])*h
        K_2 = f(t[i]+h/2, U[i]+K_1/2)*h
        K_3 = f(t[i]+h/2, U[i]+K_2/2)*h
        K_4 = f(t[i]+h, U[i]+K_3)*h
        U[i+1] = U[i] + (K_1 + 2*K_2 + 2*K_3 + K_4)/6
    return (t, U)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    ↳importing [from] it.

    def fMassSpring(t, u):
        return np.array([ u[1], -(k/M)*u[0] - (D/M)*u[1]])
    M = 1.0
    k = 1.0
    D = 0.1
    u_0 = [1.0, 0.0]
    a = 0.0
    b = 8 * np.pi # Four periods
    n = 100 # Should be enough now!

    (t, U) = rungeKuttaSystem(fMassSpring, a, b, u_0, n)

```

(continues on next page)

(continued from previous page)

```

y = U[:,0]
Dy = U[:,1]

plt.figure(figsize=[12,6])
plt.title(f"U_0, = y with {k=}, {D=} – by Runge-Kutta with {n} steps")
plt.plot(t, y)
plt.xlabel('t')
plt.ylabel('y')
plt.grid(True)

plt.figure(figsize=[12,6])
plt.title(f"U_1, = dy/dt with {k=}, {D=} – by Runge-Kutta with {n} steps")
plt.plot(t, Dy)
plt.xlabel('t')
plt.ylabel('dy/dt')
plt.grid(True)

plt.figure(figsize=[8,8]) # Make axes equal length; orbits should be circular or
↪ "circular spirals"
    if D == 0.:
        plt.title(f"The orbits of the undamped mass-spring system, {k=} – by Runge-
↪ Kutta with {n} steps")
    else:
        plt.title(f"The orbits of the damped mass-spring system, k={k}, D={D} – by
↪ Runge-Kutta with {n} steps")
    plt.plot(y, Dy)
    plt.xlabel('y')
    plt.ylabel('dy/dt')
    plt.plot(y[0], Dy[0], "g*", label="start")
    plt.plot(y[-1], Dy[-1], "r*", label="end")
    plt.legend()
    plt.grid(True)

```

36.5.2 For the future: an attempt to use the IMPLICIT Midpoint Method

Solving for now with a few fixed point iterations.

```

def FPMidpointSystem(f, a, b, u_0, n=100, iterations=1):
    """Use a few iterations of a fixed point method solution of the Implicit Midpoint
    ↪ Method
    to solve  $du/dt = f(t, u)$  for  $t$  in  $[a, b]$ , with initial value  $u(a) = u_0$ 
    Note:
        - the default of iterations=1 gives the Explicit Midpoint Method
        - iterations=0 gives Euler's Method
    """
    h = (b-a)/n
    t = np.linspace(a, b, n+1) # Note: "n" counts steps, so there are n+1 values for
    ↪ t.

    # Only the following three lines change for the systems version – the same lines
    ↪ as for eulerSystem and so on.
    n_unknowns = len(u_0)
    U = np.zeros([n+1, n_unknowns])
    U[0] = np.array(u_0)

```

(continues on next page)

(continued from previous page)

```

for i in range(n):
    K = f(t[i], U[i])*h
    # A few iterations of the fixed point method
    for iteration in range(iterations):
        K = f(t[i]+h/2, U[i]+K/2)*h
    U[i+1] = U[i] + K
return (t, U)

```

```

# Demo
if __name__ == "__main__": # Do this if running the .py file directly, but not when
    importing [from] it.

    def fMassSpring(t, u):
        return np.array([ u[1], -(k/M)*u[0] - (D/M)*u[1]])
    M = 1.0
    k = 1.0
    D = 0.0
    u_0 = [1.0, 0.0]
    a = 0.0
    b = 8 * np.pi # Four periods
    n = 200
    iterations = 2

    (t, U) = FPMidpointSystem(fMassSpring, a, b, u_0, n, iterations=1)
    y = U[:,0]
    Dy = U[:,1]

    plt.figure(figsize=[12,6])
    plt.title(f"U_0, = y with {k=}, {D=} - by FPMidpointSystem with {n} steps,
    iterations")
    plt.plot(t, y)
    plt.xlabel('t')
    plt.ylabel('y')
    plt.grid(True)

    plt.figure(figsize=[8,8]) # Make axes equal length; orbits should be circular or
    "circular spirals"
    if D == 0.:
        plt.title(f"The orbits of the undamped mass-spring system, {k=} - by
        FPMidpointSystem with {n} steps, {iterations} iterations")
    else:
        plt.title(f"The orbits of the damped mass-spring system, k={k}, D={D} - by
        FPMidpointSystem with {n} steps, {iterations} iterations")
    plt.plot(y, Dy)
    plt.xlabel('y')
    plt.ylabel('dy/dt')
    plt.plot(y[0], Dy[0], "g*", label="start")
    plt.plot(y[-1], Dy[-1], "r*", label="end")
    plt.legend()
    plt.grid(True)

```

36.6 For some examples in Chapter Initial Value Problems for Ordinary Differential Equations

```
def fMassSpring(t, u): return np.array([ u[1], -(K/M)*u[0] - (D/M)*u[1] ])

def yMassSpring(t, t_0, u_0, K, M, D):
    (y_0, v_0) = u_0
    discriminant = D**2 - 4*K*M
    if discriminant < 0: # underdamped
        omega = np.sqrt(4*K*M - D**2)/(2*M)
        A = y_0
        B = (v_0 + y_0*D/(2*M))/omega
        return np.exp(-D/(2*M)*(t-t_0)) * ( A*np.cos(omega*(t-t_0)) + B*np.
        sin(omega*(t-t_0)))
    elif discriminant > 0: # overdamped
        Delta = np.sqrt(discriminant)
        lambda_plus = (-D + Delta)/(2*M)
        lambda_minus = (-D - Delta)/(2*M)
        A = M*(v_0 - lambda_minus * y_0)/Delta
        B = y_0 - A
        return A*np.exp(lambda_plus*(t-t_0)) + B*np.exp(lambda_minus*(t-t_0))
    else:
        q = -D/(2*M)
        A = y_0
        B = v_0 - A * q
        return (A + B*t)*np.exp(q*(t-t_0))

def damping(K, M, D):
    if D == 0:
        print("Undamped")
    else:
        discriminant = D**2 - 4*K*M
        if discriminant < 0:
            print("Underdamped")
        elif discriminant > 0:
            print("Overdamped")
        else:
            print("Critically damped")
```

LINEAR ALGEBRA ALGORITHMS USING 0-BASED INDEXING AND SEMI-OPEN INTERVALS

This section describes some of the core algorithms of linear algebra using the same indexing conventions as in most modern programming languages: Python, C, Java, C++, javascript, Objective-C, C#, Swift, etc. (In fact, almost everything except Matlab and Fortran.)

The key elements of this are:

- Indices for vectors and other arrays **start at 0**.
- Ranges of indices are described with *semi-open intervals* $[a, b)$.

This “index interval” notation has two virtues: it emphasizes the mathematical fact that the order in which things are done is irrelevant (such as within sums), and it more closely resembles the way that most programming languages specify index ranges. For example, the indices i of a Python array with n elements are $0 \leq i < n$, or $[0, n)$, and the Python notations `range(n)`, `range(0, n)`, `:n` and `0:n` all describe this. Similarly, in Java, C, C++ etc., one can loop over the indices $i \in [a, b)$ with `for (i=a, i<b, i+=1)`

The one place that the indexing is still a bit tricky is counting backwards!

For this, note that the index range $i = b, b-1, \dots, a$ is $b \geq i > a-1$, which in Python is `range(b, a-1, -1)`.

I include Python code for comparison for just the three most basic algorithms: “naive” LU factorization and forward and backward substitution, without pivoting. The rest are good exercises for learning how to program loops and sums.

37.1 The naive Gaussian elimination algorithm

In this careful version, the original matrix A is called $A^{(0)}$, and the new versions at each stage are called $A^{(1)}$, $A^{(2)}$, and so on to $A^{(n-1)}$, which is the row-reduced form also called U ; likewise with the right-hand sides $b^{(0)} = b$, $b^{(1)}$ up to $b^{(n-1)} = c$.

However, in software all those super-scripts can be ignored, just updating arrays A and b .

Algorithm 2.1

```
for k in [0, n-1)
    for i in [k+1, n)
         $l_{i,k} = a_{i,k}^{(k)} / a_{k,k}^{(k)}$ 
        for j in [k+1, n)
             $a_{i,j}^{(k+1)} = a_{i,j}^{(k)} - l_{i,k} a_{k,j}^{(k)}$ 
        end
```

```

         $b_i^{(k+1)} = b_i^{(k)} - l_{i,k} b_k^{(k)}$ 
    end
end for

```

Actually this skips formulas for some elements of the new matrix $A^{(k+1)}$, because they are either zero or are unchanged from $A^{(k)}$:

the rows before $i = k$ are unchanged, and for columns before $j = k$, the new entries are zeros.

37.2 The LU factorization with L unit lower triangular, by Doolittle's direct method

Algorithm 2.2

```

for j in [0, n)
     $u_{0,j} = a_{0,j}$ 
end
for i in [1, n)
     $l_{i,0} = a_{i,0}/u_{0,0}$ 
end
for k in [1, n)
    for j in [k, n)
         $u_{k,j} = a_{k,j} - \sum_{s \in [0,k)} l_{k,s} u_{s,j}$ 
    end
    for i in [k+1, n)
         $l_{i,k} = \left( a_{i,k} - \sum_{s \in [0,k)} l_{i,s} u_{s,k} \right) / u_{k,k}$ 
    end
end
end

```

37.3 Forward substitution with a unit lower triangular matrix

For L unit lower triangular, solving $Lc = b$ by forward substitution is

Algorithm 2.3

```

 $c_0 = b_0$ 
for i in [1, n)

```


$$c_i = b_i - \sum_{j \in [0, i)} l_{i,j} c_j$$

end for

37.4 Backward substitution with an upper triangular matrix

Algorithm 2.4

```

 $x_{n-1} = c_{n-1} / u_{n-1, n-1}$ 
for i from n-2 down to 0
    
$$x_i = \frac{c_i - \sum_{j \in [i+1, n)} u_{i,j} x_j}{u_{i,i}}$$

end

```

37.4.1 Counting backwards in Python

For the Python implementation, we need a range of indices that change by a step of -1 instead of 1. This can be done with an optional third argument to range: `range(a, b, step)` generates a succession values starting with a, each value differing from its predecessor by step, and stopping just before b. That last rule requires care when the step is negative: for example, `range(3, 0, -1)` gives the sequence $\{3, 2, 1\}$. So to count down to zero, one has to use $b = -1$! That is, to count down from $n-1$ and end at 0, one uses `range(n-1, -1, -1)`.

```
import numpy as np
```

```

def solveUpperTriangular(U, c):
    n = len(c)
    x = np.zeros(n)
    x[n-1] = c[n-1]/U[n-1,n-1]
    for i in range(n-2, -1, -1):
        x[i] = (c[i] - sum(U[i,i+1:n] * x[i+1:n]))/U[i,i]
    return x

```

37.4.2 Another way to count downwards

A range of integers i from b down to a is also $i = b - d$ for d from 0 up to $b - a$,

so one can use `for d in [0, b-a)`

This gives the alternative for the algorithm:

Algorithm 2.5

```

 $x_{n-1} = c_{n-1} / u_{n-1, n-1}$ 
for d in [2, n+1):
     $i = n - d$ 

```

$$x_i = \frac{c_i - \sum_{j \in [i+1, n)} u_{i,j} x_j}{u_{i,i}}$$

end

This corresponds to the alternative Python code:

```
def solveUpperTriangular(U, c):
    n = len(c)
    x = np.zeros(n)
    x[n-1] = c[n-1]/U[n-1,n-1]
    for nmi in range(2, n+1): # nmi is n-i
        i = n - nmi
        x[i] = (c[i] - sum(U[i,i+1:n] * x[i+1:n]))/U[i,i]
    return x
```

37.5 Versions with maximal element partial pivoting

Apart the choices of pivot rows and updating of the permutation vector p , the only change from the non-pivoting version is that all *row* indices change from i to p_i and so on, in both U and c ; *column* indices are unchanged.

37.5.1 Gaussian elimination with maximal element partial pivoting

In the following description, I will discard the above distinction between the successive matrices $A^{(k)}$ and vectors $b^{(k)}$, and instead refer to A and b like variable arrays in a programming language, with their elements being updated. Likewise, the permutation will be stored in a variable array p .

Algorithm 2.6

Initialize the permutation vector as $p = [0, 1, \dots, n-1]$

for k in $[0, n-1)$

 Search elements $a_{p_i,k}$ for $i \in [k, n)$ and find the index r of the one with largest absolute value.

 If $r \neq k$, swap p_k with p_r

 for i in $[k+1, n)$

$$l_{p_i,k} = a_{p_i,k}/a_{p_k,k}$$

 for j in $[k+1, n)$

$$a_{p_i,j} = a_{p_i,j} - l_{p_i,k} a_{p_k,j}$$

 end

$$b_{p_i} = b_{p_i} - l_{p_i,k} b_{p_k}$$

 end

end

37.5.2 The Doolittle LU factorization algorithm with maximal element partial pivoting

Algorithm 2.7

```

for k in [0, n)
    Search elements  $a_{p_i,k}$  for  $i \in [k, n)$  and find the index r of the one with largest absolute value.
    If  $r \neq k$ , swap  $p_k$  with  $p_r$ 
    for j in [k, n)
        Note that for  $k = 0$ , the sum can [and should!] be omitted in the following line:

$$u_{p_k,j} = a_{p_k,j} - \sum_{s \in [0,k)} l_{p_k,s} u_{p_s,j}$$

        end
    for i in [k+1, n)
        Note that for  $k = 0$ , the sum can [and should!] be omitted in the following line:

$$l_{p_i,k} = \left( a_{p_i,k} - \sum_{s \in [0,k)} l_{p_i,s} u_{p_s,k} \right) / u_{p_k,k}$$

        end
    end
end

```

37.5.3 Forward substitution with maximal element partial pivoting

Algorithm 2.8

```

 $c_{p_0} = b_{p_0} / l_{p_0,0}$ 
for i in [1, n)

$$c_{p_i} = b_{p_i} - \sum_{j \in [0,i)} l_{p_i,j} c_{p_j}$$

end

```

37.5.4 Backward substitution with maximal element partial pivoting

Algorithm 2.9

```

 $x_{p_{n-1}} = c_{p_{n-1}} / u_{p_{n-1},n-1}$ 
for i from n-2 down to 0

$$x_i = \frac{c_{p_i} - \sum_{j \in [i+1,n)} u_{p_i,j} x_j}{u_{p_i,i}}$$

end

```

end

37.6 Tridiagonal matrix algorithms

Describe a tridiagonal matrix with three 1D arrays as $T = \begin{bmatrix} d_0 & u_0 & & & \\ l_0 & d_1 & u_1 & & \\ & l_1 & d_2 & u_2 & \\ & & \ddots & \ddots & \ddots \\ & & & l_{n-3} & d_{n-2} & u_{n-2} \\ & & & & l_{n-2} & d_{n-1} \end{bmatrix}$

with all “missing” entries being zeros, and the right hand side of system $Tx = b$ as

$$\begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_{n-1} \end{bmatrix}$$

37.6.1 Doolittle factorization of a tridiagonal matrix

The factorization has the form $T = LU$ with $L = \begin{bmatrix} 1 & & & & \\ L_0 & 1 & & & \\ & L_1 & 1 & & \\ & & \ddots & \ddots & \\ & & & L_{n-3} & 1 \\ & & & & L_{n-2} & 1 \end{bmatrix}$, $U =$

$$\begin{bmatrix} D_0 & u_0 & & & \\ & D_1 & u_1 & & \\ & & D_2 & u_2 & \\ & & & \ddots & \ddots \\ & & & & D_{n-2} & u_{n-2} \\ & & & & & D_{n-1} \end{bmatrix}$$

so just the arrays L and D are to be computed.

Algorithm 2.10

```

 $D_0 = d_0$ 
for i in [1, n)
     $L_{i-1} = l_{i-1}/D_{i-1}$ 
     $D_i = d_i - L_{i-1}u_{i-1}$ 
end

```

37.6.2 Forward substitution with a tridiagonal matrix

Algorithm 2.11

```
 $c_0 = b_0$   
for i in [1, n)  
     $c_i = b_i - L_{i-1}c_{i-1}$   
end
```

37.6.3 Backward substitution with a tridiagonal matrix

Algorithm 2.12

```
 $x_{n-1} = c_{n-1}/D_{n-1}$   
for i from n-2 down to 0  
     $x_i = (c_i - u_i x_{i+1})/D_i$   
end
```

37.7 Banded matrix algorithms

37.7.1 Doolittle factorization of a matrix of bandwidth p

That is, $A_{i,j} = 0$ when $|i - j| > p$.

In addition to loops stopping at the point beyond which value would be zero or unchanged the top row and top-right diagonal overlapping at the element of “1-based” indices $(1, p + 1)$, which is now at 0-based indices $(0, p)$.

Algorithm 2.13

The top row is unchanged:

```
for j in [0, p+1)
```

$$u_{0,j} = a_{0,j}$$

end *The top non-zero diagonal is also unchanged:*

```
for k in [1, n - p)
```

$$u_{k,k+p} = a_{k,k+p}$$

```
end for
```

The left column requires no sums:

```
for i in [1, p+1)
```

$$l_{i,0} = a_{i,0}/u_{0,0}$$

```
end for
```

The main loop:

```

for k in [1, n)
  for j in [k, min(n, k+p))
    
$$u_{k,j} = a_{k,j} - \sum_{s \in [\max(0, j-p), k)} l_{k,s} u_{s,j}$$

  end
  for i in [k+1, min(n, k+p+1))
    
$$l_{i,k} = \left( a_{i,k} - \sum_{s \in [\max(0, i-p), k)} l_{i,s} u_{s,k} \right) / u_{k,k}$$

  end
end
end

```

37.7.2 Forward substitution with a unit lower-triangular matrix of bandwidth p

Algorithm 2.14

```


$$c_0 = b_0 / l_{0,0}$$

for i in [1, n)
  
$$c_i = b_i - \sum_{j \in [\max(0, i-p), i)} l_{i,j} c_j$$

end

```

37.7.3 Backward substitution with an upper-triangular matrix of bandwidth p

Algorithm 2.15

```


$$x_{n-1} = c_{n-1} / u_{n-1,n-1}$$

for i from n-2 down to 0
  
$$x_i = \frac{c_i - \sum_{j \in [i+1, \min(n, i+p+1))} u_{i,j} x_j}{u_{i,i}}$$

end

```

REVISION NOTES AND PLANS

38.1 Recent changes

- **2024-06-13:** Moved “linear-algebra-with-0-based-indexing-and-semiopen-intervals” to folder main — this might involve changing numerous cross-references!
- **2024-06-9/13:** Proofreading, mostly typo correction along with some minor Python code clean up, so far to section 2.2, *Solving Equations by Fixed Point Iteration (of Contraction Mappings)*.
- **2023-11-12:** Slight revisions and corrections to *Approximating Derivatives by the Method of Undetermined Coefficients*.
- Added notes on rounding errors, illustrated for IEEE-64 machine arithmetic, in *Machine Numbers, Rounding Error and Error Propagation*.
- Added a draft section *Piecewise Polynomial Approximating Functions and Spline Interpolation*, based on notes by Stephen Roberts.
- Added a draft section *Choosing the collocation points: the Chebyshev method*, based on notes by Stephen Roberts.
- Added first drafts of some sections on multi-step methods: leapfrog, Adams-Bashforth methods and Adams-Moulton methods.
- Filled in several incomplete sections on ODE’s.
- Better labelling of graphs and improved examples on ODE’s.
- Brought over some revisions from the Julia edition; more to come (see TO DO list).

38.2 To Do

38.2.1 Content improvements

- Add derivation of the formula for the matrix norm $\|A\|_\infty$
- Add a derivation of the error bound $Rel(x_a) := \frac{\|x - x_a\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|r\|}{\|b\|}$
- Expand some sections that are only stubs or very brief; in particular those on
 - 1) Minimization
 - 2) Multi-step methods for ODEs and determining their stability.
- Continue to port updates from the Julia version from *Finding the Minimum of a Function of One Variable Without Using Derivatives — A Brief Introduction* onwards.

- Add more exercises.
- About document python-tutorial-introduction: either omit or add notes on speed/cost measurement, such as with module `time`.

38.2.2 Formating, layout and style improvements

- **camelCase** everything (e.g. change from *numerical_methods* to *numericalMethods*), except where an underscore is used to indicate a subscript. (For one thing that harmonises with Julia style.)
- When using `<br>`, it should appear on its own line, not at end-of-line. (For PDF output; HTML output is more forgiving.) This is relevant to some pseudo-code appearance in the PDF; e.g in *Basic Concepts and Euler's Method*:

38.2.3 MyST markdown tips

- See <https://jupyterbook.org/en/stable/content/myst.html>
- Move to MyST Markdown notation `{doc}`, `{ref}`, `{eq}` and so on.
- To number equations for referencing, use MyST-Markdown-augmented `$$ . . . $$` notation, as with `$$2+2=4$$` (`eq-obvious`)
- If the top-level section atop a file is labelled as with `(section-label)=` then the usage `{ref}<section-label>` can be used instead of `{doc}<file-base-name>` this is possibly useful if the file name can have suffix either “-python” or “-julia” but I want to use the same cross-reference text.

BIBLIOGRAPHY

BIBLIOGRAPHY

- [BFB16] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. Cengage, 10th edition, 2016.
- [CK12] Ward Cheney and David Kincaid. *Numerical Mathematics and Computing*. Cengage, 7 edition, 2012.
- [KC90] David Kincaid and Ward Cheney. *Numerical Analysis*. Brooks/Cole, 1990.
- [Sau22] Timothy Sauer. *Numerical Analysis*. Pearson, 3rd edition, 2022.

a-contraction-mapping-theorem

a-contraction-mapping-theorem (*main/fixed-point-iteration-python*), 22

a-derivative-based-fixed-point-theorem

a-derivative-based-fixed-point-theorem (*main/fixed-point-iteration-python*), 22

absolute-backward-error

absolute-backward-error (*main/error-measures-convergence-rates*), 50

absolute-error

absolute-error (*main/error-measures-convergence-rates*), 49

algorithm-Doolittle-factorization

algorithm-Doolittle-factorization (*main/linear-equations-3-lu-factorization-python*), 101

algorithm-plu-1

algorithm-plu-1 (*main/linear-equations-4-plu-factorization-python*), 109

algorithm-plu-2

algorithm-plu-2 (*main/linear-equations-4-plu-factorization-python*), 111

algorithm-plu-fragment

algorithm-plu-fragment (*main/linear-equations-4-plu-factorization-python*), 109

another-way-to-count-backwards

another-way-to-count-backwards (*main/linear-equations-1-row-reduction-python*), 78

appendix-backward-banded-0-based

appendix-backward-banded-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 494

appendix-backward-mepp-0-based

appendix-backward-mepp-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 491

appendix-backward-substitution-0-based-2

appendix-backward-substitution-0-based-2 (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 489

appendix-backward-tridiagonal-0-based

appendix-backward-tridiagonal-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 493

appendix-forward-banded-0-based

appendix-forward-banded-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 494

appendix-forward-mepp-0-based

appendix-forward-mepp-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 491

appendix-forward-substitution-0-based

appendix-forward-substitution-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 488

appendix-forward-tridiagonal-0-based

appendix-forward-tridiagonal-0-based
(*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 493

appendix-lu-banded-0-based

appendix-lu-banded-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 493

appendix-lu-doolittle-0-based

appendix-lu-doolittle-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 488

appendix-lu-mepp-0-based

appendix-lu-mepp-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 491

appendix-lu-tridiagonal-0-based

appendix-lu-tridiagonal-0-based
(*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 492

appendix:gaussian-elimination-0-based

appendix:gaussian-elimination-0-based
(*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 487

backward-error

backward-error (*main/newtons-method-python*), ??

backward-error-redux

backward-error-redux (*main/error-measures-convergence-rates*), 50

backward-substitution-

backward-substitution- (*main/linear-equations-1-row-reduction-python*), 78

backward-substitution-0-based

backward-substitution-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 489

backward-substitution-1

backward-substitution-1 (*main/linear-equations-1-row-reduction-python*), 77

backward-substitution-redux

backward-substitution-redux (*main/linear-equations-7-tridiagonal-banded-and-SDD-matrices*), 129

bisection-for

bisection-for (*main/root-finding-by-interval-halving-python*), 16

bisection-step

bisection-step (*main/root-finding-by-interval-halving-python*), 15

bisection-while

bisection-while (*main/root-finding-by-interval-halving-python*), 17

bisection-x-cosx

bisection-x-cosx (*main/root-finding-by-interval-halving-python*), 9

check-with-taylor`

check-with-taylor` (*main/derivatives-and-the-method-of-undetermined-coefficients*), 178

choose==step-size-2

choose==step-size-2 (*main/ODE-IVP-5-error-control*), 259

choose-step-size-1

choose-step-size-1 (*main/ODE-IVP-5-error-control*), 258

collocation-error-formula

collocation-error-formula (*main/polynomial-collocation-error-formulas-python*), 148

collocation-error-formula-equally-spaced-nodes

collocation-error-formula-equally-spaced-nodes (*main/polynomial-collocation-error-formulas-python*), 148

collocation-error-formula-remark

collocation-error-formula-remark
(*main/polynomial-collocation-error-formulas-python*), 153

comparison-to-taylor-error-formula

comparison-to-taylor-error-formula
(*main/polynomial-collocation-error-formulas-python*), 148

convergence-of-order-p

convergence-of-order-p (main/error-measures-convergence-rates), 51

definition-absolute-error

definition-absolute-error (main/fixed-point-iteration-python), 23

definition-columnwise-strictly-diagonally-dominant

definition-columnwise-strictly-diagonally-dominant (main/linear-equations-1-row-reduction-python), 85

definition-contraction-mapping

definition-contraction-mapping (main/fixed-point-iteration-python), 21

definition-error

definition-error (main/fixed-point-iteration-python), 23

definition-mapping

definition-mapping (main/fixed-point-iteration-python), 20

definition-psychologically-triangular

definition-psychologically-triangular (main/linear-equations-4-plu-factorization-python), 114

definition-strictly-diagonally-dominant

definition-strictly-diagonally-dominant (main/linear-equations-1-row-reduction-python), 85

definition-tridiagonal

definition-tridiagonal (main/linear-equations-7-tridiagonal-banded-and-SDD-matrices), 128

definition-vector-valued-contraction-mapping

definition-vector-valued-contraction-mapping (main/linear-equations-6-iterative-methods-python), 125

dolittle-general

dolittle-general (main/linear-equations-7-tridiagonal-banded-and-SDD-matrices), 130

error-bound-chebychev-collocation

error-bound-chebychev-collocation (main/polynomial-collocation-chebychev), 155

error-bounds-clamped-splines

error-bounds-clamped-splines (main/piecewise-polynomial-approximation-and-splines), 159

error-bounds-hermite-cubics

error-bounds-hermite-cubics (main/piecewise-polynomial-approximation-and-splines), 160

error-left-endpoint-rule

error-left-endpoint-rule (main/integrals-1-building-blocks-python), 188

error-redux

error-redux (main/error-measures-convergence-rates), 49

errors-when-approximating-derivatives

errors-when-approximating-derivatives (main/machine-numbers-rounding-error-and-error-propagation-python), 93

euler-variable-h

euler-variable-h (main/ODE-IVP-5-error-control), 256

example-1-x-4cosx

example-1-x-4cosx (main/fixed-point-iteration-python), 20

example-2

example-2 (main/newtons-method-python), ??

example-2-x-cosx

example-2-x-cosx (main/fixed-point-iteration-python), 22

example-3

example-3 (main/newtons-method-python), ??

example-3-x-cosx-fpi

example-3-x-cosx-fpi (main/fixed-point-iteration-python), 24

example-4

example-4 (main/fixed-point-iteration-python), 27

example-almost-division-by-zero

example-almost-division-by-zero
(*main/linear-equations-1-row-reduction-python*),
83

example-avoiding-small-denominators

example-avoiding-small-denominators
(*main/linear-equations-1-row-reduction-python*),
84

example-basic-forward-difference

example-basic-forward-difference
(*main/derivatives-and-the-method-of-undetermined-coefficients*), 174

example-hilbert-matrices

example-hilbert-matrices (*main/linear-equations-5-error-bounds-condition-numbers-python*), 117

example-integration

example-integration (*main/ODE-IVP-1-basics-and-Euler-python*), ??

example-less-obvious-division-by-zero

example-less-obvious-division-by-zero
(*main/linear-equations-1-row-reduction-python*),
82

example-newton-x-cosx

example-newton-x-cosx (*main/newtons-method-python*), ??

example-nonlinear-ode

example-nonlinear-ode (*main/ODE-IVP-1-basics-and-Euler-python*), ??

example-obvious-division-by-zero

example-obvious-division-by-zero
(*main/linear-equations-1-row-reduction-python*),
81

example-simplest-real-ode

example-simplest-real-ode (*main/ODE-IVP-1-basics-and-Euler-python*), ??

example-stiff-ode

example-stiff-ode (*main/ODE-IVP-1-basics-and-Euler-python*), ??

example-three-point-centered-difference

example-three-point-centered-difference (*main/derivatives-and-the-method-of-undetermined-coefficients*), 175

example-three-point-one-sided-difference

example-three-point-one-sided-difference (*main/derivatives-and-the-method-of-undetermined-coefficients*), 174

example-three-point-one-sided-difference-method-2

example-three-point-one-sided-difference-method-2 (*main/derivatives-and-the-method-of-undetermined-coefficients*),
177

explicit-midpoint-algorithm

explicit-midpoint-algorithm (*main/ODE-IVP-2-Runge-Kutta-python*), 229

explicit-trapezoid-algorithm

explicit-trapezoid-algorithm (*main/ODE-IVP-2-Runge-Kutta-python*), 225

forward-substitution

forward-substitution (*main/linear-equations-7-tridiagonal-banded-and-SDD-matrices*), 129

gaussian-elimination

gaussian-elimination (*main/linear-equations-1-row-reduction-python*), 72

gaussian-elimination-0-based

gaussian-elimination-0-based (*main/linear-equations-1-row-reduction-python*), 73

gaussian-elimination-inserting-zeros

gaussian-elimination-inserting-zeros
(*main/linear-equations-1-row-reduction-python*),
71

gemep-0-based

gemep-0-based (*main/linear-algebra-with-0-based-indexing-and-semiopen-intervals*), 490

generalized-mean-value-theorem

generalized-mean-value-theorem
(*main/integrals-2-composite-rules*), 191

geometrical-derivation-of-least-squares

geometrical-derivation-of-least-squares (*main/least-squares-fitting*), 165

integral-mean-value-theorem

integral-mean-value-theorem (*main/integrals-1-building-blocks-python*), 186

interpolation-example-1

interpolation-example-1 (*main/polynomial-collocation+approximation-python*), 140

interpolation-example-2

interpolation-example-2 (*main/polynomial-collocation+approximation-python*), 143

interpolation-example-3

interpolation-example-3 (*main/polynomial-collocation+approximation-python*), 145

inverse-power-method

inverse-power-method (*main/eigenproblems-python*), 135

linear-convergence

linear-convergence (*main/error-measures-convergence-rates*), 50

lu-banded

lu-banded (*main/linear-equations-7-tridiagonal-banded-and-SDD-matrices*), 130

lu-banded-symmetric

lu-banded-symmetric (*main/linear-equations-7-tridiagonal-banded-and-SDD-matrices*), 131

lu-factorization

lu-factorization (*main/linear-equations-7-tridiagonal-banded-and-SDD-matrices*), 129

mathematically-correct-notation

mathematically-correct-notation (*main/linear-equations-1-row-reduction-python*), 78

midpoint-rule-error

midpoint-rule-error (*main/integrals-1-building-blocks-python*), 186

module-numerical-methods

module-numerical-methods (*main/newtons-method-python*), ??

multistep-method

multistep-method (*main/ODE-IVP-6-multi-step-methods-introduction-python*), 271

multistep-method-redux

multistep-method-redux (*main/ODE-IVP-7-multi-step-methods-Adams-Bashforth-python*), 281

naive-gaussian-elimination

naive-gaussian-elimination (*main/linear-equations-1-row-reduction-python*), 71

no-scaled-partial-pivoting

no-scaled-partial-pivoting (*main/linear-equations-2-pivoting-python*), 95

numpy-math-functions

numpy-math-functions (*main/root-finding-by-interval-halving-python*), 12

numpy-matplotlib

numpy-matplotlib (*main/root-finding-by-interval-halving-python*), 9

odeivp-onestep-order-of-global-error

odeivp-onestep-order-of-global-error (*main/ODE-IVP-3-error-results-one-step-methods*), 238

power-method

power-method (*main/eigenproblems-python*), 133

proposition-1

proposition-1 (*main/newtons-method-convergence-rate*), ??

proposition-1-fpi-iterates-converge-to-fp

proposition-1-fpi-iterates-converge-to-fp (*main/fixed-point-iteration-python*), 19

proposition-2

proposition-2 (*main/newtons-method-convergence-rate*), ??

proposition-2-ivp-fpi-version

proposition-2-ivp-fpi-version (*main/fixed-point-iteration-python*), 20

proposition-3

proposition-3 ([main/fixed-point-iteration-python](#)), 23

python-array-dicing

python-array-dicing ([main/linear-equations-1-row-reduction-python](#)), 76

python-array-slicing

python-array-slicing ([main/linear-equations-1-row-reduction-python](#)), 75

python-complex-numbers

python-complex-numbers ([main/newtons-method-python](#)), ??

python-counting-backwards

python-counting-backwards ([main/linear-equations-1-row-reduction-python](#)), 78

python-dot

python-dot ([main/eigenproblems-python](#)), 133

python-splat-*

python-splat-* ([main/linear-equations-1-row-reduction-python](#)), 79

relative-error

relative-error ([main/error-measures-convergence-rates](#)), 49

remark-1

remark-1 ([main/integrals-2-composite-rules](#)), 194

remark-1-not-quite-zero-values-and-rounding

remark-1-not-quite-zero-values-and-rounding ([main/linear-equations-1-row-reduction-python](#)), 71

remark-1-to-0-easy

remark-1-to-0-easy ([main/linear-equations-1-row-reduction-python](#)), 73

remark-12

remark-12 ([main/fixed-point-iteration-python](#)), 24

remark-19

remark-19 ([main/linear-equations-1-row-reduction-python](#)), 82

remark-5

remark-5 ([main/error-measures-convergence-rates](#)), 50

remark-LU-with-P

remark-LU-with-P ([main/linear-equations-4-plu-factorization-python](#)), 111

remark-dolittle

remark-dolittle ([main/linear-equations-3-lu-factorization-python](#)), 101

remark-float-vs-int

remark-float-vs-int ([main/linear-equations-1-row-reduction-python](#)), 70

remark-module-linalg

remark-module-linalg ([main/linear-equations-1-row-reduction-python](#)), 67

remark-numpy-linalg-norm

remark-numpy-linalg-norm ([main/linear-equations-5-error-bounds-condition-numbers-python](#)), 116

remark-numpy-matrix-product

remark-numpy-matrix-product ([main/linear-equations-1-row-reduction-python](#)), 69

remark-other-matrix-norms

remark-other-matrix-norms ([main/linear-equations-5-error-bounds-condition-numbers-python](#)), 116

remark-positive-definite-also-works

remark-positive-definite-also-works ([main/linear-equations-3-lu-factorization-python](#)), 107

remark-positive-definite-matrices-also-work

remark-positive-definite-matrices-also-work ([main/linear-equations-1-row-reduction-python](#)), 86

remark-python-for-0-based

remark-python-for-0-based ([main/linear-equations-1-row-reduction-python](#)), 72

remark-python-style

remark-python-style ([main/newtons-method-python](#)), ??

remark-use-the-module

remark-use-the-module ([main/root-finding-without-derivatives-python](#)), 56

remark-vector-derivative notation

remark-vector-derivative notation
([main/newtons-method-for-systems-intro](#)), ??

richardson-forward-differences

richardson-forward-differences
([main/richardson-extrapolation](#)), 180

richardson0n-to-kn

richardson0n-to-kn ([main/richardson-extrapolation](#)), 181

rkf

rkf ([main/ODE-IVP-5-error-control](#)), 266

robust

robust ([main/machine-numbers-rounding-error-and-error-propagation-python](#)), 87

romberg-integration

romberg-integration ([main/integrals-4-romberg-integration](#)), 198

runge-kutta

runge-kutta ([main/ODE-IVP-2-Runge-Kutta-python](#)), 233

secant-method

secant-method ([main/root-finding-without-derivatives-python](#)), 61

separatrices

separatrices ([main/ODE-IVP-4-system-higher-order-equations-python](#)), 242

stiffness

stiffness ([main/ODE-IVP-4-system-higher-order-equations-python](#)), 241

super-linear

super-linear ([main/error-measures-convergence-rates](#)), 51

taylors-theorem-a

taylors-theorem-a ([main/taylors-theorem](#)), ??

taylors-theorem-h

taylors-theorem-h ([main/taylors-theorem](#)), ??

theorem-1

theorem-1 ([main/derivatives-and-the-method-of-undetermined-coefficients](#)), 176

theorem-Crout-SDD

theorem-Crout-SDD ([main/linear-equations-3-lu-factorization-python](#)), 107

theorem-LU-SDD

theorem-LU-SDD ([main/linear-equations-3-lu-factorization-python](#)), 106

theorem-collocation

theorem-collocation ([main/polynomial-collocation+approximation-python](#)), 139

theorem-gaus-seidel-convergence

theorem-gaus-seidel-convergence
([main/linear-equations-6-iterative-methods-python](#)), 127

theorem-jacobi-convergence

theorem-jacobi-convergence ([main/linear-equations-6-iterative-methods-python](#)), 126

theorem-loss-of-precision

theorem-loss-of-precision ([main/machine-numbers-rounding-error-and-error-propagation-python](#)), 93

theorem-matrix-iteration-convergence

theorem-matrix-iteration-convergence
([main/linear-equations-6-iterative-methods-python](#)), 125

theorem-row-reduction-preserves-sdd

theorem-row-reduction-preserves-sdd
([main/linear-equations-1-row-reduction-python](#)), 85

todo-ode-2

todo-ode-2 ([main/ODE-IVP-2-Runge-Kutta-python](#)), 223

trapezoid-rule-error

trapezoid-rule-error ([main/integrals-1-building-blocks-python](#)), 185

trapezoid-step-size

trapezoid-step-size ([main/ODE-IVP-5-error-control](#)), 265

triangular-matrix

triangular-matrix ([main/linear-equations-3-lu-factorization-python](#)), 99

uniformly-contracting

uniformly-contracting (main/fixed-point-iteration-python), [22](#)

vector-valued-contraction-mapping-theorem

vector-valued-contraction-mapping-theorem (main/linear-equations-6-iterative-methods-python), [125](#)

well-posed

well-posed (main/machine-numbers-rounding-error-and-error-propagation-python), [87](#)